unooria
plataforma educacional B2G

DOCUMENTAÇÃO TÉCNICA DE ARQUITETURA DE SOFTWARE, ARQUITETURA DE NUVEM E CÓDIGO

Plataforma Unooria

Hub multi-tenant e 48 experiências imersivas

Monorepo de uma plataforma de educação básica para redes públicas de ensino: um *hub* em Next.js que embeda experiências estáticas versionadas em CDN, uma API NestJS com isolamento lógico por escola em *Row-Level Security*, telemetria em dois canais e uma camada de IA generativa com quota por perfil — uma *stack* de nuvem por contrato, cem escolas por *stack*.

DOCUMENTO	**PU-DOC-001 — Documentação Técnica Consolidada**
VERSÃO	**1.1**
DATA	**27 de agosto de 2026**
AUTOR	**Lucas Salomão**
REPOSITÓRIO	**`plataforma-unooria` (*monorepo*, tronco `main`)**
CLASSIFICAÇÃO	**Documento técnico de projeto — uso interno**

Escopo do documento: contexto de negócio, requisitos, casos de uso, arquitetura de *software* nos quatro níveis do modelo C4, arquitetura de nuvem e implantação em GCP, modelo de *multi-tenancy* e segurança, modelo de dados, contratos de integração, telemetria, camada de IA, catálogo de experiências, especificação da API, *design system*, engenharia de entrega, qualidade, registro de decisões arquiteturais, riscos e evolução. Diagramas vetoriais em TikZ; fontes Mermaid equivalentes no Apêndice A.

Ficha técnica do documento

Identificação

Campo	Conteúdo
Título	Plataforma Unooria — Documentação Técnica Consolidada de Arquitetura de *Software*, Arquitetura de Nuvem e Código
Identificador	PU-DOC-001
Versão	1.1
Data de emissão	27 de agosto de 2026
Autor	Lucas Salomão (<lucas.salomao@unooria.com>)
Repositório	`plataforma-unooria` — *monorepo* pnpm + Turborepo
Tronco	`main` (histórico linear; uma *branch* por experiência)
Linguagem principal	TypeScript 5.7 (*strict*), Node.js 20
Prazo de produção	novembro de 2026
Estado	Em desenvolvimento — M0–M6 concluídos; *stack* de *dev* em GCP operacional
Repositório de infra	Separado (Terraform). Contrato em `docs/INFRA.md`

Escopo e propósito

Este documento consolida, em fonte única, o que estava distribuído entre `AGENTS.md` (as convenções e regras invioláveis do repositório), `README.md`, `docs/ARQUITETURA.md`, `docs/DECISOES.md`, `docs/INFRA.md`, `docs/INFRA-IDENTITY.md` e os comentários de decisão que vivem no próprio código-fonte — que, neste projeto, é onde as decisões mais finas estão registradas.

Acrescenta o material que não existia em forma escrita: especificação de requisitos funcionais e não funcionais rastreáveis, catálogo de casos de uso, modelo C4 completo nos quatro níveis para o *software* e para a nuvem, diagramas de sequência dos contratos de integração, dicionário de dados, especificação da API rota por rota e o registro formal das decisões arquiteturais (ADR) com suas consequências.

> **NOTA**
>
> Este documento é **normativo** para o projeto: quando ele divergir de uma suposição, ele vence. Quando divergir do código, o código vence e o documento está desatualizado — e atualizá-lo faz parte da *Definition of Done* de qualquer tarefa (Capítulo 16).

Histórico de revisões

Versão	Data	Autor	Alterações
0.1	22/08/2026	L. Salomão	Estrutura do *monorepo*; arquitetura e ADRs iniciais em `docs/`.
0.2	25/08/2026	L. Salomão	Migração para o *monorepo* mergeada na `main` (26 *commits*, *fast-forward*); *tenancy* com RLS, exp-01 e exp-02.
0.3	26/08/2026	L. Salomão	exp-70 (Vila do Lume) e exp-91 (bem-estar); decisão de retenção sem prazo declarado.
1.0	**27/08/2026**	**L. Salomão**	**Documentação técnica consolidada: requisitos, casos de uso, modelo C4 de *software* e de nuvem, *multi-tenancy*, modelo de dados, contratos, telemetria particionada, IA, API, *design system*, entrega, qualidade, ADRs e bibliografia em ABNT.**
1.1	01/09/2026	L. Salomão	Capítulo de observabilidade: estado atual por camada, sinais, comparação de ferramentas, SLOs e alertas, privacidade nos sinais, custo por contrato e roteiro em fases; **ADR-57** e **ADR-58** (propostas); `R-05` reclassificado.

Documentos de referência do projeto

Artefato	Conteúdo
`AGENTS.md`	Convenções, regras invioláveis de *tenancy* e segurança, contratos de integração e *Definition of Done*. É o documento de governança do repositório.
`README.md`	Instalação, os três modos de execução local e *smoke* visual.
`docs/ARQUITETURA.md`	Visão arquitetural resumida por contrato.
`docs/DECISOES.md`	Registro resumido de decisões (base do Capítulo 18).
`docs/INFRA.md`	Contrato entre este repositório e o de Terraform: nomes de recursos, variáveis e o que o CI publica.

`docs/INFRA-IDENTITY.md`	Contrato do gerenciador central de identidade (WSO2 IS), que é único para vários contratos.
`apps/api/prisma/schema.prisma`	Modelo de dados anotado, com o porquê de cada decisão de esquema.
`packages/sdk/src/`	Contrato *postMessage* e cliente de telemetria — a fronteira entre hub e experiência.
`.env.example`	Catálogo completo de variáveis de ambiente (Apêndice B).

Sumário

Ficha técnica do documento i

Lista de figuras xx

Lista de tabelas xxii

Lista de siglas e abreviaturas xxiii

1 Introdução 1

1.1 Propósito deste documento . . . 1

1.2 O problema . . . 1

1.3 A solução em uma página . . . 2

1.4 Como ler este documento . . . 3

1.4.1 Estrutura . . . 3

1.4.2 Convenções tipográficas . . . 4

1.5 O que este documento não cobre . . . 5

2 Contexto de negócio e restrições 6

2.1 Modelo de negócio . . . 6

2.2 Escala alvo . . . 6

2.3 Restrições de projeto . . . 7

2.4 Os *drivers* arquiteturais . . . 8

2.5 Interessados . . . 9

2.6 As sete regras invioláveis . . . 10

2.7 Fora de escopo, deliberadamente . . . 10

3 Requisitos 12

3.1 Requisitos funcionais 12

3.1.1 Identidade, papéis e acesso 12

3.1.2 Experiências e *embed* 13

3.1.3 Telemetria 14

3.1.4 Inteligência artificial 14

3.1.5 Pedagógico e relatórios 15

3.2 Requisitos não funcionais 17

3.3 Rastreabilidade 18

4 Casos de uso 20

4.1 Atores 20

4.2 Diagrama de casos de uso 21

4.3 Especificação dos casos de uso 21

4.3.1 UC-01 — Autenticar-se 22

4.3.2 UC-03 — Realizar experiência 22

4.3.3 UC-10 — Preparar atividade 24

4.3.4 UC-13 — Ler resultados da turma 24

4.3.5 UC-14 — Gerar plano de aula 25

4.3.6 UC-15 — Fazer *check-in* de bem-estar 26

4.3.7 UC-23 — Publicar manifesto de experiência 27

4.4 Rastreabilidade de casos de uso 27

5 Arquitetura de *software* 29

5.1 Visões arquiteturais 29

5.2 Estilo arquitetural e princípios 29

5.3 C4 nível 1 — Contexto do sistema 31

5.4 C4 nível 2 — Contêineres 31

5.4.1 Catálogo de contêineres e artefatos 32

5.5 C4 nível 3 — Componentes 33

5.5.1 Componentes da API 33

5.5.2 Componentes do hub 35
5.6 C4 nível 4 — Código 36
5.6.1 O contrato da ponte e o *guard* de *tenancy* 37
5.6.2 O gabarito de um módulo de domínio 37
5.7 Estrutura do *monorepo* 38
5.8 Fluxos principais 39
5.8.1 Curadoria de um bloco de *feed* (exp-01) 39
5.8.2 Estados de uma geração assíncrona 40

6 Arquitetura de nuvem e implantação 41
6.1 Princípios de nuvem 41
6.2 C4 nível 1 de implantação — o contrato 42
6.3 Diagrama de implantação detalhado 42
6.4 Catálogo de recursos por contrato 44
6.5 Caminhos de tráfego 45
6.6 Identidade de carga de trabalho e segredos 46
6.7 Dimensionamento 47
6.7.1 Volumes medidos 47
6.8 Ambientes 48
6.9 O provedor de identidade fora da *stack* 49
6.10 Observabilidade, continuidade e reversão 49

7 *Multi-tenancy* e segurança 51
7.1 Os três níveis de isolamento 51
7.2 Nível 2: *Row-Level Security* no PostgreSQL 51
7.2.1 Como a política é armada 52
7.2.2 A porta única: `comTenancy` 52
7.2.3 Como a escola chega ao contexto 54
7.2.4 A exceção: `admin-contrato` 54
7.3 Nível 3: autorização por papel 54

7.3.1 Onde cada papel entra 55
7.3.2 A matriz de capacidades 56
7.3.3 As portas que não são papel 59
7.3.4 O desenho por trás 59
7.4 Segurança do *embed* 60
7.5 Segredos e o que nunca vai ao cliente 62
7.6 Privacidade e dados de menores 62
7.6.1 Retenção: uma decisão explícita de não declarar prazo 63
7.6.2 Localidade de processamento 64
7.7 Verificação 64
7.8 Ameaças consideradas 64

8 Modelo de dados 66

8.1 A convenção de nomes 66
8.1.1 A migração de referência, e duas armadilhas registradas 67
8.2 Modelo da plataforma 67
8.3 A rede: escola, lotação e o CPF 69
8.4 Enturmamento e identidade 70
8.5 Telemetria particionada 71
8.6 Modelo da exp-01 (Instanooria) 71
8.7 Modelo das experiências 02, 70, 90 e 91 75
8.8 As cinco tabelas da exp-03 76
8.9 Decisões de esquema por experiência 77
8.9.1 exp-70 — as decisões que custam informação de propósito 77
8.9.2 exp-90 — conteúdo fora do banco 78
8.9.3 exp-91 — ausência como decisão de esquema 78
8.10 Migrações 78

9 Contratos de integração 81

9.1 Protocolo `postMessage` hub ↔ experiência 81

9.1.1 Especificação das mensagens (protocolo v1) 81
9.1.2 O *handshake*, passo a passo 82
9.1.3 Regras invioláveis do protocolo 83
9.1.4 Política de evolução 84
9.2 Manifesto de experiências 84
9.2.1 Como o hub resolve a URL do `iframe` 86
9.2.2 Publicação e reversão 86
9.3 Contrato de telemetria 87
9.4 Contrato do *provider* de IA 87
9.5 Contrato entre duas experiências 89

10 Telemetria 90

10.1 Por que dois canais 90
10.2 O canal analítico, ponta a ponta 92
10.3 A rota de ingestão e sua tranca 92
10.4 Destino: PostgreSQL, não BigQuery 93
10.4.1 `INSERT` multi-linha 93
10.4.2 Partição mensal 94
10.5 Privacidade da telemetria 94
10.6 Comportamento em desenvolvimento 95

11 Camada de inteligência artificial 96

11.1 Onde a IA é usada 96
11.2 A camada única 97
11.3 Quota por perfil 97
11.4 Credencial: ADC, nunca chave de API 98
11.5 Modelo, região e o custo de escolher 99
11.5.1 Imagem: proporção pedida ao modelo, não recortada depois 99
11.6 Raciocínio conta como saída 100
11.7 *Guardrails* do *check-in* de bem-estar 100

11.8 Variáveis de configuração da IA 102

12 Catálogo de experiências 103

12.1 Visão do catálogo 103

12.2 exp-01 — Instanooria 104

12.3 exp-02 — UnooTube 105

12.4 exp-03 — unooGPT (Missão Tempestade) 106

12.4.1 A regra de ouro: a Lacuna não cruza a fronteira do `iframe` 106

12.4.2 Três contas que o cliente não faz 107

12.4.3 Ponto cego é ausência, não zero 107

12.4.4 O Relógio da Aula 108

12.4.5 Telas do professor 108

12.4.6 `allow-forms`: o bug que só o *smoke* pega 108

12.4.7 O *smoke* mentiu duas vezes 108

12.4.8 Pendência: o guia do debate 109

12.5 exp-70 — Vila do Lume 109

12.5.1 Duas leituras de uma partida, e elas não se tocam 109

12.5.2 O servidor recalcula; nunca aceita o perfil do cliente 109

12.5.3 `grau` é invariante de API, não regra de tela 110

12.5.4 A bolha é do servidor 110

12.5.5 Telas do professor 110

12.5.6 A exceção de ADR: o jogo em JavaScript clássico 111

12.6 exp-90 — Gerador de plano de aula 111

12.7 exp-91 — Check-in de bem-estar 112

12.8 Como criar uma experiência nova 112

13 Especificação da API 114

13.1 Convenções transversais 114

13.1.1 As três rotas sem `OidcGuard` 115

13.2 Mapa de rotas por área 116

13.3 Superfície que a SDK consome 116

13.4 Grupos de rotas em detalhe 117
13.4.1 Motor de *feed* (exp-01) 117
13.4.2 Entrada de pessoas: as duas portas, e por que só uma escreve 118
13.4.3 Planos de aula (exp-90) 119
13.4.4 Bem-estar (exp-91) 119
13.4.5 Análise 120
13.5 Sanidade e prontidão 120
13.6 Documentação executável (OpenAPI) 120

14 *Design system* e camada de interface 122
14.1 Duas famílias no mesmo pacote 122
14.2 A linha que separa as seções é a REVELAÇÃO 124
14.3 Consumo: sempre pela ponte 124
14.4 CSS: uma vez, no ponto de entrada 125
14.4.1 No hub, o CSS não basta: as fontes também 125
14.5 Frames: dois níveis, não empilhar 126
14.6 Tema 126
14.7 Versão: atualizar é ato deliberado 127
14.8 Compensações locais 127
14.8.1 Lacunas abertas, verificadas na 0.1.14 127
14.8.2 Lacunas abertas na 0.1.17 (exp-02 1.1.0) 128
14.8.3 Avisos que ficaram obsoletos 128
14.9 Cor da plataforma: classe escrita à mão 129
14.10 Exceção registrada 129

15 Engenharia de entrega 130
15.1 O *monorepo* e o *build* incremental 130
15.2 Integração contínua 131
15.2.1 A trava de vulnerabilidade (Snyk) 132
15.3 Entrega contínua 133

15.3.1 O que faz o *deploy* caber no café 133
15.3.2 Nada de nome de recurso no *workflow* 134
15.3.3 Concorrência e reexecução 135
15.4 Ambientes locais 136
15.5 O provedor de identidade sobe à parte 136
15.6 Convenções de *git* 137

16 Qualidade 139

16.1 A *Definition of Done* 139
16.2 Estratégia de testes 140
16.3 A suíte de ponta a ponta, em detalhe 141
16.3.1 O manifesto: a descrição mora no teste que fotografa 141
16.3.2 Erro provocado não é erro descoberto 142
16.3.3 O roteiro tem de PERDER a corrida para o erro 142
16.3.4 Duas condições para o vermelho significar alguma coisa 143
16.3.5 O que só o *smoke* visual pega 143
16.4 Análise estática 144
16.5 Armadilhas de ferramenta registradas 145
16.6 Cobertura: onde está e onde não está 145

17 Observabilidade 147

17.1 O que se vê hoje 147
17.2 Princípios 151
17.3 Os sinais, e onde cada um nasce 152
17.3.1 Log estruturado 152
17.3.2 Métricas 153
17.3.3 Rastros distribuídos 154
17.3.4 Erros 155
17.3.5 Experiência do usuário real e sondas sintéticas 156
17.3.6 Banco, mensageria e borda 157
17.3.7 Camada de IA 158

17.3.8 Entrega 158
17.4 Ferramentas: comparação e escolha 158
17.5 SLIs, SLOs e a lista de alertas 160
17.6 Privacidade nos sinais 162
17.7 Custo estimado por contrato 163
17.8 Roteiro de implantação 165
17.9 O que fica de fora 167

18 Registro de decisões arquiteturais 168

18.1 Índice 168
18.2 ADR-01 — Experiências como *bundles* estáticos 169
18.3 ADR-02 — *Monorepo* único 170
18.4 ADR-03 — Autenticação por OIDC genérico 170
18.5 ADR-04 — Uma *stack* por contrato; escolas por RLS 171
18.6 ADR-05 — Cloud SQL HA, não AlloyDB 171
18.7 ADR-06 — Pub/Sub para telemetria desde o início 172
18.8 ADR-07 — IA atrás de um *gateway* na própria API 172
18.9 ADR-08 — RLS forçada e uma porta única de banco 172
18.10 ADR-09 — Domínio por experiência na API; gabarito *server-side* 173
18.11 ADR-10 — Bem-estar: instrumento fixo, IA só no entorno, sem gestor 173
18.12 ADR-11 — Telemetria sem PII e vocabulário fechado 174
18.13 ADR-12 — `grau` como invariante de API 174
18.14 ADR-13 — Retenção sem prazo declarado 175
18.15 ADR-14 — Telemetria no PostgreSQL particionado 175
18.16 ADR-15 — O hub assina os *tokens* de experiência 176
18.17 ADR-16 — ADC como credencial; modelo *versus* região 176
18.18 ADR-17 — *Claim* atômico em vez de fila externa 177
18.19 ADR-18 — Exceções registradas 178
18.20 ADR-19 — *Design system* externo como fonte única de interface 178

18.21 ADR-20 — Um banco, um *schema*; prefixo `exp_NN_` 178

18.22 ADR-21 — *Push subscription* para a própria API 179

18.23 ADR-22 — exp-02 1.1.0: *briefing* antes do disfarce 179

18.24 ADR-23 — *Function calling* entra no `AiProvider` 180

18.25 ADR-24 — Teto de custo de IA por SESSÃO 182

18.26 ADR-25 — O Relógio da Aula é tabela da experiência 183

18.27 ADR-26 — Sem prazo de retenção na exp-03 184

18.28 ADR-27 — Transcrição é dado do instrumento 185

18.29 ADR-28 — Identidade por pessoa; OneRoster na fronteira 185

18.30 ADR-29 — Senha provisória com troca obrigatória 188

18.31 ADR-30 — A rede: lotação por escola e escola ativa no *token* 189

18.32 ADR-31 — CPF como identidade, guardado só em *hash* 190

18.33 ADR-32 — Transferência: o dado fica, a identidade segue 191

18.34 ADR-33 — O CPF é guardado, e o gestor o vê 192

18.35 ADR-34 — A regional agrupa; quem autoriza é a lotação 194

18.36 ADR-35 — Perfil portátil: um documento assinado, não uma integração 195

18.37 ADR-36 — O OpenAPI é derivado do Zod, não escrito ao lado dele 198

18.38 ADR-37 — O *schema* de resposta é conferido, não só publicado 199

18.39 ADR-38 — O *deploy* paraleliza os *jobs*, não o portão 200

18.40 ADR-39 — Vulnerabilidade de dependência trava o portão, com prazo para os *majors* 201

18.41 ADR-40 — A raiz é a *landing*; a marca vem do DS, não do modelo 202

18.42 ADR-41 — O relatório do estudante sai do aparelho: o iframe expande, não remonta 204

18.43 ADR-42 — “O que o algoritmo escondeu” é a EXPOSIÇÃO, não o domínio 205

18.44 ADR-43 — A resposta do estudante guarda a primeira e a última 206

18.45 ADR-44 — A janela do envolvimento termina com o feed, e o total sai das listagens 207

18.46 ADR-45 — *Agent* e `fetch` vêm do MESMO pacote undici 208

18.47 ADR-46 — O `?hubOrigin=` é entrada não confiável: a experiência confirma quem a embedou 209

18.48 ADR-47 — Nem senha de demonstração nem verificação de TLS moram no fonte . . . 211

18.49 ADR-48 — O teto do CSV é de BYTES e cabe debaixo do limite do ambiente; o modelo do OneRoster vem em pares, e o exemplo leva a escola de verdade . . . 213

18.50 ADR-49 — A atividade do Instanooria declara a TURMA, e o painel dela para de ler a escola inteira . . . 215

18.51 ADR-50 — O perfil importado guarda a PARTIDA e a VERSÃO do mapeamento, e não ganha chave estrangeira . . . 216

18.52 ADR-51 — A suíte de ponta a ponta é organizada por PAPEL, e a descrição de cada tela mora no teste que a fotografa . . . 217

18.53 ADR-52 — Falha do provedor de IA na destilação é 503 com a causa, não 500 mudo . . . 219

18.54 ADR-53 — Um ambiente por *branch*: *environments* do GitHub carregam as `vars`, e a *branch* escolhe a *stack* . . . 221

18.55 ADR-54 — Pessoa nasce por IMPORTAÇÃO: não existe cadastro avulso de usuário . . . 222

18.56 ADR-55 — Instrução de agente: `AGENTS.md` canônico por fronteira, `CLAUDE.md` como ponteiro de uma linha . . . 223

18.57 ADR-56 — O fluxo por ambiente é imposto por *hooks* versionados, não por *branch protection* . . . 225

18.58 ADR-57 — Observabilidade nativa do Google Cloud, com OpenTelemetry como fronteira; nenhuma *stack* auto-hospedada . . . 226

18.59 ADR-58 — Erro de cliente entra pela API; nenhum SDK de terceiro no *bundle* da experiência . . . 228

18.60 ADR-59 — Cadastro unitário existe, e ele DELEGA ao importador; `admin-contrato` tem rota própria . . . 229

18.61 ADR-60 — Microsoft Clarity carrega no hub **e** dentro do *bundle* da experiência . . . 231

18.62 ADR-61 — Trabalho paralelo acontece em *worktree*, com a base escrita à mão . . . 233

18.63 ADR-62 — O WAF não inspeciona o corpo de rota autenticada nem de texto livre de estudante . . . 235

18.64 ADR-63 — O CI não publica a mídia pesada, mas se recusa a publicar sem ela . . . 236

19 Riscos e evolução **238**

19.1 Registro de riscos . . . 238

19.2 Estado de implementação . . . 241

19.3 Evolução planejada 242
19.3.1 Antes de novembro de 2026 (bloqueante) 242
19.3.2 Depois, por ordem de retorno 243
19.4 O que continuará fora de escopo 243

20 Conclusão 245

20.1 O que foi construído 245
20.2 As decisões que mais definiram o resultado 245
20.3 O que este projeto ensinou sobre documentação 246
20.4 Estado de prontidão para produção 246
20.5 Encerramento 247

A Fontes Mermaid dos diagramas 248

A.1 Figura 1.1 — Visão geral da plataforma 248
A.2 Figura 4.1 — Casos de uso 249
A.3 Figura 5.1 — C4 nível 1: contexto 250
A.4 Figura 5.2 — C4 nível 2: contêineres 250
A.5 Figura 5.3 — C4 nível 3: componentes da API 251
A.6 Figura 5.4 — C4 nível 3: componentes do hub 252
A.7 Figura 5.5 — C4 nível 4: tipos que carregam a regra 252
A.8 Figura 5.6 — Grafo de dependências do *monorepo* 253
A.9 Figura 5.7 — Curadoria de um bloco de *feed* 253
A.10 Figura 5.8 — Estados de uma geração assíncrona 254
A.11 Figura 6.1 — Implantação de um contrato 254
A.12 Figura 6.2 — Implantação detalhada 255
A.13 Figura 6.3 — Identidade de carga de trabalho 256
A.14 Figura 7.1 — Os três níveis de isolamento 256
A.15 Figura 7.2 — Propagação da escola 257
A.16 Figura 8.1 — Modelo de dados da plataforma 257
A.17 Figuras 8.3 e 8.4 — Modelo da exp-01 258

A.18 Figuras 8.5 e 8.6 — Demais experiências 259
A.19 Figura 9.1 — *Handshake* hub-experiência 260
A.20 Figura 9.2 — Resolução da URL do *embed* 261
A.21 Figura 9.3 — Contrato entre duas experiências 261
A.22 Figura 10.1 — Canal analítico 262
A.23 Figura 11.1 — Camada de IA 262
A.24 Figura 11.2 — *Guardrails* do bem-estar 263
A.25 Figura 12.1 — Os quatro graus de medida 263
A.26 Figura 13.1 — Mapa de rotas da API 264
A.27 Figura 14.1 — A linha da revelação 264
A.28 Figura 15.1 — *Pipeline* de entrega 265
A.29 Figura 16.1 — Estratégia de testes 265
A.30 Figura 17.1 — Cadeia de uma requisição e o que cada elo revela 266
A.31 Figura 17.2 — Erro de cliente entra pela API 266
A.32 Figura 19.1 — Matriz de riscos 266

B Variáveis de ambiente 268

B.1 API — `apps/api/.env` 268
B.1.1 Banco e identidade 268
B.1.2 Inteligência artificial 269
B.1.3 Armazenamento 270
B.1.4 Telemetria e chaves de serviço 270
B.1.5 Observabilidade (Capítulo 17) 271
B.2 Hub — `apps/hub/.env.local` 272
B.3 Experiências — variáveis de *build* do Vite 273
B.4 `vars` do repositório (CI/CD) 274
B.5 Configuração mínima para subir sem GCP 274

C Catálogo de *endpoints* 276

C.1 ai 276

C.2 analise 277
C.3 atividades 277
C.4 auth 282
C.5 bemestar 282
C.6 escolas 284
C.7 estudantes 285
C.8 experiencias 287
C.9 healthz 288
C.10 lotacoes 288
C.11 matriculas 288
C.12 perfis 289
C.13 perfis-recebidos 289
C.14 periodos-letivos 290
C.15 planos 291
C.16 posts 293
C.17 progresso 293
C.18 readyz 294
C.19 rostering 294
C.20 runs 296
C.21 telemetry 299
C.22 turmas 300
C.23 unoogpt 301
C.24 unootube 304
C.25 usuarios 306
C.26 vila 307

D *Payloads* e esquemas de validação 311

D.1 Protocolo `postMessage` (v1) 311
D.2 *Token* de experiência 312

D.3 Telemetria — canal analítico 312
D.4 exp-70 — o *payload* de uma partida 313
D.4.1 As decisões que cada campo carrega 314
D.4.2 As duas travas estruturais 315
D.4.3 O cap de eventos, e por que ele caiu de 2000 para 200 316
D.5 Manifesto de experiência 316
D.6 Validação de variáveis de ambiente 317
D.7 Importação de usuários por CSV 318

E Dicionário de dados 319

E.1 Índice de tabelas 319
E.2 Tabelas da plataforma 320
E.2.1 `escolas` 320
E.2.2 `perfis_portateis` 321
E.2.3 `periodos_letivos` (catálogo) 322
E.2.4 `turmas` 322
E.2.5 `matriculas` 323
E.2.6 `users` 323
E.2.7 `experiencias` (catálogo) 325
E.2.8 `progresso` 325
E.2.9 `ia_uso_diario` 325
E.2.10 `telemetria_eventos` (particionada) 325
E.3 Tabelas da exp-01 326
E.3.1 `exp_01_activities` 326
E.3.2 `exp_01_posts` 327
E.3.3 `exp_01_runs` 328
E.3.4 `exp_01_events` — o canal operacional 328
E.3.5 `exp_01_exp_70_perfis_importados` 329
E.4 Tabelas das demais experiências 329
E.4.1 `exp_02_sessoes` 329
E.5 Tabelas da exp-03 330

E.5.1 `exp_03_sessoes` 330
E.5.2 `exp_03_mensagens` 331
E.5.3 `exp_03_marcos` 331
E.5.4 `exp_03_simulacoes` 331
E.5.5 `exp_03_relogio_turma` 332
E.5.6 `exp_70_sessoes` 332
E.5.7 `exp_70_perfis` 333
E.5.8 `exp_70_bolhas` 334
E.5.9 `exp_90_planos` e `exp_90_referencias` 334
E.5.10 `exp_91_ciclos` e `exp_91_checkins` 335
E.6 O que o Prisma não modela 335

F Operação 337
F.1 Comandos de desenvolvimento 337
F.2 Testes de ponta a ponta 338
F.3 *Design system* 340
F.4 Publicação e reversão 340
F.5 Provisionamento de um contrato novo 341
F.6 Diagnóstico: sintoma → causa 341
F.7 *Checklist* de operação 351
F.8 Compilar esta documentação 353

Referências 354

Lista de figuras

1.1 Visão geral da plataforma . . . 3

4.1 Casos de uso da plataforma . . . 21

5.1 C4 nível 1 — contexto . . . 31

5.2 C4 nível 2 — contêineres . . . 32

5.3 C4 nível 3 — componentes da API . . . 34

5.4 C4 nível 3 — componentes do hub . . . 35

5.5 C4 nível 4 — tipos que carregam a regra . . . 37

5.6 Grafo de dependências do monorepo . . . 38

5.7 Sequência: curadoria de um bloco de feed . . . 39

5.8 Estados de uma geração assíncrona . . . 40

6.1 Implantação de um contrato . . . 42

6.2 Implantação detalhada de um contrato . . . 43

6.3 Identidade de carga de trabalho . . . 46

7.1 Os três níveis de isolamento . . . 51

7.2 Propagação da escola pela requisição . . . 54

8.1 Modelo de dados da plataforma . . . 67

8.2 Enturmamento . . . 70

8.3 Modelo de dados da exp-01: atividade e conteúdo . . . 72

8.4 Modelo de dados da exp-01: sessão do estudante . . . 73

8.5 Modelo de dados das exps 02 e 70 . . . 75

8.6 Modelo de dados das exps 90 e 91 . . . 76

9.1 Sequência do handshake hub-experiência 82
9.2 Resolução da URL do embed 86
9.3 Contrato entre duas experiências 89

10.1 Canal analítico de telemetria 92

11.1 Camada de IA 97
11.2 Guardrails do check-in de bem-estar 101

12.1 Os quatro graus de medida 110

13.1 Mapa de rotas da API 116

14.1 A linha da revelação 124

15.1 Pipeline de entrega 133

16.1 Estratégia de testes 140

17.1 Cadeia de uma requisição e o que cada elo revela hoje 151
17.2 Erro de cliente entra pela API 156

19.1 Matriz de riscos 241

Lista de tabelas

Lista de siglas e abreviaturas

Siglas técnicas

Sigla	Significado
ADC	*Application Default Credentials* — cadeia de resolução de credencial do Google Cloud
ADR	*Architecture Decision Record* — registro de decisão arquitetural
API	*Application Programming Interface*
ANPD	Autoridade Nacional de Proteção de Dados
BFF	*Backend for Frontend* — camada de *proxy* do hub para a API
BNCC	Base Nacional Comum Curricular
C4	Modelo de visualização arquitetural em quatro níveis (Contexto, Contêiner, Componente, Código)
CDN	*Content Delivery Network*
CI/CD	*Continuous Integration / Continuous Delivery*
CORS	*Cross-Origin Resource Sharing*
CSP	*Content Security Policy*
CSV	*Comma-Separated Values*
DASS-21	*Depression, Anxiety and Stress Scales* — versão de 21 itens
DoD	*Definition of Done* — critério de conclusão de tarefa
DPA	*Data Processing Agreement* — acordo de tratamento de dados
DS	*Design System*
EF	Ensino Fundamental (séries `1EF`–`9EF`)
EM	Ensino Médio (séries `1EM`–`3EM`)
ER	Entidade-Relacionamento (modelo de dados)
GCP	*Google Cloud Platform*
HA	*High Availability* — alta disponibilidade
HTTP(S)	*HyperText Transfer Protocol* (*Secure*)
IaC	*Infrastructure as Code* — infraestrutura como código
IdP	*Identity Provider* — provedor de identidade

Sigla	Significado
JSON	*JavaScript Object Notation*
JWK/JWKS	*JSON Web Key* / *JSON Web Key Set*
JWT	*JSON Web Token*
LB	*Load Balancer*
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei 13.709/2018)
LLM	*Large Language Model* — modelo de linguagem de grande escala
OIDC	*OpenID Connect*
OTel/OTLP	*OpenTelemetry* / *OpenTelemetry Protocol* — padrão aberto de instrumentação (rastros, métricas, logs) e seu protocolo de exportação
ORM	*Object-Relational Mapping*
PII	*Personally Identifiable Information* — dado pessoal identificável
PITR	*Point-in-Time Recovery* — recuperação a um instante no tempo
PK/FK	*Primary Key* / *Foreign Key*
REST	*Representational State Transfer*
RF	Requisito Funcional
RLS	*Row-Level Security* — segurança em nível de linha do PostgreSQL
RNF	Requisito Não Funcional
RS256	RSASSA-PKCS1-v1_5 com SHA-256 — algoritmo de assinatura de JWT
RUM	*Real User Monitoring* — medida de desempenho no navegador do usuário real (Web Vitals)
SA	*Service Account* — conta de serviço do GCP
SDK	*Software Development Kit*
SLA/SLO	*Service Level Agreement* / *Objective*
SLI	*Service Level Indicator* — a medida sobre a qual um SLO promete um valor
SQL	*Structured Query Language*
SSR	*Server-Side Rendering*
TLS	*Transport Layer Security*
TTL	*Time To Live* — tempo de validade
UC	Caso de Uso (*Use Case*)
UML	*Unified Modeling Language*
VPC	*Virtual Private Cloud*
WIF	*Workload Identity Federation* — federação de identidade de carga de trabalho
WSO2 IS	WSO2 *Identity Server* — o gerenciador central de identidade do produto
xAPI	*Experience API* — especificação de telemetria de aprendizagem

Termos do domínio

Os nomes de domínio deste projeto são em português brasileiro por convenção explícita (`AGENTS.md`, seção "Convenções de código"); termos técnicos permanecem em inglês. O glossário abaixo fixa o significado de cada um para todo o documento.

Termo	Significado neste projeto
`contrato`	**Tenant de nível 1**: uma rede pública (estado ou município) que contratou a plataforma. Um contrato equivale a uma *stack* de infraestrutura GCP inteira e separada.
`escola`	**Tenant de nível 2**: uma unidade escolar dentro de um contrato. Cem ou mais escolas compartilham a mesma *stack*, isoladas logicamente por `escola_id` + RLS.
`turma`	Agrupamento de estudantes de uma mesma série dentro de uma escola. Chave de balanceamento de vários instrumentos (bolhas, distribuição de lados).
`estudante`	Usuário-estudante. É o único papel cujos dados são de menores, o que governa a maior parte das decisões de retenção e de log.
`professor`	Usuário que prepara e conduz a aula; consome relatórios de turma e usa as ferramentas de professor (exp-90, exp-91).
`gestor`	Usuário administrativo da escola: importa usuários, redefine senhas. Não vê dado de bem-estar de professor.
`admin-contrato`	Único papel autorizado a atravessar escolas, e apenas em rotas explicitamente marcadas. Publica manifestos de experiência.
`experiência`	Aplicação imersiva independente (*bundle* Vite estático) que o hub embeda em `iframe`. Nunca é um serviço.
`hub`	Aplicação Next.js que autentica, roteia por papel, embeda as experiências e desenha todo o *chrome* institucional.
`aparelho`	Metáfora de interface: a casca de dispositivo (`DispositivoFrame`) que o hub desenha em volta do `iframe` quando a experiência declara `viewport`.
`clone`	Superfície de interface que imita um aplicativo real (Instagram, YouTube, WhatsApp, ChatGPT, TikTok) para que a experiência possa medir comportamento em contexto reconhecível.
`revelação`	Momento pedagógico em que a experiência sai da ficção do clone e volta à linguagem institucional. É a linha que separa as seções do *design system*.
`missão`	Unidade pedagógica: uma experiência com objetivo, guia do professor e leitura de resultado.
`manifesto`	Linha da tabela `experiencias` (ou o `public/experiencia.json` do *bundle*) que descreve id, nome, versão ativa, séries, *capabilities*, `viewport`, público e permissões.

Termo	Significado neste projeto
`versaoAtiva`	Versão do *bundle* que o hub serve. *Rollback* é a troca deste campo — não um novo *deploy*.
`bolha`	Nível de dificuldade do instrumento da exp-70 (A, B ou C), atribuído pelo servidor e estável por estudante.
`grau`	Quanto uma partida sustenta a medida de um valor: `scored`, `indicative`, `zero` ou `none`. A diferença entre `zero` e `none` é invariante de API.
`run`	Sessão de um estudante numa atividade da exp-01: carrega o estado do motor de feed adaptativo.
`comTenancy`	Porta única de acesso ao banco na API: abre transação e arma a RLS com a escola do *token*.

1

Introdução

1.1 Propósito deste documento

Este documento descreve a arquitetura, o escopo e a implementação da **Plataforma Unooria**: uma plataforma de educação básica destinada a redes públicas de ensino, composta por um *hub* web multi-tenant e um catálogo alvo de 48 experiências imersivas que cobrem do 1º ano do Ensino Fundamental ao 3º ano do Ensino Médio.

A descrição arquitetural segue a norma ISO/IEC/IEEE 42010 (International Organization for Standardization, 2022): identifica os interessados, enuncia os *drivers* que restringem o desenho, apresenta visões complementares do sistema e registra as decisões com suas consequências. A notação de visualização é o modelo C4 (BROWN, 2026; BROWN, 2024), aplicado *duas vezes* — uma para a arquitetura de *software* (Capítulo 5) e outra para a arquitetura de nuvem e implantação (Capítulo 6) — porque neste projeto os dois recortes não coincidem: há contêineres lógicos que não são serviços em execução, e há serviços em execução que não são contêineres lógicos novos.

> **NOTA**
>
> Todo diagrama deste documento é vetorial, desenhado nativamente em TikZ. Não há imagem rasterizada de diagrama, nem dependência de ferramenta externa de renderização. Para quem precisa editar, revisar ou versionar o diagrama fora do LaTeX, o Apêndice A traz a fonte Mermaid equivalente de cada figura, referenciando o mesmo número.

1.2 O problema

Uma plataforma educacional para rede pública tem três exigências que, juntas, determinam quase toda a arquitetura — e que raramente aparecem juntas em produto educacional comercial:

1. **Isolamento contratual forte, isolamento escolar barato.** Cada contrato é uma relação jurídica com um ente federativo. Vazamento de dado entre *contratos* é inaceitável; por isso

cada contrato recebe uma *stack* de infraestrutura própria. Já as cem escolas de um mesmo contrato não podem custar cem infraestruturas: elas compartilham a *stack* e são isoladas logicamente.

2. **Pico de carga previsível e brutal.** A rede toda entra na plataforma às oito da manhã. O alvo de projeto é de aproximadamente dez mil estudantes por contrato com dois a quatro mil simultâneos nesse pico — uma curva de carga que não se parece com a de um SaaS de escritório.
3. **Dados de menores.** A maior parte dos titulares dos dados tratados é criança ou adolescente. Isso não é um detalhe de conformidade tratado no fim: é um *driver* arquitetural que decide o que se registra, onde se registra, quem lê e o que o relatório está *proibido* de afirmar (Brasil, 2018; Autoridade Nacional de Proteção de Dados, 2024).

A quarta restrição não é do domínio, é do projeto, e é a mais severa: **uma pessoa desenvolve, com prazo de produção em novembro de 2026**. Isso transforma "simplicidade" de preferência estética em requisito de viabilidade. Cada peça de infraestrutura nova é uma peça que uma pessoa vai operar de plantão; cada abstração a mais é código que ninguém revisa em par.

> **DECISÃO DE PROJETO**
>
> A regra de projeto que decorre disso está escrita no documento de governança do repositório e é aplicada em todo este documento: **simplicidade** > **sofisticação**. Nenhum serviço, fila, cache ou biblioteca de estado entra sem *gargalo medido*. As duas exceções são decisões tomadas de antemão e justificadas: o Pub/Sub da telemetria (**ADR-06**) e o *provider* plugável de IA (**ADR-07**).

1.3 A solução em uma página

A plataforma é um *monorepo* com quatro tipos de artefato, e a distinção entre eles é a decisão arquitetural mais consequente do projeto:

Artefato	Tecnologia	Como é entregue
`apps/hub`	Next.js (App Router)	Contêiner no Cloud Run
`apps/api`	NestJS + PostgreSQL	Contêiner no Cloud Run
`apps/experiencias/*`	Vite + React + `@edu/sdk`	***Bundle* estático** em Cloud Storage atrás de CDN
`packages/*`	TypeScript puro	Consumido por *workspace*; nunca implantado sozinho

> **REGRA INVIOLÁVEL**
>
> **Experiências nunca viram serviços.** Uma experiência é um conjunto de arquivos estáticos em `gs://{contrato}-exp/{expId}/{versao}/`. Não há processo, não há porta, não há *scaling*. Isso vale para as 48: o custo marginal de publicar a quadragésima oi-

> tava experiência é o custo de copiar arquivos para um *bucket*. *Rollback* é o hub passar a apontar para a versão anterior — uma linha de `UPDATE`, não um *deploy*.

A Figura 1.1 resume o caminho de uma sessão de estudante, do *login* à coleta de telemetria.

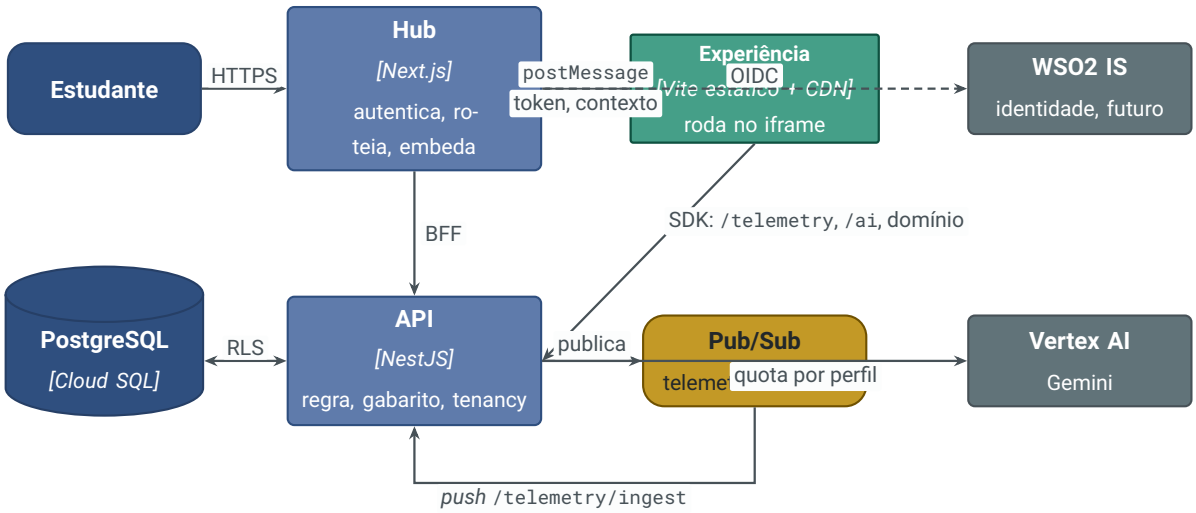


Figura 1.1 – Visão geral: o hub é a única porta do usuário; a experiência é estática e recebe o contexto por `postMessage`; a API concentra regra, gabarito e *tenancy*; a telemetria analítica sai pelo Pub/Sub e volta à própria API por *push*.

Legenda: pessoa · contêiner da plataforma · artefato estático · banco de dados · fila · sistema externo

1.4 Como ler este documento

1.4.1 Estrutura

Cap.	Título	Responde a
1–2	Introdução, contexto de negócio	Por que a plataforma existe e sob que restrições
3–4	Requisitos, casos de uso	O que ela deve fazer, e para quem
5	Arquitetura de *software* (C4)	Como o *software* está organizado
6	Arquitetura de nuvem (C4 + implantação)	Onde ele roda e quanto custa operá-lo
7	*Multi-tenancy* e segurança	Como um dado de escola não vaza para outra
8	Modelo de dados	Que dados existem e por quê
9–11	Contratos, telemetria, IA	As três fronteiras que mais custam se quebrarem
12–14	Experiências, API, *design system*	O catálogo, a superfície HTTP e a interface

15–16	Entrega e qualidade	Como o código chega à escola e o que o impede de chegar errado
17	Observabilidade	Como se sabe que ela está funcionando na escola, e quem é avisado quando não está
18–20	ADRs, riscos, conclusão	As decisões, o que pode dar errado e o estado atual
A–F	Apêndices	Mermaid, variáveis, *endpoints*, *payloads*, dicionário de dados, operação

1.4.2 Convenções tipográficas

Marca	Significado
`identificador`	Identificador de código: campo, função, tipo, coluna, valor.
`caminho/arquivo.ts`	Caminho de arquivo ou diretório no repositório.
`GET /rota/:id`	*Endpoint* HTTP da API.
`VARIAVEL`	Variável de ambiente.
RF-01, **RNF-01**	Requisito funcional / não funcional (Capítulo 3).
UC-01	Caso de uso (Capítulo 4).
ADR-01	Decisão arquitetural registrada (Capítulo 18).
DR-01	*Driver* arquitetural (Seção 2.4).

Os blocos destacados têm significado fixo ao longo do documento:

> **NOTA**
>
> Informação de contexto que ajuda a entender uma escolha, mas cuja ausência não quebra nada.

> **DECISÃO DE PROJETO**
>
> Decisão de projeto tomada deliberadamente, com a alternativa descartada e o motivo. Toda decisão deste tipo tem uma entrada correspondente no Capítulo 18.

> **REGRA INVIOLÁVEL**
>
> Invariante que não pode ser violada por nenhuma implementação futura. As sete regras invioláveis da plataforma estão consolidadas na Seção 2.6.

> **ATENÇÃO**
>
> Armadilha conhecida: algo que já falhou, ou que falha silenciosamente. Quase todos estes blocos registram um erro efetivamente cometido e medido no projeto.

LIMITAÇÃO CONHECIDA

Limitação assumida do estado atual, com o caminho de saída. Não é bug: é escopo que ficou de fora, de propósito.

1.5 O que este documento não cobre

- **O código Terraform.** Ele vive em repositório separado. O *contrato* entre os dois repositórios — nomes de recursos esperados, variáveis consumidas, artefatos publicados — está no Capítulo 6 e em `docs/INFRA.md`.
- **O conteúdo pedagógico.** O catálogo de missões, os textos e a trilha por ano são produto, não arquitetura. O Capítulo 12 descreve a *mecânica* de cada experiência apenas até onde ela impõe exigências ao *software*.
- **O *design system*.** Os componentes vivem em repositório próprio (`storybook-unooria`) e chegam aqui como pacote npm publicado. O Capítulo 14 documenta as *regras de consumo*, não o catálogo de componentes.
- **As 43 experiências ainda não escritas.** O Capítulo 12 documenta as cinco existentes e o *gabarito* que as próximas devem seguir.

2

Contexto de negócio e restrições

2.1 Modelo de negócio

A Plataforma Unooria é vendida **B2G**: o cliente é um ente federativo — uma secretaria estadual ou municipal de educação — e não a escola nem a família. A consequência arquitetural imediata é que a *unidade comercial* e a *unidade de uso* são diferentes, e o desenho precisa acomodar as duas.

Nível	Entidade	Isolamento
1	`contrato` (rede)	**Físico**: *stack* GCP inteira e separada, provisionada por um *module* Terraform por contrato. Projeto, *buckets*, banco, *Load Balancer* e contas de serviço próprios.
2	`escola`	**Lógico**: `escola_id` obrigatório em toda tabela de dado de estudante, mais *Row-Level Security* no PostgreSQL. Cem ou mais escolas por *stack*.
3	`turma` / `estudante`	**Autorização**: *claims* do JWT (`turma_id`, `perfil`) verificados na API por `RolesGuard` e por filtro explícito nas *queries*.

> **DECISÃO DE PROJETO**
>
> **Uma *stack* por contrato, escolas por RLS** (ADR-04). A alternativa "uma *stack* por escola" daria isolamento físico perfeito e transformaria a empresa em operadora de cem infraestruturas por contrato — inviável para uma pessoa. A alternativa oposta, uma *stack* única para todos os contratos, tornaria um incidente em um município um incidente em todos, e não sobreviveria à primeira análise jurídica de um edital. O ponto escolhido separa fisicamente o que é contratual e logicamente o que é operacional.

2.2 Escala alvo

Grandeza	Alvo por contrato	Consequência

Estudantes cadastrados	~10.000	Cabe folgadamente numa instância pequena de Cloud SQL.
Simultâneos no pico (8h)	2.000–4.000	`min-instances` no Cloud Run para eliminar *cold start*; *pooler* de conexões obrigatório.
Escolas por *stack*	100+	RLS e `escola_id` em todo lugar; nenhuma *query* cruza escolas fora de `admin-contrato`.
Experiências no catálogo alvo	48	Custo marginal de publicação *deve* ser próximo de zero ⇒ *bundle* estático.
Faixa etária	1º EF ao 3º EM	Instrumentos e interface variam por série; regras de instrumento mudam nos anos iniciais.

O perfil de carga é o dado mais importante desta tabela e merece ser dito por extenso: **o tráfego não é distribuído, é uma parede**. Entre 7h50 e 8h10 de um dia letivo, uma fração grande da rede faz *login* e abre uma experiência quase ao mesmo tempo. O desenho responde a isso com três medidas independentes:

1. O grosso dos *bytes* do pico — os *bundles* das experiências e as imagens geradas — não passa pela aplicação. Sai da CDN, com o caminho versionado e, portanto, imutável e cacheável indefinidamente (**ADR-01**).
2. Os serviços têm instâncias mínimas provisionadas, então o pico não paga inicialização de contêiner nem *warm-up* de *pool* do Prisma.
3. A telemetria de alto volume não é síncrona: vai para o Pub/Sub e é gravada em lote, fora do caminho da requisição do estudante (Capítulo 10).

2.3 Restrições de projeto

ID	Restrição	Origem e consequência
RE-01	Uma pessoa desenvolve e opera	Restrição de recurso. Nenhuma peça de infraestrutura sem gargalo medido; preferência sistemática por absorver capacidade na API existente em vez de criar serviço novo.
RE-02	Produção em novembro de 2026	Restrição de prazo. Ordem de implementação por milestones (M0–M6); o que não bloqueia a primeira rede fica documentado como evolução (Capítulo 19).
RE-03	Identidade centralizada em WSO2 IS	Exigência do cliente e da estratégia multi-contrato. A aplicação nunca conhece o WSO2: conhece OIDC (**ADR-03**).
RE-04	Google Cloud Platform	Decisão de plataforma. Serviços gerenciados preferidos a orquestração própria; Cloud Run em vez de GKE.

RE-05	Interface exclusivamente do *design system*	Decisão de produto e de marca. Nenhum componente de UI nasce neste repositório; o que falta nasce no `storybook-unooria` (Capítulo 14).
RE-06	Dados de menores	LGPD art. 14 e orientação da ANPD (Brasil, 2018; Autoridade Nacional de Proteção de Dados, 2024). Governa telemetria, log, retenção e *o que o relatório pode afirmar*.
RE-07	Dado de saúde de professor	LGPD art. 11 (dado sensível). Determina a ausência deliberada de visão de gestor na exp-91.

2.4 Os *drivers* arquiteturais

Um *driver* arquitetural é uma exigência que, se ignorada, invalida a solução — não a piora, invalida. Foram sete.

ID	Driver	Consequência arquitetural
DR-01	Isolamento verificável entre escolas	`escola_id NOT NULL` + `FORCE ROW LEVEL SECURITY` em toda tabela de dado de estudante; acesso ao banco por uma *única* porta (`PrismaService.comTenancy`) que arma a política antes de qualquer trabalho. *Teste* de *tenancy* com *role* sem *ownership* (`tests/tenancy.spec.ts`).
DR-02	Custo marginal quase nulo por experiência	Experiência é *bundle* estático versionado em *bucket*, nunca um serviço. *Rollback* por ponteiro (`versaoAtiva`), não por *deploy* (**ADR-01**).
DR-03	Correção e pontuação não podem estar no cliente	Gabarito, correção e qualquer nota são *server-side*, num módulo de domínio por experiência na API. O *bundle* nunca recebe a resposta certa (**ADR-09**).
DR-04	Troca de provedor de identidade sem tocar código	Validação de *token* por OIDC genérico: `OIDC_ISSUER` e `OIDC_JWKS_URI`. Hoje o emissor é o próprio hub; amanhã é o WSO2, e o `OidcGuard` não muda (**ADR-03**).
DR-05	IA com custo previsível e sem segredo no cliente	Toda chamada de IA passa por `apps/api/src/ai`, com quota diária persistida *por perfil* verificada antes da chamada. Nenhuma chave, URL interna ou *prompt* de sistema em *bundle* (**ADR-07**).
DR-06	O pico da manhã não pode derrubar o transacional	Dois canais de telemetria com garantias diferentes: o operacional é síncrono e transacional; o analítico é assíncrono via Pub/Sub, com *insert* multi-linha e tabela particionada por mês (**ADR-06**, **ADR-14**).
DR-07	O relatório não pode produzir rótulo sobre uma criança	Invariantes *de tipo*, não de tela: valor com uma única fonte de medida não recebe número; "não teve oportunidade" nunca é lido como "escolheu não"; listagem de turma não exibe valor psicométrico (**ADR-12**).

> **NOTA**
>
> **DR-07** é incomum como *driver* arquitetural, porque parece uma questão de produto. Ele está aqui porque foi implementado como *estrutura*: a API devolve uma união discriminada em que só a variante `scored` tem `numero: number`; as outras têm `null` *por tipo*. O hub fica estruturalmente incapaz de imprimir "Poder 100" a partir de um único sinal — ele nunca recebe esse número, e o compilador nem deixa procurá-lo. Prosa num comentário não é controle; um tipo é.

2.5 Interessados

Interessado	O que precisa	Onde é atendido
Estudante	Entrar, escolher a missão, jogar sem atrito, ver o próprio resultado	Hub do estudante (`/estudante`), `/exp/*`, dossiê da experiência
Professor	Preparar a aula, conduzir o momento coletivo, ler a turma, gerar plano de aula, cuidar do próprio bem-estar	`/professor/*`, exp-90, exp-91, relatórios por turma
Gestor escolar	Cadastrar e manter usuários da escola	`/admin/usuarios`, importação CSV
Secretaria (rede)	Visão agregada da rede, sem identificar criança	`/analise/*` (papel `admin-contrato`)
Encarregado de dados (DPO)	Saber o que é coletado, por quanto tempo, quem lê, e onde o dado é processado	Capítulos 7 e 10; **ADR-11**, **ADR-13**
Desenvolvedor	Adicionar experiência sem reaprender a plataforma	SDK, `_template`, *scaffolding* (`pnpm new:exp`), Capítulo 12
Operação	Subir, observar e reverter sem adivinhar	CI/CD (Capítulo 15), Apêndice F

2.6 As sete regras invioláveis

Estas regras estão registradas em `AGENTS.md` e são normativas: uma implementação que as viole está errada, independentemente de passar nos testes. Cada uma tem o capítulo onde é detalhada e verificada.

#	Regra	Detalhe em
1	Toda *query* SQL é escopada por `escola_id` (e `estudante_id` quando aplicável). Tabelas de dado de estudante têm `escola_id NOT NULL` + RLS. Nenhuma *query* atravessa escolas, exceto em rotas `admin-contrato`.	Cap. 7
2	Nenhum segredo em *bundle* de cliente: sem chave de API, sem *prompt* de sistema, sem URL interna. IA só pela API.	Cap. 11
3	Resposta correta, correção e pontuação que vale nota são *server-side*. O cliente envia a resposta do estudante; a API corrige.	Cap. 12
4	O *token* é obtido pelo hub via OIDC e repassado à experiência por `postMessage` com `targetOrigin` explícito — nunca `"*"`. A experiência mantém o *token só em memória*.	Cap. 9
5	IA tem quota diária por usuário *e por perfil*, e *rate limit*. Nenhuma chamada de IA sem passar pelo `QuotaService`: `temSaldo` antes, `consumir` depois.	Cap. 11
6	Dados de menores: nenhuma PII em log em texto claro. A telemetria carrega `estudante_id` opaco, nunca nome ou *e-mail*.	Cap. 10
7	Nenhuma migração remove `escola_id` ou afrouxa RLS. Renomeações usam `RENAME`, nunca `DROP`/`CREATE`.	Cap. 8

2.7 Fora de escopo, deliberadamente

O que *não* entra é parte do desenho. As exclusões abaixo são decisões, não omissões, e cada uma tem um gatilho explícito de reavaliação (Capítulo 19).

Descartado	Motivo e gatilho de reavaliação
Kubernetes (GKE)	Cloud Run entrega *autoscaling*, TLS e revisões sem *control plane* para operar. Gatilho: necessidade de *workload* de longa duração ou topologia de rede que o Cloud Run não expresse.
AlloyDB	A robustez vem de HA + PITR + *pooler*, não do *engine*. Ambos são PostgreSQL; migrar depois é trivial (**ADR-05**).
Microsserviço por experiência	Seriam 48 serviços para operar, cada um com *deploy*, log e escala próprios — para servir arquivos estáticos (**ADR-01**).
Redis como *buffer* de telemetria	Quem amortece o pico já é o Pub/Sub, e ele é *durável*: um *buffer* em memória perde os eventos do processo que morre antes do *flush*. O gargalo medido era o laço de `INSERT`, e ele foi corrigido (**ADR-14**).

BigQuery para o canal analítico	Nenhum relatório atual lê `telemetria_eventos`; seria uma segunda plataforma de dados para operar sem resolver problema existente (**ADR-14**).
Multi-região	Uma rede é geograficamente concentrada. Gatilho: exigência contratual de continuidade regional.
Worker dedicado de telemetria	Substituído por *push subscription* para a própria API — uma peça a menos para operar. Gatilho: volume que justifique isolar o consumo.

3

Requisitos

Este capítulo especifica os requisitos da plataforma de forma rastreável: cada requisito recebe identificador estável, prioridade, estado e o ponto do sistema onde é verificado. Requisito sem ponto de verificação é intenção, não requisito.

A prioridade segue MoSCoW: **M** (*must*) bloqueia a entrada em produção; **S** (*should*) é esperado na primeira rede; **C** (*could*) é evolução planejada. O estado reflete o repositório em 27 de agosto de 2026.

3.1 Requisitos funcionais

3.1.1 Identidade, papéis e acesso

ID	Requisito	Pri	Estado	Verificação
RF-01	Autenticar usuário por credencial da rede e emitir sessão.	M	Implementado	`apps/hub/lib/auth.ts`; `POST /auth/login`
RF-02	Emitir *token* de experiência assinado em RS256, com TTL de 15 minutos e *claims* `{sub, escola_id, turma_id?, perfil}`, audiência `plataforma-api`.	M	Implementado	`lib/chave-token.ts`; `GET /.well-known/jwks.json`
RF-03	Validar *token* na API por JWKS do emissor configurado, sem conhecer o produto de identidade.	M	Implementado	`src/auth/oidc.guard.ts`
RF-04	Restringir rota por papel (`estudante`, `professor`, `gestor`, `admin-contrato`).	M	Implementado	`src/auth/roles.ts`

ID	Requisito	Pri	Estado	Verificação
RF-05	Redirecionar usuário que acessa área de outro papel para a home do próprio papel, sem expor 403.	S	Implementado	`apps/hub/middleware.ts`; `lib/papeis.test.ts`
RF-06	Importar usuários em massa por CSV (`nome,usuario,senha,papel,tu`	M	Implementado	`POST /usuarios/importar-csv`; `tests/usuarios.spec.ts`
RF-07	Redefinir senha de usuário da escola.	S	Implementado	`POST /usuarios/:id/reset-senha`
RF-08	Trocar o emissor de identidade para o WSO2 IS alterando apenas configuração.	M	Preparado	`OIDC_ISSUER`, `OIDC_JWKS_URI`; `wso2/`

3.1.2 Experiências e *embed*

ID	Requisito	Pri	Estado	Verificação
RF-10	Manter manifesto de versões por experiência e servir a versão ativa.	M	Implementado	Tabela `experiencias`; `PUT /experiencias/:expId`
RF-11	Reverter versão de experiência trocando `versaoAtiva`, sem *build* nem *deploy*.	M	Implementado	`scripts/publicar-manifesto.mjs`
RF-12	Embutir a experiência em `iframe` com *sandbox* mínimo, concedendo capacidade extra apenas a quem a declara no manifesto.	M	Implementado	`components/ExpFrame.tsx`; `scripts/probe-pointerlock.mjs`
RF-13	Entregar *token* e contexto à experiência por `postMessage` com `targetOrigin` explícito.	M	Implementado	`packages/sdk/src/bridge.ts`; `bridge.test.ts`
RF-14	Renovar *token* da experiência sob demanda, sem recarregar o `iframe`.	M	Implementado	`exp:token-expirando` → `GET /api/exp-token` → `hub:token`
RF-15	Pausar e retomar a experiência quando a página do hub perde ou recupera visibilidade/foco.	S	Implementado	`hub:pause` / `hub:resume`
RF-16	Desenhar casca de aparelho quando a experiência declara `viewport`, lendo o manifesto *ou* o `experiencia.json` do *bundle*.	S	Implementado	`lib/embed-experiencia.ts`; `components/DispositivoFrame.tsx`

ID	Requisito	Pri	Estado	Verificação
RF-17	Persistir e recuperar progresso genérico por (`escola`, `estudante`, `experiência`).	M	Implementado	`GET/POST /progresso/:expId`
RF-18	Expor uma bancada local de *embed* para desenvolvimento (`/dev-embed?src=`).	C	Implementado	`app/dev-embed`

3.1.3 Telemetria

ID	Requisito	Pri	Estado	Verificação
RF-20	Coletar evento analítico no padrão xAPI simplificado, em lote de até 50 eventos com *flush* a cada 10 segundos.	M	Implementado	`packages/sdk/src/telemetry.ts`
RF-21	Enviar o último lote no `pagehide` por `sendBeacon`, com o *token* no corpo.	M	Implementado	`POST /telemetry/beacon`
RF-22	Derivar identidade do evento do *token*, nunca do corpo enviado pelo cliente.	M	Implementado	`src/telemetry/telemetry.controller.ts`
RF-23	Gravar o canal analítico de forma idempotente sob entrega *at least once*.	M	Implementado	`ON CONFLICT (event_id, ts) DO NOTHING`
RF-24	Autenticar o consumo do *push* do Pub/Sub, falhando fechado na ausência de configuração.	M	Implementado	`src/telemetry/pubsub-push.guard.ts`; `tests/telemetria-push.spec.ts`
RF-25	Manter o canal operacional (motor de feed) síncrono e transacional, fora do Pub/Sub.	M	Implementado	`POST /runs/:id/eventos`, `GET /runs/:id/feed`

3.1.4 Inteligência artificial

ID	Requisito	Pri	Estado	Verificação
RF-30	Concentrar toda chamada a provedor de IA num único módulo com interface neutra de *vendor*.	M	Implementado	`src/ai/provider.ts`

ID	Requisito	Pri	Estado	Verificação
RF-31	Verificar quota diária por usuário *e por perfil* antes da chamada, e registrar consumo depois.	M	Implementado	`src/ai/quota.service.ts`; tabela `ia_uso_diario`
RF-32	Autenticar no provedor por ADC, sem chave de API em variável de ambiente.	M	Implementado	`src/ai/vertex.provider.ts`; `tests/vertex-provider.spec.ts`
RF-33	Oferecer *provider* de teste (`mock`) que permita subir sem GCP.	M	Implementado	`AI_PROVIDER=mock`
RF-34	Detectar corte de saída do modelo (`finishReason` = `MAX_TOKENS`) e tratá-lo como erro, não como conteúdo.	M	Implementado	`src/planos/planos.service.ts`
RF-35	Gerar plano de aula anual de forma assíncrona, com *claim* atômico e consulta por *polling*.	M	Implementado	`POST /planos` (202), `GET /planos/:id`
RF-36	Guardar conteúdo autoral do professor em armazenamento privado, fora do *bucket* público.	M	Implementado	`{contrato}-planos`; `src/object-storage`
RF-37	Validar todo texto gerado por IA que chegue ao professor no contexto de bem-estar (sem vocabulário clínico, ordem, promessa ou *link*), com *fallback* fixo.	M	Implementado	`src/bemestar/regras.ts`; `tests/bemestar-regras.spec.ts`

3.1.5 Pedagógico e relatórios

ID	Requisito	Pri	Estado	Verificação
RF-40	Corrigir e pontuar no servidor; nunca enviar gabarito ao *bundle*.	M	Implementado	`src/unootube/`, `src/motor/`, `src/vila/`
RF-41	Curar o feed adaptativo por blocos, de forma determinística dado um gerador de aleatoriedade.	M	Implementado	`src/motor/curadoria.ts`; `tests/motor-pesos.spec.ts`
RF-42	Serializar a curadoria por `run` com *lock* consultivo, para que dois pedidos simultâneos não descartem um bloco.	M	Implementado	`pg_advisory_xact_lock`; `tests/motor-concorrencia.spec.ts`

ID	Requisito	Pri	Estado	Verificação
RF-43	Atribuir bolha (A/B/C) balanceada dentro da turma, no servidor, e mantê-la estável entre partidas.	M	Implementado	`exp_70_bolhas`; `tests/vila-regras.spec.ts`
RF-44	Não servir a bolha B nos anos iniciais (1EF–5EF).	M	Implementado	`bolhasElegiveis`
RF-45	Recalcular o perfil de valores no servidor a partir das medidas brutas, recusando *payload* que traga o perfil.	M	Implementado	`CHAVES_DE_PERFIL` em `src/vila/contrato.ts`
RF-46	Permitir reprocessar o histórico quando a versão do mapeamento subir.	M	Implementado	`exp_70_sessoes.payload`; `versaoMapeamento`
RF-47	Devolver leitura em que só a variante `scored` tem número; as outras têm `null` *por tipo*.	M	Implementado	`src/vila/leitura.ts`
RF-48	Abrir o relatório individual pelo *nível* do instrumento antes do desempenho.	M	Implementado	`/professor/entre-redes/vila-do-lume/est`
RF-49	Distinguir “cartões servidos” de “cartões que apareceram”, e nunca afirmar omissão a partir de ausência.	M	Implementado	`cartoesServidos`
RF-50	Zerar os dados da turma de uma experiência, apagando também a atribuição de bolha, e *sem* tocar dados de outra experiência.	S	Implementado	`DELETE /vila/professor/turmas/:turmaId/dados`
RF-51	Importar o perfil da Vila como largada do Instanooria, preservando redistribuição manual do professor.	S	Implementado	`POST /atividades/:id/perfis/da-vila`; `tests/vila-ato1.spec.ts`
RF-52	Exportar análise por turma e por rede em CSV.	S	Implementado	`GET /atividades/:id/analise/csv`, `GET /analise/csv`
RF-53	Não expor visão de gestor nem agregado para o dado de bem-estar do professor.	M	Implementado (por ausência)	`src/bemestar/bemestar.controller.ts`
RF-54	Permitir ao professor apagar integralmente os próprios dados de bem-estar.	M	Implementado	`DELETE /bemestar/dados`

3.2 Requisitos não funcionais

Organizados pelas características de qualidade de produto da ISO/IEC 25010 (International Organization for Standardization, 2023).

ID	Característica	Requisito e critério	Estado
RNF-01	Desempenho — capacidade	Suportar 2.000–4.000 sessões simultâneas por contrato no pico, sem *cold start*: `min-instances` $\geq$ 1 na API e no hub, concorrência 80 na API.	Configurado; teste de carga pendente
RNF-02	Desempenho — vazão de ingestão	O caminho de gravação da telemetria analítica deve custar $O(\text{lotes})$ e não $O(\text{eventos})$ em *round-trips* ao banco.	Implementado (`INSERT` multi-linha)
RNF-03	Desempenho — borda	*Bundles* e mídia servidos por CDN com caminho imutável; só o `index.html` tem *cache* curto (60 s).	Implementado
RNF-04	Confiabilidade	Banco com HA e PITR; *pooler* de conexões obrigatório; *deploy* por revisões reversíveis.	Provisionado
RNF-05	Confiabilidade — entrega	Perda de evento analítico tolerada apenas quando o processo morre antes do *flush*; a partir do Pub/Sub, a entrega é durável e *at least once* com gravação idempotente.	Implementado
RNF-06	Segurança — isolamento	Nenhuma *query* de dado de estudante pode retornar linha de outra escola, mesmo com erro de aplicação. Verificado por RLS forçada e por teste com *role* sem *ownership*.	Implementado
RNF-07	Segurança — segredo	Zero segredo em artefato de cliente. Segredos em Secret Manager; credencial de IA por ADC.	Implementado
RNF-08	Segurança — superfície	*Sandbox* de `iframe` mínimo; `targetOrigin` explícito; toda mensagem validada por esquema nos dois lados; CORS por lista explícita.	Implementado
RNF-09	Segurança — *fail closed*	Rota cuja proteção depende de configuração responde 401/503 na ausência dela, nunca fica aberta.	Implementado
RNF-10	Privacidade	Telemetria sem PII; texto livre sensível nunca vai a log nem à telemetria; IA não recebe nome, escola, pontuação nem faixa clínica.	Implementado
RNF-11	Manutenibilidade	TypeScript *strict* sem `any` (uma exceção registrada em ADR); validação de todo *input* externo por Zod, inclusive variáveis de ambiente, no *boot*.	Implementado
RNF-12	Manutenibilidade — acoplamento	Nenhum módulo fora de `src/ai` conhece o *vendor* de IA; nenhum componente fora de `packages/ui` importa o *design system* diretamente.	Implementado

ID	Característica	Requisito e critério	Estado
RNF-13	Portabilidade	Provedor de identidade e provedor de IA substituíveis por configuração ou por arquivo novo, sem alterar chamadores.	Implementado
RNF-14	Testabilidade	A plataforma sobe inteira sem GCP (`AI_PROVIDER=mock`, `ASSETS_DRIVER=local`, `PLANOS_DRIVER=local`, `TELEMETRIA_TOPIC` vazio).	Implementado
RNF-15	Observabilidade	*Smoke* visual obrigatório para mudança de tela — *typecheck* não detecta ícone virando texto nem `iframe` vazio. Em produção: log estruturado com correlação, alerta por sintoma (SLO) e erro de cliente registrado pela API (Capítulo 17).	Parcial — só o *smoke*
RNF-16	Custo	Custo marginal de experiência publicada ≈ armazenamento + tráfego de CDN. Nenhum serviço por experiência.	Implementado
RNF-17	Conformidade	Localidade de processamento de IA declarada e auditável; ausência de prazo de retenção é decisão registrada, não omissão.	Documentado

LIMITAÇÃO CONHECIDA

RNF-01 está *configurado* mas não *verificado*: o teste de carga com 4.000 *logins* e aberturas de experiência em dez minutos é obrigatório antes de novembro de 2026 e ainda não foi executado. É o maior risco técnico aberto do projeto (Capítulo 19, `R-01`).

3.3 Rastreabilidade

A matriz abaixo liga *drivers*, requisitos-chave e as decisões que os realizam. Ela é a resposta à pergunta “por que o sistema é assim” em uma única página.

Driver	Requisitos	Decisões (ADR)
DR-01	**RNF-06**, **RF-04**	**ADR-04** (*stack* por contrato), **ADR-08** (RLS forçada e porta única)
DR-02	**RF-10**, **RF-11**, **RNF-03**, **RNF-16**	**ADR-01** (*bundle* estático), **ADR-02** (*monorepo*)
DR-03	**RF-40**, **RF-45**	**ADR-09** (domínio por experiência na API)
DR-04	**RF-03**, **RF-08**, **RNF-13**	**ADR-03** (OIDC genérico), **ADR-15** (hub como emissor provisório)
DR-05	**RF-30**–**RF-34**, **RNF-07**	**ADR-07** (*gateway* de IA na API), **ADR-16** (ADC e modelo *versus* região)

DR-06	**RF-20**–**RF-25**, **RNF-02**	**ADR-06** (Pub/Sub), **ADR-14** (Postgres particionado, sem Redis nem BigQuery)
DR-07	**RF-47**–**RF-49**, **RF-53**	**ADR-12** (`grau` como invariante de API), **ADR-10** (bem-estar sem visão de gestor)

4

Casos de uso

Os casos de uso descrevem o sistema pela ótica de quem o usa. Cada um recebe identificador estável, ator primário, pré-condição, fluxo principal, fluxos alternativos e a regra de negócio que o governa. A rastreabilidade para requisitos está na Seção 4.4.

4.1 Atores

Ator	Tipo	Descrição
Estudante	Primário	Estudante da rede. Escolhe a missão no hub próprio e entra na experiência. É titular de dado de menor.
Professor	Primário	Prepara e conduz a aula, lê a turma, gera plano de aula e faz o próprio *check-in* de bem-estar.
Gestor escolar	Primário	Administra usuários da escola.
Admin do contrato	Primário	Único papel autorizado a atravessar escolas. Publica manifestos e lê o agregado da rede.
Provedor de identidade	Secundário	WSO2 Identity Server (futuro). Hoje o papel de emissor é exercido pelo próprio hub.
Vertex AI (Gemini)	Secundário	Geração de texto e de imagem, sempre intermediada pela API.
Pub/Sub	Secundário	Transporte durável do canal analítico de telemetria.
CI/CD (GitHub Actions)	Secundário	Publica *bundles* e faz *upsert* do manifesto. É um ator: age no sistema por API autenticada.

4.2 Diagrama de casos de uso

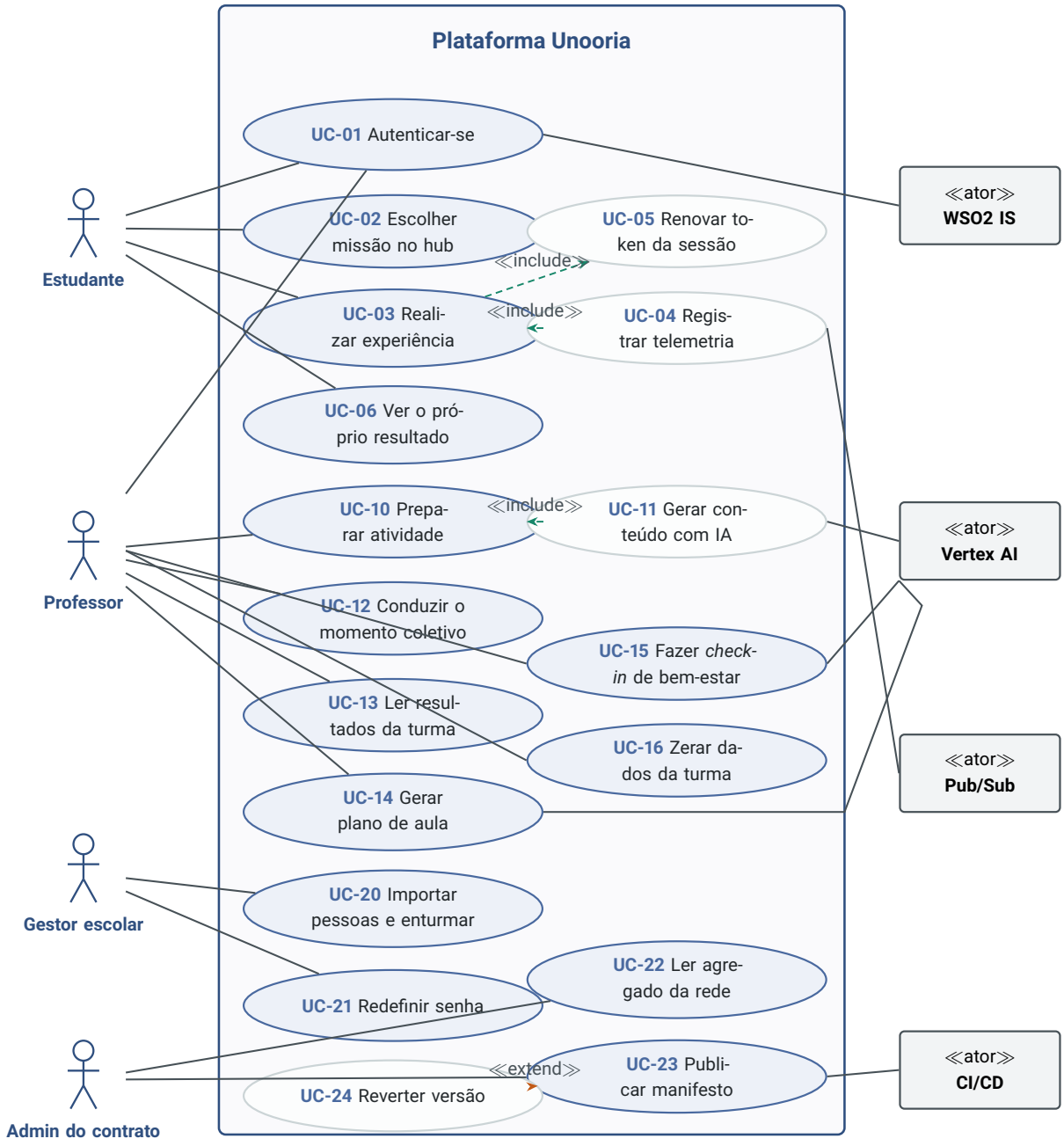


Figura 4.1 – Casos de uso da plataforma. As elipses em tom claro são casos de uso de *suporte*, incluídos por outros e nunca iniciados diretamente por um ator humano. Os quatro papéis são mutuamente exclusivos: o *middleware* do hub redireciona quem entra na área de outro papel.

Legenda: caso de uso primário · caso de uso de suporte · - - - ≪include≫ · - - - ≪extend≫

4.3 Especificação dos casos de uso

4.3.1 UC-01 — Autenticar-se

> **UC-01 Autenticar-se**
>
> | | |
> |---|---|
> | **Ator primário** | Qualquer usuário |
> | **Pré-condição** | Usuário existe na escola (criado por **UC-20**) |
> | **Pós-condição** | Sessão do hub estabelecida; usuário na *home* do próprio papel |
> | **Requisitos** | **RF-01**, **RF-02**, **RF-05** |

Fluxo principal

1. O usuário informa credencial em `/login`.
2. O hub valida a credencial e cria a sessão, com `papel`, `escolaId`, `turmaId` e `serie`.
3. O *middleware* redireciona para a *home* do papel: `/estudante`, `/professor` ou `/admin/usuarios`.

Fluxo do PRIMEIRO acesso. Se a senha veio de uma importação ou de um reajuste (`senha_provisoria`), o passo 3 vira `/trocar-senha`, e não há saída: a mesma marca faz o BFF recusar dados e o `/api/exp-token` recusar *token* de experiência — só com o *middleware*, bastaria digitar `/exp/...` para pular. Trocada a senha, a sessão é **refeita** com a nova, porque o *cookie* antigo ainda diz que falta trocar (**ADR-29**).

Fluxos alternativos

- **A1 — credencial inválida:** o formulário permanece com mensagem; nenhuma sessão é criada.
- **A2 — usuário já autenticado acessa área de outro papel:** redirecionado para a *home* do próprio papel. *Não* recebe 403 — um estudante que digita `/professor` volta ao lugar dele sem mensagem de erro, o que é decisão de produto, não descuido de segurança (a autorização real é verificada de novo na API).
- **A3 — WSO2 como emissor:** o passo 2 passa a ser um *redirect* OIDC. Nada muda nos passos 1 e 3, nem na API.

4.3.2 UC-03 — Realizar experiência

Este é o caso de uso central da plataforma e o que mais determina a arquitetura.

> **UC-03 Realizar experiência**
>
> | | |
> |---|---|
> | **Ator primário** | Estudante (ou Professor, nas ferramentas de professor) |
> | **Pré-condição** | Sessão válida; experiência disponível para o papel |
> | **Pós-condição** | Progresso e telemetria registrados; conclusão notificada ao hub |

Inclui	**UC-04** (telemetria), **UC-05** (renovação de *token*)
Requisitos	**RF-12**–**RF-17**, **RF-40**

Fluxo principal

1. O estudante escolhe a missão em `/estudante` e o hub navega para `/exp/{expId}`.
2. O hub monta o *embed*: assina o *token* de experiência (RS256, 15 min), consulta o manifesto e resolve a URL versionada do *bundle*.
3. Se o manifesto declara `viewport`, o hub desenha a casca de aparelho em volta do `iframe`.
4. O *bundle* carrega, a SDK emite `exp:ready` e o hub responde `hub:init` com *token*, dados do estudante, `apiUrl` e `config`.
5. A experiência opera: chama rotas de domínio da API, registra telemetria em lote e salva progresso.
6. Ao terminar, emite `exp:concluir`; o hub registra progresso e libera a tela seguinte.

Fluxos alternativos

- **A1 — *token* perto de expirar:** a experiência emite `exp:token-expirando`; o hub busca `GET /api/exp-token` e responde `hub:token`. O `iframe` não recarrega e o estado não se perde.
- **A2 — versão de protocolo incompatível:** os dois lados verificam a versão *maior*. Sem `hub:init`, o *handshake* expira e a experiência mostra erro. Falha alta, nunca silenciosa.
- **A3 — sem manifesto (ambiente novo ou desenvolvimento):** o hub lê o `experiencia.json` servido ao lado do `index.html` do *bundle* para obter `viewport` e `permissoes`.
- **A4 — página do hub perde foco:** o hub emite `hub:pause`; a experiência congela cronômetros. Ao voltar, `hub:resume`.
- **A5 — experiência de professor acessada por `/exp/*`:** a rota responde 404. Ferramenta de professor não é missão de estudante.

> **ATENÇÃO**
>
> O passo 4 falhava de forma intermitente numa versão anterior da SDK: com os *assets* em *cache* (um F5), a experiência ficava pronta *antes* de o hub hidratar o *listener*, e o único `exp:ready` se perdia — o *handshake* expirava sem culpa de ninguém. A correção foi *re-enviar* `exp:ready` a cada 400 ms até o hub responder, e tratar `hub:init` repetido como evento normal, não como ruído.

4.3.3 UC-10 — Preparar atividade

UC-10 Preparar atividade (exp-01)

Ator primário	Professor
Pré-condição	Sessão de professor válida
Pós-condição	Atividade `ativa`, com *feed* gerado e revisado
Inclui	**UC-11** (geração com IA)
Requisitos	**RF-30**, **RF-31**, **RF-41**

Fluxo principal

1. O professor cria a atividade com tema, recorte e contexto de aula.
2. A API destila o antagonismo (dois lados) e propõe a pergunta-chave; o professor *aprova ou reescreve* — a origem fica registrada em `pergunta_origem`.
3. O professor dispara a geração. O servidor planeja os lotes (o *mix* é decisão do servidor; a IA apenas preenche) e o *worker* executa em segundo plano.
4. O professor acompanha o progresso por *polling*, revisa os posts, reprova ou regenera itens e ativa a atividade.

Fluxos alternativos

- **A1 — sem saldo de quota:** a geração é recusada *antes* da chamada ao provedor. Gastar *token* e depois recusar seria pagar pelo que não se entrega.
- **A2 — geração órfã (instância morta no meio):** o *polling* do painel detecta a fila órfã e redispara; itens `rodando` há tempo demais voltam à fila por `updatedAt`.
- **A3 — falha de geração de imagem:** o post recebe arte procedural de *fallback* e `image_status = "fallback"`. A atividade não trava.

4.3.4 UC-13 — Ler resultados da turma

UC-13 Ler resultados da turma

Ator primário	Professor
Pré-condição	Turma com pelo menos uma sessão registrada
Pós-condição	Nenhuma — é leitura
Requisitos	**RF-47**–**RF-49**, **RF-52**

Regras de negócio que governam este caso de uso — e que são a parte mais importante dele:

1. **A listagem da turma não mostra valor psicométrico.** Listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças. Os valores vivem na tela individual, uma criança por vez.
2. **O relatório individual abre pelo nível do instrumento.** "Caiu em 2 de 3" sem dizer que era o nível C faz o adulto concluir que a criança é ingênua — exatamente o rótulo que o projeto existe para não produzir.
3. **Ausência de oportunidade nunca é lida como omissão.** Zero cartões aparecidos produz "as mensagens não chegaram nesta partida", nunca "ela ignorou tudo".
4. **Valor com uma única fonte de medida não recebe número.** A API não devolve o número; o hub não pode imprimi-lo.
5. **O agregado é de gestor; a linha de uma criança, não.** São *controllers* separados, e não um `@Roles` misto com exceção por método.

4.3.5 UC-14 — Gerar plano de aula

> **UC-14 Gerar plano de aula (exp-90)**
>
> | | |
> |---|---|
> | **Ator primário** | Professor |
> | **Pré-condição** | Referências curriculares do contrato importadas |
> | **Pós-condição** | Plano `pronto` em armazenamento privado, editável |
> | **Requisitos** | **RF-35**, **RF-36**, **RF-34** |

Fluxo principal

1. O professor escolhe disciplina e série; a experiência chama `POST /planos`, que responde **202** imediatamente.
2. A API faz *claim* atômico da linha (`pendente` → `gerando`) e chama o modelo de saída longa com o material real — currículo da rede e as quatro unidades do ano, algo em torno de 190.000 *tokens* só de *prompt*.
3. O Markdown gerado é gravado imutável em `planos/{escolaId}/{id}/gerado.md`; a conversão para HTML alimenta o editor.
4. O professor edita e salva; o HTML é sanitizado por *allowlist* e gravado em `atual.html`. O versionamento de objetos do *bucket* dá histórico de cada salvamento sem uma linha de código.

Fluxos alternativos

- **A1 — saída cortada:** `finishReason = MAX_TOKENS` é *erro*, não conteúdo. O plano vai para `erro` com mensagem, em vez de entregar meio plano.
- **A2 — *restart* no meio da geração:** a linha fica `gerando` sem *worker*; após 12 minutos é considerada órfã e retomada, no máximo duas vezes.

- **A3 — restaurar o original:** o professor descarta a edição e volta ao `gerado.md`, que nunca foi sobrescrito.

4.3.6 UC-15 — Fazer *check-in* de bem-estar

UC-15 *Check-in* de bem-estar (exp-91)

Ator primário	Professor (e somente ele)
Pré-condição	Sessão de professor válida
Pós-condição	*Check-in* do dia registrado; ao sétimo, ciclo pontuado e classificado
Requisitos	**RF-37**, **RF-53**, **RF-54**

Fluxo principal

1. `GET /bemestar/hoje` abre o *check-in* do dia com três itens do DASS-21 — *verbatim*, na versão brasileira validada — e os textos de entorno gerados pela IA.
2. O professor responde os três itens (0–3) e, se quiser, escreve texto livre.
3. `POST /bemestar/checkins/:id/respostas` grava. No sétimo *check-in* a API soma por subescala ($\times 2$) e classifica pelas faixas publicadas.

Fluxos alternativos

- **A1 — texto de IA barrado, sem saldo ou sem *provider*:** entra o texto fixo do catálogo. O *check-in* nunca deixa de abrir.
- **A2 — sinal de apoio** (item 17 ou 21 ≥ 2, ou termo de risco no texto livre): mensagem *fixa* de apoio com CVV 188, SAMU 192 e CAPS/UBS, e a IA *não é chamada*.
- **A3 — ciclo expirado:** 21 dias sem fechar encerra o ciclo como `expirado`, sem pontuação.
- **A4 — apagar tudo:** `DELETE /bemestar/dados` remove ciclos e *check-ins* do professor.

REGRA INVIOLÁVEL

O instrumento é **fixo e *verbatim***. A IA nunca reescreve um item do DASS-21: reformular quebra a validação psicométrica e invalida as faixas de corte. O que a IA gera é o entorno — abertura do dia, pergunta aberta, acolhimento — e cada texto passa por validação antes de chegar ao professor. A IA nunca recebe nome, escola, pontuação ou faixa.

4.3.7 UC-23 — Publicar manifesto de experiência

> **UC-23 Publicar manifesto**
>
> | | |
> |---|---|
> | **Ator primário** | CI/CD (em nome de `admin-contrato`) |
> | **Pré-condição** | *Bundle* publicado em `gs://{contrato}-exp/{expId}/{versao}/` |
> | **Pós-condição** | Linha da tabela `experiencias` atualizada; hub passa a servir a nova versão |
> | **Estendido por** | UC-24 (reverter versão) |
> | **Requisitos** | RF-10, RF-11 |

Fluxo principal

1. O CI sincroniza `dist/` para o caminho versionado do *bucket* e marca o `index.html` com `Cache-Control: max-age=60`.
2. O CI assina um *token* de cinco minutos com a chave de assinatura e chama `PUT /experiencias/{expId}` com o conteúdo do `public/experiencia.json`.
3. A API valida o *token* contra o JWKS que o *hub* publica e faz o *upsert*.

> **NOTA**
>
> A ordem importa e está codificada no *workflow*: as experiências são publicadas *depois* do hub, porque o *token* do passo 2 é validado contra o JWKS servido pelo hub. Hub fora do ar significa manifesto rejeitado — e um manifesto rejeitado é preferível a um manifesto aceito apontando para um hub que não subiu.

UC-24 — **Reverter versão:** o mesmo *script* com `--versao <anterior>`. Não há *build*, não há imagem, não há *deploy*: é um `UPDATE` de `versao_ativa`, e o próximo estudante que abrir a experiência recebe o *bundle* anterior. É a operação de emergência mais rápida da plataforma, e existe por desenho (ADR-01).

4.4 Rastreabilidade de casos de uso

UC	Requisitos	Superfície	Módulo da API
UC-01	RF-01, RF-02, RF-05	`/login`	`auth`
UC-02	RF-10	`/estudante`	`experiencias`
UC-03	RF-12–RF-17	`/exp/:expId`	`motor`, `unootube`, `vila`, `progress`
UC-04	RF-20–RF-25	SDK	`telemetry`
UC-05	RF-14	`/api/exp-token`	`auth`
UC-06	RF-40	dossiê da experiência	`dossie`

UC-10	**RF-41**	`/professor/atividades-ti`	`atividades`, `geracao`
UC-11	**RF-30–RF-33**	—	`ai`, `geracao`
UC-12	**RF-41**	painel de etapas	`atividades`, `motor`
UC-13	**RF-47–RF-52**	`/professor/entre-re`	`analise`, `vila`, `unootube`
UC-14	**RF-34–RF-36**	`/professor/plano-de-aula`	`planos`
UC-15	**RF-37**, **RF-53**	`/professor/bem-esta`	`bemestar`
UC-16	**RF-50**	botão de zerar turma	`vila`
UC-20	**RF-06**	`/admin/importar`, `/admin/turmas`	`rostering`
UC-21	**RF-07**	`/admin/usuarios`	`usuarios`
UC-25	**RF-01**	`/trocar-senha`	`auth`
UC-26	**RF-06**	`/professor/turmas`	`rostering`
UC-27	**RF-06**	`/admin/escolas` (lotação em lote), `/admin/minhas-escol`	`rostering`
UC-28	**RF-07**	`/admin/turmas/:id` (perfil portátil)	`portabilidade`
UC-22	**RF-52**	`/analise`	`analise`
UC-23	**RF-10**	CI/CD	`experiencias`
UC-24	**RF-11**	*script*	`experiencias`

5

Arquitetura de *software*

Este capítulo descreve a organização interna do *software*: que peças existem, o que cada uma faz, como conversam e por que a fronteira está onde está. A arquitetura de *nuvem* — onde essas peças rodam — é o Capítulo 6.

5.1 Visões arquiteturais

O sistema é descrito por cinco visões complementares, no espírito do modelo 4+1 (KRUCHTEN, 1995). Nenhuma delas é suficiente isoladamente.

Visão	Notação	Onde está
Lógica	C4 níveis 1–3; diagrama de classes	Seções 5.3 a 5.6
Processos	Diagramas de sequência e de estado	Seção 5.8; Cap. 9
Desenvolvimento	Estrutura do *monorepo* e grafo de dependências	Seção 5.7
Física / implantação	Diagramas de implantação e de rede	Cap. 6
Cenários	Casos de uso	Cap. 4

5.2 Estilo arquitetural e princípios

A plataforma é, deliberadamente, um **monólito modular** de *backend* (uma API NestJS com módulos de domínio bem separados) servindo uma **constelação de artefatos estáticos** (as experiências). Não é microsserviços, e não é por acaso.

DECISÃO DE PROJETO

Um serviço de *backend*, muitos artefatos de *frontend* (**ADR-01**, **ADR-02**). Microsserviços resolvem escala organizacional — times independentes que precisam implantar sem coordenação. Aqui há uma pessoa. O que precisa escalar independentemente é a *entrega* das experiências, e isso foi resolvido tirando-as do plano de execução: *bundles* estáticos numa CDN escalam sem que exista um serviço. A modularidade que os microsserviços dariam por processo é obtida por *módulo NestJS* e por convenção de prefixo de tabela — com a vantagem de um `JOIN` continuar sendo um `JOIN`, e não uma chamada de rede.

Os princípios que atravessam todo o código:

Validação na fronteira, sempre.
Todo *input* externo passa por Zod (MCDONNELL, 2026): corpo, *query*, *postMessage* recebido e as variáveis de ambiente — estas no *boot*, de modo que configuração errada derruba o processo na subida, não no meio de uma requisição de estudante.

Uma porta por recurso crítico.
O banco só é acessado por `PrismaService.comTenancy`; a IA só por `src/ai`; o *design system* só pela ponte `packages/ui`; a entrada de pessoas e turmas só por `src/rostering` — que é também o único lugar da API que conhece o OneRoster, do mesmo jeito que só o *provider* conhece o fornecedor de IA (**ADR-28**). Recurso com duas portas tem duas chances de esquecer a tranca.

Regra de domínio pura, casca fina.
Cada módulo de experiência separa `catalogo.ts` (dados), `regras.ts` (funções puras, testáveis sem banco) e um *service* que persiste. O *controller* é só a casca HTTP.

Falhar alto, ou falhar fechado.
Incompatibilidade de protocolo falha alto e visível; rota cuja proteção depende de configuração ausente responde 401, nunca fica aberta.

O tipo carrega a regra.
Quando uma regra pode ser expressa no sistema de tipos, ela é — porque comentário não é controle (Seção 5.6).

5.3 C4 nível 1 — Contexto do sistema

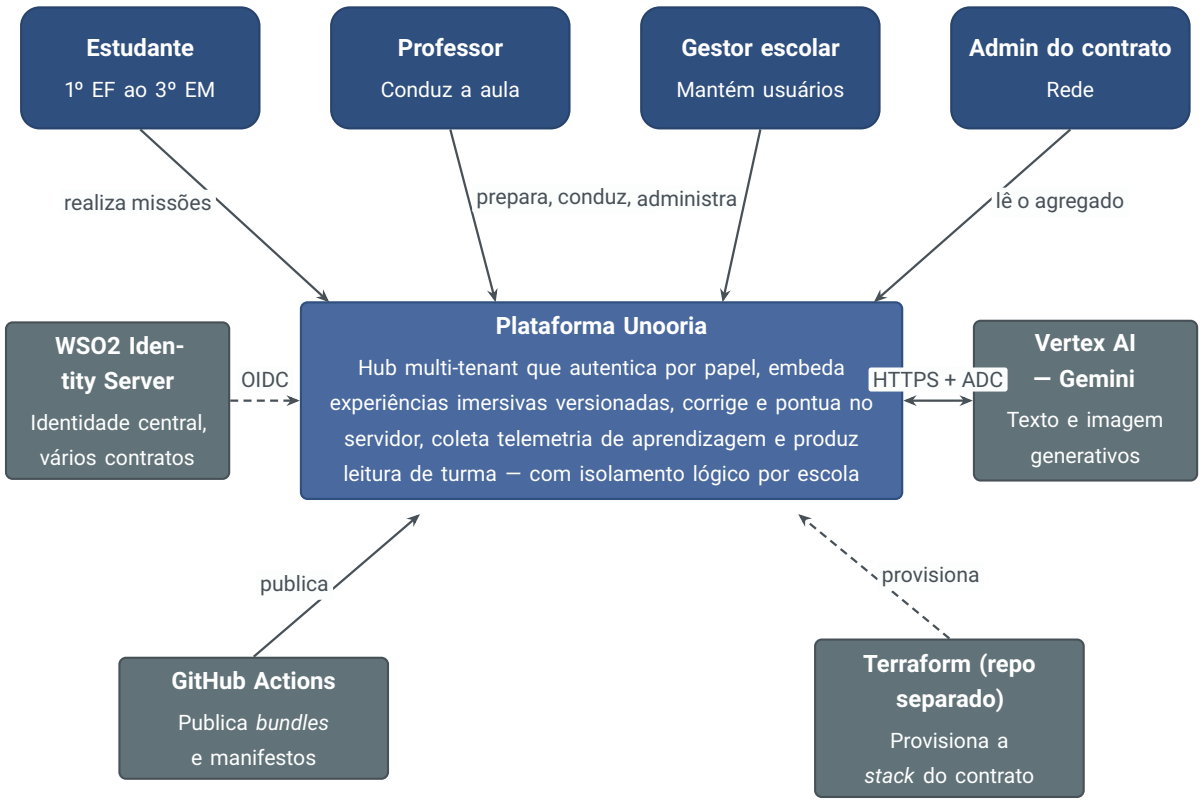


Figura 5.1 – C4 nível 1: contexto. Quatro papéis humanos mutuamente exclusivos, um provedor de identidade compartilhado entre contratos, um único serviço externo na dependência de IA, e dois atores de automação. O Terraform aparece porque a fronteira entre os dois repositórios é um contrato explícito, não um detalhe de operação.

> **NOTA**
>
> A dependência externa *não substituível em tempo de execução* é o Vertex AI, e mesmo assim sua indisponibilidade não derruba a plataforma: as experiências que não usam IA (exp-02, exp-70) seguem íntegras, o *feed* da exp-01 continua sendo curado a partir de conteúdo já gerado, e as ferramentas de professor degradam para texto fixo (exp-91) ou recusam a geração com mensagem clara (exp-90). Essa contenção de acoplamento é deliberada (**RNF-12**).

5.4 C4 nível 2 — Contêineres

O nível de contêineres é onde a assimetria da arquitetura aparece. Há *quatro* contêineres de execução e um número crescente de *artefatos* que não são contêineres — e tratá-los como se fossem seria o erro que este desenho existe para evitar.

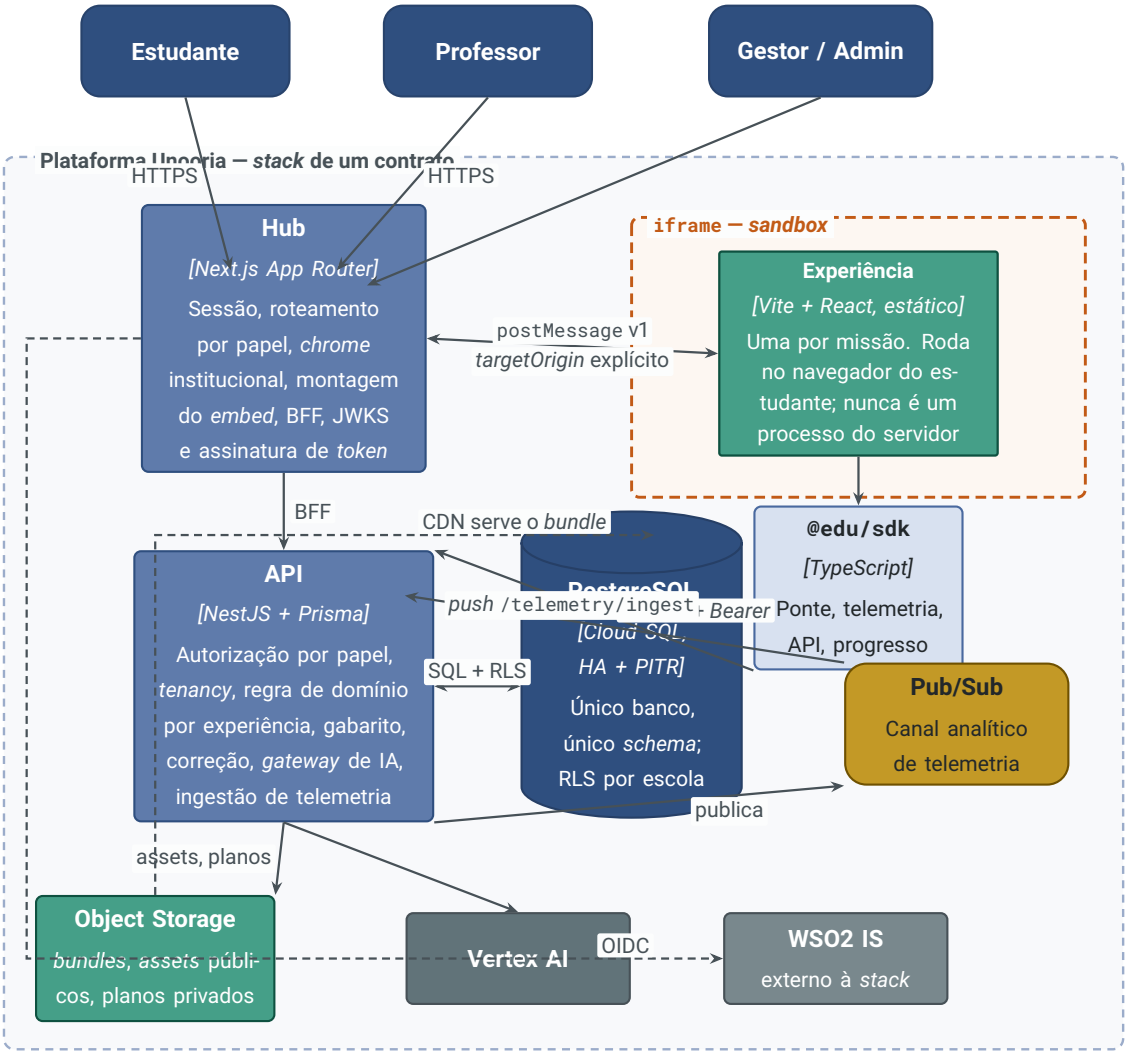


Figura 5.2 – C4 nível 2: contêineres. Só o hub, a API, o banco e o Pub/Sub são peças em execução. A experiência é um *artefato* servido pela CDN e executado no navegador; a SDK é biblioteca dentro dele. A linha tracejada laranja é a fronteira do *sandbox* do `iframe` — a fronteira de segurança mais importante do desenho de *frontend*.

Legenda: contêiner em execução · artefato estático · biblioteca · banco · fila · armazenamento de objetos · externo

5.4.1 Catálogo de contêineres e artefatos

Peça	Tipo	Tecnologia	Responsabilidade e limite
`apps/hub`	Contêiner	Next.js 16, App Router	Autenticar, roteirizar por papel, desenhar *todo* o *chrome* institucional, montar o *embed*, assinar *token* de experiência e publicar JWKS. **Não** contém regra de domínio de experiência.
`apps/api`	Contêiner	NestJS, Prisma	Autorizar, garantir *tenancy*, aplicar regra de domínio, guardar gabarito, corrigir, intermediar IA, ingerir telemetria. É o único que fala com o banco.

Peça	Tipo	Tecnologia	Responsabilidade e limite
`apps/experiencias/*`	Artefato	Vite, React	Interface e interação de uma missão. **Nunca** guarda gabarito, nem calcula nota que valha, nem persiste *token*.
`packages/sdk`	Biblioteca	TypeScript puro, Zod	Contrato `postMessage`, cliente HTTP autenticado, telemetria em lote, progresso. É a *única* forma legítima de uma experiência falar com a plataforma.
`packages/ui`	Biblioteca	TypeScript, React	Ponte para o *design system*: marca `'use client'`, desfaz colisão de nomes entre seções e preserva *tree-shaking*.
`packages/conteu`	Biblioteca	TypeScript puro	A copy da revelação da exp-01, em UMA fonte. A experiência a exibe e o guia da aula a mostra ao professor — que antes conduzia a conversa mais importante das duas aulas sem ter lido o que a turma acabara de ver. Não passa pela api de propósito: a revelação abre no meio da aula, e texto que depende de *fetch* é texto que pode não aparecer.
PostgreSQL	Contêiner gerenciado	Cloud SQL	Único banco, único *schema* lógico. Guarda tudo, com RLS forçada.
Pub/Sub	Contêiner gerenciado	Pub/Sub	Amortecedor *durável* do pico de telemetria. Entrega *at least once* por *push* autenticado.
Object Storage	Artefato	Cloud Storage	Três *buckets* com regimes diferentes: *bundles* (público, imutável), *assets* (público, CDN), planos (**privado**, versionado).

REGRA INVIOLÁVEL

Um banco, um *schema*. Nem banco por experiência — elas não são isoladas (o perfil da Vila alimenta a largada do Instanooria, e o PostgreSQL não faz `JOIN` entre bancos; além disso, um *pool* do Prisma por experiência estouraria `max_connections`) — nem *schema* por experiência: a única coisa que um *schema* daria além do prefixo é fronteira de `GRANT`, e existe *uma role* de aplicação. A separação é por **prefixo de tabela**: `exp_NN_` para tabela de experiência, sem prefixo para tabela de plataforma (Capítulo 8).

5.5 C4 nível 3 — Componentes

5.5.1 Componentes da API

A API é organizada em módulos NestJS. A Figura 5.3 separa-os em quatro faixas, e a faixa a que um módulo pertence determina o que ele pode conhecer. Desde 2026-09-02 a faixa trans-

versal tem um quinto componente que a figura não desenha, `observabilidade/` (Capítulo 17): o *middleware* que abre o contexto da requisição antes dos *guards* (o `TenancyInterceptor` passou a *preencher* a escola nesse mesmo objeto, em vez de abri-lo), o *logger* pino com *redaction*, o `FiltroDeExcecao` global, o *sampler* por rota e o `otel.ts` — que é a **primeira importação** do `main.ts`, porque a instrumentação automática do OpenTelemetry precisa alterar `http`, `express`, `undici` e o Prisma antes de o Nest os carregar.

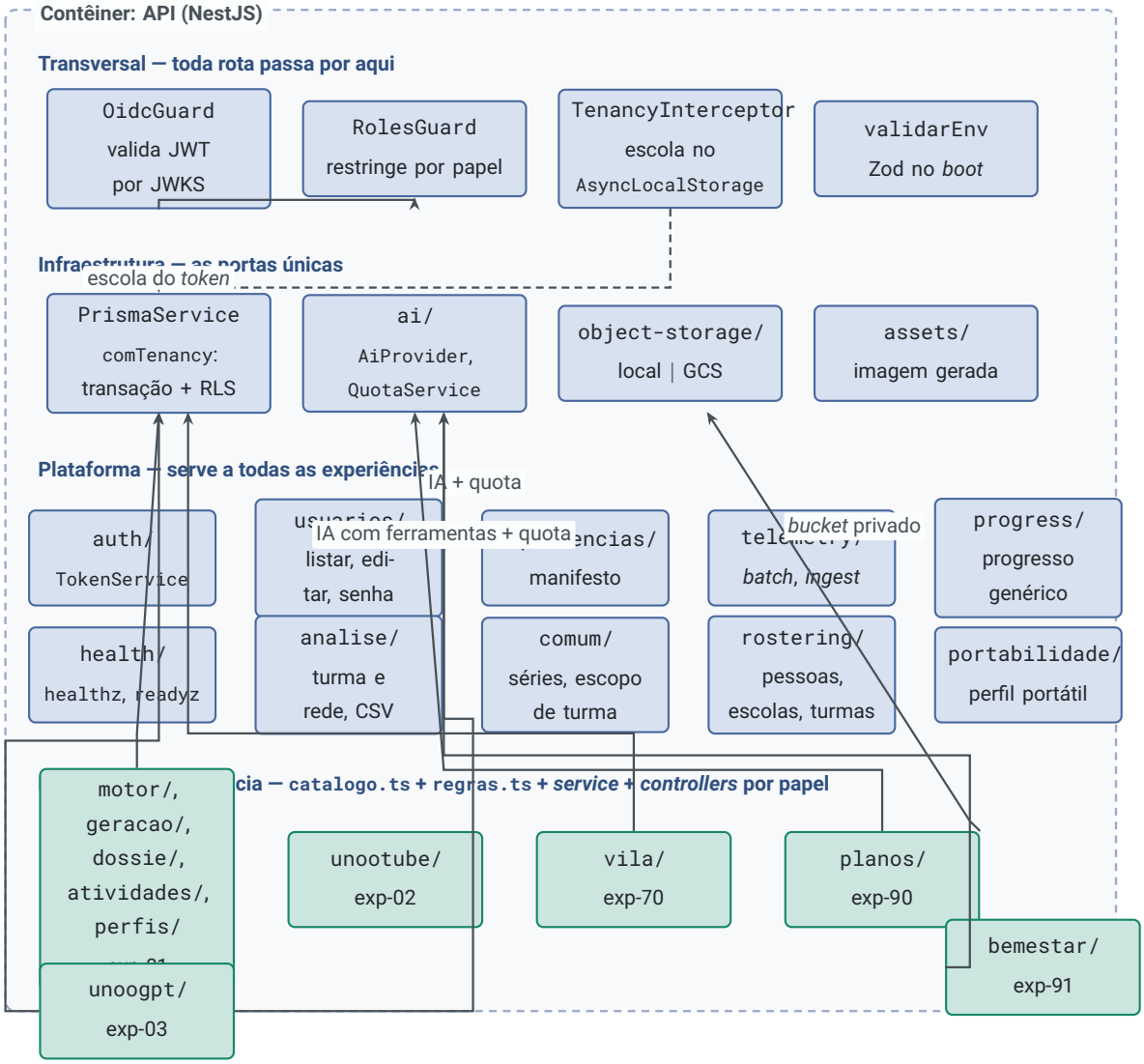


Figura 5.3 – C4 nível 3: componentes da API. A faixa transversal roda em toda rota; a faixa de infraestrutura contém as portas únicas (banco, IA, armazenamento); a faixa de plataforma serve a todas as experiências; a faixa de domínio tem um módulo por experiência, e nenhum módulo dessa faixa conhece outro.

REGRA INVIOLÁVEL

> **NOTA**
>
> `unoogpt/` é o módulo de domínio mais fundo da API: 3.285 linhas de motor probabilístico, agente e regras puras, com subpastas (`modelo/`, `regras/`, `agente/`) em vez do par `catalogo.ts` + `regras.ts`. A granularidade veio do projeto de origem e foi **mantida de propósito**: cada arquivo tem teste próprio, e colar arquivos bem fatorados produziria colisão de nomes e falha mais difícil de localizar. O padrão do Capítulo 12 continua valendo para experiência nova; este é um porte.

Módulo de domínio não conhece módulo de domínio. `vila/` não importa `unootube/`. Quando duas experiências precisam se comunicar — o perfil da Vila alimentando a largada do Instanooria — a comunicação é por *tabela de contrato* explícita (`exp_01_exp_70_perfis_importados`, que carrega os dois números no nome) e por um *endpoint* de fronteira (`POST /atividades/:id/perfis/da-vila`). Chamar a tabela por um número só mentiria sobre quem depende dela.

5.5.2 Componentes do hub

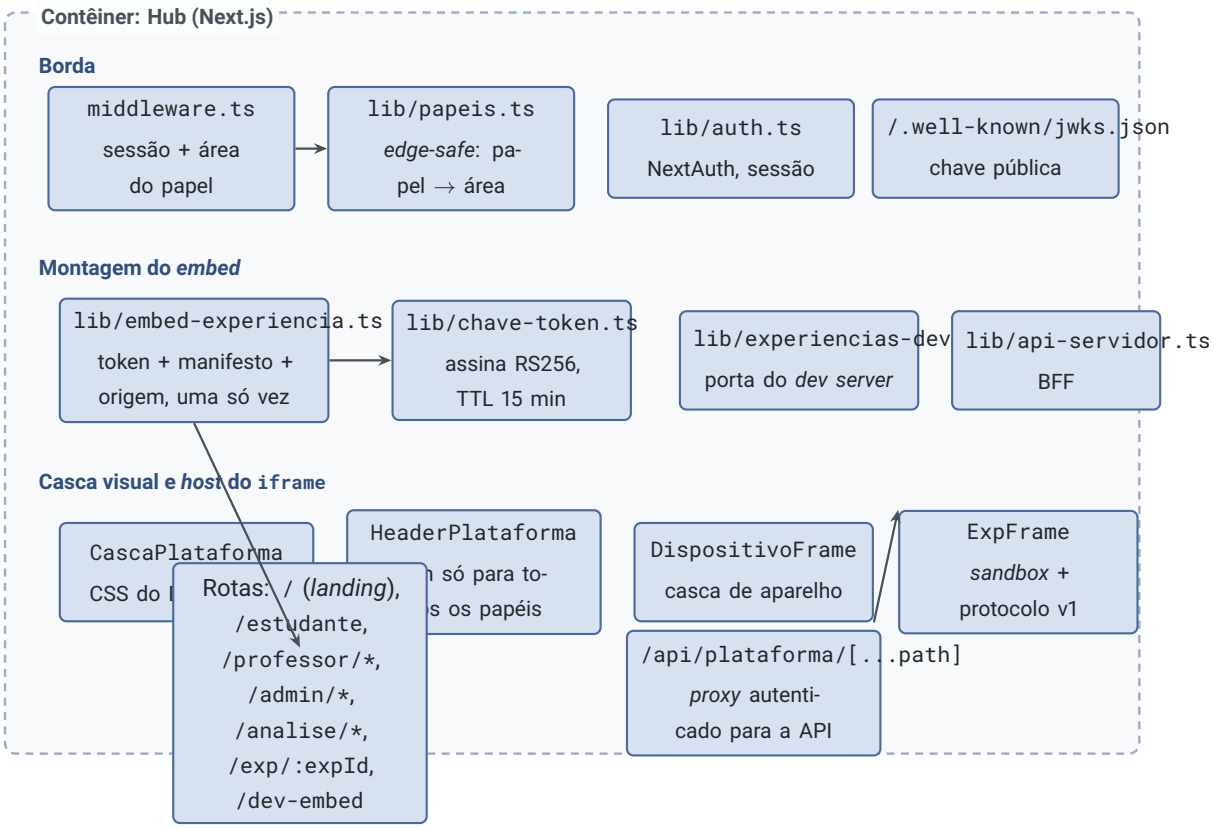


Figura 5.4 – C4 nível 3: componentes do hub. A montagem do *embed* é *uma só função* usada pelas quatro superfícies que embedam (`/exp/[expId]`, `/dev-embed`, `/professor/plano-de-aula` e `/professor/bem-estar`) — duplicá-la era o caminho mais curto para uma delas passar a assinar o *token* com *claims* diferentes.

DECISÃO DE PROJETO

Um único *header* para todos os papéis, e uma única casca visual. Antes, cada área copiava o próprio *header* e a experiência não tinha nenhum. Hoje `HeaderPlataforma` serve `/estudante`, `/professor`, `/admin`, `/analise` e `/exp/*`, e a casca visual saiu do *layout* do grupo de rotas para `CascaPlataforma`, porque `/exp/*` e `/dev-embed` estão fora do grupo e também precisam dela. A regra de ouro passou a valer um nível abaixo: o *header* é *chrome* do hub, desenhado *acima* do aparelho — dentro do `iframe` nada da marca institucional entra.

NOTA

A raiz é a *landing* pública, e continua roteando quem tem sessão (**ADR-40**). `/` sem sessão renderiza `components/landing/` — porte da tela desenhada no Stitch, com a estrutura, a copy e o movimento do modelo (navegação fixa em vidro, *shader* WebGL de fundo, revelação ao rolar) e a cor, a tipografia, os ícones, o logo e os botões do DS. Com sessão, `/` redireciona para a *home* do papel (ou para `/trocar-senha`): o `login-form` e a troca de senha fazem `router.replace("/")` e dependem desse roteamento para cair na área certa sem o *client* conhecer o papel. A *landing* está fora do `matcher` do *middleware*, não lê nada da sessão e é, com `/login` e o JWKS, a única superfície pública do hub. Os dois pedaços com ciclo de vida (`FundoAnimado`, `RevelarAoRolar`) são *client components*; o resto é servidor.

5.6 C4 nível 4 — Código

O nível 4 do C4 raramente vale a pena desenhar por inteiro. Vale aqui para *três* lugares, porque neles o desenho de tipos *é* a arquitetura.

5.6.1 O contrato da ponte e o *guard* de *tenancy*

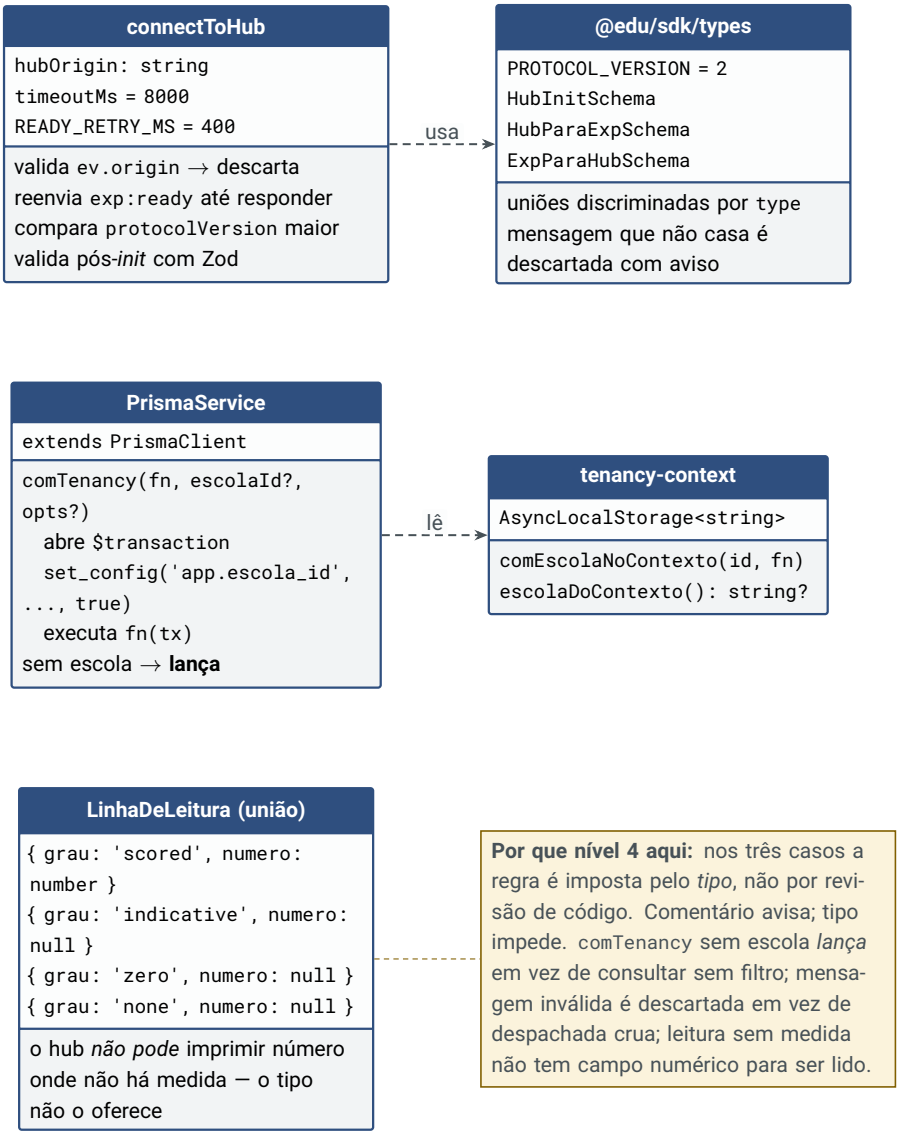


Figura 5.5 – C4 nível 4: os três pontos em que a regra arquitetural está no sistema de tipos. `comTenancy` é a única porta do banco e falha alto sem escola; o contrato da ponte descarta o que não valida; a leitura de valores não oferece número onde não há medida.

5.6.2 O gabarito de um módulo de domínio

Todo módulo de experiência segue a mesma forma. Não é convenção estética: é o que permite testar a regra sem banco e trocar a persistência sem tocar a regra.

Listing 5.1 – Forma canônica de um módulo de domínio de experiência

```
apps/api/src/<slug>/
  catalogo.ts     // DADOS: itens, textos, gabarito. Constantes puras.
  regras.ts       // REGRA: funcoes puras. Sem Prisma, sem fetch, sem Date.
  <slug>.service.ts   // PERSISTENCIA: le com lock, aplica a regra, grava.
                      // Toda query dentro de comTenancy e filtrando escola_id.
  <slug>.controller.ts        // casca HTTP do papel "estudante"
  <slug>-professor.controller.ts  // casca HTTP do papel "professor"
```

```
  <slug>.module.ts

// A separacao de controllers POR PAPEL e deliberada: um @Roles misto com
// excecao por metodo e o caminho mais curto para um GET de professor herdar
// silenciosamente a permissao de um POST de estudante.
```

NOTA

A consequência prática dessa forma aparece nos testes: `vila-regras.spec.ts`, `unootube-regras.spec.ts` e `bemestar-regras.spec.ts` rodam *milissegundos* sem PostgreSQL, porque exercitam `regras.ts` puro. Os testes que precisam de banco (`*-postgres.spec.ts`, `tenancy.spec.ts`) são separados e cobrem exatamente o que não dá para afirmar sem ele: *tenancy*, *lock* e concorrência.

5.7 Estrutura do *monorepo*

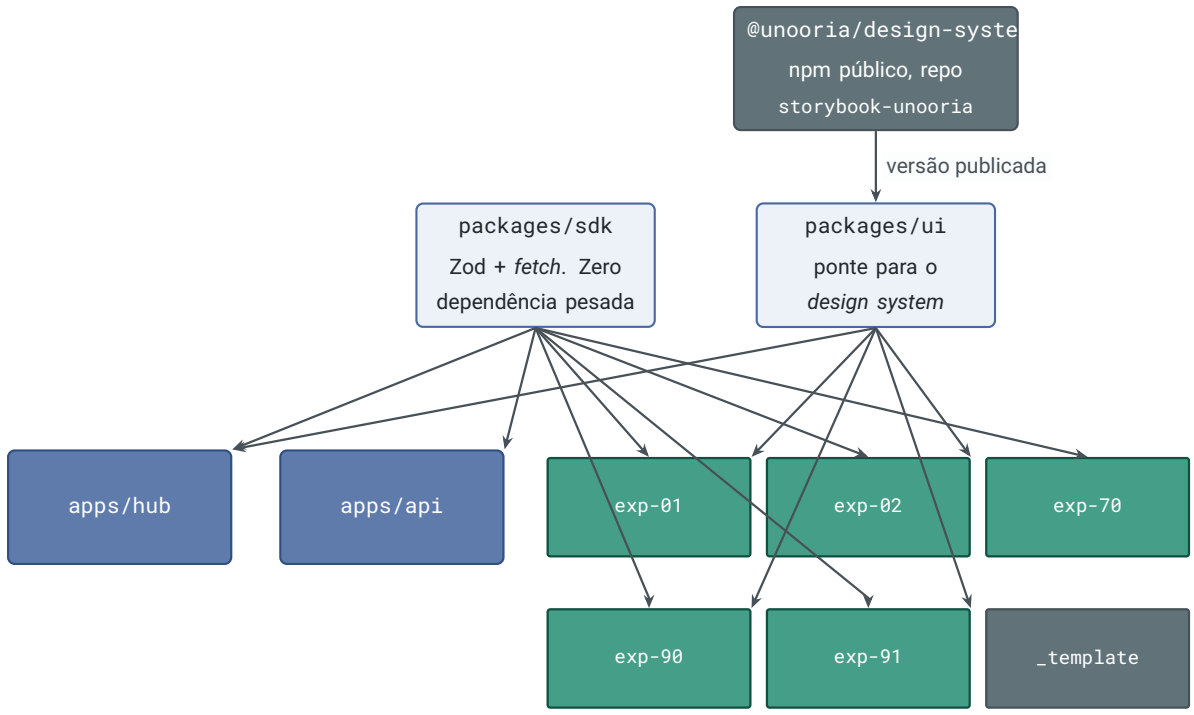


Figura 5.6 – Grafo de dependências do *monorepo*. A SDK é consumida por tudo — é por isso que uma mudança nela precisa testar tudo no mesmo *pull request*, e por isso o *monorepo* (**ADR-02**). A exp-70 não depende de `packages/ui`: é jogo em *canvas* 3D, sem DOM de interface, e é a exceção registrada em ADR à regra do *design system*.

Diretório	Conteúdo
`apps/hub`	Aplicação Next.js: rotas por papel, *embed*, BFF, JWKS.
`apps/api`	Aplicação NestJS: 22 módulos, Prisma, 9 migrações.
`apps/experiencias/*`	Uma pasta por experiência, mais `_template`.

`packages/sdk`	Contrato da ponte e telemetria.
`packages/ui`	Ponte do *design system* e CSS da plataforma.
`docs/`	Contratos com infraestrutura e registro de decisões.
`scripts/`	*Scaffolding*, chave de assinatura, importação de referências, a suíte de ponta a ponta (`scripts/e2e/`), *probe* de *pointer lock*.
`wso2/`	*Compose* próprio do provedor de identidade, com ciclo de vida separado.
`documentacao-latex/`	Este documento.

A orquestração é Turborepo (Vercel, 2026b), com *pipeline* declarado em `turbo.json`: `build` depende do `build` das dependências; `typecheck` e `test` também. É isso que permite ao CI rodar apenas o que mudou:

Listing 5.2 – O comando que o CI executa

```
pnpm turbo typecheck lint test build --filter='...[HEAD^]'
```

5.8 Fluxos principais

5.8.1 Curadoria de um bloco de *feed* (exp-01)

Este é o fluxo com mais concorrência da plataforma e o que mais exige do banco.

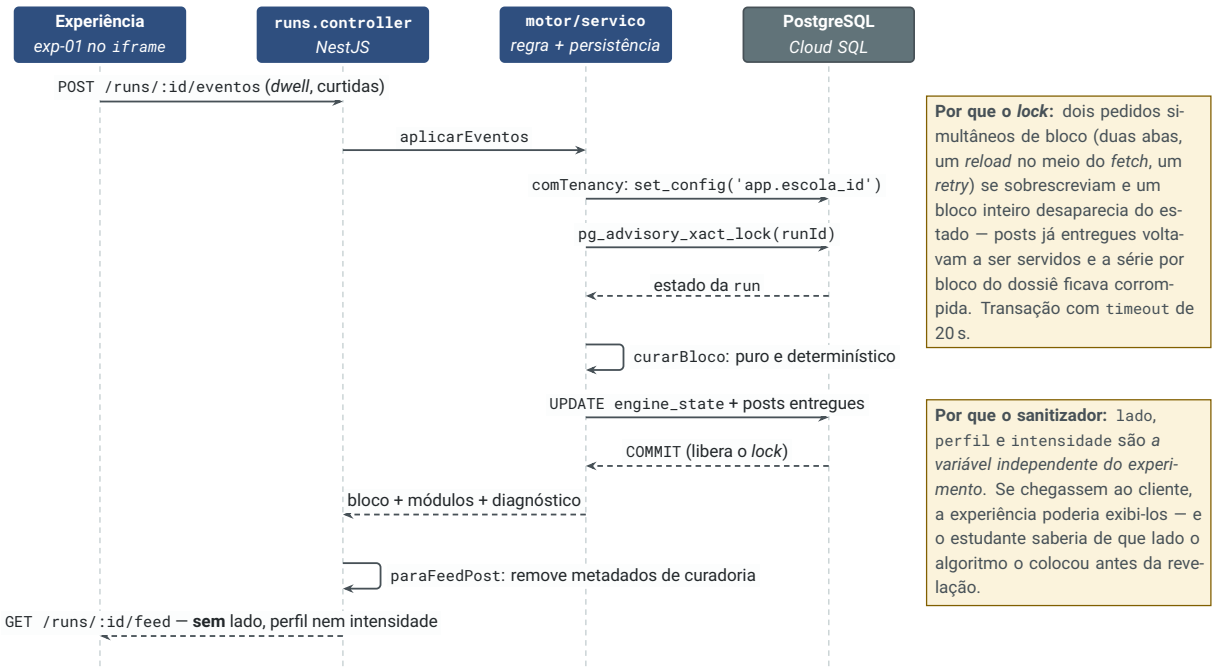
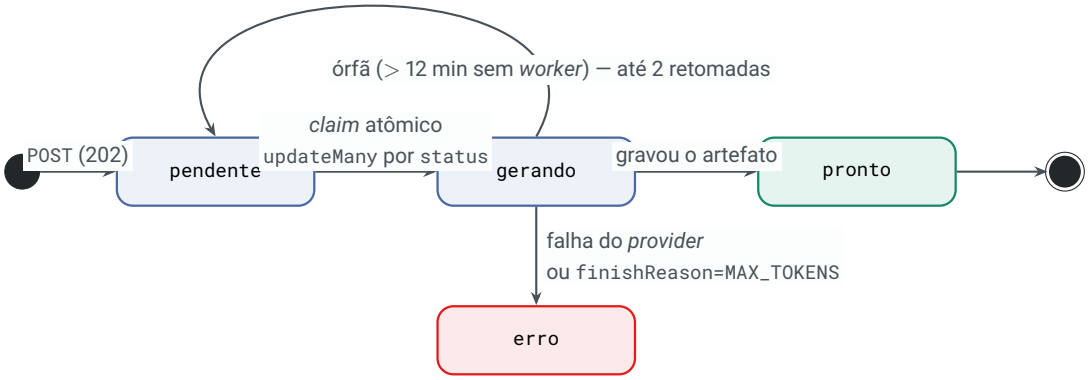


Figura 5.7 – Sequência da curadoria de um bloco de *feed*. A regra (`curarBloco`) é pura; o *service* lê com *lock*, aplica e grava; o *controller* sanitiza. As três responsabilidades ficam em arquivos diferentes de propósito.

5.8.2 Estados de uma geração assíncrona

Duas rotinas da plataforma são assíncronas e longas: a geração de *feed* da exp-01 e a geração de plano de aula da exp-90. Ambas seguem a mesma máquina de estados, com *claim* atômico em vez de fila externa.



O *claim* atômico (`UPDATE ... WHERE status = 'pendente'` com contagem de linhas afetadas) substitui uma fila externa: duas instâncias do Cloud Run podem tentar, e só uma ganha a linha. É a menor peça possível que resolve o problema — e não exige operar um serviço a mais (**ADR-17**).

Figura 5.8 – Máquina de estados das gerações assíncronas (exp-01 e exp-90). O *claim* atômico dispensa fila externa; a retomada de órfã é disparada pelo *polling* do próprio painel, não por um *scheduler*.

LIMITAÇÃO CONHECIDA

Não há retomada no *boot*. Se todas as instâncias morrerem e ninguém abrir o painel, uma geração órfã fica parada até que alguém abra. É aceito porque o *polling* do painel é justamente o momento em que alguém se importa com aquela geração — e porque a alternativa é um *scheduler*, que é uma peça nova para operar. O gatilho de reavaliação é a primeira reclamação real de geração parada.

6

Arquitetura de nuvem e implantação

Este capítulo descreve *onde* o *software* do Capítulo 5 roda: quais recursos gerenciados existem, como o tráfego atravessa a plataforma, como a identidade de carga de trabalho substitui chaves e onde estão as fronteiras de rede.

A infraestrutura é provisionada por Terraform (BRIKMAN, 2022) num **repositório separado**. Este documento descreve o *contrato* entre os dois repositórios — nomes de recursos esperados, variáveis consumidas, artefatos publicados — e não o código Terraform.

> **REGRA INVIOLÁVEL**
>
> **Este repositório nunca cria recurso GCP imperativamente.** Nenhum *script* de `scripts/` chama `gcloud ... create`. O CI apenas *usa* recursos que já existem: envia imagem para um Artifact Registry existente, executa um Cloud Run Job existente, sincroniza um *bucket* existente. Recurso criado por *script* é recurso que o Terraform não sabe que existe — e que ninguém destrói quando o contrato termina.

6.1 Princípios de nuvem

Uma *stack* por contrato.
Todos os nomes de recurso derivam de `{contrato} = {app_name}-{env}`. No piloto, `plataforma-unooria-dev`. Replicar um contrato é instanciar o *module* Terraform com outro prefixo.

Serviço gerenciado antes de orquestração.
Cloud Run em vez de GKE; Cloud SQL em vez de PostgreSQL autogerido; Pub/Sub em vez de *broker* próprio. O critério não é moda: é quantas peças uma pessoa consegue operar de plantão às oito da manhã.

Sem chave de longa duração.
O CI autentica por Workload Identity Federation; a API autentica no Vertex por ADC. Nenhum arquivo JSON de conta de serviço existe em nenhum lugar do fluxo.

Reversão por ponteiro.
Três níveis: revisão do Cloud Run (hub e API), `versaoAtiva` no manifesto (experiências) e versionamento de objetos (planos de aula). Nenhum deles exige *build*.

Estático na borda, dinâmico no mínimo.
O que pode ser servido pela CDN é servido pela CDN. O que sobra para o Cloud Run é regra de negócio.

6.2 C4 nível 1 de implantação — o contrato

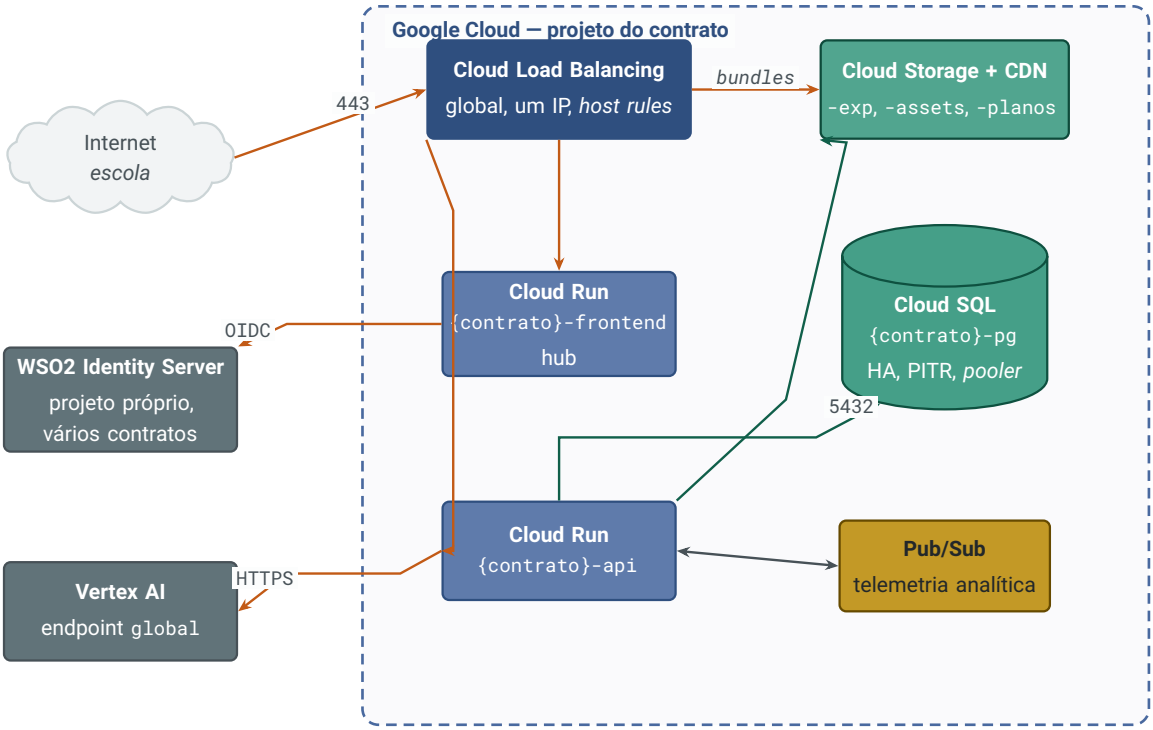


Figura 6.1 – Implantação de um contrato. Um único *Load Balancer* global recebe todo o tráfego HTTPS e distribui por *host rules*: hub, API, *bundles* e *assets* respondem em domínios diferentes com o *mesmo* IP. O Cloud SQL só tem IP privado — daí a existência do Cloud Run Job de migração dentro da VPC.

Legenda: tráfego público (TLS) tráfego privado (VPC) serviço de borda serviço de aplicação banco mensageria fora da *stack*

6.3 Diagrama de implantação detalhado

A Figura 6.2 abre o mesmo desenho no nível em que ele é operado: com os *buckets* separados por regime de acesso, os segredos, o registro de imagens e os dois caminhos de identidade de máquina.

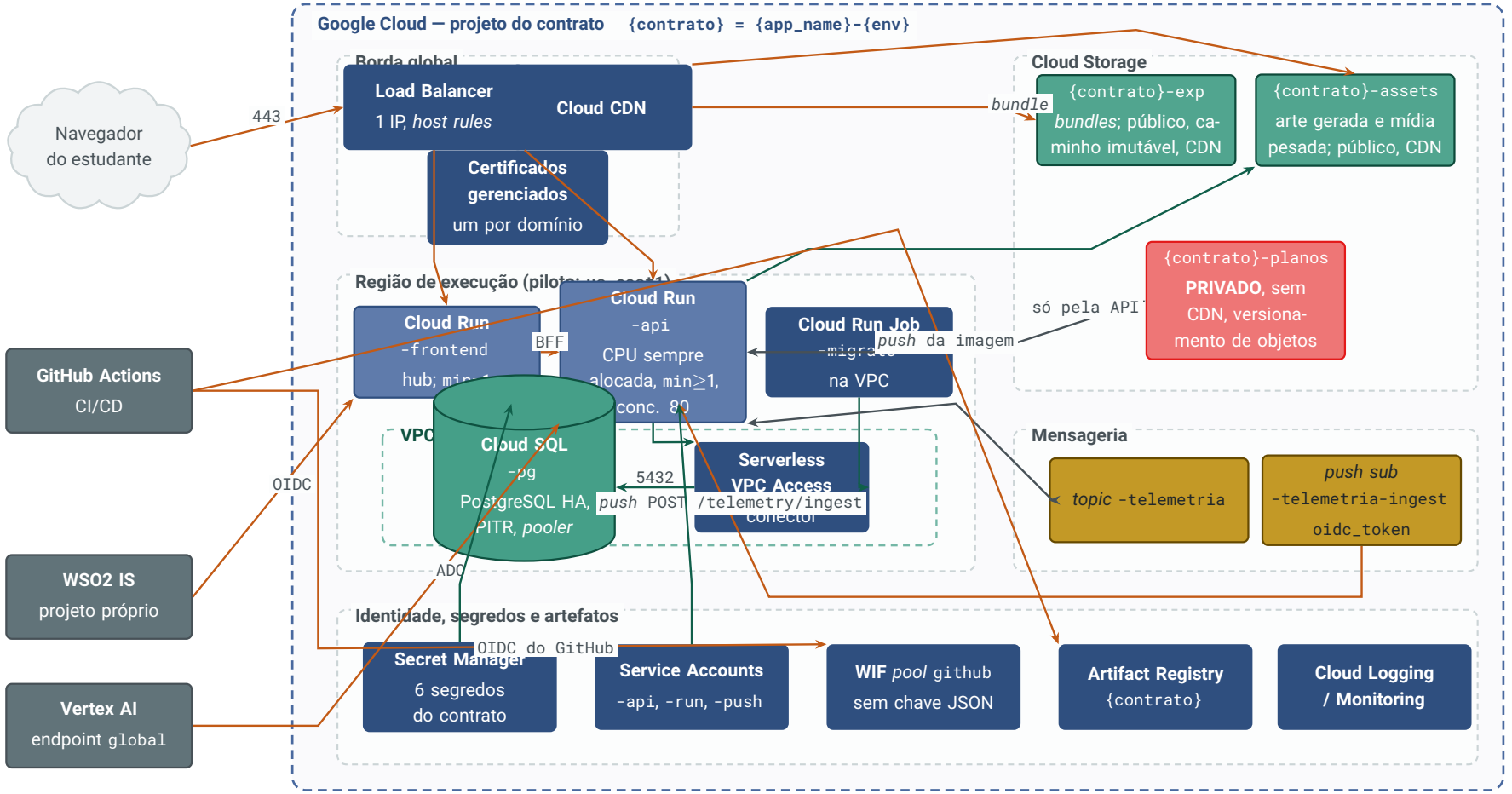


Figura 6.2 – Implantação detalhada. Os três *buckets* têm regimes de acesso deliberadamente diferentes: o de *bundles* e o de *assets* são públicos atrás da CDN (caminho versionado, portanto imutável); o de planos de aula é **privado**, sem CDN, com versionamento de objetos, e só é lido pela API depois de verificar papel e posse. O Cloud SQL não tem IP público: é isso que obriga a migração a rodar como *Job* dentro da VPC.

6.4 Catálogo de recursos por contrato

Este é o contrato formal com o repositório de Terraform. Nome divergente significa CI quebrado.

Recurso	Nome esperado	Consumido por
Cloud Run (serviço)	`{contrato}-frontend` (hub), `{contrato}-api`	`deploy.yml`
Cloud Run (*job*)	`{contrato}-migrate` — na VPC, roda `prisma migrate deploy`	`deploy.yml`
Artifact Registry	`{contrato}` (mesma região do Cloud Run)	`deploy.yml`
Bucket de experiências	`{contrato}-exp` + Cloud CDN	`deploy.yml`; hub (URL do `iframe`)
Bucket de *assets*	`{contrato}-assets` + Cloud CDN, leitura pública	API (`ASSETS_BUCKET`); mídia pesada das experiências
Bucket de planos	`{contrato}-planos` — **privado**, sem CDN, versionamento de objetos ligado	API (`PLANOS_BUCKET`)
Cloud SQL	`{contrato}-pg` — HA, PITR, *pooler* ativo	API (`DATABASE_URL`)
Pub/Sub	*topic* `{contrato}-telemetria` + *push sub* `{contrato}-telemetria-in` → `POST {api}/telemetry/ingest`	API
Secret Manager	`{contrato}-database-url`, `{contrato}-db-password`, `{contrato}-token-signing-key`, `{contrato}-auth-secret`, `{contrato}-perfil-import-token`, `{contrato}-planos-import-token`	*env* do Cloud Run
Service Accounts	`{contrato}-api` (Vertex, *publisher* do Pub/Sub, `objectAdmin` nos *buckets* de *assets* e de planos), `{contrato}-run` (hub), `{contrato}-push` (assina o *push* da telemetria)	Cloud Run, Pub/Sub

Recurso	Nome esperado	Consumido por
Load Balancer	Um só, global: hub em `{hub_domain}`, API em `{api_domain}`, *bundles* em `{exp_domain}`, *assets* em `{assets_domain}` — mesmo IP, *host rules*	DNS
WIF para GitHub Actions	*pool* `github`	`deploy.yml` (sem chave JSON)

DECISÃO DE PROJETO

Uma *role* de banco, não duas. O desenho original previa duas *roles*: uma dona do *schema* para as migrações e uma `app_runtime` sem *ownership* para a aplicação. Na prática ficou **uma**, porque as tabelas usam `FORCE ROW LEVEL SECURITY` — que sujeita o próprio dono às políticas e torna a separação desnecessária. O que *continua* obrigatório é a *role* não ter `BYPASSRLS` nem `SUPERUSER`; há SQL de verificação no *runbook* e um teste local (`tests/tenancy.spec.ts`) que cobre o mesmo contrato com uma *role* sem *ownership*.

6.5 Caminhos de tráfego

Caminho	Rede	Observação
Navegador → LB → hub	Pública, TLS gerenciado	Sessão do hub; SSR e BFF.
Navegador → LB → CDN → *bucket* de *bundles*	Pública, TLS	O grosso dos *bytes* do pico. Caminho versionado, portanto imutável; só o `index.html` tem `max-age=60`.
Navegador (dentro do `iframe`) → LB → API	Pública, TLS + *Bearer*	CORS por lista explícita (`CORS_ORIGINS`).
hub → API	Pública, TLS	`API_INTERNAL_URL`; o hub repassa o *token* do usuário.
API → Cloud SQL	**Privada**, VPC	O banco não tem IP público.
Cloud Run Job → Cloud SQL	**Privada**, VPC	Único caminho para rodar migração.
API → Vertex AI	Pública, TLS + ADC	Endpoint `global`.
API → Pub/Sub *topic*	Pública, TLS + SA	Publicação do lote analítico.
Pub/Sub → API (`/telemetry/ingest`)	Pública, TLS + *ID token*	Verificado por `PubSubPushGuard`: emissor Google, `aud` e `email` da SA de *push*.

hub → WSO2 IS	Pública, TLS	OIDC. Fora da *stack* do contrato.

ATENÇÃO

O *Load Balancer* exige **dois** certificados gerenciados quando o hub e a API estão em domínios distintos — um por domínio. Provisionar apenas o do hub faz a API responder com erro de TLS que *parece* problema de aplicação: o navegador não completa o *handshake* e a experiência mostra falha de rede sem nenhuma linha no log da API.

6.6 Identidade de carga de trabalho e segredos

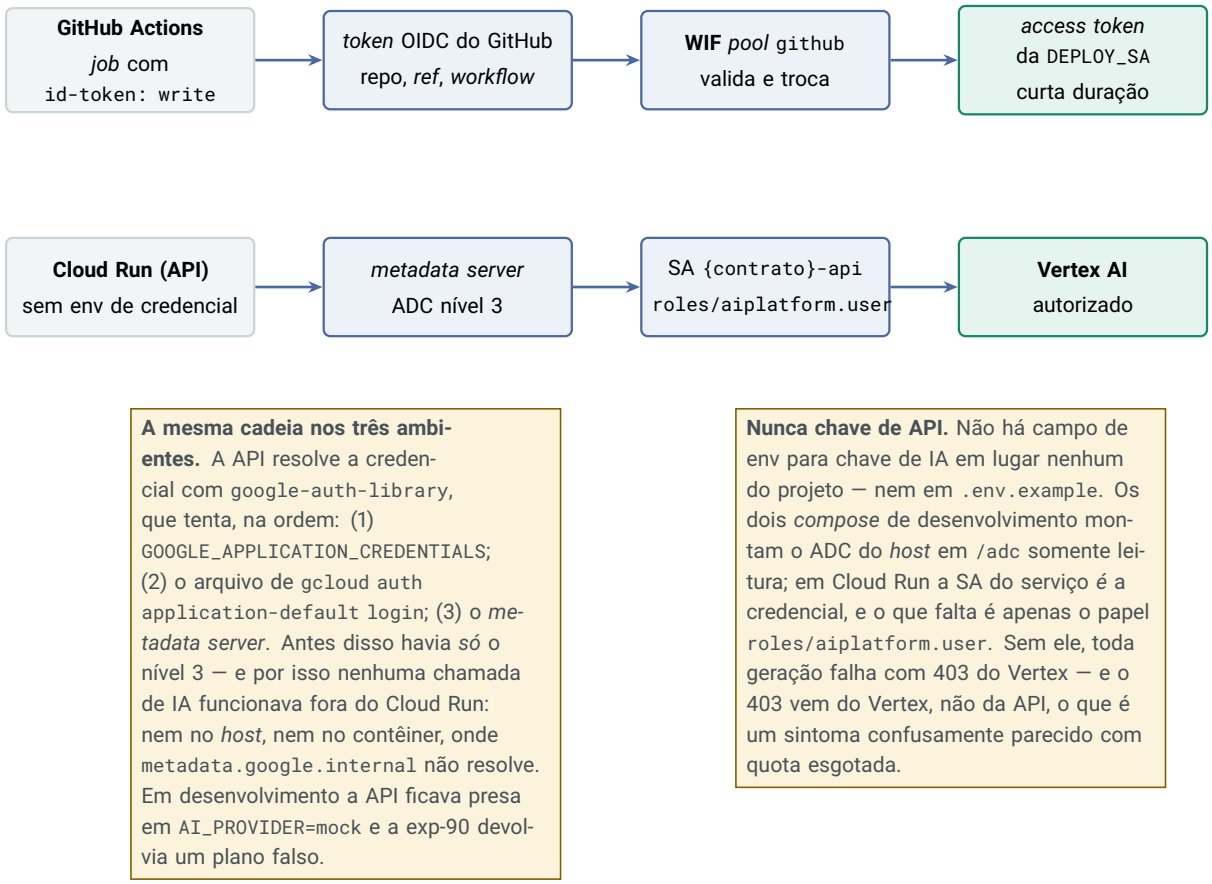


Figura 6.3 – Os dois caminhos de identidade de máquina, ambos sem chave de longa duração: WIF para o CI e ADC para a aplicação.

Segredo / variável	Onde vive	Por quê
`{contrato}-database-url`, `{contrato}-db-password`	Secret Manager	Credencial de banco.
`{contrato}-token-signing-k`	Secret Manager	Chave privada RS256 que assina o *token* de experiência. Também usada pelo CI para assinar o *token* de cinco minutos do *upsert* de manifesto.

`{contrato}-auth-secret`	Secret Manager	Segredo de sessão do NextAuth.
`{contrato}-perfil-import-` `{contrato}-planos-import-`	Secret Manager	Chaves de serviço de duas rotas de importação. **Ausentes = rota desligada** (503), nunca aberta.
`OIDC_ISSUER`, `OIDC_JWKS_URI`	*env* em texto plano	**Não são segredo**: são URLs públicas do hub. Ficam em texto plano derivadas de `hub_domain` — é isso que garante que o `iss` do *token* e o esperado pela API *não podem* divergir.
`TELEMETRIA_PUSH_SA`, `TELEMETRIA_PUSH_AUDIENCE`	*env* em texto plano	Identificam quem pode chamar `/telemetry/ingest`. Ausentes = rota 401.

6.7 Dimensionamento

Recurso	Configuração	Motivo
Cloud Run — API	CPU sempre alocada, `min-instances` ≥ 1, concorrência 80	Há trabalho *in-process* fora do ciclo de requisição: o *worker* de geração de conteúdo da exp-01 e a geração de plano de aula da exp-90. Com CPU liberada entre requisições, a geração morreria no meio.
Cloud Run — hub	`min-instances` ≥ 1	Elimina *cold start* no pico de *login*.
Cloud SQL	Instância pequena, HA, PITR, *pooler*	10.000 estudantes e o volume medido de telemetria cabem folgadamente. O *pooler* é obrigatório porque o *autoscaling* do Cloud Run multiplica *pools* do Prisma e estoura `max_connections`.
CDN	*cache* longo no caminho versionado; `max-age=60` no `index.html`	O caminho carrega a versão, logo o conteúdo é imutável. O `index.html` é o ponteiro: *cache* longo nele atrasaria um *rollback*.
Pub/Sub	*Push subscription* com `oidc_token`	Amortecedor durável do pico, e uma peça a menos que um *worker* dedicado.

6.7.1 Volumes medidos

Grandeza	Valor medido	Como se chegou a ele
Payload de uma partida da exp-70	1,1 kB de JSON (1,5 kB em JSONB)	Medido em 2026-08-26. O jogo tem sete pontos de log, todos atos discretos com guarda própria: 8 Chamas, 4 aldeões, 1 Borda + 3 Destaques, 4 cartões, 1 primeira conversa — teto real de ~22 eventos por partida. Nada por quadro, nada por metro andado.

Armazenamento da exp-70 por contrato	**~30** MB	10.000 estudantes jogando uma vez. A estimativa *anterior* — centenas de MB a 2,4 GB — partia do teto do contrato (2000 eventos) e errava por duas ordens de grandeza. O teto do contrato caiu de 2000 para 200, que é o que de fato limita a linha se um cliente adulterado mandar lixo.
Custo de gravação da telemetria	~16 `INSERT`/s no pico (era ~800)	Um lote de 50 eventos custava 50 *round-trips*; em `INSERT` multi-linha custa um. No pico de ~4000 simultâneos a diferença é entre saturar e não notar.
Prompt de um plano de aula anual	~190.000 *tokens*	Currículo da rede + as quatro unidades do ano. É o que justifica a quota separada de professor (2.000.000/dia ≈ 6 a 9 planos).
Mídia da exp-02	52 MB (21 arquivos MP4)	Fora do *git* e fora do *bundle*: vai para o *bucket* de *assets*, referenciada por `VITE_VIDEOS_BASE_URL`.

6.8 Ambientes

Ambiente	Como sobe	Diferenças relevantes
Host (desenvolvimento)	`docker compose up -d pg` + `pnpm dev`	Tudo no *host*; só o banco em contêiner. `AI_PROVIDER` pode ser `mock`; *drivers* de armazenamento locais; sem Pub/Sub.
Contêiner (desenvolvimento)	`pnpm docker:dev`	A *stack* inteira em contêiner — banco, API, hub e as cinco experiências — com o código do *host* por *bind mount*: salvar recompila lá dentro. Mesmas portas do `pnpm dev`, então é um *ou* o outro.
Imagens congeladas	`docker compose up -d --build`	Aproxima-se da produção: imagens construídas, migração e *seed* automáticos no primeiro `up`.
`dev` em GCP	CI no *push* para `main`	*Stack* real do piloto (`plataforma-unooria-dev`), com todos os recursos da Figura 6.2.
Produção	Mesmo CI, outras `vars` de repositório	O *workflow* não conhece o nome de nenhum recurso: tudo entra por `vars`, cujos valores saem de `terraform output`.

> **NOTA**
>
> Nos ambientes de contêiner, a leitura *server-side* do `experiencia.json` exige `EXP_INTERNAL_HOST` — que troca apenas o *host* e **preserva a porta**. `EXP_INTERNAL_URL`, que é um endereço fixo, só serve quando existe *uma* origem de *bundles* (a CDN, em produção): em desenvolvimento, cada experiência tem um *dev server* numa porta diferente, e um endereço fixo faria todas lerem o manifesto de uma só. O sintoma dessa configu-

ração errada é específico e memorável: o UnooTube, que não declara `viewport`, aparece dentro da casca de celular da Instanooria.

ATENÇÃO

Os *dev servers* das experiências declaram `allowedHosts: ["host.docker.internal"]`. O Vite 5.4+ recusa cabeçalho `Host` que não conheça, e sem essa declaração o hub em contêiner não lê o manifesto — e a experiência aparece esticada na janela em vez de dentro do aparelho.

6.9 O provedor de identidade fora da *stack*

O WSO2 Identity Server foge do padrão `{contrato}-*` por desenho: é **único para vários contratos** e tem ciclo de vida próprio. Vive em projeto GCP próprio e, em desenvolvimento, num *compose* separado (`wso2/docker-compose.yml`, IS em `:9443` e banco próprio em `:5433`).

DECISÃO DE PROJETO

O hub assina os *tokens* de experiência enquanto o IdP central não entra (**ADR-15**). O hub emite RS256 com TTL de 15 minutos e publica o JWKS em `/.well-known/jwks.json`; a API valida por `OIDC_ISSUER` e `OIDC_JWKS_URI`. Trocar para o WSO2 é trocar essas duas variáveis — o `OidcGuard` não muda uma linha. A alternativa (esperar o IdP para começar a integrar) faria toda a plataforma depender de um servidor Java em *bootstrap* para que qualquer experiência pudesse ser testada.

ATENÇÃO

O `issuer` do WSO2 é *tenant-qualified* (`.../t/{contrato}/oauth2/token`) e deve ser copiado **literalmente** do *discovery*, nunca montado por concatenação. Um `iss` montado à mão que difere por uma barra faz toda validação falhar com "*issuer* inválido" — e a mensagem não diz qual dos dois lados está errado.

6.10 Observabilidade, continuidade e reversão

Aspecto	Como é atendido hoje
Sanidade de processo	`GET /healthz` (processo vivo) e `GET /readyz` (dependências prontas). São rotas sem *guard*, de propósito.
Log	Cloud Logging pelo *stdout* do Cloud Run. **Nenhuma PII em texto claro**; texto livre de bem-estar nunca vai a log (só contagens).
Métricas	Métricas nativas do Cloud Run e do Cloud SQL. Não há métrica de aplicação customizada — é lacuna conhecida (Cap. 19).

Reversão do hub e da API	Revisões do Cloud Run. `gcloud run services update-traffic` devolve o tráfego à revisão anterior.
Reversão de experiência	`node apps/hub/scripts/publicar-manifesto.mjs <exp> --versao <anterior>`. Sem *build*, sem imagem.
Reversão de conteúdo de professor	Versionamento de objetos do *bucket* de planos: cada “Salvar” é uma versão, sem uma linha de código de histórico.
Recuperação do banco	HA + PITR do Cloud SQL.
Migração de *schema*	Cloud Run Job dentro da VPC, executado **antes** do *deploy* do serviço, com `--wait` para que a falha interrompa o *pipeline*.

LIMITAÇÃO CONHECIDA

Não há *alerting* configurado, nem painel. Há log e métrica nativa, e há sanidade por `/readyz` — o que basta para diagnosticar, mas não para ser avisado; e `/readyz` não é chamada por ninguém, porque o *deploy* não configura sonda. O estudo completo — o que se vê hoje, elo por elo, os sinais a acrescentar, a comparação de ferramentas e o roteiro em fases com custo por contrato — é o Capítulo 17. A fase 0 dele (canais, *uptime checks*, políticas, SLOs e painel, tudo em Terraform) é o que fecha o risco `R-05` antes de novembro de 2026.

7

Multi-tenancy e segurança

Este é o capítulo que, se estiver errado, invalida o produto. Uma plataforma que serve cem escolas de uma rede não pode, sob nenhuma circunstância — inclusive sob erro de programação — devolver o dado de uma escola a outra.

7.1 Os três níveis de isolamento

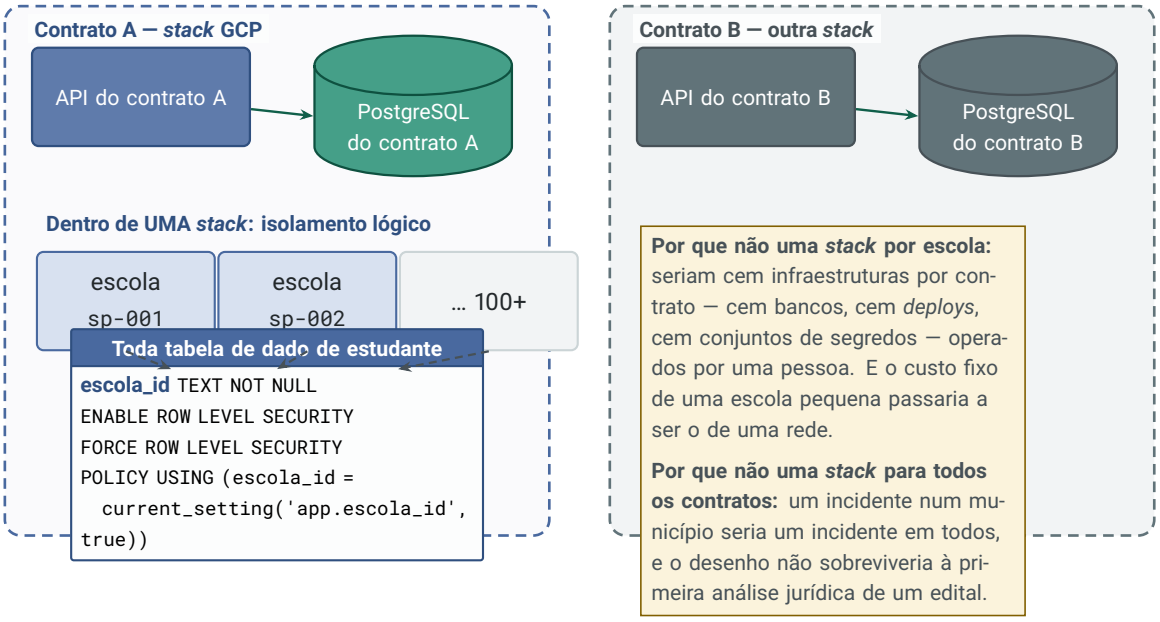


Figura 7.1 – Isolamento em três níveis: físico entre contratos, lógico entre escolas de um contrato, e por autorização entre turmas e estudantes de uma escola.

7.2 Nível 2: *Row-Level Security* no PostgreSQL

7.2.1 Como a política é armada

Toda tabela de dado de estudante tem `escola_id TEXT NOT NULL` desnormalizado e uma política de RLS (The PostgreSQL Global Development Group, 2026a). A migração inicial aplica isso em laço sobre a lista de tabelas:

Listing 7.1 – Aplicação da RLS na migração inicial

```
DO $$
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY[ /* ... tabelas de dado de estudante ... */ ] LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
    EXECUTE format('ALTER TABLE %I FORCE ROW LEVEL SECURITY', t);
    EXECUTE format(
      'CREATE POLICY %I ON %I USING (escola_id = current_setting(''app.escola_id'', true))',
      t || '_por_escola', t
    );
  END LOOP;
END $$;
```

DECISÃO DE PROJETO

`FORCE ROW LEVEL SECURITY`, não apenas `ENABLE`. `ENABLE` isenta o *dono* da tabela das políticas. Com `FORCE`, nem o dono escapa — e é isso que dispensou a segunda *role* de banco que o desenho original previa. O que continua obrigatório é a *role* da aplicação não ter `BYPASSRLS` nem `SUPERUSER`, porque esses atributos derrotam até o `FORCE`.

7.2.2 A porta única: `comTenancy`

Listing 7.2 – A única porta de acesso ao banco (`src/prisma/prisma.service.ts`)

```
async comTenancy<T>(
  fn: (tx: Prisma.TransactionClient) => Promise<T>,
  escolaId?: string,
  opts?: { timeout?: number },
): Promise<T> {
  const escola = escolaId ?? escolaDoContexto();
  if (!escola) {
    // Falha ALTA. A alternativa — consultar sem filtro — e exatamente o
    // acidente que este metodo existe para tornar impossivel.
    throw new Error(
      "comTenancy sem escola: rota sem OidcGuard/TenancyInterceptor ou " +
      "chamada fora de requisicao sem escolaId explicito",
    );
  }
  return this.$transaction(async (tx) => {
    // `true` = local a TRANSACAO. Um SET de sessao vazaria a escola para a
    // proxima requisicao que reusasse a conexao do pool.
    await tx.$executeRaw`SELECT set_config('app.escola_id', ${escola}, true)`;
    return fn(tx);
  }, opts);
}
```

Três detalhes desse método carregam a segurança inteira:

1. **O terceiro argumento de `set_config` é `true`**, ou seja, o valor é local à transação. Um `SET` de sessão sobreviveria ao `COMMIT` e a conexão voltaria ao *pool contaminada* — a próxima requisição, de outra escola, herdaria a política errada. Esse é o bug clássico de RLS com *pool* de conexões, e é evitado por um booleano.
2. **Sem escola, lança.** Não consulta "sem filtro", não devolve lista vazia: lança. Rota nova que esqueceu o `OidcGuard` quebra no primeiro teste, em vez de funcionar devolvendo tudo.
3. **A escola vem do contexto, não do parâmetro.** O parâmetro explícito existe só para código fora de requisição — *seed*, *worker*, testes.

REGRA INVIOLÁVEL

O `comTenancy` arma a RLS; ele NÃO filtra a *query*. Ele executa um `set_config` que as POLÍTICAS leem — e tabela sem política não é alcançada por ele. `users`, `turmas` e `lotacoes` estão deliberadamente fora da RLS (são lidas no *login*, antes de se saber a escola), então nelas o `escola_id` no `where` não é defesa em profundidade: **é a única tranca**.
Isso já custou um vazamento real. `distribuicaoDaTurma`, que monta o painel de perfis do Instanooria, rodava `user.findMany({ where: { role: 'estudante' } })` — dentro de um `comTenancy`, o que dava a impressão de estar protegido. O painel listava nome e login de crianças de TODAS as escolas do contrato, e os contadores que decidem quantos estudantes o botão "Redistribuir a turma" move eram calculados sobre esse universo. Ver **ADR-49**; a regressão é coberta por `tests/perfis-distribuicao-tenancy.spec.ts`, que roda como superusuário justamente para que a RLS não mascare a ausência do `where`.

7.2.3 Como a escola chega ao contexto

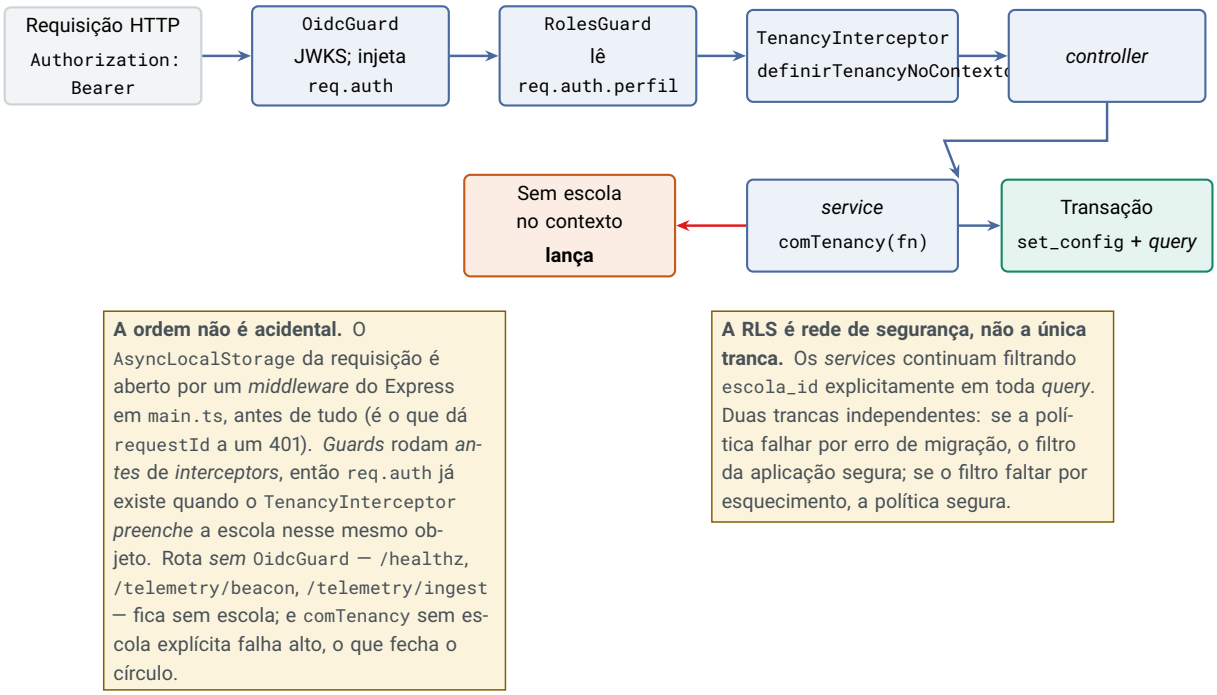


Figura 7.2 – Como a escola do *token* chega à política de RLS. Duas trancas independentes protegem o mesmo invariante.

7.2.4 A exceção: `admin-contrato`

> **REGRA INVIOLÁVEL**
>
> `admin-contrato` é o **único** papel autorizado a atravessar escolas, e só em rotas explicitamente marcadas com `@Roles("admin-contrato")`. Para esse papel, `escolaId` nas *claims* pode ser vazio — e o *CHECK* da migração permite `escola_id NULL` em `users` apenas quando `role = 'admin-contrato'`. Nenhuma outra combinação é aceita pelo banco.

As rotas cross-escola vivem em *controllers* separados (`src/analise/analise.controller.ts`, o *controller* de `/analise/*`) e não num `@Roles` misto com exceção por método — porque um `@Roles` misto é o caminho mais curto para um `GET` novo herdar silenciosamente a permissão do vizinho.

7.3 Nível 3: autorização por papel

Papel	Escopo de dados	Pode

`estudante`	Os próprios dados, na própria escola	Realizar experiência, salvar progresso, ver o próprio resultado. **Nunca** vê gabarito, nem dado de colega.
`professor`	A própria escola; **as turmas em que tem matrícula**	Preparar e conduzir atividade, ler turma e estudante individual, gerar plano de aula, fazer o próprio *check-in*. **Não** vê bem-estar de outro professor, nem turma em que não leciona.
`gestor`	A própria escola	Importar pessoas, enturmar, redefinir senha, ler agregado da escola. **Não** vê a linha de uma criança nos instrumentos de valores, nem qualquer dado de bem-estar.
`admin-contrato`	Toda a rede	Ler agregado da rede, publicar manifesto. **Não** tem rota para a linha individual de uma criança.

7.3.1 Onde cada papel entra

O *middleware* do hub (`apps/hub/middleware.ts`) não conhece telas: ele conhece **áreas**, e `apps/hub/lib/papeis.ts` é a única fonte do mapa papel → área. Quem digita a URL de outra área não recebe 403 — é redirecionado para a própria casa, porque estudante que tropeça em `/professor` não precisa de uma mensagem de erro, precisa voltar.

Papel	Casa	O que existe na área
`estudante`	`/estudante`	O *hub* de missões e o aparelho da experiência (`/exp/:expId`). São a MESMA área: o hub escolhe, o `iframe` joga.
`professor`	`/professor`	Atividades (exp-01), turmas, guia, plano de aula (exp-90), bem-estar (exp-91) e "entre redes" — os resultados por unidade.
`gestor`	`/admin/usuarios`	Usuários, turmas, importar, ano letivo. E "minhas escolas", que só aparece para quem tem mais de uma lotação.
`admin-contrato`	`/admin/usuarios`	O mesmo, mais `/admin/escolas` (abrir unidade) e `/analise` (o nível de produto, na rede inteira).

> **ATENÇÃO**
>
> **Gestor e `admin-contrato` moram na mesma área do *middleware*** — `areaDoPapel` devolve `/admin/usuarios` para os dois. A distinção entre eles **não** é feita ali: é feita item a item na navegação (`SO_ADMIN_DO_CONTRATO` em `components/nav-admin.tsx`), por `redirect` na página que não admite gestor (`/admin/escolas`) e, definitivamente, pelo `RolesGuard` da API. Um gestor que digite `/analise` vê a tela abrir e ficar **vazia**: a API responde 403, o BFF converte em `null` e não há o que renderizar. Nenhum dado atravessa — mas a tranca que segurou foi a segunda, não a primeira.

7.3.2 A matriz de capacidades

As duas tabelas a seguir são o **contrato de autorização** lido do código: cada linha existe porque há um `@Roles` que a sustenta. ✓ é "pode"; – é "não existe rota que faça isso para este papel" — a ausência é estrutural, não disciplinar.

Identidade, enturmamento e rede	Rota	Est.	Prof.	Gest.	Adm.
Criar pessoa — em lote, por IMPORTAÇÃO	`POST /rostering/importar`, `/rostering/oneroster/:arquivo`	–	–	✓	✓
Criar UMA pessoa — o formulário que delega ao importador	`POST /usuarios`	–	–	✓	✓
Criar outro `admin-contrato`	`POST /usuarios/admin-contrato`	–	–	–	✓
Listar as pessoas da escola	`GET /usuarios`	–	–	✓	✓
Editar pessoa (nome, CPF, e-mail, papel, `ativo`)	`PATCH /usuarios/:id`	–	–	✓	✓
Redefinir senha (temporária, mostrada uma vez)	`POST /usuarios/:id/reset-senha`	–	–	✓	✓
Diagnosticar contas sem matrícula da rede	`GET /rostering/contas-de-assento`	–	–	✓	✓
Criar e editar turma	`POST`, `PATCH /turmas`	–	–	✓	✓
Matricular, ajustar e encerrar matrícula	`/turmas/:id/matriculas`, `/matriculas/:id`	–	–	✓	✓
Ler as turmas em que atua	`GET /turmas/minhas`	–	✓	✓	✓
Abrir uma turma e ver quem está nela	`GET /turmas/:id`	–	✓*	✓	✓
Abrir o ano letivo	`POST /periodos-letivos`	–	–	✓	✓
Ler os períodos letivos	`GET /periodos-letivos`	–	✓	✓	✓
Abrir ou editar ESCOLA	`POST`, `PATCH /escolas`	–	–	–	✓
Conceder lotações em lote (nível da rede)	`POST /escolas/lotacoes-em-lote`	–	–	–	✓
Lotar alguém numa escola, encerrar lotação	`POST /escolas/:id/lotacoes`, `DELETE /lotacoes/:id`	–	–	✓	✓

Identidade, enturmamento e rede	Rota	Est.	Prof.	Gest.	Adm.
Transferir estudante entre escolas do contrato	`POST /estudantes/:id/transferir`	–	–	✓	✓
Ler o histórico de quem foi transferido	`GET /estudantes/:id/historico`	–	✓	✓	✓
Emitir, importar e remover perfil portátil	`/estudantes/:id/perfil-portatil`, `/perfis-recebidos`	–	–	✓	✓
Ver as escolas em que tem lotação	`GET /escolas/minhas`	✓	✓	✓	✓
Publicar ou atualizar experiência no catálogo	`PUT /experiencias/:expId`	–	–	–	✓
Ler o catálogo de experiências	`GET /experiencias`	✓	✓	✓	✓

* O professor só abre a turma **em que tem matrícula** (`professorLeciona`); gestor e admin enxergam a escola inteira. É a diferença entre "dá aula nesta turma" e "trabalha nesta escola" (**ADR-28**).

Vida pedagógica e leitura de dados	Rota	Est.	Prof.	Gest.	Adm.
Realizar experiência e salvar progresso	`/progresso/:expId`, `/runs/*`	✓	–	–	–
Preparar e conduzir atividade (exp-01)	`/atividades/*`	–	✓	–	–
Destilar, gerar e regenerar o *feed* com IA	`/atividades/:id/destilar`, `/gerar`, `/posts/:id/regenerar`	–	✓	–	–
Monitor da turma e análise da atividade (+CSV)	`GET /atividades/:id/monitor`, `/analise`	–	✓	–	–
Reiniciar o *run* de um estudante	`POST /runs/:id/reset`	–	✓	–	–
Ler o dossiê de um *run*	`GET /runs/:id/dossie`	✓†	✓	–	–
Vila do Lume: agregado da turma	`/vila/agregado/*`	–	✓	✓	–
Vila do Lume: a linha de uma criança	`/vila/professor/*`	–	✓	–	–
unooGPT: a própria partida	`/unoogpt/*`	✓	–	–	–
unooGPT: turma e transcrição individual	`/unoogpt/professor/`	–	✓	–	✓
unooGPT: agregado da escola (sem `estudanteId`)	`/unoogpt/agregado/escola`	–	–	✓	✓
unooTube: a própria sessão	`/unootube/*`	✓	–	–	–

Vida pedagógica e leitura de dados	Rota	Est.	Prof.	Gest.	Adm.
unooTube: resultados por turma	`/unootube/turmas/*`	–	✓	✓	✓
Bem-estar: o próprio *check-in*, histórico, apagar	`/bemestar/*`	–	✓	–	–
Plano de aula: criar, versionar, restaurar	`/planos/*`	–	✓	–	–
Análise de produto, na rede (+CSV)	`/analise/*`	–	–	–	✓
Chamar a IA (com quota diária POR PERFIL)	`POST /ai/gerar`	✓	✓	✓	✓
Enviar telemetria	`POST /telemetry/batch`	✓	✓	✓	✓

† O estudante lê o dossiê do **próprio** *run*; a posse é verificada no *service*, não pelo papel.

REGRA INVIOLÁVEL

As três ausências mais importantes desta tabela. *Bem-estar* não tem uma única rota de gestor nem de agregado — o instrumento é do professor, sobre o professor. No *unooGPT*, o controller do gestor **não tem rota que aceite** `estudanteId`: não é que ele não deva ver a linha de uma criança, é que não existe endereço onde ela caiba. E `admin-contrato` não alcança a Vila nem o dossiê: ele lê a rede em agregado, nunca a sala.

DECISÃO DE PROJETO

Pessoa nasce por importação — e o cadastro unitário DELEGA a ela. Existe `POST /usuarios`, e ele não escreve em `users`: monta a MESMA linha da planilha e chama o MESMO importador (**ADR-59**, acionando o gatilho da **ADR-54**). Quem grava continua sendo `POST /rostering/importar` — ou `/rostering/oneroster/users`, para a rede que já exporta o padrão — e por isso as regras de CPF, série, turno, login gerado, lotação e matrícula são as mesmas nas duas telas.

A diferença é de SEMÂNTICA: a importação é idempotente pela matrícula da rede (e pelo CPF), então reimportar não duplica nem reseta a senha de quem já entrou; o formulário, que se chama "Novo usuário", **recusa com 409** quem já existe, em vez de atualizar em silêncio — e o 409 de um CPF achado em outra escola sai sem id, sem nome e sem dizer qual unidade.

A exceção é `admin-contrato`: ele não tem `escola_id`, lotação nem turma, não cabe numa planilha DA ESCOLA, e por isso tem rota própria, `POST /usuarios/admin-contrato`, restrita a ele mesmo. Gestor recebe 403 ali — conceder o alcance da rede inteira é decisão do contrato, nunca de uma unidade.

7.3.3 As portas que não são papel

Cinco grupos de rotas não passam pelo `RolesGuard` porque não há papel para checar — e cada uma prova outra coisa no lugar. Elas são a superfície que um leitor da matriz acima poderia concluir, erradamente, que não existe.

Rota	Prova	Por quê
`POST /auth/login`, `POST /auth/trocar-senha`	A credencial	Quem chega ainda não tem *token*. Trocar a senha provisória exige a ATUAL — mesmo nível de prova do login. 8 tentativas por minuto por usuário.
`GET /healthz`, `/readyz`	Nada	*Probe* do Cloud Run. Não tocam dado de ninguém.
`POST /telemetry/beacon`	O *token* no CORPO	`navigator.sendBeacon` não manda cabeçalho. Responde 204 sempre: a página já foi descartada e ninguém leria o erro.
`POST /telemetry/ingest`	`PubSubPushGuard`	O *push* do Pub/Sub, autenticado pelo OIDC da conta de serviço — não por pessoa.
`POST /perfis/importar`, `/planos/referencias`	Chave de serviço (`x-perfil-token`, `x-import-token`)	Máquina falando com máquina no *provisionamento*. Nenhuma delas é alcançável a partir do navegador de um usuário.

7.3.4 O desenho por trás

> **DECISÃO DE PROJETO**
>
> **Agregado e linha individual são *controllers* diferentes.** Em `src/vila/` há `VilaProfessorController` com dois `@Controller` distintos: `/vila/agregado/*` abre para `professor` e `gestor`; `/vila/professor/*`, que devolve a linha de uma criança, só para `professor`. A separação física é o que garante que a próxima rota adicionada ao arquivo herde a permissão *certa*.

> **DECISÃO DE PROJETO**
>
> **Dentro da escola, o escopo do professor é a MATRÍCULA.** Até a migração do enturmamento não havia vínculo professor↔turma no modelo: os seletores listavam todas as turmas da escola e a única prova exigida para abrir os resultados de uma delas era "mesma escola do *token*" — que é a regra de *tenancy*, não a de quem dá aula ali. O professor do 8A abria bem-estar e Vila do 9B.
>
> Agora `matriculas` responde por isso: `ondeAsTurmasSaoVisiveis` filtra as listas e `professorLeciona` vira 403 nas telas de uma turma só. Gestor e admin-contrato seguem enxergando a escola inteira, de propósito — é a mesma distinção que o OneRoster faz entre `roles`, no nível da organização, e `enrollments`, no nível da turma (**ADR-28**).

DECISÃO DE PROJETO

Numa REDE, a pessoa atravessa escolas; o *token* não. O professor lotado em duas unidades escolhe em qual está trabalhando, e o *token* leva **essa** — então a política de RLS e as 31 tabelas com `escola_id` continuam exatamente como estavam (**ADR-30**). A troca de escola ativa é validada contra as lotações *pela API*, nunca contra a lista que o *cookie* carrega: *cookie* é dado do cliente.
Acima da escola existe um nível — a **regional** (Diretoria de Ensino, polo, secretaria municipal) —, e ele **agrupa sem autorizar**. Estar numa diretoria não dá acesso a nada: quem dá acesso é a lotação, uma por unidade. A hierarquia serve para MONTAR a lista de escolas que o admin do contrato confirma antes de conceder (`POST /escolas/lotacoes-em-lote`), e nenhuma consulta de autorização lê `escolas.pai_id`. Autorizar por hierarquia seria uma segunda fonte de verdade sobre acesso, divergente da lotação no dia em que alguém corrigisse um pai (**ADR-34**).
E na transferência de um estudante, o dado **fica onde foi produzido**. O que atravessa é uma leitura única e estreita (`GET /estudantes/:id/historico`), restrita a quem tem lotação na escola onde a criança está hoje, devolvendo participações e nunca o conteúdo dos instrumentos (**ADR-32**).

DECISÃO DE PROJETO

Sair da REDE é outra coisa, e sai como DOCUMENTO. Transferência entre escolas do contrato é lotação. Já a criança que vai para outra rede sai da instalação — outro banco, nenhum *JOIN* —, e não há garantia de que o destino seja cliente. Por isso a saída é um arquivo assinado pelo hub (a mesma chave dos *tokens*, com `aud` próprio), emitido a pedido do titular (LGPD art. 18, V), e o que chega de fora vive numa tabela só dele: nunca vira matrícula, progresso nem perfil, e nunca entra em média. Assinatura inválida é RECUSA; emissor inalcançável é um rótulo visível na tela (**ADR-35**).

DECISÃO DE PROJETO

A senha do primeiro acesso é travada em três lugares. Enquanto `senha_provisoria` for verdadeira, o *middleware* prende as páginas em `/trocar-senha`, o BFF recusa dados e `/api/exp-token` não emite *token* de experiência. Só com o *middleware*, bastaria digitar `/exp/...` para pular a troca — a experiência abriria por baixo, com o *token* que o hub continuaria assinando (**ADR-29**).

7.4 Segurança do *embed*

A fronteira mais exposta da plataforma não é a API: é o `iframe`. Dentro dele roda código que, em produção, vem de outra *origin* (a CDN) e que recebe um *token* do usuário.

Medida	Implementação e por quê

`targetOrigin` explícito	Nunca `"*"`, nos dois sentidos. O hub usa a *origin* derivada da *mesma* URL do `iframe`; a SDK usa o `hubOrigin` recebido por *query string*. `"*"` entregaria o *token* a qualquer documento que conseguisse virar o pai do *frame* (Mozilla Developer Network, 2026b).
`hubOrigin` normalizado	A *query string* é entrada NÃO CONFIÁVEL — o *bundle* é público, e quem embedar a experiência escolhe aquele parâmetro. `normalizarHubOrigin()` o reduz a uma origem canônica, recusa esquema fora de http(s), credencial e origem opaca, exige hub em https quando a experiência está em https, e — a trava que fecha o caminho — confere contra `location.ancestorOrigins[0]`, a origem REAL de quem embedou, que vence a *query string* (**ADR-46**).
Validação de `ev.origin`	Os dois lados descartam mensagem cuja *origin* não bate, *antes* de olhar o conteúdo.
Validação por esquema	Toda mensagem pós-*init* passa por Zod nos dois lados; o que não casa é descartado com aviso, **nunca** despachado cru aos *listeners*.
Verificação de versão maior	Hub e experiência comparam `protocolVersion`. Incompatível falha alto: sem `hub:init`, o *handshake* expira e a experiência mostra erro.
Token só em memória	A experiência nunca escreve o *token* em `localStorage` nem em *cookie*. TTL de 15 minutos, renovado por mensagem.
Sandbox mínimo	`allow-scripts allow-same-origin allow-modals`, mais `allow-pointer-lock` **apenas** para quem declara `pointer-lock` no manifesto (Mozilla Developer Network, 2026a).
`allow` do *frame*	`autoplay; fullscreen; clipboard-write`. A *Permissions-Policy* tem *default* `'self'`: sem delegar `autoplay`, um `<video muted autoplay>` cross-origin rejeita `play()` em silêncio.

NOTA

Cada token do *sandbox* está lá por um motivo medido: `allow-same-origin` porque, sem ele, a *origin* do `iframe` cross-origin vira `"null"` e a validação de *origin* quebra dos dois lados; `allow-modals` porque, sem ele, o navegador *ignora* `confirm()` do `iframe` silenciosamente (devolve `false`), e a exp-90 confirma exclusão assim; `allow-pointer-lock` porque, num `iframe` com *sandbox*, `requestPointerLock()` morre com `SecurityError` sem ele — verificado em `scripts/probe-pointerlock.mjs`, que precisa do Chrome do sistema porque o Chromium do Playwright não concede *pointer lock* nem no topo da página.

DECISÃO DE PROJETO

Permissão de `iframe` é por experiência, não *default* do *frame*. Prender o ponteiro do mouse de um estudante é capacidade de quem a declarou — e concedê-la a todas as experiências significaria que qualquer *bundle* publicado pode capturar o mouse. Já `fullscreen` *não* entra no manifesto: o `allow` do *frame* já o delega a todas, e campo de

> manifesto que não muda nada é pior que campo nenhum, porque quem lê acredita ter declarado algo.

7.5 Segredos e o que nunca vai ao cliente

REGRA INVIOLÁVEL

Nenhum segredo em *bundle* de cliente. Sem chave de API, sem *prompt* de sistema, sem URL interna. Consequências concretas:

- Chamada de IA só existe como `POST /ai/gerar` na API. O *prompt* de sistema mora em `apps/api/src/ai/prompts.ts` e nunca é devolvido.
- O gabarito da exp-02 (`factual`) não é persistido no estado do estudante nem devolvido em nenhuma rota de estudante.
- Os textos dos cartões da exp-70 chegam ao jogo **sem gabarito**, e apenas os da bolha daquela criança.
- `lado`, `perfil` e `intensidade` dos posts da exp-01 são removidos pelo sanitizador antes de o *feed* sair da API — são a variável independente do experimento.

7.6 Privacidade e dados de menores

ATENÇÃO

O CPF é guardado, e o que o protege é a CONTENÇÃO. Ele é a identidade imutável na rede — o que torna possível reconhecer a mesma criança numa transferência — e o gestor precisa vê-lo, para conferir identidade contra o sistema da rede e atender a família no balcão (**ADR-33**). CPF é dado pessoal comum, não do art. 11.
O que o cerca: `users.cpf` é o **único** lugar do banco onde o número existe; ele nunca vai a *log* (nem em mensagem de erro), nunca a telemetria, JSONB, *claim* de *token* ou *bundle* de experiência; e só gestor e admin-contrato alcançam a rota que o devolve. O teste estrutural cobra os dois lados — que só exista essa coluna, e que nenhum JSONB de telemetria ou progresso carregue onze dígitos seguidos, que é o caminho mais fácil de ele escapar.

Prática	Como é garantida
Telemetria sem PII	O evento carrega `estudante_id` opaco (UUID), nunca nome ou *e-mail*. A identidade é derivada do *token* no servidor, não do corpo enviado pelo cliente — o que é, ao mesmo tempo, anti-*spoof* e garantia de que o cliente não *pode* enviar PII no lugar do id.

Sem canal de texto arbitrário	Em `exp_70_sessoes.payload`, o campo `eventos[].dados` aceita apenas número, booleano ou *string* de um **vocabulário fechado**. O campo `texto` livre que existia era o único canal de texto arbitrário do *payload* — e o caminho por onde o nome de uma criança chegaria a um JSONB de longa duração. É trava estrutural, não lista de proibidos.
Texto sensível fora de log	O texto livre do *check-in* de bem-estar nunca vai a log nem à telemetria; só contagens.
IA sem contexto identificável	O *prompt* de bem-estar recebe apenas dia da semana, mês, número do *check-in* e o texto livre *delimitado como conteúdo*. Nunca nome, escola, pontuação ou faixa.
Ranking sem menor real	O ranking da exp-02 usa colegas **fictícios**, com distribuição desenhada para a lição. Não há pontuação de criança real exposta a outra criança.
Dado de saúde com posse	`exp_91_ciclos` e `exp_91_checkins` têm RLS e filtro por `professor_id` em toda *query*, e não existe rota de gestor nem agregado. O próprio titular apaga tudo.

7.6.1 Retenção: uma decisão explícita de não declarar prazo

DECISÃO DE PROJETO

Não há prazo de retenção declarado para `exp_70_sessoes` nem para `telemetria_eventos` (decisão de 2026-08-26, **ADR-13**).
Havia uma coluna `expires_at` de 90 dias no *schema* e *nenhuma rotina que a cumprisse*. Coluna de prazo que ninguém lê é pior que prazo nenhum, porque quem lê o *schema* acredita nela — é a mesma armadilha de um campo `consentimento_em` que existia no aplicativo de origem: a regra saiu do código, a coluna ficou, e todo mundo continuou acreditando. A coluna foi removida na migração `20260826170000_vila_sem_prazo`.
O que se ganha em troca é o **reprocessamento**, que é requisito real deste instrumento: `payload` é a única fonte para recomputar o perfil quando a versão do mapeamento subir, e o mapeamento é hipótese de *design* que vai mudar. 90 dias matavam exatamente a comparação entre duas gerações do mapeamento.
Se um contrato exigir prazo, ele volta **junto com a rotina que o cumpre** — nunca só a coluna. Com a tabela particionada por mês, expurgo passou a ser `DROP TABLE particoes.<mês>` em vez de um `DELETE` que trava a tabela.

LIMITAÇÃO CONHECIDA

A pergunta "qual é o prazo de retenção do dado comportamental de menor" está **aberta** e é decisão de produto e de contrato, não de módulo: ela vale para a plataforma inteira, porque `telemetria_eventos` guarda evento por `estudante_id` de *todas* as experiências. Enquanto não for respondida, o que existe é esta seção — que é melhor que uma coluna mentindo.

7.6.2 Localidade de processamento

> **ATENÇÃO**
>
> O modelo de IA em uso (`gemini-3.6-flash`) só é publicado no endpoint `global`: medido no projeto do contrato, ele responde 404 em `southamerica-east1`, `us-east1` e `us-central1`. A consequência é explícita e foi aceita: **o tráfego de IA sai de São Paulo, inclusive o do estudante**, e isso precisa constar do contrato e do DPA da rede. Voltar à região é possível a qualquer momento, mas exige trocar *também* o modelo pelo que a região publica (lá, a família 2.5) — as duas variáveis mudam juntas ou a geração para de funcionar.

7.7 Verificação

Segurança não documentada por teste é intenção. O que é efetivamente verificado:

Teste	O que afirma
`tests/tenancy.spec.ts`	Com uma *role* sem *ownership* e a política ativa, uma *query* da escola A não retorna linha da escola B — nem quando o filtro da aplicação é omitido.
`tests/planos-tenancy.s`	Um professor não lê nem edita plano de outro professor, mesmo na mesma escola (RLS + posse).
`tests/telemetria-push.spec.ts`	`POST /telemetry/ingest` rejeita *token* sem a `aud` correta, sem a SA correta, e **rejeita tudo** quando as variáveis estão ausentes.
`packages/sdk/src/bridg`	Mensagem de *origin* errada é ignorada; mensagem que não valida é descartada; versão maior divergente rejeita a conexão.
`tests/motor-concorrencia.spec.ts`	Dois pedidos simultâneos de bloco não descartam um bloco (*lock* consultivo).
`tests/vila-regras.spec`	*Payload* que traga chave de perfil é recusado; bolha B não é servida nos anos iniciais; `zero` e `none` não se confundem.
`scripts/probe-pointerlo`	Sem o token de *sandbox*, `requestPointerLock()` falha; com ele, funciona.

7.8 Ameaças consideradas

Análise no espírito do OWASP Top 10 (OWASP Foundation, 2021), restrita ao que é específico desta plataforma.

Ameaça	Categoria	Mitigação
Estudante lê dado de outra escola	*Broken access control*	RLS forçada + filtro explícito + *claims* do *token*. Duas trancas independentes, uma delas no banco.

Estudante lê o gabarito	Exposição de dado sensível ao negócio	Correção *server-side*; gabarito não persistido no estado do estudante nem devolvido; sanitizador remove metadados de curadoria.
Cliente adulterado envia perfil pronto	Integridade de medida	Servidor *recalcula* e recusa *payload* que traga chave de perfil; cap de eventos por partida.
Injeção de telemetria falsa	Autenticação ausente em *endpoint* de serviço	`PubSubPushGuard` verifica emissor, audiência e SA; *fail closed* sem configuração.
Roubo de *token* do `iframe`	Exposição de credencial	TTL de 15 min, escopo mínimo, só em memória, `targetOrigin` explícito, HTTPS. E a correção é *server-side* de qualquer forma, então o *token* não compra nota.
Esgotamento de quota de IA por abuso	Consumo de recurso	Quota diária persistida por (escola, usuário, dia), com incremento atômico, verificada *antes* da chamada; *rate limit* por usuário.
Vazamento de PII por log	Privacidade	Proibição explícita; telemetria com id opaco; vocabulário fechado no *payload*; texto sensível nunca logado.
Chave de serviço vazada	Gestão de segredo	Não existe chave de longa duração: WIF no CI, ADC na aplicação.
Rota nova esquecendo a *tenancy*	Erro de programação	`comTenancy` *lança* sem escola; não há caminho alternativo ao banco.

LIMITAÇÃO CONHECIDA

Não foi feita revisão de segurança externa nem teste de intrusão. Para uma plataforma que trata dado de menor numa rede pública, isso é item de contratação — e está registrado como risco `R-06` (Capítulo 19).

8

Modelo de dados

Um banco, um *schema* lógico, 27 tabelas em `public` e um *schema* auxiliar (`particoes`) fora do alcance do ORM. Este capítulo explica a convenção de nomes, o modelo por área e as decisões de esquema que não são óbvias — que, neste projeto, são a maioria.

O dicionário de dados campo a campo está no Apêndice E.

8.1 A convenção de nomes

REGRA INVIOLÁVEL

Tabela de experiência leva o número dela: `exp_NN_`. Assim: `exp_70_sessoes`, `exp_02_sessoes`, `exp_03_sessoes`, `exp_91_checkins`.
Tabela da plataforma — que serve a todas — fica sem prefixo: `escolas`, `turmas`, `users`, `experiencias`, `progresso`, `telemetria_eventos`, `ia_uso_diario`.
Tabela que é contrato entre duas experiências carrega os dois números: `exp_01_exp_70_perfis_importados` — a exp-70 escreve, a exp-01 lê. Chamá-la de uma só mentiria sobre quem depende dela.

DECISÃO DE PROJETO

Um banco, um *schema* — nem banco por experiência, nem *schema* por experiência.
Banco por experiência não funciona porque *as experiências não são isoladas*: o perfil de valores medido na Vila do Lume alimenta a largada do Instanooria, e o PostgreSQL não faz `JOIN` entre bancos. Além disso, um *pool* do Prisma por experiência estouraria `max_connections` com o *autoscaling* do Cloud Run.
Schema por experiência não funciona porque a única coisa que um *schema* daria além do prefixo é fronteira de `GRANT` — e existe **uma** *role* de aplicação. Seriam 48 *schemas* para obter exatamente o que um prefixo de nome já dá.

8.1.1 A migração de referência, e duas armadilhas registradas

A renomeação para o padrão `exp_NN_` está em `20260827090000_tabelas_por_experiencia`, e é a migração de referência para qualquer renomeação futura: `RENAME`, nunca `DROP`/`CREATE`, e renomeando *constraints*, índices e políticas de RLS junto.

ATENÇÃO

Duas armadilhas registradas naquela migração, ambas produzindo o mesmo sintoma — nomes duplicados como `exp_01_exp_01_..._pkey`:

1. `FOR ... IN SELECT` sobre o catálogo do sistema **relê o que o próprio laço alterou**. A lista de tabelas a renomear tem de ser materializada antes do laço.
2. `RENAME CONSTRAINT` renomeia **também** o índice que a sustenta. Renomear os dois em sequência aplica o prefixo duas vezes.

8.2 Modelo da plataforma

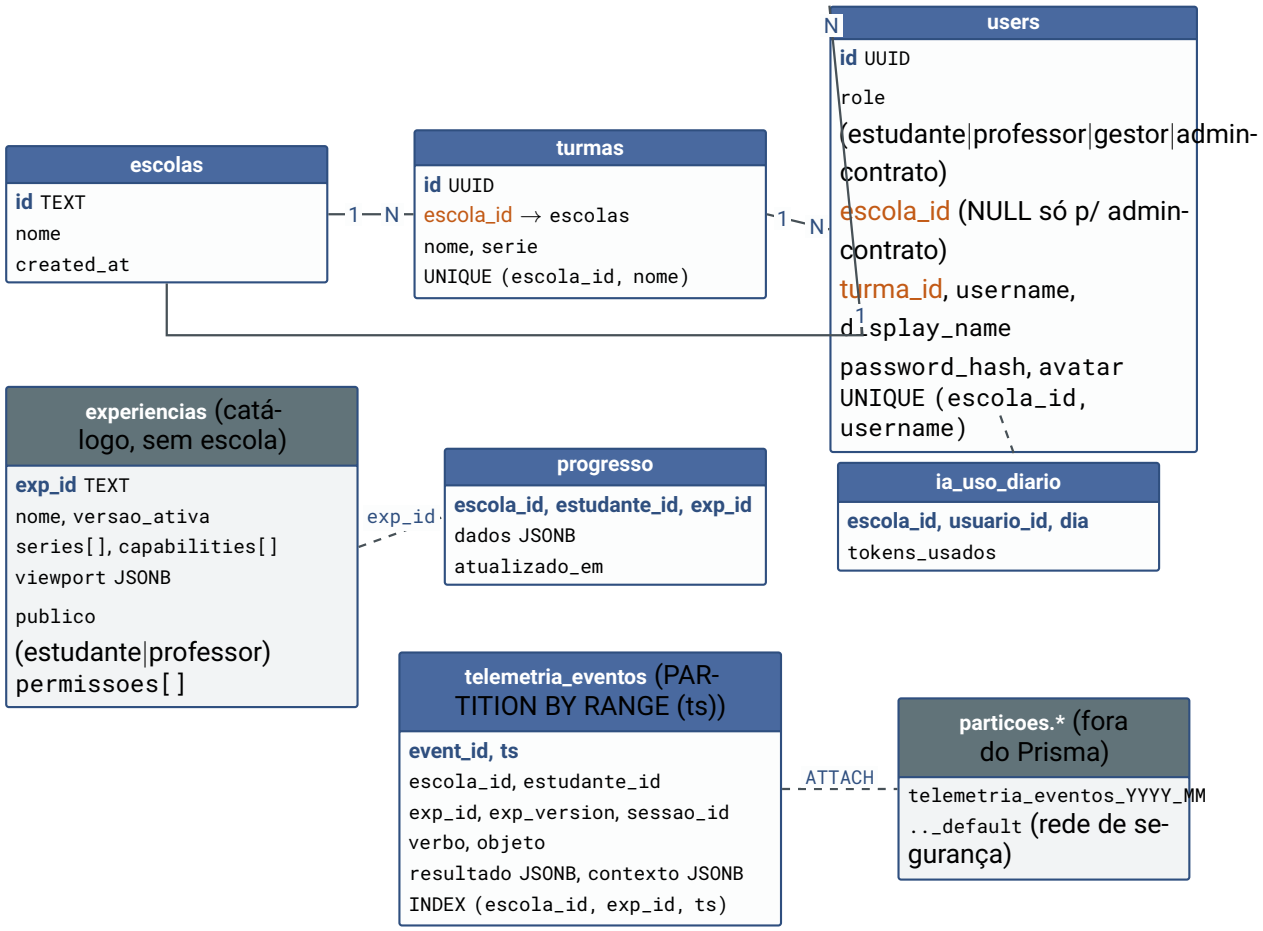


Figura 8.1 – Modelo de dados da plataforma. As caixas em azul são tabelas de *tenant* (têm `escola_id` e RLS); as cinzas são **catálogo do contrato** — `experiencias` e as partições de telemetria não pertencem a uma escola e ficam fora da RLS de propósito.

Tabela	Regime	Nota de esquema
`escolas`	*Tenant* raiz	O `id` é **texto**, e é o *mesmo* valor que circula no *claim* `escola_id` do JWT e no `set_config` da RLS. Um id legível (`sp-001`) vale mais que um UUID quando o valor aparece em três lugares diferentes de um diagnóstico. A tabela guarda TRÊS níveis (`tipo`: `rede` → `regional` → `escola`), mas só a `escola` é *tenant*: regional e rede não têm turma, estudante nem lotação.
`turmas`	*Tenant*	`UNIQUE (escola_id, periodo_letivo_id, nome)`: duas escolas podem ter "8A", e a mesma escola tem o "8A" de 2026 e o de 2027 — são turmas **diferentes**, e é essa diferença que preserva o histórico na virada do ano.
`users`	*Tenant*	`username` é único **por escola**, não globalmente — uma rede tem muitos `joao.silva`. *CHECK*: `escola_id NOT NULL` exceto quando `role = 'admin-contrato'`. A identidade da PESSOA é `matricula_externa` (o RA da rede), com índice único **parcial** por escola.
`periodos_letivos`	Catálogo	Ano letivo e seus recortes. Sem `escola_id` e sem RLS: o calendário é do contrato, e duplicá-lo por escola só criaria cem linhas divergentes.
`lotacoes`	Identidade	O vínculo PESSOA × ESCOLA. Professor **acumula** escolas; estudante tem uma ativa (índice único parcial no banco). Fora da RLS pelo mesmo motivo de `users`: é lida no *login*, antes de se saber a escola (**ADR-30**).
`matriculas`	*Tenant*	O enturmamento. Ver Seção 8.4.
`experiencias`	Catálogo	Sem `escola_id`: o manifesto é do contrato, não da escola. Escrita só por `admin-contrato`.
`progresso`	*Tenant*	PK composta `(escola_id, estudante_id, exp_id)`. É o progresso *genérico* da SDK; estado rico de experiência vai para a tabela da experiência.
`ia_uso_diario`	*Tenant*	Contador de quota, uma linha por (escola, usuário, dia). Incremento atômico por `ON CONFLICT DO UPDATE ... RETURNING`.
`telemetria_eventos`	*Tenant*, particionada	Ver Seção 8.5.
`perfis_portateis`	*Tenant*	O documento assinado que veio de OUTRA instalação (**ADR-35**). Nunca se mistura com o dado local: não vira matrícula, progresso nem perfil, e não entra em média. É a única exceção à contenção do CPF — o número é o conteúdo do documento, e alterá-lo destruiria a assinatura.

8.3 A rede: escola, lotação e o CPF

Uma rede estadual tem milhares de unidades, professores que dão aula em mais de uma e crianças que se transferem entre elas. Três colunas e uma tabela sustentam isso, e a decisão que as torna baratas é o que **não** muda: o *token* continua carregando UMA escola, então as 31 tabelas com `escola_id` e a política de RLS ficam intocadas (**ADR-30**).

Peça	O que resolve, e o que ela não é
`lotacoes`	Pessoa × escola, datado. Professor e gestor **acumulam** — lotação nova não encerra a anterior. Estudante tem UMA ativa, e é o banco que garante: "estudante em duas escolas" é indistinguível de uma transferência que ficou pela metade.
Escola ativa	Estado de SESSÃO, não coluna. Quem tem mais de uma escolhe no cabeçalho, e a troca é validada contra as lotações *pela API* — nunca contra a lista que o *cookie* carrega.
`users.cpf`	A identidade IMUTÁVEL do estudante. Nome, *login* e matrícula mudam; o CPF não. É o que transforma "esse CPF já existe na escola B" em transferência reconhecida, em vez de duas crianças com o histórico partido. Guardado em 11 dígitos, único por contrato, e **exibido ao gestor** — é como ele confere identidade contra o sistema da rede. O que sustenta a decisão é a contenção: esta é a única coluna onde o número existe (**ADR-33**).
`escolas.codigo_inep` `municipio`, `uf`	Identificar a unidade numa rede de milhares. Sem município, a lista de escolas é impossível de navegar.
`escolas.tipo` `escolas.pai_id`	O nível REGIONAL — Diretoria de Ensino, polo, secretaria municipal. `tipo` tem vocabulário fechado (`escola`, `regional`, `rede`) e um *trigger* recusa `pai_id` que aponte para uma `escola` ou para a própria linha. **Agrupa, não autoriza**: serve para MONTAR a lista de unidades de uma diretoria — o insumo da lotação em lote — e nenhuma consulta de autorização a lê (**ADR-34**).

ATENÇÃO

Coluna que ninguém lê é dívida com cara de recurso. `tipo` e `pai_id` existiram por um tempo sem nenhuma consulta que as usasse — a mesma armadilha do `expires_at` sem rotina e do `turmas.consentimento_em` do aplicativo de origem. Ou a coluna nasce com o consumidor, ou não nasce.

DECISÃO DE PROJETO

Na transferência, o dado FICA e a identidade SEGUE (**ADR-32**). Tudo o que a criança produziu continua com o `escola_id` da escola onde produziu — verdade histórica que um *UPDATE* em massa apagaria. O que atravessa é a leitura: `GET /estudantes/:id/historico`, uma porta só, restrita a quem tem lotação na escola onde ela está hoje, devolvendo participações e nunca o conteúdo dos instrumentos.

> É também a única operação que troca o `app.escola_id` *no meio de uma transação*: sem isso o *update* na escola de origem não veria linha nenhuma, a RLS o esconderia sem erro, e a criança ficaria matriculada nas duas.

8.4 Enturmamento e identidade

Duas tabelas respondem "quem é a pessoa" e "em que turma ela está, desde quando". São isomórficas ao OneRoster 1.2 (`academicSessions` e `enrollments`) por decisão registrada em **ADR-28**, com nomes em pt-BR — o padrão em si só aparece na fronteira de importação.

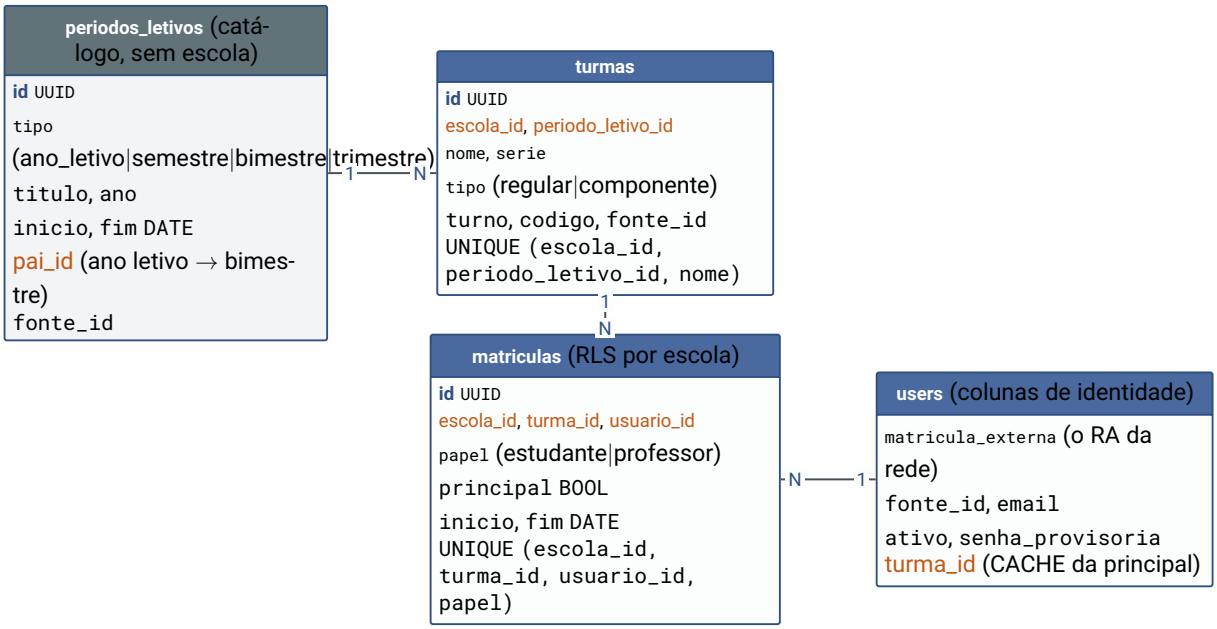


Figura 8.2 – Enturmamento. A verdade é `matriculas`; `users.turma_id` permanece como **cache** da matrícula principal, porque o *login* e o *claim* `turma_id` do *token* o leem antes de haver escola no contexto de tenancy.

Decisão de esquema	Por quê
`papel` só admite `estudante` e `professor`	Gestor é papel no **nível da escola** (o `roles.csv` do padrão), não matrícula de turma — e é por isso que ele continua enxergando a escola inteira enquanto o professor passa a ver só as turmas dele.
`fim` em vez de `DELETE`	Há dado de estudante pendurado em oito tabelas. Apagar a matrícula deixaria esse histórico órfão de turma; encerrar preserva "esteve no 8A até março".
`matriculas` tem RLS, `turmas` e `users` não	A exceção de identidade existe porque o *login* precisa achar a pessoa **antes** de saber a escola — e a matrícula nunca é lida nesse momento. *Armadilha*: consulta fora de `comTenancy` devolve lista **vazia**, sem erro.

Índice único **parcial** em `matricula_externa`	Duas pessoas sem RA não colidem entre si (`NULL` não é valor), mas o mesmo RA não se repete dentro de uma escola. O Prisma não modela índice parcial: ele vive na *migration*, como os *CHECKs* e a RLS.

8.5 Telemetria particionada

> **DECISÃO DE PROJETO**
>
> `telemetria_eventos` é **particionada por mês** (`PARTITION BY RANGE (ts)`), na migração `20260827100000_telemetria_particionada` (The PostgreSQL Global Development Group, 2026b). As partições vivem no *schema* `particoes`, **fora do alcance do Prisma** — em `public` cada partição viraria *drift* de *schema* a cada `prisma migrate dev`.

Três consequências do particionamento merecem registro:

1. **A chave primária é composta: `(event_id, ts)`.** O PostgreSQL exige a chave de partição em todo índice único. A *dedupe* do Pub/Sub segue correta, porque a reentrega da mesma mensagem traz o *mesmo* `ts` — então `ON CONFLICT (event_id, ts) DO NOTHING` continua sendo idempotência real, não aproximação.
2. **Há uma partição `DEFAULT`, como rede de segurança.** Sem ela, um evento com `ts` fora das faixas seria **recusado** — e telemetria perdida em silêncio é pior que telemetria mal guardada. As faixas declaradas vão até 2028-12.
3. **Expurgo passou a ser `DROP TABLE`.** Se um contrato exigir prazo de retenção, ele é cumprido por `DROP TABLE particoes.<mês>` — instantâneo — em vez de um `DELETE` que trava a tabela.

> **ATENÇÃO**
>
> A partição `DEFAULT` encher é **sinal de que faltou criar partição**, não funcionamento normal. Não há *job* agendado nesta plataforma — de propósito, porque seria a única peça de *scheduling* do sistema — e as faixas vão até 2028-12. Verificar a contagem da partição `DEFAULT` é item do *checklist* de operação (Apêndice F).

8.6 Modelo da exp-01 (Instanooria)

A exp-01 é a experiência mais rica em dados: um motor de *feed* adaptativo com geração de conteúdo por IA, sessão por estudante e dossiê de fechamento. São doze tabelas `exp_01_` mais a tabela de contrato com a exp-70.

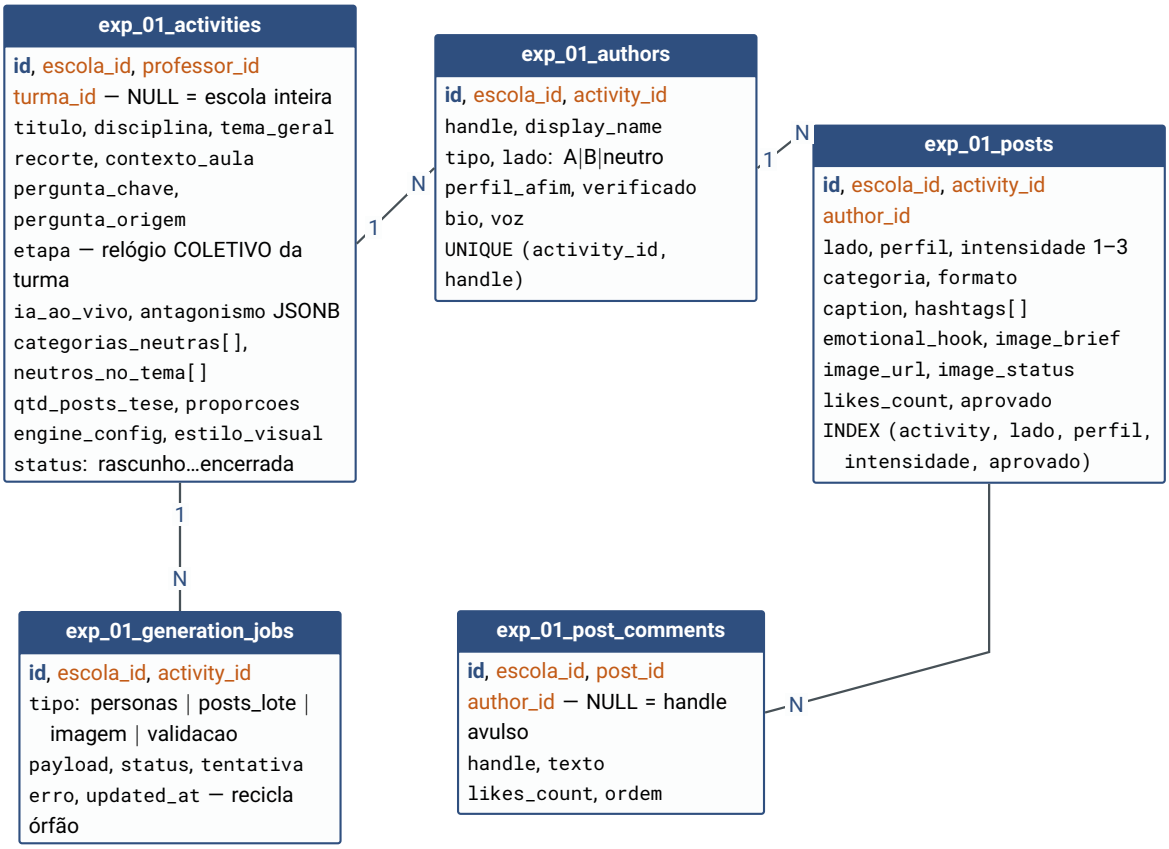


Figura 8.3 – Modelo de dados da exp-01, parte 1: a atividade preparada pelo professor e o conteúdo gerado. `lado`, `perfil` e `intensidade` em `exp_01_posts` são a variável independente do experimento — estão no banco, e o sanitizador da API os remove antes de o *feed* sair (Seção 5.8).

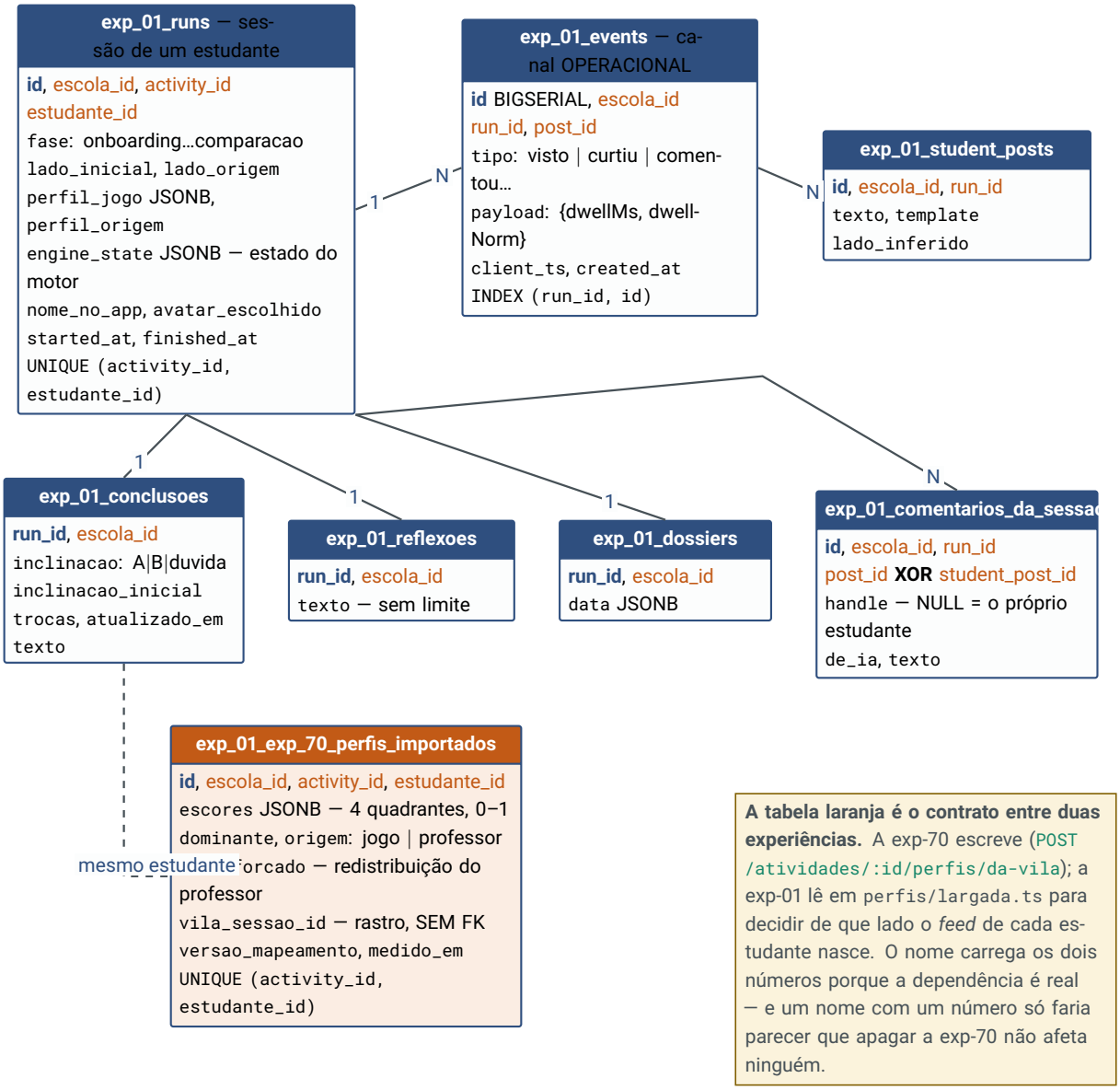


Figura 8.4 – Modelo de dados da exp-01, parte 2: a sessão do estudante. A `run` é o centro — carrega o estado do motor em JSONB e é a unidade de *lock* da curadoria. `exp_01_events` é o canal *operacional* de telemetria, síncrono e transacional, e não deve ser confundido com `telemetria_eventos`, o canal analítico (Capítulo 10).

Decisão de esquema	Por quê
`engine_state` em JSONB, não em tabela	O estado do motor é um agregado lido e escrito *inteiro*, sempre dentro do mesmo *lock*. Normalizá-lo daria *joins* sem nenhum ganho de consulta.
`nome_no_app` e `avatar_escolhido` na `run`, não em `users`	A conta é **do aparelho**: outra turma reusa o mesmo *login*. O nome que a criança digitou pertence àquela sessão.
`pergunta_origem`	Distingue pergunta gerada por IA de pergunta escrita ou editada pelo professor. É auditoria de autoria pedagógica, e custa uma coluna.

`lado_origem` na `run`	`perfil`, `sorteio` ou `redistribuido`: auditoria de *por que* aquele estudante caiu naquele lado. Sem isso, uma reclamação de professor não tem resposta.
`post_id` **XOR** `student_post_id` em `comentarios_da_sessao`	O comentário ancora num post do banco *ou* no post que o estudante publicou. Nunca nos dois — e o *CHECK* está na migração.
`de_ia`	Permite ao professor distinguir, na auditoria, o comentário escrito pela persona (modelo) do escrito pelo estudante.

8.7 Modelo das experiências 02, 70, 90 e 91

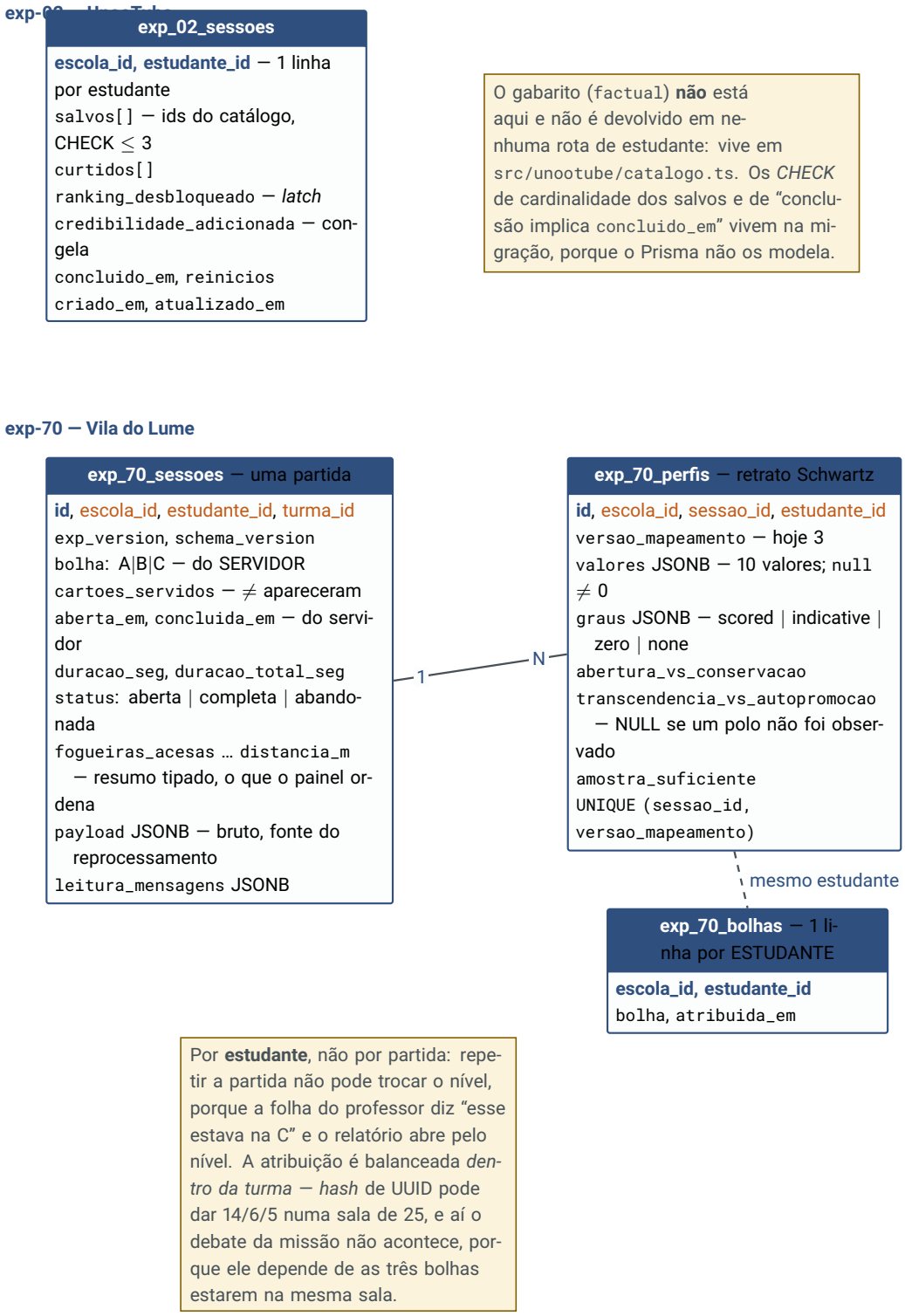


Figura 8.5 – Modelo de dados da exp-02 e da exp-70. Em `exp_70_perfis`, `null` **não é zero**: é valor que não recebeu número. `graus` diz por quê, e é essa distinção que impede um relatório de afirmar recusa onde houve ausência de oportunidade.

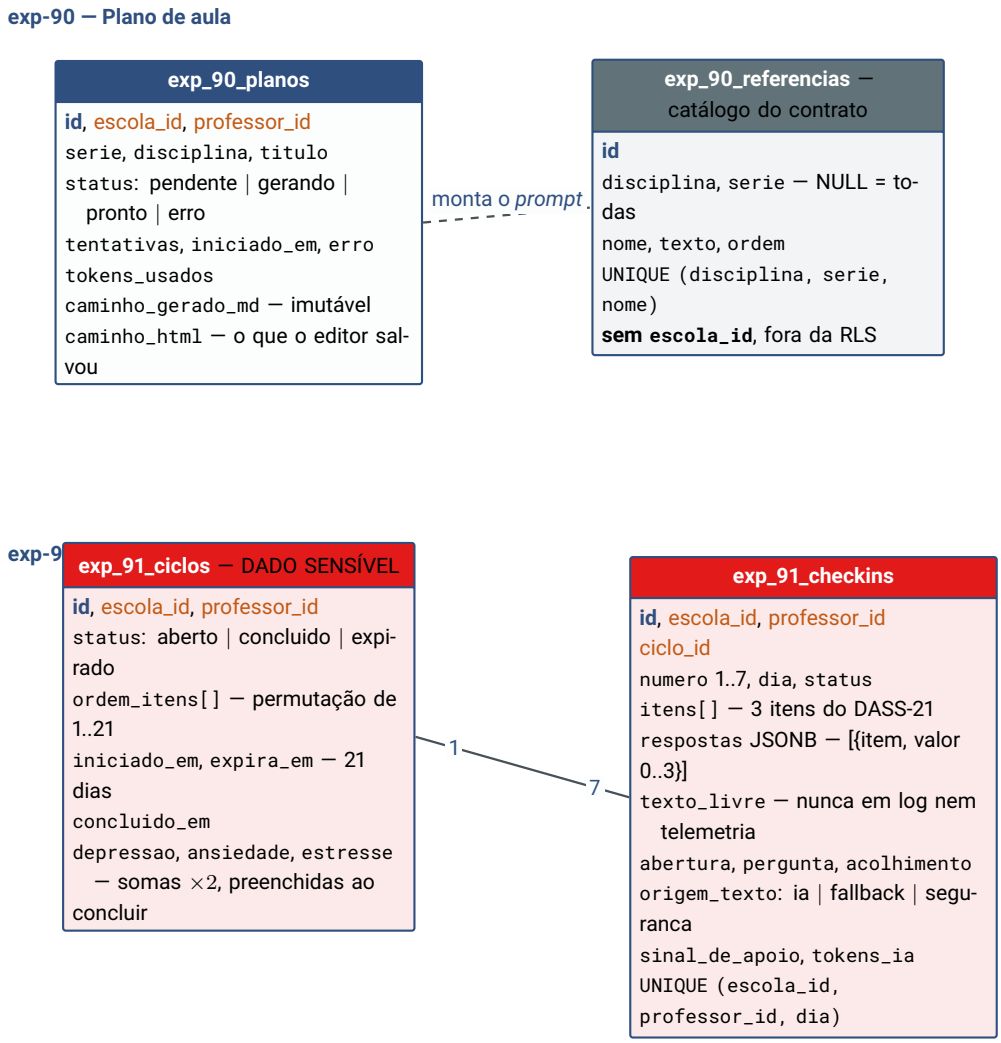


Figura 8.6 – Modelo de dados da exp-90 e da exp-91. As caixas vermelhas são dado *sensível* de saúde (LGPD art. 11): RLS *e* posse por `professor_id` em toda *query*, sem nenhuma rota de gestor ou agregado. `exp_90_referencias` é catálogo do contrato e fica **fora** da RLS, como `experiencias`.

8.8 As cinco tabelas da exp-03

Tabela	O que guarda
`exp_03_sessoes`	O estado da partida, uma linha por estudante. Chave `(escola_id, estudante_id)` — **não há id de sessão**: a identidade vem do *token*, e foi isso que dispensou a guarda de posse do projeto de origem, onde a sessão era um *uuid* na URL. Inclui `tokens_gastos` e `chamadas_ia` (**ADR-24**).
`exp_03_mensagens`	A **Transcrição**: a conversa entre o estudante e o agente. `texto` é caixa livre escrita por menor de idade — o dado mais sensível da experiência (**ADR-27**).
`exp_03_marcos`	A trilha da decisão: o que a criança fez, em que minuto da cidade. `dados` traz **só dado público** — nada de viés, Lacuna ou parâmetro oculto, porque este JSON chega ao navegador.

`exp_03_simulacoes`	O desfecho de cada tentativa, com `versao_prompt` gravada junto: sem ela não se sabe qual *prompt* produziu o que está em produção.
`exp_03_relogio_turma`	O Relógio da Aula, por **turma** (**ADR-25**). Chave `(escola_id, turma_id)`.

REGRA INVIOLÁVEL

Os filhos apontam para o **par** `(escola_id, estudante_id)`, não para um id de sessão. Cada linha filha carrega `escola_id` por construção — que é o que a RLS precisa — e nenhuma consulta nova pode esquecê-lo.

Cinco colunas do projeto de origem não vieram, porque estavam mortas lá, e foram verificadas uma a uma antes do descarte: `simulacoes_restantes` e `custo_cents` (zero leituras), `camadas_reveladas` e `pergunta_id` (só escritas vazias) e `evidencias` (só escrita, e o que ela guardava já está em `rodadas`). Centavo virou *token*: preço depende de tabela do *vendor*, que muda sem aviso e não é conhecida pela aplicação.

Nenhuma coluna de prazo (**ADR-26**). E 18 *CHECKs* de estado impossível na migração, entre eles: justificativa sem destino, encerrar relógio que nunca começou, duração de relógio igual a zero (dividiria por zero no minuto narrativo) e `debitou` numa fala que não é do estudante.

ATENÇÃO

As duas tabelas com `BIGSERIAL` (`exp_03_mensagens` e `exp_03_marcos`) exigem `GRANT USAGE` na **sequência**, além do *grant* de tabela. Sem ele o `INSERT` falha para a *role* de *runtime* mesmo com todos os privilégios de tabela concedidos.

8.9 Decisões de esquema por experiência

8.9.1 exp-70 — as decisões que custam informação de propósito

Decisão	Por quê
`turma_id` desnormalizado na sessão	O estudante troca de turma entre anos letivos, e o resultado precisa continuar *na turma em que foi medido*.
Datas cravadas pelo **servidor**	O aplicativo de origem derivava do relógio do cliente, e documentava que a hora numa máquina de escola costuma estar errada.
O `id` da sessão é emitido pela API	No aplicativo de origem o cliente escolhia a própria chave de idempotência, e duas crianças na mesma máquina colidiam — a segunda partida voltava "duplicada" e desaparecia.

`valores` com `null`, e `graus` separados	`null` **não é zero**: é valor que não recebeu número, porque tem uma fonte de medida só ou porque a oportunidade não chegou. `graus` diz *por quê*, e é essa distinção que impede um relatório de afirmar recusa onde houve ausência.
Eixos do circumplexo anuláveis	Eixo com polo não observado não é zero: é *indefinido*. Colocar zero produziria um ponto no meio do gráfico, que se lê como "equilibrado".
`amostra_suficiente` como coluna	Falso ⇒ o painel abre avisando que **não sabe**. Não é rigor estatístico: é impedir que uma partida sem escolha vire uma frase sobre quem a criança é.
Nova linha por `versao_mapeamento`	O mapeamento é hipótese de *design* e vai mudar. Guardar só o resultado tornaria todo o passado incomparável no dia da revisão. `UNIQUE (sessao_id, versao_mapeamento)`.
Sem tabela de evento fino	O bruto vive em `payload` (fonte do reprocessamento) e o canal analítico da SDK já grava em `telemetria_eventos`. Duas tabelas de log seriam duas histórias de retenção para o mesmo dado.

8.9.2 exp-90 — conteúdo fora do banco

> **DECISÃO DE PROJETO**
>
> O **conteúdo** do plano de aula não fica no PostgreSQL: fica em armazenamento de objetos *privado* (`planos/{escolaId}/{planoId}/gerado.md` imutável + `atual.html` do editor). A tabela guarda metadados, estado da geração e *caminhos*.
> O ganho não é performance: é que o versionamento de objetos do *bucket* dá histórico de cada "Salvar" **sem uma linha de código de histórico**. Um professor que sobrescreveu o plano por engano tem recuperação, e ninguém escreveu uma tabela de versões.

8.9.3 exp-91 — ausência como decisão de esquema

> **DECISÃO DE PROJETO**
>
> Não existe tabela de agregado de bem-estar, e não existe *view* por escola. Isso não é lacuna: é a decisão. Dado de saúde (LGPD art. 11) tem RLS e posse por `professor_id` em toda *query*; o titular apaga tudo com `DELETE /bemestar/dados`. Uma visão agregada e anonimizada para a rede é decisão *futura*, com DPA e *k*-anonimato — e não sai de um `GROUP BY`.

8.10 Migrações

Migração	Conteúdo

`20260824145301_init`	Esquema inicial: *tenancy* (`escolas`, `turmas`, `users`), tabelas da exp-01, tabelas de plataforma, e o laço que aplica `ENABLE`/`FORCE ROW LEVEL SECURITY` + política por escola.
`20260825011950_unootube_sessoe`	Estado *server-side* da exp-02, com *CHECK* de cardinalidade dos salvos.
`20260825120000_planos_de_aula`	Planos e referências curriculares.
`20260825225016_bem_estar`	Ciclos e *check-ins* do DASS-21, com RLS e `GRANT`.
`20260826120000_permissoes_de_if0alua`	Coluna `permissoes` no manifesto, com *CHECK* do vocabulário aceito.
`20260826140000_vila_do_lume`	Sessões, perfis e bolhas da exp-70.
`20260826170000_vila_sem_prazo`	Remoção da coluna `expires_at` (**ADR-13**).
`20260827090000_tabelas_por_exp`	Renomeação para o padrão `exp_NN_`: `RENAME` de tabelas, *constraints*, índices e políticas. É a migração de referência.
`20260827100000_telemetria_particionada`	Particionamento mensal de `telemetria_eventos` no *schema* `particoes`, com partição `DEFAULT` e faixas até 2028-12.
`20260828093000_enturmamento_e_`	`periodos_letivos`, `matriculas` (com RLS), colunas de identidade em `users` e o *backfill* de `users.turma_id` para matrícula principal.
`20260828140000_rede_lotacoes_e_bolsas`	`lotacoes` e o índice único parcial que impede estudante ativo em duas escolas.
`20260828170000_cpf_em_claro`	Troca de `cpf_hash` por `users.cpf` (11 dígitos, *CHECK* e único parcial) — **ADR-33**.
`20260828200000_nivel_regional`	Vocabulário `escola` | `regional` | `rede` em `escolas.tipo` (`distrito` renomeado), o *trigger* que exige que o pai AGRUPE, e índices em `pai_id`/`tipo` — **ADR-34**.
`20260828230000_perfil_portatil`	`perfis_portateis`, com RLS, *CHECK* do vocabulário de `verificacao` e único por (escola, estudante, emissor, emissão) — **ADR-35**.

REGRA INVIOLÁVEL

Nenhuma migração remove `escola_id` ou afrouxa RLS. Renomeação usa `RENAME`; nada de `DROP`/`CREATE` em tabela com dado. Em produção, a migração roda como Cloud Run Job dentro da VPC **antes** do *deploy* do serviço, com `--wait` — porque o *schema* novo tem de estar no banco antes de a revisão nova receber tráfego.

ATENÇÃO

O que o Prisma **não** modela e por isso vive na migração: políticas de RLS, *CHECK constraints*, partições, `GRANT`. Ler apenas `schema.prisma` dá uma visão incompleta do banco —

> é por isso que os campos correspondentes são `TEXT`/`SMALLINT` no *schema* e a validação de aplicação é feita com Zod.

9

Contratos de integração

Há quatro fronteiras nesta plataforma cuja quebra é caríssima, porque atinge código que já está publicado e não pode ser atualizado em conjunto: o protocolo `postMessage` entre hub e experiência, o formato do evento de telemetria, o manifesto de experiências e a interface do *provider* de IA. Este capítulo especifica os quatro e a política de versionamento de cada um.

> **REGRA INVIOLÁVEL**
>
> **Mudança em contrato exige cuidado extremo e atualização deste documento.** Um *bundle* publicado há três meses no *bucket* de um contrato continua sendo servido a estudantes: ele não é reconstruído quando a API muda. Toda alteração de contrato é, por definição, uma alteração *retrocompatível* ou um *bump* de versão maior com suporte à anterior.

9.1 Protocolo `postMessage` hub ↔ experiência

9.1.1 Especificação das mensagens (protocolo v1)

Sentido	Mensagem	Carga e semântica
exp → hub	`exp:ready`	`{ protocolVersion }`. Anúncio de que o *bundle* carregou. É **reenviado** a cada 400 ms até o hub responder.
hub → exp	`hub:init`	`{ protocolVersion, token, estudante{id,nome,serie,turmaId,escolaId}, apiUrl, config }`. Resposta ao `exp:ready`; só a primeira conta. `config` é o saco aditivo do protocolo: hoje carrega `expId` e `clarityProjectId` (**ADR-60**) — campo novo aqui não muda `PROTOCOL_VERSION`, porque a experiência que não o conhece o ignora.
exp → hub	`exp:concluir`	`{ payload: { score?, dados? } }`. Conclusão da missão. O hub registra progresso.
exp → hub	`exp:token-expirando`	Sem carga. Pedido de renovação; o hub responde `hub:token`.
hub → exp	`hub:token`	`{ token }`. Novo *token*, sem recarregar o `iframe`.

exp → hub	`exp:estado`	`{ estado: {…} }`. Estado de alto nível para o *chrome* opcional do hub. **Aditivo** ao v1; o hub *não* interpreta domínio, só repassa a quem plugou um *callback*.
hub → exp	`hub:pause` / `hub:resume`	Sem carga. Emitidos por visibilidade e foco da página do hub.

NOTA — o que o hub faz com o `exp:estado`

O `ExpHost` guarda o último estado publicado e reconhece dois campos, ambos da exp-01: `acao: "sair"` (a sessão é do HUB, em *cookie* `httpOnly` fora do alcance do *iframe*, então a experiência pede e o hub executa) e `fase`, que acende a moldura dos posts perdidos e — com `passo: "relatorio"` — solta o aparelho para o relatório ocupar a área inteira (ADR-41, 18.42).

Duas consequências de o hub SUBSTITUIR o estado inteiro a cada mensagem: publicar só o `passo` apagaria a `fase`, então quem publica leva as duas; e é a própria troca de fase seguinte que desfaz o estado anterior, sem mensagem de "desligar". As outras experiências publicam estado (`{ tela }` na exp-02, `{ passo, tentativa }` na exp-03) e o hub ignora — o canal é genérico de propósito.

9.1.2 O *handshake*, passo a passo

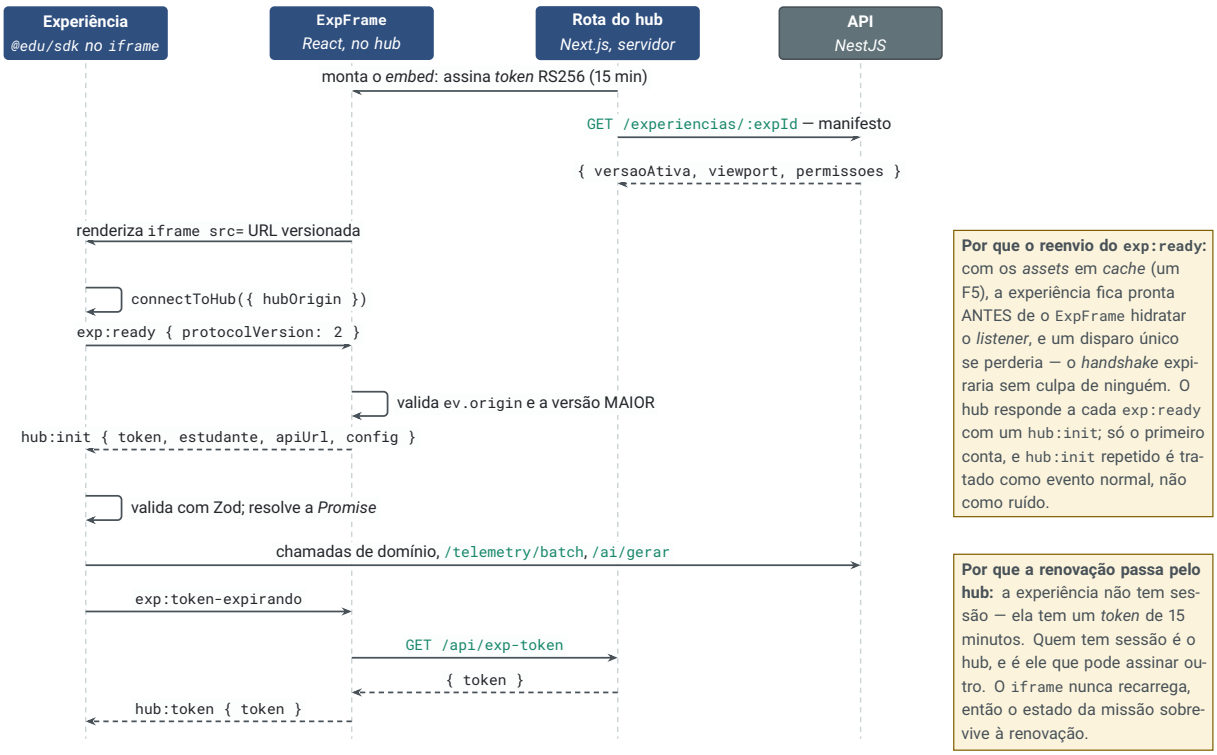


Figura 9.1 – Sequência do *handshake* e da renovação de *token*. O `targetOrigin` é explícito nos dois sentidos, e as duas pontas descartam mensagem de *origin* divergente antes de olhar o conteúdo.

9.1.3 Regras invioláveis do protocolo

REGRA INVIOLÁVEL

1. **`targetOrigin` explícito, nunca `"*"`.** O hub usa a *origin* derivada da *mesma* URL que colocou no `iframe`; a SDK usa o `hubOrigin` que recebeu por *query string*. Com `"*"`, o *token* iria para qualquer documento que conseguisse virar o pai do *frame*.
2. **Aquela *query string* é entrada NÃO CONFIÁVEL, e passa por `normalizarHubOrigin()`.** O *bundle* é público: quem embeda a experiência num `iframe` próprio escreve o parâmetro, e com ele escolheria de quem se aceita o `hub:init` e para onde vai tudo o que sai depois. A função reduz o valor a uma origem canônica, recusa esquema fora de http(s), credencial e origem opaca, exige hub em https quando a experiência está em https, e confere contra `location.ancestorOrigins[0]` — a origem REAL de quem embedou, que VENCE o parâmetro (**ADR-46**).
3. **Toda mensagem pós-*init* é validada com Zod nos DOIS lados.** Mensagem inválida é descartada com aviso, nunca despachada crua aos *listeners*.
4. **Versão MAIOR verificada nos dois lados; incompatível falha alto.** O hub não envia `hub:init` e registra erro no console; a experiência expira o *handshake* e mostra a própria mensagem de erro.
5. **O *token* vive só em memória** na experiência. Nunca `localStorage`, nunca *cookie*.
6. **Nunca quebrar compatibilidade sem *bump* de versão maior e suporte à anterior.**

Listing 9.1 – O núcleo da validação, em `packages/sdk/src/bridge.ts`

```
function handler(ev: MessageEvent) {
  if (ev.origin !== hubOrigin) return;           // regra 1: valida origin

  if (!conectado) {
    const init = HubInitSchema.safeParse(ev.data);
    if (!init.success) return;                   // antes do init, tudo mais e ruido
    // Versao MAIOR precisa bater — hub e experiencia de geracoes diferentes
    // falham ALTO, nao silenciosamente.
    if (Math.trunc(init.data.protocolVersion) !== PROTOCOL_VERSION) {
      reject(new Error(`Protocolo incompativel: hub v${init.data.protocolVersion}, ` +
                       `experiencia v${PROTOCOL_VERSION}`));
      return;
    }
    conectado = true;
    resolve({ init: init.data, send, onMessage });
    return;
  }

  // hub:init repetido (resposta a um exp:ready reenviado) nao e ruido nem erro.
  if ((ev.data as { type?: unknown })?.type === "hub:init") return;

  const msg = HubParaExpSchema.safeParse(ev.data);
  if (!msg.success) {
    console.warn("[sdk] mensagem do hub descartada (schema):", ev.data?.type);
    return;                                      // regra 2: descarta, nao despacha
  }
```

```
  listeners.forEach((cb) => cb(msg.data));
}
```

9.1.4 Política de evolução

Tipo de mudança	Versão	Regra
Nova mensagem opcional (como `exp:estado`)	v1, aditivo	Permitido sem *bump*: hub antigo ignora mensagem que não conhece, e experiência antiga nunca recebe a nova sem pedir.
Novo campo opcional em `hub:init`	v1, aditivo	Permitido. O esquema Zod tem `.default()` ou `.optional()`, e a experiência antiga simplesmente não o lê.
Campo existente muda de tipo ou semântica	**v2**	Exige *bump* de versão maior *e* que o hub continue respondendo v1 enquanto houver *bundle* v1 publicado em algum contrato.
Remoção de mensagem ou de campo	**v2**	Mesma regra. A remoção só é efetiva quando o último *bundle* v1 tiver sido republicado.

> **ATENÇÃO**
>
> `hub:pause` e `hub:resume` existem porque **esconder um `iframe` não dispara `visibilitychange` nele**. Sem essas mensagens, o cronômetro de *dwell* da experiência correria com o estudante de costas — e *dwell* é a medida central do motor de *feed* da exp-01. O hub escuta `visibilitychange` e `blur`/`focus` da janela, porque trocar de janela ou de aplicativo não muda `visibilityState`.
>
> **Uma exceção, e ela custou um bug:** clicar em qualquer lugar **dentro** do `iframe` move o foco para a experiência, e o navegador dispara `blur` no documento de cima. Isso *não* é o estudante saindo — e como o `focus` correspondente não volta enquanto ele continua ali, um `hub:pause` nesse caso é definitivo. Na exp-02 o vídeo parava no primeiro toque em "Salvar" e não voltava mais. Quando o foco foi para a nossa experiência, `document.activeElement` é o próprio `iframe`, e `ExpFrame.tsx` não emite nada.

9.2 Manifesto de experiências

O manifesto é o contrato entre o CI, a API e o hub. Ele existe em duas cópias que devem concordar: o arquivo `public/experiencia.json` dentro do *bundle*, e a linha da tabela `experiencias` na API — que o CI escreve lendo o arquivo.

Listing 9.2 – `apps/experiencias/exp-70-vila-do-lume/public/experiencia.json`

```
{
  "id": "exp-70-vila-do-lume",
```

```
  "nome": "Vila do Lume — A Noite Longa",
  "versao": "1.0.0",
  "series": [],
  "capabilities": ["telemetria"],
  "permissoes": ["pointer-lock"],
  "verbos": {
    "attempted":  "entrou na vila (partida aberta)",
    "interacted": "acendeu fogueira, gastou tocha, gravou na Pedra dos Nomes ou respondeu a uma mensagem",
    "progressed": "acendeu mais uma das 8 fogueiras",
    "completed":  "acendeu as 8 fogueiras e reacendeu o Fogo Sagrado"
  }
}
```

Campo	Tipo	Semântica
`id`	*string*	Slug da experiência; é a chave primária da tabela e o primeiro segmento do caminho no *bucket*.
`nome`	*string*	Nome de exibição.
`versao`	*semver*	Vem do `package.json` da experiência. É o segundo segmento do caminho no *bucket*, o que torna o *bundle* imutável.
`series`	*string[]*	Séries em que a experiência se aplica. **Vazio significa TODAS as séries**, não nenhuma — ver o alerta abaixo.
`capabilities`	*string[]*	`telemetria`, `progresso`, `ia`, `avaliacao`, `tempo-real`. Declara o que a experiência usa da plataforma.
`viewport`	`{largura, altura}?`	Presente ⇒ o hub embeda dentro do `DispositivoFrame` (casca de aparelho com escala). A exp-01 é 375×812; a exp-02 *não* tem `viewport`, porque foi desenhada para *desktop*.
`publico`	`estudante|professor`	`professor` ⇒ ferramenta do professor, e `/exp/*` responde **404** para ela.
`permissoes`	*string[]*	Permissões que o *iframe* daquela experiência precisa. Hoje só `pointer-lock`. Vazio (o *default*) = nenhuma.
`verbos`	*objeto*	Documentação dos verbos de telemetria que a experiência emite. Não é lido por código; é contrato de leitura para quem analisa os dados.

REGRA INVIOLÁVEL

`series: []` significa **todas as séries**, não nenhuma: é assim que uma experiência *transversal* se declara (é o caso da exp-70, que não pertence a ano nenhum). Quando o filtro por série voltar a valer no hub do estudante, vazio tem de continuar querendo dizer "todas" — o contrário faz a experiência desaparecer da tela sozinha, sem erro em lugar nenhum.

9.2.1 Como o hub resolve a URL do `iframe`

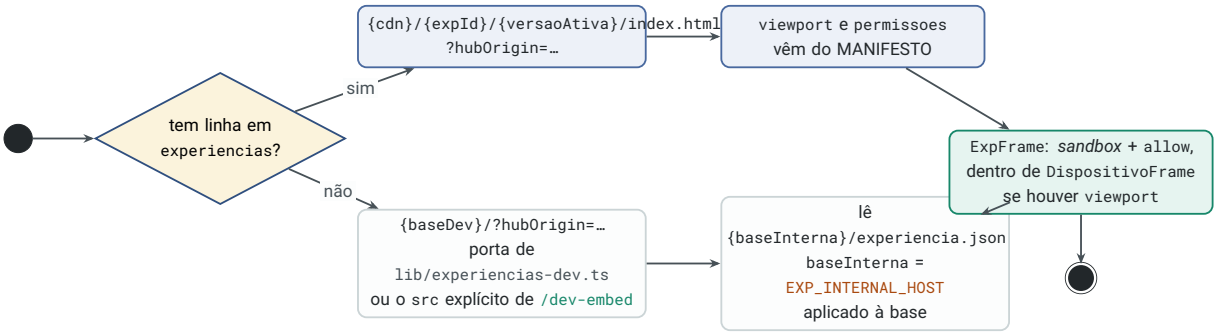


Figura 9.2 – Resolução da URL do *embed* e da casca de aparelho. Os dois caminhos convergem: mesmo sem manifesto, o hub obtém `viewport` e `permissoes` lendo o `experiencia.json` servido ao lado do `index.html` do *bundle*.

ATENÇÃO

A leitura do `experiencia.json` do *bundle* **não é só de desenvolvimento**. Antes era, e uma *stack* com a tabela `experiencias` vazia — *compose* local, ou ambiente novo antes do primeiro *publish* do CI — servia a exp-01 *esticada na janela* em vez de dentro do aparelho de 375×812. Pelo mesmo motivo, a exp-70 abriria sem `allow-pointer-lock`: o jogo monta, a câmera não anda, e nada no console diz por quê.

NOTA

Essa leitura é *server-side*, e `NEXT_PUBLIC_CDN_EXP_URL` é o endereço do **navegador** — de dentro de um contêiner, `localhost:5173` é o próprio contêiner. Daí as duas variáveis: `EXP_INTERNAL_HOST` troca só o *host* e **preserva a porta** (é o que serve em desenvolvimento, onde cada experiência tem um *dev server* numa porta diferente), e `EXP_INTERNAL_URL` é um endereço fixo, que só serve quando há *uma* origem de *bundles* — a CDN, em produção.

9.2.2 Publicação e reversão

Operação	Comando
Publicar (feito pelo CI)	`gcloud storage rsync -r {dir}/dist gs://{bucket}/{exp}/{ver}/` + `node apps/hub/scripts/publicar-manifesto.mjs {dir}`
Reverter	`node apps/hub/scripts/publicar-manifesto.mjs <exp> --versao x.y.z`

O *script* assina um *token* de cinco minutos com a chave de assinatura e chama `PUT /experiencias/:expId` como `admin-contrato`. A API valida esse *token* contra o JWKS que o *hub* publica — o que é a razão de o *workflow* publicar as experiências *depois* do hub.

9.3 Contrato de telemetria

O formato do evento é inspirado em xAPI (Advanced Distributed Learning Initiative, 2023) e simplificado ao que a plataforma efetivamente usa.

Listing 9.3 – Evento de telemetria (canal analítico)

```
{
  "eventId":    "uuid",          // gerado pela SDK; chave de dedupe
  "ts":         "ISO-8601",      // relogio do cliente
  "estudanteId":    "...",           // SOBRESCRITO pelo servidor, a partir do token
  "escolaId":   "...",           // SOBRESCRITO pelo servidor
  "expId":      "exp-01",
  "expVersion": "1.2.0",
  "sessaoId":   "uuid",
  "verbo":      "completed | attempted | interacted | scored | progressed | ...",
  "objeto":     "fase-3",
  "resultado":  { },             // livre, por experiencia
  "contexto":   { }              // livre, por experiencia
}
```

Cláusula	Regra
Quem preenche o quê	A SDK preenche tudo **exceto** `verbo`, `objeto` e `resultado` — esses são a única coisa que a experiência informa.
Identidade	`estudanteId` e `escolaId` enviados pelo cliente são **descartados**: o servidor os deriva do *token*. É anti-*spoof* e, ao mesmo tempo, garantia de que o cliente não *pode* colocar PII no lugar do id.
Lote	Até 50 eventos por requisição, *flush* a cada 10 s ou quando o *buffer* enche.
Fechamento de página	No `pagehide`, `POST /telemetry/beacon` com o *token* **no corpo** — `sendBeacon` não envia cabeçalho.
Idempotência	`ON CONFLICT (event_id, ts) DO NOTHING`. O Pub/Sub entrega *at least once*, então reentrega é rotina, não exceção.
Documentação dos verbos	Cada experiência declara seus `verbos` no manifesto e no próprio README. Verbo não documentado é dado que ninguém consegue interpretar seis meses depois.

O detalhamento dos dois canais — por que são dois, e o que acontece em cada um — é o Capítulo 10.

9.4 Contrato do *provider* de IA

Listing 9.4 – `apps/api/src/ai/provider.ts` — a interface neutra de *vendor*

```
export interface AiRequest {
  system: string;
  user: string;
  maxTokens?: number;
```

```ts
  /** Familia de modelo, NEUTRA de vendor. `padrao` e o modelo do dia a dia;
   *  `longo` e o de saida grande (planos anuais). Quem chama nao sabe qual e. */
  modelo?: "padrao" | "longo";
  temperatura?: number;
}

export interface AiResult {
  texto: string;
  tokensUsados: number;
  /** `STOP` (natural) ou outro motivo do vendor (`MAX_TOKENS`, `SAFETY`...).
   *  Aditivo: quem nao liga, ignora. Quem gera texto longo TEM de olhar. */
  finishReason?: string;
}

export interface AiFerramenta {
  nome: string;
  descricao: string;
  /** Reusa o AiJsonSchema neutro. `description` por PROPRIEDADE nao e
   *  comentario: e por ela que o modelo descobre o que pode mandar. */
  parametros: AiJsonSchema;
}

export interface AiConversaRequest {
  system: string;
  historico: Array<{ papel: "estudante" | "agente"; texto: string }>;
  ferramentas?: {
    declaracoes: AiFerramenta[];
    executar: (nome: string, args: Record<string, unknown>) => Promise<unknown>;
  };
  /** Motivos para RECUSAR a fala. Motivo derruba a tentativa e o modelo tenta
   *  de novo sabendo por que. As regras sao do dominio; o laco e do provider. */
  validar?: (texto: string) => string[];
  socorro?: string;              // o que pedir na ULTIMA tentativa
  maxVoltas?: number;            // teto de idas e voltas de ferramenta (6)
  tentativas?: number;           // default 3
}

export interface AiProvider {
  generate(req: AiRequest): Promise<AiResult>;
  /** Saida JSON estruturada, com retry e reforco do erro anterior no prompt. */
  generateJson<T>(req: AiJsonRequest<T>): Promise<T>;
  /** Bytes originais (jpeg/png). Falha => quem chama cai no fallback procedural. */
  generateImage(prompt: string): Promise<Buffer>;
  /** Conversa multi-turno com ferramentas (ADR-23). `tokensUsados` soma TODAS
   *  as voltas e TODAS as tentativas, inclusive as rejeitadas: e o que foi pago. */
  conversar(req: AiConversaRequest): Promise<AiResult>;
}

export const AI_PROVIDER = "AI_PROVIDER";   // token de injecao do Nest
```

REGRA INVIOLÁVEL

Nenhum módulo fora de `src/ai` conhece o *vendor*. Trocar de provedor é criar um arquivo novo (`src/ai/<vendor>.provider.ts`) e registrá-lo no módulo por `AI_PROVIDER`. Implementações atuais: `vertex.provider.ts` (*default*) e `mock.provider.ts` (testes e desenvolvimento sem GCP).

NOTA

`AiJsonSchema` é um subconjunto deliberadamente pequeno de JSON Schema (`object`, `array`, `string`, `number`, `boolean`, `enum`), e cada *provider* o traduz para o formato do seu modelo. O campo `validar` do `AiJsonRequest` recebe a saída bruta e *lança* quando ela não serve — o que faz a tentativa ser regerada com o erro anterior reforçado no *prompt*. Validação como parte do contrato, não como verificação posterior.

9.5 Contrato entre duas experiências

Quando uma experiência precisa do resultado de outra, a comunicação nunca é direta — módulos de domínio não se conhecem (Seção 5.5). O contrato é uma tabela nomeada com os dois números e um *endpoint* de fronteira.

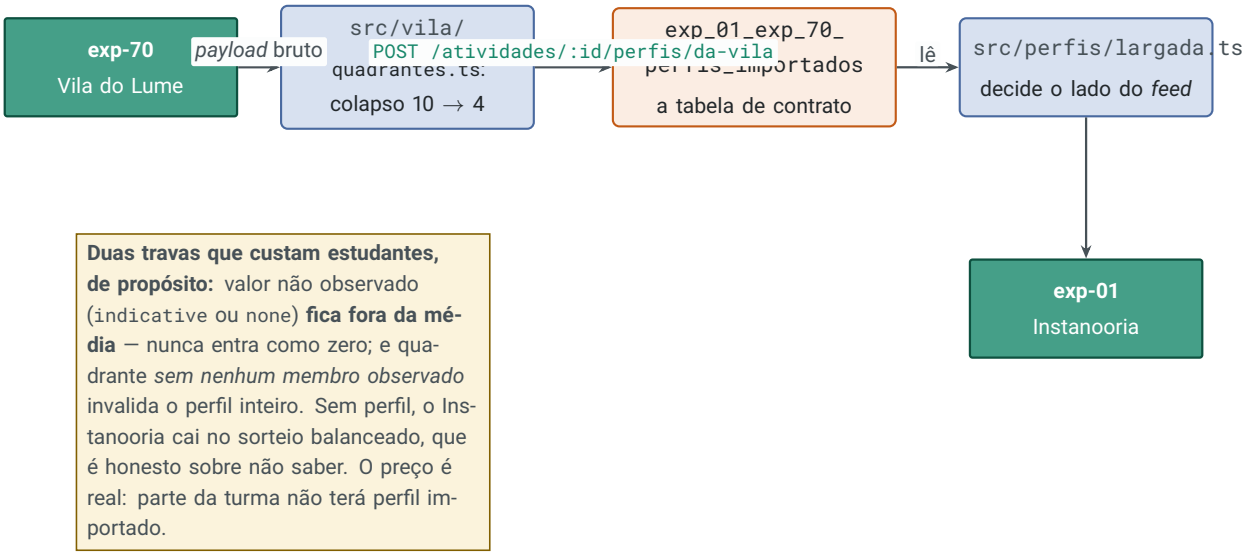


Figura 9.3 – O único acoplamento entre experiências, e como ele é explícito: uma tabela cujo nome carrega os dois números, escrita por um *endpoint* de fronteira acionado pelo professor.

REGRA INVIOLÁVEL

O disparo é do **professor**, e `lado_forcado` é **preservado** no *upsert*. Importar sozinho — ou levar o objeto inteiro no `update` — desfaria em silêncio uma redistribuição de turma que o professor fez antes da aula. `PERFIL_IMPORT_TOKEN` e `POST /perfis/importar` continuam vivos apenas como compatibilidade com a ponte externa original.

10

Telemetria

DECISÃO DE PROJETO

Dois canais de telemetria, não um. A plataforma tem dois fluxos de evento com garantias, latências e destinos diferentes, e confundi-los quebra um dos dois:

- o canal **operacional** (motor), *síncrono e transacional*, lido em tempo quase real para curar o próximo bloco de *feed*;
- o canal **analítico** (SDK), *assíncrono e em lote*, para análise posterior.

10.1 Por que dois canais

Aspecto	Operacional (motor)	Analítico (SDK)
Rota	`POST /runs/:id/eventos`, `GET /runs/:id/feed`	`POST /telemetry/batch`, `/beacon`, `/ingest`
Transporte	HTTP direto, dentro da transação	HTTP → Pub/Sub → *push* → HTTP
Latência	Milissegundos; a próxima tela depende dela	Segundos a minutos; ninguém espera
Ordenação	**Garantida** pela transação e pelo *lock* da `run`	Sem garantia
Destino	`exp_01_events`	`telemetria_eventos` (particionada)
Entrega	Exatamente uma vez (transacional)	*At least once*; *dedupe* por `(event_id, ts)`

Quem lê	O motor de curadoria, o painel do professor, o dossiê	Ninguém, hoje — é dado de análise futura

REGRA INVIOLÁVEL

***Dwell* NUNCA passa pelo Pub/Sub.** A latência e a falta de ordenação quebrariam a adaptação do *feed* e o *lock* por `run`: o motor precisa saber, no instante em que cura o próximo bloco, exatamente quais posts o estudante viu e por quanto tempo. Um evento que chega dois minutos depois, fora de ordem, produziria uma curadoria baseada em passado errado — e o estudante não teria como perceber.

NOTA

A recíproca também vale, e é o que justifica o segundo canal: mandar *tudo* pelo caminho transacional colocaria, no pico de quatro mil simultâneos, um `INSERT` sincronizado por interação de estudante dentro da mesma transação que serve a tela. O canal analítico existe para absorver volume sem competir com o que o estudante está esperando.

10.2 O canal analítico, ponta a ponta

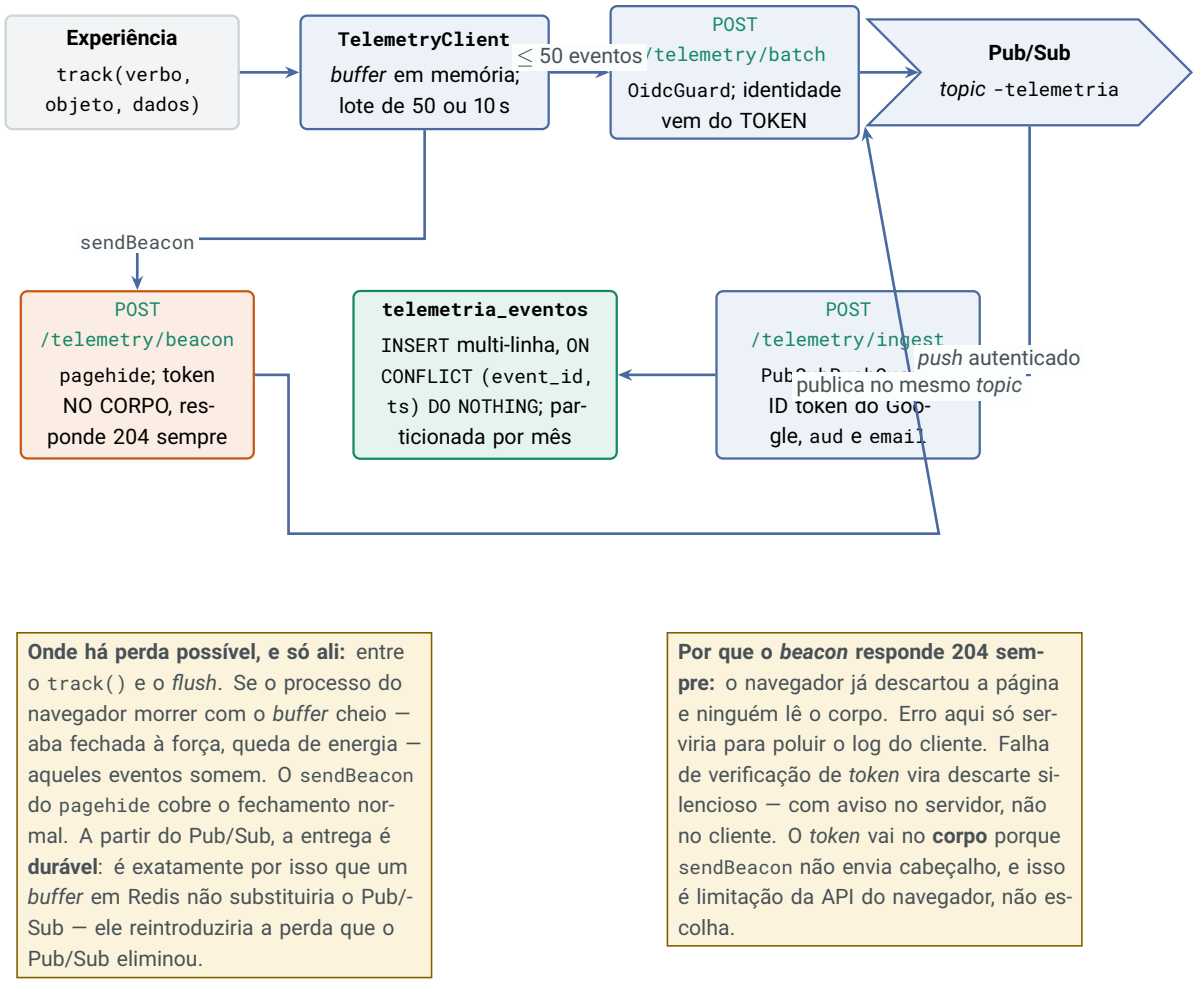


Figura 10.1 – O canal analítico ponta a ponta. Os únicos eventos que podem se perder são os que estão no *buffer* em memória do navegador no instante em que o processo morre; do Pub/Sub em diante a entrega é durável.

10.3 A rota de ingestão e sua tranca

`POST /telemetry/ingest` é a rota mais delicada da API: ela grava dado de estudante, o `escolaId` vem no corpo (já enriquecido pelo `/batch`) e ela **não passa pelo `OidcGuard`** — porque quem chama é o Pub/Sub, não um usuário, e não existe *token* do hub nessa chamada.

REGRA INVIOLÁVEL

Sem `PubSubPushGuard`, a rota seria um *endpoint* público que escreve telemetria falsa em qualquer escola. O *guard* verifica **três** coisas no ID token que o Pub/Sub assina:

1. **emissor Google** (`accounts.google.com`), pelo JWKS público;
2. `aud` igual a `TELEMETRIA_PUSH_AUDIENCE`;
3. `email` igual a `TELEMETRIA_PUSH_SA`.

As três juntas é que fazem a prova. A assinatura sozinha só diz “algum projeto GCP”, que não é ninguém em particular.
Falha fechada: sem as duas variáveis, a rota rejeita tudo com 401 — nunca fica aberta. O Terraform casa os dois lados: a *subscription* é criada com `oidc_token` dessa mesma conta de serviço e audiência.

DECISÃO DE PROJETO

***Push subscription* para a própria API, e não um *worker* dedicado.** O desenho original previa `{contrato}-telemetry-worker` como serviço próprio. A *push subscription* entrega o mesmo resultado com uma peça a menos para operar — um serviço a menos com *deploy*, log, escala e *alerting* próprios. O *worker* separado volta se o volume justificar isolar o consumo do tráfego de estudante.

10.4 Destino: PostgreSQL, não BigQuery

DECISÃO DE PROJETO

O canal analítico grava no mesmo PostgreSQL (decisão de 2026-08-27, **ADR-14**).
Avaliou-se mandar o canal analítico para o BigQuery por *subscription* nativa do Pub/Sub. Não foi, por dois motivos concretos: (a) os relatórios do professor **não leem** `telemetria_eventos` — eles leem `exp_01_events`, o canal síncrono do motor —, então o BigQuery não resolvia problema nenhum *hoje*; e (b) criava uma segunda plataforma de dados para operar, com IAM, custo e ferramentas próprios.
Ficando no mesmo banco, se um relatório precisar desse dado um dia, é um `JOIN` local com `turmas` e `users` — e é uma plataforma de dados a menos.

O que sustenta essa escolha na escala alvo são duas medidas, e nenhuma delas é um serviço novo:

10.4.1 `INSERT` multi-linha

Listing 10.1 – O gargalo real, e a correção (`src/telemetry/telemetry.controller.ts`)

```
// UM INSERT multi-linha, nao um por evento. E o gargalo real deste caminho:
// um lote de 50 eventos custava 50 round-trips ao banco, e no pico da manha
// (~4k estudantes simultaneos) isso e ~800 INSERTs/s numa instancia pequena.
// Em lote vira ~16/s, que e trivial.
//
// Nao e caso para Redis: quem amortece o pico ja e o Pub/Sub, que ainda por
// cima e DURAVEL — um buffer em memoria perderia eventos no processo que
// morre antes do flush.
const linhas = doGrupo.map(
  (e) => Prisma.sql`(
    ${e.eventId}::uuid, ${new Date(e.ts)}, ${e.escolaId}, ${e.estudanteId}, ${e.expId},
    ${e.expVersion}, ${e.sessaoId}::uuid, ${e.verbo}, ${e.objeto},
```

```
    ${JSON.stringify(e.resultado ?? null)}::jsonb,
    ${JSON.stringify(e.contexto  ?? null)}::jsonb)`,
);
await tx.$executeRaw(Prisma.sql`
  INSERT INTO telemetria_eventos
    (event_id, ts, escola_id, estudante_id, exp_id, exp_version, sessao_id,
     verbo, objeto, resultado, contexto)
  VALUES ${Prisma.join(linhas)}
  ON CONFLICT (event_id, ts) DO NOTHING`);
```

NOTA

O lote é **agrupado por escola** antes da gravação, e cada grupo grava na sua própria *tenancy* (`comTenancy(fn, escolaId)`). Um lote do Pub/Sub pode conter eventos de escolas diferentes, e uma única transação não poderia ter duas políticas de RLS armadas ao mesmo tempo.

10.4.2 Partição mensal

Detalhada na Seção 8.5. Em resumo: `PARTITION BY RANGE (ts)`, partições no *schema* `particoes` (fora do alcance do Prisma), chave primária composta `(event_id, ts)` porque o PostgreSQL exige a chave de partição em todo índice único, partição `DEFAULT` como rede de segurança, e faixas declaradas até 2028-12.

DECISÃO DE PROJETO

Nada de Redis como *buffer* (**ADR-14**). Quem amortece o pico já é o Pub/Sub — e ele é *durável*. Um *buffer* em memória perde os eventos do processo que morre antes do *flush*, ou seja, reintroduz exatamente a perda que o Pub/Sub eliminou. Redis só entra com gargalo medido, e o gargalo medido aqui era o laço de `INSERT`.

10.5 Privacidade da telemetria

REGRA INVIOLÁVEL

- O evento carrega `estudante_id` **opaco** (UUID), nunca nome ou *e-mail*.
- A identidade é **derivada do *token*** no servidor. O que o cliente enviou nesses campos é descartado — então o cliente não *pode* colocar PII ali.
- Nenhuma PII em log em texto claro.
- Texto livre sensível (o *check-in* de bem-estar) **nunca** vai à telemetria; só contagens.
- Em `exp_70_sessoes.payload`, `eventos[].dados` aceita apenas número, booleano ou *string* de um **vocabulário fechado**. O campo de texto livre que existia era o único canal de texto arbitrário do *payload* — e o caminho por onde o nome de uma criança chegaria a um JSONB de longa duração. Trava estrutural, não lista de proibidos.

LIMITAÇÃO CONHECIDA

Retenção sem prazo declarado também aqui. `telemetria_eventos` guarda evento por `estudante_id` de *todas* as experiências e não tem prazo — pela mesma razão registrada na Seção 8.5 e em **ADR-13**: prazo só existe junto com a rotina que o cumpre. A diferença é que, particionada, essa rotina passou a ser trivial de escrever (`DROP TABLE particoes.<mês>`), o que remove o último argumento técnico para não ter prazo. A pergunta que sobra é de produto e de contrato, e é da plataforma inteira.

10.6 Comportamento em desenvolvimento

Configuração	Comportamento
`TELEMETRIA_TOPIC` vazio	Os eventos são apenas *logados* no console (`[telemetria/dev] N eventos`). Não há Pub/Sub em desenvolvimento e nada chega a `/ingest`.
`TELEMETRIA_PUSH_SA` / `..._AUDIENCE` vazias	`/telemetry/ingest` responde 401 a tudo. É o comportamento correto: em desenvolvimento ninguém deveria chamá-la.

NOTA

Essa combinação é o que permite rodar a plataforma inteira, com telemetria funcionando de ponta a ponta do lado do cliente, sem nenhuma credencial de GCP — e é o que faz **RNF-14** (testabilidade sem nuvem) ser verdade e não aspiração.

11

Camada de inteligência artificial

A IA generativa é usada em quatro lugares da plataforma, com propósitos e riscos muito diferentes. Este capítulo descreve a camada única por onde todos passam, a economia de *tokens* que a sustenta e os *guardrails* que impedem cada uso de produzir dano.

11.1 Onde a IA é usada

Uso	Onde	O que gera, e o que NÃO gera
Geração de *feed*	exp-01, `src/geracao/`	Personas fictícias, *captions*, *hashtags* e arte dos posts. O *mix* (quantos posts de cada lado, perfil e intensidade) é decidido pelo **servidor**; a IA só preenche.
Reação ao vivo	exp-01, `src/motor/`	Comentários de persona reagindo ao que o estudante publicou. Desligável pelo professor no meio da aula, sem *deploy* (`ia_ao_vivo`).
Plano de aula	exp-90, `src/planos/`	Um plano anual completo em Markdown, a partir do currículo da rede e das unidades do ano. **Não** decide nota nem avalia estudante.
Entorno do *check-in*	exp-91, `src/bemestar/`	Abertura do dia, pergunta aberta e acolhimento. **Nunca** reescreve um item do DASS-21, nunca vê pontuação, nome ou escola.

REGRA INVIOLÁVEL

Correção, pontuação e gabarito nunca são gerados por IA. A IA produz *conteúdo* e *texto de entorno*. Tudo que vale nota, classifica ou mede é código determinístico e testável — e roda no servidor (Capítulo 12).

11.2 A camada única

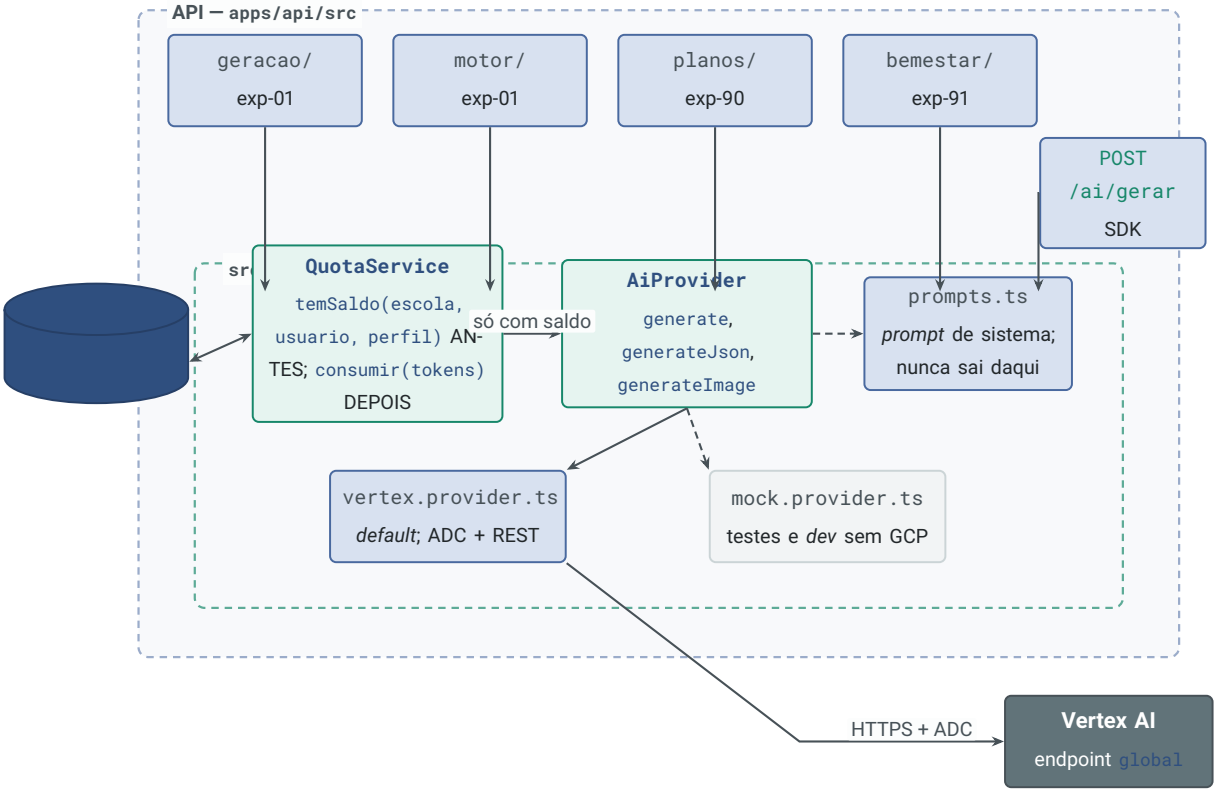


Figura 11.1 – A camada de IA. Quatro módulos de domínio e uma rota de SDK entram; todos passam por `QuotaService` antes do *provider*. Nenhum módulo fora da fronteira verde conhece o *vendor* do modelo.

11.3 Quota por perfil

> **REGRA INVIOLÁVEL**
>
> **Nunca exponha chamada de IA sem passar pelo `QuotaService`: `temSaldo(escola, usuario, perfil)` antes, `consumir` depois.**

Listing 11.1 – Incremento atômico da quota (`src/ai/quota.service.ts`)

```ts
/** Incremento atomico; devolve o total do dia depois do consumo. */
async consumir(escolaId: string, usuarioId: string, tokens: number): Promise<number> {
  const dia = this.hoje();
  return this.prisma.comTenancy(async (tx) => {
    const linhas = await tx.$queryRaw<Array<{ tokens_usados: number }>>`
      INSERT INTO ia_uso_diario (escola_id, usuario_id, dia, tokens_usados)
      VALUES (${escolaId}, ${usuarioId}, ${dia}, ${tokens})
      ON CONFLICT (escola_id, usuario_id, dia)
        DO UPDATE SET tokens_usados = ia_uso_diario.tokens_usados + ${tokens}
      RETURNING tokens_usados`;
    return Number(linhas[0]?.tokens_usados ?? tokens);
  }, escolaId);
}
```

DECISÃO DE PROJETO

A quota é persistida, não um `Map` em memória. O `Map` que existia antes tinha dois furos: zerava a cada *restart* e era *por instância* — com `min-instances: 3` no Cloud Run, a mesma criança tinha três quotas. Uma linha por (escola, usuário, dia), com incremento atômico, resolve os dois.

DECISÃO DE PROJETO

O limite depende do PERFIL. O estudante conversa com o tutor, e 20.000 *tokens*/dia bastam. O professor gera planos anuais em que só o *prompt* — currículo da rede mais material didático — passa de 200.000 *tokens*: o teto dele é outro (`AI_DAILY_TOKEN_QUOTA_PER_PROFESSOR`, *default* 2.000.000/dia, o que dá seis a nove planos). Um teto único obrigaria a escolher entre "estudante com quota absurda" e "professor que não consegue gerar um plano".

NOTA

A verificação acontece **antes** de chamar o *provider*. Gastar *token* e depois recusar seria pagar pelo que não se entrega. Há também *rate limit* por usuário (`src/comum/rate-limit.ts`), que é uma proteção diferente: a quota limita custo diário, o *rate limit* limita rajada.

11.4 Credencial: ADC, nunca chave de API

REGRA INVIOLÁVEL

O *provider* obtém o *token* com `google-auth-library`, que resolve na mesma ordem nos três ambientes:

1. `GOOGLE_APPLICATION_CREDENTIALS`;
2. o arquivo de `gcloud auth application-default login`;
3. o *metadata server* (Cloud Run).

Não existe variável de ambiente para chave de API em nenhum lugar do projeto — nem em `.env.example`.

ATENÇÃO

Antes dessa mudança havia **só** o *metadata server*, e por isso *nenhuma* chamada de IA funcionava fora do Cloud Run — nem no *host*, nem no contêiner, onde `metadata.google.internal` não resolve. Em desenvolvimento a API ficava presa em `AI_PROVIDER=mock` e a exp-90 devolvia um plano **falso**, o que é pior que falhar: parecia funcionar. Os dois *compose* montam o ADC do *host* em `/adc` somente lei-

tura; em Cloud Run a conta de serviço do serviço já *é* a credencial, e ela precisa de `roles/aiplatform.user`. Sem o papel, toda geração falha com 403 do Vertex.

11.5 Modelo, região e o custo de escolher

DECISÃO DE PROJETO

Modelo `gemini-3.6-flash`, localização `global` — para texto *e* para o plano de aula (decisão de 2026-08, **ADR-16**).
Medido no projeto do contrato: o `3.6-flash` dá **404** em `southamerica-east1`, `us-east1` e `us-central1`. Só o endpoint `global` (sem *host* regional) o serve.
A consequência é explícita e foi aceita: **o tráfego de IA sai de São Paulo, inclusive o do estudante**, e isso precisa constar do contrato e do DPA da rede. Voltar para `VERTEX_LOCATION=southamerica-east1` é possível a qualquer momento, mas exige trocar **também** o modelo pelo que a região publica — a família 2.5, já que a 2.0 saiu do catálogo e a 3.x nunca esteve lá. As duas variáveis mudam juntas, ou a geração para de funcionar.

11.5.1 Imagem: proporção pedida ao modelo, não recortada depois

Fato	Consequência
O *default* anterior era `gemini-2.0-flash-exp`	Esse modelo **não existe mais** em lugar nenhum (404 na região *e* em `global`), e por isso *toda* geração de imagem caía no *fallback* procedural — sem erro visível para o professor.
Modelo atual: `gemini-3.1-flash-image`	Responde em `global`. Alternativas verificadas: `gemini-2.5-flash-image` e `gemini-3-pro-image`.
A proporção `4:5` é PEDIDA no `imageConfig`	Sem isso, o 3.1 devolve 1408×768, e o `cover` do `AssetsService` (1080×1350) descarta mais da metade da largura.
O modelo aceita 1:1, 2:3, 3:2, 3:4, 4:3, 4:5, 5:4, 9:16, 16:9 e 21:9 pelo MESMO custo em *tokens*	Experiência com outro formato deve **pedir** o formato, não recortar depois.

11.6 Raciocínio conta como saída

ATENÇÃO

Nos modelos das famílias 2.5 e 3.x, os *tokens de pensamento* saem do `maxOutputTokens`. Com pensamento livre, o plano anual voltava **vazio**, com `finishReason: MAX_TOKENS` — o modelo gastava todo o orçamento pensando e não sobrava saída. `VERTEX_THINKING_BUDGET` (*default* `0`, desligado) mantém a paridade com o aplicativo de origem, que usava um modelo sem essa etapa; `-1` volta ao dinâmico.

REGRA INVIOLÁVEL

Quem gera texto longo checa `AiResult.finishReason`. `MAX_TOKENS` significa *conteúdo cortado*, e isso é **erro** — não resultado parcial aceitável. Entregar meio plano de aula a um professor é pior que dizer que falhou.

Parâmetro	Valor e motivo
`MAX_TOKENS_SAIDA` do plano	**64k**, não os 32k do aplicativo de origem (que eram o limite do modelo usado lá). Medido: com o material real (currículo da rede + as 4 unidades do ano) o plano é cortado em 32k e o professor recebe "o plano ficou grande demais" sem plano nenhum. A família 2.5 entrega até 65k; 64k dá a folga sem chegar ao limite duro.
`AiRequest.modelo`	`"padrao"` ou `"longo"`. O `longo` mapeia para `VERTEX_MODEL_LONGO`. Quem chama **não sabe** qual modelo é — a família é neutra de *vendor*.

11.7 *Guardrails* do *check-in* de bem-estar

Este é o uso de IA com maior risco de dano na plataforma, e por isso o mais cercado. O instrumento é o DASS-21 (LOVIBOND; LOVIBOND, 1995), na versão brasileira validada (VIGNOLA; TUCCI, 2014), em domínio público.

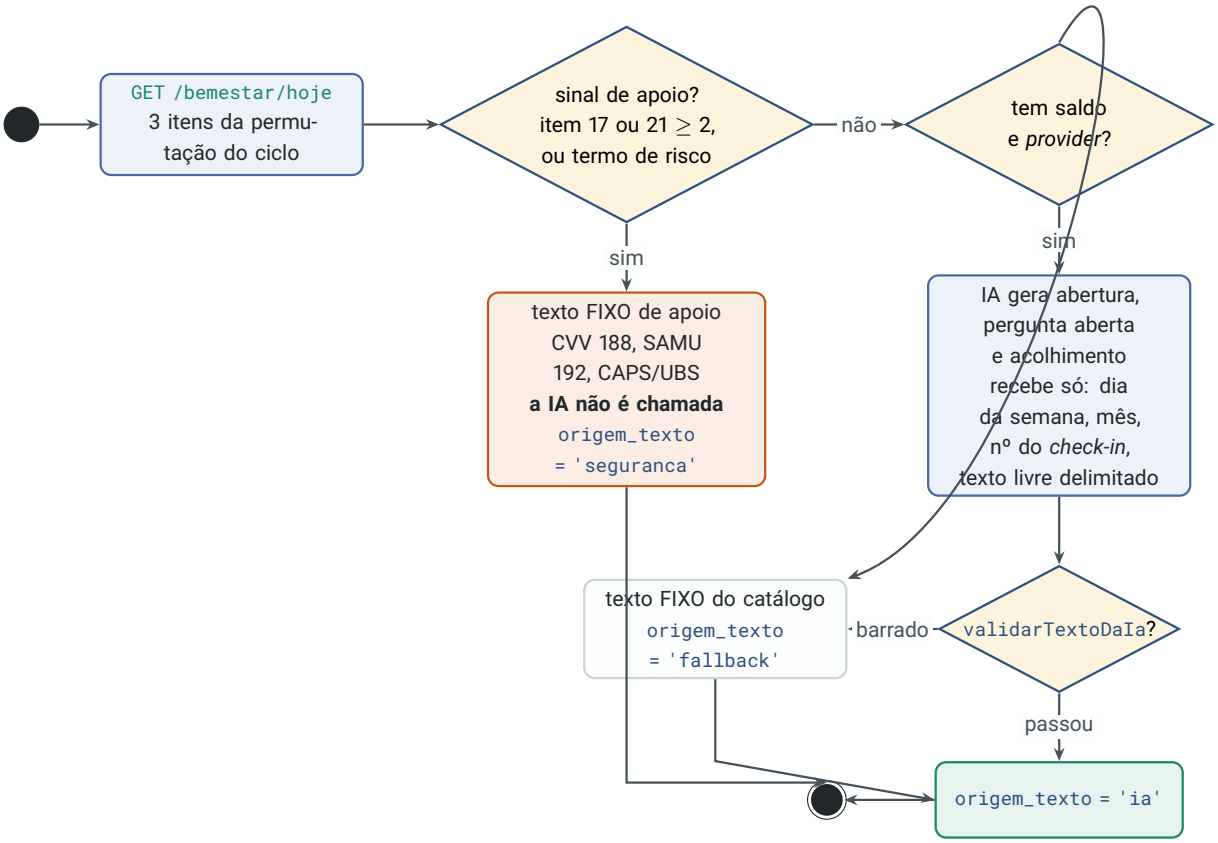


Figura 11.2 – Os *guardrails* do *check-in*. Três caminhos levam a texto seguro, e em nenhum deles o *check-in* deixa de abrir: se a IA falha, é barrada ou não há saldo, entra o texto fixo do catálogo; se há sinal de apoio, a IA nem é chamada.

REGRA INVIOLÁVEL

1. **Os 21 itens são fixos e *verbatim*** (`catalogo.ts`). A IA **nunca** reescreve um item: reformular quebra a validação psicométrica e invalida as faixas de corte publicadas.
2. **O que a IA gera é o entorno**, e cada texto passa por `regras.validarTextoDaIa`: sem vocabulário clínico, sem ordens, sem promessas, sem linguagem de gestão, sem *links*, com limite de tamanho.
3. **Barrado, sem saldo ou sem *provider* ⇒ texto fixo do catálogo.** O *check-in* nunca deixa de abrir.
4. **A IA nunca recebe** nome, escola, pontuação ou faixa — só dia da semana, mês, número do *check-in* e o texto livre, *delimitado como conteúdo*.
5. **Sinal de apoio bypassa a IA**: item 17 ou 21 ≥ 2, ou termo de risco no texto ⇒ mensagem **fixa** de apoio, e a IA não é chamada.
6. **A correção é *server-side***: ao sétimo *check-in* a API soma por subescala ($\times 2$) e classifica pelas faixas publicadas (`pontuar` e `faixaDe` em `regras.ts`).

NOTA

O texto livre do professor *vai* ao Vertex (endpoint `global`), delimitado como conteúdo e sem nenhum outro contexto identificável. Isso consta do DPA junto com o resto do uso de IA — e é a razão de a Seção 7 tratar localidade de processamento como item de conformidade, não de arquitetura.

11.8 Variáveis de configuração da IA

Variável	Default	Papel
`AI_PROVIDER`	`vertex`	`vertex` ou `mock`.
`VERTEX_PROJECT_ID`	—	**Obrigatória** com `AI_PROVIDER=vertex`: o *boot* recusa sem ela. Sem projeto, a URL viraria `.../projects/undefined/...` e o erro só apareceria na primeira geração.
`VERTEX_LOCATION`	`global`	Ver **ADR-16**.
`VERTEX_MODEL`	`gemini-3.6-fl`	Modelo do dia a dia.
`VERTEX_MODEL_LONGO`	`gemini-3.6-flash`	Modelo de saída longa. A variável separada existe para trocar um sem tocar no outro.
`VERTEX_IMAGE_MODEL`	`gemini-3.1-fl`	Quem trocar precisa conferir a proporção devolvida.
`VERTEX_THINKING_BUDGET`	`0`	`0` desligado, `-1` dinâmico, > 0 teto.
`AI_DAILY_TOKEN_QUOTA_PER_I`	20.000	Quota diária do estudante.
`AI_DAILY_TOKEN_QUOTA_PER_PROFESSOR`	2.000.000	Quota diária de professor, gestor e admin.

DECISÃO DE PROJETO

Validação de *env* no *boot*, com Zod. `VERTEX_PROJECT_ID` ausente com `AI_PROVIDER=vertex` derruba o processo na subida, com mensagem que diz o que fazer ("em *dev*, use `AI_PROVIDER=mock` para subir sem GCP"). A alternativa — descobrir na primeira geração de um professor real — é um incidente em vez de um erro de configuração.

12

Catálogo de experiências

O catálogo alvo é de 48 experiências. Cinco existem, e são elas que definiram o gabarito que as próximas seguem. Este capítulo documenta cada uma até onde a mecânica impõe exigência ao *software* — e não como material pedagógico.

12.1 Visão do catálogo

Experiência	Séries	Público	Capabilities	Módulo na API
`exp-01-instanooria`	3EF	estudante	telemetria, progresso, ia	`motor`, `geracao`, `atividades`, `perfis`, `dossie`
`exp-02-unootube`	4EF	estudant	telemetria, progresso	`unootube`
`exp-03-unoogpt`	2EM	estudante	telemetria, ia	`unoogpt`
`exp-70-vila-do-l`	**transversa**	estudant	telemetria	`vila`
`exp-90-plano-de-au`	—	**professor**	telemetria, ia	`planos`
`exp-91-bem-estar`	—	**professo**	telemetria, ia	`bemestar`
`_template`	—	—	telemetria, progresso	—

Experiência	Porta em *dev*	Onde aparece no hub
`exp-01-instanooria`	5173	`/exp/exp-01-instanooria`
`exp-02-unootube`	5174	`/exp/exp-02-unootube`
`exp-90-plano-de-au`	5175	`/professor/plano-de-aula`
`exp-91-bem-estar`	5176	`/professor/bem-estar`
`exp-70-vila-do-lume`	5177	`/exp/exp-70-vila-do-lume`

```
exp-03-unoogpt    5178        /exp/exp-03-unoogpt
```

> **NOTA**
>
> As portas vivem em `apps/hub/lib/experiencias-dev.ts` — uma por experiência. É esse mapa que o hub usa quando não há manifesto, e é por isso que `EXP_INTERNAL_HOST` tem de *preservar a porta* (Seção 9.2).

12.2 exp-01 — Instanooria

Clone de rede social de imagens. O estudante navega um *feed* curado por um motor adaptativo que o empurra progressivamente para um dos dois lados de uma controvérsia; na virada da missão, o dossiê mostra o que ele não viu.

Exigência ao *software*	Como é atendida
Feed adaptativo por blocos, determinístico	`motor/curadoria.ts`: função pura que recebe um gerador de aleatoriedade. É isso que permite ao teste afirmar "só engaja A ⇒ domínio > 0,75 até o bloco 4".
Serialização por sessão	`pg_advisory_xact_lock(runId)` dentro da transação, com `timeout` de 20 s.
Gabarito de curadoria fora do cliente	`motor/sanitizador.ts` remove `lado`, `perfil` e `intensidade` antes de o *feed* sair.
Geração de conteúdo em lote, assíncrona	`geracao/`: fila em `exp_01_generation_jobs`, *worker in-process*, reciclagem de órfão por `updated_at`.
Momento coletivo controlado pelo professor	`activities.etapa` é um relógio **coletivo**: o professor abre e fecha, e a tela do estudante trava conforme.
Reação ao vivo, desligável sem *deploy*	`activities.ia_ao_vivo`, alterável por `PATCH /atividades/:id/ia`.
Largada por perfil, sorteio ou redistribuição	`perfis/largada.ts`, com precedência explícita e `lado_origem` gravado para auditoria.
Casca de aparelho de 375×812	`viewport` no manifesto ⇒ `DispositivoFrame`.

Listing 12.1 – Precedência da largada (`src/perfis/largada.ts`)

```
// 1. `lado_forcado` — o professor redistribuiu antes da aula. Essa decisao NAO
//    pode ser desfeita por um recalculo de afinidade no login.
// 2. Afinidade do perfil — o mecanismo normal.
// 3. Sorteio balanceado — sem perfil, ou perfil `misto`. Escolhe o lado com
//    menos runs, para a turma ainda sair perto de 50/50.
//
```

```
// Existe como modulo proprio porque DUAS rotas precisam da MESMA regra: o
// primeiro inicio e o reset da sessao. Quando o reset tinha regra propria, ele
// mantinha o lado antigo — e reaproveitar o aparelho repetia a semente sem
// ninguem perceber.
```

Verbo	Significado documentado
`attempted`	entrou no aparelho
`progressed`	mudou de fase
`completed`	concluiu a missão (dossiê visto)

12.3 exp-02 — UnooTube

Clone de plataforma de vídeo, para o 4º ano. O estudante salva até três vídeos; ao terceiro, o ranking desbloqueia; "adicionar credibilidade" reordena o ranking por fontes e é a virada pedagógica.

Fluxo (1.1.0). Entrada (*briefing* da missão) → *feed* → três favoritos na bandeja → "Concluir missão" → diálogo → *ranking* → "Adicionar credibilidade" (a virada). Até a 1.0.0 o estudante caía direto no *feed* sem saber o que estava fazendo, e o terceiro salvo abria o diálogo sozinho — a palavra "missão" só aparecia depois de a medida já ter sido tomada.

Decisão	Motivo
Gabarito (`factual`) fora do *bundle*	Vive em `src/unootube/catalogo.ts`. A API corrige e devolve inscritos e fontes — nunca a resposta (regra 3).
Ranking com colegas **fictícios**	Distribuição desenhada para a lição. Não há pontuação de menor real exposta a outro menor (LGPD).
URLs das fontes como **texto**	O *sandbox* do `iframe` bloqueia *popups*, e o estudante não sai da plataforma.
Salvos **congelados** depois de "adicionar credibilidade"	O *score* da experiência já foi reportado — nota e estado não podem divergir. Curtir segue livre; reiniciar descongela.
`ranking_desbloqueado` é *latch*	Remover um salvo depois não tranca o ranking de novo. Estado que anda para trás confunde a criança e o professor.
Sem `viewport`	Foi desenhada para *desktop*: não entra em casca de aparelho.
Tema escolhido pelo estudante	Perfil → APARÊNCIA, lembrado em `localStorage` — que a regra 4 proíbe para *token*, não para preferência de tela.
Entrada só com a missão **zerada**	`src/lib/missao.ts`. Recarregar no meio cai direto no *feed*: repetir o *briefing* a cada `F5` faria o estudante pular a leitura justamente na vez em que ele leria. Reiniciar pelo perfil volta ao *briefing*.

Conclusão pelo **botão**, não pelo 3º salvo	O terceiro salvo habilita “Concluir missão”; quem fecha é o estudante. A distância entre `missao-3-salvos` e `concluir-missao` é quanto tempo ele passou revendo a escolha. Nada muda na API: `ranking_desbloqueado` já vem do servidor no 3º salvo.
Limite de 3 vira **diálogo**, não *toast*	O *toast* sumia em 5,2 s e não dava saída. O diálogo põe a troca na tela, com os três favoritos e um “Remover” em cada — e não repreende. Usa o `Modal` do **clone**: quem fala é o aplicativo sobre a própria mecânica.
Métricas de popularidade **visíveis**	`visualizacoesRotulo` e `curtidasRotulo` no catálogo, sem nenhuma marcação de verdadeiro ou falso. São cosméticas — o gabarito continua sendo `factual` — mas não são aleatórias: o vídeo *fake* é o que tem os números grandes, e há teste que garante isso.

NOTA

A exp-02 é o caso canônico da **linha da revelação** (Capítulo 14): *feed*, cromo, *playlists* e canal usam a seção do clone; ranking, os dois diálogos da missão, a tela de entrada e o botão “Concluir missão” usam a seção da plataforma. O *toast* e o diálogo de limite **não** cruzam a linha — os dois falam de dentro da ficção (“Vídeo salvo!”, “sua *playlist* tem três lugares”). O critério nunca foi “é depois no tempo?”, e sim “quem está falando?” — ver **ADR-22**.

REGRA INVIOLÁVEL

O *badge* `+N` de ganho de inscritos, ao lado do chip do canal, torna legível **durante** a missão o padrão *fake* $= +1.000$ contra confiável $= +50$. Foi aceito de propósito (**ADR-22**): o total já se movia e o delta sempre foi derivável de dois quadros seguidos. É o primeiro suspeito se a distribuição de `fontesConfiaveis` da turma subir de forma anômala.

12.4 exp-03 — unooGPT (Missão Tempestade)

Porte de um projeto autônomo (`gpturia`), publicado na trilha do Entre Redes em **2ª série do Ensino Médio, unidade 1**. Um clone de assistente de IA em que o estudante decide, sob incerteza e com o relógio correndo, onde alocar duas equipes de resgate entre quatro bairros de uma cidade fictícia antes de uma tempestade.

> O propósito **não é acertar a alocação**. É experimentar que perguntas melhores e dados melhores produzem respostas melhores — e que nenhuma delas produz a resposta certa.

12.4.1 A regra de ouro: a Lacuna não cruza a fronteira do `iframe`

Cada um dos oito conjuntos de dados distorce a cidade de um jeito: o satélite vê telhado e não vê morador, e infla o bairro industrial em 1,6 vez; o cadastro de moradores está velho onde a

cidade cresceu. Essa distorção — a **Lacuna** — é o **material do debate do professor**, e a experiência nunca a revela.

A consequência é arquitetural: **não há revelação no software**. A exp-02 troca da seção do *design system* do clone para a da plataforma na virada da missão; aqui não existe virada, e por isso a exp-03 é 100% da seção `chatgpt` e não importa `unooria.css` nenhum. A marca institucional entra só como *chrome* do hub, desenhada acima do `iframe`.

O que isso proíbe no *bundle*, verificado por `grep` no `dist/` como parte da conferência de entrega: `instrucao` (o viés de cada fonte), `pSevero`, `expostosSevero`, `expostosBrando` e `fracaoVulneravel` (os parâmetros ocultos do mundo), `CAPACIDADE_DA_EQUIPE`, `JANELA_CONFORTAVEL`, `EFICIENCIA_MINIMA` e `TETO_DE_COBERTURA` (a calibragem), e o *prompt* de sistema.

12.4.2 Três contas que o cliente não faz

Três cálculos que o projeto de origem fazia no navegador subiram para a API no porte, e o motivo é o mesmo nos três: um *bundle* capaz de computá-los deixaria o estudante antecipar o desfecho antes de enviar.

Conta	Por que subiu, e como chega ao cliente
Minuto da cidade	Depende do **pivô** da extensão de tempo (**ADR-25**), uma conta sutil que existia nos dois lados — duas cópias, duas chances de divergir. Chega pronta em `EstadoDoRelogio`; o cliente só faz o número descer entre dois *pollings*.
Projeção das rotas	Usa `chegaATempo` e a capacidade da equipe. Chega em `Projecao`, e é ela que produz o aviso "chega com N de sobra" ao lado do botão que manda.
Alcance por bairro	Idem, com o `aperto` ("chega, mas raspando") já resolvido como booleano — era a constante de calibragem `JANELA_CONFORTAVEL` viajando no *bundle*. Chega em `BairroPublico.alcance`, com o mesmo portão do `tempos`: só existe com a malha viária carregada.

12.4.3 Ponto cego é ausência, não zero

> **REGRA INVIOLÁVEL**
>
> `BairroPublico.estimativa === null` significa "nenhuma fonte carregada cobre este bairro". A tela **nunca** pode escrever "0 pessoas em risco" aí — a verdade é "não sabemos".

É a mesma distinção que separa `zero` de `none` no relatório do professor, e era um **defeito no projeto de origem**: `montarMapa` devolvia uma estimativa para todo bairro, inclusive os sem cobertura, com todos os números zerados — e ela atravessava até a tela. O comentário do próprio módulo do modelo já dizia a intenção ("precisa aparecer como ausência, não como número ruim") e o tipo do sanitizador já prometia `null`; só o código não cumpria. A correção está em `regras/camadas.ts`, e o teste de camadas fixa o invariante.

12.4.4 O Relógio da Aula

O relógio é da **turma**, não do estudante: quem dá a largada é o professor, e ela vale para a sala ao mesmo tempo. Detalhes e alternativas em **ADR-25**. O que o professor controla é *quando dá para falar* com o assistente — nunca o Passo de ninguém.

12.4.5 Telas do professor

- `/professor/entre-redes/2-serie/unidade-1` — a ficha, que **abre com a faixa da largada**: sem ela a aula não começa, e "não abre" é o suporte que a faixa evita.
- `.../unidade-1/resultados` — o painel do relógio (largada, +2 min, encerrar, modo livre, redefinir) e o **retrato de conduta** da turma. Não há coluna de resgatados: listagem é ordenável, e ordenar crianças por acerto viraria *ranking* (**ADR-27**). O *smoke* visual falha se uma aparecer.
- `.../unidade-1/estudantes/:estudanteId` — a linha de uma criança: os graus e a **transcrição**. Só o professor da escola chega aqui.
- "Zerar a turma" fica no fim da página, longe da largada: ação destrutiva ao lado do botão mais apertado é acidente esperando acontecer.

12.4.6 `allow-forms`: o bug que só o *smoke* pega

> **ATENÇÃO**
>
> O manifesto declara `"permissoes": ["forms"]`, e **sem isso a missão não funciona**. O `Composer` do *design system* é um `<form>`, e num `iframe` com *sandbox* sem `allow-forms` o navegador bloqueia o *submit*: o estudante digita a pergunta, aperta Enter e nada acontece.

O sintoma engana de propósito — a conversa parece viva, porque a fala de abertura do agente já está na tela, e o único rastro é um aviso de console sobre "*form's frame is sandboxed*". Passou pelo *typecheck*, pelo *lint* e por duas versões do *smoke* antes de ser encontrado. A correção usa o mecanismo que a plataforma já tinha, `permissoes` no manifesto, como o `pointer-lock` da exp-70: por experiência, porque permitir *submit* de formulário é capacidade de quem declarou.

12.4.7 O *smoke* mentiu duas vezes

O roteiro de `scripts/e2e/estudante/04-unoogpt.mjs` usa **dois contextos** de navegador — o professor dá a largada, o estudante joga — porque um contexto só passaria com o relógio quebrado. Duas asserções foram descartadas no caminho, e as duas passavam com a funcionalidade quebrada:

1. *esperar por texto do assistente*: a fala de abertura dele já está na tela desde o começo;

2. *esperar pela fala do estudante*: o cliente a mostra otimistamente e só a retira cerca de 30 segundos depois, quando a API falha.

A asserção honesta é a conversa **sobreviver a um F5**, porque aí ela veio de `exp_03_mensagens`. Teste que passa com a funcionalidade quebrada é pior que teste nenhum.

12.4.8 Pendência: o guia do debate

LIMITAÇÃO CONHECIDA

O guia do professor do projeto de origem está **vazio** — o material de condução do debate nunca foi escrito. E é no debate que a lição acontece, porque a missão deliberadamente não revela nada por software. A ficha publicada no catálogo (competência, quatro atos, preparo) foi redigida a partir das ADRs e dos dois *playtests* documentados, e está marcada no código como ponto de partida para revisão pedagógica, não palavra final.

12.5 exp-70 — Vila do Lume

Jogo 3D em primeira pessoa, **transversal** (`series: []`): não pertence a ano nenhum. É o "Ato 1" que alimenta a largada da exp-01.

12.5.1 Duas leituras de uma partida, e elas não se tocam

REGRA INVIOLÁVEL

O retrato Schwartz (`mapeamento.ts`: 10 valores + 2 eixos) lê *escolhas com custo* dentro do jogo. A missão "A Noite Longa" (`mensagens.ts`) lê *julgamento sobre 5 mensagens*. A separação é **de medida**: um golpe que custasse fagulha faria a criança que caiu aparecer como *menos generosa* no retrato, e nada no relatório mostraria isso.

12.5.2 O servidor recalcula; nunca aceita o perfil do cliente

REGRA INVIOLÁVEL

O jogo sabe computar o retrato e **deliberadamente não o envia**: `CHAVES_DE_PERFIL` em `src/vila/contrato.ts` *recusa payload* que traga o perfil. É isso que permite reprocessar o histórico quando `VERSAO_MAPEAMENTO` subir (hoje 3, herdada do piloto para os dados serem comparáveis).

12.5.3 `grau` é invariante de API, não regra de tela

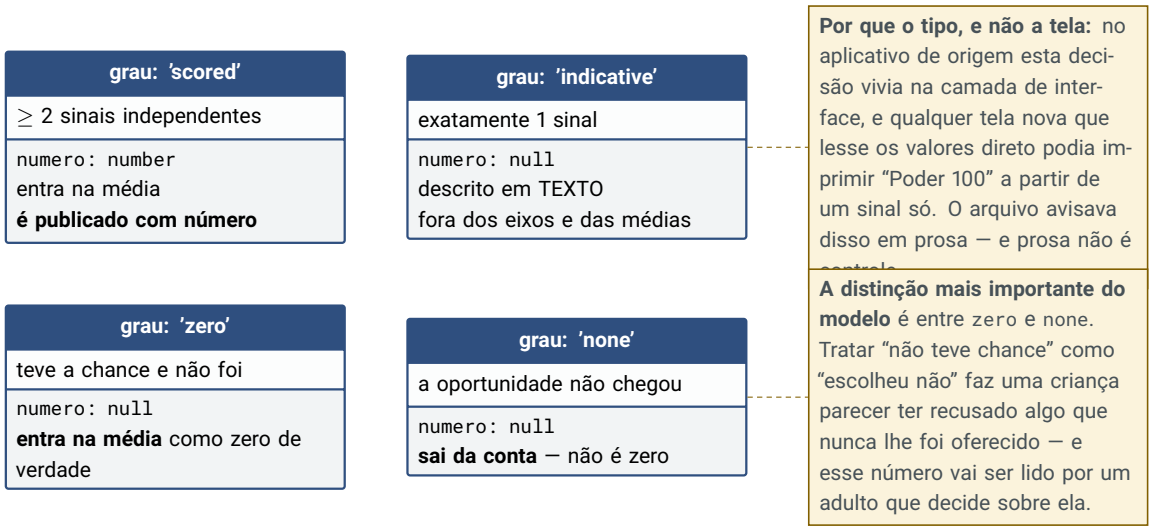


Figura 12.1 – Os quatro graus de medida da exp-70. A API devolve uma união discriminada em que **só** a variante `scored` tem `numero: number`; as outras têm `null` *por tipo*. O hub fica estruturalmente incapaz de imprimir número onde não há medida.

12.5.4 A bolha é do servidor

REGRA INVIOLÁVEL

`POST /vila/sessoes` atribui a bolha A/B/C **balanceada dentro da turma**, persiste em `exp_70_bolhas` (estável entre partidas) e devolve os cartões — **só os desta criança e sem gabarito**.

Antes era `estudanteSlot % 3` no navegador, o que punha no *bundle* a condição do próprio instrumento e deixava as três versões legíveis antes da aula.

Nos anos iniciais (1EF–5EF) a bolha B **não é servida** ("suporte" exige entender o que é suporte) — regra que só a API pode aplicar, porque só ela sabe a série.

O *lock* consultivo por turma não é zelo: a sala clica junto, e sem ele os 25 caem todos na bolha A.

ATENÇÃO

`cartoesServidos` ≠ "apareceram". O primeiro é quantos a API entregou; o segundo, quantos chegaram a aparecer na partida. Zero aparecidos ⇒ a tela diz "as mensagens não chegaram nesta partida", **nunca** "ela ignorou tudo" — é a mesma distinção entre `zero` e `none`.

12.5.5 Telas do professor

Rota	Conteúdo e regra

`.../vila-do-lume`	Ficha: para que serve, quem consome o perfil, passo a passo.
`.../vila-do-lume/guia`	O debate. Projeta as mensagens das **três** bolhas lado a lado — é a revelação da missão, e a primeira vez que a turma vê as outras versões.
`.../resultados?turmaId=`	Listagem da turma, com o botão de zerar.
`.../estudantes/:id`	A leitura individual, que abre pelo **nível**.

REGRA INVIOLÁVEL

1. **O `/guia` precisa saber a TURMA.** Nos anos iniciais a API não serve a bolha B, e uma tela que projetasse as três mandaria o professor esperar a reação de um grupo que a sala dele não tem. `GET /vila/professor/cartoes?turmaId=` filtra pela *mesma* regra de `abrir` (`bolhasElegiveis`).
2. **Os textos vêm da API, nunca de cópia no hub.** Os textos *são* o instrumento, e uma vírgula divergente entre o que o jogo serviu e o que o professor projeta põe a sala discutindo uma mensagem que ninguém recebeu.
3. **A listagem da turma não mostra valor de Schwartz nenhum.** Listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças. Os valores vivem na tela individual, uma criança por vez, com o grau ao lado.
4. **O agregado abre para gestor; a linha de uma criança, não.** São *controllers* separados, e não um `@Roles` misto com exceção por método.
5. **Zerar a turma apaga as BOLHAS junto.** Rodar de novo com a mesma bolha serviria à criança o mesmo golpe: ela reconhece, e a segunda rodada não mede nada. O que **não** é tocado é `exp_01_exp_70_perfis_importados` — aquilo pertence à atividade da exp-01, e apagar de lá trocaria os lados de uma aula em andamento.

12.5.6 A exceção de ADR: o jogo em JavaScript clássico

DECISÃO DE PROJETO

`apps/experiencias/exp-70-vila-do-lume/public/jogo/` — cerca de dez mil linhas de JavaScript clássico com variáveis globais compartilhadas e `three.min.js` vendorizado — **fica como está**, e está nos `ignores` do ESLint (**ADR-18**).
É um instrumento com distâncias e tempos calibrados, e o compilador não tem como saber que uma refatoração mudou a *medida*. O *shim* (`src/`, TS *strict*) é a única porta: `window.VILA`. A rede que substitui o `tsc` ali é o *smoke* visual.

12.6 exp-90 — Gerador de plano de aula

Ferramenta do professor. Detalhada nas Seções 11 e 4; o que resta registrar é a divisão de responsabilidade de armazenamento.

Artefato	Onde vive, e por quê
`gerado.md`	*Bucket* privado, **imutável**. É o original da IA: restaurar é voltar a ele.
`atual.html`	*Bucket* privado. O que o editor salvou, **sanitizado por *allowlist***. Versionamento de objetos dá histórico de cada "Salvar".
Referências curriculares	PostgreSQL (`exp_90_referencias`), como **texto**. Catálogo do contrato, sem `escola_id`, fora da RLS. Importadas por `scripts/importar-referencias-plano.mjs` com `PLANOS_IMPORT_TOKEN`.
Metadados e estado	PostgreSQL (`exp_90_planos`).

ATENÇÃO

`PLANOS_IMPORT_TOKEN` ausente ⇒ a rota de importação responde **503**, não fica aberta. O mesmo vale para `PERFIL_IMPORT_TOKEN`.

12.7 exp-91 — Check-in de bem-estar

Ferramenta do professor, detalhada na Seção 11. O que resta registrar é a mecânica de micro-doses e a decisão de escopo de leitura.

Mecânica	Implementação
21 itens em 7 *check-ins*	`ordem_itens` é a permutação de 1..21 sorteada na abertura do ciclo; cada *check-in* leva 3 itens, um de cada subescala.
Um *check-in* por dia	`UNIQUE (escola_id, professor_id, dia)`, com `dia` no fuso `America/Sao_Paulo`.
Ciclo expira em 21 dias	`expira_em`; ciclo não fechado vira `expirado`, sem pontuação.
Resumo leve para o *card* do hub	`GET /bemestar/resumo`, sem IA. É o que a home do professor lê — e a home não pode depender de uma chamada de modelo.
Apagar tudo	`DELETE /bemestar/dados`.

DECISÃO DE PROJETO

Nenhuma rota de gestor nem agregado (**ADR-10**). Dado de saúde (LGPD art. 11) com RLS *e* posse por `professor_id` em toda *query*; só o professor lê. Uma visão agregada e anonimizada para a rede é decisão *futura*, com DPA e *k*-anonimato — e não sai de um `GROUP BY`.

12.8 Como criar uma experiência nova

Listing 12.2 – O caminho completo

```
# 0. Uma branch por experiencia, com o nome do slug
git switch -c exp-07-fracoes

# 1. Scaffolding do _template (copia e ajusta nomes)
pnpm new:exp exp-07-fracoes

# 2. Implementar em src/App.tsx+
#    - auth, api e telemetria: SOMENTE pela SDK
#    - interface: SOMENTE o @unooria/design-system, pela ponte packages/ui,
#      na secao do clone que a experiencia imita
#    Falta componente? Ele nasce no repo storybook-unooria e chega por versao
#    nova — nunca como arquivo local.

# 3. Declarar capabilities e verbos no public/experiencia.json

# 4. Documentar verbo/objeto de telemetria no README da experiencia

# 5. Build gera dist/ 100% estatico. Testar embedado:
pnpm --filter @edu/exp-07-fracoes dev     # porta nova em lib/experiencias-dev.ts
# e abrir a bancada do hub:
#   http://localhost:3000/dev-embed?src=http://localhost:5178
```

Checklist	Verificação
Módulo de domínio na API	Necessário sempre que houver gabarito, estado de missão ou visão de professor. Prefixo minúsculo igual ao slug (`src/<slug>/`), com `catalogo.ts` + `regras.ts` + *service* com `comTenancy` + *controllers* por papel.
Tabelas	Prefixo `exp_NN_`, `escola_id NOT NULL` + RLS.
Porta de *dev*	Registrada em `apps/hub/lib/experiencias-dev.ts`.
`allowedHosts` do Vite	Deve incluir `host.docker.internal`, senão o hub em contêiner não lê o manifesto.
`viewport`	Só se a experiência exigir casca de aparelho.
`permissoes`	Só o que o `iframe` realmente precisa.
Mídia > 2 MB	Vai para o *bucket* de *assets*, referenciada por variável de *build*. Nunca no *git*, nunca no *bundle*.
Ponta a ponta	Uma suíte em `scripts/e2e/<papel>/` se a experiência tem tela nova no hub.

> **NOTA**
>
> O `_template` continua **cru de propósito**: o que ele não traz, o *design system* traz. Um *template* rico em componentes locais seria a primeira violação da regra de interface, replicada 48 vezes.

13

Especificação da API

A API é um serviço NestJS (NestJS, 2026) com 22 módulos. Este capítulo especifica a superfície HTTP: as convenções transversais, os grupos de rotas e a autorização de cada um. O catálogo completo, rota por rota, está no Apêndice C.

13.1 Convenções transversais

Aspecto	Convenção
Autenticação	`Authorization: Bearer <JWT>`, validado por `OidcGuard` contra o JWKS de `OIDC_ISSUER`. Três rotas fogem disso, e cada uma tem substituto explícito (abaixo).
Autorização	`@Roles(...)` + `RolesGuard`, *sempre* depois do `OidcGuard` na lista de *guards*. Sem `@Roles`, a rota aceita qualquer papel autenticado.
Tenancy	`TenancyInterceptor` global abre o contexto com a escola do *token*; toda *query* passa por `comTenancy` e filtra `escola_id` explicitamente.
Validação de entrada	Zod em todo corpo e *query*. Não há `class-validator` nem *pipes* de DTO: o esquema Zod é a fonte única do tipo e da validação.
Corpo	JSON com teto de 2 MB; `text/csv` cru com 30 MB, para a importação de pessoas — o teto do *bodyParser* e o da rota saem da MESMA constante (`TETO_DO_ARQUIVO`, em `rostering.service.ts`). O número cabe debaixo dos 32 MiB que o Cloud Run recusa por requisição HTTP/1, para o 413 do ambiente nunca ser o que o gestor vê (**ADR-48**).
CORS	Lista explícita em `CORS_ORIGINS` — hub mais os *dev servers* das experiências. Nunca `*`.
Erros	*Exceptions* do Nest (`BadRequestException`, `ForbiddenException`, `NotFoundException`). Domínio com vocabulário próprio de erro usa um `CodigoErro` tipado (ex.: `src/unootube/erros.ts`).
Assincronia longa	**202 + *polling***. Nenhuma rota fica aberta esperando modelo de linguagem.

13.1.1 As três rotas sem `OidcGuard`

Rota	Substituto	Por quê
`GET /healthz`, `GET /readyz`	Nenhum — públicas	Sonda de saúde. Exigir *token* tornaria a sonda dependente do IdP.
`POST /telemetry/beacon`	Verificação manual do *token*, pelo `TokenService`	`sendBeacon` não envia cabeçalho, então o *token* vem no corpo. O rigor da verificação é o mesmo; só o transporte muda.
`POST /telemetry/ingest`	`PubSubPushGuard`	Quem chama é o Pub/Sub, não um usuário. Verifica emissor Google, `aud` e `email` da conta de serviço. **Falha fechada** sem configuração.

> **ATENÇÃO**
>
> Há ainda duas rotas de *serviço* protegidas por chave em cabeçalho, e não por JWT: `POST /perfis/importar` (`x-perfil-token`) e `POST /planos/referencias` (`x-import-token`). Ambas são chamadas por *script* local ou por integração externa, e ambas respondem **503** quando a variável correspondente está ausente — desligadas, não abertas.

13.2 Mapa de rotas por área

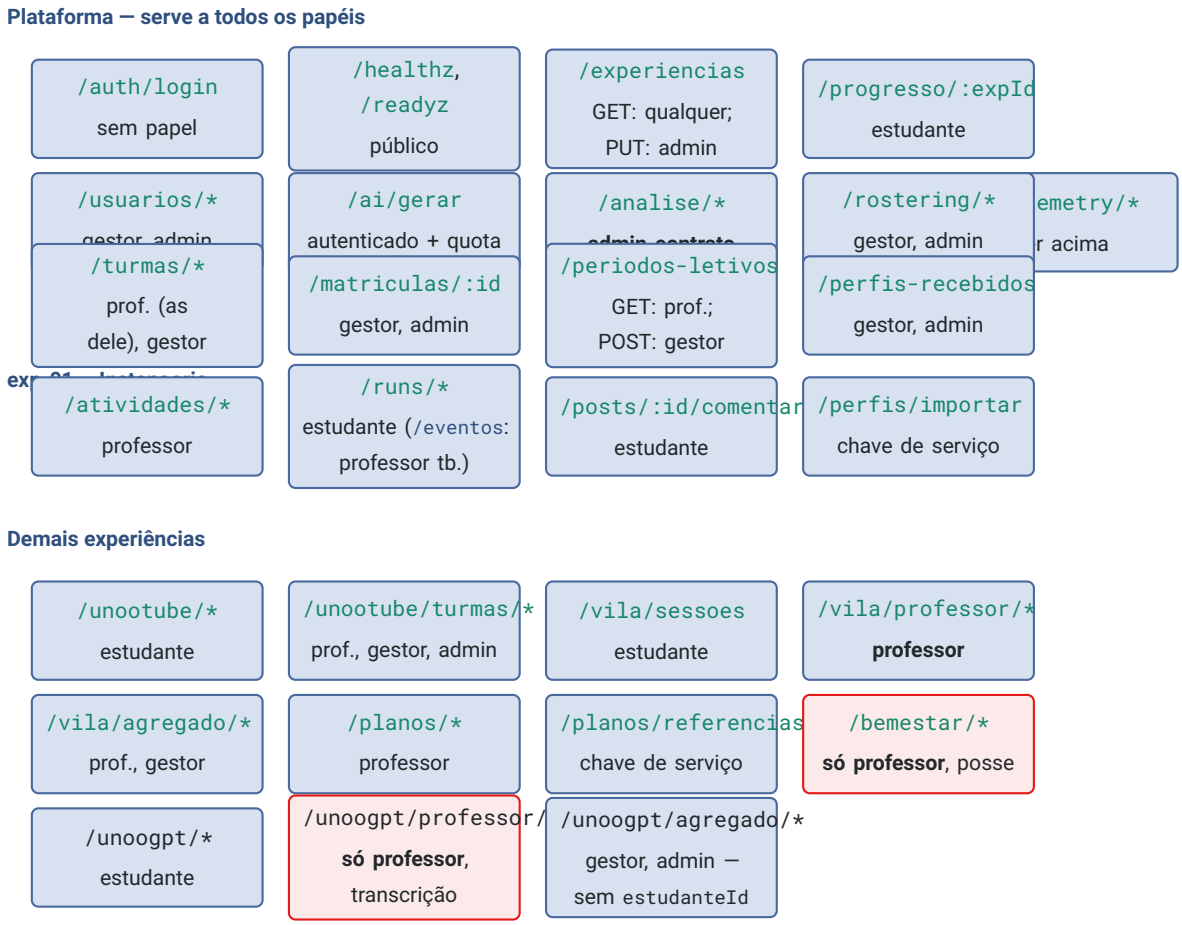


Figura 13.1 – Mapa das rotas por área e papel autorizado. Duas áreas estão destacadas por **ausência deliberada**: bem-estar não tem nenhuma rota de gestor nem de agregado; e no unooGPT a transcrição do estudante só existe no controller do professor — o do gestor **não tem uma rota que aceite `estudanteId`**, o que torna estrutural, e não disciplinar, o limite do que ele alcança (**ADR-27**). Em `/turmas/*` a autorização do professor é por **matrícula**, não por escola: é a diferença entre "dá aula nesta turma" e "trabalha nesta escola" (**ADR-28**).

13.3 Superfície que a SDK consome

Uma experiência nunca chama a API "à mão": chama pela SDK, e a SDK oferece exatamente quatro capacidades.

Listing 13.1 – `ExperienciaHandle` — tudo que a experiência pode fazer

```
export interface ExperienciaHandle {
  estudante: Estudante;
  config: Record<string, unknown>;

  /** Cliente HTTP autenticado. Sem retry fora do GET — repetir um POST
   *  de dominio pode duplicar efeito. */
  api: {
    get<T>(path: string): Promise<T>;
    post<T>(path: string, body?: unknown): Promise<T>;
```

```
    put<T>(path: string, body?: unknown): Promise<T>;
    patch<T>(path: string, body?: unknown): Promise<T>;
    delete<T>(path: string): Promise<T>;
  };

  /** Unica porta para IA. A experiencia NAO conhece modelo nem prompt. */
  ai: { gerar(input: { atividade: string; mensagem: string }): Promise<{ texto: string }> };

  /** Telemetria em lote. A experiencia informa SO verbo, objeto e dados. */
  track(verbo: TelemetryEvent["verbo"], objeto: string,
        dados?: Omit<TelemetryEvent, "verbo" | "objeto">): void;

  progresso: {
    salvar(dados: Record<string, unknown>): Promise<void>;
    carregar<T>(): Promise<T | null>;
  };

  concluir(payload?: { score?: number; dados?: Record<string, unknown> }): void;
  on(evento: "pause" | "resume", cb: () => void): void;

  /** Estado de alto nivel para o chrome opcional do hub (exp:estado). */
  enviarEstado(estado: Record<string, unknown>): void;
}
```

> **NOTA**
>
> Não há método de “corrigir” nem de “pontuar” na SDK, e isso é intencional: correção é rota de domínio da experiência na API (`/unootube/*`, `/vila/*`, `/runs/*`), não capacidade genérica. Uma API genérica de avaliação convidaria a mandar a resposta certa junto.

13.4 Grupos de rotas em detalhe

13.4.1 Motor de *feed* (exp-01)

O grupo com mais rotas da plataforma, todas escopadas por `run` e todas de papel `estudante`, com duas exceções marcadas.

Rota	Papel	Semântica
`POST /runs`	estudante	Cria ou recupera a sessão; resolve a largada (perfil, sorteio ou redistribuição).
`GET /runs/boot`	estudante	Estado inicial que a experiência precisa para montar a primeira tela.
`POST /runs/:id/entrar`	estudante	Nome no aplicativo e avatar escolhidos.
`GET /runs/:id/feed`	estudante	**Próximo bloco curado.** Abre transação, toma o *lock* da `run`, aplica eventos pendentes, cura, persiste e sanitiza.
`POST /runs/:id/eventos`	**estudante, professor**	Canal operacional de telemetria (*dwell*, curtidas, salvos). Síncrono e transacional.

`GET /runs/:id/fase`	estudante	Fase atual, para travar a tela conforme o relógio coletivo da turma.
`POST /runs/:id/conclusao`	estudante	Inclinação escolhida (A, B ou dúvida).
`POST /runs/:id/reflexao`, `GET .../reflexao`	estudante	Texto livre de fechamento, sem limite de caracteres.
`POST /runs/:id/student-post`	estudante	Post publicado pelo estudante.
`POST /runs/:id/reacoes`	estudante	Dispara reações de persona ao vivo (se `ia_ao_vivo`).
`GET /runs/:id/dossie`	**estudante, professor**	O dossiê de fechamento — a única rota do grupo que o professor também lê.
`GET /runs/:id/perdidos`	estudante	Os posts que o estudante *não* viu. É a virada pedagógica da missão.
`POST /runs/:id/reset`	**professor**	Zera a sessão. Recalcula a largada pela *mesma* regra do primeiro início.

13.4.2 Entrada de pessoas: as duas portas, e por que só uma escreve

Criar pessoa tem duas superfícies e **um** caminho de escrita. Em lote é `POST /rostering/importar` (ou o *binding* OneRoster); uma por vez é `POST /usuarios`, o formulário do balcão — que não escreve em `users`: serializa os campos como uma linha da MESMA planilha e chama o MESMO importador (**ADR-59**). É o que mantém CPF com dígito verificador, série, turno, login gerado, turma, lotação e matrícula com uma definição só.

Rota	Papel	Semântica
`POST /rostering/importar`	gestor, admin	O lote. Idempotente pela matrícula da rede e pelo CPF: reimportar atualiza, não duplica, e não reseta a senha de quem já entrou.
`POST /usuarios`	gestor, admin	Uma pessoa. **Criar é criar**: CPF, matrícula ou login já existentes devolvem 409 e nada é escrito. A senha sai uma vez no corpo, sempre provisória.
`POST /usuarios/admin-contrato`	**admin-contrato**	O único papel que não passa pelo importador: sem `escola_id`, sem lotação, sem turma. O login dele é único na REDE, não na escola.

ATENÇÃO

O 409 de um CPF encontrado em **outra escola** da rede não devolve id, nome nem o nome da unidade — é dado de outro *tenant*. A mensagem diz o caminho legítimo (transferência é pela importação, que move a matrícula junto) e nada além disso.

13.4.3 Planos de aula (exp-90)

Rota	Código	Semântica
`POST /planos`	**202**	Enfileira a geração e devolve o id. *Não* espera o modelo.
`GET /planos/:id`	200	Estado (`pendente`, `gerando`, `pronto`, `erro`) e, quando pronto, o conteúdo. É o alvo do *polling*.
`PUT /planos/:id/conteudo`	200	Salva o HTML do editor, sanitizado por *allowlist*.
`POST /planos/:id/restaurar`	200	Volta ao `gerado.md`, que nunca foi sobrescrito.
`PATCH /planos/:id`	200	Renomeia.
`DELETE /planos/:id`	200	Exclui.
`GET /planos/referencias/disponiveis`	200	Que disciplinas e séries têm material importado — é o que popula o formulário.

13.4.4 Bem-estar (exp-91)

Rota	Semântica
`GET /bemestar/resumo`	**Leve, sem IA.** É o que o *card* do hub lê — a home do professor não pode depender de uma chamada de modelo.
`GET /bemestar/hoje`	Abre o *check-in* do dia: 3 itens *verbatim* + textos de entorno (IA ou *fallback*).
`POST /bemestar/checkins/:id/respostas`	Grava. No sétimo, pontua e classifica *server-side*.
`GET /bemestar/historico`	Série do próprio professor.
`DELETE /bemestar/dados`	Apaga tudo do próprio professor.

REGRA INVIOLÁVEL

Todas as cinco são `@Roles("professor")` e filtram `professor_id = req.auth.usuarioId` em toda *query*. RLS por escola não basta aqui: dois professores da mesma escola não podem se ler.

13.4.5 Análise

Rota	Papel	Escopo
`GET /atividades/:id/analise`	professor	Uma atividade da própria escola.
`GET /atividades/:id/analise/c`	professor	Exportação da mesma visão.
`GET /analise/resumo`	**admin-contrato**	Toda a rede.
`GET /analise/atividades/:id`	**admin-contrato**	Uma atividade, cross-escola.
`GET /analise/csv`	**admin-contrato**	Exportação da rede.

> **DECISÃO DE PROJETO**
>
> São **dois *controllers* no mesmo arquivo**, com `@Roles` diferentes: um em `/atividades` para o professor, outro em `/analise` para o `admin-contrato`. A separação física é o que garante que a próxima rota adicionada herde a permissão certa — e é a mesma decisão tomada em `src/vila/`.

13.5 Sanidade e prontidão

Rota	Semântica	Uso
`GET /healthz`	O processo está vivo	*Liveness* do Cloud Run.
`GET /readyz`	As dependências estão prontas	*Readiness*: verifica o banco. É o que impede uma revisão nova de receber tráfego antes de conseguir consultar.

> **NOTA**
>
> As duas são públicas de propósito. Uma sonda de saúde que precise de *token* depende do provedor de identidade para dizer se a API está viva — o que inverte a relação de diagnóstico exatamente quando ela mais importa.

13.6 Documentação executável (OpenAPI)

A API publica um documento OpenAPI 3.0 em `/docs` (Swagger UI navegável) e `/docs-json`. Ele é **derivado** dos *schemas* Zod que as próprias rotas usam para validar — não escrito ao lado deles (**ADR-36**).

Peça	Papel
`openapi/contrato.ts`	O decorator `@Contrato`: resumo, prosa, corpo, *query*, resposta e notas. Só grava metadata.
`openapi/rotas.ts`	Percorre os *controllers* pelo `DiscoveryService` e devolve verbo, caminho, `@HttpCode`, `@Roles`, *guards* e contrato de cada rota. Fonte única dos três consumidores.
`openapi/zod-para-openapi.ts`	Zod → JSON Schema no dialeto do OpenAPI 3.0.
`openapi/documento.t`	Monta cada operação. O esqueleto de caminhos vem do roteador do Nest; o conteúdo, do `@Contrato`.
`openapi/porta-do-doca.gu`	A porta de senha (*Basic*), com comparação de tempo constante.
`openapi/contrato.in`	Confere a resposta contra o *schema* declarado, fora de produção (**ADR-37**).

> **NOTA**
>
> **Os papéis no documento vêm do `@Roles` real**, não de um campo repetido no `@Contrato`. Repetir abriria espaço para o documento dizer `gestor` enquanto o *guard* exige `admin-contrato` — e uma documentação de autorização errada é pior que nenhuma.

A trava. `tests/openapi-cobertura.spec.ts` percorre o roteador — não uma lista escrita à mão — e cobra `@Contrato` de toda rota, salvo as nomeadas numa lista explícita de pendências que só encolhe. Ela confere também que o acesso declarado bate com os *guards* que a rota realmente tem (rota que se diz pública e carrega `OidcGuard`, ou o contrário), e que a lista de pendências não guarda rota que já não existe. Rota nova sem contrato fica vermelha.

O documento e o Apêndice C são GERADOS. O `docs/openapi.json` é versionado no repositório — é o artefato que um cliente ou um edital consome sem precisar subir a plataforma — e o catálogo de *endpoints* sai dele, não mais de prosa escrita à mão.

Listing 13.2 – Regerar o documento e o apêndice

```
npx ts-node --transpile-only apps/api/scripts/gerar-openapi.ts
node scripts/gerar-endpoints-tex.mjs
cd documentacao-latex && ./compilar.sh
```

> **NOTA**
>
> A coluna de autorização do apêndice vem das extensões `x-papeis` e `x-acesso` que o documento carrega, e não da prosa da descrição. Um gerador que fizesse *parsing* do próprio texto procurando "**Papéis:**" quebraria na primeira vez que alguém reescrevesse a frase — e quebraria em silêncio, produzindo uma tabela de autorização errada, que é o pior defeito possível num catálogo de rotas.

14

Design system e camada de interface

> **REGRA INVIOLÁVEL**
>
> **Todo elemento de interface do hub e das experiências vem do `@unooria/design-system`, sempre na última versão publicada.** Componente de interface novo se escreve *no repositório do design system* e chega aqui por versão — não em `packages/ui`, não dentro de uma experiência. Antes de criar um botão, um *card* ou um campo: procure na seção correspondente do pacote, ela provavelmente já tem.

O pacote é npm **público** no escopo `@unooria` — sem *registry* privado, sem `.npmrc` no repositório. A fonte é o repositório `storybook-unooria` (Storybook em `:6007`, publicado por *tag* `v*` no GitHub Actions).

14.1 Duas famílias no mesmo pacote

O pacote publica **seções** independentes, cada uma com *namespace* de token e prefixo de classe próprios. Elas nunca se cruzam.

Seção	Import	Tokens	Onde vale
`unooria` (39)	raiz do pacote	`--unooria-*`, classe `.unooria-theme-light`	Hub: *landing* (`/`), *login*, `/professor`, `/admin`, `/analise`; *header* de todos os papéis; telas **pós-revelação** do estudante
`instanooria` (21)	`/instanooria`	`--insta-*`, `insta-`	exp-01
`unitube` (30)	`/unitube`	`--uni-*`, `uni-` — **só escuro**	exp-02
`whatsappmobile` (29)	`/whatsappmobi`	`--wa-*` — só escuro	—
`chatgpt` (25)	`/chatgpt`	`--gpt-*`, `data-gpt-theme`	**exp-03 inteira** — ela não tem revelação no software, então nunca troca para a seção da plataforma

`tiktooria` (35)	`/tiktooria`	`--tik-*`	— (dois *contextos* de superfície: cromo claro de lista e perfil, imersivo preto de *feed* e câmera — não dois temas)

NOTA — "pós-revelação" virou uma tela de verdade

Por muito tempo a linha "telas pós-revelação do estudante" descrevia um único componente: o `DonutChart` do relatório, num *chunk* tardio. O resto do relatório era Tailwind do clone sobre `bg-night`.
Com a ADR-41 (18.42) o relatório sai do aparelho e passa a ser claro e largo, e a autorização deixa de ser teórica. A fronteira não mudou: quem a marca é a REVELAÇÃO — sete telas, fundo `night`, sem nada da plataforma. Antes dela, o disfarce; depois, a língua da escola.
A trava técnica continua a mesma e não pode cair: o CSS do DS entra por `React.lazy`, no *chunk* do relatório, e nunca no *bundle* do feed — lá ele seria legível no DevTools antes da hora.

ATENÇÃO

A seção `unitube` é **só escura**: o Figma não publica variante clara, e **derivar uma é proibido**. Uma paleta clara inventada não é o clone — é uma aproximação, e a experiência mede reconhecimento.

14.2 A linha que separa as seções é a REVELAÇÃO

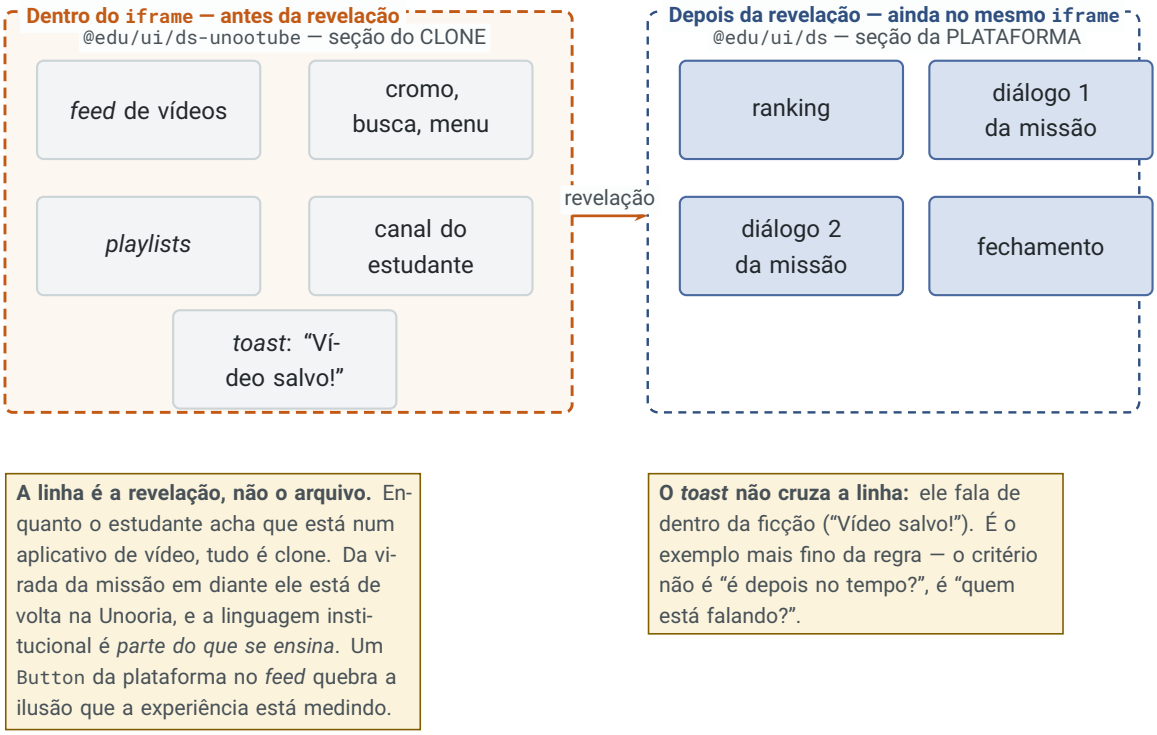


Figura 14.1 – A linha da revelação, na exp-02. As duas seções do *design system* coexistem no mesmo *bundle* e a fronteira entre elas é pedagógica, não técnica.

> **NOTA**
>
> **A linha tem duas pontas, não uma.** A versão 1.1.0 da exp-02 abriu com uma **tela de entrada** que explica a missão, e ela usa a seção da **plataforma** — antes do *feed*, ou seja, *antes* da revelação. Não é exceção à regra: é a regra aplicada pelo critério que ela mesma enuncia, "quem está falando?". No *briefing* quem fala é a escola, e não há disfarce a proteger — a criança ainda não viu um *Short*. O disfarce começa no toque em "Começar missão". Ver **ADR-22**.

14.3 Consumo: sempre pela ponte

> **REGRA INVIOLÁVEL**
>
> **Consumir sempre pela ponte em `packages/ui`**, nunca importando o pacote direto num componente:

`@edu/ui/ds`	`unooria`	plataforma
`@edu/ui/ds-insta`	`instanooria`	exp-01
`@edu/ui/ds-unootube`	`unitube`	exp-02

A ponte existe por dois motivos, e os dois estão registrados nos arquivos:

1. **Marcar `'use client'` de uma vez**, para o App Router do Next.js.
2. **Desfazer a colisão de nomes.** `Icon`, `Avatar`, `Badge`, `Button`, `Logo` e `TextField` existem nas *duas* famílias. Os clones só saem pelo *namespace* (`InstanooriaUI.Icon`), e **destruturar na ponte** é o que mantém o *tree-shaking* e evita o erro `TS2742` causado pelo `@types/react` interno do pacote.

> **NOTA**
>
> Clone novo em uso = arquivo novo `packages/ui/src/ds-<clone>.ts` + `exports` no `packages/ui/package.json`. Mesmo padrão, sempre.

14.4 CSS: uma vez, no ponto de entrada

Aplicação	Onde o CSS entra
Hub	`components/casca-plataforma.tsx` — e é *por isso* que `/exp/*` e `/dev-embed`, que estão fora do grupo de rotas `(plataforma)`, também têm o *design system*.
Experiência	O próprio `src/main.tsx`, com o CSS da *sua* seção.

> **REGRA INVIOLÁVEL**
>
> **Nunca importe o consolidado `/styles.css` numa experiência**: ele arrasta as seis seções e a tipografia da plataforma para dentro do clone.

14.4.1 No hub, o CSS não basta: as fontes também

> **ATENÇÃO**
>
> O `unooria.css` declara `Open Sans` e `Material Symbols Rounded` por `@import` de URL remota — e o **Next remove `@import` externo** de CSS importado por JS, *silenciosamente*: a regra continua lá, o `getComputedStyle` responde "Open Sans" e o navegador desenha com a fonte do sistema.
> Como `Material Symbols Rounded` é fonte de **ligadura**, o sintoma é o **ícone virando texto**: onde deveria haver um ícone de adicionar, aparece "addInvestigação".
> Por isso `components/casca-plataforma.tsx` também monta `FontesDaPlataforma` (`@edu/ui/fontes`). Numa experiência Vite o `@import` sobrevive e isso não é necessário.

ATENÇÃO

Segundo caminho para o mesmo sintoma: `icon` como *string*. Componentes de navegação do pacote (`SideBar`, `Tab`) tipam `icon` como `React.ReactNode`, não como nome de ícone. Passando a *string*, o item renderiza o **texto cru**: a barra lateral da área administrativa escrevia "group", "school" e "upload_file" na lateral — com a mesma cara de fonte que não carregou, e com a fonte perfeitamente carregada.
A forma certa é `icon={<Icon name="group" />}`. Quem pegou foi o *smoke* visual: *typecheck* não reclama, porque `string` é um `ReactNode` válido.

REGRA INVIOLÁVEL

Quem importa, declara. Todo aplicativo que escreve `@unooria/design-system` no código tem a dependência no seu `package.json` — hub incluído, porque ele importa o `unooria.css`. Depender do *hoisting* do pnpm via `packages/ui` funcionava *por acaso*: bastava a ponte trocar de versão (ou o `node_modules` ser reinstalado com outro *layout*) para o CSS do hub sumir sem ninguém ter mexido nele.

14.5 Frames: dois níveis, não empilhar

Componente	Onde vale
`PhoneFrame`, `DesktopFrame`	Cascas de aparelho **do clone**, publicadas pelo pacote. Servem ao Storybook e a telas soltas.
`DispositivoFrame` (hub)	**Dentro da plataforma, quem desenha o aparelho é o hub**, dirigido pelo `viewport` do manifesto. A experiência embedada preenche o `iframe` e não traz casca própria.

14.6 Tema

Seção	Como recebe o tema
Clone (`unitube`)	Lê `data-uni-theme` na raiz do documento.
Plataforma (`unooria`)	Recebe `theme` por **prop**.

ATENÇÃO

Quem troca o tema escreve nos **dois** (`exp-02/src/lib/tema.ts`), senão o estudante escolhe escuro no *feed* e o ranking volta branco — porque o ranking é da seção da plataforma, que não lê o atributo da raiz. Na exp-02 a escolha é do estudante (Perfil →

APARÊNCIA) e é lembrada em `localStorage`, que a regra 4 proíbe para *token*, não para preferência de tela.

14.7 Versão: atualizar é ato deliberado

ATENÇÃO

`^0.1.11` **não** pega `0.2.0`: *caret* em `0.x` só sobe *patch*. "Última versão" é, portanto, ativo:

```
npm view @unooria/design-system version     # qual e a ultima publicada
pnpm up -r @unooria/design-system --latest  # sobe em ui/hub/exps de uma vez
# commitando o lockfile E os package.json juntos
```

14.8 Compensações locais

REGRA INVIOLÁVEL

Componente que falta no pacote não nasce na experiência. Vira *compensação local* marcada no CSS (comentário `(b) COMPENSAÇÃO`) e um pedido ao `storybook-unooria`. A compensação é **exceção controlada e decrescente**: cada bloco compensa uma lacuna real, anotada no comentário, e sai quando o pacote publicar a correção.

14.8.1 Lacunas abertas, verificadas na 0.1.14

NOTA

As quatro primeiras da tabela abaixo foram **atendidas na 0.1.17** e as compensações correspondentes foram apagadas da exp-02. Duas lacunas novas apareceram na 1.1.0 da experiência e estão registradas logo depois.

Lacuna	Consequência
`unitube` não publica *toast*, diálogo, tabela nem estado vazio	Quatro compensações locais na exp-02.
`ChannelHeader` não tem `showSubscribe`	O botão de inscrever-se é *sempre* desenhado, e o canal da exp-02 é o do **próprio estudante**.
`.uni-channel-header__cover` usa a primitiva `--uni-color-neutral-50`, não um token semântico	No escuro vira uma laje branca.

`styles-unootube.css` repete o `@import` da Roboto no **meio** do arquivo	O Vite descarta com aviso; o do topo continua valendo, então a fonte carrega. É ruído, não bug.
Não existe componente de seleção (só `TextField`, `TextArea`, `PasswordInput`, `SearchInput`)	`<select>` do hub usa `.unooria-select`, com os tokens do pacote — mesma altura, raio e cores do `TextField`.

14.8.2 Lacunas abertas na 0.1.17 (exp-02 1.1.0)

Lacuna	Consequência
`unootube` não publica `Tag` nem `Badge`	Nenhum dos dois arquivos de origem desenha um. A etiqueta “LIMITE ATINGIDO” do diálogo de limite é compensação local — e sai **neutra**, não âmbar como o Figma pede: a seção não tem token de aviso, e derivar um seria inventar valor.
O véu do `ShortsPlayer` tem altura **fixa** de 194 px	Ele existe para a legenda ficar legível. Numa miniatura de 88 px sem legenda (`showChannel false`), o véu é mais alto que a miniatura inteira e só a faixa mais escura do gradiente aparece: o sintoma é **retângulo preto no lugar do vídeo**, com o vídeo carregado por baixo. Compensado com `display: none` nas miniaturas da bandeja e do diálogo. Deveria ser proporcional, ou preso ao `showChannel`.
`Button` do clone não tem variante de ênfase	“Entendi!” e “Salvar” recebem preenchimento por ligação local, com os tokens da própria marca.

14.8.3 Avisos que ficaram obsoletos

> **NOTA**
>
> Dois avisos herdados do repositório de origem estão **obsoletos na 0.1.11**, medidos no pacote:
>
> - o `MetricCard` **não inventa mais tendência** — a prop é `tagText?` e a documentação diz “sem ela a etiqueta não é renderizada”;
> - o `PositiveNegativeBarChart` tem escala **fixa** em ± 100, que é exatamente o que os eixos do circumplexo pedem — então a barra divergente *não* é mais escrita à mão.
>
> O terceiro segue valendo: `AlertBox` é **diálogo** de 436 px; para faixa de contexto, o componente é `FeedbackAlert` (e a prop é `type`, não `variant`).

ATENÇÃO

Sobrescrever classe do pacote exige DOIS seletores (`.app-shell > .x`): o CSS do pacote é injetado pelo módulo do componente, ou seja, *depois* do CSS da experiência no grafo de importação. Com peso igual, quem ganha é sempre o pacote.

14.9 Cor da plataforma: classe escrita à mão

ATENÇÃO

A cor das superfícies da plataforma vem de classes escritas à mão em `packages/ui/styles/plataforma.css`, e o tema do Tailwind está **vazio** (`theme.extend: {}`) — de propósito, para que nada da marca institucional vaze para a superfície do estudante, nem por nome de classe.

A consequência é uma armadilha real: **classe de cor que não existe ali não é erro em lugar nenhum**. Em texto, ela herda a cor e passa desapercebida; em **fundo**, o elemento fica branco no branco. Foi o que aconteceu com `bg-unooria-neutral-900`: a letra da bolha ficou invisível. O *smoke* da exp-70 checa o `backgroundColor` **computado** do círculo por isso — `toBeVisible()` não pega.

Existem hoje apenas: `bg-unooria-{navy-800, neutral-50, neutral-100, neutral-200}`, `border-unooria-neutral-200` e `text-unooria-{danger, navy-900, neutral-500, neutral-700, neutral-800, neutral-950}`.

NOTA

O *preflight* do Tailwind fica **ligado**. Desligá-lo parecia inofensivo — o pacote traz o próprio CSS — mas o pacote estiliza **componentes**, não elementos: sem o *reset*, todo `<a>` volta a ser sublinhado e todo `<ul>` volta a ter marcador. Foi o que o *smoke* visual pegou nos *cards* de produto.

14.10 Exceção registrada

DECISÃO DE PROJETO

A exp-70 está fora da regra do *design system*, deliberadamente: é jogo em *canvas* 3D, sem DOM de interface (**ADR-18**). A pendência da exp-02 — que tinha CSS próprio em `src/estilos` — foi **resolvida**: ela consome a seção `unitube` pela ponte `@edu/ui/ds-unootube`, com a aparência escolhida pelo estudante.

15

Engenharia de entrega

15.1 O *monorepo* e o *build* incremental

DECISÃO DE PROJETO

***Monorepo* único, com pnpm *workspaces* + Turborepo** (ADR-02). A SDK é consumida por 48 aplicações: uma mudança nela precisa testar tudo no *mesmo pull request*, ou a incompatibilidade aparece num *bundle* publicado semanas depois. Repositórios separados só se houver estúdio externo produzindo experiência.

Listing 15.1 – `turbo.json` — o *pipeline* inteiro

```
{
  "tasks": {
    "build":     { "dependsOn": ["^build"],
                   "outputs": ["dist/**", ".next/**", "!.next/cache/**"] },
    "dev":       { "cache": false, "persistent": true },
    "lint":      { },
    "typecheck": { "dependsOn": ["^build"] },
    "test":      { "dependsOn": ["^build"] }
  }
}
```

O `dependsOn: ["^build"]` é o que faz o grafo funcionar: `typecheck` de uma experiência exige o `build` da SDK, então o *filtro* por mudança arrasta as dependências corretas.

Comando	Uso
`pnpm turbo build --filter=@edu/exp-01`	Builda uma experiência mais as dependências dela.
`pnpm turbo build --filter='...[HEAD^]'`	Só o que mudou. É o que o CI usa.
`pnpm turbo typecheck lint test`	O portão local da *Definition of Done*.

15.2 Integração contínua

Listing 15.2 – `.github/workflows/ci.yml` (abreviado)

```yaml
name: ci
on:
  # Todo PR: de branch de trabalho para dev, de dev para hml e, quando
  # producao existir, de hml para main — o mesmo portao nas tres pontes.
  pull_request:
  # Chamado pelo deploy.yml no push para dev/hml. NAO tem `push` proprio:
  # rodaria duas vezes no mesmo commit, e o deploy precisa ESPERAR por ele —
  # dois workflows em paralelo deixariam um teste vermelho subir para escola.
  workflow_call:
    inputs:
      completo:   # `forcar` no deploy builda tudo; o portao cobre o mesmo
        type: boolean

jobs:
  verify:
    runs-on: ubuntu-latest
    services:
      postgres:   # 9 das 23 suites da api sao de INTEGRACAO (RLS, locks)
        image: postgres:16
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 2 }
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 24, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm --filter @edu/api migrate:deploy
      - run: |
          if [ "$COMPLETO" = "true" ]; then
            pnpm turbo typecheck lint test build
          else
            pnpm turbo typecheck lint test build --filter='...[HEAD^]'
          fi
      - name: Snyk (trava em high)   # high/critical em dependencia = vermelho
        env: { SNYK_TOKEN: "${{ secrets.SNYK_TOKEN }}" }
        run: |   # o --exclude e obrigatorio: o passo roda DEPOIS do build
          npm install -g snyk
          snyk test --all-projects --dev --severity-threshold=high \
            --exclude=.next,.next-prod,dist,build,storage,storage-privado,jogo
```

DECISÃO DE PROJETO

O CI é chamado pelo *deploy* por `workflow_call`, e **não** tem gatilho próprio em `push`. Com gatilho próprio, ele rodaria duas vezes no mesmo *commit* — e, pior, em *paralelo* com o *deploy*, o que deixaria um teste vermelho subir para uma escola. Assim o *deploy* **espera** pelo portão. O gatilho de `pull_request` não filtra a *branch* de destino de propósito: o portão é o mesmo nas três pontes do fluxo (*branch* de trabalho → `dev`, `dev` → `hml`, `hml` → `main`).

15.2.1 A trava de vulnerabilidade (Snyk)

O portão termina com `snyk test --all-projects --dev --severity-threshold=high`: vulnerabilidade `high` ou `critical` em qualquer dependência do *workspace* deixa o CI vermelho — e, como o *deploy* espera pelo portão, nada sobe (**ADR-39**). O passo roda sempre no *workspace* inteiro, sem o filtro por mudança do resto do portão: o *lockfile* é um só, e a vulnerabilidade nova chega pelo banco de dados do Snyk, não por um *commit* — "dependência que não mudou" não existe como categoria.

O `--dev` não é detalhe: sem ele o Snyk pula as `devDependencies`, e foi por aí que um *critical* no `vitest` passou verde na primeira semana da trava (achado pelo `pnpm audit`, que olha tudo). Ferramenta de desenvolvimento não embarca no *bundle*, mas roda na máquina de quem desenvolve e no *runner* do CI — e é lá que um *dev server* furado vaza código.

> **NOTA**
>
> **O que exige *major* de *framework* não bloqueia — expira.** O `apps/api/.snyk` ignora, com **prazo e motivo**, as vulnerabilidades cujo *fix* é o Nest 11 (`@nestjs/core`, e o `file-type` que é ESM-only e quebraria o `require` do Nest 10) ou um *bump* que só o Prisma pode fazer (`deepmerge-ts`). Quando o prazo vence sem o *upgrade*, o próprio passo volta a falhar — é o prazo **com** a rotina que o cumpre, a resposta à armadilha do `expires_at`. O resto foi consertado na hora: o hub subiu para o **Next 16** (as *criticals* do 15 não tinham *patch* na linha 15.x), e `pnpm.overrides` no `package.json` da raiz cobre o transitivo (`next` 16.3.3 por baixo do *design system*, `multer` 2.2.0, `js-yaml` 4.3.1, `lodash` 4.18.1).

O segredo `SNYK_TOKEN` vem dos *secrets* do repositório (`snyk config get api | gh secret set SNYK_TOKEN`). Ausente, o passo **falha alto** — nunca pula, pela mesma regra do *guard* do Pub/Sub: env ausente é porta fechada, não porta aberta.

15.3 Entrega contínua

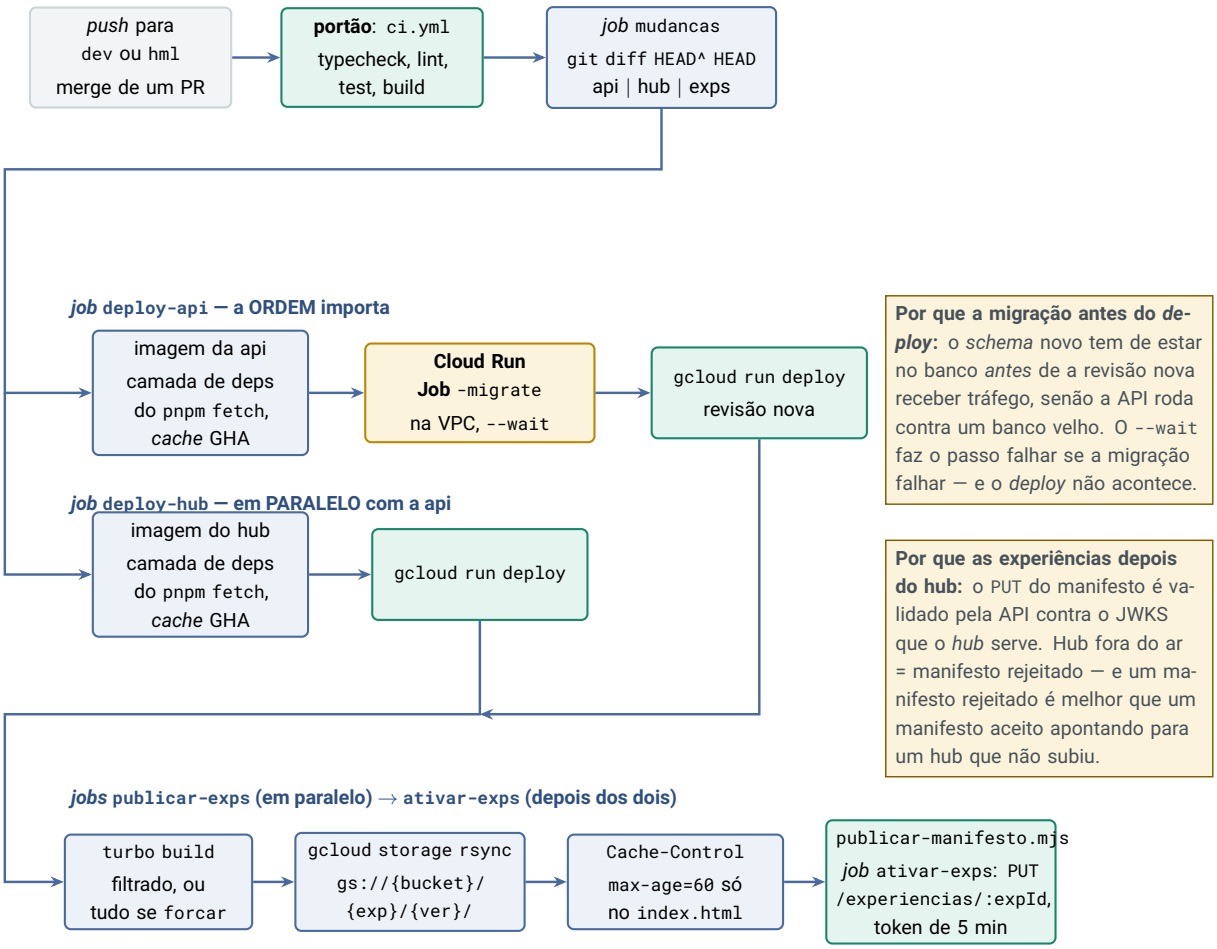


Figura 15.1 – O *pipeline* de entrega. O portão e o *job* `mudancas` decidem o que roda; api e hub sobem em *jobs* paralelos (cada um autentica por WIF, sem chave JSON); as experiências publicam por último, depois dos dois.

15.3.1 O que faz o *deploy* caber no café

DECISÃO DE PROJETO

As imagens de api e hub sobem em *jobs* paralelos, e a camada de dependências dos *Dockerfiles* é cacheável por si (ADR-38). Medido nos *runs* de 2026-08-28: as duas imagens eram 80% do tempo de *deploy* — 4m53s a api num *push* comum, mais 6m30s o hub em série num `forcar` — porque `COPY . .` vinha *antes* do `pnpm install`, e qualquer mudança de código invalidava a camada inteira. O *cache* GHA que o *workflow* sempre teve só acertava apt e corepack.

Os dois *Dockerfiles* passaram a abrir com uma camada que depende **só do *lockfile***:

Listing 15.3 – A camada de dependências (api e hub)

```
# Camada de dependencias: so invalida quando o lockfile muda.
COPY pnpm-lock.yaml pnpm-workspace.yaml ./
```

```
RUN pnpm fetch
# Codigo por cima; o install resolve do store local, sem rede.
COPY . .
RUN pnpm install --frozen-lockfile --offline
```

`pnpm fetch` existe para isto: baixa o *store* inteiro lendo apenas o *lockfile*, sem precisar dos `package.json`. Num *commit* que só toca código, a camada cara vem do *cache* e sobra o que é daquele *app*: `nest build` (ou `next build`), `pnpm deploy --prod` e o `prisma generate`.

E como as duas imagens não dependem uma da outra, cada uma virou um *job* — `deploy-api` e `deploy-hub` — atrás do mesmo portão, com o *diff* calculado uma única vez num *job* `mudancas`. A ordem que **importa** continua explícita no grafo, agora em duas partes: SUBIR o *bundle* não depende de nada além do portão, porque o caminho carrega a versão (`{expId}/{versao}/`) e ninguém pede por ele até o manifesto mudar — o *upload* é inerte. ATIVAR depende dos dois *deploys*, e isso não é preferência: a api valida o *PUT* contra o JWKS que o hub serve. Separar os dois encurta a janela em que o hub novo conviveria com o *bundle* antigo: de um `turbo build` inteiro para a duração de um *PUT*. E dentro de `deploy-api` a migração continua antes da revisão nova. A única ordem que se perdeu foi acidental: o hub não espera mais a api — os dois conversam por HTTP e o *rollout* do Cloud Run nunca garantiu revisão casada mesmo.

15.3.2 Nada de nome de recurso no *workflow*

DECISÃO DE PROJETO

Tudo que muda de um ambiente para outro entra por `vars` de ***environment*** do GitHub (`dev`, `hml` e, no futuro, `prd`) — o *workflow* **não sabe** o nome de nenhum recurso. Cada *job* de *deploy* declara `environment:` com o nome da *branch* que disparou o *push* (`main` mapeia para `prd`), e é esse vínculo que resolve as `vars` da *stack* certa. Os valores saem de `terraform output` do *workspace* correspondente, e há um *script* que os grava no *environment* certo (`scripts/configurar-github-ci.sh`) — ver a **ADR-53**.
É isso que faz o mesmo *workflow* servir a `dev`, a `hml` e à produção, e ao próximo contrato, sem *fork*.

ATENÇÃO

As `vars` de *environment* **só resolvem dentro de um *job* que declara `environment:`** — um bloco `env:` no nível do *workflow* as receberia vazias, sem erro. Por isso o `deploy.yml` não tem `env:` global: cada *job* de *deploy* declara o *environment* e as próprias variáveis. E referenciar um *environment* que não existe o **cria em silêncio**, com todas as `vars` vazias — o *job* `mudancas` tem um passo de guarda que derruba um `workflow_dispatch` disparado de *branch* que não é de ambiente.

	Conteúdo
`GCP_PROJECT`, `REGION`	Projeto e região.

`AR_REPO`	Artifact Registry.
`API_SERVICE`, `HUB_SERVICE`, `MIGRATE_JOB`	Nomes dos serviços e do *job*.
`EXP_BUCKET`, `ASSETS_BASE_URL`	*Buckets* e base pública de *assets*.
`API_URL`, `HUB_ORIGIN`	Endereços públicos.
`WIF_PROVIDER`, `DEPLOY_SA`	Identidade federada do CI.
`CLARITY_PROJECT_ID`	Id do projeto Microsoft Clarity (**ADR-60**). Única `vars` que **não** sai de `terraform output`: é criada à mão em cada *environment*.

ATENÇÃO

`CLARITY_PROJECT_ID` entra na *imagem*, não no serviço. É a única variável de todo o *deploy* que chega ao *bundle* do **cliente**, e `NEXT_PUBLIC_*` é substituída pelo literal durante o `next build` — defini-la no Cloud Run depois não altera um *bundle* já compilado. Por isso o passo *Imagem do hub* a passa como `build-arg`, e o `apps/hub/Dockerfile` a recebe num `ARG` antes do `turbo build`. As duas consequências: trocar o id exige *rebuild* (não basta reiniciar o serviço), e uma imagem passa a ser específica do ambiente que a construiu — promover `hml` para produção reconstrói, não repromove a mesma *tag*. Ausente, o `ARG` cai no *default* vazio e o hub sobe com o Clarity desligado, sem erro.

15.3.3 Concorrência e reexecução

Listing 15.4 – A política de concorrência do *deploy*

```yaml
# Dois deploys ao mesmo tempo brigam pela mesma revisao do Cloud Run e pelo
# mesmo banco. `cancel-in-progress: false` de proposito: abortar no meio de
# uma migration deixa schema e imagem fora de sincronia — melhor enfileirar.
concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false
```

NOTA

Há um `workflow_dispatch` com *input* `forcar`, para publicar tudo ignorando o *diff* — primeiro *deploy* de um contrato, ou reexecução depois de falha. Com `forcar`, o *build* também tem de ser de **todas** as experiências, senão o laço de publicação encontra pasta sem `dist/`.

> **NOTA**
>
> O laço de publicação é tolerante por desenho: pasta de experiência sem `package.json`, sem `public/experiencia.json` ou sem `dist/` é **pulada com aviso**, não é erro. Experiência incompleta é trabalho em andamento, e trabalho em andamento não deve quebrar o *deploy* do resto.

15.4 Ambientes locais

Modo	Comando	Quando usar
Banco em contêiner, resto no *host*	`docker compose up -d pg` + `pnpm dev`	Iteração rápida com *debugger* do editor. O caminho mais direto.
Tudo em contêiner, sincronizado	`pnpm docker:dev`	**Recomendado.** Banco, API, hub e as seis experiências, cada um com o próprio *watcher*, com o código do *host* por *bind mount*: salvar um arquivo recompila lá dentro (API ~1 s, hub e Vite na hora), sem `build` e sem `up` de novo.
Imagens congeladas	`docker compose up -d --build`	Aproximar-se da produção. Migrações e *seed* rodam sozinhos no primeiro `up`.

> **ATENÇÃO**
>
> As portas são as mesmas nos três modos (hub `:3000`, API `:3001`, experiências `:5173`–`:5177`), então **não dá para rodar `pnpm docker:dev` e `pnpm dev` ao mesmo tempo** — é um *ou* o outro.

Situação	O que fazer
Mexeu em dependência (`package.json` ou *lockfile*)	`pnpm docker:dev:deps` e reiniciar. O `node_modules` de cada contêiner é **volume próprio** — binário de macOS não roda no Linux da imagem — então instalar no *host* não basta.
Salvou um arquivo e nada recompilou	`DEV_POLLING=1 pnpm docker:dev`. O *inotify* pode não atravessar o *bind mount* em alguns *setups*.

15.5 O provedor de identidade sobe à parte

Listing 15.5 – O WSO2 tem ciclo de vida próprio

```
docker compose -f wso2/docker-compose.yml up -d     # IS :9443, banco dele :5433
docker compose -f wso2/docker-compose.yml watch     # olha conf/ e bootstrap/
```

> **NOTA**
>
> O `watch` é o equivalente do *hot reload* para um servidor Java: `conf/` já entra por volume (nada de *rebuild*), e mudar o `deployment.toml` **reinicia** o servidor — que é o ciclo mais curto possível ali, porque o *entrypoint* só copia o volume de configuração para o *server home* no *boot* (∼2 min). Mudar `bootstrap/` reexecuta o *bootstrap* (idempotente); só mudar o `Dockerfile` (o *driver* JDBC) refaz imagem.

15.6 Convenções de *git*

Convenção	Regra
Fluxo	*Branch* de trabalho → PR para `dev` → PR de `dev` para `hml` → (futuro) PR de `hml` para `main`, que será a produção. **Ninguém commita direto em *branch* de ambiente**: o *merge* do PR é o que dispara o *deploy* da *stack* correspondente.
Merge	*Merge commit* ou *squash*; **rebase fica desabilitado** nas opções do repositório. O *job* `mudancas` compara `HEAD^` (o primeiro pai), que num *merge commit* é a ponta antiga da *branch* de ambiente — um *rebase-merge* faria o *diff* enxergar só o último *commit* do PR.
Branches	**Uma por experiência**, com o nome do slug (`exp-07-fracoes`). Trabalho que não é experiência usa o prefixo do tipo do *commit*: `fix/quota-por-perfil`, `chore/deps`, `ci/deploy`.
Commits	`feat(exp-12): ...`, `fix(api): ...`, `chore(sdk): ...` — o escopo é o slug da experiência ou o aplicativo tocado. Validado pelo *hook* `.githooks/commit-msg`.
Promoção	O PR de `hml` para `main` leva **tag semver** (`v1.4.0`) e *release*, criadas depois do *merge*. MAJOR só em quebra de contrato externo (`PROTOCOL_VERSION` da ponte, manifesto, telemetria, contrato do Terraform); MINOR é o caso comum. A *tag* é o nome do que está em produção — sem ela, "voltar para a anterior" vira caça a *SHA* no meio de um incidente.
Hotfix	Sai de `main`, entra por PR em `main`, leva *tag* de PATCH — e volta para `hml` e `dev` por **retro-merge** no mesmo dia. *Hotfix* que não volta renasce como *bug* na promoção seguinte.
Imposição	`branch protection` e `rulesets` respondem **403** (repositório privado, plano *Free*). As regras vivem em `.githooks/` — `pre-push` recusa *push* direto em *branch* de ambiente e nome fora do padrão — instaladas pelo `prepare` do `pnpm install` (`core.hooksPath`). Ver **ADR-56**.

> **NOTA**
>
> O fluxo completo, com os comandos de cada ponte (promoção, *hotfix*, *retro-merge*, *tag*, *rollback* por camada), está em `CONTRIBUTING.md`, na raiz do repositório.

ATENÇÃO

Nada de *build* no *git*. O `.gitignore` cobre `dist/`, `.next/`, `.next-prod/`, `.turbo/`, `*.tsbuildinfo`, `storage/`, `storage-privado/` e os *prints* das suítes de ponta a ponta. `apps/hub/.next-prod/` já entrou uma vez por um `git add -A` distraído: 358 arquivos e 250 MB de *cache* de *webpack* no histórico, que só saíram reconstruindo a *branch* inteira. **Stage por caminho, nunca `-A` sem olhar o `git status`.**

NOTA

`apps/hub/next-env.d.ts` **é** versionado (o Next espera isso), mas alterna entre `./.next` e `./.next-prod` conforme o último *build*. Modificado depois de um `build:isolado` = descartar, não *commitar*.

16

Qualidade

16.1 A *Definition of Done*

REGRA INVIOLÁVEL

Nenhuma tarefa está concluída sem estes seis itens. É o portão que substitui a revisão em par que uma pessoa não tem.

1. ***Typecheck* + *lint* + testes passando** (`pnpm turbo typecheck lint test`).
2. **Se mexeu em tela: a suíte de ponta a ponta do papel afetado** (`node scripts/e2e/rodar.mjs <papel>`) — *typecheck* verde não pega ícone virando texto, *link* sublinhado nem `iframe` vazio. Tela nova é passo novo na suíte, com `titulo`/`mostra`/`prova` preenchidos: são eles que viram slide e legenda de vídeo.
3. ***Input* externo validado com Zod.**
4. ***Queries* novas escopadas por *tenancy*** (revisar mentalmente a regra 1).
5. **Sem segredo nem URL interna em código de cliente.**
6. **Se tocou contrato** (ponte, telemetria, *provider*, manifesto): **documentação atualizada** — em `AGENTS.md` e neste documento.

16.2 Estratégia de testes

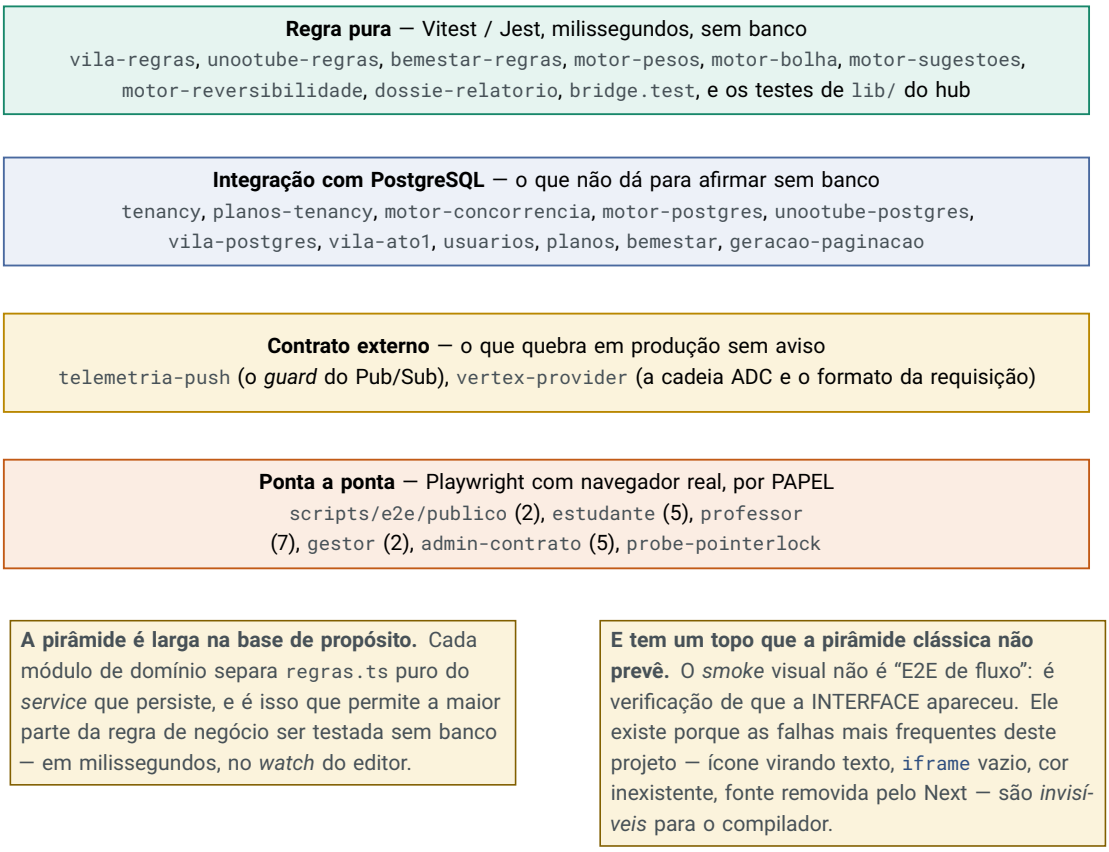


Figura 16.1 – As quatro camadas de verificação. A quarta é incomum e é a mais importante deste projeto: ela pega a classe de falha que o *typecheck* estruturalmente não vê.

> **NOTA**
>
> **Caso registrado (2026-08-27).** O *smoke* da exp-02 foi o único verificador a pegar o `hub:pause` espúrio do `ExpFrame.tsx` (**ADR-22**): 31 tarefas de `typecheck`, `lint` e teste passavam, e o que denunciou foi uma linha do relatório — "autoplay BLOQUEADO". O defeito já existia antes da mudança que o expôs; nenhuma camada abaixo tinha como vê-lo, porque ele mora no encontro entre foco de janela, *postMessage* e política de mídia do navegador.

Camada	Ferramenta	Mínimo obrigatório
Regra pura	Vitest (SDK, experiências) / Jest (NestJS)	Contratos: ponte, telemetria, *guards* de *tenancy*. Toda função de `regras.ts`.
Integração	Jest + PostgreSQL de teste	*Tenancy*, *lock*, concorrência, idempotência.

Contrato externo	Jest com *fake* de rede	*Guard* do Pub/Sub e formato da requisição ao Vertex.
Smoke visual	Playwright, navegador real	Toda mudança de tela.

16.3 A suíte de ponta a ponta, em detalhe

Ela vive em `scripts/e2e/`, organizada por PAPEL — `publico` (2 suítes), `estudante` (5), `professor` (7), `gestor` (2) e `admin-contrato` (5). Por papel, e não pelo nome da experiência, porque é o crachá que decide o que a pessoa alcança, e porque a organização antiga escondia as telas da PLATAFORMA, que são a maioria (ADR-51, Seção 18.52).

Listing 16.1 – A suíte de ponta a ponta

```
pnpm exec playwright install chromium    # uma vez por maquina. NAO `npx playwright install`:
                                        # o npx baixa a build de navegador de OUTRA versao
node scripts/e2e/rodar.mjs               # tudo: publico, estudante, professor, gestor, admin
node scripts/e2e/rodar.mjs professor     # so um papel
node scripts/e2e/rodar.mjs estudante/02  # so uma suite
node scripts/e2e/rodar.mjs --deck        # tudo + deck (.pptx) + videos (.webm)
node scripts/probe-pointerlock.mjs
```

ATENÇÃO

Duas coisas exigem o **Chrome do sistema**, não o Chromium do Playwright: a suíte da exp-70 e o *probe* de *pointer lock*. O Chromium empacotado **não concede *pointer lock* nem no topo da página**, então usá-lo produziria um falso negativo — o teste diria que a permissão de *sandbox* não funciona quando o problema é o navegador.

NOTA

As suítes precisam da plataforma no ar (contêineres ou `pnpm dev`) e gravam *prints* na pasta indicada — que está no `.gitignore`. A senha de todos os usuários de demonstração é `unooria123`; os *logins* do *seed* são `gestor.demo`, `prof.demo` e `estudante01`–`estudante40`. Nenhum roteiro fixa *login* no código: eles saem de `credenciais-demo.csv`, que o *seed* escreve — fixá-los fazia qualquer mudança no *seed* quebrar seis scripts com “timeout esperando o feed”, que não diz que a conta mudou de nome.

16.3.1 O manifesto: a descrição mora no teste que fotografa

Cada passo é registrado com `r.registrar(pagina, nome, {titulo, mostra, prova, detalhe})`, e isso vira um `passos.json` por suíte, ao lado dos *prints*. O *deck* de evidências (`.pptx`) e os vídeos (`.webm`, um por papel e um por missão do estudante) são gerados DELE — os geradores não conhecem o produto, só desenham.

Antes existia um roteiro de 66 KB mantido à mão, separado dos scripts, e ele divergiu: descrevia telas que já não eram fotografadas e omitia passos novos. É a mesma armadilha do `expires_at` — o registro fica, a regra sai, e quem lê acredita nele.

16.3.2 Erro provocado não é erro descoberto

Quando um roteiro exercita uma trava de propósito — ativar uma atividade com conteúdo de menos (422), subir um perfil portátil adulterado (400), abrir a análise de produto com o papel errado (403) — o erro que aparece na tela é a PROVA de que a trava existe. Ele é declarado com `r.provocado(padrão, motivo)` e sai numa seção própria do relatório.

Sem isso, o vermelho aparecia justamente na corrida que demonstrava o comportamento correto — e vermelho que não significa nada treina a pessoa a ignorar o vermelho, que é o único jeito de um *smoke* deixar de servir.

16.3.3 O roteiro tem de PERDER a corrida para o erro

Peças compartilhadas em `scripts/e2e/lib/` — `roteiro.mjs` (a casca: navegador, *login* com portão de hidratação, *print* com manifesto, asserções e relatório) e `smoke.mjs` (as esperas que correm contra o erro). Antes cada roteiro carregava sua própria cópia, e as doze cópias divergiram: uma tirava `fullPage` e outra não, três reprovavam por erro de console e nove não.

Medido em 2026-08-29: ao gerar um plano de aula, a API falhou em **306 ms** e o roteiro ficou **180 s** esperando uma `<table>` que nunca viria — 588 vezes o tempo que a resposta levou. O erro estava escrito em vermelho na tela desde o primeiro segundo, e o que saiu no console foi `Timeout waiting for locator('table')`: um sintoma que não aponta para a causa, dentro do próprio instrumento que existe para encontrá-la.

Os 3 minutos são o menor dos problemas. **Nenhum roteiro olhava estado de erro** — nenhuma ocorrência de `FeedbackAlert` em `scripts/`. Uma falha que ainda deixasse uma `<table>` na tela (plano cortado por `MAX_TOKENS` caindo num *fallback*) passaria **verde**.

Peça	O que faz, e onde vale
`esperarOuFalharCedo`	Corrida entre o sinal de sucesso e o alerta de erro; sobe a mensagem que o professor LEU, não o nome do seletor que faltou. O *timeout* continua alto (a geração real leva de 30 a 60 s) — o que muda é que o caminho de falha não o paga.
`ALERTA_DE_ERRO`	`.unooria-feedback-alert--error`. É a CLASSE, não `[role="alert"]`: o *role* é o mesmo para *info*, *warning* e *success*, e esperar por ele faria um aviso benigno derrubar o roteiro. Vale para quem fala pela PLATAFORMA (exp-90, exp-91).

`vigiarFalhasDaApi`	Anota respostas 4xx/5xx para o relatório, com rótulo do contexto. É o sinal das experiências DISFARÇADAS (exp-01, exp-03), que mostram erro em `<p>` avulso, com classe diferente em cada tela, e por isso não têm gancho estável. **Não derruba o roteiro**: um 404 de sondagem é esperado, e derrubar por *status* inventaria falha. Para 5xx e 422 ele anexa o CORPO — "500 POST /destilar" manda procurar no *log* da API; "422 … 12 posts aprovados, o recomendado é 300" resolve na hora.
`esperarOuFalharPorHttp`	O irmão do primeiro para quem não tem alerta no DOM: corre entre o sinal de sucesso e a RESPOSTA de erro da rota, e põe o corpo dela na mensagem. Foi ele que trocou "180 s esperando *Perguntas propostas pela IA*" por "9 s, HTTP 500 em /destilar".

> **ATENÇÃO**
>
> Os dois mecanismos são complementares, e o caso da exp-90 mostra por quê: o `POST /planos` devolve **202**, e o erro só aparece no CORPO do *polling*. Um vigia de *status* HTTP não o pegaria — ali o sinal é o alerta no DOM. Na exp-03 é o contrário: a rota devolve **503** e não há classe estável para esperar.

16.3.4 Duas condições para o vermelho significar alguma coisa

Idempotência. O roteiro do estudante cobre *login* → **onboarding** → *feed*, e o *onboarding* só acontece uma vez: na segunda execução a criança cai direto no *feed*, a espera pelo texto da tela de entrada estoura em 30 s, e o relatório diz *iframe* vazio quando o *iframe* está perfeito, só adiantado. Ele passou a zerar a turma no começo, como o da explicabilidade e o da exp-03 já faziam. ***Smoke* que só passa uma vez por banco não é *smoke*** — e o sintoma dessa falha é indistinguível de um defeito de produto.

Ruído de dev não pode pintar de vermelho. O `performance.measure(routeChangeToRender, …)` de `next/dist/client/index.js` pode receber *timestamp* negativo numa navegação cliente e lança `Failed to execute 'measure' on 'Performance'`. Aparece em 11 dos 12 roteiros; como o da exp-02 e o da exp-03 reprovam por QUALQUER erro de console, os dois viviam vermelhos por um defeito que não é nosso — e vermelho permanente treina a pessoa a ignorar o vermelho, que é o único jeito de um *smoke* deixar de servir. A lista de ruído (`ehRuidoDeDev`) é explícita, curta, e quem filtra **conta e diz quantos ignorou**: silenciar por padrão devolveria o problema em outra forma.

16.3.5 O que só o *smoke* visual pega

Falha	Por que o *typecheck* não vê

Ícone virando texto	O Next remove `@import` externo de CSS importado por JS, *silenciosamente*. `Material Symbols Rounded` é fonte de ligadura, então o sintoma é "addInvestigação" no lugar do ícone. O `getComputedStyle` *responde* "Open Sans" — a regra continua no CSS.
Cor inexistente em fundo	Classe de cor que não existe em `plataforma.css` não é erro em lugar nenhum. Em texto herda a cor; em **fundo** o elemento fica branco no branco. O *smoke* da exp-70 checa o `backgroundColor` **computado** — `toBeVisible()` não pega.
`iframe` vazio	*Handshake* que expira, *dev server* fora do ar, manifesto ausente: o componente monta, o `iframe` existe, e não há nada dentro.
Experiência esticada na janela	`viewport` não lido (manifesto vazio + `EXP_INTERNAL_HOST` errado). Nenhum tipo muda.
Link sublinhado, marcador de lista de volta	*Preflight* do Tailwind desligado. O pacote estiliza *componentes*, não elementos.
Câmera do jogo que não anda	`allow-pointer-lock` ausente: `requestPointerLock()` morre com `SecurityError` e nada no console diz por quê.

> **ATENÇÃO**
>
> **Testes de navegador precisam de portão de hidratação.** Em desenvolvimento, o SSR chega *antes* do *bundle*, e o `fill` do Playwright não move o estado do React — o campo mostra o texto e o `onChange` nunca rodou. Toda suíte espera um sinal de hidratação antes de interagir.

16.4 Análise estática

Regra	Aplicação
TypeScript *strict* em tudo	`tsconfig.base.json`, herdado por todos os *workspaces*. Inclui `noUncheckedIndexedAccess` — é por isso que o código de curadoria usa `!` em índices provadamente válidos, com comentário.
Sem `any`	Use `unknown` + *narrow*. Uma exceção, registrada em ADR: `exp-70/public/jogo/**` está nos `ignores` do ESLint.
Zod em todo *input* externo	Corpo, *query*, `postMessage` recebido, variáveis de ambiente.
ESLint	`eslint.config.mjs` na raiz, com `typescript-eslint`, `eslint-plugin-react-hooks` e `@next/eslint-plugin-next`.

> **ATENÇÃO**
>
> O ESLint **ignora a pasta `storage`**. Um módulo com esse nome passaria no *lint* sem ser lido — foi por isso que o módulo de armazenamento de objetos se chama `object-storage`, e não `storage`.

16.5 Armadilhas de ferramenta registradas

Cada linha abaixo é um erro efetivamente cometido, com o sintoma que ele produz. Estão aqui porque o sintoma nunca aponta para a causa.

Armadilha	Sintoma, e o que fazer
Build isolado apagando o `.next` do *dev*	Sintoma: "tela branca" ou "CSS bugado" depois de validar um *build* de produção. Validar sempre com `pnpm build:isolado` (`HUB_DIST_DIR=.next-prod`).
Standalone do Next não copia `static` nem `public`	Sintoma: CSS 404 e página sem estilo. Distinguir do caso acima pelo **código** do erro, não pela aparência.
Substituição de texto sem asserção	Edição por *replace* que não casa passa em **silêncio**, e o `tsc` não acusa. Sempre afirmar o casamento.
Arquivo em `public/` escrito em *runtime*	Dá 404: `public/` é do *build*. E `Cache-Control` de rota dinâmica só cola via `next.config`.
Probe de *hot reload* como comentário	O *esbuild* remove comentários — o falso negativo faz parecer que o *inotify* não atravessa o *bind mount*.
`sanitize-html` recente é ESM-only	Fixar `~2.13.1` na API (CJS, Node 20).
Design system quebrando o *dev server* do Vite	ESM importando diretório: o *build* passa, o *dev* morre. Sintoma: `iframe` vazio ou `ERR_CONNECTION_REFUSED`.
Vite pulando de porta	Uma segunda instância da mesma experiência ocupa a porta seguinte — e o hub passa a ler o manifesto da experiência errada.

16.6 Cobertura: onde está e onde não está

Área	Cobertura	Observação
Tenancy e RLS	✓ Forte	Teste com *role* sem *ownership*; o caso mais crítico do sistema.
Regra de domínio	✓ Forte	`regras.ts` puro em todos os módulos.
Concorrência do motor	✓ Forte	*Lock* consultivo, reversibilidade, paginação da geração.

Contrato da ponte	✓ Forte	*Origin*, esquema, versão maior.
Guard do Pub/Sub	✓ Forte	Inclui o caso *fail closed*.
Provider de IA	✓ Parcial	Cadeia ADC e formato da requisição. Não há teste contra o Vertex real — e não deveria haver.
Telas	✓ Ponta a ponta	Vinte e uma suítes, cinco papéis; não é cobertura de interação, é verificação de que apareceu.
Carga	**– Ausente**	O teste de 4.000 *logins* em dez minutos é obrigatório antes de novembro de 2026 e não foi executado. Risco `R-01`.
Segurança externa	– Ausente	Sem revisão externa nem teste de intrusão. Risco `R-06`.
Acessibilidade	– Ausente	Nenhuma verificação automatizada de contraste, foco ou leitor de tela. Risco `R-07`.

LIMITAÇÃO CONHECIDA

Não há métrica de cobertura de linha configurada, e não há meta numérica. A escolha é deliberada: com uma pessoa desenvolvendo, uma meta de percentual incentiva testar o que é fácil em vez do que é caro se falhar. O critério em uso é o da *Definition of Done* — contratos, *tenancy* e regra pura — e ele é verificável por inspeção da lista de arquivos de teste.

17

Observabilidade

Este capítulo é um estudo com recomendação: levanta o que a plataforma revela de si mesma hoje, define o que ela precisa revelar para ser operada por uma pessoa em novembro de 2026, compara as ferramentas que fazem isso — com preferência declarada por Google Cloud e por software livre — e fecha num roteiro em fases, com custo estimado por contrato. As duas decisões que ele recomenda estão registradas como **ADR-57** e **ADR-58**, no estado *proposta*, e o risco `R-05` passa a apontar para aqui.

> **DECISÃO DE PROJETO**
>
> **Telemetria não é observabilidade.** O Capítulo 10 descreve o instrumento *pedagógico*: eventos xAPI do estudante, gravados para análise do que ele fez na experiência. Este capítulo trata do instrumento *operacional*: o que a plataforma emite sobre *si mesma* — latência, erro, saturação, custo de IA, fila — para quem a opera. Os dois usam a palavra "evento" e os dois passam pela API, e é justamente por isso que a confusão é frequente (MAJORS; FONG-JONES; MIRANDA, 2022). Um mede a criança; o outro mede o software. Nenhum substitui o outro, e a regra 6 (nada de PII em log) vale igual para os dois.

17.1 O que se vê hoje

O levantamento foi feito sobre o código em `dev` em 1º de setembro de 2026, com busca por biblioteca, por padrão de código e por configuração de implantação. O resultado é o mesmo em todas as camadas: **não há nenhuma biblioteca de observabilidade instalada em nenhum pacote do *monorepo*** — nem OpenTelemetry, nem `pino`/`winston`, nem `prom-client`, nem Sentry, nem `web-vitals`. O que existe é o que a infraestrutura dá de graça e o que a suíte de ponta a ponta prova quando alguém a roda.

Camada	O que existe	O que falta, e onde se vê
API	`ConsoleLogger` padrão do Nest em **texto**, quatro `Logger` nomeados e 19 `console.*` com prefixo manual. `GET /healthz` (processo) e `GET /readyz` (`SELECT 1`). Disciplina manual da regra 6, com comentário em cada ponto sensível.	Sem JSON ⇒ cada linha vira `textPayload` sem severidade. Sem *request-id* nem `traceparent` (`tenancy-context.ts` carrega só `escolaId`). Sem log de requisição na aplicação. Sem `ExceptionFilter`: erro não previsto vira 500 genérico com *stack* no *stdout*. Sem `LOG_LEVEL`. `new PrismaClient()` sem `log` nem métrica. Um desvio da regra 6: `auth.controller.ts` loga o `username` em claro.
Cloud Run	Log de requisição e métricas nativas (contagem, latência, instâncias, CPU, memória) chegam ao Cloud Logging e ao Cloud Monitoring sem configuração.	O `gcloud run deploy` do `deploy.yml` não configura sonda: vale a *startup probe* TCP padrão, e `/readyz` **não é usada por nada**. O `docs/INFRA-IDENTITY.md` manda "nunca usar probe TCP" para o WSO2; a API não segue o próprio conselho.
Hub	`console.*` esparso (`[hub]`, `[chave-token]`); o único log diagnóstico real é `avisarManifestoIlegivel`, escrito depois de um incidente. Nas telas, falha vira `FeedbackAlert` de erro.	Sem `error.tsx`, `global-error.tsx`, `instrumentation.ts`, `reportWebVitals` ou rota de saúde. Sem `HEALTHCHECK` no Dockerfile nem no *compose*. `apiDoServidor` **engole qualquer falha em `null`**, sem log: "API caiu" e "não há dados" são indistinguíveis. Nenhum `fetch` tem *timeout*. Os cinco *route handlers* do BFF não registram nada, inclusive o 404 de rota fora da *allowlist*.
Experiências e SDK	O protocolo da ponte tem `exp:ready`, `exp:concluir`, `exp:estado`, `exp:token-expirando`. O `TelemetryClient` faz lote de 50 a cada 10 s e `sendBeacon` no `pagehide`.	**Não existe `exp:erro`**, nem `window.onerror`, nem `unhandledrejection` em lugar nenhum: um *crash* na exp-70 durante a aula não deixa rastro fora do console do estudante. Falha ao renovar *token* ou ao registrar conclusão é `console.error` no navegador. Sem *sourcemap* nos *bundles*. Fila de telemetria só em memória, sem *backoff*, sem teto, sem `visibilitychange` (a exp-01 registra os três eventos e comenta por quê; a SDK não aplicou a lição).
Pipeline de telemetria	Guarda do *push* falha fechada; `INSERT` multi-linha; *dedupe* por `(event_id, ts)`.	Quatro caminhos de descarte silencioso no *controller*, só com `console.warn`. `recebidos` conta o publicado, não o gravado. Nenhum sinal de atraso da *subscription*, de *dead letter* ou de lote perdido no cliente.

Camada	O que existe	O que falta, e onde se vê
IA	`ia_uso_diario` guarda *tokens* por (escola, usuário, dia) e o `AiResult` já carrega `finishReason`, `voltas` e `tentativas`.	O `VertexProvider` (438 linhas) **não mede tempo e não loga**. Ninguém consome `voltas`/`tentativas`. Custo não é calculado. Degradação deliberada (503 amigável, texto fixo, arte procedural) é boa para o estudante e **invisível** para quem opera: a falha "dá certo" e não conta em lugar nenhum.
Trabalhos em segundo plano	Fila de geração e de planos persistida em tabela, com `status`, `erro` e `tentativas` consultáveis.	A visibilidade é `SELECT` numa tabela: sem alerta, sem taxa de erro, sem *dead letter*. Não há *job* agendado — por decisão — e portanto nenhuma verificação recorrente roda sozinha.
CI/CD	`deploy.yml` separa subir de ativar, roda a migração com `--wait` e reprova o *pipeline* se ela falhar.	**Sem *smoke* pós-*deploy*** e **sem notificação em falha**: o único sinal é o *exit code* do `gcloud`, que volta assim que a revisão recebe tráfego, não quando ela responde. Snyk comentado.
Infra (Terraform)	`docs/INFRA-IDENTITY.md` desenha *uptime checks*, métrica de log, dez alertas com severidade e painel — **só para o WSO2**.	`docs/INFRA.md` não tem **uma linha** sobre alerta, sonda, SLO, retenção de log ou painel para hub, API, CDN, Cloud SQL ou Pub/Sub.
Suíte e2e	`scripts/e2e`: 19 suítes por papel, `esperarOuFalharCedo`, `vigiarFalhasDaApi`, classificação de ruído/pendência/defeito, relatório estruturado e *exit code* honesto.	Só roda com operador. Não corre depois de um *deploy*, não roda em horário, não emite métrica: o melhor detector de defeito da plataforma depende de alguém lembrar de ligá-lo.

LIMITAÇÃO CONHECIDA

O resumo honesto é: **a plataforma é diagnosticável, mas não é observável.** Quem sabe onde olhar reconstrói uma falha a partir do log do Cloud Run e de um `SELECT`; ninguém é avisado de que ela aconteceu, e uma classe inteira de falha — a que acontece dentro do `iframe`, no navegador do estudante — não deixa rastro em lugar nenhum. Para o piloto em `dev`, com uma pessoa olhando, isso bastou. Para cem escolas às oito da manhã, não basta.

ATENÇÃO

Medição de 2026-09-01, na *stack* de `dev`: num dia inteiro de operação, o serviço da API produziu **exclusivamente** entradas no *log* `run.googleapis.com/requests`. Nenhuma linha em `varlog/stdout` ou `stderr` depois do *banner* de subida do contêiner — ou seja, os quatro `Logger` nomeados e os 19 `console.*` da tabela acima **não aparecem no Cloud Logging** como sinal consultável.

A consequência é concreta e já foi encontrada em diagnóstico: quando o Vertex recusa uma geração, o `unoogpt.service` escreve `turno sem resposta: <motivo>` com o

corpo da resposta do provedor — a única pista de *qual* foi o erro (modelo inexistente, papel de IAM faltando, região errada), já que o estudante vê a mesma frase amigável nos três casos. Esse texto não está em lugar nenhum que se possa consultar depois. O efeito colateral perverso é que a **ausência** de erro no *log* não prova ausência de erro no serviço, e foi preciso recorrer ao *log* do *load balancer* — que enxerga o *status* HTTP e a latência, não a razão — para separar "não falhou" de "falhou em silêncio".
Isso reforça a prioridade do log estruturado em JSON (Seção 17.3.1 adiante): o problema não é o formato do texto, é o sinal não existir.

NOTA

O que mudou na API em 2 de setembro de 2026 (fase 1). O `ConsoleLogger` saiu: o Nest loga pelo `nestjs-pino` (`observabilidade/logger.ts`), em JSON com `severity`, `message`, `time` RFC 3339, `serviceContext` e os campos `logging.googleapis.com/trace`, `spanId` e `trace_sampled` preenchidos a partir do contexto da requisição. Esse contexto (`tenancy/tenancy-context.ts`) é aberto por um **middleware do Express** antes dos *guards* — um 401 sai com `x-request-id` como qualquer 200 — e o `TenancyInterceptor` só o preenche com escola e perfil. O `requestId` é o `traceId` que o *load balancer* mandou (`traceparent` ou `X-Cloud-Trace-Context`), então o código que o professor lê no 500 é o que quem opera cola no Logs Explorer. A *redaction* é mecânica (`observabilidade/redaction.ts`): por nome de chave em qualquer profundidade e por padrão na mensagem (e-mail, CPF, `Bearer`, *query string*); o `username` do `auth.controller.ts` deixou de existir no log. O `FiltroDeExcecao` global responde 500 `{codigo, requestId}` com UMA linha `ERROR` e `stack_trace`, e converteu em 400 um caso que era 500 mudo: o `ZodError` que escapava de um `parse(body)` no *handler*, devolvido como `path: code`, sem o valor recebido. Os 19 `console.*` viraram `logger.*` com um campo `codigo` estável (`telemetria.descartado`, `planos.geracao-com-erro`…) — é por ele que se filtra e que se deriva métrica de log. Os descartes do `/telemetry/beacon`, rota sem *guard*, saem em `debug`: uma linha por lote recusado seria vetor de inundação; o sinal contável entra com as métricas da fase 2. O que ainda não existe: *traces*, métricas da aplicação, e tudo do hub e da SDK.

A Figura 17.1 percorre a cadeia de uma requisição e marca, elo por elo, o que se enxerga hoje e o que está cego.

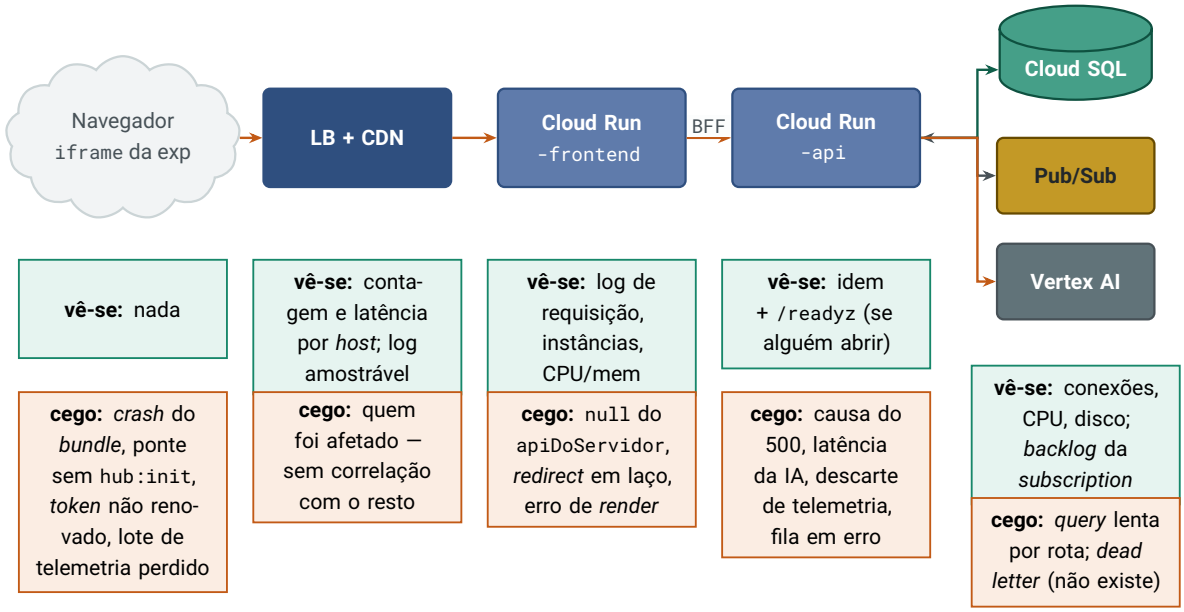


Figura 17.1 – A cadeia de uma requisição e o que cada elo revela hoje. O verde é o que a infraestrutura entrega sem configuração; o laranja é onde a plataforma está cega. O primeiro elo — o navegador, onde a experiência roda — é o mais cego e o que mais importa: é lá que a criança está.

17.2 Princípios

As escolhas deste capítulo derivam de sete princípios, na ordem em que decidem um empate.

1. **Alerta é sintoma, não causa.** Quem é avisado é avisado de que o estudante não consegue abrir a experiência — não de que a CPU passou de 80%. Causa vai para painel e para log; sintoma vai para o telefone (BEYER et al., 2016). A consequência prática é que a lista de alertas é **curta**: uma pessoa que recebe vinte alertas por dia para de lê-los na primeira semana, e a suíte e2e já pagou para aprender isso com o ruído de console (Capítulo 16).
2. **Uma pessoa opera.** Dois níveis de urgência e só dois: *acorda* (indisponibilidade, erro em massa, dado em risco) e *amanhã* (saturação que cresce, custo, fila parada). Nada de escalonamento, rodízio ou ferramenta de plantão enquanto o time é de um.
3. **Dado de menor não entra em sinal operacional.** Log, métrica, *trace* e relato de erro carregam identificadores opacos (`estudante_id`, `escola_id`, `sessao_id`), nunca nome, CPF, *username*, e-mail ou texto livre. É a regra 6 estendida da telemetria para tudo o que sai do processo — e ela precisa ser **mecânica** (*redaction* no *logger*), não convenção.
4. **Nativo do Google Cloud antes de peça nova.** O contrato com o Terraform (Capítulo 6) já proíbe serviço sem necessidade demonstrada. Cloud Logging, Cloud Monitoring, Cloud Trace e Error Reporting já recebem o que o Cloud Run emite; a franquia gratuita cobre um contrato; e um contrato a mais não é uma instância a mais para operar. Uma *stack* auto-

hospedada só entra quando algo que ela faz e o nativo não faz for demonstrado, e isso não aconteceu.

5. **OpenTelemetry é a fronteira.** A instrumentação da API e do hub fala OpenTelemetry (OpenTelemetry Authors, 2026); o destino é o Cloud Trace/Monitoring *hoje*. Trocar de destino (uma rede que exija ferramenta própria, um SIEM, um Grafana) é trocar o exportador, não a instrumentação — o mesmo raciocínio do *provider* de IA (**ADR-07**) e do OIDC genérico (**ADR-03**).
6. **Por contrato, e visão única para quem opera.** Log e métrica ficam no projeto do contrato (isolamento da rede, mesmo argumento da *stack* por contrato). A pessoa que opera vê todos os contratos por um *metrics scope* do Cloud Monitoring num projeto de observação sem recurso nenhum — métrica atravessa projeto de graça; log **não** atravessa, de propósito.
7. **Custo previsível e declarado.** Cada sinal tem volume estimado e franquia ao lado (Seção 17.7). Sinal cujo custo cresce com o tráfego sem teto (log de requisição a 100%, *trace* sem amostragem) é amostrado *na origem*, não cortado na fatura.

17.3 Os sinais, e onde cada um nasce

17.3.1 Log estruturado

O *stdout* do Cloud Run já é o transporte: uma linha que seja JSON válido é lida pelo agente de log como `jsonPayload`, e três campos com nome especial viram metadado de primeira classe (Google Cloud, 2026b): `severity` (o nível), `logging.googleapis.com/trace` (o identificador que junta esta linha ao *trace* e ao log de requisição do próprio Cloud Run) e `logging.googleapis.com/spanId`. Um erro com *stack trace* e `severity: ERROR` é agrupado pelo Error Reporting sem mais nada.

> **O que muda na API**
>
> - `nestjs-pino` substitui o `ConsoleLogger`: `pino` em JSON, com `level` mapeado para `severity` (`warn` → `WARNING`, `error` → `ERROR`, `fatal` → `CRITICAL`) e `messageKey: "message"`. Em desenvolvimento, `pino-pretty`; em produção, JSON cru. Uma dependência de produção e uma de desenvolvimento.
> - **Contexto por requisição** no mesmo `AsyncLocalStorage` da *tenancy* (`tenancy-context.ts`): `escolaId`, `perfil`, `requestId` e `trace`. O *logger* filho lê dali, e toda linha emitida dentro de um *handler* sai com os quatro campos sem que quem escreve o log precise lembrar deles.
> - **Correlação**: o `trace` vem do cabeçalho `traceparent` (W3C (W3C, 2021)) quando existe, e do `X-Cloud-Trace-Context` que o *load balancer* injeta quando não. O valor gravado é `projects/{id}/traces/{traceId}`, que exige o identificador do projeto: entra como `GCP_PROJECT_ID`, injetada pelo Terraform (Apêndice B).

- ***Redaction* mecânica**: a opção `redact` do `pino` apaga, por caminho, `authorization`, `cookie`, `token`, `senha`, `cpf`, `username`, `email`, `nome` e qualquer campo de texto livre (`texto`, `mensagem`, `transcricao`, `observacao`) em qualquer profundidade. É a regra 6 saindo da convenção e entrando na ferramenta; o `username` do `auth.controller.ts` deixa de vazar por construção.
- **Um `ExceptionFilter` global**: `HttpException` mantém o corpo de hoje; erro não previsto (inclusive `ZodError` que escapou) vira 500 com `{ codigo: "erro-interno", requestId }` no corpo e **uma** linha `ERROR` com *stack* e contexto no log. O `requestId` no corpo é o que o professor lê na tela e o que quem opera cola no Logs Explorer — é a ponte entre o relato e o registro.
- **Log de requisição da aplicação: não.** O Cloud Run já grava um por requisição, com método, rota, *status*, latência e `trace`. Duplicá-lo dobraria o volume do sinal mais caro da lista para acrescentar zero informação. O que a aplicação loga é o que o Cloud Run não sabe: o `escolaId`, o motivo de um 4xx, a decisão de degradar.
- **Prisma**: `log: [{ emit: "event", level: "query" }]` ligado só quando `LOG_LEVEL=debug`, e um *listener* que emite `WARNING` para *query* acima de 500 ms em qualquer nível. A *preview feature* `metrics` do Prisma (contadores de pool e histograma de *query*) alimenta a Seção 17.3.2.

No hub, o mesmo `pino` (sem o módulo do Nest) em `lib/log.ts`, usado por `apiDoServidor`, pelos *route handlers* e pelo *middleware*. `apiDoServidor` **continua devolvendo `null`** para não quebrar as páginas que dependem disso, mas passa a registrar `WARNING` com rota, *status* e `requestId` antes de devolver — e ganha `AbortSignal.timeout(10000)`, porque uma API pendurada não pode pendurar o hub junto. O 404 silencioso da *allowlist* do BFF vira `WARNING` com o prefixo recusado: é o sintoma de uma tela nova que esqueceu de pedir passagem.

REGRA INVIOLÁVEL

Exclusão na origem, não na fatura. Dois filtros de exclusão no *sink* `_Default` de cada contrato, criados pelo Terraform: (1) o log de requisição do Cloud Run para `/healthz`, `/readyz` e os *user agents* dos *uptime checks*; (2) amostragem de 20% (`sample(insertId, 0.2)`) do log de requisição com *status* 2xx e latência abaixo de 1 s. Requisição com erro, ou lenta, fica em 100%. Log excluído não é ingerido e não é cobrado, e é o que mantém o sinal mais volumoso dentro da franquia (Seção 17.7).

17.3.2 Métricas

Três origens, do mais barato para o mais informativo:

Origem	O que dá	Custo e esforço

Nativas do Google Cloud	Cloud Run: `request_count` por *status*, `request_latencies` (distribuição), `instance_count`, CPU, memória, latência de *startup*. Cloud SQL: conexões, CPU, disco, *replication lag*, *failover*. Pub/Sub: `oldest_unacked_message_age`, `num_undelivered_messages`, `dead_letter_message_count`. LB: contagem e latência por *host* e classe de *status*; CDN: taxa de acerto.	**Grátis, zero código.** É a base de todos os alertas da fase 0.
Métricas de log	Um contador ou distribuição derivado de um filtro do Cloud Logging: "linhas com `codigo=telemetria-descartada`", "`severity=ERROR` da revisão X", "execução do *job* `-migrate` com falha".	As *de sistema* são grátis; as *definidas pelo usuário* contam na franquia de métrica. Zero código: é um filtro no Terraform.
Métricas da aplicação (OpenTelemetry)	O que só o processo sabe: **IA** — latência por modelo e rota (histograma), *tokens* de entrada/saída, `finishReason`, `voltas`, `tentativas`, recusas de quota; **telemetria** — lotes recebidos, eventos publicados, gravados e descartados por motivo; **fila** — geração e planos por `status`, idade do mais antigo em `gerando`; **pool do Prisma** — conexões em uso e espera; **ponte** — *handshakes* concluídos e expirados (via hub).	Exportadas ao Cloud Monitoring pelo SDK OpenTelemetry com o exportador do Google, agregadas a cada 60 s. Cerca de 60 séries por contrato: cabe na franquia de 150 MiB com folga.

ATENÇÃO

Cardinalidade. `escola_id` **não** é rótulo de métrica: cem escolas multiplicam cada série por cem, e a franquia acaba na primeira semana. Escola vai para o log (que se filtra) e para a telemetria (que se agrega em SQL). Rótulos de métrica são conjuntos **fechados** e pequenos: rota, perfil, modelo, `finishReason`, motivo de descarte, *status* de fila. A mesma regra que a telemetria aplica ao vocabulário do *payload* (**ADR-11**).

17.3.3 Rastros distribuídos

Um *trace* responde à pergunta que nem log nem métrica respondem: *onde* os 1.800 ms de uma abertura de missão foram gastos — no hub, no BFF, na API, no *lock* do *feed*, no Vertex. A instrumentação automática do OpenTelemetry para Node cobre HTTP de entrada e saída (`undici`, portanto o `fetch` do Vertex e o do hub), Express (o Nest), `pg` (o Prisma) e Pub/Sub, sem tocar no código dos módulos. No hub, o `instrumentation.ts` do Next (Vercel, 2026a) registra o mesmo SDK, e o `traceparent` atravessa do navegador ao Cloud SQL.

DECISÃO DE PROJETO

Exportador direto, sem *sidecar* de coletor. O Google recomenda um OpenTelemetry Collector como segundo contêiner do serviço Cloud Run (Google Cloud, 2026a). Ele faz sentido quando há vários destinos, ou quando se quer *tail sampling* (guardar só os *traces* com erro, decidido no fim). Nenhuma das duas condições existe hoje, e o *sidecar* é mais um contêiner com CPU, memória, versão e falha própria em cada serviço de cada contrato. A API e o hub exportam direto, com ADC (a mesma credencial de *service account* que já autentica o Vertex), pelos exportadores `@google-cloud/opentelemetry-cloud-trace-exporter` e `-cloud-monitoring-exporter`. O *sidecar* é o gatilho de reavaliação registrado na **ADR-57**.

Amostragem: `ParentBased` sobre `TraceIdRatioBased(0.02)` — 2% das requisições, com uma diferença deliberada em relação ao *default* do SDK (implementada em 2026-09-02, `observabilidade/sample` **o pai remoto não decide**. O `traceparent` chega da internet — da SDK no navegador, do hub, de um `curl` — e honrar `sampled=01` deixaria qualquer cliente forçar 100% de amostragem, que é custo. Como a decisão por razão é determinística no `traceId`, hub e API com o mesmo *ratio* tomam a *mesma* decisão sem confiar um no outro; o pai *local* (span filho no mesmo processo) segue o pai, como sempre. O identificador de *trace* existe em **100%** das requisições (ele vem do *load balancer*, e com o SDK ligado o span de servidor existe mesmo não amostrado), então todo log de erro carrega um `trace` para pesquisar mesmo quando os *spans* não foram guardados; o que a amostragem decide é só quanta *linha do tempo* se armazena. Rotas de IA (`/ai`, `/unoogpt`, `/planos`, `/bemestar`) e `/runs/:id/feed` usam dez vezes o *ratio*, até 100%: são as em que a latência é o produto. `/healthz` e `/readyz` não viram *span*: o *uptime check* bate 4.320 vezes por dia.

No hub a instrumentação é **menor de propósito**: o Next já emite os *spans* de servidor e já extrai o `traceparent` de entrada assim que existe um `TracerProvider` global (medido: um `traceparent` forjado na requisição sai como `traceId` de todos os *spans* do *render*); instalar o `instrumentation-http` ali duplicaria o *span* raiz. Só o `undici` entra, para o `fetch` de saída à API levar o cabeçalho. O gate é o mesmo nos dois processos: `OBS_TRACE_RATIO` $>$ 0 **e** (`GCP_PROJECT_ID` ou `OBS_DEBUG=1`, que imprime os *spans* no console sem GCP). Desligado, nenhum objeto do SDK é criado — é o que mantém o Jest e o `nest start --watch` sem GCP intocados.

17.3.4 Erros

Erros de servidor chegam ao Error Reporting pelo log estruturado, sem biblioteca: linha com `severity: ERROR` e *stack trace* é agrupada por assinatura, com contagem, primeira e última ocorrência e a revisão do Cloud Run em que apareceu. “Erro novo nesta revisão” é o alerta de *deploy* mais barato que existe.

Erros de **cliente** são o problema que a Figura 17.1 mostra em laranja no primeiro elo, e a escolha aqui é a **ADR-58**: a experiência **não** carrega SDK de terceiro. Um *bundle* público que carrega um *DSN* do Sentry é um *bundle* que manda o *user agent*, a URL, a *stack* e o que mais o erro arrastar para um processador fora do contrato e do DPA da rede — para um erro que aconteceu na tela de uma criança. O caminho é a API, que já tem o *token*, a escola e a regra 6 (Figura 17.2).

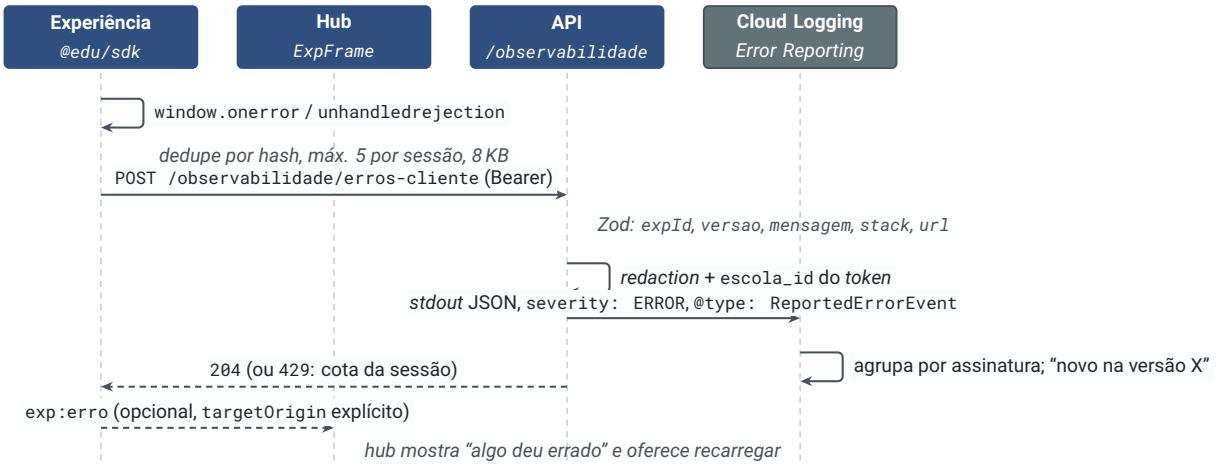


Figura 17.2 – Um erro não capturado dentro do `iframe` chega ao Error Reporting pela API, nunca por um serviço de terceiros: a API já tem o *token*, a escola e a *redaction*. A mensagem `exp:erro` da ponte é opcional e só serve para o hub trocar a tela; o registro não depende dela.

Três detalhes que decidem se isso funciona ou vira ruído. **Cota por sessão**: um laço de erro numa experiência gera milhares de eventos por segundo; a SDK manda no máximo cinco por sessão, com *dedupe* por *hash* de mensagem e *stack*, e a API responde 429 acima disso, por `(escola, usuario)`. ***Sourcemaps* privados**: o *build* das experiências passa a gerar `build.sourcemap:"hidden"`, publicado em `gs://{contrato}-exp/{expId}/{versao}/mapas/` com ACL privada (o *bundle* continua sem o comentário que aponta para eles); a *stack* minificada é legível com o mapa da versão exata, e a versão já está no caminho. **Nada de texto do estudante**: o *payload* é um *schema* fechado; `mensagem` e `stack` passam pela *redaction* da API e têm tamanho máximo.

No hub, `error.tsx` e `global-error.tsx` fazem o equivalente para erro de *render*: registram pelo `pino` do servidor (o `global-error` do cliente posta na mesma rota) e mostram a tela de erro do *design system* com o `requestId`.

17.3.5 Experiência do usuário real e sondas sintéticas

Web Vitals. O `useReportWebVitals` do Next envia LCP, INP e CLS das telas do hub — amostrados a 10%, sem identificador de pessoa, com `perfil` e rota — para a mesma rota da API, que os emite como distribuição OpenTelemetry. Nas experiências, o mesmo pacote `web-vitals` (4 KB) entra pela SDK. É o único sinal de que "a plataforma está lenta na escola X" que não depende de alguém reclamar — e rede pública tem escola com conectividade ruim, o que é exatamente onde o problema aparece e o *trace* do servidor não vê.

Uptime checks do Cloud Monitoring, de três regiões, a cada 60 s, criados pelo Terraform, com validação de **conteúdo** e não só de *status*:

Alvo	O que um único *probe* prova
`GET {hub}/.well-known/jwks.json` contém `"keys"`	DNS, TLS, LB, hub de pé, chave de assinatura carregada. É o *probe* de maior valor: sem ele nenhum *token* de experiência é validável.
`GET {api}/readyz` = `{"ok":true}`	API de pé **e** Cloud SQL respondendo, pela VPC. Hoje ninguém chama esta rota; passa a ser chamada 4.320 vezes por dia.
`GET {hub}/api/healthz`	Rota nova do hub, sem dependência: processo vivo. Separa "hub caiu" de "API caiu" no alerta.
`GET {exp}/exp-01-instanooria/{versao}/index.html`	CDN e *bucket* servindo a versão ativa. A versão vem do manifesto, então o *check* é regerado no passo *ativar* do *deploy*.
`GET {hub}/` contém o texto da *landing*	O *render* do Next funciona, não só o processo.

A suíte e2e vira monitor sintético. É o ativo mais valioso do levantamento e o que menos custa aproveitar: `rodar.mjs publico` já é "o canário mais barato que existe" (nas palavras do próprio arquivo), e cada suíte já sai com *exit code* honesto e relatório classificado. Faltam três coisas, todas fora do código da suíte: (1) um *workflow* do GitHub Actions com `schedule` (a cada 30 minutos em horário escolar, a cada 2 horas fora dele) rodando `publico` e `estudante` contra `hml` com contas de sonda; (2) o mesmo *workflow* chamado como *smoke* ao fim do `deploy.yml`, com `if: failure()` notificando; (3) o resultado convertido em métrica: `gcloud monitoring` escrevendo uma série `e2e/suite_ok` por suíte, para que um alerta do Cloud Monitoring trate a suíte como qualquer outro sinal. O GitHub Actions dá 2.000 minutos por mês no plano *Free*; a suíte pública leva menos de dois minutos, o que cabe.

> **NOTA**
>
> Os *synthetic monitors* do Cloud Monitoring (Playwright rodando numa Cloud Function) fariam o mesmo dentro do projeto, mas custam US$ 1,20 por mil execuções com só cem grátis — e a suíte já existe, com a lógica de espera que corre contra o erro. Reescrevê-la para outra plataforma seria pagar duas vezes pela mesma prova.

17.3.6 Banco, mensageria e borda

Cloud SQL: *Query Insights* ligado no Terraform (grátis) dá a lista das *queries* mais caras por *hash*, com plano; a extensão `pg_stat_statements` é o mesmo dado por SQL, para o *checklist* do Apêndice F. A sonda de *tenancy* entra aqui também: uma métrica de log que conta `WARNING` de *query* fora de `comTenancy` (a RLS devolve zero linhas em silêncio, e o Apêndice F já registra dois incidentes disso).

Pub/Sub: a *subscription* de telemetria ganha `dead_letter_policy` para um tópico `{contrato}-telemetria-dlq` depois de cinco tentativas. Hoje, um lote que a API recusa (401 por *env* ausente, 400 por envelope inválido) é reentregue para sempre — e é invisível. Com a *dead letter*, ele vira uma contagem, e a contagem vira alerta.

Load balancer e CDN: log do LB ligado com amostragem de 10% (suficiente para taxa de erro por *host*), e uma métrica de acerto de CDN por *bucket*. Uma queda na taxa de acerto do *bucket* de *bundles* depois de um *deploy* é o sintoma de *index.html* sem `Cache-Control` ou de caminho publicado sem subir a versão — as duas armadilhas que o `AGENTS.md` da raiz já descreve.

17.3.7 Camada de IA

A IA é o custo variável da plataforma e o sinal de que menos se sabe hoje. O `AiProvider` ganha um invólucro de medida — **dentro** de `apps/api/src/ai`, o único lugar onde uma chamada de IA acontece — que emite, por chamada: latência (histograma por modelo e rota), *tokens* de entrada e saída, `finishReason`, `voltas`, `tentativas` e o resultado (`ok`, `erro-provider`, `quota`). O `QuotaService` emite as recusas por perfil. Custo é métrica *derivada*: uma tabela de preço por modelo em `catalogo-modelos.ts`, multiplicada no painel, não no processo — preço muda sem *deploy*, e a métrica de *tokens* é a que vale.

Três alertas saem daqui: erro do *provider* acima de 10% em 5 minutos (*acorda*, se em horário escolar: é a exp-03 inteira), latência p95 acima de 30 s (*amanhã*: é o `R-13` materializado), e consumo do dia acima de duas vezes a média móvel de 7 dias (*amanhã*: é custo ou laço). O `R-14` mediu uma partida em 71.397 *tokens* à mão; com isto, a medida é contínua.

17.3.8 Entrega

O `deploy.yml` ganha três passos que hoje não existem: um *smoke* pós-*deploy* (o *workflow* da suíte, acima), notificação de falha (`if: failure()` para o mesmo canal dos alertas) e uma **anotação de *deploy*** — uma linha de log `codigo=deploy`, com SHA, revisão e versão de cada experiência ativada, que vira marcador vertical nos painéis. "O erro começou às 14h07" ao lado de "a revisão 00042 recebeu tráfego às 14h06" é metade de um diagnóstico.

17.4 Ferramentas: comparação e escolha

A comparação é por função, não por fornecedor. Em cada linha, a escolha recomendada vem primeiro; as alternativas trazem a condição em que passariam a valer.

Função	Escolha	Custo	Alternativas e quando
Log	**Cloud Logging**, via *stdout* JSON do Cloud Run	50 GiB/projeto/mês grátis; US$ 0,50/GiB além; 30 dias inclusos	*Grafana Loki* auto-hospedado: uma VM, retenção e *backup* por conta própria, e o log sairia do projeto do contrato. *Grafana Cloud Free* (50 GB, 14 dias): idem quanto à saída do projeto. *OpenObserve* ou *SigNoz*: só se o nativo faltar em consulta (não falta: Logs Explorer e Log Analytics em SQL).
Métricas e painéis	**Cloud Monitoring**; painel por contrato em JSON no Terraform; *metrics scope* central	Métricas do Google grátis; custom: 150 MiB grátis, US$ 0,258/MiB além	*Prometheus + Grafana* auto-hospedados: um coletor por serviço (*scrape* não atravessa Cloud Run sem *sidecar*) e uma instância central para N contratos. *Grafana Cloud Free* (10 k séries) como **painel** sobre a fonte de dados Cloud Monitoring, se a pessoa preferir a ergonomia do Grafana — é leitura, não muda a coleta. Entra quando houver mais de um operador com opinião sobre painel.
Rastros	**Cloud Trace** pelo SDK OpenTelemetry + exportador Google	2,5 M *spans*/mês grátis; US$ 0,20/M além	*Grafana Tempo* ou *Jaeger*: exigem coletor e armazenamento. Entra se uma rede exigir retenção de *trace* acima de 30 dias (Cloud Trace guarda 30).
Erros	**Error Reporting** pelo log; erro de cliente **pela API** (ADR-58)	Grátis	*Sentry SaaS* (Developer: 5 k erros/mês, 1 usuário; Team: US$ 26/mês): *session replay* e *breadcrumbs* são melhores, mas o dado sai do contrato — só com DPA que o cubra. *GlitchTip* ou *Sentry* auto-hospedados: fala o protocolo do Sentry dentro do projeto; entra se o agrupamento do Error Reporting se mostrar insuficiente para 48 experiências.
Analytics de produto	**Microsoft Clarity**, no hub e dentro do *bundle* (ADR-60)	Grátis (teto de 100.000 gravações/dia por projeto)	*PostHog* ou *Matomo* auto-hospedados: ficariam dentro do projeto do contrato, e é o caminho se o *replay* em processador de terceiro não passar no DPA de uma rede. *Hotjar*, *FullStory*: pagos, mesmo problema de residência. **Não substituem** a telemetria própria — ela é que corrige, pontua e vira relatório de professor; o Clarity só mostra a tela.

Função	Escolha	Custo	Alternativas e quando
Sondas	**Uptime checks** do Cloud Monitoring + **suíte e2e** agendada no GitHub Actions	1 M execuções/mês grátis; Actions: 2.000 min/mês grátis	*Uptime Kuma* auto-hospedado: bonito, mas uma VM a mais para fazer o que a franquia já faz. *Synthetic monitors* do Cloud Monitoring: US$ 1,20/mil execuções; reescreveria a suíte. *Checkly*, *Grafana Synthetic*: pagos, só com vários operadores.
Roteamento de alerta	**Canais nativos**: e-mail + *push* do app Cloud Console (acorda) e Google Chat/Slack (amanhã)	Grátis; a cobrança anunciada de política (Seção 17.7)	*ntfy* auto-hospedado ou o plano grátis do *Pushover*: *push* sem depender do app do Google. *Grafana OnCall* (grátis até 3 usuários) ou *PagerDuty*: escalonamento e rodízio — **só faz sentido com o segundo operador**.
Teste de carga	**k6** (livre), da máquina de quem opera, contra `hml`	Grátis	*Grafana Cloud k6*: 500 VUh/mês grátis, executores distribuídos — útil se 4.000 usuários virtuais não saírem de um só *laptop*.
APM completo	—	—	*Datadog*, *New Relic*, *Dynatrace*: por *host* ou por GB, na casa de centenas de dólares/mês por contrato. Entram quando houver vários contratos em várias nuvens ou um time de operação com SLA formal — e a instrumentação OpenTelemetry já estará pronta para eles.

> **Onde o pago se justifica hoje**: em nada. **Onde se justifica no primeiro contrato em produção**: num canal de *push* confiável para o alerta *acorda* — o app do Cloud Console serve, e um *Pushover* custa US$ 5 uma vez. **Onde se justificará com o segundo operador**: rodízio de plantão (Grafana OnCall grátis; PagerDuty se houver SLA contratual). **Onde nunca, por desenho**: SDK de **erro** de terceiro dentro de um *bundle* de experiência.
> **E onde a regra acima abriu exceção**: *analytics de produto*. A **ADR-60** pôs o Microsoft Clarity no hub e dentro do *bundle*, porque o `iframe` é cross-origin e gravar só o hub deixaria de fora exatamente a tela em que o estudante passa a sessão. A distinção que sobra é estreita e vale dizê-la: erro de cliente continua entrando pela API, com *redaction* e `escola_id` do *token*; o que sai para terceiro é gravação de sessão, com identificador opaco e sem a resposta do estudante.

17.5 SLIs, SLOs e a lista de alertas

Um SLO é uma promessa numérica sobre um sintoma, e o alerta é a taxa com que a promessa está sendo consumida (BEYER et al., 2018). Os SLOs abaixo são por contrato, com janela de

30 dias, medidos no Cloud Monitoring (que tem *service monitoring* nativo para Cloud Run, com alerta de *burn rate* pronto). A meta de 99,5% dá 3 h 36 min de orçamento de erro por mês — honesto para uma pessoa e um Cloud Run com `min-instances=1`; 99,9% (43 minutos) só com o segundo operador.

SLI	Definição	SLO
Disponibilidade do hub	Requisições ao `-frontend` sem 5xx, exceto sondas	99,5%
Disponibilidade da API	Requisições ao `-api` sem 5xx, exceto `/ai/*` e `/unoogpt/*` (medidas à parte)	99,5%
Latência da API	p95 de `/runs/:id/feed`, `/experiencias/:id`, `/auth/*` abaixo de 800 ms	99%
Turno de IA	Chamadas ao *provider* concluídas (não `erro-provider`)	98%
Ingestão de telemetria	`oldest_unacked_message_age` abaixo de 5 minutos	99%
Ponte	*Handshakes* `exp:ready` → `hub:init` concluídos em 8 s (via métrica do hub)	99%

A lista de alertas cabe numa página, e isso é deliberado. Cada linha tem um dono (a pessoa), uma severidade e uma ação de primeira resposta que aponta para o Apêndice F.

Sev.	Condição	Por que este e não outro	Primeira resposta
A	*Uptime check* do JWKS ou do `/readyz` falha em 2 de 3 regiões por 2 minutos	Ninguém entra, ou nenhuma experiência valida *token*. É **o** sintoma.	Revisão anterior (`update-traffic`); estado do Cloud SQL
A	*Burn rate* do SLO de disponibilidade (hub ou API) $>$ 14,4 em 1 h (consome 2% do orçamento mensal numa hora)	Erro em massa, independentemente da causa. Substitui "5xx $>$ 1%": a mesma taxa às 8h e às 23h significa coisas diferentes.	Error Reporting: o erro novo da revisão
A	Erro do *provider* de IA $>$ 10% em 5 min, entre 7h e 19h	A exp-03 inteira depende disso, e a sala espera.	Quota do Vertex; catálogo do modelo (`R-12`); `error.cause`
A	`instance_count` da API = `max-instances` por 5 min	Saturação: a próxima requisição espera na fila do Cloud Run.	Subir `max`; olhar p95 e pool do Prisma
A	Execução do *job* `-migrate` com falha (métrica de log)	O *deploy* parou no meio, com *schema* velho e código novo prontos para divergir.	Não reexecutar às cegas: ler o log do *job*

Sev.	Condição	Por que este e não outro	Primeira resposta
B	*Burn rate* > 6 em 6 h	Degradação lenta que vai estourar o mês.	Painel; Query Insights
B	Conexões do Cloud SQL > 80% do `max_connections` por 10 min	O pool do Prisma é por instância; escala de instância multiplica conexão.	`connection_limit` na `DATABASE_URL`; *pooler*
B	`oldest_unacked_message_age` > 10 min, ou `dead_letter_message_count` > 0	A telemetria está parando ou sendo recusada — hoje ninguém saberia.	`/telemetry/ingest` no log: 401 é *env*, 400 é envelope
B	Latência p95 da IA > 30 s por 15 min	É o `R-13` acontecendo.	`streamGenerateContent` é a saída conhecida
B	Descartes de telemetria > 1% dos eventos em 1 h (métrica da aplicação)	Uma experiência publicou verbo fora do vocabulário, ou *payload* acima do teto.	Motivo do descarte no rótulo
B	Fila de geração ou de planos com item em `gerando` há mais de 15 min, ou taxa de `erro` > 20% em 1 h	O *worker* vive na API; se ela reciclou, o *job* ficou órfão.	`SELECT` do *checklist*; reciclagem em 5 min é automática
B	Erros de cliente de uma `(expId, versao)` acima de 50 em 10 min, ou erro **novo** nas 2 h após ativação	Uma versão quebrou no navegador da escola.	`publicar-manifesto.mjs --versao anterior`: reversão sem *build*
B	Certificado gerenciado a menos de 30 dias; disco do Cloud SQL > 80%; consumo de IA do dia > 2× a média de 7 dias	Três “amanhãs” clássicos.	Terraform; disco; quem consumiu
B	Suíte e2e agendada em vermelho duas vezes seguidas	Uma única falha é ruído de rede; duas é defeito.	Relatório da suíte diz qual tela

Quatorze políticas, seis de severidade A. Os “A” vão para e-mail e *push* (app do Cloud Console); os “B” vão para e-mail e um canal de Google Chat ou Slack, e são lidos de manhã. Nenhum alerta de CPU, memória ou latência de *startup*: são causas, e ficam no painel.

17.6 Privacidade nos sinais

Os sinais operacionais carregam dado de menor com mais facilidade que a telemetria, porque nascem em pontos que ninguém desenhou para isso: uma mensagem de exceção que embute o valor recebido, uma URL com *query string*, um *stack trace* de cliente com o estado da tela. As regras:

Ponto	Regra
Log	*Redaction* mecânica por caminho no `pino` (lista acima), nos dois processos. Mensagem de `ZodError` nunca vai inteira ao log — só `path: code`, como o `ContratoInterceptor` já faz. Identificador de pessoa em log é sempre o UUID opaco.
Trace	Atributos automáticos incluem a URL: o *hook* `ignoreIncomingRequestHook` exclui sondas e o `urlFilter` remove *query string*. Nenhum atributo com corpo de requisição.
Erro de cliente	*Schema* fechado, tamanho máximo, *redaction* na API, sem *cookie*, sem `localStorage`, sem *DOM*. O *stack* é o único texto livre, e ele é código, não dado.
Web Vitals	Sem identificador de pessoa; `perfil` e rota apenas.
Session replay	O único sinal que sai do projeto do contrato (**ADR-60**). `identify` manda o UUID opaco — nunca nome, nunca e-mail, nunca o `friendlyName` da API do Clarity. Escola e turma vão por `id`. Do `track()` sai só `verbo:objeto`: `resultado` e `contexto` carregam resposta de estudante e ficam na telemetria própria. O mascaramento de campo de entrada, número e e-mail é o *default* do Clarity e não foi afrouxado.
Retenção	Log do `_Default`: 30 dias (padrão, sem custo). Um *bucket* de auditoria separado, alimentado por *sink* das linhas `codigo=auth.*`, `codigo=importacao.*`, `codigo=deploy` e dos *Admin Activity audit logs*: 400 dias. É o que uma rede pública pergunta num incidente, e é barato porque é pequeno.
Residência	O *bucket* de log de cada contrato é criado na região do contrato (`us-east1` no piloto), não em `global`. Log não sai do projeto: o *metrics scope* central vê métrica, não log.
Acesso	Quem lê log de produção lê dado operacional de uma rede pública: IAM por pessoa, `roles/logging.viewer` por projeto, e os *Data Access audit logs* do próprio Cloud Logging ligados — quem leu o log também fica em log.

17.7 Custo estimado por contrato

Volumes estimados para um contrato em regime: dez mil estudantes, vinte dias letivos, cerca de 40 milhões de requisições por mês entre hub e API (pico de 4.000 simultâneos às 8h), e

os preços de referência do Google Cloud Observability em setembro de 2026 (Google Cloud, 2026c). Os preços mudam; as **proporções** são o que importa.

Sinal	Volume/mês	Franquia	Custo estimado
Log de requisição do Cloud Run, **sem** exclusão (≈1 KB por linha)	≈40 GiB	50 GiB	US$ 0 — mas a 80% da franquia, sem folga para o log da aplicação nem para o do LB
Idem, **com** exclusão de sondas e amostragem de 20% dos 2xx rápidos	≈12 GiB	idem	US$ 0, com 38 GiB de folga
Log da aplicação (`WARNING`+, eventos de negócio, ≈0,5 KB)	≈1–2 GiB	idem	US$ 0
Log do LB a 10%	≈4 GiB	idem	US$ 0
Traces a 2% (20% em IA e *feed*), ≈8 *spans* cada	≈8 M *spans*	2,5 M	≈US$ 1,10
Métricas da aplicação, ≈60 séries a 1 ponto/min	≈20 MiB	150 MiB	US$ 0
Métricas de log definidas pelo usuário (6)	≈2 MiB	idem	US$ 0
Uptime checks: 5 alvos × 3 regiões × 60 s	≈657 k execuções	1 M	US$ 0
Políticas de alerta: 14, ≈16 referências de métrica	—	—	≈US$ 5,60 quando a cobrança anunciada (US$ 0,35 por referência por mês) entrar em vigor; US$ 0 até lá
Retenção estendida do *bucket* de auditoria (≈0,5 GiB × 13 meses)	—	—	≈US$ 0,07
Error Reporting, Query Insights, painéis, *metrics scope*	—	—	US$ 0
Suíte e2e agendada no GitHub Actions (≈1.400 min)	—	2.000 min	US$ 0 (plano *Free*, repositório privado)
Total por contrato			**≈US$ 1 hoje; ≈US$ 7 com a cobrança de alerta**

> **NOTA**
>
> Para comparar: uma VM `e2-small` para um Grafana + Loki + Prometheus auto-hospedados custa cerca de US$ 15 por mês antes do disco e do *backup* — **por instância**, e

> ela teria de ser central (senão é uma por contrato) e receber dados de N projetos por rede e IAM próprios. O nativo custa menos que isso *no pior caso*, e a diferença não é o dinheiro: é a peça que não existe para ser atualizada às 2h da manhã. Os dois riscos de custo do nativo são conhecidos e têm teto: log de requisição sem exclusão (por isso a exclusão é do Terraform, não de quem lembra) e cardinalidade de métrica (por isso `escola_id` não é rótulo).

17.8 Roteiro de implantação

Quatro fases. A fase 0 é a que fecha o `R-05` antes de novembro de 2026 e **não toca em código de aplicação**: é Terraform e um *workflow*. As fases seguintes são incrementais e cada uma tem uma prova de pronto que a suíte ou o painel exibe.

Fase	Entrega	Onde	Prova de pronto
0 antes de nov./2026	Canais de notificação; 5 *uptime checks*; as 14 políticas (as que dependem de métrica da aplicação entram na fase 2); SLOs de disponibilidade; painel por contrato; filtros de exclusão; *bucket* de auditoria e Data Access audit logs do Logging; *dead letter* da telemetria; Query Insights; sondas HTTP (`startup` em `/healthz`, `liveness` em `/healthz`) no recurso Cloud Run; *metrics scope* central. *Smoke* pós-*deploy* e `if: failure()` no `deploy.yml`; suíte `publico`+`estudante` agendada contra `hml`.	Repositório de Terraform (`docs/INFRA.md` ganha a seção); `.github/workflows/`	Derrubar a API em `hml` e receber o *push* em menos de 3 minutos. *Deploy* com migração quebrada notifica.

Fase	Entrega	Onde	Prova de pronto
1 1–2 semanas	**API** (*entregue em 2026-09-02*, `feat/obs-api-log`): `nestjs-pino` com *redaction*, contexto de requisição no ALS, `ExceptionFilter` global, `LOG_LEVEL`, `GCP_PROJECT_ID`, Prisma com aviso de *query* lenta. **Hub** (*entregue em 2026-09-02*, `feat/obs-hub-log`): `lib/log.ts`, `apiDoServidor` logando e com *timeout*, `/api/healthz`, `error.tsx` e `global-error.tsx`, `onRequestError` com o `digest` da tela (o `HEALTHCHECK` do Dockerfile ficou de fora: o Cloud Run o ignora). **SDK**: `visibilitychange`, *backoff* e teto de 500 eventos no *buffer*, retorno do `sendBeacon` verificado.	`apps/api`, `apps/hub`, `packages/sdk`	Um 500 provocado em `hml` aparece no Error Reporting agrupado, com `requestId` igual ao da tela, em menos de um minuto. Teste unitário prova que `username` e `cpf` saem redigidos.

Fase	Entrega	Onde	Prova de pronto
2 2–3 semanas	OpenTelemetry na API e no hub (`instrumentation.ts`), exportador Google, amostragem 2%/20%, `traceparent` do navegador ao banco (*traces entregues em 2026-09-02*, `feat/obs-traces`: `otel.ts` como primeira importação do `main.ts`, `@prisma/instrumentation` no lugar do `pg`, *sampler* por rota que ignora a *flag* do pai remoto). Métricas da aplicação: IA, telemetria, filas, pool, ponte. Rota `POST /observabilidade/erros-cliente` e `/observabilidade/vitals` com Zod, cota e *redaction*; `window.onerror`/`unhandledrejection` e `web-vitals` na SDK; `exp:erro` na ponte (v1 continua compatível: mensagem nova é ignorada por hub antigo); *sourcemaps* ocultos publicados com ACL privada no passo *subir*. Políticas de alerta que faltavam.	`apps/api/src/observabilidade/`, `apps/api/src/ai`, `packages/sdk`, `deploy.yml`	Um `throw` provocado na exp-02 em `hml` vira erro agrupado por `(expId, versao)` com *stack* mapeada. Um *trace* de abertura de missão mostra hub → API → Postgres → Vertex numa só linha do tempo.
3 condicional	Grafana (Cloud Free ou local) como painel de leitura sobre o Cloud Monitoring; coletor *sidecar* com *tail sampling*; Grafana OnCall; Sentry/GlitTip auto-hospedado; k6 distribuído.	—	Cada item tem gatilho na **ADR-57**. Nenhum começa por conta própria.

ATENÇÃO

A fase 0 depende do repositório de Terraform, que é separado e tem seu próprio ciclo. O que cabe a este repositório é **o contrato**: a lista de recursos que `docs/INFRA.md` passa a esperar (canais, políticas, *checks*, painel, exclusões, *bucket* de auditoria, *dead letter*, sondas do serviço, `GCP_PROJECT_ID`) e o *workflow* da suíte. Sem esse contrato escrito, a fase 0 vira “clicar no console” — que funciona em `dev` e desaparece no segundo contrato.

17.9 O que fica de fora

Fora	Condição para reavaliar
Stack de observabilidade auto-hospedada (Grafana LGTM, SigNoz, OpenObserve)	Uma rede exigir que os sinais fiquem fora do Google Cloud, ou o nativo faltar numa consulta demonstrada. Ver **ADR-57**.
APM comercial (Datadog, New Relic, Dynatrace)	Vários contratos em várias nuvens, ou time de operação com SLA formal. A instrumentação OpenTelemetry já estará pronta para eles.
SDK de erro de terceiro no *bundle* da experiência	Nunca, por desenho (**ADR-58**). A versão auto-hospedada dentro do projeto é o caminho, se o Error Reporting não bastar. **Não confundir** com o *analytics* de produto da **ADR-60**, que está DENTRO do *bundle*: são ferramentas e decisões diferentes.
Ferramenta de plantão (PagerDuty, Opsgenie, Grafana OnCall)	O segundo operador.
Log de requisição da aplicação a 100%	Nunca: o Cloud Run já grava um.
`escola_id` como rótulo de métrica	Nunca: cardinalidade. Escola se filtra em log e se agrega em SQL.
BigQuery como destino de log	Um relatório de auditoria que o Log Analytics (SQL sobre o *bucket*, incluso) não consiga responder.

18

Registro de decisões arquiteturais

Cada decisão abaixo segue a forma de um ADR (NYGARD, 2011): contexto, decisão, alternativas descartadas, consequências — inclusive as ruins — e o gatilho que faria reavaliá-la. Decisão sem alternativa descartada e sem gatilho de reavaliação não é decisão registrada: é justificativa.

18.1 Índice

ID	Decisão	Data	Estado
ADR-01	Experiências como *bundles* estáticos, não serviços	2026-08	Aceita
ADR-02	*Monorepo* único (pnpm + Turborepo)	2026-08	Aceita
ADR-03	Autenticação por OIDC genérico	2026-08	Aceita
ADR-04	Uma *stack* por contrato; escolas por RLS	2026-08	Aceita
ADR-05	Cloud SQL HA, não AlloyDB	2026-08	Aceita
ADR-06	Pub/Sub para telemetria desde o início	2026-08	Aceita
ADR-07	IA atrás de um *gateway* na própria API	2026-08	Aceita
ADR-08	RLS forçada e uma porta única de banco	2026-08-24	Aceita
ADR-09	Domínio por experiência na API; gabarito *server-side*	2026-08-24	Aceita
ADR-10	Bem-estar: DASS-21 fixo, IA só no entorno, sem visão de gestor	2026-08-25	Aceita
ADR-11	Telemetria sem PII e vocabulário fechado no *payload*	2026-08-26	Aceita
ADR-12	`grau` como invariante de API, não regra de tela	2026-08-26	Aceita
ADR-13	Retenção sem prazo declarado, em troca de reprocessamento	2026-08-26	Aceita
ADR-14	Telemetria no PostgreSQL particionado; sem BigQuery, sem Redis	2026-08-27	Aceita

ADR-15	O hub assina os *tokens* enquanto o IdP central não entra	2026-08	Aceita, transitória
ADR-16	Credencial por ADC; modelo `3.6-flash` em `global`	2026-08	Aceita, com custo
ADR-17	*Claim* atômico em vez de fila externa	2026-08	Aceita
ADR-18	Exceções registradas: jogo em JS clássico, exp-70 fora do DS	2026-08-26	Aceita
ADR-19	*Design system* externo como fonte única de interface	2026-08	Aceita
ADR-20	Um banco, um *schema*; prefixo `exp_NN_`	2026-08-27	Aceita
ADR-21	*Push subscription* para a própria API, sem *worker*	2026-08	Aceita
ADR-22	exp-02 1.1.0: *briefing* na voz da escola, conclusão por botão	2026-08-27	Aceita, com custo
ADR-23	*Function calling* entra no `AiProvider`, não na experiência	2026-08-27	Aceita
ADR-24	Teto de custo de IA por SESSÃO; a quota diária é fusível	2026-08-27	Aceita, com número a medir
ADR-25	O Relógio da Aula é tabela da experiência, não coluna em `turmas`	2026-08-27	Aceita
ADR-26	Sem prazo de retenção na exp-03; exclusão por "zerar a turma"	2026-08-27	Aceita
ADR-27	Transcrição do estudante é dado do instrumento: professor lê, gestor não	2026-08-27	Aceita

18.2 ADR-01 — Experiências como *bundles* estáticos

> **Contexto**
>
> O catálogo alvo é de 48 experiências, cada uma uma aplicação React independente. O pico de carga é uma parede às oito da manhã, e o grosso dos *bytes* é o código e a mídia dessas aplicações. Uma pessoa opera tudo.

Decisão. Cada experiência é um *bundle* Vite 100% estático, publicado em `gs://{contrato}-exp/{expId}/{v` e servido pela CDN. Não há processo, porta, nem *scaling* por experiência. O hub monta a URL do `iframe` com `versaoAtiva` do manifesto.

Alternativas descartadas.

- *Microsserviço por experiência*: 48 serviços com *deploy*, log, escala e *alerting* próprios — para servir arquivos estáticos.
- *Rotas do hub monolítico*: cada experiência entraria no *bundle* do hub, e o hub cresceria sem limite. Uma mudança em qualquer experiência exigiria *deploy* do hub inteiro.

Consequências.

- Custo marginal de publicação ≈ armazenamento + tráfego de CDN.
- O caminho carrega a versão ⇒ o *bundle* é imutável e pode ficar um ano na borda.
- ***Rollback* é um `UPDATE`**: trocar `versaoAtiva`. Sem *build*, sem imagem, sem *deploy*. É a operação de emergência mais rápida da plataforma.
- *Ruim*: a experiência não tem *backend* próprio, então toda lógica de domínio dela vive na API — o que concentra responsabilidade num serviço só (mitigado por **ADR-09**).
- *Ruim*: um *bundle* publicado não é reconstruído quando a API muda, o que torna os contratos do Capítulo 9 rígidos.

Gatilho de reavaliação. Uma experiência que precise de execução no servidor por natureza — multiplayer em tempo real com autoridade de servidor, ou processamento pesado por sessão.

18.3 ADR-02 — *Monorepo* único

Contexto. A SDK é consumida por 48 aplicações. O *design system* é externo, mas a ponte é interna.

Decisão. Um *monorepo* com pnpm *workspaces* e Turborepo (Vercel, 2026b). `apps/*`, `apps/experiencias/*`, `packages/*`.

Alternativa descartada. Repositório por experiência, com a SDK publicada em npm. Uma mudança na SDK só seria testada contra as 48 aplicações depois de publicada — e a incompatibilidade apareceria num *bundle* servido a estudantes semanas depois.

Consequências. Mudança na SDK testa tudo no mesmo *pull request*; o CI builda só o que mudou (`--filter='...[HEAD^]'`). *Ruim*: o repositório cresce, e `pnpm install` completo é lento.

Gatilho. Estúdio externo produzindo experiência — aí a experiência sai para repositório próprio e passa a consumir a SDK publicada.

18.4 ADR-03 — Autenticação por OIDC genérico

Contexto. O WSO2 Identity Server é exigência do cliente e da estratégia multi-contrato: é único para várias redes.

Decisão. A aplicação **não conhece o WSO2**: conhece OIDC. A API valida *token* por `OIDC_ISSUER`, `OIDC_JWKS_URI` e `OIDC_AUDIENCE`. Trocar de provedor de identidade é trocar essas variáveis.

Alternativa descartada. SDK do WSO2 na aplicação. Amarraria o código a um produto que pode mudar por decisão comercial, e tornaria o teste local dependente de um servidor Java em *bootstrap*.

Consequências. `OIDC_ISSUER` e `OIDC_JWKS_URI` vão como *env* em texto plano (não são segredo: são URLs públicas), derivadas de `hub_domain` — e é isso que garante que o `iss` do *token* e o esperado pela API **não podem** divergir.

Gatilho. Nenhum previsto. É a decisão mais estável do conjunto.

18.5 ADR-04 — Uma *stack* por contrato; escolas por RLS

Contexto. Cada contrato é uma relação jurídica com um ente federativo; cada contrato tem 100+ escolas.

Decisão. Isolamento **físico** entre contratos (*stack* GCP inteira, por *module* Terraform) e **lógico** entre escolas (`escola_id` + RLS).

Alternativas descartadas.

- *Uma stack por escola*: cem infraestruturas por contrato, operadas por uma pessoa. E o custo fixo de uma escola pequena passaria a ser o de uma rede.
- *Uma stack para todos os contratos*: um incidente num município seria um incidente em todos, e o desenho não sobreviveria à primeira análise jurídica de um edital.

Consequências. `escola_id` em toda tabela de dado de estudante, RLS forçada, e `admin-contrato` como única exceção cross-escola. *Ruim*: cada contrato novo é uma *stack* nova para provisionar, monitorar e faturar.

Gatilho. Exigência contratual de isolamento físico *por escola* — o que mudaria o modelo comercial, não só a arquitetura.

18.6 ADR-05 — Cloud SQL HA, não AlloyDB

Decisão. PostgreSQL gerenciado (Cloud SQL) com HA, PITR e *pooler* de conexões obrigatório.

Alternativa descartada. AlloyDB. **Motivo:** a robustez que este sistema precisa vem de HA + PITR + *pooler*, não do *engine*. Ambos são PostgreSQL, então migrar depois é trivial se a carga justificar.

Consequências. O *pooler* é **obrigatório**: o *autoscaling* do Cloud Run multiplica *pools* do Prisma e estouraria `max_connections`. E o banco só tem IP privado — o que obriga a migração a rodar como Cloud Run Job dentro da VPC.

Gatilho. Carga medida que sature uma instância de Cloud SQL com *pooler* bem dimensionado.

18.7 ADR-06 — Pub/Sub para telemetria desde o início

Contexto. No pico, quatro mil estudantes simultâneos gerando evento a cada poucos segundos.

Decisão. O canal *analítico* de telemetria vai para o Pub/Sub. **O resto da aplicação fala PostgreSQL direto** — inclusive o canal *operacional* do motor.

Alternativa descartada. Tudo síncrono no PostgreSQL. Colocaria um `INSERT` sincronizado por interação de estudante dentro da mesma transação que serve a tela.

Consequências. Uma peça de infraestrutura a mais — e é a única exceção declarada à regra "nada de fila sem gargalo medido", junto com **ADR-07**. Em troca: o pico é amortecido por um *buffer* **durável** (ver **ADR-14**).

Gatilho. Nenhum. É decisão tomada de antemão e mantida.

18.8 ADR-07 — IA atrás de um *gateway* na própria API

Decisão. Toda chamada de IA passa por `apps/api/src/ai`, com interface neutra de *vendor*, *prompt* de sistema no servidor, quota diária persistida por perfil e *rate limit*. Pré-gerar em lote sempre que a pedagogia permitir.

Alternativas descartadas.

- *Chamada direta do bundle*: exporia chave e *prompt* de sistema, e tornaria a quota incontrolável.
- *Serviço de IA separado*: um Cloud Run a mais para operar, sem ganho — a API já tem CPU sempre alocada.

Consequências. Nenhum módulo fora de `src/ai` conhece o *vendor*; trocar de provedor é criar um arquivo. *Ruim*: a API carrega o custo de latência das gerações longas, o que obriga o padrão 202 + *polling*.

Gatilho. Volume de IA que justifique isolar o consumo, ou requisito de GPU própria.

18.9 ADR-08 — RLS forçada e uma porta única de banco

Decisão. `ENABLE` e `FORCE ROW LEVEL SECURITY` em toda tabela de dado de estudante; acesso ao banco **exclusivamente** por `PrismaService.comTenancy`, que abre transação e executa `set_config('app.` `..., true)` antes do trabalho.

Alternativas descartadas.

- *Apenas filtro na aplicação*: uma *query* nova que esqueça o filtro devolve dado de outra escola, e ninguém percebe.
- *Apenas RLS*: um erro de migração desliga a proteção inteira.
- *Duas roles* (dona para migração, sem *ownership* para a aplicação): desnecessário com `FORCE`, que sujeita o próprio dono às políticas.

Consequências. Duas trancas independentes. O terceiro argumento `true` do `set_config` é **crítico**: local à transação, então a conexão volta limpa ao *pool*. `comTenancy` sem escola **lança** — não consulta sem filtro. *Ruim*: toda leitura, inclusive trivial, abre transação.

Gatilho. Nenhum. É a decisão que sustenta o produto.

18.10 ADR-09 — Domínio por experiência na API; gabarito *server-side*

Contexto. A exp-02 (2026-08-24) foi a primeira experiência com gabarito.

Decisão. Não há módulo genérico de avaliação. Cada experiência com lógica de domínio ganha um módulo próprio na API (`src/<slug>/`), com `catalogo.ts` + `regras.ts` + *service* + *controllers* por papel. O gabarito nunca vai ao *bundle*; a API corrige.

Alternativa descartada. Um módulo genérico de avaliação com "itens" e "respostas corretas". Uma API genérica de correção convida a mandar a resposta certa junto — e cada experiência tem uma mecânica que não cabe num formato comum.

Consequências. Decisões concretas herdadas desta ADR: ranking com colegas **fictícios** (sem pontuação de menor real exposta a outro menor); URLs das fontes como **texto** (o *sandbox* bloqueia *popups*, e o estudante não sai da plataforma); salvos **congelados** depois da conclusão (nota e estado não podem divergir); estudante roteado pela **série**. *Ruim*: um módulo novo por experiência — 48 módulos no limite.

Gatilho. Se três experiências consecutivas tiverem mecânica idêntica, extrair o comum. Nunca antes.

18.11 ADR-10 — Bem-estar: instrumento fixo, IA só no entorno, sem gestor

Contexto. 2026-08-25. Um *check-in* de bem-estar *do professor*, com o DASS-21.

Decisão. O instrumento entra ***verbatim*** (versão brasileira de Vignola & Tucci (VIGNOLA; TUCCI, 2014)) em microdoses de três itens por dia. A IA gera abertura, pergunta aberta e acolhimento, atrás de `regras.validarTextoDaIa`, com *fallback* fixo, e **nunca vê** pontuação, nome ou escola. Correção *server-side* ao sétimo *check-in*. Item sentinela (17 ou 21 ≥ 2) ou termo de risco $\Rightarrow$ mensagem fixa de apoio, sem IA.

Por ser dado de saúde (LGPD art. 11): não há visão de gestor nem agregado por escola. RLS + posse por `professor_id`, e o próprio professor apaga tudo.

Alternativas descartadas.

- *IA reescrevendo os itens* para "soar mais natural": quebra a validação psicométrica e invalida as faixas de corte.
- *Questionário completo de uma vez*: engajamento vem de custar menos de um minuto, não de ser curto uma vez.
- *Painel agregado para a rede*: sai de um `GROUP BY` e é exatamente o que não se deve fazer com dado de saúde de uma equipe pequena, onde *n* baixo reidentifica.

Consequências. O *check-in* nunca deixa de abrir. *Ruim*: a rede não tem nenhuma visão de bem-estar dos professores — o que é uma perda real de informação de gestão, aceita.

Gatilho. Uma visão agregada e anonimizada é decisão futura, com DPA e *k*-anonimato explícito.

18.12 ADR-11 — Telemetria sem PII e vocabulário fechado

Decisão. A telemetria carrega `estudante_id` **opaco**, e a identidade é derivada do *token* no servidor — o que o cliente enviou nesses campos é descartado. Em `exp_70_sessoes.payload`, `eventos[].dados` aceita apenas número, booleano ou *string* de um **vocabulário fechado**.

Alternativa descartada. Campo `texto` livre no *payload* (que existia). Era o único canal de texto arbitrário — e o caminho por onde o nome de uma criança chegaria a um JSONB de longa duração.

Consequências. Trava **estrutural**, não lista de proibidos: não há como enviar texto arbitrário, então não há como revisar mal. *Ruim*: um evento novo que precise de rótulo textual exige acrescentar o rótulo ao vocabulário — ou seja, exige uma decisão explícita.

Gatilho. Nenhum.

18.13 ADR-12 — `grau` como invariante de API

Contexto. O mapeamento comportamento → valor de Schwartz é **hipótese de *design*, não instrumento validado**: Schwartz se valida por questionário (PVQ para adultos, PBVS-C para crianças (SCHWARTZ, 2012; DöRING et al., 2010)), não por telemetria de jogo.

Decisão. A API devolve uma **união discriminada** em que só a variante `scored` tem `numero: number`; as outras têm `null` **por tipo**. E a distinção entre `zero` ("teve a chance e não foi", entra na média) e `none` ("a oportunidade não chegou", sai da conta) é preservada em todo o caminho.

Alternativa descartada. Manter a decisão na camada de interface, com um comentário avisando. Foi assim no aplicativo de origem, e qualquer tela nova podia imprimir "Poder 100" a partir de um sinal só. **Prosa não é controle; um tipo é.**

Consequências. O hub fica estruturalmente incapaz de desenhar o número errado — ele nunca o recebe, e o compilador nem deixa procurar. Decisões herdadas: o relatório abre pelo **nível** do instrumento antes do desempenho; a listagem de turma **não** mostra valor psicométrico (listagem é ordenável, e uma coluna "Poder" viraria ranking de crianças); `cartoesServidos` ≠ "apareceram".

Gatilho. Se um instrumento validado por questionário for adotado, a união ganha uma variante — não perde a distinção.

18.14 ADR-13 — Retenção sem prazo declarado

Contexto. 2026-08-26. Havia `expires_at` de 90 dias no *schema* da exp-70 e **nenhuma rotina que o cumprisse**.

Decisão. A coluna **saiu** (`20260826170000_vila_sem_prazo`). Não há prazo de retenção declarado para `exp_70_sessoes` nem para `telemetria_eventos`.

Alternativa descartada. Manter a coluna. **Coluna de prazo que ninguém lê é pior que prazo nenhum**, porque quem lê o *schema* acredita nela — é a mesma armadilha de um campo `consentimento_em` que existia no aplicativo de origem: a regra saiu do código, a coluna ficou, e todo mundo continuou acreditando.

Consequências. Ganha-se o **reprocessamento**, que é requisito real do instrumento: `payload` é a única fonte para recomputar o perfil quando `VERSAO_MAPEAMENTO` subir, e 90 dias matavam exatamente a comparação entre duas gerações do mapeamento. *Ruim, e é explícito*: dado comportamental de menor fica armazenado indefinidamente, e isso é uma exposição de conformidade em aberto.

Gatilho. Exigência contratual de prazo. Quando vier, ele volta **junto com a rotina que o cumpre** — e, com a tabela particionada (**ADR-14**), essa rotina é `DROP TABLE particoes.<mês>`. A pergunta é da plataforma inteira, não de um módulo.

18.15 ADR-14 — Telemetria no PostgreSQL particionado

Contexto. 2026-08-27. Avaliou-se mandar o canal analítico para o BigQuery por *subscription* nativa do Pub/Sub. Mediu-se também o custo real do caminho de gravação.

Decisão. O canal analítico grava no **mesmo PostgreSQL**, com `INSERT` multi-linha e `telemetria_eventos` particionada por mês. **Sem BigQuery. Sem Redis.**

Alternativas descartadas.

- *BigQuery*: os relatórios do professor **não leem** `telemetria_eventos` (leem `exp_01_events`), então não resolvia problema nenhum hoje — e criava uma segunda plataforma de dados para operar.
- *Redis como buffer*: quem amortece o pico já é o Pub/Sub, e ele é **durável**. Um *buffer* em memória perde os eventos do processo que morre antes do *flush*, ou seja, reintroduz a perda que o Pub/Sub eliminou. **O gargalo medido era o laço de `INSERT`** — um lote de 50 custava 50 *round-trips*, o que no pico dava ∼800 `INSERT`/s; em lote, ∼16/s.

Consequências. A chave primária é composta `(event_id, ts)`, porque o PostgreSQL exige a chave de partição em todo índice único — e a *dedupe* segue correta, porque a reentrega traz o mesmo `ts`. As partições vivem no *schema* `particoes`, fora do alcance do Prisma. Há partição `DEFAULT` como rede de segurança, porque evento recusado é telemetria perdida em silêncio. *Ruim*: as faixas vão até 2028-12 e **não há *job* agendado** — criar partição é item de operação.

Gatilho. Volume que sature o PostgreSQL, ou um relatório que efetivamente precise de análise sobre `telemetria_eventos` em escala.

18.16 ADR-15 — O hub assina os *tokens* de experiência

Contexto. O IdP central (WSO2) tem ciclo de vida próprio e não estava pronto quando as experiências precisaram de *token*.

Decisão. O **hub** assina os *tokens* de experiência (RS256, TTL 15 min, *claims* `{sub, escola_id, turma_id?, perfil}`, `aud plataforma-api`) e serve o JWKS em `/.well-known/jwks.json`. A API valida por `OIDC_ISSUER` e `OIDC_JWKS_URI`.

Alternativa descartada. Esperar o WSO2. Faria toda a plataforma depender de um servidor Java em *bootstrap* para que qualquer experiência pudesse ser testada.

Consequências. Trocar para o WSO2 é trocar duas variáveis — o `OidcGuard` não muda uma linha. **Efeito colateral útil:** o CI usa a mesma chave para assinar o *token* de cinco minutos do *upsert* de manifesto, o que é o motivo de as experiências serem publicadas *depois* do hub. *Ruim*: o hub acumula responsabilidade de emissor, e `TOKEN_SIGNING_KEY_PEM` passa a ser obrigatória em produção.

Gatilho. WSO2 pronto e configurado por contrato. A ADR é **transitória por desenho**.

18.17 ADR-16 — ADC como credencial; modelo *versus* região

Decisão, parte 1 — credencial. O *provider* de IA obtém o *token* com `google-auth-library`, que resolve `GOOGLE_APPLICATION_CREDENTIALS` → arquivo de `gcloud auth application-default login` → *metadata server*. **Nunca chave de API** — não existe variável para ela no projeto.

Contexto. Antes havia *só* o *metadata server*, e por isso *nenhuma* chamada de IA funcionava fora do Cloud Run. Em desenvolvimento a API ficava presa em `mock` e a exp-90 devolvia um plano **falso** — o que é pior que falhar, porque parecia funcionar.

Decisão, parte 2 — modelo e região. Modelo `gemini-3.6-flash`, localização `global`, para texto *e* para plano de aula.

Medição. O `3.6-flash` dá **404** em `southamerica-east1`, `us-east1` e `us-central1`. Só o endpoint `global` o serve.

Consequência aceita, e ela é séria. O tráfego de IA sai de São Paulo, inclusive o do estudante, e isso precisa constar do contrato e do DPA da rede.

Gatilho. Exigência de residência de dados. Voltar à região é trocar `VERTEX_LOCATION` **e** o modelo, juntos — a região publica a família 2.5, não a 3.x.

Decisões relacionadas. `VERTEX_THINKING_BUDGET = 0`: nos modelos 2.5 e 3.x os *tokens* de pensamento saem do `maxOutputTokens`, e com pensamento livre o plano anual voltava **vazio** com `finishReason: MAX_TOKENS`. Teto de saída do plano em **64k**, não 32k. Modelo de imagem `gemini-3.1-flash-image`, com proporção `4:5` **pedida** ao modelo — o *default* anterior não existe mais, e sem pedir a proporção o `cover` descarta metade da largura.

18.18 ADR-17 — *Claim* atômico em vez de fila externa

Contexto. Duas rotinas longas e assíncronas: geração de *feed* (exp-01) e geração de plano de aula (exp-90). Duas ou três instâncias do Cloud Run rodando ao mesmo tempo.

Decisão. *Claim* atômico na própria linha (`UPDATE ... WHERE status = 'pendente'`, com contagem de linhas afetadas) e *worker in-process*. Para o motor de *feed*, *lock* consultivo por `run` (`pg_advisory_xact_lock`).

Alternativas descartadas.

- *Fila externa* (Pub/Sub, Cloud Tasks): peça a mais para operar, para um problema que um `UPDATE` condicionado resolve.
- *Sem lock no motor*: dois pedidos simultâneos de bloco (duas abas, *reload* no meio do *fetch*, *retry*) se sobrescreviam e um bloco inteiro desaparecia do estado — posts entregues voltavam a ser servidos e a série por bloco do dossiê ficava corrompida.

Consequências. A API precisa de **CPU sempre alocada** e `min-instances` ≥ 1, porque há trabalho fora do ciclo de requisição. Retomada de órfã é disparada pelo *polling* do painel, não por *scheduler*. *Ruim*: se todas as instâncias morrerem e ninguém abrir o painel, uma geração órfã fica parada até que alguém abra.

Gatilho. Primeira reclamação real de geração parada.

18.19 ADR-18 — Exceções registradas

Contexto. Duas regras da plataforma têm exceção, e exceção não registrada vira precedente.

Exceção 1 — `any` e ESLint. `apps/experiencias/exp-70-vila-do-lume/public/jogo/` — cerca de dez mil linhas de JavaScript clássico com globais compartilhadas e `three.min.js` vendorizado — fica como está, e está nos `ignores` do ESLint. **Motivo:** é um instrumento com distâncias e tempos calibrados, e o compilador não tem como saber que uma refatoração mudou a *medida*. O *shim* (`src/`, TS *strict*) é a única porta: `window.VILA`. A rede que substitui o `tsc` ali é o *smoke* visual.

Exceção 2 — *design system*. A exp-70 não usa o `@unooria/design-system`. **Motivo:** é jogo em *canvas* 3D, sem DOM de interface.

Consequências. As duas exceções são **nominais** — valem para uma pasta e uma experiência, não para uma categoria. Qualquer outra experiência que peça a mesma exceção precisa de uma ADR nova.

18.20 ADR-19 — *Design system* externo como fonte única de interface

Decisão. Todo elemento de interface vem do `@unooria/design-system`, consumido pela ponte `packages/ui`. Componente que falta **nasce no repositório do *design system*** e chega por versão nova — não como arquivo local.

Alternativa descartada. Componentes locais em `packages/ui`. Com 48 experiências, um componente local viraria 48 divergências — e a fidelidade ao clone é o que a experiência *mede*.

Consequências. Duas famílias no mesmo pacote que **nunca se cruzam**; a linha entre elas é a **revelação**, não o arquivo. Compensação local só como exceção marcada no CSS, com pedido correspondente ao repositório do *design system*. *Ruim*: uma lacuna do pacote bloqueia a experiência até a próxima publicação — e `^0.1.x` não sobe *minor*, então atualizar é ato deliberado.

Gatilho. Nenhum. É decisão de produto e de marca, não de engenharia.

18.21 ADR-20 — Um banco, um *schema*; prefixo `exp_NN_`

Decisão. Um banco, um *schema* lógico. Tabela de experiência leva `exp_NN_`; tabela de plataforma fica sem prefixo; tabela que é contrato entre duas experiências carrega **os dois números** (`exp_01_exp_70_perfis_importados`).

Alternativas descartadas.

- *Banco por experiência*: elas **não** são isoladas — o perfil da Vila alimenta a largada do Instanooria, e o PostgreSQL não faz `JOIN` entre bancos. E um *pool* do Prisma por experiência estouraria `max_connections`.

- *Schema por experiência*: a única coisa que daria além do prefixo é fronteira de `GRANT`, e existe **uma** *role* de aplicação.

Consequências. Migração de referência `20260827090000_tabelas_por_experiencia`: `RENAME`, nunca `DROP`/`CREATE`, renomeando *constraints*, índices e políticas junto. Duas armadilhas registradas lá: `FOR ... IN SELECT` sobre o catálogo **relê o que o próprio laço alterou**, e `RENAME CONSTRAINT` renomeia **também** o índice que a sustenta — as duas produzem `exp_01_exp_01_..._pkey`.

Gatilho. Necessidade de fronteira de `GRANT` por experiência — o que só faria sentido com mais de uma *role* de aplicação.

18.22 ADR-21 — *Push subscription* para a própria API

Decisão. O canal analítico é consumido por *push subscription* direto num *endpoint* da própria API (`POST /telemetry/ingest`), e não por um `{contrato}-telemetry-worker` separado.

Alternativa descartada. *Worker* dedicado, como previa o desenho original de infraestrutura. Seria um Cloud Run a mais com *deploy*, log, escala e *alerting* próprios.

Consequências. A rota grava dado de estudante e **não** passa pelo `OidcGuard`, então precisa de tranca própria: `PubSubPushGuard` verifica emissor Google, `aud` e `email` da conta de serviço de *push*, e **falha fechada** sem configuração. O Terraform casa os dois lados. Gravação idempotente por `ON CONFLICT`, porque a entrega é *at least once*.

Gatilho. Volume que justifique isolar o consumo do tráfego de estudante.

18.23 ADR-22 — exp-02 1.1.0: *briefing* antes do disfarce

Contexto. Até a 1.0.0 o estudante da exp-02 caía direto no *feed* de *Shorts* sem nenhum enquadramento: a primeira vez que a palavra "missão" aparecia para ele era no diálogo que abria sozinho depois do terceiro salvo — isto é, *depois* de a medida já ter sido tomada. Não havia contador visível de favoritos, não havia bandeja, e o teto de três se manifestava como um *toast* de 5,2 s. O Figma de redesenho (arquivo `oX1NGR8sPyduAS52hDaJFX`, seis telas de 1440×900, público de nove anos) acrescentou o enquadramento antes da ação e o retorno durante ela.

Decisão. Três mudanças, tomadas juntas:

1. **Tela de entrada na voz da escola.** O *briefing* usa a seção da **plataforma** (`@edu/ui/ds`) — a primeira vez que ela aparece *antes* da revelação. O critério da linha da revelação (Capítulo 14) continua sendo "quem está falando?": no *briefing* quem fala é a escola, e não há disfarce a proteger, porque a criança ainda não viu um *Short*.
2. **Conclusão pelo botão.** O terceiro salvo apenas habilita "Concluir missão" na bandeja; o diálogo abre no clique.

3. **Ganho de inscritos visível.** Um *badge* `+N` ao lado do chip do canal, por 1,4 s, quando o número sobe.

Alternativas descartadas. (a) Reproduzir o *wireframe* ao pé da letra, com o azul e o tema claro fixo dele — forçaria a paleta da plataforma para dentro da ficção e quebraria o seletor claro/escuro do estudante; o Figma é *wireframe*, e vale como especificação de estrutura, texto e estados, não de cor. (b) Reduzir o catálogo a oito vídeos, como o *wireframe* ilustra — mexeria no instrumento (o balanço 10 *fake* / 11 factual) e na comparabilidade com dados já coletados. (c) Manter a abertura automática do diálogo no terceiro salvo, com o botão como caminho alternativo — duas portas para a mesma coisa, e a primeira empurra a criança para a virada no instante do toque. (d) Animar só o total, sem o *badge* de delta.

Consequências ruins, aceitas. O *badge* torna legível **durante** a missão que um vídeo vale +1.000 inscritos e outro +50, ou seja, expõe o padrão que o *ranking* existe para revelar. Foi aceito porque o total já se movia — o delta sempre foi derivável de dois quadros seguidos — e porque a nota da etapa 3 do Figma decide isso explicitamente: o ganho varia, e o motivo **não** é explicado ali. A tela de entrada também gasta um toque a mais antes do *feed*, e um estudante apressado a atravessa sem ler.

O que não mudou. As curtidas continuam fora do cálculo; o gabarito (`factual`) continua no servidor e fora do *wire* — há teste que enumera os campos públicos do catálogo; o *score* continua saindo de `POST /unootube/credibilidade`. Nenhuma rota nova, nenhuma migração.

Efeito colateral corrigido no hub. O *feed* passou a montar depois de um clique *dentro* do `iframe`, e isso revelou um defeito que já existia: clicar dentro do `iframe` tira o foco da janela do hub, o `ExpFrame.tsx` traduzia esse `blur` em `hub:pause`, e o `focus` correspondente nunca voltava enquanto o estudante continuasse ali — o vídeo parava no primeiro toque em "Salvar" e não voltava mais, em qualquer versão. A correção é uma linha: quando `document.activeElement` é o próprio `iframe`, o foco foi para a experiência e não há perda de visibilidade nenhuma.

Gatilho de reavaliação. Distribuição de `fontesConfiaveis` da turma subindo de forma anômala — o *badge* é o primeiro suspeito. Ou o professor relatando que a turma atravessa o *briefing* sem ler, o que pediria que ele fosse do professor e não da tela.

18.24 ADR-23 — *Function calling* entra no `AiProvider`

> **Contexto**
>
> A exp-03 (Missão Tempestade) tem um agente conversacional em que **nenhum número pode sair do modelo de linguagem**: as estimativas vêm de um motor probabilístico determinístico, e o agente apenas narra o que a ferramenta devolveu. O projeto de origem implementava isso com um laço de *function calling* do SDK do vendor — até seis idas e voltas por turno, com três tentativas que reaproveitam o motivo da recusa anterior. A in-

> terface `AiProvider` desta plataforma tinha `generate`, `generateJson` e `generateImage`: nenhuma delas conversa.

Decisão. A interface ganha um quarto método, `conversar`, com tipos neutros de vendor (`AiFerramenta` reusa o `AiJsonSchema` que já existia). O laço de ferramentas, a preservação da assinatura de raciocínio e o *retry* que aprende ficam dentro de `vertex.provider.ts`; as regras de recusa ficam no domínio, passadas como *callback* `validar`. `tokensUsados` é a soma de todas as voltas e de todas as tentativas, **inclusive as rejeitadas** — é o que foi pago.

Alternativas descartadas.

- *A experiência falar com o vendor*: violaria a regra de que só `src/ai` conhece o provedor, e poria a chave e o protocolo do vendor no caminho de uma experiência.
- *Módulo da experiência chamar o SDK do vendor diretamente*: mesmo problema um nível acima — o dia de trocar de provedor passaria a exigir mexer em cada experiência que conversa.
- *Resposta estruturada em vez de ferramentas* (`generateJson` devolvendo uma lista de ações): funcionaria para um turno simples, mas o agente encadeia *carregar fonte* → *cruzar dados* → *narrar* decidindo cada passo com o resultado do anterior. Uma lista de ações declarada de antemão não tem essa realimentação.

Consequências.

- Qualquer experiência futura ganha agente com ferramentas de graça, e a troca de provedor continua sendo um arquivo novo em `src/ai`.
- *Ruim*: `conversar` é obrigatório na interface, então as três implementações existentes tiveram de ganhá-lo — inclusive os *fakes* dos testes de outros módulos.
- *Ruim*: o `MockProvider` não conhece os *slugs* do domínio, então chama a primeira ferramenta declarada com argumento vazio. Em desenvolvimento o mapa não enche pela conversa; enche pelo painel.
- Três defeitos do provedor ficaram cobertos por teste com `fetch` dublado: a assinatura de raciocínio reenviada *verbatim* (sem ela o Vertex recusa a continuação com 400 e a narrativa nunca chega), o descarte das partes de pensamento (senão o raciocínio do modelo aparece na tela do estudante) e o teto de voltas (modelo preso pedindo ferramenta vira recusa, não bolha vazia).

Gatilho de reavaliação. Um provedor cujo protocolo de ferramentas não caiba na assinatura de `conversar` — por exemplo, ferramentas paralelas na mesma volta. Ou latência de turno acima do que a sala tolera, que empurraria para `streamGenerateContent` e mudaria a forma do retorno.

18.25 ADR-24 — Teto de custo de IA por SESSÃO

> **Contexto**
>
> O `QuotaService` limita tokens por usuário e por dia, com teto por perfil (`AI_DAILY_TOKEN_QUOTA_PER_ESTUDANTE` valia 20.000). Esse número foi dimensionado para o tutor da exp-01: uma conversa curta. O agente da exp-03 tem *prompt* de sistema de cerca de 11.500 caracteres, reenviado inteiro a cada volta de ferramenta e a cada tentativa; uma partida de sete turnos fica na ordem de 100 a 200 mil tokens — duas ordens de grandeza acima.

Decisão. O controle que importa passa a ser **por sessão**, no módulo da experiência: `tokens_gastos` e `chamadas_ia` na linha da partida, com tetos em `regras/defaults.ts`. A quota diária continua como **fusível da plataforma** e sobe para acomodar duas partidas. A checagem acontece antes de chamar o provedor, e o consumo é registrado no `finally` — inclusive quando o turno falha, porque o token foi gasto de todo modo.

Alternativas descartadas.

- *Quota por experiência no `QuotaService`*: um limite por `expId` em variável de ambiente. Não mexeria no teto do tutor, mas complicaria um serviço que hoje é simples e por perfil — e continuaria sendo por dia, quando o que tem começo e fim é a sessão.
- *Encolher o instrumento para caber em 20.000*: reduzir o *prompt*, as voltas de ferramenta e o histórico. Cabe na quota atual e mexe justamente no que foi calibrado em *playtest*.
- *Teto em centavos*, como no projeto de origem: preço depende de tabela do vendor, que muda sem aviso e não é conhecida pela aplicação. Token é o que o provedor devolve.

> **Medição — 2026-08-27, `gemini-3.6-flash` no endpoint `global`**
>
> Uma **partida completa** (8 turnos, 7 chamadas de IA, do briefing ao desfecho) custou **71.397 tokens** em 40 segundos.
> Por turno: 7,5k a 8k na maioria, com dois picos (20,4k e 16,4k) nos turnos de mais voltas.
> O turno de conversa pura — "não entendi, explica de novo" — custou 3.846 e **não debitou pergunta**, que é a regra funcionando.
> E o que domina o custo **não é o tamanho do contexto: é o número de voltas** de ferramenta, porque cada volta reenvia o histórico inteiro. Um turno de duas voltas custou 11.481; o mesmo turno com quatro fontes já carregadas e seis falas de histórico, mas uma volta, custou 8.463 — *menos*, com mais contexto.

Os números, com a aritmética. `TETO_TOKENS_SESSAO` = 150.000: o dobro de uma partida completa medida, o que cabe a missão inteira com folga e ainda corta antes de o custo virar problema. `TETO_CHAMADAS_SESSAO` = 24: ao custo medido o teto de *tokens* morde primeiro, por volta do 19º turno — este é a rede contra o laço, não o controle de custo.

ATENÇÃO

A medição encontrou um defeito que a estimativa não teria encontrado: com `AI_DAILY_TOKEN_QUOTA_PER_ESTUDANTE` em 20.000 — o valor dimensionado para o tutor da exp-01, e que estava **fixo no *compose* de produção** — a missão recusava o **quarto turno** com "a sala de operações fechou por hoje". A quota subiu para 300.000 (cerca de quatro partidas, porque "Recomeçar" abre partida nova) em quatro lugares: os dois *composes*, o *default* da validação de ambiente e o `.env.example`.

Consequências. O custo fica limitado onde ele acontece, e a mensagem de estouro é uma frase de jogo ("a sala de operações fechou por hoje") em vez de vocabulário de infraestrutura — o estudante não sabe o que é quota. *Ruim*: são dois tetos no mesmo caminho, e um estouro exige olhar os dois para saber qual disparou.

Gatilho de reavaliação. A primeira medição real. E a terceira experiência com agente conversacional, que tornaria o teto por sessão um padrão a subir para a plataforma.

18.26 ADR-25 — O Relógio da Aula é tabela da experiência

Contexto

A Missão Tempestade tem uma restrição de tempo que é parte do instrumento: cinco minutos de relógio de parede valem a hora do cenário. No projeto de origem esse cronômetro armava sozinho quando o estudante entrava na investigação — e numa sala isso é uma corrida com largadas diferentes: quem chegou primeiro terminava antes de o professor acabar de explicar. As colunas do relógio moravam em `turmas`.

Decisão. O relógio é da **turma**, e vive em `exp_03_relogio_turma` — tabela *da experiência*, com `escola_id` e RLS como as outras. Quem dá a largada é o professor, no painel, e ela vale para a sala inteira ao mesmo tempo. Enquanto `iniciado_em` é nulo o estudante vê o cronômetro parado e o campo de fala desabilitado.

Alternativa descartada. *Colunas em `turmas`*, como no origem. `turmas` serve a plataforma inteira: uma coluna de cronômetro de uma missão faria toda experiência futura herdar o vocabulário dela, e a próxima que precisasse de tempo teria de escolher entre reusar colunas com nome errado ou adicionar as suas. É o mesmo raciocínio do prefixo `exp_NN_` (**ADR-20**).

Consequências.

- A sala começa junto, e a espera passa a ser a aula — não uma tela travada. A ficha do professor abre com essa faixa, porque "não abre" é o suporte que ela evita.
- A extensão de tempo usa um **pivô**: sem ele, esticar de cinco para sete minutos jogaria a cidade do minuto 48 para o 34, e a chuva andaria para trás na cara da turma.
- A largada é **idempotente** — é exatamente o clique que um professor nervoso dá duas vezes.

- Estudante sem turma vê "aguardando", que é a verdade. O *fallback* do origem armava um cronômetro individual, e com ele um estudante solto jogava sem largada.
- *Ruim*: o professor passa a ter uma ação sem a qual a aula não acontece. É dependência operacional nova, e o suporte precisa saber dela.

Gatilho de reavaliação. Uma segunda experiência com tempo de sala — aí o relógio da turma vira serviço da plataforma, não tabela de uma missão.

18.27 ADR-26 — Sem prazo de retenção na exp-03

> **Contexto**
>
> O projeto de origem tinha `expires_at` de 30 dias em `messages` e `marcos`, com uma rota de *cron* que o cumpria. Esta plataforma não tem *job* agendado, por decisão — e já registrou duas vezes o custo de uma coluna de prazo sem rotina: o `expires_at` da exp-70 e o `turmas.consentimento_em` do aplicativo de origem, onde a regra saiu do código, a coluna ficou, e quem leu o *schema* acreditou nela.

Decisão. Nenhuma das cinco tabelas da exp-03 tem coluna de prazo. No lugar entra **um** caminho de exclusão que existe de fato: "zerar a turma" (`DELETE /unoogpt/professor/turmas/:turmaId`), acionado por quem tem a posse pedagógica, apagando as partidas *daquela turma* e devolvendo o relógio ao início.

Alternativas descartadas.

- *Portar o `expires_at` de 30 dias*: prazo que nenhuma rotina cumpre é pior que prazo nenhum, porque cria a crença de que há retenção.
- *Trazer o prazo E o job*: seria o primeiro agendador da plataforma, com o custo operacional que a decisão de não ter agendador evitou. Se um contrato exigir prazo, ele volta **junto com a rotina**, nunca só a coluna.
- *`DELETE /unoogpt/dados` no estilo do bem-estar*: lá o professor é dono das linhas; aqui o dado é do estudante, e um botão "apagar tudo" no professor seria posse que ele não tem.

Consequências. O relógio é apagado junto, e o diálogo diz por quê: rodar de novo com a aula "já começada" faria a segunda rodada nascer no meio da tempestade. *Ruim*: sem prazo, a transcrição de um menor de idade fica no banco até alguém apagar — e a pergunta de retenção continua aberta para `telemetria_eventos` e para a plataforma inteira, não só para este módulo.

Gatilho de reavaliação. Exigência contratual de prazo, ou a primeira rotina agendada da plataforma — a partir dela o custo marginal de cumprir um prazo cai muito.

18.28 ADR-27 — Transcrição é dado do instrumento

> **Contexto**
>
> A missão inteira acontece numa conversa em texto livre, escrita por menor de idade. O professor precisa ler essa conversa: é nela que aparece *como* a criança perguntou, e o relatório sem ela é uma lista de contagens. Ao mesmo tempo, o `AGENTS.md` registra a trava oposta na exp-70 — o campo de texto livre foi removido do *payload* porque era o caminho por onde o nome de uma criança chegaria a um JSONB de 90 dias.

Decisão. A transcrição vive em coluna dedicada (`exp_03_mensagens.texto`), com RLS por escola, e é servida **apenas ao professor**: `UnoogptTurmasController` tem `@Roles("professor")`, e o agregado do gestor está em `UnoogptAgregadoController`, que **não tem nenhuma rota que aceite `estudanteId`**. Ela nunca vai a log nem à telemetria.

Alternativa descartada. Um controller só, com `@Roles` incluindo gestor e uma exceção dentro do método que devolve a criança. É a forma que vaza: basta um método novo esquecer a exceção. Com dois controllers, nenhum descuido futuro leva o gestor à conversa de uma criança, porque não existe rota para isso no controller dele.

Consequências.

- O relatório individual abre pelo **nível** — graus, com `zero` (“teve a chance e não foi”) distinto de `none` (“a oportunidade não chegou”) — antes do que a criança fez. É a regra da exp-70, e existe para o adulto não concluir sobre a criança o que o instrumento não mediu.
- A **listagem** da turma não tem coluna de resgatados: listagem é ordenável, e ordenar crianças por acerto viraria *ranking*. O *smoke* visual falha se uma aparecer.
- A telemetria analítica carrega contagem, nunca o texto.
- *Ruim*: o dado mais sensível da experiência não tem prazo (**ADR-26**), e a única forma de apagá-lo é uma ação do professor.

Gatilho de reavaliação. Pedido de visão de rede sobre conduta — aí o caminho é agregado sem identidade no controller do gestor, nunca abrir a linha da criança.

18.29 ADR-28 — Identidade por pessoa; OneRoster na fronteira

> **Contexto**
>
> As experiências foram portadas de projetos onde **cada atividade tinha o próprio cadastro de usuário**. No monorepo a unificação técnica já existia — toda experiência recebe `estudante.id` pelo `hub:init` e grava `estudante_id` com chave estrangeira para `users` —, mas três resíduos impediam que “acompanhar por estudante” fosse verdade:

> 1. A conta era de **assento**, não de pessoa. Está escrito no *schema* desde o porte: `exp_01_runs.nome_no_app` existe "porque a conta é do APARELHO: outra turma reusa o mesmo login". O nome real da criança vivia dentro de UMA experiência, e nenhuma das outras o enxergava.
> 2. Não existia **enturmamento**. O vínculo era a coluna `users.turma_id`: um estudante cabia em uma turma só, o PROFESSOR não tinha turma nenhuma — e por isso todo professor enxergava todas as turmas da escola — e não havia ano letivo. Em fevereiro o 8A vira 9A e todo o acumulado do ano anterior passaria a ser lido pela turma nova, em silêncio.
> 3. A entrada de pessoas fazia *upsert* por `usuario` e exigia **senha em texto claro no arquivo**. Corrigir um login criava um segundo estudante, com o histórico preso no primeiro.

Decisão. O modelo interno passa a ser **isomórfico ao OneRoster 1.2** (1EdTech): entram `periodos_letivos` (`academicSessions`) e `matriculas` (`enrollments`, a tripla usuário × turma × escola com papel e datas), mais as colunas de identidade em `users`, `turmas` e `escolas` (`matricula_externa`, `fonte_id`, `codigo_inep`).

O padrão vive **só na fronteira** `apps/api/src/rostering/` — a mesma disciplina de `src/ai`, onde apenas o *provider* conhece o fornecedor. Nada do domínio conhece `sourcedId`, o contrato do `hub:init` não muda, e os nomes seguem em pt-BR.

Alternativas descartadas.

- *Modelar do zero, sem padrão*: chegaríamos a quase a mesma tabela. O padrão não é o motivo — é a conferência de que o desenho está certo, o vocabulário de graça e um argumento de edital.
- *LTI 1.3 / NRPS*: responde "quem chegou agora", em tempo real, e pressupõe estar embutido num LMS. Não provisiona quem ainda não entrou, que é exatamente o caso da matrícula da rede antes do ano letivo.
- *Ed-Fi*: padrão de *data warehouse* de rede; escopo muito maior que o problema, para uma equipe de uma pessoa.
- *Esperar o padrão nacional*: o MEC publicou a Portaria 269, de 24/03/2026, criando a Infraestrutura Nacional de Dados da Educação (EducaDados) e o Padrão Nacional de Interoperabilidade, com implantação gradual e especificação técnica ainda não publicada. Adotar o *modelo* agora e manter o *formato* atrás de adaptadores é o que serve aos dois futuros.
- *Adotar também os bindings do padrão*: o CSV do OneRoster são sete arquivos com identificadores globais; nenhum secretário de escola os preenche. A tela oferece uma **planilha simples**, de uma linha por pessoa, e o mesmo núcleo aplica o lote.
- *`demographics` do OneRoster*: raça e etnia no vocabulário do censo dos Estados Unidos, além de nascimento e sexo. Dado sensível (LGPD art. 11) que a plataforma não usa para nada — não entra nem como coluna.

Consequências.

- `users.turma_id` **sobrevive**, com novo significado: cache da matrícula principal. O *login* e o *claim* `turma_id` do *token* (contrato congelado) o leem antes de haver escola no contexto, onde `matriculas` — que tem RLS — não é alcançável.
- "Coordenação" continua sendo `gestor`: `PAPEIS` é contrato congelado entre o hub e a API, e um quinto papel atravessaria *middleware*, *guards* e RLS. A planilha aceita sinônimos (`coordenacao`, `diretor`, `principal`) e normaliza.
- *Ruim*: quatro tabelas e um módulo a mais, e uma migração nas duas tabelas mais centrais três meses antes de produção.
- *Armadilha registrada*: consulta que leia `matriculas` fora de `comTenancy` volta **vazia**, sem erro. O sintoma é lista de pessoas "sem turma".

Nota de implementação — o binding CSV, arquivo por arquivo. O padrão empacota os sete arquivos num zip com `manifest.csv`. Aqui cada um é enviado em uma chamada (`POST /rostering/oneroster` na ordem de dependência: empacotamento é embalagem, não semântica, e aplicar um por vez dá o mesmo resultado sem dependência de descompactação e sem lote pela metade guardado em lugar nenhum. Fora de ordem também não corrompe: a linha que aponta para um `sourcedId` inexistente volta no relatório **dizendo qual arquivo falta**, em vez de criar meia entidade. Quatro decisões dentro dessa leitura:

- `orgs.csv` só **aprende** o código INEP da escola desta importação. Escola é *tenant*: criar um a partir de arquivo enviado por *upload* seria abrir um inquilino com *stack*, RLS e dados de menores por trás de um clique.
- Na 1.2 o papel saiu de `users.csv` e foi para `roles.csv`. A coluna `role` é aceita quando vem (provedores seguem emitindo a da 1.1); sem ela, a pessoa nasce como **estudante** — o menor privilégio — e o relatório diz quantas linhas dependem do `roles.csv`. Falhar fechado: um professor que entra como estudante perde acesso e reclama; um estudante que entrasse como professor leria a turma inteira.
- A coluna `password`, que o padrão permite, é **ignorada** (**ADR-29**) — e a contagem de linhas que a traziam vai ao relatório, para não sumir em silêncio.
- Em `bulk`, ausência de linha **não** desativa ninguém: é contada e listada. Lote completo que chegou pela metade não pode apagar uma rede por omissão.

Gatilho de reavaliação. Publicação da especificação técnica do EducaDados — em especial se ela adotar o CPF como identificador do estudante, que entra em coluna de reconciliação de acesso restrito e **nunca** em telemetria, JSONB ou *claim* de *token*.

18.30 ADR-29 — Senha provisória com troca obrigatória

> **Contexto**
>
> Antes da integração com o WSO2, alguém precisa criar as contas e entregar uma credencial a 10 mil pessoas que nunca acessaram a plataforma. O formato anterior resolvia isso com uma coluna `senha` na planilha — arquivo que circula por e-mail, WhatsApp e *pen drive*, e que fica no computador da secretaria.

Decisão. A planilha **não tem coluna de senha**. A senha inicial é escolhida na tela de importação, em dois modos: uma para o lote inteiro, ou uma por pessoa (recomendado para estudantes). Quem nasce da importação recebe `senha_provisoria = true`, e o primeiro acesso cai em `/trocar-senha` sem saída. Reimportar **não** redefine a senha de quem já entrou.

Alternativas descartadas.

- *Manter a senha na planilha*: é o vazamento por construção, e a reimportação passaria a redefinir a senha de quem já trocou.
- *Delegar tudo ao WSO2 agora*: o piloto precisa de contas antes do provedor de identidade existir, e a plataforma continuaria sem caminho para a escola sem federação.
- *Link mágico por e-mail*: a maioria dos estudantes da rede pública não tem e-mail institucional em uso, e a plataforma não envia e-mail hoje.

Consequências.

- A trava é **tripla**: *middleware* (páginas), BFF (dados) e `/api/exp-token`. Só com o *middleware*, bastaria digitar `/exp/...` para pular a troca.
- Ao trocar, a sessão é **refeita** com a senha nova, não remendada: o *cookie* antigo ainda diz "precisa trocar", e sem refazê-lo o *middleware* devolveria a pessoa à mesma tela — agora com a senha anterior já inválida. É o beco sem saída mais fácil de criar neste fluxo.
- As senhas geradas são devolvidas **uma vez**, para impressão. Não ficam em coluna nem em *log*; depois disso o caminho é o reajuste de senha pelo gestor, que remarca a conta como provisória.
- *Ruim, e dito na própria tela*: com uma senha só para o lote, qualquer estudante entra na conta de outro até o primeiro acesso dele. Por isso o modo "uma por pessoa" existe e é o recomendado para turma grande.
- O *seed* de desenvolvimento nasce com `senha_provisoria = false`: prender os usuários de demonstração na troca pararia os seis suítes de ponta a ponta na mesma tela.

Gatilho de reavaliação. Entrada do WSO2 em produção — a partir daí a senha local vira caminho de exceção (escola sem federação), não o principal.

18.31 ADR-30 — A rede: lotação por escola e escola ativa no *token*

> **Contexto**
>
> Vender a uma rede estadual não é vender a uma escola grande. São milhares de unidades, professores que dão aula em mais de uma, e crianças que se transferem entre elas. O modelo tinha `users.escola_id` **único**, e ele é a chave de *tenancy*: circula no *claim* `escola_id` do JWT e no `set_config` da RLS. Um professor em duas escolas não caberia nele.
>
> O tamanho do problema, medido: **31 tabelas** carregam `escola_id` e **19 arquivos** da API derivam a escola do *token*.

Decisão. O *token* continua carregando **UMA** escola. Nenhuma das 31 tabelas muda, nenhum dos 19 arquivos muda, a RLS continua sendo `escola_id = current_setting('app.escola_id')` e a regra inviolável nº 1 continua literalmente verdadeira. O que entra é a camada *acima*:

- `lotacoes` — o vínculo PESSOA × ESCOLA, datado. "Lotação" é o termo que a rede pública usa para a alocação de um servidor a uma unidade. Professor e gestor **acumulam**: lotação nova não encerra a anterior. Estudante tem **uma ativa**, garantida por índice único parcial no banco.
- **Escola ativa na sessão.** Quem tem mais de uma escolhe no cabeçalho; a troca passa pelo servidor (`POST /api/escola-ativa`), que valida contra `GET /escolas/minhas` — **nunca** contra a lista que o *cookie* carrega. *Cookie* é dado do cliente, e aceitar a escola que ele pedir deixaria qualquer professor ler qualquer unidade da rede.
- `users.escola_id` passa a ser **cache** da lotação principal — o mesmo papel que `turma_id` já tinha em relação a `matriculas`.

Alternativas descartadas.

- *Token com várias escolas e RLS em* `= ANY(...)`: tocaria as 31 tabelas e trocaria a garantia "esta conexão vê UMA escola" por "vê um conjunto". A propriedade que sustenta o isolamento de um contrato B2G é justamente a primeira.
- *Um usuário por escola (dois cadastros para o mesmo professor)*: é o que acontece hoje sem esta camada, e é o problema — duas senhas, dois logins, e nada ligando as duas metades da mesma pessoa.
- *`lotacoes` com RLS*: ela é lida no LOGIN, antes de se saber a escola. Uma política por `escola_id` esconderia justamente a resposta ("quais escolas são suas"), e o sintoma seria o professor de duas unidades não conseguindo trocar para a segunda. Fica fora, como `users`/`turmas`, com filtro explícito nos *services*.

Consequências.

- Criar escola é `admin-contrato` e ninguém mais — inclusive no `orgs.csv`. Escola é *tenant*: abrir inquilino com RLS, dados de menores e *stack* por trás não é operação de quem administra uma unidade.
- *Ruim*: a escola ativa é estado de SESSÃO. Duas abas do mesmo professor em escolas diferentes brigam pelo mesmo *cookie* — a última troca vence, e a outra aba passa a mostrar a escola nova sem avisar.
- *Ruim*: o campo extra que eu pus dentro do item da `SideBar` para filtrar o menu virou atributo no DOM (o tipo do DS tem índice aberto e ele espalha o que não conhece). A restrição foi para fora do item; fica registrado porque é o tipo de erro que o *typecheck* não pega.

Gatilho de reavaliação. Um contrato em que a MESMA pessoa precise ver duas escolas ao mesmo tempo na mesma tela — aí a conversa volta para o *token* com conjunto de escolas, e com ela o custo nas 31 tabelas.

18.32 ADR-31 — CPF como identidade, guardado só em *hash*

> **ATENÇÃO**
>
> **SUPERADA pela ADR-33.** O *hash* entregava o que a *plataforma* precisa do CPF e não o que o *gestor* precisa: ver o número. Fica registrada porque o raciocínio continua valendo para qualquer dado que a plataforma só precise *comparar* — e porque a lição de processo é a parte mais útil dela.

> **Contexto**
>
> Numa rede, a criança troca de escola e o dado dela precisa continuar sendo dela. Nome muda (casamento dos pais, correção de registro), *login* muda, matrícula muda a cada rede — e a escola de origem e a de destino não compartilham nenhuma dessas chaves. Sem um identificador imutável, cada transferência cria uma segunda pessoa e o histórico nasce partido em duas.
> O identificador que o Brasil usa é o CPF, e é o que a Portaria MEC 269/2026 adota como chave do estudante no EducaDados. É também o dado pessoal mais sensível que esta plataforma tocaria, de menores de idade, num banco que já guarda perfil de valores e *check-in* de saúde.

Decisão. O CPF é a chave de identidade e **nunca** é guardado. O banco tem `users.cpf_hash` = `HMAC-SHA256(cpf, CPF_PEPPER)` e `users.cpf_final`, os três últimos dígitos em claro para confirmação humana na tela. O *hash* é **determinístico** de propósito: é o que permite procurar (basta *hashear* o que o gestor digitou) e reconhecer a mesma pessoa. Único por **contrato**, não por escola — é a identidade da pessoa na rede inteira.

Alternativas descartadas.

- *Guardar o número em claro*: dá tudo o que o *hash* dá e mais um acervo de CPF de menores, que a plataforma não precisa para nada — ela não emite documento nem presta contas ao fisco. O SIS da rede já tem o número.
- *Hash com sal por linha* (`bcrypt`): inutilizaria o campo como identidade. Sem determinismo não há busca nem reconhecimento, que é a única razão de o CPF estar aqui.
- *Hash sem pimenta*: CPF tem 11 dígitos e espaço de busca trivial — um SHA-256 puro é reversível por força bruta em minutos. A pimenta é o que torna o *hash* inútil fora desta instalação.
- *Cifrar em vez de hashear*: exigiria guardar a chave capaz de devolver o número, e o número é exatamente o que não se quer poder devolver.

Consequências.

- `CPF_PEPPER` **não muda**. Trocá-la invalida todos os *hashes* e a plataforma deixa de reconhecer quem já cadastrou — o sintoma é toda criança transferida virando cadastro novo. Ausente, a coluna `cpf` da importação é **recusada** com mensagem explícita, nunca aceita e guardada de outro jeito.
- O dígito verificador é conferido na leitura da planilha. Não é preciosismo: CPF digitado errado numa planilha de 10 mil linhas cria uma identidade fantasma que nunca vai casar com a da rede, e o erro só aparece meses depois, na primeira transferência.
- O CPF é **opcional**. Rede sem o dado continua funcionando com `matricula_externa` como chave — e perde só o reconhecimento automático na transferência.
- *Ruim*: três dígitos em claro são identificáveis em combinação com o nome. É o preço de o gestor conseguir confirmar que é a pessoa certa antes de transferir uma criança.
- O teste estrutural `identidade-unica.spec.ts` falha se aparecer qualquer coluna com `cpf` no nome fora de `users.cpf_hash` e `users.cpf_final`.

Gatilho de reavaliação. A especificação técnica do EducaDados exigir *enviar* o CPF de volta à rede — aí ele volta a entrar, e a decisão passa a ser onde a chave de cifra mora, não se o número existe.

18.33 ADR-32 — Transferência: o dado fica, a identidade segue

> **Contexto**
>
> "Os dados do estudante seguem o estudante, independente da escola" é requisito de produto. E colide de frente com a regra inviolável nº 1: tudo o que a criança produziu tem `escola_id` da escola onde produziu, com RLS por cima. Depois de uma transferência, a escola nova não alcança nada disso — e a antiga continua alcançando tudo.

Decisão. O **dado fica onde foi produzido**; o que segue o estudante é a **identidade**. A transferência encerra a lotação e as matrículas na origem, abre na destino e move o *cache* `users.escola_id` — e mais nada. Para a leitura atravessada existe **uma** porta: `GET /estudantes/:id/historico`,

que devolve **participações** (em que escola, em que turma, em que missão, quando), nunca o conteúdo dos instrumentos.

Alternativas descartadas.

- *Reescrever `escola_id` nas 31 tabelas na transferência*: apaga de onde o fato veio. "Esta criança respondeu isso na escola A, em 2026" deixaria de ser recuperável, e um *UPDATE* em massa por transferência é a operação mais fácil de deixar pela metade.
- *Afrouxar a RLS para o `estudante_id` atravessar*: transformaria a exceção numa regra geral. A porta única é auditável; uma política mais larga não é.
- *O histórico devolver o instrumento completo*: abriria, por uma tela de secretaria, o retrato de valores da Vila e a transcrição do unooGPT — o que **ADR-12**, **ADR-27** e **ADR-10** fecharam. Quem lê o instrumento é quem tem a posse pedagógica na escola onde ele aconteceu.

Consequências.

- O `historico` só abre para quem tem **lotação na escola onde o estudante está hoje** — é a unidade responsável por ele — ou para `admin-contrato`.
- A transferência é a única operação que troca o `app.escola_id` **no meio de uma transação**: ela encerra na origem e abre no destino, e sem trocar o contexto o *update* na origem não veria linha nenhuma (a RLS o esconderia, sem erro, e a criança ficaria matriculada nas duas escolas). O `set_config(..., true)` é local à transação, então a conexão volta limpa ao *pool*.
- Quem recebe a criança é a escola de **destino**: um gestor só transfere PARA a própria escola, e o caminho normal é importar a planilha com o CPF — o importador reconhece e transfere.
- *Ruim*: o professor da escola nova vê que a criança participou de uma missão na escola anterior, mas não o que ela respondeu. Para continuar um trabalho pedagógico iniciado em outra unidade, isso é pouco — e é deliberado até haver uma regra de consentimento que diga o contrário.

Gatilho de reavaliação. Pedido pedagógico formal de continuidade entre escolas da mesma rede — aí a conversa é sobre consentimento e sobre QUAL instrumento atravessa, um a um, nunca sobre abrir a fronteira inteira.

18.34 ADR-33 — O CPF é guardado, e o gestor o vê

> **Contexto**
>
> **ADR-31** decidiu guardar apenas `HMAC-SHA256(cpf, CPF_PEPPER)` e os três últimos dígitos. A decisão pesou o risco de o banco acumular CPF de menores e **não perguntou quem usa o dado**.
> Quem usa é o gestor da escola, e ele precisa **ver o número**: confere identidade contra o sistema de gestão da rede, preenche documento de transferência e atende a família no

> balcão com o documento na mão. *Hash* é de mão única — não existe configuração que o exiba, porque não existe chave. Não era uma limitação a ajustar; era a funcionalidade ausente por construção.

Decisão. O CPF é guardado: `users.cpf`, os 11 dígitos sem pontuação (a formatação é da tela, e um *CHECK* no banco impede que "529.982.247-25" e "52998224725" coexistam como duas pessoas). Único por **contrato**, não por escola. Exibido formatado na lista do gestor, buscável com ou sem pontuação, e corrigível na edição — o balcão erra, e um dígito trocado numa planilha de dez mil linhas só aparece meses depois, na primeira transferência.

Sobre a base legal. CPF é dado pessoal **comum**, não dado sensível do art. 11 da LGPD — que trata de saúde, raça, religião, filiação e biometria. O tratamento tem base clara num contrato de execução de política pública de educação, e é o que todo sistema de gestão escolar brasileiro guarda. O cuidado que a plataforma tem com `exp_91_checkins` (saúde) não é o mesmo caso.

Alternativas descartadas.

- *Manter o hash e exibir os três dígitos*: é o que havia, e não atende — três dígitos não servem para conferir documento nem para preencher formulário.
- *Hash para identidade + coluna cifrada para exibição*: duas representações do mesmo fato, uma chave a operar e a rotacionar, e um vazamento de *dump* + chave devolve o número de todo modo. O ganho real sobre a cifra de disco que o Cloud SQL já aplica é pequeno; a complexidade é permanente.
- *Exibir mascarado na lista e completo só no cadastro*: foi considerado e recusado pelo dono do produto — o gestor confere pela lista, e obrigá-lo a abrir cada pessoa transforma uma conferência de turma em trinta cliques.

Consequências.

- **O que sustenta a decisão não é a coluna, é a contenção.** `users.cpf` é o ÚNICO lugar onde o número existe: nunca em *log* (nem em mensagem de erro), nunca em telemetria, JSONB, *claim* de *token* ou *bundle* de experiência. O teste estrutural cobra pelos dois lados — que só exista essa coluna, e que nenhum JSONB de `telemetria_eventos` ou `progresso` carregue onze dígitos seguidos, que é o caminho mais fácil de o número escapar.
- O índice único é da **rede**, não da escola. Duas importações simultâneas com o mesmo CPF colidem no banco — é o comportamento correto, e o efeito colateral apareceu primeiro nos testes, onde duas suítes usavam o mesmo número.
- `CPF_PEPPER` deixou de existir.
- *Ruim*: o banco passa a ser um acervo de CPF de menores de idade. A proteção é de acesso (só gestor e admin-contrato alcançam a rota) e de contenção, não de criptografia de aplicação.

- *Lição de processo, registrada porque custou retrabalho*: uma escolha **irreversível** — *hash* não volta atrás — foi tomada sem o requisito de quem usa o dado estar escrito. A pergunta "quem precisa LER isso, e como?" devia ter vindo antes da primeira migração.

Gatilho de reavaliação. Exigência contratual de cifra em nível de aplicação, ou um incidente de acesso indevido — aí a conversa é sobre onde a chave mora e quem a usa, não sobre se o número existe.

18.35 ADR-34 — A regional agrupa; quem autoriza é a lotação

> **Contexto**
>
> Uma rede estadual não chega organizada em escolas soltas. Em São Paulo são **Diretorias de Ensino**; em outras redes, polos ou secretarias municipais. Entre a escola e o contrato existe um nível intermediário real, com gente que trabalha nele: o supervisor que acompanha as cinquenta unidades da sua diretoria.
> A **ADR-30** deixou a plataforma pronta para a pessoa em várias escolas — a lotação existe, o *token* carrega uma escola por vez e o cabeçalho troca a escola ativa. O que faltava era o *agrupamento*: sem ele, pôr um supervisor nas cinquenta unidades é abrir cinquenta telas. As colunas `escolas.tipo` e `escolas.pai_id` tinham sido acrescentadas antes, "porque custam zero", e **nenhuma consulta as lia**. Era exatamente a armadilha que este projeto já documentou duas vezes — o `expires_at` sem rotina e o `turmas.consentimento_em` do aplicativo de origem: a regra sai do código, a coluna fica, e quem lê o *schema* acredita nela.

Decisão. O nível regional passa a ser **real e lido**, com um vocabulário fechado de três valores em `escolas.tipo` — `escola` (a unidade, onde há turma, estudante e dado), `regional` (o que agrupa unidades) e `rede` (a raiz). `pai_id` liga unidade a regional, e um *trigger* no banco recusa `pai_id` apontando para uma `escola` ou para si mesma.

E a decisão que dá sentido a tudo: **a hierarquia agrupa e nunca autoriza**. Estar numa diretoria não dá acesso a nada. Quem dá acesso é a lotação, uma por unidade. A regional é o *insumo* da lotação em lote (`POST /escolas/lotacoes-em-lote`) — ela monta a lista de escolas que o admin do contrato confirma antes de conceder.

Alternativas descartadas.

- *Um papel novo,* `supervisor`: `PAPEIS` é contrato congelado entre o *middleware* do *hub*, os *guards* da api e a RLS; um quinto papel toca os três. E não resolveria nada que a lotação já não resolva — o supervisor faz o que um gestor faz, em mais de uma escola.
- *Autorizar por hierarquia* ("quem está na regional lê as unidades dela"): é uma segunda fonte de verdade sobre acesso, ao lado da lotação, e as duas divergem no dia em que alguém corrige um `pai_id`. Pior: transformaria uma coluna *organizacional* em coluna de segurança, e a RLS deixaria de ser legível numa linha.

- *`regional_id` no token, com a api somando as escolas da diretoria*: quebraria a regra 1 em todas as 31 tabelas com `escola_id` e daria ao supervisor uma tela que mistura duas escolas — que é precisamente o que o isolamento lógico existe para impedir.
- *Aceitar o critério no lugar da lista* ("lote todos da regional X"): o servidor expandiria o critério e o humano confirmaria um número que não viu. A rota exige a **lista** de `escolaIds`, no máximo mil.

Consequências.

- **Nada mudou na tenancy.** O *token* continua carregando uma escola, a política de RLS continua sendo a mesma linha, e as 31 tabelas com `escola_id` não foram tocadas. O nível regional inteiro vive acima delas.
- A regional **não recebe lotação**: `lotarEmLote` ignora o que não for `tipo = 'escola'` e devolve a lista de ignoradas. Lotar alguém numa regional daria uma escola sem turma nem estudante no seletor do cabeçalho.
- O panorama (`GET /escolas/minhas/resumo`, tela `/admin/minhas-escolas`) **lista, não soma**: uma linha por unidade, com turmas, estudantes e quantos ainda não fizeram o primeiro acesso. Somar as escolas numa métrica só seria a tela que a decisão acima recusa.
- O `orgs.csv` do OneRoster passa a criar regionais a partir de `district`/`local` e a ligar unidades por `parentSourcedId`. Pai desconhecido vira *aviso* e a escola entra sem regional — perder a escola por causa do agrupamento seria trocar o essencial pelo acessório.
- *Ruim*: a lotação em lote é uma operação de poucos cliques que concede acesso a muitas escolas. A contenção é ser exclusiva de `admin-contrato`, exigir a lista explícita e mostrar os nomes antes de confirmar.
- *Lição de processo*: coluna acrescentada "porque custa zero" e nunca lida é dívida com aparência de recurso. Ou nasce com o consumidor, ou não nasce.

Gatilho de reavaliação. Uma rede que exija leitura agregada de fato — "o desempenho da diretoria inteira" — muda a pergunta: aí é relatório com escopo próprio (como `/vila/agregado/*` faz para o gestor), nunca alargamento do *token*.

18.36 ADR-35 — Perfil portátil: um documento assinado, não uma integração

> **Contexto**
>
> Um contrato é uma rede e uma *stack*. A **ADR-32** resolveu a criança que troca de escola DENTRO da rede: o dado fica onde foi produzido e a leitura atravessa por uma porta só. Fica de fora o caso que o dono do produto levantou: a criança que sai da rede estadual para a municipal — outro contrato, outro banco, nenhum *JOIN* possível.

> E há a restrição que decide o desenho: **não há garantia nenhuma de que a rede de destino comprou o produto**. Qualquer solução que exija duas instalações desta plataforma conversando resolve só o caso raro em que as duas existem.

Decisão. A saída é um **documento assinado**, e não uma integração. Um arquivo JSON `{ perfil, assinatura }` que a escola de origem emite a pedido do titular (LGPD art. 18, V — portabilidade) e que a escola de destino importa como um ato dela. Para uma rede que nunca vai instalar esta plataforma, o mesmo arquivo continua sendo um documento legível.

Quatro decisões dentro dessa:

- **O hub assina; a api verifica.** Assinar exige a chave privada, que só o hub tem — é a MESMA que assina os *tokens* de experiência e é publicada em `/.well-known/jwks.json`, então nenhum material de chave novo entrou. `aud: "perfil-portatil"` separa os dois usos. Verificar acontece na api porque é o passo que **não pode confiar em quem chama**: se a api recebesse o veredito pronto do hub, bastaria forjar o campo.
- **O que vem de fora nunca se mistura.** O documento vive em `perfis_portateis` e em lugar nenhum mais: não vira matrícula, progresso nem perfil da Vila, e não entra em média. É a afirmação de OUTRA instalação sobre a criança, possivelmente de outra `versao_mapeamento`. Separado, o professor lê os dois e sabe qual é qual.
- **Assinatura inválida é recusa; emissor inalcançável é rótulo.** São coisas diferentes e a tela diz qual foi. Não conseguir falar com o emissor é limitação de rede; assinatura que não confere é adulteração, e essa não entra.
- **O documento se liga a alguém que já está aqui.** Importar não cria estudante: a matrícula é ato da escola (planilha ou OneRoster), e só depois o perfil se anexa — pelo CPF, que é a única chave que significa alguma coisa numa base onde nenhum id da origem existe.

Alternativas descartadas.

- *API entre as duas instalações*: exige que as duas existam, que se conheçam e que uma libere acesso à outra. Não cobre o caso comum, que é a rede de destino não ser cliente.
- *Um banco só para várias redes* (`rede_id` em tudo): foi medido e descartado antes — e não resolveria este caso, porque a rede de destino continuaria fora da instalação.
- *Exportar CSV sem assinatura*: quem recebe não tem como saber se o arquivo foi editado no caminho, e um histórico escolar sem procedência não serve para decidir nada.
- *Assinar o documento como payload do JWS*: o arquivo viraria uma *string* opaca. Ele é entregue à FAMÍLIA, que tem direito de lê-lo; por isso o JWS cobre o *hash* canônico e o documento fica legível ao lado.
- *Buscar o JWKS de qualquer emissor*: o `emissor.hub` vem de dentro do arquivo, ou seja, de um desconhecido — seria SSRF. A allowlist `PORTABILIDADE_EMISSORES` é a decisão de confiança: o documento **seleciona** entre origens que o operador escreveu, e nunca introduz uma URL.

Consequências.

- **O CPF passa a existir num segundo lugar**, e é a única exceção à contenção da **ADR-33**: `perfis_portateis.documento`. O CPF é o CONTEÚDO do documento — é o que reconhece a criança lá fora — e é guardado *verbatim* porque alterá-lo destruiria a assinatura. A exceção está numa lista explícita dentro do teste estrutural, para que a próxima seja uma decisão e não um vazamento que ninguém viu.
- **Bem-estar (exp-91) nunca entra no documento.** DASS-21 é dado de saúde, a posse é do professor e não existe rota de gestor para ele em lugar nenhum. Um documento que o gestor emite não pode carregar o que nem o gestor lê.
- `PORTABILIDADE_EMISSORES` vazia (o default, e o caso de dev) faz TODO documento entrar como `nao-verificada`. Degradar visivelmente é melhor que recusar o direito do titular — e a tela sai em âmbar, não em verde.
- **A entrada da allowlist pode apontar o JWKS** (`origem=urlDoJwks`), e é isso que torna o caminho criptográfico exercitável fora de produção. O `emissor.hub` do documento é o endereço do **navegador** — a mesma distinção de `EXP_INTERNAL_HOST`, e pela mesma razão: de dentro do contêiner da api, `localhost:3000` é a própria api. A regra de canal passou a valer sobre a URL do JWKS: `https` sempre, exceto **fora de produção**, onde `http` é aceito e avisado no *log* (quem escreveu a entrada foi o operador, no servidor, e o tráfego não sai da máquina). Em produção, `http` é recusado — a chave pública viria por um canal que qualquer um reescreve.
- Sem isso, a garantia mais importante do recurso — documento adulterado é RECUSADO — não teria prova no ambiente onde se trabalha todo dia, e o *smoke* visual registrava o caminho degradado como se fosse o normal.
- O caminho criptográfico é coberto por teste com as chaves injetadas, e foi esse teste que pegou o pior erro possível deste recurso: a classificação do erro olhava o TEXTO da exceção, e a palavra "failed" de "signature verification failed" caía na regra de erro de rede. Documento FORJADO entraria marcado "não deu para conferir" em vez de recusado. Hoje a classificação é pelo `code` da biblioteca.
- *Ruim*: reemitir gera outro documento, e a escola de destino pode acabar com dois do mesmo estudante. A lista mostra data *e hora* e permite apagar; unificar por conteúdo esconderia que foram duas emissões.

Gatilho de reavaliação. A especificação técnica do EducaDados (Portaria MEC 269/2026) definir um formato nacional de transferência de histórico: aí este documento vira um *binding* daquele formato, e o que sobrevive é a decisão de ser documento assinado em vez de integração.

18.37 ADR-36 — O OpenAPI é derivado do Zod, não escrito ao lado dele

> **Contexto**
>
> A api tem 146 rotas em 27 *controllers* e nenhuma documentação executável: quem quisesse saber o que `POST /rostering/importar` aceita lia o *controller*. A única descrição existente é o Apêndice C deste documento — prosa escrita à mão, que nada obriga a concordar com o código.
>
> A complicação está em como a api valida. Não há DTO de classe: são 42 *schemas* Zod declarados como `const` dentro dos *controllers* e aplicados por `.parse()`/`.safeParse()` no corpo do método, sem *pipe* compartilhado. O `@nestjs/swagger` infere esquema de classe anotada com `class-validator`; sobre Zod ele não infere nada.

Decisão. O documento é **derivado** do *schema* que a rota já usa para validar. Um decorator único, `@Contrato`, declara resumo, prosa, corpo, *query* e **resposta**; `zod-to-json-schema` traduz para o dialeto do OpenAPI 3.0; e `openapi/documento.ts` monta cada operação.

Três decisões dentro dessa:

- **O `@Contrato` não chama `@ApiOperation`/`@ApiResponse`.** Ele só grava metadata. Um decorator não enxerga o verbo da rota — o `@Get`/`@Post` pode ser aplicado depois dele, e a ordem de avaliação é de baixo para cima — então teria de ADIVINHAR o código de sucesso, e também não enxerga o `@Roles`. Quem monta a operação é o construtor do documento, que percorre as rotas já registradas e tem o verbo, o `@HttpCode` e o `@Roles` à mão.
- **Os papéis vêm do `@Roles` real**, nunca de um campo repetido no `@Contrato`: repetir abriria espaço para o documento dizer `gestor` enquanto o *guard* exige `admin-contrato`.
- **O esqueleto de caminhos vem do roteador do Nest** (`SwaggerModule.createDocument`), que é a única fonte incapaz de inventar uma rota que não existe. As duas travessias — a do roteador e a do `DiscoveryService` — precisam concordar, e divergência de um caminho sequer é erro alto na subida.

Alternativas descartadas.

- *Anotar `@ApiBody`/`@ApiResponse` à mão nas 146 rotas*: criaria uma segunda descrição do contrato ao lado da primeira, sem nada que as obrigue a concordar. É literalmente a armadilha do `expires_at` que ficou no *schema* depois que a rotina saiu do código.
- *Migrar a validação para DTOs com `class-validator`*: reescreveria a validação das 146 rotas para ganhar inferência automática, e trocaria o Zod — que a convenção da casa usa em TODO input externo, *env* incluído — por um segundo validador só nas rotas.
- *`nestjs-zod`*: exige DTO de classe (`createZodDto`) e *pipe* global. É a mesma reescrita, com uma dependência a mais.
- *`@asteasolutions/zod-to-openapi`*: monta o documento a partir de um registro próprio, fora do roteador do Nest — ou seja, pode listar rota que não existe e omitir rota que existe, que é exatamente o defeito que se quer evitar.

Consequências.

- **`.refine()` some da conversão, em silêncio.** JSON Schema não expressa função JavaScript, e a biblioteca descarta. O `AjustarMatriculaSchema` exige "`turmaDestinoId` OU `principal`" e isso não aparece como esquema. Por isso o `@Contrato` tem o campo `notas`: restrição que o Zod checa e o JSON Schema não sabe dizer vira texto na descrição. A validação não muda — o que as notas consertam é o documento não mentir por omissão.
- **Esquemas saem embutidos, sem `$ref`** (`$refStrategy: "none"`). É seguro porque não há `z.lazy` em lugar nenhum da api. Custa repetição no JSON e ganha um documento que não depende de um registro de nomes estáveis.
- *Ruim*: a assinatura publicada do `zodToJsonSchema` estoura o compilador com **TS2589** ("type instantiation is excessively deep") — o tipo de retorno instancia `JsonSchema7Type`, uma união recursiva grande, mesmo quando `target: "openApi3"` resolve para `object`. O módulo fixa localmente a assinatura que usa. Some quando a biblioteca simplificar o retorno.
- **A entrega foi em etapas, com lista que só encolhe.** Documentar 146 rotas num commit seria um *diff* que ninguém revisa, e uma resposta mal descrita passaria batida. `tests/openapi-cobertura.spe` percorre o roteador e cobra `@Contrato` de toda rota, salvo as nomeadas numa lista explícita — o mesmo molde do `SEM_CHAVE_POR_DECISAO` de `identidade-unica.spec.ts`, com a diferença de que aquelas exceções são permanentes e estas eram dívida. **A lista está vazia**: as 146 rotas declaram contrato, e rota nova sem ele é teste vermelho.
- **O Apêndice C deixou de ser escrito à mão.** Ele é gerado do `docs/openapi.json`, que é gerado do `@Contrato`. Eram 547 linhas de prosa descrevendo 146 rotas sem que nada obrigasse essa prosa a concordar com o código — exatamente o defeito que esta ADR existe para fechar, e que teria sobrevivido dentro do próprio documento que a registra.

Gatilho de reavaliação. Subir para Zod 4, que traz `z.toJSONSchema` nativo: aí a dependência de conversão sai, e o TS2589 provavelmente com ela.

18.38 ADR-37 — O *schema* de resposta é conferido, não só publicado

> **Contexto**
>
> O *schema* de ENTRADA não tem como envelhecer: a api o usa para validar em toda requisição, então errar nele quebra a rota na hora. O de SAÍDA não tinha nenhum — os *services* devolvem entidade do Prisma direto, *spreads* e formas ad-hoc — e passa a existir só para o documento.
> Afirmação que nada exercita é a mesma coisa que a coluna de prazo sem rotina que a cumpra: quem lê acredita, e ninguém percebe quando deixa de ser verdade.

Decisão. Três camadas, em ordem de força.

1. **Teste**, com `.parse()` estrito, nas suítes que já exercitam rota de verdade. É a conferência que dói.

2. **Runtime fora de produção**: o `ContratoInterceptor` roda `safeParse` na resposta e avisa no *log*. Nunca lança.
3. **Produção**: o interceptor é *no-op*.

Alternativas descartadas.

- *Validar a resposta também em produção*: um *schema* estrito demais transformaria rota que funciona em 500. O documento não vale uma indisponibilidade.
- *Só publicar, sem conferir nada*: é o `expires_at` de novo, agora em forma de *schema*.

Consequências.

- **O interceptor serializa antes de conferir** (`JSON.parse(JSON.stringify(...))`). Ele roda ANTES da serialização, onde ainda há `Date` do Prisma; sem essa volta, todo campo de data acusaria divergência falsa contra um `z.string().datetime()` correto. É também parte do motivo de ele não rodar em produção.
- **O aviso carrega só caminho e código do problema, nunca o valor.** A mensagem do Zod embute o recebido em alguns casos (`invalid_enum_value`), e a resposta desta api carrega nome de criança e CPF — regra de não registrar PII em *log*, nem dentro de mensagem de erro.
- Funcionou na primeira rota documentada: o *schema* do `POST /auth/login` declarava `escolaId` como `uuid`, e `Escola.id` é `String @id` sem *default* — o id vem da rede. O aviso apareceu no primeiro *login* real.

Gatilho de reavaliação. A conferência aparecer no perfil de latência de desenvolvimento a ponto de atrapalhar — aí ela passa a valer só em teste.

18.39 ADR-38 — O *deploy* paraleliza os *jobs*, não o portão

> **Contexto**
>
> Medido nos *runs* de 2026-08-28, um *push* comum levava ~9,6 min até a escola e um `forcar`, ~26 min — e 80% disso eram as duas imagens Docker, por duas causas que se somavam: `COPY . .` antes do `pnpm install` invalidava a camada de dependências em todo *commit* (o *cache* GHA só acertava apt e corepack), e api e hub buildavam em série no mesmo *job* sem depender uma da outra.

Decisão. Duas mudanças, nenhuma nova peça de infra: (1) os *Dockerfiles* abrem com `COPY pnpm-lock.yaml pnpm-workspace.yaml` + `pnpm fetch` — que baixa o *store* lendo só o *lockfile* — e o `pnpm install --offline` roda depois do `COPY . .`; (2) o *workflow* vira quatro *jobs* — `mudancas` (o *diff*, calculado uma vez), `deploy-api` e `deploy-hub` em paralelo atrás do portão, e `publicar-exps` depois dos dois, porque o `PUT` do manifesto é validado contra o JWKS que o hub serve.

Alternativas descartadas.

- *Buildar as imagens em paralelo com o portão*, deixando só `gcloud run deploy` e a migração esperarem o verde: ganharia 2–6 min de caminho crítico, mas gastaria *runner* em todo *commit* vermelho, e o pipeline passaria a ter dois regimes de *gating* para raciocinar. O portão continua na frente de tudo.
- *Cache remoto do Turborepo* ligando `verificar`, o *build* do *runner* e os *builds* dentro das imagens: é a otimização certa quando o `tsc`/`next build` virarem o gargalo — hoje o gargalo era o `pnpm install` repetido, e um serviço de *cache* é uma peça a mais para operar.
- *Copiar o `dist/` do runner para dentro da imagem* em vez de buildar lá dentro: acopla o *Dockerfile* ao *workflow* (um `docker build` local deixaria de funcionar sozinho) para economizar um passo que, com a camada de dependências cacheada, custa segundos.

Consequências.

- O hub não espera mais o *deploy* da api. A ordem era acidental — os dois conversam por HTTP e o *rollout* do Cloud Run nunca garantiu revisão casada — mas fica registrado: se um dia um contrato exigir hub-depois-da-api, é um `needs` de uma linha.
- `publicar-exps` usa `!cancelled()` com `result != 'failure'` nos `needs`: os *deploys* podem ter sido PULADOS (um *push* que só toca experiência), e pulado não pode bloquear a publicação.
- Cada *job* que autentica no GCP repete o *setup* de ~1 min (WIF, gcloud, buildx). É o preço do paralelismo, pago em paralelo.
- O `pnpm fetch` depende só do *lockfile*: mudou dependência, a camada refaz inteira. É o comportamento certo — e é raro na frequência de *commit* deste repo.

Gatilho de reavaliação. O `nest build`/`next build` dentro das imagens passar de ~2 min com a camada de dependências já em *cache* — aí o cache remoto do Turborepo entra na conversa.

18.40 ADR-39 — Vulnerabilidade de dependência trava o portão, com prazo para os *majors*

> **Contexto**
>
> A plataforma guarda dado de menor e roda em rede pública; dependência com vulnerabilidade conhecida é passivo que cresce em silêncio. Na primeira varredura (Snyk, 2026-08-29) havia *criticals* reais: o Next 15 do hub e o `next@16.3.0` que o *design system* arrasta para dentro de todo consumidor, mais *highs* transitivas na api (`multer`, `js-yaml`, `lodash`) e três cujo *fix* exige *major* de *framework* (Nest 11) ou *bump* de terceiro (Prisma).

Decisão. O portão (`ci.yml`) ganha `snyk test --all-projects --severity-threshold=high`: `high` ou `critical` = CI vermelho = nada sobe. O estoque existente foi tratado em três movimentos — o hub subiu para o **Next 16** (a linha 15.x não tinha *patch* para as *criticals*), o tran-

sitivo consertável entrou em `pnpm.overrides` na raiz, e o que exige Nest 11/Prisma está em `apps/api/.snyk` com **motivo e prazo** (2026-10-31): vencido o prazo sem o *upgrade*, o próprio passo volta a falhar. `SNYK_TOKEN` ausente = passo vermelho, nunca pulado (a regra do *guard* do Pub/Sub).

Alternativas descartadas.

- *Só relatório, sem travar* (`continue-on-error`): vira o `expires_at` — todo mundo acredita que há uma verificação, e ninguém lê o relatório de um passo que nunca fica vermelho.
- *Bloquear até consertar TUDO, sem* `.snyk`: com *fix* que é *major* de *framework*, o pipeline ficaria vermelho por semanas e o *deploy* pararia — e um portão que todo mundo precisa contornar ensina a contornar portões.
- *Rodar só no push para* `main`: o PR mergearia vulnerável e o autor descobriria depois do merge, no pior momento para reverter.

Consequências.

- Cada execução do portão testa os 12 projetos do *workspace* contra a API do Snyk — em plano gratuito há cota mensal de testes. Se a cota virar o problema, a válvula é mover a trava para o *push* em `main` + um agendado diário, nunca desligar.
- O `.snyk` expira em 2026-10-31: o *upgrade* do Nest 10→11 tem prazo de verdade, cumprido pelo próprio CI.
- `pnpm.overrides` força versão por baixo de quem não atualizou (`@unooria/design-system>next`, `multer`, `js-yaml`, `lodash`); cada entrada deve SUMIR quando o pai atualizar — *override* esquecido é *pin* que ninguém lembra por quê.
- O *fix* definitivo do `next` transitivo é do repo `storybook-unooria`: o DS não deveria arrastar `next` para dentro de experiência Vite que nunca o executa.

Gatilho de reavaliação. A cota do Snyk barrar o portão, ou o `.snyk` vencer: os dois têm a mesma resposta — resolver a causa (plano ou *upgrade*), não afrouxar a trava.

18.41 ADR-40 — A raiz é a *landing*; a marca vem do DS, não do modelo

> **Contexto**
>
> A raiz do hub (`/`) só roteava: sem sessão, `/login`; com sessão, a *home* do papel. Quem chegava ao endereço da plataforma via um formulário de usuário e senha e nada sobre o produto — não havia *landing*. A tela foi desenhada no Stitch (projeto `16689192725057015920`, "Unooria — Educação Digital Infinita"), e o pedido era implementá-la fiel ao modelo.
> O HTML exportado pelo Stitch traz duas coisas que não são da marca: o tema Material que a ferramenta gera sozinha (primária `#396093`, Nunito Sans, Inter, Material Symbols *Outlined*, Tailwind por CDN) e o logo como PNG hospedado no Google. E o `login-form`

> e a troca de senha fazem `router.replace("/")` contando com o roteamento da raiz — é o que leva a pessoa à área certa sem o *client* conhecer o papel.

Decisão. Três partes.

1. **`/` sem sessão é a *landing*; com sessão, continua roteando** para a *home* do papel (ou para `/trocar-senha`). Nada muda no `login-form`, na troca de senha nem no *middleware* (a raiz segue fora do `matcher`). Quem já está dentro e quer ver a *landing* sai da conta.
2. **Fiel ao modelo em estrutura, copy, seções e movimento** — navegação fixa em vidro, *hero* sobre o *shader* WebGL do modelo (portado do `<script>` inline para um *client component* com ciclo de vida), cards flutuantes, revelação ao rolar, rodapé — **e do DS em cor, tipografia, ícones, logo e botões**: tokens `--unooria-*` (a primária do modelo vira `brandnavy-900`; os roxos, `brandpurple-600/700`; o vermelho, `danger-600`), Open Sans, `Icon` (Material Symbols *Rounded*, os mesmos nomes de ícone), `Logo`/`LogoSymbol` vetoriais no lugar dos PNG e `Button` nas duas chamadas. O painel "conceito" de Nossa Resposta, que no modelo era um PNG do símbolo sob um gradiente, é o `--unooria-color-gradient-1` com o `LogoSymbol` — mesma imagem, sem *raster*.
3. **O CSS da *landing* vive ao lado do componente** (`components/landing/landing.css`), não em `plataforma.css`: é composição de página, não compensação de lacuna do DS. Toda cor ali é *token*; translucidez sai de `color-mix()` sobre o *token*, não de literal `rgba`.

Alternativas descartadas.

- *Landing sempre visível, com uma rota nova (`/inicio`) para o roteamento por sessão*: exigiria trocar o destino do `login-form` e da troca de senha e criar uma segunda porta que só redireciona. Mais peças para o mesmo resultado — e quem está logado raramente quer a página de venda.
- *Landing fora do hub, como HTML estático no bucket*: a página teria de duplicar o CSS e as fontes do DS, e o botão "Começar Agora" atravessaria origens. O hub já serve a casca do DS (`CascaPlataforma`) — a *landing* é a quinta superfície a usá-la.
- *Reproduzir a paleta e as fontes do Stitch*: seria a primeira superfície da plataforma fora da marca, e por definição a mais vista. A regra do DS (Capítulo 14) existe exatamente para isso.
- *Trazer o Tailwind por CDN e o tema do modelo*: o hub já tem Tailwind só para utilitárias de *layout*, com o tema VAZIO de propósito — cor por nome de classe é o que a decisão de `plataforma.css` proíbe.

Consequências.

- **O botão primário sai azul-céu, não navy** (`#0d0d35` no modelo): é o `Button` do DS. Se a marca quiser o CTA em navy, o lugar é o pacote, não uma classe local.
- **O cleanup do *shader* NÃO chama `loseContext()`.** O Strict Mode do React (dev) monta, desmonta e remonta o efeito no MESMO canvas, e um canvas só tem um contexto WebGL — perdido de propósito na primeira desmontagem, volta perdido na segunda montagem e

o fundo fica em branco. Foi o primeiro sintoma do *smoke* (`contextLost: true`). Com `prefers-reduced-motion` desenha um quadro e para; na aba oculta, para até voltar; sem WebGL, o gradiente do DS declarado no CSS ocupa o lugar.

- **A revelação ao rolar só esconde depois que o JS montou** (`.landing--animada`). Esconder no CSS e revelar no JS deixaria a página em branco para quem o *bundle* não alcança.
- **Cor de ícone vai pela *prop* `color` do `Icon`**, não por classe: a folha do pacote vem depois no grafo de importação e, com peso igual, vence — os ícones saíam pretos.
- Os *links* de Privacidade, Termos e Contato do rodapé apontam para `#`, como no modelo: as páginas não existem. Pendência de produto, não de código.
- Suíte própria (`scripts/e2e/publico/01-landing.mjs`): fontes do DS carregadas (Open Sans e a de LIGADURA dos ícones), canvas com contexto, cor de fundo COMPUTADA do card de vidro, todo `.landing-revelar` visível ao fim, nada transbordando em celular, e o CTA levando ao `/login`. Só precisa do hub no ar.

Gatilho de reavaliação. A plataforma ganhar uma área pública maior (institucional, ajuda, termos): aí a *landing* vira um grupo de rotas `(publico)` com *layout* próprio, e os *links* do rodapé deixam de ser `#`.

18.42 ADR-41 — O relatório do estudante sai do aparelho: o iframe expande, não remonta

> **Contexto**
>
> A explicabilidade da exp-01 é a peça mais densa do produto e rodava inteira dentro do aparelho de 375×812: seis blocos, um gráfico só e três dos cinco posts de maior *dwell* — quando o *payload* já carregava mais. As barras por bloco e a linha da escalada existiam em `graficos.tsx` e não tinham consumidor de tela; a "escalada" virara uma frase por falta de espaço.
> Ela acontece DEPOIS da revelação, e nesse ponto não há mais disfarce a proteger: a regra da ponte (`packages/ui/src/ds.ts`) já autorizava o DS da plataforma nas "telas pós-revelação do estudante".

Decisão. No passo do relatório, a experiência publica `exp:estado { fase: "explicabilidade", passo: "relatorio" }` e o `ExpHost` liga `expandido` no `DispositivoFrame`, que troca APENAS o CSS do próprio `<div>` interno. O relatório vira claro, responsivo, com os três gráficos e o *top* 5. A revelação continua no aparelho, escura: ela é a fronteira, e não estava comprimida.

Alternativas descartadas.

- *Um `Modal size="full"` do hub buscando `/runs/:id/dossie`*: o BFF já permitiria (a rota autoriza o estudante a ler o próprio) e havia precedente em `modal-estudante.tsx`. Descartado porque duplicaria a renderização do relatório em dois apps — ou obrigaria a experiência a perder os *exports* em texto e PNG, que são dela.

- *Uma rota própria do hub* (`/estudante/relatorio/:runId`): tira o estudante do aparelho no meio de uma aula conduzida por botão.
- *`requestFullscreen()` de dentro do iframe*: o `allow="…fullscreen"` já está delegado a todas as experiências e ninguém usa. O hub perderia o controle da moldura, e a largura lógica continuaria a mesma — ficaria maior, não menos comprimido.

Consequências.

- **Expandir é ESTILO, nunca árvore.** Tirar o `children` do `DispositivoFrame` para renderizá-lo fora desmontaria o `<iframe>` e o estudante voltaria à revelação com a sessão recarregada. É por isso que o modo é uma *prop*, e não um `if` trocando o *wrapper*.
- **A condição é POSITIVA, e o *default* é contraído.** Qualquer outro estado — inclusive mensagem perdida — cai no aparelho de 375px. O `IntersectionObserver` do *dwell* se refere ao *iframe*: um FEED numa área larga inflaria o sinal central do motor.
- **O layout largo é `md:`, o estreito é o *default*.** Se a expansão não acontecer, o estudante vê exatamente o que via.
- **Nenhuma mensagem hub→exp para o layout**: o *iframe* muda de largura de verdade, e as *media queries* de dentro respondem sozinhas. Os dois gráficos a mais são renderizados e escondidos por CSS — perguntar a largura em JS custaria um `matchMedia` e um render a mais na hidratação.
- A volta usa o caminho que já existia: o professor avança, o *polling* de 5s vê a fase nova, a experiência publica `{ fase: "perdidos" }` sem `passo`, e o aparelho volta. Até 5s de atraso, que é a latência que a troca de tela já tinha.
- **Paridade**: o relatório do professor (`modal-estudante.tsx`) ganhou a seção *"por que o feed dele começou desse lado"*, que ele não lia — e é a frase que a criança acabou de ler.

Gatilho de reavaliação. Uma segunda experiência precisar do mesmo gesto: aí `expandido` deixa de ser lido de dois campos conhecidos no `ExpHost` e vira mensagem própria no `ExpParaHubSchema`, com *bump* de `protocolVersion`.

18.43 ADR-42 — "O que o algoritmo escondeu" é a EXPOSIÇÃO, não o domínio

> **Contexto**
>
> Havia duas contas para a mesma frase. A tela do estudante (`/runs/:id/perdidos`) decidia qual era "o lado dele" CONTANDO os posts servidos (`consumo.A >= consumo.B`); o relatório do professor (`dossie/computar.ts`) decidia pelo `dominioDe()`, o *score* do motor — *dwell*, sinais, escada de intensidade.
> Enquanto as duas apontam o mesmo lado, os números batem. Elas divergem exatamente quando o estudante VIRA O JOGO, porque virar o jogo *é* a definição de a atenção ter ter-

> minado do lado oposto ao que o feed empurrou. Ou seja: as duas telas se contradiziam justamente sobre os estudantes que o professor mais quer discutir.
> O caso está registrado numa auditoria de UX: sobre o MESMO estudante, o painel dizia "terminou dominante em Processo Lento e Gradual, virou o jogo" e a tela dele dizia "32 posts que nunca chegaram até você, todos de Processo Lento e Gradual". Duas frases sobre a mesma criança, nomeando lados opostos. O relatório do professor ("escondeu 30") contava os posts do lado que o algoritmo estava EMPURRANDO nele.

Decisão. Uma conta só, em `apps/api/src/dossie/exposicao.ts`, consumida pelas duas rotas: "o lado dele" é o que o feed mais SERVIU, e o número é quantos aprovados do outro lado nunca foram servidos. `ladoDominante` continua existindo e continua no relatório — ele responde "para onde a atenção dele foi", que é outra pergunta, e é a que sustenta o "virou o jogo".

Alternativa descartada. *Padronizar no domínio.* É a leitura mais fiel ao que a atenção do estudante fez, mas "o algoritmo escondeu" é uma afirmação sobre o que o feed ENTREGOU: medir pelo domínio credita à máquina o resultado do comportamento da criança. E com `dominio` $\leq$ `0,5` o resultado era `indefinido`, que virava "escondeu 0 posts".

Consequências.

- **O número do professor MUDA** em atividades já rodadas — `VERSAO_DO_RELATORIO` sobe para 4 e os dossiês congelados são recomputados na leitura. É o preço de ter contado errado.
- **Sem post de tese servido, não se nomeia lado nenhum.** A rota devolve `ladoPerdido: null` e a tela diz que a sessão terminou antes de o feed abrir. "0 posts que nunca chegaram até você" afirmaria o contrário do que houve — é a mesma distinção entre `zero` e `none` da Vila do Lume.
- **Post desaprovado não conta como escondido**: nunca teve chance de aparecer para ninguém, e culpar o algoritmo por uma decisão editorial do professor seria outra mentira pequena.
- A trava é `apps/api/tests/exposicao-lado-oposto.spec.ts`, e o teste do meio monta exatamente o estudante que virou o jogo.

Gatilho de reavaliação. Alguém precisar de "o oposto do lado dominante" para outra tela: é uma terceira grandeza, e ela nasce com nome próprio — nunca reaproveitando esta.

18.44 ADR-43 — A resposta do estudante guarda a primeira e a última

> **Contexto**
>
> A tela da resposta não tem botão de confirmar: tocar na pílula já registra, e o quadro diz "pode trocar de ideia enquanto espera — vale a última". O estudante oscila de propósito enquanto os colegas terminam.

> O `upsert` sobrescrevia `inclinacao` e a primeira escolha desaparecia. O `created_at` ficava congelado no primeiro envio enquanto a inclinação mudava, ou seja, passava a mentir sobre a própria linha. E a correlação "lado em que o feed nasceu × resposta dada" — o dado mais forte que esta atividade produz — não tinha como dizer se estava lendo a reação imediata ao feed ou o que sobrou depois de dois minutos de espera e de olhar a tela do colega.

Decisão. Migração aditiva em `exp_01_conclusoes`: `inclinacao_inicial` (gravada uma vez, nunca mais tocada), `trocas` e `atualizado_em`. A correlação continua usando a ÚLTIMA — é a que o estudante assumiu e é a que ele vai defender no debate — e a análise da turma passa a DIZER isso, com quantos mexeram na resposta e quantos terminaram noutro lado.

Alternativas descartadas.

- *Emitir evento de telemetria a cada toque*: o canal analítico tem retenção indefinida e `estudante_id`; guardar a oscilação de opinião de uma criança num JSONB de eventos é mais dado do que a pergunta pede.
- *Trocar a correlação para a primeira escolha*: seria decidir pelo professor qual das duas conta. A decisão é dele, e agora ele tem os dois números.

Consequências. `inclinacao_inicial` é NULL nas linhas anteriores à migração, e é assim que se lê "esta resposta é anterior à mudança": preenchê-la com a inclinação atual inventaria uma primeira escolha que ninguém observou. Reenviar a mesma opção (toque duplo, rede lenta) não conta como troca. As duas colunas novas vão ao CSV da análise.

Gatilho de reavaliação. Aparecer demanda por ver a oscilação completa (não só a primeira e a última): aí é tabela de histórico, não mais colunas.

18.45 ADR-44 — A janela do envolvimento termina com o feed, e o total sai das listagens

> **Contexto**
>
> Duas coisas, medidas na mesma auditoria.
> A primeira: o painel ao vivo mostrava a imersão da turma DERRETENDO conforme a aula avançava ($52{,}7 \rightarrow 42{,}8 \rightarrow 38{,}7 \rightarrow 35{,}1$) e isso lia como adolescente desengajando. Não era. O *buffer* de eventos da experiência vive enquanto a *run* vive: durante o debate, a explicação e a reflexão — mais de metade do roteiro de 100 minutos — cada `hub:pause` ainda registra `perdeu_foco`. Com a conta fechando no último evento da SESSÃO, `feedMs` crescia depois de o feed fechar, cada vão virava `ociosoMs`, e `ativoMs / feedMs` caía sozinha.

> A segunda: o total 0–100 aparecia como número numa COLUNA de tabela, por criança, no monitor e na análise — ordenável, comparável. O próprio código dizia, em comentário, que "o total existe para ordenar uma lista, não para ser exibido sozinho", e a tela o exibia sozinho. E ele é somável justamente onde menos se sustenta: *Gestos* e *Exploração* ficam em 0/20 na maioria das sessões, então o teto real não é 100.

Decisão. `fimDoFeedDe(eventos)` — o último `visto`, que é o fim do feed por definição — corta a janela em `calcularTempo` E no desconto por perda de foco de `calcularImersao`. E o total sai das duas listagens por estudante, ficando as cinco barras (com o eixo no `title`); o número segue no popup do estudante, que é uma criança por vez, e nos agregados de turma. Um nome só: *Envolvimento*, que estava "Imersão" em quatro telas e "Envolvimento" numa.

Alternativas descartadas.

- *Cortar por `run.finishedAt`*: `clientTs` é o relógio do APARELHO e `finishedAt` é o do servidor. Comparar os dois numa escola com relógio desregulado traria o mesmo erro de volta por outro caminho.
- *Parar de registrar eventos fora do feed, no cliente*: corrigiria daqui para a frente e não tocaria em nenhum dado já coletado — e depende de a experiência estar atualizada.
- *Recalibrar os tetos de Gestos e Exploração*: exige medição em turmas reais. Fica em aberto; mexer nos tetos por intuição trocaria um número errado por outro.

Consequências. `VERSAO_DO_RELATORIO` sobe para 4 e recomputa o que está congelado. Um teste do `dossie-relatorio.spec.ts` precisou de um `visto` para delimitar o feed: ele montava trinta `perdeu_foco` e nenhum post visto, ou seja, descrevia perdas de foco fora de qualquer feed — uma distinção que antes não existia. O professor perde a leitura de um número por estudante na tabela; ganha o padrão dos cinco eixos, que é o que a conversa com ele usa.

Gatilho de reavaliação. Dados de turmas reais mostrarem que *Gestos* e *Exploração* pontuam: aí os tetos são recalibrados e a pergunta de exibir o total volta a fazer sentido.

18.46 ADR-45 — *Agent* e `fetch` vêm do MESMO pacote undici

> **Contexto**
>
> O provider do Vertex usa um *dispatcher* próprio: `generateContent` não-*streaming* só devolve os *headers* quando a geração termina, e o teto de 300 s do `fetch` do Node estourava no meio de um plano anual. A solução era um `Agent` do pacote `undici` com 15 minutos, passado como opção `dispatcher` ao `fetch` GLOBAL.
> Isso acopla duas coisas que ninguém obriga a concordar. O `fetch` global usa o undici EMBUTIDO na *runtime* (`process.versions.undici`), que é outro pacote, com outra versão. Com os `Dockerfile` fixados em **Node 26.7** (undici **8.x**) e a API declarando `"undici": "^6"`, o `Agent` da 6 deixou de satisfazer a interface de *handler* da 8.

> O sintoma não aponta para nada: `fetch failed`. A razão real (`invalid onError method`) vive só em `error.cause`, e o `catch` da geração logava apenas `error.message`. **TODA** chamada de IA morria — plano de aula, *unooGPT*, arte — e por dias isso pareceu falha de credencial, porque havia de fato um ADC vencido por cima.

Decisão. Importar o `fetch` do PRÓPRIO undici (`import { Agent, fetch } from "undici"`). Vindos do mesmo pacote, os dois concordam por construção, e a versão embutida na *runtime* deixa de ser uma dependência invisível. O *log* da geração passa a imprimir o `cause` junto — só no *log*: a mensagem que vai ao banco é a que o professor lê.

Alternativas descartadas.

- *Subir `undici` para `^8`*: funciona hoje e quebra no próximo *bump* de Node, porque a condição continua sendo "duas versões coincidirem" — e nada no CI verifica isso. Seria a mesma armadilha, adiada.
- *Largar o dispatcher e usar o `fetch` global puro*: devolveria o teto de 300 s que motivou o *dispatcher*, e o plano anual voltaria a morrer no meio.
- *Fixar o `undici` pela versão da runtime*: exigiria ler `process.versions.undici` e casar no `package.json`. Duas fontes de verdade sobre a mesma versão, uma delas fora do controle do repo.

Consequências. O teste `ai-conversar.spec.ts` dublava `global.fetch` e passou a não interceptar nada — voltava *array* vazio. Ele agora mocka o módulo `undici` encaminhando para `global.fetch`, o que mantém a dublagem em uma linha e o que se testa segue sendo a lógica do provider. Fica registrado que o `dispatcher` é a razão de o `fetch` não poder ser o global: quem apagar a importação "porque o Node já tem `fetch`" reabre exatamente este defeito.

Gatilho de reavaliação. O Vertex passar a ser consumido por `streamGenerateContent?alt=sse`: com *streaming* os *headers* chegam de imediato, o teto de 300 s deixa de incomodar, e o *dispatcher* — junto com todo este acoplamento — pode simplesmente sair.

18.47 ADR-46 — O `?hubOrigin=` é entrada não confiável: a experiência confirma quem a embedou

> **Contexto**
>
> A regra 4 diz que o *token* passa do hub para a experiência por `postMessage` com `targetOrigin` explícito, nunca `"*"`. A bridge cumpria a letra da regra e não o espírito: o valor do `targetOrigin` — e, o que é pior, o valor comparado em `ev.origin !== hubOrigin` para decidir de quem se aceita o `hub:init` — vinha CRU da *query string* (`new URLSearchParams(location.search).get("hubOrigin")`).

> O *bundle* da experiência é público por construção (ADR-01: ele mora num *bucket* servido por CDN). Quem abrisse `https://cdn/exp-01/1.0.0/index.html?hubOrigin=https://evil.tld` dentro de um *iframe* próprio escrevia, com aquele parâmetro, a política de confiança da experiência: passava a poder entregar um `hub:init` forjado — com `estudante`, `apiUrl` e `token` escolhidos por ele — e a receber **tudo** o que a experiência emitisse depois (`exp:concluir`, `exp:estado`). O Snyk classificou como *code injection* no ponto em que o parâmetro entra; a leitura é discutível, o caminho não.

Decisão. Todo `hubOrigin` passa por `normalizarHubOrigin()` (`packages/sdk/src/bridge.ts`), chamado tanto onde o parâmetro é lido quanto dentro do `connectToHub` — a função é idempotente, e a segunda chamada existe porque é ali que o valor vira `targetOrigin`. São três travas:

1. `new URL(...).origin`: o valor vira uma ORIGEM canônica, nunca uma *string* com caminho, credencial ou *query*. Não é higiene: `postMessage` compara só a origem, mas a igualdade do *handler* nunca casaria, e o sintoma seria *timeout* de *handshake* em vez de "o parâmetro está errado". Esquema fora de http(s) e origem opaca (`"null"`) são recusados.
2. Experiência servida por https exige hub em https — a mesma regra do JWKS da portabilidade. Em desenvolvimento tudo é http e a trava não pesa.
3. `location.ancestorOrigins[0]`, quando o navegador publica: é a origem REAL de quem embedou, e ela VENCE a *query string*. É a trava que fecha o caminho de fato — o atacante controla o parâmetro, não o endereço do documento que abriu o *iframe*.

Alternativas descartadas.

- *Lista de origens permitidas embutida no bundle*: a experiência é a mesma em todos os contratos, e cada contrato tem o seu domínio de hub. A lista viraria *env* de *build*, ou seja, um *bundle* por contrato — exatamente o que a ADR-01 evita.
- *Deduzir a origem de `document.referrer`*: depende da *referrer policy* do hub e some com `no-referrer`. O `ancestorOrigins` não é forjável pela página.
- *Deixar como estava, porque o token forjado é do próprio atacante*: verdadeiro para o roubo direto e irrelevante para o resto — o que o atacante ganha é uma experiência que fala com a API dele e devolve a ele o que a criança produziu.

Consequências. O Firefox não implementa `ancestorOrigins`; lá sobram as duas primeiras travas. `hubOrigin` malformado deixa de morrer por *timeout* de *handshake* e passa a lançar com a razão escrita — diagnóstico melhor, mas é uma falha NOVA para quem embedar a experiência fora do hub. `HUB_ORIGIN` mal configurado no hub, que antes só travava o *handshake*, agora recusa explicitamente.

NOTA

O apontamento do Snyk que originou esta ADR NÃO some com ela, e está certo assim. A regra *Code Injection* (CWE-94) desenha o fluxo `location.search` → `setInterval` e afirma que o valor "é executado como JavaScript". O *sink* apontado é o `setInterval(anunciar, READY_RETRY_MS)` do *retry* do `exp:ready`, e `setInterval` só AVALIA código quando o primeiro argumento é uma *string*: ali ele recebe uma referência de FUNÇÃO, e o `hubOrigin` entra por captura de *closure* para terminar como `targetOrigin` de um `postMessage`, que não executa nada. O valor tem três usos no pacote — dois `postMessage` e uma comparação com `ev.origin` —, e nenhum é *sink* de execução.

O que aquele fluxo tinha de verdadeiro era a origem de confiança vindo da URL, e é isso que esta ADR fecha. Fica registrado para que ninguém CONTORÇA o código para agradar o *scanner*: esconder a captura do *closure* deixaria o *retry* ilegível sem tirar risco nenhum. A nota vive junto da chamada, em `packages/sdk/src/index.ts`.

Gatilho de reavaliação. O dia em que uma experiência precisar ser embedada por um hub de terceiros (LTI, por exemplo): a trava 3 passa a exigir uma lista de embedadores conhecidos, e ela terá de vir do manifesto — não do parâmetro.

18.48 ADR-47 — Nem senha de demonstração nem verificação de TLS moram no fonte

Contexto

Três apontamentos do Snyk com a mesma raiz: credencial e política de segurança escritas como constante no código, onde não há como diferenciar desenvolvimento de produção.

- `apps/api/prisma/seed.ts` trazia `SENHA_PADRAO = "unooria123"`. O seed é idempotente e roda contra o `DATABASE_URL` que estiver no ambiente: apontado por engano para o banco de um contrato, ele criaria 61 contas — inclusive o `admin-contrato`, o único papel que atravessa escolas — com uma senha publicada no `README`.
- `wso2/bootstrap/bootstrap.mjs` abria com `process.env.NODE_TLS_REJECT_UNAUTHORIZED = "0"`, porque o IS local usa o certificado *self-signed* da distribuição (CN=`localhost`, host `identity-server`). Aquilo desliga a verificação do PROCESSO INTEIRO, para toda origem: o *script* que carrega a senha do *admin* do *tenant* passava a aceitar qualquer certificado de qualquer servidor.
- `apps/api/src/auth/auth.controller.ts` calculava o *hash* "dummy" anti-*timing* sobre uma *string* literal.

Decisão. O princípio é um só: o que é credencial vem do AMBIENTE, e o que é confiança se PROVA em vez de ser dispensada.

- `SEED_SENHA` é obrigatória e não tem *default* no fonte — ausente, o seed falha alto. O *default* de desenvolvimento vive nos dois `docker-compose`, exatamente como o `docs`/`unooria-docs-local` do Swagger (ADR-36). O seed também deixou de imprimir a senha.
- O *bootstrap* do WSO2 PINA o certificado: lê o certificado do IS no primeiro contato (o único `rejectUnauthorized: false` que resta, e ele não autentica nada — serve para DESCOBRIR a chave), grava o PEM que api e hub já consumiam por `NODE_EXTRA_CA_CERTS` e o usa como a ÚNICA CA aceita nas chamadas seguintes. Só a conferência de NOME do host é dispensada, com `checkServerIdentity`, porque o CN da distribuição não bate com o nome do serviço — a identidade da chave continua obrigatória.
- O *hash* dummy passa a ser calculado sobre `randomBytes(32)`, sorteado a cada subida e descartado no mesmo instante.

Alternativas descartadas.

- *Manter o literal do seed e ignorar o apontamento no `.snyk`*: o `.snyk` deste repo é para vulnerabilidade cujo *fix* exige *major* de *framework*, com prazo cobrado pelo CI. Aqui o *fix* custa dez linhas.
- *Sortear a senha do seed quando a env falta*: nada ficaria no fonte, mas as suítes de ponta a ponta passariam a depender de ler `stdout`, e o `README` não teria o que documentar.
- *Passar a CA do IS por `NODE_EXTRA_CA_CERTS` no bootstrap*: a variável é lida na SUBIDA do processo, e o certificado só existe depois que o *script* o busca. Exigiria dois processos.
- *Aceitar também o nome do host, reescrevendo o certificado do IS*: trocar a distribuição por um certificado gerado no *up* é mais infra local para operar, num serviço que já leva dois minutos para subir.

Consequências. `pnpm --filter @edu/api seed` no *host* exige `SEED_SENHA` — no `apps/api/.env` (que o Prisma carrega) ou na linha de comando. Nos dois *compose* nada muda. O *bootstrap* do WSO2 trocou o `fetch` global pelo `node:https`, porque o `fetch` do Node não aceita agente e é por agente que se passa uma CA; são vinte linhas a mais, e é o preço de não desligar o TLS do processo. O *bootstrap* passou a FALHAR se não conseguir ler o certificado, onde antes seguia com um aviso: sem CA não há como falar com o IS sem voltar a aceitar qualquer certificado.

Gatilho de reavaliação. O WSO2 deixar de ser *self-signed* em desenvolvimento (certificado de uma CA local no *up*): a pinagem e o `checkServerIdentity` saem juntos, e o `chamar` pode voltar ao `fetch`.

18.49 ADR-48 — O teto do CSV é de BYTES e cabe debaixo do limite do ambiente; o modelo do OneRoster vem em pares, e o exemplo leva a escola de verdade

> **Contexto**
>
> A tela de importação tem duas abas. A da *planilha da escola* oferece dois arquivos para *download* — o modelo em branco e um exemplo preenchido — e os dois são GERADOS da mesma constante que o *parser* consome. A do **OneRoster 1.2** lista os sete arquivos do *binding* CSV e não oferecia nenhum: quem não tem o *export* pronto do sistema de gestão — e é a maioria antes da primeira integração — tinha de montar o cabeçalho lendo a especificação da 1EdTech.
> O teto do corpo era 10 MB, dimensionado para a planilha de UMA escola, e o `MAX_LINHAS` de 50 mil, que existe do lado da planilha, não vale para o caminho OneRoster. Medido com o formato real do padrão (~130 B por linha de `enrollments.csv`), o pior caso realista de UM contrato — ~10 mil estudantes, cinco componentes cada — dá ~7 MB. Os 10 MB apertavam exatamente ali.
> Acima da aplicação há um teto que não é nosso: o Cloud Run recusa requisição maior que 32 MiB em HTTP/1, no *frontend* do Google e ANTES do contêiner. Não é quota que se peça para aumentar — é como a camada de *serving* funciona, porque ela bufferiza o corpo inteiro antes de repassar.

Decisão.

- O teto de BYTES sobe para 30 MB e passa a viver numa constante só, `TETO_DO_ARQUIVO` em `rostering.service.ts`, que o *bodyParser* de `main.ts` também lê. O número é escolhido para ficar DEBAIXO dos 32 MiB do ambiente: a aplicação não promete o que o Cloud Run não entrega, e a mensagem que o gestor vê é sempre a nossa — a que diz para dividir o arquivo. Ainda são ~4× o pior caso medido.
- A ORDEM entre os dois tetos importa: com o *parser* menor que a rota, o corpo morre no *middleware* e o gestor recebe um `PayloadTooLarge` do Express, sem o texto que diz o que fazer. O `mb` do *parser* é MEBIbyte (`30mb` = 31,4 MB), então ele fica ~5% ACIMA da rota — e é essa folga que faz a nossa mensagem ser a que aparece.
- O `MAX_LINHAS` de 50 mil NÃO sobe: ele é da planilha de uma escola, e 50 mil pessoas numa unidade é um arquivo errado, não um arquivo grande.
- Cada arquivo do padrão ganha o mesmo par da planilha: `GET /rostering/modelos/oneroster/:arquivo` (só o cabeçalho) e `.../:arquivo/exemplo` (preenchido). As colunas do modelo são um SUBCONJUNTO declarado de `COLUNAS_POR_ARQUIVO`, a constante que o leitor confere, e `tests/oneroster-modelos.spec.ts` cobra as DUAS direções — obrigatória que falte no modelo, e coluna do modelo que o *parser* não conheça (o dado entraria e sumiria).
- O exemplo é gerado com a escola de VERDADE no `sourcedId` do `orgs.csv` e no `schoolSourcedId` das turmas e matrículas, e com o ano corrente no calendário.

- Os sete exemplos formam um lote COERENTE — uma escola, um ano letivo, duas turmas, quatro pessoas e o enturmamento delas. O teste aplica os sete contra o banco, na ordem da tela, e afirma turma, papéis e matrículas.

Alternativas descartadas.

- *Escrever os sete arquivos à mão, como texto fixo*: é a armadilha que o modelo da planilha já evitou uma vez. Modelo escrito à mão diverge do *parser* no primeiro campo novo, e o sintoma chega como "baixei o modelo de vocês e deu erro de cabeçalho".
- *Publicar o modelo com TODAS as colunas que a api sabe ler*: `users.csv` tem 24. Modelo com 24 colunas não é modelo, é a especificação — e a escola preencheria `sms`, `phone` e `agentSourcedIds` sem que nada os leia.
- *Um `esc.exemplo` fixo no `orgs.csv`*: para o gestor, o lote inteiro voltaria como "escola não é a desta importação", e o arquivo que existe para conferir o caminho provaria o contrário. Para o `admin-contrato`, que PODE criar escola por CSV (ADR-34), o exemplo abriria um *tenant* de mentira — com RLS e dado de menor atrás dele.
- *Um único arquivo por item, sem o modelo em branco*: é o exemplo que o gestor abre para entender, e o modelo que ele preenche. Linha de exemplo dentro do modelo vira pessoa fantasma importada no primeiro uso — a mesma razão pela qual `planilha.ts` mantém os dois separados.
- *Empacotar os sete num zip*: empacotamento é embalagem, não semântica. A rota de importação já é um arquivo por chamada, e um *zip* para baixar exigiria dependência nova para produzir algo que o gestor descompactaria antes de usar.
- 100 MB — foi o primeiro número escolhido, e ele é PIOR que 30 MB: entre 33,5 MB e 100 MB a plataforma prometeria algo que o ambiente recusa, e o gestor receberia um 413 que não é nosso, sem mensagem útil e sem linha nenhuma no log da api. Teto que promete mais do que o ambiente entrega não é folga, é o erro que ninguém diagnostica.
- *HTTP/2 ponta a ponta (h2c) para tirar o limite do Cloud Run*: é a saída documentada, porque em HTTP/2 a requisição é *streamada* em vez de bufferizada. Custa trocar o servidor HTTP do Nest e fazer o hub repassar em HTTP/2 — um modo de transporte novo para operar, por um problema que os números acima dizem que não existe.
- *Streaming do corpo em vez de teto maior*: resolveria a memória, mas a leitura inteira do CSV é o que permite recusar o lote com o número da linha e o nome do campo. Trocar isso por um *parser* incremental é reescrever as duas fronteiras para um caso que ainda não apareceu.

Consequências. O corpo do CSV é lido inteiro para a memória em três lugares — o *route handler* do BFF do hub, o *bodyParser* da api e o `parseCsv` —, então 30 MB de arquivo custam alguns múltiplos disso em pico de *heap* por importação simultânea. O *rate limit* de 5 por minuto por usuário é o que segura isso hoje, e ele ficou ~3× mais barato do que ficaria com 100 MB.

A rede que precisar de mais que 30 MB num arquivo só NÃO tem saída pela tela: ela divide o `enrollments.csv` por escola ou por diretoria, que é uma operação que o formato já suporta

(um arquivo por chamada). Os sintomas dos três tetos — o nosso, o do *parser* e o do ambiente — estão no Apêndice F.

Gatilho de reavaliação. Uma rede chegar com *export* que não caiba em 30 MB depois de dividido: aí a conversa é sobre o CAMINHO — *upload* direto para o *bucket* com URL assinada e importação assíncrona a partir dele —, não sobre subir o número. Aquele caminho tira o limite do ambiente e o pico de *heap* de uma vez.

18.50 ADR-49 — A atividade do Instanooria declara a TURMA, e o painel dela para de ler a escola inteira

> **Contexto**
>
> A `exp_01_activities` tinha `escola_id` e `professor_id`, e nenhum vínculo com `turmas` — embora o próprio esquema já falasse como se tivesse: o comentário de `etapa` chama o campo de "relógio COLETIVO da turma", e a função que monta o painel de perfis se chama `distribuicaoDaTurma`.
>
> Ao investigar por que a importação dos perfis da Vila do Lume (`POST /atividades/:id/perfis/da-vila`) varria a escola, apareceu uma coisa pior: aquela função rodava `user.findMany({ where: { role: 'estudante' } })` — **sem `escola_id`**. E `users` está FORA da RLS: a lista de tabelas com política em `20260824145301_init` não a inclui, e o `comTenancy` apenas arma `app.escola_id` para o Postgres, sem tocar no `where` do Prisma. O painel do professor listava nome e login de crianças de TODAS as escolas do contrato, e `contagem`, `desequilibrio` e `precisaRedistribuir` — que decidem quantos estudantes o botão "Redistribuir a turma" move — eram calculados sobre esse universo.
>
> O `DaVilaSchema` já aceitava `turmaId` opcional desde que a rota nasceu, e a tela nunca o enviou: mandava `{}`. O relatório dizia "N ainda não jogaram" sobre crianças de salas que o professor não tem.

Decisão. Duas coisas, e a primeira vale sozinha.

1. **`escola_id` explícito** em `distribuicaoDaTurma`. É a regra inviolável nº 1 — toda *query* filtra por escola ALÉM da RLS — e aqui a RLS nem existia como rede.
2. **`exp_01_activities.turma_id`**, ANULÁVEL e sem *backfill* (`20260830090000_atividade_por_turma`). Com turma, o painel, a listagem de perfis, a importação da Vila e a entrada do estudante em `POST /runs` falam todos da MESMA sala. Sem turma — que é o caso de toda atividade anterior à migração — vale a escola inteira, o comportamento de antes. A turma é conferida contra a MATRÍCULA do professor (`professorLeciona`), nunca contra a escola do *token*.

Alternativas descartadas.

- *Só um seletor de turma na tela, sem coluna*: o professor escolheria a sala a cada visita e nada registraria para qual turma a atividade foi feita. Pior: `POST /runs` não teria como saber, e o estudante do 8B continuaria entrando na atividade do 8A.
- *`turma_id` NOT NULL, com backfill*: escolher uma turma para as atividades que já existem afirmaria uma decisão que nenhum professor tomou. É a mesma regra do `inclinacao_inicial` em `20260829120000_conclusao_primeira_e_ultima`: NULL quer dizer "anterior à mudança", e isso é informação.
- *Confiar só na RLS para o vazamento*: `users` não tem política, e não deve ter — ela é lida no *login*, antes de se saber a escola. A tranca correta é o `where`.
- *Uma atividade por turma, obrigatória*: transformaria a criação de atividade num laço para o professor que dá a mesma aula em quatro salas, sem que nada no produto peça isso hoje.

Consequências. Atividades antigas seguem escolares, e o painel agora DIZ isso em uma faixa — antes ele mostrava a lista sem explicar de quem era. O `POST /runs` ganhou uma leitura a mais (`users.turma_id`, o cache da matrícula principal) antes de escolher a atividade; a ordenação põe a atividade DA SALA à frente da geral quando as duas estão ativas. O professor que criar atividade sem escolher turma continua no comportamento antigo, sem aviso de erro: é opção, não omissão.

Gatilho de reavaliação. Uma rede rodar o Instanooria em várias turmas simultâneas com a MESMA pergunta: aí a pergunta é se a atividade deve ter várias turmas (tabela de junção) em vez de uma, e a resposta muda o painel inteiro.

18.51 ADR-50 — O perfil importado guarda a PARTIDA e a VERSÃO do mapeamento, e não ganha chave estrangeira

> **Contexto**
>
> `vila_sessoes` não tem prazo de retenção por uma decisão registrada em `20260826170000_vila_sem_prazo`: o bruto do jogo é a ÚNICA fonte para recalcular o perfil quando `VERSAO_MAPEAMENTO` subir, e o mapeamento é hipótese de design que vai mudar.
> Só que a cópia que alimenta o Instanooria (`exp_01_exp_70_perfis_importados`) guardava `escores`, `dominante`, `origem`, `lado_forcado` e `medido_em` — e nem a sessão de onde veio, nem a versão sob a qual foi medida. Ao subir a constante, os `exp_70_perfis` seriam recalculados e as cópias ficariam na geração anterior, decidindo o lado do feed, sem nada em lugar nenhum dizendo isso. É exatamente a armadilha do `expires_at`: o registro fica, a regra sai, e quem lê acredita.

Decisão. Duas colunas anuláveis, sem *backfill* (`20260830100000_perfil_importado_sabe_a_partida`): `vila_sessao_id` e `versao_mapeamento`. Gravadas no `create` E no `update` do *upsert* — se só o `create` gravasse, reimportar não tiraria a linha do estado "desatualizada" e o aviso da tela

nunca sumiria. O painel do professor passa a dizer quantos perfis vieram de uma versão anterior, como AVISO: perfil de geração antiga ainda é melhor informação que o sorteio.

SEM chave estrangeira para `vila_sessoes`, e isso é o centro da decisão. Zerar a turma na Vila apaga sessões e bolhas; com FK, isso levaria em cascata — ou travaria com `RESTRICT` — o perfil que está decidindo o lado de uma aula em andamento. Hoje zerar a Vila NÃO toca `perfis_importados`, de propósito, e a coluna não pode mudar isso: ela é RASTRO da medida, não vínculo de integridade. Sessão apagada vira um id que não resolve, que é exatamente a verdade do que aconteceu.

Alternativas descartadas.

- *Um valor novo em `origem`* (por exemplo `jogo-v3`): `perfis_importados_origem_check` e `runs_perfil_origem_check` congelam o vocabulário em dois lugares, e o contrato Zod num terceiro. `vila_sessao_id IS NULL` já responde "não veio da Vila desta instalação" sem mexer em nenhum deles.
- *Preencher a versão das linhas existentes com a atual*: afirmaria uma medida que ninguém fez. NULL é a leitura honesta de "anterior a esta migração, ou preenchido à mão pelo professor".
- *Reimportar sozinho quando a versão sobe*: desfaria em silêncio a redistribuição que o professor fez antes da aula. O disparo continua sendo dele (ADR do Ato 1), e o que muda é que agora a tela AVISA que vale reimportar.
- *Uma rotina de reprocessamento agora*: ela não existe nem para a própria Vila. Escrever a coluna sem o consumidor seria a armadilha de novo, ao contrário — por isso a coluna nasce COM o aviso na tela que a lê.

Consequências. A capacidade de reprocessar continua sendo capacidade, e não rotina: nada recalcula sozinho. O que existe agora é a informação para decidir — o painel sabe dizer que a medida envelheceu. Um perfil cuja sessão foi apagada mantém o id órfão, e isso é aceito.

Gatilho de reavaliação. A primeira subida real de `VERSAO_MAPEAMENTO` com dado de contrato em produção: aí se descobre se o aviso basta ou se a reimportação precisa virar rotina assistida.

18.52 ADR-51 — A suíte de ponta a ponta é organizada por PAPEL, e a descrição de cada tela mora no teste que a fotografa

> **Contexto**
>
> A validação visual eram doze arquivos soltos em `scripts/`, nomeados pela EXPERIÊNCIA que cobriam (`walkthrough-estudante`, `walkthrough-rede`, `walkthrough-ato1-vila`). Três problemas mediram-se sozinhos.

> O primeiro é de cobertura: telas inteiras não tinham dono. Criar uma atividade do zero, os períodos letivos, a análise de produto, o binding do OneRoster, o compositor de post do Instanooria — nada disso era exercitado, porque nenhum arquivo se chamava por elas.
> O segundo é de divergência. Cada roteiro carregava sua própria cópia de `registrar`, de `entrar`, do vigia de console e do relatório final, e as doze cópias divergiram: uma tirava `fullPage` e outra não, três reprovavam por erro de console e nove não. Divergência em código de teste é pior que em código de produção — quem lê o resultado não sabe qual regra valeu.
> O terceiro é o pior. O deck de evidências nascia de um segundo arquivo (`testes/.gerador-ppt/roteiro.mjs`, 66 KB) mantido À MÃO, com a descrição de cada slide escrita longe do teste que tirava o *print*. Ele divergiu do que os scripts faziam: descrevia telas que já não eram fotografadas e omitia passos novos. É a mesma armadilha do `expires_at` e do `turmas.consentimento_em` — o registro fica, a regra sai, e quem lê acredita nele.

Decisão. A suíte passa a viver em `scripts/e2e/<papel>/`, com uma casca compartilhada em `scripts/e2e/lib/roteiro.mjs` e um *runner* único (`scripts/e2e/rodar.mjs`).

Por PAPEL, não por experiência. Quem usa o produto é uma pessoa com um crachá, e é o crachá que decide o que ela alcança. Organizar por papel faz a LACUNA aparecer: "o gestor não tem suíte de períodos letivos" é uma frase que alguém diz em voz alta; "não existe `walkthrough-periodos`" não é. A contrapartida está registrada abaixo.

O manifesto é a fonte. Cada `registrar()` grava, junto com o *print*, o TÍTULO da tela, o que ela MOSTRA e a regra que ela PROVA, num `passos.json` por suíte. O deck (`gerar-deck.mjs`) e os vídeos (`gerar-videos.mjs`) só desenham — não conhecem o produto. A descrição passou a morar onde o passo mora, e sair de sincronia deixou de ser possível.

Alternativas descartadas.

- *Manter a organização por experiência*: é a que casa com o `apps/experiencias/`, e para a exp-70 e a exp-03 seria mais direta. Mas ela esconde as telas da PLATAFORMA, que são a maioria e não pertencem a experiência nenhuma — e foram justamente elas que ficaram sem cobertura por dois anos de projeto.
- *Um processo só para todas as suítes*, mais rápido por reaproveitar o navegador: uma suíte que trava levaria as outras vinte junto, e o código de saída deixaria de ser por suíte. O *runner* paga o custo do `spawn` de propósito.
- *Playwright Test* (`@playwright/test`) no lugar da casca própria: traz *runner*, *fixtures*, *retries* e relatório HTML de graça. O que ele NÃO traz é o manifesto descritivo, que é o objetivo aqui — a saída dele é um relatório de teste, não um documento de evidência com uma frase de produto por tela. Adotá-lo significaria manter o roteiro do deck à mão de novo, que é o problema que esta ADR resolve.
- *Manter o roteiro do deck à mão, só arrumando o que divergiu*: arruma a divergência de hoje e garante a de amanhã.

- *Deck em PDF ou HTML*: o entregável pedido é `.pptx` — é o que a rede abre, edita e projeta.

Consequências. As suítes de dois contextos (o professor conduz o relógio, o estudante joga) ficam no papel do PROTAGONISTA, e não em ambos: a missão da exp-01 mora em `estudante/`, ainda que metade dos *prints* sejam da tela do docente. O passo carrega de qual contexto veio, e o deck e o vídeo dizem isso na tarja.

A ORDEM de execução passa a importar e está fixada no *runner*: o estudante produz o estado que o professor lê, e a administração vem por último porque importa gente e troca senha. Rodar uma suíte isolada continua possível; rodar o professor antes do estudante devolve telas de resultado vazias.

E há um custo aceito: uma experiência nova não tem mais um lugar óbvio. Ela entra em `estudante/` se for missão, em `professor/` se for ferramenta — e a `AGENTS.md` da pasta `apps/experiencias/` passa a dizer isso.

Gatilho de reavaliação. Um papel novo no produto (o *responsável*, por exemplo, se o acesso da família for adiante), ou a suíte passar de 40 minutos de execução — aí a paralelização por papel deixa de ser opcional e o *runner* precisa crescer.

18.53 ADR-52 — Falha do provedor de IA na destilação é 503 com a causa, não 500 mudo

Contexto

Quatro lugares chamam `generateJson`, e três decidem o que fazer quando ele falha: as reações ao vivo caem em lista vazia (o feed fica quieto, como um app em que ninguém respondeu), e a geração de posts e de personas vira *job* falho que o painel de progresso mostra. O quarto, `geracao.controller.ts` (`POST /atividades/:id/destilar`), deixava a exceção subir.
O Nest a convertia em `{"statusCode":500,"message":"Internal server error"}`. E o *log* não salvava: o `ErroDeGeracao` guarda a causa real em `detalhe`, e o `ExceptionsHandler` imprime apenas a `message` — que no `VertexProvider` é a genérica "Falha ao gerar conteúdo estruturado.".
Com `AI_PROVIDER=mock` isso deixa de ser um caso raro e vira certeza: `MockProvider.generateJson` lança SEMPRE, por decisão (o mock não sabe satisfazer schema arbitrário). Ou seja, no modo que existe para subir sem GCP — o recomendado em desenvolvimento, e o *default* dos dois *compose* — **criar atividade era impossível, e nada em lugar nenhum dizia por quê**. Custou uma tarde de investigação: a destilação reproduzida fora da api funcionava, o `UPDATE` do Prisma funcionava, e a resposta só apareceu num `ps eww` do processo, onde se via o `AI_PROVIDER=mock` que o `.env` não declarava.

Decisão. A chamada entra num `try/catch` e a falha vira **503** com `mensagem` legível, acrescida da causa quando existe. A causa é lida como `detalhe ?? message`: o `VertexProvider` enche o `detalhe` e deixa a `message` genérica; o `MockProvider` faz o contrário. Ler só um dos dois deixaria sem diagnóstico exatamente o modo em que a falha é garantida.

503 e não 500 porque a requisição está correta e repetir pode funcionar: o que falhou é uma dependência. É a mesma distinção que a api já faz em `/ativar` com o 422.

Alternativas descartadas.

- *Um `ExceptionFilter` global para `ErroDeGeracao`*: pareceria mais econômico e seria pior — transformaria em 503 também os três chamadores que hoje DEGRADAM de propósito, e a lista vazia das reações ao vivo viraria erro na cara da criança no meio da aula. A decisão de degradar é de cada chamador, e por isso o tratamento fica em cada um.
- *Fazer o `MockProvider` devolver um antagonismo plausível*: seria o mock passando a conhecer o domínio, e um dia divergiria dele em silêncio. Pior: esconderia o caminho de falha em vez de torná-lo legível — em produção o provedor também cai, e é esse caminho que precisa estar exercido.
- *Deixar o 500 e melhorar só o log*: resolveria para quem tem acesso ao *log* da api. O professor continuaria vendo "Internal server error", e a suíte de ponta a ponta continuaria reportando um sintoma que não aponta para a causa.

Consequências. `tests/destilar-ia-indisponivel.spec.ts` trava o comportamento em cinco asserções, incluindo que a atividade NÃO é movida pela metade (nem `status`, nem `pergunta_chave`). O roteiro `scripts/e2e/professor/02-atividade-do-zero.mjs` passou a falhar em 9 s com a causa impressa, no lugar dos 180 s esperando "Perguntas propostas pela IA" — é a mesma lição do `esperarOuFalharCedo`, agora do lado do servidor.

Ruim: a mensagem expõe texto do provedor ao cliente. Aceito porque a rota é de professor autenticado e a causa é operacional ("sem implementação", "resposta vazia"), nunca dado de estudante — e o corte em 200 caracteres impede que um *stack* inteiro vaze para a tela.

Gatilho de reavaliação. Um quarto consumidor de `generateJson` sem fallback, ou o dia em que a destilação ganhar um — por exemplo, um antagonismo escrito à mão pelo professor sem passar pela IA, que hoje ainda depende dela para avaliar o equilíbrio.

18.54 ADR-53 — Um ambiente por *branch*: *environments* do GitHub carregam as `vars`, e a *branch* escolhe a *stack*

> **Contexto**
>
> Até agosto de 2026 havia uma *stack* GCP só (`dev`) e o *deploy* disparava no *push* para `main`, lendo doze `vars` do nível do **repositório**. Com a *stack* de `hml` provisionada (projeto GCP próprio, *workspace* Terraform próprio, WIF *pool* e *deployer* SA próprios), esse desenho não comporta dois ambientes: `vars` de repositório são um conjunto único, e `main` não pode significar "dev" e "hml" ao mesmo tempo.
> O fluxo de trabalho também mudou: ninguém mais commita direto em `main`. *Branch* de trabalho entra por PR em `dev`; `dev` promove por PR para `hml`; e, quando produção existir, `hml` promoverá por PR para `main`.

Decisão. Um *environment* do GitHub por ambiente (`dev`, `hml`, futuramente `prd`), cada um com o conjunto completo de `vars` e o *secret* `TOKEN_SIGNING_KEY_PEM` escopados nele. O `deploy.yml` dispara em `push` para `dev` e `hml`, e cada *job* de *deploy* declara `environment: ${{ github.ref_name == 'main' && 'prd' || github.ref_name }}` — a *branch* **é** o seletor de *stack*, com `main` como a única exceção de nome. Cada *environment* tem *deployment branch policy* restrita à própria *branch*: o de `hml` só aceita *job* vindo de `hml`, então nem um `workflow_dispatch` de outra *branch* consegue assumir a identidade WIF do ambiente errado. O `scripts/configurar-github-ci.sh` deriva o *environment* de `terraform workspace show` — a mesma fonte dos valores — e cria *environment*, *policy*, `vars` e *secret* de uma vez.

Alternativas descartadas.

- *Prefixar as `vars` de repositório* (`DEV_API_SERVICE`, `HML_API_SERVICE`) e escolher no *shell*: dobra os nomes, espalha `if` por todos os *jobs* e não dá nenhuma trava — qualquer *branch* leria as `vars` de qualquer ambiente. O *environment* dá o escopo E a *branch policy* de graça.
- *Um `deploy-dev.yml` e um `deploy-hml.yml`*: os dois divergiriam em silêncio — é a mesma armadilha do roteiro de 66 KB da suíte de ponta a ponta, mantido à mão até descrever telas que não existiam. Um *workflow* só, parametrizado pelo *environment*, não tem como divergir de si mesmo.
- *Deploy por tag* (`v1.2.3-hml`): desacopla o histórico da promoção, mas cria um segundo vocabulário (qual *commit* está em `hml`? o da *tag*, não o da ponta de nenhuma *branch*) e dispensa o PR de promoção — que é justamente onde o *diff* `dev..hml` fica visível e revisável.

Consequências. O bloco `env:` global do `deploy.yml` desceu para dentro dos *jobs*: `vars` de *environment* só resolvem em *job* que declara `environment:`, e no nível do *workflow* chegariam vazias, sem erro. Referenciar um *environment* inexistente o cria em silêncio (com `vars` vazias), então o *job* `mudancas` ganhou um passo de guarda que derruba *dispatch* de *branch* que não é de ambiente. O *cache* do Docker ganhou o ambiente no escopo (`scope=api-dev`, `scope=api-hml`): `dev` e `hml` não invalidam o *cache* um do outro a cada *push* alternado. E o critério de *diff* por

`HEAD^` passou a depender do método de *merge*: *merge commit* e *squash* preservam o primeiro pai como a ponta antiga da *branch* de ambiente; *rebase* o quebraria, e fica desabilitado nas opções do repositório.

Ruim: são $12 \times N$ `vars` para manter (N = ambientes), e a UI do GitHub não compara *environments* lado a lado — um valor desatualizado só aparece como falha no meio do *deploy*. Mitigado porque nenhum valor é digitado: o *script* os copia de `terraform output`, e reexecutá-lo é idempotente.

Gatilho de reavaliação. A chegada de `main`/produção (que testará o mapeamento `main` → `prd` e pedirá *required reviewers* no *environment*); ou o segundo contrato — *environments* escalam por ambiente, não por contrato, e aí a conversa vira um repositório de *deploy* por contrato ou uma matriz.

18.55 ADR-54 — Pessoa nasce por IMPORTAÇÃO: não existe cadastro avulso de usuário

Estado: vigente, com o gatilho acionado (2026-09-01). O cadastro unitário passou a existir — e existe do jeito que o gatilho abaixo prescreve: um formulário que monta a MESMA linha de CSV e chama o MESMO importador, nunca um segundo caminho de escrita em `users`. Ver **ADR-59**, que também abriu a rota separada de `admin-contrato`. Tudo o mais nesta ADR continua valendo, inclusive as três alternativas descartadas.

> **Contexto**
>
> Quem pergunta "como eu crio um usuário?" procura, naturalmente, um botão "Novo" na tela `/admin/usuarios`. Ele não existe, e a ausência nunca esteve escrita em lugar nenhum — estava só implícita no fato de a API não ter `POST /usuarios`.
> O provisionamento desta plataforma não é auto-serviço: a lista de pessoas vem da REDE (a secretaria, o sistema de matrícula), e a identidade que importa é a **matrícula da rede** — com o CPF como chave que promove uma linha repetida a transferência entre unidades (**ADR-28**, **ADR-32**). Uma pessoa não é só um `users`: é lotação, matrícula em turma, período letivo e senha provisória, nascidos juntos na mesma transação.

Decisão. As **únicas** duas chamadas que criam linha em `users` são `POST /rostering/importar` (a planilha simples, uma linha por pessoa) e `POST /rostering/oneroster/users` (o *binding* do padrão, para a rede que já o exporta). *Desde a **ADR-59**, some-se a elas `POST /usuarios/admin-contrato` — o único papel que não cabe numa planilha da escola; `POST /usuarios` não é uma terceira, porque delega à primeira.* As duas exigem `gestor` — na própria escola — ou `admin-contrato`, com `?escolaId=`. Criar UMA pessoa é importar uma planilha de uma linha: a operação é idempotente pela matrícula (e pelo CPF), então repetir não duplica nem reseta a senha de quem já entrou.

O que existe depois da criação são as operações de **balcão**, e essas sim são unitárias: `PATCH /usuarios/:id` corrige nome, CPF, e-mail, papel e `ativo`; `POST /usuarios/:id/reset-senha` devolve uma senha temporária uma única vez.

Alternativas descartadas.

- *Um `POST /usuarios` com formulário na tela*: é uma segunda porta de escrita sobre o mesmo modelo, e portas de escrita divergem. Na primeira regra nova — o CPF que vira transferência, a matrícula que precisa de período letivo aberto, o papel `coordenacao` que normaliza para `gestor` — uma das duas a implementaria e a outra não, em silêncio. É exatamente o motivo pelo qual `POST /usuarios/importar-csv` sobreviveu apenas como *delegação* para o importador, e não como um segundo importador.
- *Auto-cadastro pelo próprio usuário*: são menores. Não há prova de vínculo com a escola que uma pessoa possa apresentar sozinha, e a consequência de aceitar uma seria uma criança dentro do *tenant* errado.
- *Provisionamento just-in-time no primeiro login* (criar a pessoa a partir do *token* do IdP): tentador quando o WSO2 entrar, e recusado desde já — o *token* do IdP não carrega turma nem série, e a pessoa nasceria sem enturmamento, invisível para o professor e fora de qualquer conferência da rede. Quando o WSO2 chegar, ele autentica; quem provisiona continua sendo a importação.

Consequências. *Ruim*: o gestor que percebe "esqueci de incluir a Ana" precisa montar um arquivo para uma linha — atrito real de balcão, e a tela hoje só aceita *upload*, sem campo de colar. Mitigado em parte porque o balcão de verdade (nome errado, CPF trocado, senha perdida, saída da rede) é editar, não criar, e essas rotas existem.

Gatilho de reavaliação. Se a operação pedir cadastro unitário com frequência, a saída é um formulário que monta a MESMA linha de CSV e chama o MESMO importador — nunca um segundo caminho de escrita em `users`.

18.56 ADR-55 — Instrução de agente: `AGENTS.md` canônico por fronteira, `CLAUDE.md` como ponteiro de uma linha

> **Contexto**
>
> Uma pessoa desenvolve esta plataforma, e boa parte do trabalho passa por agentes de codificação. O que eles sabem sobre o projeto vem dos arquivos de instrução que a ferramenta carrega sozinha ao tocar uma pasta — e esses arquivos viraram um padrão *de fato*, nunca escrito: `AGENTS.md` por domínio, com um `CLAUDE.md` de uma linha ao lado. Padrão que não está escrito não se verifica, e os desvios apareceram. A doutrina da Vila do Lume — as regras que protegem a Camada 2 do instrumento — viveu meses em `exp-70-vila-do-lume/docs/CLAUDE.md`: o carregamento é por caminho, então quem editava o jogo em `public/jogo/` nunca a recebia. Ela ainda mandava rodar dois

> `scripts/walkthrough-*.mjs` que deixaram de existir quando a suíte passou a ser organizada por papel (**ADR-51**) — instrução desatualizada, que é pior que instrução nenhuma, porque o agente obedece.

Decisão. Três arquivos, três papéis que não se misturam. `AGENTS.md` é **canônico**: o único lugar onde a doutrina de uma pasta é escrita, e o nome que a maioria das ferramentas lê. `CLAUDE.md` é **uma linha**, `@AGENTS.md` — o Claude Code procura esse nome, o ponteiro importa o canônico, e a regra existe uma vez só. `README.md` continua sendo para pessoas: o que a pasta é e como rodar, e ele *não* entra sozinho no contexto do agente.

Um `AGENTS.md` nasce numa pasta onde se trabalha uma tarefa inteira sem sair dela — um app, um pacote, uma experiência, a suíte de ponta a ponta — e sempre na RAIZ dela, nunca numa subpasta. Entra nele o que, se você não souber, você quebra: regra já violada e o que custou, contrato com quem consome, zona congelada, armadilha que passa no `typecheck`. Não entra o que o código já diz nem o que o `README.md` ao lado já explica. Onde o `README.md` já é a doutrina — `scripts/e2e`, `documentacao-latex` — o `AGENTS.md` é curto e serve de *carregador*: as poucas regras invioláveis e a ordem de abrir o README. Alterar a regra e o arquivo é o mesmo *commit*, e isso entra na *Definition of Done*.

Decisão irmã, do mesmo movimento: **varredura de vulnerabilidade pertence à esteira**, com o segredo e a política de prazos do repositório (**ADR-39**), não à sessão do agente. O agente não dispara `snyk` nem as ferramentas do MCP correspondente; as regras estão em `.claude/settings.json` (escopo do projeto) e no equivalente do usuário. Varredura na sessão gasta cota do plano, produz apontamento sem o portão que o cumpre, responde por uma árvore de trabalho local que não é o que vai a produção — e recicla falsos positivos já analisados e registrados, como o *code injection* do `setInterval` da *bridge*. Quem precisar de uma varredura avulsa a roda na mão.

Alternativas descartadas.

- *Um arquivo só, na raiz*: já foi assim, e o arquivo cresceu até o ponto em que a regra da api ocupava contexto de quem mexia numa experiência. A divisão por pasta existe porque o carregamento é por caminho: cada sessão recebe o que é dela.
- *`CLAUDE.md` com conteúdo próprio ao lado do `AGENTS.md`*: duas fontes de verdade sobre a mesma pasta. Elas divergem, sempre na direção de quem parou de ler a outra — a mesma armadilha do `expires_at` que saiu do *schema* e ficou no registro.
- *Só `README.md`, sem `AGENTS.md`*: o README não entra no contexto sozinho. Foi o que fez a doutrina da Vila do Lume ficar invisível para quem ela tentava avisar.
- *Gerar os `AGENTS.md` a partir do código*: o que vale ali é justamente o que o código não diz — por que a medida é 0,6 e não 0,5, o que custou descobrir. Gerador nenhum sabe disso.

Consequências. *Ruim*: são dezenove pares de arquivos a manter, e cada um pode envelhecer sozinho — o custo é real e recai sobre a mesma pessoa. Mitigado por duas coisas: a proibição explícita de repetir conteúdo entre eles (cópia é o que ninguém atualiza junto) e o item novo na

Definition of Done. *Ruim também*: bloquear o MCP de varredura no escopo do usuário vale para todos os projetos da máquina, não só para este.

Gatilho de reavaliação. Um `AGENTS.md` que ninguém consultou ao mudar a pasta que ele governa — sinal de que ele descreve o que o código já diz, e deve encolher ou desaparecer. Do lado da varredura: a esteira deixar de cobrir uma classe de vulnerabilidade que só a análise local pega.

18.57 ADR-56 — O fluxo por ambiente é imposto por *hooks* versionados, não por *branch protection*

> **Contexto**
>
> A **ADR-53** definiu o fluxo — *branch* de trabalho → `dev` → `hml` → `main` — e o *deploy* passou a depender dele: o *merge* do PR é o que sobe código para a escola. A regra existia em prosa no `AGENTS.md` e nada a sustentava: `git push origin dev` funcionava, e funcionaria de madrugada, no fim de uma sessão longa, sem portão e sem PR.
> A resposta natural seria `branch protection` ou `rulesets`. As duas APIs respondem **403 — "Upgrade to GitHub Pro or make this repository public"**: o repositório é privado numa conta de plano *Free*. Tornar público não é opção (é código de plataforma com dado de menor no desenho), e pagar por assento para uma pessoa, hoje, não se justifica — mas a regra precisa valer **agora**, porque `hml` já é um ambiente real.
> Faltavam também as duas pontas que só aparecem quando produção existe: como se corrige produção sem furar a fila, e como se diz *qual* versão está lá. Não havia uma *tag* no repositório.

Decisão. A barreira é local e versionada: `.githooks/`, ativado por `core.hooksPath` no `prepare` do `package.json` — ou seja, `pnpm install` instala. `pre-push` recusa *push* direto em `dev`, `hml` e `main` (inclusive deleção) e nome de *branch* fora do padrão; `commit-msg` valida *Conventional Commits* com os escopos deste monorepo. Os escapes são **nomeados** (`PERMITIR_PUSH_DIRETO=1`, `PERMITIR_BRANCH_FORA_DO_PADRAO=1`), e não a *flag* genérica `--no-verify`: emergência acontece, e é melhor que ela deixe rastro numa variável que diz o que se está furando.

Junto com isso, duas regras que o fluxo não tinha. **Promoção para `main` leva *tag* semver e *release***, criadas depois do *merge*: MAJOR só em quebra de contrato com quem está fora do repositório (`PROTOCOL_VERSION` da ponte, manifesto, telemetria, contrato do Terraform), MINOR no caso comum, PATCH em correção. ***Hotfix* sai de `main`, entra por PR em `main` e volta para `hml` e `dev` por retro-merge** no mesmo dia. O fluxo operacional inteiro vive em `CONTRIBUTING.md` — a raiz do repositório, onde o GitHub o exibe ao abrir um PR.

Alternativas descartadas.

- *Deixar só a prosa do* `AGENTS.md`: é o estado que produziu o risco. Regra que depende de lembrar dela, às 2h da manhã, não é regra — e o custo de esquecer aqui é *deploy* para escola sem portão.
- *Tornar o repositório público para liberar* `rulesets`: trocar a superfície de exposição do produto inteiro por um recurso de processo. Desproporcional.
- *Husky + commitlint + lint-staged*: três dependências, uma cadeia de configuração e um *hook* gerado, para o que são dois arquivos de *shell* que o repositório inteiro consegue ler. A regra da raiz vale aqui ("simplicidade > sofisticação"), e `core.hooksPath` é nativo do git desde a 2.9.
- *Um workflow de release que cria a tag sozinho no push para* `main`: automação que decide o número da versão precisa de uma fonte para ele, e a única honesta seria um arquivo que alguém edita — o mesmo trabalho manual, com uma engrenagem a mais para falhar. Enquanto a promoção for um ato deliberado de uma pessoa, o comando documentado basta.
- *Trunk-based puro, com feature flags no lugar de* `hml`: exige disciplina de *flag* e um custo de teste que uma pessoa não paga; e `hml` existe porque a rede cliente precisa de um lugar para homologar antes da escola.

Consequências. *Ruim*: *hook* é conselho, não muralha — quem clonar e não rodar `pnpm install` não tem nenhum, e `--no-verify` fura. A barreira protege contra descuido, não contra intenção, e isso é aceitável enquanto o time é de uma pessoa; deixa de ser no primeiro colaborador externo. *Ruim também*: o `prepare` roda em CI e em contêiner, onde o *hook* não serve para nada — é inofensivo (`git config` num *checkout* que existe), mas é ruído.

Gatilho de reavaliação. O primeiro colaborador com *commit* no repositório, ou a conta virar *Pro* / organização: nos dois casos as mesmas regras viram `ruleset` no servidor, e os *hooks* passam a ser conveniência local (retorno rápido) em vez de barreira única.

18.58 ADR-57 — Observabilidade nativa do Google Cloud, com OpenTelemetry como fronteira; nenhuma *stack* auto-hospedada

Estado: aceita (2026-09-02; proposta em 2026-09-01 a partir do estudo do Capítulo 17). Aceita com a primeira peça no código: log JSON com *redaction* na API e no hub (fase 1) e *traces* OpenTelemetry com o exportador direto do Cloud Trace nos dois processos (fase 2, *traces*); as métricas da aplicação e o erro de cliente seguem o roteiro do capítulo.

> **Contexto**
>
> A plataforma é diagnosticável e não é observável: log em texto sem correlação, nenhuma métrica de aplicação, nenhum alerta, nenhuma sonda configurada no Cloud Run, erro dentro do `iframe` sem rastro. O risco `R-05` exige alerta antes de novembro de 2026. Uma pessoa opera; um contrato é um projeto GCP; haverá vários contratos. A regra da raiz proíbe serviço novo sem necessidade demonstrada, e a franquia gratuita do Google Cloud

> Observability (50 GiB de log, 150 MiB de métrica, 2,5 M de *spans*, 1 M de *uptime checks* por mês) cobre um contrato em regime com folga, desde que o log de requisição seja amostrado na origem.

Decisão. Os sinais operacionais vão para o Cloud Logging, Cloud Monitoring, Cloud Trace e Error Reporting **do projeto do contrato**. A API e o hub emitem log JSON pelo *stdout* (`pino`, com *redaction* mecânica e `severity`/`trace` nos campos que o Cloud Logging reconhece) e *traces* e métricas pelo SDK OpenTelemetry com os exportadores do Google, **direto do processo, sem *sidecar***. Alertas são poucos e por sintoma (SLO com *burn rate*), em duas severidades. Tudo o que é recurso — canais, políticas, *checks*, painel, exclusões, *bucket* de auditoria, *dead letter*, sondas do serviço — é Terraform, e passa a constar do contrato em `docs/INFRA.md`. A visão entre contratos é um *metrics scope* num projeto de observação sem recursos; log não atravessa projeto.

Alternativas descartadas.

- *Grafana + Loki + Tempo + Prometheus auto-hospedados*: uma VM central (ou uma por contrato), com retenção, *backup*, atualização e autenticação por conta própria, recebendo dado de N projetos por rede e IAM próprios — e custando, no mínimo, mais que o nativo no pior caso. Para uma pessoa, é a peça que quebra às 2h da manhã. Volta se uma rede exigir que os sinais fiquem fora do Google Cloud, ou se uma consulta demonstrada não couber no Logs Explorer e no Log Analytics.
- *SigNoz / OpenObserve / HyperDX* (tudo-em-um OpenTelemetry): mesma objeção operacional, com a vantagem de um só binário. Gatilho igual ao anterior.
- *Grafana Cloud Free* como destino (10 k séries, 50 GB de log, 14 dias): o dado sairia do projeto do contrato para uma conta de terceiro sem DPA da rede. Aceitável apenas como **painel** de leitura sobre a fonte de dados Cloud Monitoring, se a ergonomia do Grafana compensar — é leitura, não coleta.
- *Datadog / New Relic / Dynatrace*: centenas de dólares por mês por contrato, para um time de operação que não existe. A instrumentação OpenTelemetry deixa a porta aberta sem pagar por ela agora.
- *OpenTelemetry Collector como sidecar* (a recomendação do Google): justifica-se com vários destinos ou *tail sampling*; hoje há um destino e amostragem de cabeça basta. É um contêiner a mais em cada serviço de cada contrato, com versão e falha próprias.
- *Log de requisição na aplicação* (`morgan`, interceptor): duplicaria o log que o Cloud Run já grava, dobrando o sinal mais caro para acrescentar nada.
- *Continuar como está, com o checklist semanal do Apêndice F*: é o estado que produziu o `R-05`. Funciona em `dev` com uma pessoa olhando; não funciona com cem escolas às 8h.

Consequências. *Boa*: zero peças novas para operar; custo estimado em cerca de US$ 1 por contrato por mês hoje e US$ 7 quando a cobrança de política de alerta entrar; um contrato novo herda tudo do Terraform. *Ruim*: dependência do fornecedor nos **destinos** (painel, política,

consulta são em formato do Google); mitigada porque a instrumentação é OpenTelemetry e o log é JSON — trocar de destino é trocar exportador e reescrever painéis, não reinstrumentar. *Ruim também*: dois custos crescem com o tráfego e precisam de teto na origem — log de requisição (por isso a exclusão e a amostragem são recursos do Terraform) e cardinalidade de métrica (por isso `escola_id` não é rótulo). E a cobrança anunciada de política de alerta muda a economia de "uma política por curiosidade": a lista fica curta também por dinheiro.

Gatilho de reavaliação. Uma rede exigir sinais fora do Google Cloud; um segundo operador (plantão, rodízio); um segundo destino (SIEM, Grafana) ou a necessidade de *tail sampling* — aí o *sidecar* entra; ou a franquia estourar em dois meses seguidos apesar da exclusão e da amostragem.

18.59 ADR-58 — Erro de cliente entra pela API; nenhum SDK de terceiro no *bundle* da experiência

Estado: proposta (2026-09-01).

> **Contexto**
>
> A classe de falha mais cega da plataforma é a que acontece no navegador do estudante, dentro de um `iframe` de origem diferente: não há `window.onerror`, `unhandledrejection` nem mensagem `exp:erro` na ponte, e os *bundles* não têm *source-map*. O caminho padrão da indústria é um SDK de rastreio de erro (Sentry e semelhantes) dentro do *bundle*, apontando para um *DSN*. A regra 2 da raiz proíbe segredo e URL interna em *bundle*; a regra 6 proíbe PII fora de controle. Um *DSN* não é segredo, mas é um destino: o SDK manda *user agent*, URL, *stack*, *breadcrumbs* e o que mais o erro arrastar para um processador fora do contrato — para um erro que aconteceu na tela de uma criança de uma rede pública, coberta por um DPA que não menciona esse processador.

Decisão. A `@edu/sdk` captura `window.onerror` e `unhandledrejection` e envia para `POST /observabilidade` da **própria API**, com o *token* que já tem: *schema* Zod fechado (`expId`, `versao`, `mensagem`, `stack`, `url`, `userAgent`), 8 KB no máximo, *dedupe* por *hash* e no máximo cinco por sessão no cliente, cota por `(escola, usuario)` na API. A API redige, acrescenta `escola_id` do *token* e emite **uma** linha `severity: ERROR` com `@type` de `ReportedErrorEvent`; o Error Reporting agrupa por `(expId, versao)`. O *build* das experiências gera *sourcemaps* ocultos, publicados com ACL privada ao lado do *bundle*, na mesma pasta versionada. A ponte ganha `exp:erro`, opcional e compatível com hub antigo, só para o hub trocar a tela. O hub faz o equivalente com `error.tsx` e `global-error.tsx`. Web Vitals seguem o mesmo caminho, amostrados e sem identificador de pessoa.

Alternativas descartadas.

- *SDK do Sentry (SaaS) no bundle*: melhor produto da categoria (*session replay*, *breadcrumbs*, *release health*); plano Developer grátis para uma pessoa. Descartado pelo destino, não pela ferramenta: dado de menor saindo do contrato sem DPA.
- *GlitchTip ou Sentry auto-hospedados no projeto do contrato*: resolve o destino, mas é uma peça a mais (Postgres, Redis, workers) por contrato — ou uma central com o mesmo problema de N projetos. Fica como saída se o agrupamento do Error Reporting se mostrar insuficiente para 48 experiências.
- *Só `exp:erro` na ponte, e o hub registra*: cria dependência do hub para um sinal que a experiência já consegue mandar sozinha (tem *token* e base da API), e perde o erro que acontece antes do *handshake* — justamente o mais comum. A ponte fica com o papel de interface, não de transporte.
- *Sourcemaps públicos no bundle*: expõem o fonte da experiência — e, na exp-01, a lógica da revelação que o estudante não deve ver antes da hora.
- *Não capturar erro de cliente*: é o estado atual, e é a maior cegueira da Figura 17.1.

Consequências. *Boa*: nenhum terceiro no *bundle*; a regra 6 vale por construção, porque a *redaction* é a da API; a versão da experiência já está no caminho, então a *stack* é mapeável sem *release* à parte. *Ruim*: sem *session replay* nem *breadcrumbs* — o que se tem é mensagem, *stack* e contexto; um laço de erro numa experiência é contido pela cota, mas o que passa da cota se perde de propósito. *Ruim também*: mais uma rota pública autenticada na API, com Zod e cota, para manter.

Gatilho de reavaliação. O Error Reporting agrupar mal os erros das experiências (assinaturas instáveis por minificação), ou o diagnóstico exigir *replay*: nos dois casos a saída é um Sentry-compatível **dentro do projeto**, nunca o SaaS.

18.60 ADR-59 — Cadastro unitário existe, e ele DELEGA ao importador; `admin-contrato` tem rota própria

Estado: aceita (2026-09-01). Aciona o gatilho de reavaliação da **ADR-54**, que segue válida em tudo o mais.

> **Contexto**
>
> A **ADR-54** previu este dia por escrito: "se a operação pedir cadastro unitário com frequência, a saída é um formulário que monta a MESMA linha de CSV e chama o MESMO importador". A operação pediu. O atrito que a própria ADR registrou como consequência ruim — "o gestor que percebe que esqueceu de incluir a Ana precisa montar um arquivo para uma linha" — é diário no balcão da escola, e a tela só aceitava *upload*.
> O que NÃO mudou é o motivo da decisão original: uma pessoa não é uma linha em `users`. É lotação, matrícula em turma, período letivo aberto e senha provisória, nascidos na mesma transação, com o CPF decidindo se aquilo é gente nova ou transferência

> (**ADR-28**, **ADR-32**). Duas portas de escrita sobre esse conjunto divergem na primeira regra nova.

Decisão. Existe `POST /usuarios`, e ele **não escreve em `users`**: valida a forma do corpo com Zod, pergunta se a identidade já existe e, não existindo, serializa os campos como UMA linha da planilha (`planilhaDeUmaPessoa`, na mesma constante `COLUNAS_MODELO` que o leitor consome) e chama `importarPlanilha`. Toda a regra de domínio — CPF com dígito verificador, série e turno normalizados, login gerado do nome, turma criada quando falta, lotação, matrícula — continua existindo em UM lugar só.

Duas diferenças em relação a importar um arquivo de uma linha, e as duas são deliberadas:

- **Criar é criar.** A importação é idempotente de propósito: reimportar a planilha do ano não pode duplicar ninguém. Num formulário chamado "Novo usuário" a mesma virtude vira armadilha — o gestor digita a Ana de novo, a linha antiga é atualizada e a tela diz "criado". Por isso o cadastro unitário **pergunta antes** (CPF na rede; matrícula e login na escola) e responde **409 sem escrever**, apontando o caminho certo: editar, transferir ou importar.
- **CPF de outra escola não revela nada.** O 409 desse caso sai sem id, sem nome e sem dizer qual unidade — é dado de outro *tenant*. A transferência continua sendo pela importação, que reconhece o CPF e move a matrícula junto.

E `admin-contrato` ganha rota separada (`POST /usuarios/admin-contrato`, `@Roles("admin-contrato")`): ele não passa pelo importador porque não cabe numa planilha DA ESCOLA — `escola_id` nulo, sem lotação, sem turma, e `PAPEIS_IMPORTAVEIS` sempre o excluiu. O login dele é único na REDE inteira, não na escola: o índice `(escola_id, username)` não cobre linha com `escola_id` nulo, e a autenticação procura o *username* na rede toda — dois donos do mesmo login viram recusa no login dos dois.

Alternativas descartadas.

- *Escrever direto em `users` a partir do formulário*: é exatamente a segunda porta que a **ADR-54** recusou. Na primeira regra nova — um papel que normaliza, um período letivo que precisa estar aberto — uma das duas aprenderia e a outra não, em silêncio.
- *Extrair do importador um método que receba objetos em vez de CSV*: mais limpo à primeira vista, e pior no que importa. A validação por campo vive no leitor da planilha; um caminho que a contorne precisa reescrevê-la em Zod, e aí são duas validações para manter — a divergência de novo, um nível abaixo. O custo aceito é a conversão JSON → CSV → pessoa, invisível em latência para uma linha.
- *Montar o CSV no NAVEGADOR e postar em `/rostering/importar`*: põe a regra de escape RFC 4180 no cliente e faz a api receber um arquivo que ela não sabe que veio de um formulário — sem chance de dar 409 no lugar de atualizar.

- `admin-contrato` *pela MESMA rota, com* `papel` *no corpo*: metade dos campos ficaria inerte, a autorização viraria um `if` escondido no *service* em vez do `@Roles` do método, e a matriz papel x rota perderia a linha que hoje se lê sozinha.

Consequências. *Ruim*: são duas rotas para "criar pessoa", e quem lê o apêndice C precisa entender por que a segunda existe. A conversão em três saltos é indireta o bastante para assustar quem depura pela primeira vez — daí o comentário no serializador. E o 409 tem uma janela de corrida de milissegundos: se alguém criar a mesma pessoa entre a pergunta e a escrita, o importador atualiza (corretamente) e a rota responde 409 `JA_EXISTIA` em vez de "criado", que é a resposta honesta. *Bom*: a regra de domínio segue em um lugar, o formulário ganha de graça tudo o que a importação aprendeu, e o gestor deixa de abrir o Excel para incluir uma criança.

Gatilho de reavaliação. Se o cadastro unitário precisar de um campo que a planilha não tem — e que portanto não caiba numa coluna de `COLUNAS_MODELO` —, a delegação deixou de servir: a saída é acrescentar a coluna ao modelo (e ao leitor), não abrir o caminho direto de escrita.

18.61 ADR-60 — Microsoft Clarity carrega no hub e dentro do *bundle* da experiência

> **Contexto**
>
> A plataforma media comportamento por um caminho só: a telemetria própria da `@edu/sdk` (`POST /telemetry/batch`), que registra os *verbos* que cada experiência declara no manifesto — `attempted`, `interacted`, `progressed`, `scored`, `completed`. Ela responde "o que a pessoa fez", e responde bem, porque é desenhada com o domínio dentro. Ela não responde "o que a pessoa *tentou* fazer e não conseguiu": o clique repetido num elemento que não é botão, o *scroll* que sobe e desce numa tela que não explica, a tela abandonada antes do primeiro evento. Nenhum *verbo* nasce de uma frustração — só de uma ação que o código previu.
>
> Há uma decisão anterior em sentido contrário, e ela é explícita: a **ADR-58** tirou o SDK de erro de terceiro do *bundle* da experiência, com o argumento de que um *bundle* público manda dado "para um processador fora do contrato e do DPA da rede — para um erro que aconteceu na tela de uma criança". Esta ADR **abre exceção** a ela, para analytics de produto, e o faz de olhos abertos: o argumento da **ADR-58** continua verdadeiro, e as consequências abaixo o repetem.
>
> O pedido foi cobertura total: hub e missões. A restrição técnica que decide o desenho é que o `iframe` da experiência é **cross-origin** (o hub sai do Cloud Run, os *bundles* do *bucket*). Um *script* de gravação carregado no hub não enxerga nada dentro dele: grava um retângulo vazio exatamente onde o estudante passa a maior parte da sessão. Cobrir a missão exige carregar o *script* **dentro** do *bundle*.

Decisão. O *snippet* oficial do Microsoft Clarity carrega nas duas superfícies, a partir de **uma** fonte: `packages/sdk/src/clarity.ts`, que exporta o corpo do *snippet* e as chamadas tipadas

(`identify`, `set`, `event`, `upgrade`), todas *no-op* quando `window.clarity` não existe — bloqueador de anúncio e rede de escola barrando domínio são o caso comum, não a exceção.

Superfície	Como carrega
Hub	`components/clarity-hub.tsx` monta o *snippet* com `next/script` (`afterInteractive`) no `app/layout.tsx` **raiz** — o único que cobre a *landing* pública, o login, as quatro áreas de papel, `/exp/**` e `/dev-embed`.
Identificação	`components/clarity-identidade.tsx`, montado na `casca-plataforma.tsx` — o único componente por onde `(plataforma)/**` e `/exp/**` passam os dois. Re-dispara a cada mudança de rota: o App Router navega sem recarregar a página, e o Clarity pede `identify` por **página**.
Experiência	`createExperiencia()` chama `carregarClarity` depois do *handshake*, com o id que chega no `config` do `hub:init` — campo aditivo, sem mexer no `PROTOCOL_VERSION`.

O id vai por **uma** variável, `NEXT_PUBLIC_CLARITY_PROJECT_ID`, e **ausente significa desligado** nas duas superfícies. Ela é inlinada no *bundle* do *client* durante o `next build`, então entra por `--build-arg` no `apps/hub/Dockerfile`, alimentado por `vars.CLARITY_PROJECT_ID` do *environment* no `deploy.yml`.

Duas escolhas de conteúdo fecham a decisão. O `identify` manda o **UUID opaco** do estudante e nada mais — o `friendlyName`, que levaria o nome, fica de fora, e escola e turma vão por `id`. E o `track()` da `@edu/sdk` passa a alimentar os dois destinos: a telemetria própria com a carga útil inteira, o Clarity só com `verbo:objeto`. A *resposta* do estudante não sai do contrato.

Alternativas descartadas.

- *Só nas superfícies adultas* (professor, gestor, `admin-contrato`): respeita o aviso do próprio Clarity sobre menores e não abre exceção nenhuma. Descartada porque deixa de fora o público principal — é no estudante que a dúvida de interface custa aprendizagem.
- *Só no hub, missões pela telemetria própria*: manteria terceiro fora do *bundle* público. Descartada pelo motivo técnico do contexto: o `iframe` é cross-origin, e "cobrir o hub" deixaria a maior parte da sessão do estudante como um retângulo vazio na gravação.
- *Pacote `@microsoft/clarity` (npm)*: 7,6 KB, zero dependências, com tipos. É CJS puro — sem campo `module` — e o Vite das experiências já custou tempo neste repositório com ESM / CJS. Internamente ele só injeta o mesmo *script* que está no `clarity.ts`; pagar uma dependência por isso não se justifica.
- *`VITE_CLARITY_PROJECT_ID` no bundle de cada experiência*: seria o precedente do `VITE_VIDEOS_BASE_URL`. Descartada porque `import.meta.env` dentro da `@edu/sdk` quebra o *build* do hub (que transpila o mesmo arquivo), e porque desligar passaria a exigir republicar as seis experiências.
- *Um projeto Clarity separado para o `iframe`*: dois painéis para uma sessão só. O mesmo id com a *tag* `superficie` separa o que precisa ser separado e mantém a costura pelo `identify`.

Consequências. *Ruim, e o principal*: a documentação do Clarity declara que ele **não deveria ser usado em sites ou aplicações destinadas a menores de 18 anos**, e a plataforma vai do 1º ano do Fundamental ao 3º do Médio. Isto é decisão de produto assumida, não descuido, e precisa constar de qualquer conversa de DPA com uma rede — ver `R-16` no Capítulo 19.

Ruim: a gravação de sessão sai do contrato e do projeto GCP da rede. Ao contrário do log e da métrica, que ficam no projeto do contrato, o *replay* vai para um processador de terceiro, sob os termos dele.

Ruim: as experiências disfarçadas (`exp-01`, `exp-02`, `exp-03`) carregam no `index.html` o aviso de que nada ali pode denunciar o exercício. Requisições para `clarity.ms` aparecem no DevTools, e um estudante que abra o inspetor vê um domínio que o clone imitado não teria.

Ruim: mexer na `@edu/sdk` republica as seis experiências (o filtro de mudanças do `deploy.yml` inclui o pacote), então toda alteração aqui custa um *bump* de versão em seis manifestos.

Bom: o `track()` existente virou instrumentação de funil sem tocar em nenhuma experiência, e o kill-switch é a ausência de uma variável — desligar não exige alterar código, e a máquina de quem desenvolve e a suíte e2e já nascem fora do painel.

Gatilho de reavaliação. Uma rede pública recusar o processamento em contrato, um parecer jurídico sobre o aviso de menores, ou a gravação deixar de responder pergunta que a telemetria própria não responda — nos três casos o caminho de volta é apagar uma variável de *environment*, e o de frente é um *replay* auto-hospedado dentro do projeto do contrato.

18.62 ADR-61 — Trabalho paralelo acontece em *worktree*, com a base escrita à mão

> **Contexto**
>
> O fluxo da **ADR-53** e da **ADR-56** descreve *branches*, não diretórios: pressupõe uma pessoa, uma cópia do repositório, uma tarefa por vez. Passou a haver mais de um agente trabalhando ao mesmo tempo no mesmo repositório, e alternar de *branch* na mesma cópia não serve para isso — há uma árvore de trabalho só, e ela carrega as alterações não commitadas de quem estava antes. Duas tarefas em paralelo na mesma cópia produzem um estado misturado, e é dele que sai o *commit*.

Decisão. Cada frente de trabalho paralela ganha um *worktree* em `.claude/worktrees/<nome>`, criado com a base **escrita explicitamente** — e a base é `origin/dev`.

Um clone antigo deste repositório carrega `origin/HEAD` apontando para `origin/main` (a *branch* padrão no GitHub já é `dev`; o ponteiro local não acompanha sozinho), então qualquer ferramenta que "parta da *branch* padrão" abre a *branch* sobre produção, e a **ADR-53** diz que trabalho nasce de `dev`. O erro é silencioso: a *branch* compila, os testes passam, e o PR mostra um diff cheio de *commits* que já estavam em `dev`. A raiz se conserta uma vez por clone (`git remote set-head`

`origin --auto`); escrever a base no comando é o que evita depender do *default*, consertado ou não.

O paralelismo é de edição e teste unitário; o *runtime* é *singleton*. Portas fixas com `strictPort`, *compose* de nome fixo e um banco com um *schema*: dois *worktrees* não rodam a *stack* ao mesmo tempo, e isso é deliberado — é o que faz `pnpm dev` falhar alto em vez de servir a experiência errada. No *worktree*: editar, `typecheck`, `lint`, os testes unitários e de contrato, o *build*, e o Vite de **uma** experiência em porta alternativa, embedado pelo hub que já está no ar (`/dev-embed?src=`). Na cópia principal, uma frente por vez: a *stack* inteira, a suíte de ponta a ponta e as migrações. A *Definition of Done* exige a suíte do papel afetado para toda tela — numa frente em *worktree* isso significa trazer a *branch* para a cópia principal e rodá-la de lá, um agente por vez.

`.gitignore` já cobre o caminho (`.claude/*`), então o *worktree* não aparece no estado da cópia principal. O ciclo de *commit*, PR e promoção não muda em nada — o *worktree* é onde o trabalho acontece, não um fluxo novo. O passo a passo está em `CONTRIBUTING.md`.

Alternativas descartadas.

- *Alternar de branch na mesma cópia*: é o estado que produziu o problema. Serializa o trabalho e mistura alteração não commitada de tarefas diferentes.
- *Clones separados do repositório*: funciona, e custa uma cópia inteira do histórico e um `pnpm install` do zero por frente. O *worktree* compartilha o diretório de metadados e só duplica o que precisa ser duplicado.
- *Worktree fora de `.claude/`*: exigiria uma entrada nova no `.gitignore`, e um esquecimento ali significa *worktree* inteiro entrando num `add -A` — o mesmo acidente que já custou 250 MB de *cache* de *webpack* no histórico.

Consequências. *Ruim*: `node_modules` e o *client* do Prisma são por *worktree*, então cada frente paga um `pnpm install` (e um `prisma generate`) na criação. *Ruim*: `pnpm docker:dev` de dentro de um *worktree* recria os contêineres da *stack* principal — o nome do projeto do compose vem do diretório, e os dois disputam as mesmas portas. Quem precisa da *stack* inteira usa a cópia principal. *Ruim*: quatro arquivos que toda frente toca brigam no *merge* — a numeração de ADR (dois agentes já escreveram a mesma "ADR-N" ao mesmo tempo: reserve o número conferindo `origin/dev`), o `main.pdf` binário (resolve-se regerando, nunca escolhendo um lado), o `AGENTS.md` da raiz e o `pnpm-lock.yaml`. *Ruim*: cada *merge* em `dev` dispara um *deploy* que não cancela o anterior; PRs em rajada viram *deploys* em fila. *Bom*: a árvore principal fica intocada, com o que estiver em andamento nela preservado.

Gatilho de reavaliação. O trabalho paralelo deixar de acontecer, ou o repositório ganhar CI que rode as frentes em contêiner — nos dois casos o *worktree* vira conveniência local em vez de regra. E o inverso: a fila da suíte de ponta a ponta na cópia principal virar gargalo **medido** (duas frentes esperando uma pela outra, mais de uma vez) é o que justifica parametrizar porta e nome do *compose* por *worktree* e um banco por *branch* — não antes.

18.63 ADR-62 — O WAF não inspeciona o corpo de rota autenticada nem de texto livre de estudante

> **Contexto**
>
> O Cloud Armor da **ADR-57** entrou com sete regras pré-configuradas do OWASP CRS 3.3 em `sensitivity 1`, e em `dev` já saiu de *preview* — bloqueando de verdade. Nos primeiros dias, **242 requisições** `POST /telemetry/ingest` responderam 403 com `body_denied_by_security_policy`, todas do agente `APIs-Google`: é o *push* do Pub/Sub, negado pela assinatura `owasp-crs-v030301-id921150-protocolattack` — "HTTP Header Injection via payload (CR/LF detectado)".
>
> O envelope do *push* é um JSON com a mensagem em *base64*, e a 921150 casa qualquer corpo que carregue `\r\n`. O prejuízo é silencioso duas vezes: a *subscription* reentrega para sempre (não há DLQ), e a tela de resultados do professor fica vazia sem erro em lugar nenhum — a requisição morre no *load balancer*, antes da api, então não há log de aplicação a consultar.
>
> A mesma assinatura ameaça a exp-03: a pergunta do estudante é texto livre, e *Shift+Enter* ou colar de outra aba produzem quebra de linha legítima. O 403 do WAF não carrega cabeçalho de CORS, então o navegador reporta falha de rede: o estudante lê "A conexão falhou" e não há nada no servidor apontando para o WAF.

Decisão. Duas exceções cirúrgicas na política, e nenhuma regra desligada.

1. Uma regra `allow` em **priority 900** — antes das WAF, que começam em 1000 — casando `request.path.startsWith('/telemetry/ingest')`. Abrir o caminho não abre a rota: ela exige o *token* OIDC do Pub/Sub (`TELEMETRIA_PUSH_SA` e `TELEMETRIA_PUSH_AUDIENCE`) e responde 401 sem ele. O WAF ali era redundância que só custava dado.
2. Um `preconfigured_waf_config` na regra `protocolattack` (1006) excluindo **apenas** o `target_rule_id 921150` nos prefixos `/unoogpt/` e `/ai/` — as rotas que recebem texto livre de estudante. As outras seis assinaturas de protocolo continuam valendo nesses caminhos, e a 921150 continua valendo em todo o resto da api.

Alternativas descartadas.

- *Desligar a regra `protocolattack` inteira*: resolveria os dois sintomas com uma linha a menos, e removeria *smuggling* e *splitting* da superfície inteira da api por causa de duas rotas.
- *Voltar a política para preview*: transforma um bloqueio visível em bloqueio nenhum, e junto some a proteção contra os varredores que já batem no IP do *load balancer* todos os dias.
- *Fazer a api aceitar o push por outro caminho* (endpoint interno, fora do LB): exigiria expor a api sem o LB ou montar rede nova para um único endpoint — muito mais superfície do que a exceção.

- *Sanitizar o CR/LF antes de enviar* (no SDK e no publicador da telemetria): trata o sintoma no emissor errado. A quebra de linha na pergunta do estudante é conteúdo legítimo, e removê-la mudaria o dado que a missão mede.

Consequências ruins. `/telemetry/ingest` passa a não ter WAF nenhum — se o *guard* de autenticação daquela rota regredir, não há segunda camada. E as exclusões são por prefixo de caminho: uma rota nova sob `/ai/` herda a exclusão sem que ninguém decida isso.

Gatilho de reavaliação. A telemetria deixar de entrar por *push* no LB (um *worker* separado, como o desenho original previa) devolve o caso 1 ao WAF. E o Cloud Armor publicar exclusão por *método* além de caminho permitiria estreitar o caso 2 para os `POST` que de fato carregam texto do estudante.

18.64 ADR-63 — O CI não publica a mídia pesada, mas se recusa a publicar sem ela

> **Contexto**
>
> Mídia acima de 2 MB não entra no git nem no *bundle*: os 21 *mp4* da exp-02 (52 MB) vivem no *bucket* de *assets* do contrato, e o *build* recebe a base pública em `VITE_VIDEOS_BASE_URL`. O *upload* sempre foi manual, uma vez por contrato, e isso estava registrado como lacuna conhecida em `docs/INFRA.md`.
> Em 2026-09-01 a lacuna cobrou. A exp-02 foi publicada no contrato `entre-redes-dev` com o *bucket* de *assets* vazio: os 21 arquivos responderam 404, e como o `ShortsPlayer` do *design system* deriva `waiting` só de `waiting`/`playing` — sem escutar `error` — o evento `playing` nunca chegou e o feed ficou em "Carregando o vídeo…" para sempre. Salvar e Curtir são desabilitados enquanto o vídeo carrega (para não salvar sem assistir), então a missão ficou **impossível de concluir**. Nada falhou: nem o *deploy*, nem o log da api. Quem descobriu foi um estudante, na frente da tela.

Decisão. O CI continua **não** publicando a mídia — o *runner* faz *checkout* do git, e os arquivos não estão lá — e passa a **conferi-la**. Uma experiência com mídia externa declara um `midia-externa.json` versionado ao lado do `package.json`, com o prefixo no *bucket* e a lista de arquivos esperados. Antes de subir o *bundle* daquela experiência, o *job* `publicar-exps` lista o prefixo no *bucket* de *assets* e **falha o deploy** se faltar algum, imprimindo o que falta e o comando de *upload*.

Isso exige a var `ASSETS_BUCKET` no *environment* do GitHub, escrita por `scripts/configurar-github-ci.sh` a partir do *output* `bucket_assets` do Terraform. Sem a var, o passo apenas avisa — um *environment* ainda não reconfigurado não deve travar *deploy* por uma verificação que ele não tem como fazer.

Em paralelo, e independente disto, a experiência passou a tratar o `error` do `<video>` por conta própria: vídeo que falha sai do estado de carregamento, as ações voltam e a missão continua

concluível. As duas coisas respondem à mesma falha em camadas diferentes — uma impede publicar quebrado, a outra impede que o estudante fique preso quando algo quebrar mesmo assim.

Alternativas descartadas.

- *Versionar os mp4 no git*: 52 MB por experiência, num repositório que já teve 250 MB de *cache* de *webpack* entrando por um `add -A` distraído. Para 48 experiências, o histórico ficaria impraticável.
- *Git LFS*: resolveria o peso e acrescentaria um serviço com cota, um passo de *checkout* novo e uma forma nova de o *deploy* falhar — por mídia que muda uma vez por contrato.
- *O CI baixar a mídia de uma origem canônica e republicar*: exige escolher e operar essa origem (outro *bucket*, *release assets*), e transfere o problema em vez de resolvê-lo — alguém continua tendo que colocar o arquivo lá na primeira vez.
- *Só documentar melhor o passo manual*: já estava documentado. A lacuna estava escrita em `docs/INFRA.md` e mesmo assim o contrato subiu sem a mídia.

Consequências ruins. O *deploy* de uma experiência com mídia passa a depender de um recurso que o *job* não controla: *bucket* renomeado ou permissão de leitura retirada da *service account* do CI quebra a publicação de um *bundle* que estava correto. E a lista de arquivos no `midia-externa.json` é mantida à mão — acrescentar um vídeo sem acrescentar a linha faz a conferência passar sem conferir.

Gatilho de reavaliação. A terceira experiência com mídia externa: com uma, a lista à mão é honesta; com três, o `midia-externa.json` deve passar a ser gerado do catálogo da api, que já sabe o nome de cada arquivo.

19

Riscos e evolução

19.1 Registro de riscos

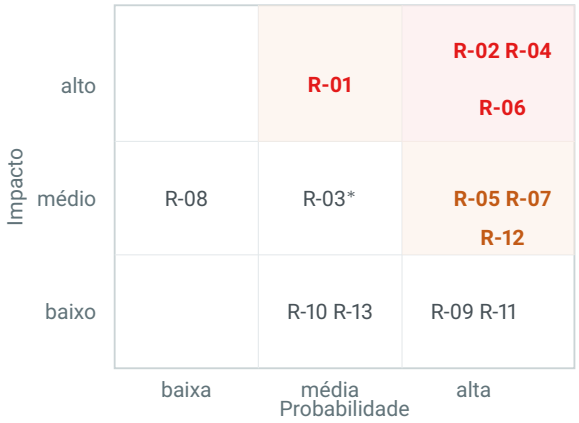
Probabilidade e impacto em escala de três (baixo, médio, alto). A ordem é por severidade, e a severidade considera o prazo de novembro de 2026.

ID	Risco	Prob	Imp	Mitigação e estado
R-01	**Carga não verificada.** O comportamento no pico de 4.000 simultâneos é projetado, não medido.	Média	**Alto**	Teste de carga com k6 — 4.000 *logins* e aberturas de experiência em dez minutos — **obrigatório antes de novembro de 2026**. **Não executado.** É o maior risco técnico aberto.
R-02	**Retenção indefinida de dado de menor.** Não há prazo declarado para `exp_70_sessoes` nem para `telemetria_eventos`.	Alta	**Alto**	Decisão consciente (**ADR-13**) em troca de reprocessamento. Com a tabela particionada, a rotina de expurgo passou a ser trivial — o que remove o último argumento técnico. **Pendente de decisão de produto e contrato**, e é da plataforma inteira.
R-13	**Latência de IA numa sala inteira.** O turno da exp-03 é síncrono e leva de 10 a 30 segundos, e a largada é simultânea: 30 estudantes perguntam ao mesmo tempo.	Média	**Alto**	O desenho aceita a latência (a tela cobre com indicador de "consultando", e a SDK espera 45 s). O que **não** foi medido é a concorrência: 30 turnos simultâneos por turma, com até seis voltas de ferramenta cada. Entra no mesmo teste de carga de `R-01`. Saída conhecida se reprovar: `streamGenerateContent` no provider — mudança dentro de `src/ai`, sem tocar o módulo da experiência.

ID	Risco	Prob	Imp	Mitigação e estado
R-14	**Custo de IA da exp-03.** Uma partida custa duas ordens de grandeza mais que um turno de tutor.	Certa	Médio	**MEDIDO** em 2026-08-27: 71.397 tokens numa partida completa de 8 turnos (**ADR-24**). Os tetos foram fixados sobre esse número, e a medição achou de passagem que a quota diária de 20.000 recusava o quarto turno — corrigida para 300.000. **Fechado**, com um resíduo: a medição é de UMA partida, e o custo varia com o número de voltas que o modelo escolhe dar.
R-15	**Guia de condução do debate da exp-03 não existe.** A missão deliberadamente não revela a Lacuna de nenhuma fonte por software: a lição acontece no debate.	Certa	**Alto**	O guia do projeto de origem está vazio. A ficha publicada (competência, quatro atos, preparo) foi redigida a partir das ADRs e dos dois *playtests*, e está marcada como ponto de partida. **Pendente de revisão pedagógica** — é a lacuna mais séria desta experiência, e não é técnica.
R-03	**Localidade de processamento de IA.** O tráfego de IA sai de São Paulo, inclusive o do estudante.	Certa	Médio	Consequência aceita e documentada (**ADR-16**). **Precisa constar do contrato e do DPA de cada rede.** Voltar à região exige trocar modelo e localização juntos.
R-04	**Fator humano único.** Uma pessoa desenvolve e opera.	Certa	**Alto**	Mitigado *por arquitetura*: menos peças, reversão por ponteiro, decisões registradas em ADR e este documento. Não é mitigável por engenharia sozinha.
R-05	**Sem *alerting*.** Há log e métrica nativa; não há alerta nem painel, e nenhuma sonda chama `/readyz`. Erro no navegador do estudante não deixa rastro.	Alta	Médio	**Estudado e planejado** (Capítulo 17, **ADR-57**, **ADR-58**): a fase 0 — canais, cinco *uptime checks*, quatorze políticas por sintoma, SLOs, painel, exclusões e *dead letter*, tudo em Terraform — fecha o risco e **continua obrigatória antes de novembro de 2026**. Custo estimado por contrato: cerca de US$ 1 por mês. **Não implementado.**
R-06	**Sem revisão de segurança externa.** Nenhum teste de intrusão.	Alta	**Alto**	Item de contratação, não de engenharia. Para plataforma que trata dado de menor em rede pública, é exigência prática de edital.
R-07	**Acessibilidade não verificada.** Nenhuma checagem de contraste, foco ou leitor de tela.	Alta	Médio	Rede pública tem exigência legal e prática. O *design system* é o lugar certo para resolver de uma vez — o que torna isto um pedido ao `storybook-unooria`, não 48 correções.

ID	Risco	Prob	Imp	Mitigação e estado
R-08	**Partição de telemetria sem *job*.** As faixas vão até 2028-12 e não há rotina que crie novas.	Baixa	Médio	A partição `DEFAULT` é rede de segurança (evento não é recusado). Verificar a contagem dela é item do *checklist* de operação. Encher = faltou criar partição.
R-09	**Mídia pesada publicada à mão.** O CI publica `dist/`; os 52 MB de MP4 da exp-02 vão por `gsutil` manual.	Alta	Baixo	Aceitável para uma experiência; **não escala para 48**. O passo `gsutil rsync` de mídia precisa entrar no CI.
R-10	**Geração órfã sem retomada no *boot*.** Se todas as instâncias morrerem e ninguém abrir o painel, a geração fica parada.	Média	Baixo	Aceito (**ADR-17**): o *polling* do painel é justamente quando alguém se importa com aquela geração. Gatilho: primeira reclamação real.
R-11	**Dependência do *design system* externo.** Lacuna do pacote bloqueia a experiência até a próxima publicação.	Média	Baixo	Compensação local marcada no CSS, decrescente por política, com pedido correspondente. Cinco lacunas abertas, verificadas na 0.1.14.
R-12	**Catálogo de modelos de IA instável.** Dois modelos *default* já saíram do catálogo sem aviso.	Alta	Médio	*Provider* plugável (**ADR-07**), modelo em variável de ambiente, e `finishReason` tratado como erro. O sintoma da última vez foi silencioso: toda geração de imagem caindo no *fallback* procedural.
R-13	**WSO2 ainda não é o emissor.** A plataforma roda com o hub assinando.	Média	Baixo	**ADR-15** é transitória por desenho: trocar são duas variáveis. O risco real é o `iss` *tenant-qualified* do WSO2 ser montado por concatenação em vez de copiado do *discovery*. Ficou **mais barato** com o enturmamento: `users.fonte_id` existe para casar o `sub` do IdP com a pessoa, sem depender do `username`.
R-14	**Virada do ano letivo.** Em fevereiro de 2027 — três meses depois da entrada em produção — o 8A vira 9A em toda a rede.	Alta	**Alto**	**Mitigado** por **ADR-28**: com `periodos_letivos` e matrícula datada, o "8A de 2026" e o "8A de 2027" são turmas diferentes, e o acumulado do ano anterior continua apontando para a turma certa. Antes disso, `users.turma_id` era coluna: todo relatório histórico passaria a ser lido pela turma nova, **em silêncio**. O que **falta** é a operação da virada — criar o período de 2027 e importar o enturmamento novo — e ela é manual hoje.

ID	Risco	Prob	Imp	Mitigação e estado
R-15	**Senha compartilhada até o primeiro acesso.** Importação com uma senha para o lote inteiro deixa qualquer estudante entrar na conta de outro antes do primeiro acesso dele.	Média	Médio	Aceito e **dito na própria tela** (ADR-29), com o modo "uma senha por pessoa" ao lado, recomendado para turma grande. Fecha de vez quando o WSO2 for o emissor.
R-16	***Session replay* de terceiro sobre tela de menor.** O Microsoft Clarity grava a sessão no hub e dentro do `iframe` da missão (ADR-60); a documentação do próprio Clarity diz que ele **não deveria ser usado em aplicações destinadas a menores de 18 anos**, e a plataforma vai do 1º EF ao 3º EM.	Alta	**Alto**	**Aceito como decisão de produto**, não mitigado. O que existe hoje: o `identify` manda só o UUID opaco (nunca nome, nunca e-mail), o `track()` manda `verbo:objeto` sem a resposta do estudante, e o Clarity mascara por *default* campo de entrada, número e e-mail. O que **falta** é a ponta contratual: a gravação sai do projeto GCP do contrato para um processador de terceiro, e isso precisa estar no DPA da rede **antes** do primeiro contrato em produção. Desligar é apagar `CLARITY_PROJECT_ID` do *environment* — sem *deploy* de código, mas com *rebuild* da imagem.



* R-03 tem probabilidade *certa* — é consequência aceita, não eventualidade. Está posicionado em "média" apenas para caber na matriz.

A zona vermelha concentra o que **tem de ser resolvido antes de novembro de 2026**: carga verificada, retenção decidida e revisão de segurança contratada.

Figura 19.1 – Matriz de riscos. Três dos quatro riscos da zona crítica não são resolvidos por código: retenção é decisão de produto, segurança externa é contratação, e o fator humano único é estrutural.

19.2 Estado de implementação

Fase	Escopo	Estado	Observação

M0	Contratos congelados (*claims*, ponte, telemetria)	✓ Concluído	`packages/sdk`, `src/auth/claims.ts`
M1	*Tenancy*, RLS, papéis, importação de usuários	✓ Concluído	Teste com *role* sem *ownership*
M2	exp-01 completa: motor, geração, dossiê	✓ Concluído	Migração do aplicativo de origem
M3	exp-02, domínio por experiência	✓ Concluído	**ADR-09**
M4	Ferramentas de professor: exp-90, exp-91	✓ Concluído	Armazenamento privado, *guardrails*
M5	exp-70 e o Ato 1 integrado	✓ Concluído	`exp_01_exp_70_perfis_importados`
M6	Hub do estudante, *header* único, casca compartilhada	✓ Concluído	`/estudante`, `CascaPlataforma`
—	*Stack* `dev` em GCP operacional	✓ Concluído	CI/CD completo com WIF
—	**Teste de carga**	– Pendente	`R-01`
—	**Observabilidade (fases 0–2)**	– Pendente	`R-05`; estudo e roteiro no Capítulo 17
—	**WSO2 como emissor**	– Pendente	**ADR-15**
—	43 experiências restantes	– Em aberto	Gabarito estabelecido

19.3 Evolução planejada

19.3.1 Antes de novembro de 2026 (bloqueante)

1. **Teste de carga com k6**: 4.000 *logins* e aberturas de experiência em dez minutos, contra a *stack* `dev`. Verificar saturação de conexões, latência do `/runs/:id/feed` sob *lock*, e o caminho de ingestão de telemetria.
2. **Observabilidade, fase 0** (Capítulo 17): canais de notificação, *uptime checks* do JWKS e de `/readyz`, SLO de disponibilidade com *burn rate*, falha do Cloud Run Job de migração, saturação do Cloud SQL, *dead letter* da telemetria e *smoke* pós-*deploy* — tudo em Terraform e no `deploy.yml`, sem tocar em código de aplicação. A fase 1 (log estruturado com *redaction* e `ExceptionFilter`) entra logo depois, porque é o que torna o alerta diagnosticável.

3. **Decisão de retenção**, com a rotina que a cumpre. Se a resposta for “sem prazo”, ela precisa estar no contrato — não só neste documento.
4. **Revisão de segurança externa** e teste de intrusão.
5. **DPA por rede** contemplando a localidade de processamento de IA.
6. **WSO2 como emissor**, com o `iss` *tenant-qualified* copiado literalmente do *discovery*.

19.3.2 Depois, por ordem de retorno

Evolução	Por quê, e o que ela exige
Rsync de mídia no CI	`R-09`. Com 48 experiências, publicação manual de mídia deixa de ser aceitável.
Filtro por série vindo do manifesto	Hoje o hub do estudante lista missões de um mapa estático. Quando o manifesto virar a fonte, `series: []` tem de **continuar** querendo dizer “todas” — o contrário faz a experiência transversal desaparecer da tela sozinha.
Acessibilidade no *design system*	`R-07`. Resolver no pacote é resolver uma vez; resolver nas experiências é resolver 48 vezes.
Telemetria offline real (IndexedDB)	Hoje o *buffer* é em memória, e há um `TODO` explícito na SDK. Rede pública tem escola com conectividade ruim, e é lá que o dado se perde.
Métricas de aplicação	Hoje só métrica nativa de infraestrutura. Nada mede “quantos estudantes concluíram a missão hoje”.
Reprocessamento do mapeamento	`VERSAO_MAPEAMENTO` vai subir, e o *payload* está guardado justamente para isso (**ADR-13**). Falta a rotina que reprocessa e o relatório que compara duas gerações.
Visão agregada de bem-estar	Só com DPA e *k*-anonimato explícitos (**ADR-10**). **Não sai de um `GROUP BY`.**
Worker dedicado de telemetria	Se o volume justificar isolar o consumo (**ADR-21**).
BigQuery	Só se um relatório efetivamente precisar de análise em escala sobre `telemetria_eventos` (**ADR-14**).

19.4 O que continuará fora de escopo

Descartado	Condição para reavaliar
Kubernetes (GKE)	*Workload* de longa duração ou topologia de rede que o Cloud Run não expresse.
Microsserviço por experiência	Nenhuma. Contraria **ADR-01** na raiz.

Multi-região	Exigência contratual de continuidade regional.
Redis	Gargalo medido que o Pub/Sub e o `INSERT` em lote não resolvam.
Módulo genérico de avaliação	Três experiências consecutivas com mecânica idêntica.
Banco ou *schema* por experiência	Necessidade de fronteira de `GRANT` por experiência.

> **NOTA**
>
> A lista acima é tão importante quanto a de evolução. Num projeto com uma pessoa e um prazo, **o que se recusa a construir é o que permite terminar o resto** — e recusar sem registrar a condição de reavaliação transforma a recusa em teimosia.

20

Conclusão

20.1 O que foi construído

A Plataforma Unooria é um *monorepo* com dois serviços em execução — um hub Next.js e uma API NestJS — servindo um catálogo crescente de experiências imersivas entregues como *bundles* estáticos versionados em CDN. Em 27 de agosto de 2026, o estado é:

Dimensão	Estado
Serviços em execução	2 (hub, API) + 2 gerenciados (PostgreSQL, Pub/Sub)
Experiências publicadas	5 de 48, cobrindo os quatro casos estruturais: estudante com IA, estudante sem IA, jogo transversal e ferramenta de professor
Módulos da API	22, com um módulo de domínio por experiência
Tabelas	27 em `public` + partições mensais em `particoes`
Migrações	9, todas com RLS preservada
Arquivos de teste	34 (regra pura, integração, contrato, *smoke*)
Walkthroughs visuais	7
Decisões registradas	56 ADRs
Milestones concluídos	M0–M6

20.2 As decisões que mais definiram o resultado

Se este documento tivesse de ser reduzido a cinco frases, seriam estas.

1. **Experiência é artefato, não serviço** (ADR-01). É a decisão que torna 48 experiências viáveis para uma pessoa: o custo marginal de publicar a próxima é copiar arquivos, e a reversão de emergência é um `UPDATE` de uma coluna.
2. **Isolamento físico entre contratos, lógico entre escolas** (ADR-04, ADR-08). Uma *stack* por rede satisfaz o edital; RLS forçada com uma porta única de banco satisfaz a engenharia. E as duas trancas — política no banco e filtro na aplicação — são independentes de propósito.

3. **Regra que pode ser expressa em tipo é expressa em tipo** (ADR-12). `comTenancy` *lança* sem escola em vez de consultar sem filtro; a leitura de valores não *tem* campo numérico onde não há medida. Comentário avisa; tipo impede. É a diferença entre uma regra documentada e uma regra que vale.
4. **O que se recusa a construir é o que permite terminar** (Capítulo 19). Sem GKE, sem microsserviço por experiência, sem Redis, sem BigQuery, sem *worker* de telemetria, sem módulo genérico de avaliação — cada um com a condição de reavaliação escrita, para que a recusa não vire teimosia.
5. **Medir antes de otimizar, e medir antes de estimar** (ADR-14, ADR-16). O gargalo da telemetria não era o transporte, era um laço de `INSERT`. O armazenamento da exp-70 não era 2,4 GB, era 30 MB. O modelo de IA não existia na região que parecia óbvia. Nenhuma dessas três coisas era previsível — e todas as três mudaram uma decisão.

20.3 O que este projeto ensinou sobre documentação

Vale registrar, porque muda como este documento deve ser mantido.

> **NOTA**
>
> As decisões mais finas deste projeto não estavam em documento nenhum: estavam em **comentários no código**, ao lado da linha que elas explicam — por que o terceiro argumento do `set_config` é `true`, por que `exp:ready` é reenviado, por que a tabela de contrato carrega dois números, por que `cartoesServidos` não é "apareceram".
> Isso é um bom padrão e deve continuar. O que este documento acrescenta não é substituir aqueles comentários — é dar-lhes um *lugar*: a estrutura em que uma pessoa que nunca viu o repositório consegue entender por que ele é assim, e a matriz que liga cada decisão ao requisito que a exige e ao teste que a verifica.

> **ATENÇÃO**
>
> O risco desta documentação é o mesmo do `expires_at` que foi removido do *schema* (ADR-13): **documento que ninguém atualiza é pior que documento nenhum**, porque quem o lê acredita nele.
> Por isso a atualização deste documento é item explícito da *Definition of Done* (Capítulo 16) e está registrada em `AGENTS.md`, não apenas aqui. Uma implementação nova que mude arquitetura, contrato, modelo de dados, decisão ou risco **tem de** atualizar o capítulo correspondente e regerar o PDF no mesmo *commit*.

20.4 Estado de prontidão para produção

Dimensão	Estado	Falta

Funcionalidade	✓ Pronta	As 43 experiências restantes são *conteúdo*, não plataforma.
Isolamento e segurança de *tenancy*	✓ Pronta e verificada	Revisão externa (`R-06`).
Entrega e reversão	✓ Pronta	*Rsync* de mídia no CI (`R-09`).
Identidade	✓ Funcional	WSO2 como emissor (**ADR-15**).
Desempenho sob pico	**– Não verificado**	**Teste de carga** (`R-01`). É o item que separa "funciona" de "funciona às oito da manhã".
Operabilidade	– Parcial	**Observabilidade, fase 0** (`R-05`; Capítulo 17).
Conformidade	– Parcial	**Retenção decidida** (`R-02`) e **DPA com a localidade de IA** (`R-03`).
Acessibilidade	– Não verificada	Checagem no *design system* (`R-07`).

LIMITAÇÃO CONHECIDA

A plataforma está **funcionalmente pronta** e **operacionalmente incompleta**. As quatro lacunas em negrito acima são bloqueantes para novembro de 2026, e três delas não são resolvidas por código: retenção é decisão de produto, DPA é jurídico, revisão de segurança é contratação. Reconhecer isso explicitamente é mais útil que declarar prontidão.

20.5 Encerramento

O desenho descrito aqui é conservador por escolha. Não há tecnologia interessante nele: há PostgreSQL com RLS, contêineres sem orquestrador, arquivos estáticos numa CDN e uma fila onde a fila era inevitável. A parte difícil não foi escolher as peças — foi recusar as que não cabiam, e escrever por que.

O que este documento tenta preservar não é o *estado* do sistema, que o código descreve melhor. É o *porquê*: as vinte e uma decisões, as medições que as sustentam, as armadilhas que já custaram tempo e as consequências ruins que foram aceitas de olhos abertos. Um sistema pode ser lido a partir do código. Um projeto, não.

Apêndice A

Fontes Mermaid dos diagramas

Todo diagrama deste documento é vetorial, desenhado nativamente em TikZ — o que garante tipografia única, nitidez em qualquer *zoom* e ausência de dependência externa de renderização. Este apêndice traz a fonte **Mermaid equivalente** de cada figura, para quem precisa editar, revisar ou versionar o diagrama fora do LaTeX (num *pull request*, num Storybook, num quadro).

> **REGRA INVIOLÁVEL**
>
> **Ao acrescentar um diagrama TikZ, inclua aqui a fonte Mermaid equivalente**, referenciando o mesmo número de figura. Diagrama sem par Mermaid é diagrama que só uma pessoa consegue alterar.

> **NOTA**
>
> As fontes abaixo usam `C4Context`, `C4Container` e `C4Component` onde o diagrama é C4, `sequenceDiagram` onde é sequência, `erDiagram` onde é entidade-relacionamento, `stateDiagram-v2` onde é máquina de estados e `flowchart` nos demais. Não são idênticas ao TikZ: o Mermaid não expressa posicionamento fino nem notas ancoradas, e as notas longas dos diagramas TikZ aparecem aqui como `note` ou comentário.

A.1 Figura 1.1 — Visão geral da plataforma

```
C4Container
  title Visao geral da Plataforma Unooria

  Person(estudante, "Estudante", "1o EF ao 3o EM")

  Container_Boundary(plat, "Plataforma Unooria") {
    Container(hub, "Hub", "Next.js", "Autentica, roteia por papel, embeda")
    Container(exp, "Experiencia", "Vite estatico + CDN", "Roda no iframe do navegador")
    Container(api, "API", "NestJS", "Regra, gabarito, tenancy")
    ContainerDb(pg, "PostgreSQL", "Cloud SQL", "RLS por escola")
    ContainerQueue(ps, "Pub/Sub", "", "Telemetria analitica")
  }

  System_Ext(idp, "WSO2 Identity Server", "Identidade, futuro")
  System_Ext(vertex, "Vertex AI", "Gemini")
```

```
  Rel(estudante, hub, "HTTPS")
  BiRel(hub, exp, "postMessage v1", "token, contexto")
  Rel_L(hub, idp, "OIDC")
  Rel(hub, api, "BFF")
  Rel(exp, api, "SDK: /telemetry, /ai, dominio")
  BiRel(api, pg, "SQL + RLS")
  Rel(api, ps, "publica")
  Rel(ps, api, "push POST /telemetry/ingest")
  Rel(api, vertex, "quota por perfil")
```

A.2 Figura 4.1 — Casos de uso

```
flowchart LR
  estudante([Estudante]); prof([Professor]); gestor([Gestor escolar]); admin([Admin do contrato])
  idp[[WSO2 IS]]; vertex[[Vertex AI]]; pubsub[[Pub/Sub]]; ci[[CI/CD]]

  subgraph SIS["Plataforma Unooria"]
    UC01(("UC-01 Autenticar-se"))
    UC02(("UC-02 Escolher missao no hub"))
    UC03(("UC-03 Realizar experiencia"))
    UC04(("UC-04 Registrar telemetria"))
    UC05(("UC-05 Renovar token da sessao"))
    UC06(("UC-06 Ver o proprio resultado"))
    UC10(("UC-10 Preparar atividade"))
    UC11(("UC-11 Gerar conteudo com IA"))
    UC12(("UC-12 Conduzir o momento coletivo"))
    UC13(("UC-13 Ler resultados da turma"))
    UC14(("UC-14 Gerar plano de aula"))
    UC15(("UC-15 Check-in de bem-estar"))
    UC16(("UC-16 Zerar dados da turma"))
    UC20(("UC-20 Importar usuarios por CSV"))
    UC21(("UC-21 Redefinir senha"))
    UC22(("UC-22 Ler agregado da rede"))
    UC23(("UC-23 Publicar manifesto"))
    UC24(("UC-24 Reverter versao"))
  end

  estudante --- UC01 & UC02 & UC03 & UC06
  prof  --- UC01 & UC10 & UC12 & UC13 & UC14 & UC15 & UC16
  gestor --- UC20 & UC21
  admin --- UC22 & UC23

  UC03 -.->|include| UC04
  UC03 -.->|include| UC05
  UC10 -.->|include| UC11
  UC23 -.->|extend| UC24

  UC01 --- idp
  UC11 --- vertex
  UC14 --- vertex
  UC15 --- vertex
  UC04 --- pubsub
  UC23 --- ci
```

A.3 Figura 5.1 — C4 nível 1: contexto

```
C4Context
  title C4 nivel 1 - Contexto do sistema

  Person(estudante, "Estudante", "1o EF ao 3o EM")
  Person(prof, "Professor", "Conduz a aula")
  Person(gestor, "Gestor escolar", "Mantem usuarios")
  Person(admin, "Admin do contrato", "Rede")

  System(sis, "Plataforma Unooria", "Hub multi-tenant que autentica por papel, embeda experiencias imersivas versionadas, corrige e pontua no servidor, coleta telemetria e produz leitura de turma - com isolamento logico por escola")

  System_Ext(idp, "WSO2 Identity Server", "Identidade central, varios contratos")
  System_Ext(vertex, "Vertex AI - Gemini", "Texto e imagem generativos")
  System_Ext(ci, "GitHub Actions", "Publica bundles e manifestos")
  System_Ext(tf, "Terraform (repo separado)", "Provisiona a stack do contrato")

  Rel(estudante, sis, "realiza missoes")
  Rel(prof, sis, "prepara, conduz, le")
  Rel(gestor, sis, "administra")
  Rel(admin, sis, "le o agregado")
  Rel(idp, sis, "OIDC")
  BiRel(sis, vertex, "HTTPS + ADC")
  Rel(ci, sis, "publica")
  Rel(tf, sis, "provisiona")
```

A.4 Figura 5.2 — C4 nível 2: contêineres

```
C4Container
  title C4 nivel 2 - Conteineres (stack de um contrato)

  Person(estudante, "Estudante")
  Person(prof, "Professor")
  Person(adm, "Gestor / Admin")

  Container_Boundary(stack, "Plataforma Unooria - stack de um contrato") {
    Container(hub, "Hub", "Next.js App Router", "Sessao, roteamento por papel, chrome institucional, montagem do embed, BFF, JWKS e assinatura de token")
    Container(exp, "Experiencia", "Vite + React, ESTATICO", "Uma por missao. Roda no navegador; nunca e um processo do servidor")
    Component(sdk, "@edu/sdk", "TypeScript", "Ponte, telemetria, API, progresso")
    Container(api, "API", "NestJS + Prisma", "Autorizacao por papel, tenancy, regra de dominio, gabarito, correcao, gateway de IA, ingestao de telemetria")
    ContainerDb(pg, "PostgreSQL", "Cloud SQL HA + PITR", "Unico banco, unico schema; RLS por escola")
    ContainerQueue(ps, "Pub/Sub", "", "Canal analitico de telemetria")
    Container(buckets, "Object Storage", "Cloud Storage", "bundles, assets publicos, planos privados ")
  }

  System_Ext(vertex, "Vertex AI")
  Container_Ext(idp, "WSO2 IS", "", "Externo a stack")
```

```
  Rel(estudante, hub, "HTTPS")
  Rel(prof, hub, "HTTPS")
  Rel(adm, hub, "HTTPS")
  BiRel(hub, exp, "postMessage v1 - targetOrigin explicito")
  Rel(exp, sdk, "usa")
  Rel(sdk, api, "HTTPS + Bearer")
  Rel(hub, api, "BFF")
  BiRel(api, pg, "SQL + RLS")
  Rel(api, ps, "publica")
  Rel(ps, api, "push /telemetry/ingest")
  Rel(api, buckets, "assets, planos")
  Rel(api, vertex, "HTTPS + ADC")
  Rel(hub, idp, "OIDC")
  Rel(buckets, exp, "CDN serve o bundle")
```

A.5 Figura 5.3 — C4 nível 3: componentes da API

```
C4Component
  title C4 nivel 3 - Componentes da API (NestJS)

  Container_Boundary(api, "API") {
    %% --- transversal: roda em toda rota
    Component(guard,  "OidcGuard", "", "Valida JWT por JWKS")
    Component(roles,  "RolesGuard", "", "Restringe por papel")
    Component(tenint, "TenancyInterceptor", "", "Preenche escola/perfil no AsyncLocalStorage aberto pelo middleware")
    Component(env,    "validarEnv", "Zod", "Valida env no boot")
    Component(obs,    "observabilidade/", "pino + OpenTelemetry", "Contexto da requisicao ( middleware), logger com redaction, filtro global, sampler por rota; otel.ts e a primeira importacao do main.ts")

    %% --- infraestrutura: as portas unicas
    Component(prisma, "PrismaService", "Prisma", "comTenancy: transacao + RLS")
    Component(ai,     "ai/", "", "AiProvider, QuotaService")
    Component(store,  "object-storage/", "", "local | GCS")
    Component(assets, "assets/", "", "Imagem gerada")

    %% --- plataforma: serve a todas as experiencias
    Component(auth,   "auth/", "", "TokenService")
    Component(usu,    "usuarios/", "", "CSV, senha")
    Component(expmod, "experiencias/", "", "Manifesto")
    Component(tel,    "telemetry/", "", "batch, beacon, ingest")
    Component(prog,   "progress/", "", "Progresso generico")
    Component(health, "health/", "", "healthz, readyz")
    Component(analise,"analise/", "", "Turma e rede, CSV")
    Component(comum,  "comum/", "", "Series, rate limit")

    %% --- dominio por experiencia (nenhum conhece outro)
    Component(motor,  "motor/ geracao/ dossie/ atividades/ perfis/", "", "exp-01")
    Component(utube,  "unootube/", "", "exp-02")
    Component(vila,   "vila/", "", "exp-70")
    Component(planos, "planos/", "", "exp-90")
    Component(bem,    "bemestar/", "", "exp-91")
    Component(ugpt,   "unoogpt/", "", "exp-03 - motor probabilistico, agente com ferramentas, relogio da turma")
```

```
  }

  Rel(guard, roles, "antes de")
  Rel(tenint, prisma, "escola do token")
  Rel(motor, prisma, "comTenancy")
  Rel(vila, prisma, "comTenancy")
  Rel(planos, ai, "IA + quota")
  Rel(bem, ai, "IA + quota")
  Rel(planos, store, "bucket privado")
```

A.6 Figura 5.4 — C4 nível 3: componentes do hub

```
C4Component
  title C4 nivel 3 - Componentes do Hub (Next.js)

  Container_Boundary(hub, "Hub") {
    Component(mw,      "middleware.ts", "", "Sessao + area do papel")
    Component(papeis,  "lib/papeis.ts", "", "edge-safe: papel -> area")
    Component(authlib, "lib/auth.ts", "NextAuth", "Sessao")
    Component(jwks,    "/.well-known/jwks.json", "", "Chave publica")

    Component(embed,  "lib/embed-experiencia.ts", "", "Token + manifesto + origem, UMA vez")
    Component(chave,  "lib/chave-token.ts", "", "Assina RS256, TTL 15 min")
    Component(devsrv, "lib/experiencias-dev.ts", "", "Porta do dev server")
    Component(apiserv,"lib/api-servidor.ts", "", "BFF")

    Component(casca,  "CascaPlataforma", "", "CSS do DS + fontes")
    Component(header, "HeaderPlataforma", "", "Um so para todos os papeis")
    Component(disp,   "DispositivoFrame", "", "Casca de aparelho")
    Component(frame,  "ExpFrame", "", "Sandbox + protocolo v1")

    Component(landing, "components/landing", "", "Rota / sem sessao: landing publica (ADR-40)")
  }

  Rel(mw, papeis, "usa")
  Rel(landing, casca, "usa a casca do DS")
  Rel(embed, chave, "assina")
  Rel(embed, devsrv, "porta em dev")
  Rel(disp, frame, "envolve")
```

A.7 Figura 5.5 — C4 nível 4: tipos que carregam a regra

```
classDiagram
  class connectToHub {
    +hubOrigin: string
    +timeoutMs = 8000
    +READY_RETRY_MS = 400
    valida ev.origin, senao descarta
    reenvia exp:ready ate responder
    compara protocolVersion MAIOR
    valida pos-init com Zod
```

```
  }

  class SdkTypes {
    +PROTOCOL_VERSION = 2
    +HubInitSchema
    +HubParaExpSchema
    +ExpParaHubSchema
    unioes discriminadas por type
  }

  class PrismaService {
    +comTenancy(fn, escolaId?, opts?)
    abre $transaction
    set_config('app.escola_id', ..., true)
    sem escola: LANCA
  }

  class TenancyContext {
    +AsyncLocalStorage~string~
    +comEscolaNoContexto(id, fn)
    +escolaDoContexto(): string?
  }

  class LinhaDeLeitura {
    <<union>>
    scored:      numero: number
    indicative:  numero: null
    zero:        numero: null
    none:        numero: null
  }

  connectToHub ..> SdkTypes : usa
  PrismaService ..> TenancyContext : le
  note for LinhaDeLeitura "Comentario avisa; TIPO impede.\nO hub nunca recebe numero onde nao ha
↪ medida."
```

A.8 Figura 5.6 — Grafo de dependências do *monorepo*

```
flowchart TB
  ds[["@unooria/design-system (npm publico)"]]
  sdk["packages/sdk"]
  ui["packages/ui"]
  hub["apps/hub"]; api["apps/api"]
  e01["exp-01"]; e02["exp-02"]; e70["exp-70"]; e90["exp-90"]; e91["exp-91"]
  tpl["_template"]

  ds -->|versao publicada| ui
  sdk --> hub & api & e01 & e02 & e70 & e90 & e91 & tpl
  ui --> hub & e01 & e02 & e90 & e91
  %% exp-70 NAO depende de packages/ui: canvas 3D, sem DOM de UI (ADR-18)
```

A.9 Figura 5.7 — Curadoria de um bloco de *feed*

```
sequenceDiagram
  autonumber
  participant E as Experiencia (exp-01 no iframe)
  participant C as runs.controller (NestJS)
  participant S as motor/servico
  participant P as PostgreSQL

  E->>C: POST /runs/:id/eventos (dwell, curtidas)
  C->>S: aplicarEventos
  S->>P: comTenancy: set_config('app.escola_id')
  S->>P: pg_advisory_xact_lock(runId)
  Note over S,P: Sem o lock, dois pedidos simultaneos de bloco<br/>se sobrescrevem e um bloco inteiro desaparece.<br/>Transacao com timeout de 20s.
  P-->>S: estado da run
  S->>S: curarBloco (puro e deterministico)
  S->>P: UPDATE engine_state + posts entregues
  P-->>S: COMMIT (libera o lock)
  S-->>C: bloco + modulos + diagnostico
  C->>C: paraFeedPost: remove lado, perfil, intensidade
  Note over C: Esses campos sao a variavel independente<br/>do experimento. No cliente, o estudante saberia<br/>de que lado o algoritmo o colocou.
  C-->>E: GET /runs/:id/feed (sanitizado)
```

A.10 Figura 5.8 — Estados de uma geração assíncrona

```
stateDiagram-v2
  [*] --> pendente : POST (202)
  pendente --> gerando : claim atomico (updateMany por status)
  gerando --> pronto : gravou o artefato
  gerando --> erro : falha do provider OU finishReason=MAX_TOKENS
  gerando --> pendente : orfa (>12 min sem worker), ate 2 retomadas
  pronto --> [*]

  note right of pendente
    O claim atomico substitui uma fila externa:
    duas instancias podem tentar, so uma ganha a linha.
    Menor peca possivel que resolve o problema (ADR-17).
  end note
```

A.11 Figura 6.1 — Implantação de um contrato

```
C4Deployment
  title Implantacao de um contrato

  Deployment_Node(internet, "Internet", "escola") {
    Container(browser, "Navegador do estudante", "", "")
  }

```

```
  Deployment_Node(gcp, "Google Cloud - projeto do contrato", "GCP") {
    Deployment_Node(borda, "Borda global", "") {
      Container(lb,  "Cloud Load Balancing", "", "1 IP, host rules")
      Container(cdn, "Cloud CDN", "", "")
    }
    Deployment_Node(regiao, "Regiao de execucao (us-east1)", "") {
      Container(hub, "Cloud Run {contrato}-frontend", "hub", "min>=1")
      Container(api, "Cloud Run {contrato}-api", "", "CPU sempre alocada, conc. 80")
      Deployment_Node(vpc, "VPC - rede privada", "") {
        ContainerDb(sql, "Cloud SQL {contrato}-pg", "PostgreSQL HA, PITR, pooler", "")
      }
    }
    Deployment_Node(gcs, "Cloud Storage", "") {
      Container(bexp, "{contrato}-exp", "publico, imutavel, CDN", "")
      Container(bass, "{contrato}-assets", "publico, CDN", "")
      Container(bpla, "{contrato}-planos", "PRIVADO, versionado", "")
    }
    ContainerQueue(ps, "Pub/Sub {contrato}-telemetria", "", "")
  }

  System_Ext(idp, "WSO2 IS", "projeto proprio")
  System_Ext(vertex, "Vertex AI", "endpoint global")

  Rel(browser, lb, "443")
  Rel(lb, cdn, "bundles e assets")
  Rel(cdn, bexp, "serve")
  Rel(lb, hub, "HTTPS")
  Rel(lb, api, "HTTPS")
  Rel(api, sql, "5432 - IP PRIVADO")
  Rel(api, bpla, "so pela API")
  BiRel(api, ps, "publica / recebe push")
  Rel(hub, idp, "OIDC")
  Rel(api, vertex, "HTTPS + ADC")
```

A.12 Figura 6.2 — Implantação detalhada

```
flowchart TB
  browser([Navegador do estudante]); gh[[GitHub Actions]]; wso2[[WSO2 IS]]; vertex[[Vertex AI - global]]

  subgraph GCP["Google Cloud - projeto do contrato"]
    subgraph BORDA["Borda global"]
      lb["Load Balancer (1 IP, host rules)"]; cdn["Cloud CDN"]; cert["Certificados gerenciados (um por dominio)"]
    end
    subgraph EXEC["Regiao de execucao (us-east1)"]
      hub["Cloud Run -frontend (min>=1)"]
      api["Cloud Run -api (CPU sempre alocada, conc. 80)"]
      job["Cloud Run Job -migrate (na VPC)"]
      subgraph VPC["VPC - rede privada"]
        sql[("Cloud SQL -pg (HA, PITR, pooler)")]
        conn["Serverless VPC Access (conector)"]
      end
    end
    subgraph STORAGE["Cloud Storage"]
      bexp["{contrato}-exp - publico, imutavel, CDN"]
```

```
      bass["{contrato}-assets - publico, CDN"]
      bpla["{contrato}-planos - PRIVADO, sem CDN, versionado"]
    end
    subgraph MSG["Mensageria"]
      topic["topic -telemetria"]; sub["push sub -telemetria-ingest (oidc_token)"]
    end
    subgraph GOV["Identidade, segredos e artefatos"]
      sm["Secret Manager (6 segredos)"]; sa["Service Accounts (-api, -run, -push)"]
      wif["WIF pool github (sem chave JSON)"]; ar["Artifact Registry"]
      log["Cloud Logging / Monitoring"]
    end
  end

  browser -->|443| lb
  lb --> cdn --> bexp & bass
  lb --> hub & api
  hub -->|BFF| api
  api -->|5432 privado| conn --> sql
  job --> conn
  api -->|so pela API| bpla
  api --> bass
  api --> topic --> sub -->|POST /telemetry/ingest| api
  gh -->|OIDC do GitHub| wif
  gh -->|push da imagem| ar
  wso2 -->|OIDC| hub
  api -->|ADC| vertex
  sm --> api & hub
  sa --> api & hub
```

A.13 Figura 6.3 — Identidade de carga de trabalho

```
flowchart LR
  subgraph CI["CI: Workload Identity Federation"]
    gh["GitHub Actions (id-token: write)"] --> tok["token OIDC do GitHub (repo, ref, workflow)"]
    tok --> wif["WIF pool github (valida e troca)"]
    wif --> sadeploy["access token da DEPLOY_SA (curta duracao)"]
  end

  subgraph APP["Aplicacao: Application Default Credentials"]
    run["Cloud Run (API) - sem env de credencial"] --> md["metadata server (ADC nivel 3)"]
    md --> saapi["SA {contrato}-api (roles/aiplatform.user)"]
    saapi --> vx["Vertex AI - autorizado"]
  end

  %% A mesma cadeia nos tres ambientes:
  %%   1. GOOGLE_APPLICATION_CREDENTIALS
  %%   2. arquivo de `gcloud auth application-default login`
  %%   3. metadata server (Cloud Run)
  %% Antes havia SO o nivel 3 — e por isso nenhuma chamada de IA funcionava
  %% fora do Cloud Run. NUNCA chave de API.
```

A.14 Figura 7.1 — Os três níveis de isolamento

```
flowchart TB
  subgraph A["Contrato A - stack GCP"]
    apiA["API do contrato A"] --> pgA[("PostgreSQL do contrato A")]
    subgraph ESC["Dentro de UMA stack: isolamento LOGICO"]
      e1["escola sp-001"]; e2["escola sp-002"]; e3["... 100+"]
    end
    tab["Toda tabela de dado de estudante:<br/>escola_id TEXT NOT NULL<br/>ENABLE + FORCE ROW LEVEL SECURITY<br/>POLICY USING (escola_id = current_setting('app.escola_id', true))"]
    e1 -.-> tab; e2 -.-> tab; e3 -.-> tab
  end
  subgraph B["Contrato B - OUTRA stack"]
    apiB["API do contrato B"] --> pgB[("PostgreSQL do contrato B")]
  end
```

A.15 Figura 7.2 — Propagação da escola

```
flowchart LR
  req["Requisicao HTTP (Authorization: Bearer)"] --> guard["OidcGuard (JWKS; injeta req.auth)"]
  guard --> roles["RolesGuard (le req.auth.perfil)"]
  roles --> int["TenancyInterceptor (definirTenancyNoContexto)"]
  int --> ctrl["controller"] --> svc["service: comTenancy(fn)"]
  svc --> tx["Transacao: set_config + query"]
  svc -->|sem escola no contexto| fail["LANCA"]

  %% Guards rodam ANTES de interceptors, entao req.auth ja existe.
  %% Rota sem OidcGuard nao abre contexto — e comTenancy sem escola falha alto.
  %% A RLS e rede de seguranca, nao a unica tranca: os services continuam
  %% filtrando escola_id explicitamente em toda query.
```

A.16 Figura 8.1 — Modelo de dados da plataforma

```
erDiagram
  escolas ||--o{ turmas : tem
  escolas ||--o{ users  : tem
  turmas  ||--o{ users  : agrupa
  experiencias ||..o{ progresso : "exp_id"
  telemetria_eventos ||..o{ particoes : "ATTACH por mes"

  escolas {
    text id PK "MESMO valor do claim escola_id e do set_config"
    text nome
    timestamptz created_at
  }
  turmas {
    uuid id PK
    text escola_id FK
    text nome "UNIQUE (escola_id, nome)"
    text serie "1EF..9EF | 1EM..3EM"
```

```
  }
  users {
    uuid id PK
    text role "estudante|professor|gestor|admin-contrato"
    text escola_id FK "NULL SO p/ admin-contrato (CHECK)"
    uuid turma_id FK
    text username "UNIQUE (escola_id, username)"
    text display_name
    text password_hash
  }
  experiencias {
    text exp_id PK
    text nome
    text versao_ativa "rollback = UPDATE deste campo"
    text_array series "VAZIO = TODAS as series"
    text_array capabilities
    jsonb viewport "presente => casca de aparelho"
    text publico "estudante|professor"
    text_array permissoes "hoje so pointer-lock"
  }
  progresso {
    text escola_id PK
    text estudante_id PK
    text exp_id PK
    jsonb dados
  }
  ia_uso_diario {
    text escola_id PK
    text usuario_id PK
    text dia PK "YYYY-MM-DD"
    int tokens_usados
  }
  telemetria_eventos {
    uuid event_id PK "PK COMPOSTA: o Postgres exige a chave de particao"
    timestamptz ts PK
    text escola_id
    text estudante_id
    text exp_id
    text exp_version
    uuid sessao_id
    text verbo
    text objeto
    jsonb resultado
    jsonb contexto
  }
```

A.17 Figuras 8.3 e 8.4 — Modelo da exp-01

```
erDiagram
  exp_01_activities ||--o{ exp_01_authors : tem
  exp_01_authors    ||--o{ exp_01_posts   : escreve
  exp_01_posts      ||--o{ exp_01_post_comments : tem
  exp_01_activities ||--o{ exp_01_generation_jobs : enfileira
  exp_01_activities ||--o{ exp_01_runs : hospeda
  exp_01_runs ||--o{ exp_01_events : registra
  exp_01_runs ||--o{ exp_01_student_posts : publica
```

```
  exp_01_runs ||--|| exp_01_conclusoes : conclui
  exp_01_runs ||--|| exp_01_reflexoes : reflete
  exp_01_runs ||--|| exp_01_dossiers : gera
  exp_01_runs ||--o{ exp_01_comentarios_da_sessao : ancora
  exp_01_activities ||--o{ exp_01_exp_70_perfis_importados : "largada"

  exp_01_activities {
    uuid id PK
    text escola_id FK
    uuid professor_id FK
    text pergunta_chave
    text pergunta_origem "ia|professor - auditoria de autoria"
    text etapa "relogio COLETIVO da turma"
    boolean ia_ao_vivo "desligavel sem deploy"
    jsonb antagonismo
    jsonb engine_config
    text status
  }
  exp_01_posts {
    uuid id PK
    text lado "VARIAVEL INDEPENDENTE - sanitizada antes de sair"
    text perfil "idem"
    smallint intensidade "idem"
    text image_status "pendente|ok|fallback"
    boolean aprovado
  }
  exp_01_runs {
    uuid id PK
    text lado_inicial
    text lado_origem "perfil|sorteio|redistribuido - auditoria"
    jsonb engine_state "estado do motor; unidade de LOCK"
    text nome_no_app "a conta e do APARELHO, nao da crianca"
  }
  exp_01_events {
    bigint id PK
    text tipo "canal OPERACIONAL: sincrono e transacional"
    jsonb payload "{dwellMs, dwellNorm}"
  }
  exp_01_comentarios_da_sessao {
    uuid post_id FK "XOR com student_post_id (CHECK na migration)"
    uuid student_post_id FK
    boolean de_ia "para o professor distinguir na auditoria"
  }
  exp_01_exp_70_perfis_importados {
    jsonb escores "4 quadrantes, 0-1"
    text dominante
    text origem "jogo|professor"
    text lado_forcado "PRESERVADO no upsert"
  }
```

A.18 Figuras 8.5 e 8.6 — Demais experiências

```
erDiagram
  exp_70_sessoes ||--o{ exp_70_perfis : "recalculado pelo servidor"
  exp_70_perfis  ||..|| exp_70_bolhas : "mesmo estudante"
  exp_90_planos  ||..o{ exp_90_referencias : "monta o prompt"
```

```
  exp_91_ciclos  ||--o{ exp_91_checkins : "7 check-ins"

  exp_02_sessoes {
    text escola_id PK
    uuid estudante_id PK
    text_array salvos "CHECK cardinality <= 3"
    boolean ranking_desbloqueado "LATCH: nao volta"
    boolean credibilidade_adicionada "congela os salvos"
  }
  exp_70_sessoes {
    uuid id PK "EMITIDO pela API - o cliente colidia"
    uuid turma_id FK "desnormalizado: turma NO MOMENTO da partida"
    text bolha "A|B|C - do SERVIDOR"
    smallint cartoes_servidos "!= apareceram"
    timestamptz aberta_em "relogio do SERVIDOR"
    jsonb payload "bruto - unica fonte do reprocessamento"
    jsonb leitura_mensagens
  }
  exp_70_perfis {
    smallint versao_mapeamento "sobe => nasce linha nova"
    jsonb valores "null NAO e zero"
    jsonb graus "scored|indicative|zero|none"
    float abertura_vs_conservacao "NULL se polo nao observado"
    boolean amostra_suficiente "falso => o painel avisa que NAO SABE"
  }
  exp_70_bolhas {
    text escola_id PK
    uuid estudante_id PK "por ESTUDANTE, nao por partida"
    text bolha
  }
  exp_90_planos {
    text status "pendente|gerando|pronto|erro"
    text caminho_gerado_md "IMUTAVEL, bucket privado"
    text caminho_html "o que o editor salvou, sanitizado"
  }
  exp_90_referencias {
    text disciplina PK
    text serie PK "NULL = todas"
    text nome PK
    text texto "catalogo do CONTRATO - sem escola_id, FORA da RLS"
  }
  exp_91_ciclos {
    uuid id PK
    int_array ordem_itens "permutacao de 1..21"
    int depressao "somas x2, ao concluir - DADO SENSIVEL"
  }
  exp_91_checkins {
    smallint numero "1..7"
    text dia "UNIQUE (escola_id, professor_id, dia)"
    text texto_livre "NUNCA em log nem telemetria"
    text origem_texto "ia|fallback|seguranca"
    boolean sinal_de_apoio
  }
```

A.19 Figura 9.1 — *Handshake* hub-experiência

```
sequenceDiagram
  autonumber
  participant E as Experiencia (@edu/sdk no iframe)
  participant F as ExpFrame (React, no hub)
  participant R as Rota do hub (servidor)
  participant A as API (NestJS)

  R->>F: monta o embed: assina token RS256 (15 min)
  R->>A: GET /experiencias/:expId (manifesto)
  A-->>R: { versaoAtiva, viewport, permissoes }
  F->>E: renderiza iframe src = URL versionada
  E->>E: connectToHub({ hubOrigin })
  loop a cada 400ms, ate o hub responder
    E->>F: exp:ready { protocolVersion: 2 }
  end
  F->>F: valida ev.origin e a versao MAIOR
  F-->>E: hub:init { token, estudante, apiUrl, config }
  E->>E: valida com Zod; resolve a Promise
  E->>A: chamadas de dominio, /telemetry/batch, /ai/gerar
  E->>F: exp:token-expirando
  F->>R: GET /api/exp-token
  R-->>F: { token }
  F-->>E: hub:token { token }

  Note over E,F: Com assets em cache (F5), a experiencia fica pronta ANTES<br/>de o ExpFrame hidratar . Um disparo unico se perderia.<br/>hub:init repetido e evento normal, nao ruido.
```

A.20 Figura 9.2 — Resolução da URL do *embed*

```
flowchart TB
  ini([inicio]) --> q{tem linha em experiencias?}
  q -->|sim| com["{cdn}/{expId}/{versaoAtiva}/index.html?hubOrigin=..."]
  q -->|nao| sem["{baseDev}/?hubOrigin=... (porta de lib/experiencias-dev.ts<br/>ou o src explicito de /dev-embed)"]
  com --> vp1["viewport e permissoes vem do MANIFESTO"]
  sem --> vp2["le {baseInterna}/experiencia.json<br/>baseInterna = EXP_INTERNAL_HOST aplicado a base "]
  vp1 --> frame["ExpFrame: sandbox + allow,<br/>dentro de DispositivoFrame se houver viewport"]
  vp2 --> frame
  frame --> fim([fim])

  %% A leitura do experiencia.json NAO e so de dev: stack com a tabela vazia
  %% servia a exp-01 esticada na janela em vez de dentro do aparelho.
```

A.21 Figura 9.3 — Contrato entre duas experiências

```
flowchart LR
  v["exp-70 (Vila do Lume)"] -->|payload bruto| mv["src/vila/ - quadrantes.ts: colapso 10 -> 4"]
  mv -->|"POST /atividades/:id/perfis/da-vila (professor)"| t["exp_01_exp_70_perfis_importados"]
  t -->|le| mi["src/perfis/largada.ts - decide o lado do feed"]
  mi --> i["exp-01 (Instanooria)"]
```

```
%% Duas travas que CUSTAM estudantes, de proposito:
%%  1. valor nao observado (indicative|none) fica FORA da media — nunca zero
%%  2. quadrante sem nenhum membro observado INVALIDA o perfil inteiro
%% Sem perfil, o Instanooria cai no sorteio balanceado, que e honesto sobre
%% nao saber. E `lado_forcado` e PRESERVADO no upsert.
```

A.22 Figura 10.1 — Canal analítico

```
flowchart LR
  exp["Experiencia: track(verbo, objeto, dados)"] --> sdk["TelemetryClient<br/>buffer em memoria; lote de 50 ou 10s"]
  sdk -->|"<= 50 eventos"| batch["POST /telemetry/batch<br/>OidcGuard; identidade vem do TOKEN"]
  batch --> topic[["Pub/Sub topic -telemetria"]]
  sdk -->|sendBeacon no pagehide| beacon["POST /telemetry/beacon<br/>token NO CORPO; responde 204 SEMPRE"]
  beacon --> topic
  topic -->|push autenticado| ingest["POST /telemetry/ingest<br/>PubSubPushGuard: ID token do Google, aud, email"]
  ingest --> pg[("telemetria_eventos<br/>INSERT multi-linha<br/>ON CONFLICT (event_id, ts) DO NOTHING<br/>particionada por mes")]

  %% Onde ha perda possivel, e SO ali: entre o track() e o flush.
  %% Do Pub/Sub em diante a entrega e DURAVEL — e por isso um buffer em Redis
  %% nao o substituiria: reintroduziria a perda que ele eliminou.
```

A.23 Figura 11.1 — Camada de IA

```
C4Component
  title Camada de IA - src/ai e a UNICA porta

  Container_Boundary(api, "API") {
    Component(g, "geracao/", "", "exp-01")
    Component(m, "motor/", "", "exp-01")
    Component(p, "planos/", "", "exp-90")
    Component(b, "bemestar/", "", "exp-91")
    Component(a, "POST /ai/gerar", "", "SDK")

    Container_Boundary(ai, "src/ai/ - a UNICA porta") {
      Component(q,  "QuotaService", "", "temSaldo(escola, usuario, perfil) ANTES; consumir DEPOIS")
      Component(pr, "AiProvider", "", "generate, generateJson, generateImage")
      Component(pp, "prompts.ts", "", "prompt de sistema; NUNCA sai daqui")
      Component(vx, "vertex.provider.ts", "", "default; ADC + REST")
      Component(mk, "mock.provider.ts", "", "testes e dev sem GCP")
    }
    ComponentDb(ia, "ia_uso_diario", "", "quota persistida por (escola, usuario, dia)")
  }
  System_Ext(vertex, "Vertex AI", "endpoint global")

  Rel(g, q, "1")
  Rel(m, q, "1")
```

```
  Rel(p, q, "1")
  Rel(b, q, "1")
  Rel(a, q, "1")
  BiRel(q, ia, "incremento atomico")
  Rel(q, pr, "SO com saldo")
  Rel(pr, pp, "usa")
  Rel(pr, vx, "AI_PROVIDER=vertex")
  Rel(pr, mk, "AI_PROVIDER=mock")
  Rel(vx, vertex, "HTTPS + ADC")
```

A.24 Figura 11.2 — *Guardrails* do bem-estar

```
flowchart TB
  ini([inicio]) --> abre["GET /bemestar/hoje - 3 itens da permutacao do ciclo"]
  abre --> sinal{"sinal de apoio?<br/>item 17 ou 21 >= 2, ou termo de risco"}
  sinal -->|sim| fixo["texto FIXO de apoio (CVV 188, SAMU 192, CAPS/UBS)<br/>a IA NAO e chamada<br/>origem_texto = 'seguranca'"]
  sinal -->|nao| saldo{tem saldo e provider?}
  saldo -->|nao| fall["texto FIXO do catalogo<br/>origem_texto = 'fallback'"]
  saldo -->|sim| ia["IA gera abertura, pergunta aberta e acolhimento<br/>recebe SO: dia da semana, mes, no do check-in,<br/>texto livre delimitado"]
  ia --> val{validarTextoDaIa?}
  val -->|barrado| fall
  val -->|passou| ok["origem_texto = 'ia'"]
  fixo --> fim([fim]); fall --> fim; ok --> fim

  %% Os 21 itens do DASS-21 sao FIXOS e VERBATIM. A IA nunca reescreve um item:
  %% reformular quebra a validacao psicometrica e invalida as faixas de corte.
  %% O check-in NUNCA deixa de abrir.
```

A.25 Figura 12.1 — Os quatro graus de medida

```
classDiagram
  class scored {
    >= 2 sinais independentes
    numero: number
    entra na media
    E PUBLICADO com numero
  }
  class indicative {
    exatamente 1 sinal
    numero: null
    descrito em TEXTO
    fora dos eixos e das medias
  }
  class zero {
    teve a chance e NAO foi
    numero: null
    ENTRA na media, como zero de verdade
  }
  class none {
```

```
    a oportunidade NAO chegou
    numero: null
    SAI da conta - nao e zero
  }
  note "A distincao mais importante do modelo e entre zero e none.\nTratar 'nao teve chance' como '
↪ escolheu nao' faz uma crianca\nparecer ter recusado algo que nunca lhe foi oferecido."
```

A.26 Figura 13.1 — Mapa de rotas da API

```
flowchart TB
  subgraph PLAT["Plataforma - serve a todos os papeis"]
    au["/auth/login - sem papel"]; he["/healthz, /readyz - PUBLICO"]
    ex["/experiencias - GET qualquer; PUT admin-contrato"]
    pg["/progresso/:expId - estudante"]; te["/telemetry/* - ver capitulo 10"]
    us["/usuarios/* - gestor, admin"]; ai["/ai/gerar - autenticado + quota"]
    an["/analise/* - ADMIN-CONTRATO"]
  end
  subgraph E01["exp-01 - Instanooria"]
    atv["/atividades/* - professor"]; rn["/runs/* - estudante (/eventos: professor tb.)"]
    po["/posts/:id/comentarios - estudante"]; pf["/perfis/importar - chave de servico"]
  end
  subgraph OUT["Demais experiencias"]
    ut["/unootube/* - estudante"]; utt["/unootube/turmas/* - prof, gestor, admin"]
    vi["/vila/sessoes - estudante"]; vp["/vila/professor/* - PROFESSOR"]
    va["/vila/agregado/* - prof, gestor"]; pl["/planos/* - professor"]
    pref["/planos/referencias - chave de servico"]
    be["/bemestar/* - SO PROFESSOR, com posse"]
    ug["/unoogpt/* - estudante"]
    ugp["/unoogpt/professor/* - SO PROFESSOR, transcricao"]
    uga["/unoogpt/agregado/* - gestor, admin - sem estudanteId"]
  end

  %% Duas areas por AUSENCIA deliberada: bem-estar nao tem rota de gestor nem
  %% de agregado; e no unoogpt o controller do gestor nao tem NENHUMA rota que
  %% aceite estudanteId, o que torna estrutural (nao disciplinar) o limite do que
  %% ele alcanca.
```

A.27 Figura 14.1 — A linha da revelação

```
flowchart LR
  subgraph CLONE["Dentro do iframe, ANTES da revelacao - @edu/ui/ds-unootube"]
    feed["feed de videos"]; cromo["cromo, busca, menu"]
    play["playlists"]; canal["canal do estudante"]; toast["toast: 'Video salvo!'"]
  end
  subgraph PLATF["DEPOIS da revelacao, no MESMO iframe - @edu/ui/ds"]
    rank["ranking"]; d1["dialogo 1 da missao"]; d2["dialogo 2 da missao"]; fim["fechamento"]
  end
  CLONE ==>|REVELACAO| PLATF

  %% A linha e a REVELACAO, nao o arquivo. O toast NAO cruza: ele fala de
  %% dentro da ficcao. O criterio nao e "e depois no tempo?", e "quem fala?".
```

```
  %%
  %% CONTRAEXEMPLO, exp-03 (unooGPT): nao ha revelacao no software. O vies de
  %% cada fonte e o material do debate do professor, e nenhuma tela o entrega.
  %% Logo a exp-03 e 100% da secao do clone (@edu/ui/ds-chatgpt) e nunca troca
  %% para a da plataforma. A marca institucional entra so como chrome do HUB,
  %% desenhada ACIMA do iframe.
```

A.28 Figura 15.1 — *Pipeline* de entrega

```
flowchart TB
  push["push para main"] --> ver["PORTAO: ci.yml (typecheck, lint, test, build)"]
  push --> diff["job mudancas: git diff HEAD^ HEAD -> api | hub | exps"]

  subgraph api["job deploy-api (paralelo)"]
    img["imagem da api (camada de deps do pnpm fetch + cache GHA)"]
    img --> mig["Cloud Run Job -migrate (na VPC, --wait)"]
    mig --> dep["gcloud run deploy api (revisao nova)"]
  end

  subgraph hub["job deploy-hub (paralelo)"]
    imgh["imagem do hub"] --> deph["gcloud run deploy hub"]
  end

  subgraph exps["job publicar-exps — DEPOIS do hub e da api"]
    build["turbo build (filtrado, ou tudo se 'forcar')"]
    build --> rsync["gcloud storage rsync -> gs://{bucket}/{exp}/{ver}/"]
    rsync --> cc["Cache-Control: max-age=60 SO no index.html"]
    cc --> man["publicar-manifesto.mjs -> PUT /experiencias/:expId (token de 5 min)"]
  end

  ver --> api
  ver --> hub
  diff --> api
  diff --> hub
  dep --> exps
  deph --> exps

  %% ORDEM 1: migracao ANTES do deploy — o schema novo tem de estar no banco
  %%          antes de a revisao nova receber trafego. --wait faz falhar.
  %% ORDEM 2: experiencias DEPOIS do hub — o PUT do manifesto e validado contra
  %%          o JWKS que o HUB serve. Deploys PULADOS nao bloqueiam
  %%          (!cancelled() + result != 'failure').
```

A.29 Figura 16.1 — Estratégia de testes

```
flowchart TB
  v["PONTA A PONTA - Playwright, navegador real<br/>scripts/e2e por papel: publico, estudante,<br/>↪ professor, gestor, admin-contrato + probe-pointerlock"]
  c["CONTRATO EXTERNO<br/>telemetria-push (guard do Pub/Sub), vertex-provider (cadeia ADC)"]
  i["INTEGRACAO COM POSTGRES<br/>tenancy, planos-tenancy, motor-concorrencia, motor-postgres,<br/>↪ unootube-postgres, vila-postgres, vila-ato1, usuarios, planos, bemestar"]
```

```
  u["REGRA PURA - Vitest/Jest, milissegundos, sem banco<br/>vila-regras, unootube-regras, bemestar-regras, motor-pesos,<br/>motor-bolha, motor-sugestoes, motor-reversibilidade, bridge.test"]
  u --- i --- c --- v

  %% A base e larga de proposito: regras.ts puro em todo modulo de dominio.
  %% E o TOPO nao e "E2E de fluxo": e verificacao de que a INTERFACE apareceu.
  %% Icone virando texto, iframe vazio, cor inexistente, fonte removida pelo
  %% Next — tudo INVISIVEL para o compilador.
```

A.30 Figura 17.1 — Cadeia de uma requisição e o que cada elo revela

```
flowchart LR
  nav[Navegador - iframe da exp] --> lb[LB + CDN] --> hub[Cloud Run -frontend] --> api[Cloud Run - api]
  api --> sql[(Cloud SQL)]
  api <--> ps[Pub/Sub]
  api --> vx[Vertex AI]

  nav -.- v0["ve-se: nada<br/>cego: crash do bundle, ponte sem hub:init, token nao renovado, lote perdido"]
  lb  -.- v1["ve-se: contagem e latencia por host<br/>cego: quem foi afetado (sem correlacao)"]
  hub -.- v2["ve-se: log de requisicao, instancias, CPU/mem<br/>cego: null do apiDoServidor, redirect em laco, erro de render"]
  api -.- v3["ve-se: idem + /readyz (se alguem abrir)<br/>cego: causa do 500, latencia da IA, descarte de telemetria, fila em erro"]
  sql -.- v4["ve-se: conexoes, CPU, disco, backlog<br/>cego: query lenta por rota; dead letter (nao existe)"]

  %% Nenhum elo propaga identificador de correlacao.
```

A.31 Figura 17.2 — Erro de cliente entra pela API

```
sequenceDiagram
  participant E as Experiencia (@edu/sdk)
  participant H as Hub (ExpFrame)
  participant A as API (/observabilidade)
  participant L as Cloud Logging / Error Reporting

  E->>E: window.onerror / unhandledrejection
  Note over E: dedupe por hash, max. 5 por sessao, 8 KB
  E->>A: POST /observabilidade/erros-cliente (Bearer)
  Note over A: Zod: expId, versao, mensagem, stack, url
  A->>A: redaction + escola_id do token
  A->>L: stdout JSON, severity ERROR, @type ReportedErrorEvent
  L->>L: agrupa por assinatura; "novo na versao X"
  A-->>E: 204 (ou 429: cota da sessao)
  E-->>H: exp:erro (opcional, targetOrigin explicito)
  Note over H: mostra "algo deu errado" e oferece recarregar
```

A.32 Figura 19.1 — Matriz de riscos

```
quadrantChart
  title Matriz de riscos - probabilidade x impacto
  x-axis "Probabilidade baixa" --> "Probabilidade alta"
  y-axis "Impacto baixo" --> "Impacto alto"
  quadrant-1 "CRITICO - resolver antes de nov/2026"
  quadrant-2 "Impacto alto, prob. baixa"
  quadrant-3 "Monitorar"
  quadrant-4 "Prob. alta, impacto baixo"

  "R-01 carga nao verificada": [0.55, 0.92]
  "R-02 retencao indefinida":  [0.90, 0.90]
  "R-04 fator humano unico":   [0.98, 0.88]
  "R-06 sem revisao externa":  [0.85, 0.86]
  "R-05 sem alerting":         [0.85, 0.58]
  "R-07 acessibilidade":       [0.85, 0.55]
  "R-12 catalogo de modelos":  [0.82, 0.50]
  "R-03 localidade de IA":     [0.98, 0.55]
  "R-08 particao sem job":     [0.20, 0.50]
  "R-09 midia manual":         [0.85, 0.22]
  "R-10 orfa sem retomada":    [0.50, 0.20]
  "R-11 dependencia do DS":    [0.50, 0.22]
  "R-13 WSO2 nao e o emissor": [0.50, 0.18]
```

Apêndice B

Variáveis de ambiente

Catálogo completo, derivado de `.env.example` e de `apps/api/src/config/env.ts`. Toda variável da API é validada com Zod **no *boot***: valor errado derruba o processo na subida, não no meio de uma requisição de estudante.

REGRA INVIOLÁVEL

Variável marcada **fecha** significa: *ausente* ⇒ a funcionalidade responde 401 ou 503, **nunca** fica aberta. Essa é a convenção de *fail closed* da plataforma (**RNF-09**).

DECISÃO DE PROJETO

Identidade, enturmamento e importação não têm variável nenhuma (**ADR-28**, **ADR-29**). É afirmação, não omissão deste apêndice: o período letivo vigente é derivado das *datas* em `periodos_letivos` — sem coluna `ativo` e sem `ANO_LETIVO` —, a senha inicial do lote é escolhida na tela de importação e não fica em configuração, e o teto do CSV está no código (`main.ts` e `rostering.service.ts`). Configuração é lugar de endereço e segredo; regra de negócio em variável de ambiente é regra que ninguém encontra quando ela sai do esperado.
Houve por um dia uma `CPF_PEPPER`, quando o CPF era guardado em *hash*; ela saiu junto com a decisão (**ADR-33**).

B.1 API — `apps/api/.env`

B.1.1 Banco e identidade

Variável	Default	Papel
`DATABASE_URL`	— (obrigatória)	URL do PostgreSQL. Validada com `startsWith("postgresql://")`. Em produção vem do Secret Manager.

`OIDC_ISSUER`	— (obrigatória)	Emissor aceito. Hoje é o hub; amanhã o WSO2. **Não é segredo** — é URL pública, e vai em texto plano derivada de `hub_domain`, o que garante que `iss` e esperado não possam divergir.
`OIDC_JWKS_URI`	— (obrigatória)	JWKS do emissor.
`OIDC_AUDIENCE`	`plataforma-ap`	Audiência exigida no *token*.
`CORS_ORIGINS`	hub + *dev servers*	Lista explícita, separada por vírgula. **Nunca** `*`.
`PORT`	`3001`	Porta HTTP.

> **ATENÇÃO**
>
> O `issuer` do WSO2 é *tenant-qualified* (`.../t/{contrato}/oauth2/token`) e deve ser copiado **literalmente** do *discovery*, nunca montado por concatenação. Um `iss` montado à mão que difere por uma barra faz toda validação falhar com "*issuer* inválido" — e a mensagem não diz qual dos dois lados está errado.

B.1.2 Inteligência artificial

Variável	Default	Papel
`AI_PROVIDER`	`vertex`	`vertex` ou `mock`. Com `mock` a plataforma sobe sem nenhuma credencial de GCP.
`VERTEX_PROJECT_ID`	—	**Obrigatória com `AI_PROVIDER=vertex`**: o *boot* recusa sem ela. Sem projeto, a URL viraria `.../projects/undefined/...` e o erro só apareceria na primeira geração real.
`VERTEX_LOCATION`	`global`	O `3.6-flash` **só** existe em `global` (medido: 404 em `southamerica-east1`, `us-east1`, `us-central1`). Trocar por uma região exige trocar *também* o modelo.
`VERTEX_MODEL`	`gemini-3.6-fla`	Modelo do dia a dia.
`VERTEX_MODEL_LONGO`	`gemini-3.6-flas`	Modelo de saída longa (planos anuais). A variável separada existe para trocar um sem tocar no outro.
`VERTEX_IMAGE_MODEL`	`gemini-3.1-fla`	Arte da exp-01. O antigo `gemini-2.0-flash-exp` **não existe mais** — e por isso toda geração de imagem caía no *fallback* procedural. Quem trocar precisa conferir a proporção devolvida.

VERTEX_THINKING_BUDGET	0	Tokens de *raciocínio* saem do maxOutputTokens. 0 desligado (paridade com o aplicativo de origem); -1 dinâmico; > 0 teto. Com pensamento livre, o plano anual voltava **vazio**.
AI_DAILY_TOKEN_QUOTA_PER_I	300.000	Quota diária do estudante.
AI_DAILY_TOKEN_QUOTA_PER_PROFESSOR	2.000.000	Quota diária de professor, gestor e admin. Um plano anual gasta ~220.000 *tokens* só de *prompt*.

> **NOTA**
>
> **Não existe variável para chave de API de IA** em lugar nenhum do projeto — nem aqui, nem em .env.example. A credencial é sempre ADC (**ADR-16**).

B.1.3 Armazenamento

Variável	Default	Papel
ASSETS_DRIVER	local	local (a API serve em /assets) ou gcs (*bucket* público atrás da CDN).
ASSETS_BUCKET	—	{contrato}-assets.
ASSETS_PUBLIC_URL	—	Base pública da CDN de *assets*.
ASSETS_LOCAL_DIR	—	Diretório local em *dev*.
PLANOS_DRIVER	local	local grava em storage-privado/; gcs usa o *bucket* **privado**.
PLANOS_BUCKET	—	{contrato}-planos. **Privado**, sem CDN, com versionamento de objetos.
PLANOS_LOCAL_DIR	—	Diretório local em *dev*.

B.1.4 Telemetria e chaves de serviço

Variável	Ausente	Papel
TELEMETRIA_TOPIC	console	Vazio em *dev* = eventos apenas *logados*. Não há Pub/Sub local.
TELEMETRIA_PUSH_SA	**fecha (401)**	*E-mail* da conta de serviço que assina o *push* do Pub/Sub.
TELEMETRIA_PUSH_AUDIENCE	**fecha (401)**	Audiência configurada na *subscription*. Sem as duas, /telemetry/ingest rejeita tudo.
PERFIL_IMPORT_TOKEN	**fecha (503)**	Cabeçalho x-perfil-token da ponte externa do Ato 1.
PLANOS_IMPORT_TOKEN	**fecha (503)**	Cabeçalho x-import-token do *script* de importação de referências curriculares.

`PORTABILIDADE_EMISSOR`	degrada	De quem a api aceita conferir um perfil portátil. O `emissor.hub` vem de DENTRO do arquivo, ou seja, de um desconhecido: sem esta lista a api sairia buscando a URL que um arquivo mandou (SSRF). O documento **seleciona** entre estas origens; nunca introduz uma. Cada entrada é `origem` ou `origem=urlDoJwks` — a segunda porque o `emissor.hub` é o endereço do **navegador** (mesma distinção de `EXP_INTERNAL_HOST`): de dentro do contêiner, `localhost:3000` é a própria api. JWKS por `http` só fora de produção, com aviso no *log*. Vazia = nenhum documento é verificado e todos entram `nao-verificada` — não é fechar a rota, porque isso recusaria um direito do titular (**ADR-35**).
`SWAGGER_USUARIO`	**some (404)**	Usuário da documentação OpenAPI em `/docs`.
`SWAGGER_SENHA`	**some (404)**	Senha, mínimo 16 caracteres. **Sem as duas, `/docs` e `/docs-json` não são montadas** — quem responde é o 404 do roteador, não um 401. As rotas de serviço acima devolvem 401/503 porque a existência delas já é pública; aqui o ativo protegido É o mapa das rotas, e um 401 confirmaria que ele está atrás daquela porta. **Meia credencial derruba a subida**: uma porta que não abre com alguém achando que configurou a documentação é pior que nenhuma (**ADR-36**). Em *dev* os dois *compose* trazem um default (`docs` / `unooria-docs-local`) — sem ele o `/docs` só funcionaria para quem descobrisse que precisa exportar variável, isto é, para ninguém. Em produção vêm do Secret Manager.
`SEED_SENHA`	**o seed falha**	Senha das contas de demonstração criadas por `prisma/seed.ts` (mínimo 8 caracteres). **Não tem *default* no fonte**: o seed é idempotente e roda contra o `DATABASE_URL` que estiver no ambiente, e uma constante ali criaria 61 contas com senha conhecida — inclusive o `admin-contrato`, o único papel que atravessa escolas — se ele fosse apontado por engano ao banco de um contrato. O *default* de *dev* vive nos dois *compose*, como o do Swagger acima; no *host*, no `apps/api/.env` (o Prisma o carrega) ou na linha de comando. O seed não imprime mais a senha (**ADR-47**). Só *dev*: nenhum *pipeline* de entrega roda o seed.

B.1.5 Observabilidade (Capítulo 17)

Validadas pelo Zod de `config/env.ts` desde 2 de setembro de 2026 (fase 1 da API). Nenhuma é segredo: entram como *env* em texto plano no Cloud Run, injetadas pelo Terraform. O logger raiz lê `LOG_LEVEL`, `GCP_PROJECT_ID` e `NODE_ENV` **do ambiente cru**, antes do Nest — ele nasce na importação, e é usado pelo `main.ts` e pelo *bootstrap* do OpenTelemetry.

Variável	Default	Papel
`LOG_LEVEL`	`info`	Nível mínimo do `pino` (`debug` liga o log de *query* do Prisma). Em produção, `info`; nunca `debug` por mais de um incidente.
`GCP_PROJECT_ID`	— (vazio)	Identificador do projeto do contrato, para montar `logging.googleapis.com/trace` como `projects/{id}/traces/{traceId}`. Vazio = log sem correlação com o *trace* (dev). Injetada pelo Terraform; também lida pelo hub.
`OBS_TRACE_RATIO`	`0`	Fração de requisições com *spans* exportados ao Cloud Trace (`0.02` em produção; `0` desliga o exportador, que é o *default* de dev). Rotas de IA e o *feed* usam dez vezes este valor, até 1.
`OBS_DEBUG`	`0`	`1` imprime *spans* e métricas no console local, sem GCP. Só para desenvolvimento; nunca no Terraform.
`LOG_FORMATO`	— (vazio)	Fora de produção o log sai em `pino-pretty`; `json` força o formato de produção no terminal, para conferir uma linha como o Cloud Logging a verá.

B.2 Hub — `apps/hub/.env.local`

Variável	Escopo	Papel
`NEXT_PUBLIC_API_URL`	**cliente**	Endereço da API que o **navegador** usa. Vai ao `hub:init` como `apiUrl`.
`API_INTERNAL_URL`	servidor	Endereço da API que o *hub* usa (*authorize*, BFF, leitura de manifesto).
`NEXT_PUBLIC_CDN_EXP_URL`	**cliente**	Base dos *bundles*. Em produção, a CDN; em *dev* sem manifesto, cada experiência usa a porta de `lib/experiencias-dev.ts`.
`NEXT_PUBLIC_CLARITY_PR`	**cliente**	Id do projeto Microsoft Clarity (**ADR-60**). **Ausente = Clarity desligado**, no hub e dentro das missões — o hub não renderiza a *tag* e não manda o id no `config` do `hub:init`. É a única `NEXT_PUBLIC_*` que chega ao *bundle* do **cliente**, e por isso é inlinada no `next build`: entra por `--build-arg` no `Dockerfile`, não como env de *runtime* do Cloud Run. Trocar o id exige *rebuild* da imagem.
`EXP_INTERNAL_URL`	servidor	De onde o hub **lê** o `experiencia.json`. Endereço **fixo** — serve quando há *uma* origem de *bundles* (a CDN, em produção).
`EXP_INTERNAL_HOST`	servidor	Troca **só o host** e **preserva a porta**. É o que serve em *dev* e em contêiner, onde cada experiência tem um *dev server* numa porta diferente.

`HUB_ORIGIN`	servidor	*Origin* do hub, passada à experiência por *query string* e usada como `targetOrigin`.
`AUTH_SECRET`	servidor	Segredo de sessão do NextAuth.
`TOKEN_SIGNING_KEY_PEM`	servidor	Par RS256 que assina os *tokens* de experiência. Gerar com `node apps/hub/scripts/gerar-chave-token.mjs`. **Obrigatória em produção**; vem do Secret Manager.
`AUTH_WSO2_ISSUER`	servidor	A **presença** desta variável registra o *provider* OIDC no NextAuth.
`AUTH_WSO2_CLIENT_ID`	servidor	Cliente OIDC do hub.
`AUTH_WSO2_CLIENT_SECRE`	servidor	Segredo do cliente OIDC.
`CONTRATO_ID`	servidor	Identifica o EMISSOR dentro do perfil portátil. Ausente: a emissão responde 503 — documento cujo emissor não se identifica não vale nada na rede de destino, que precisa dele (e do `HUB_ORIGIN`) para achar o JWKS.
`CONTRATO_NOME`	servidor	Nome legível da rede, mostrado a quem receber o documento. Opcional.

ATENÇÃO

`EXP_INTERNAL_URL` *versus* `EXP_INTERNAL_HOST` é a confusão de configuração mais custosa do projeto, porque o sintoma não aponta para a causa: com um `EXP_INTERNAL_URL` fixo em *dev*, as seis experiências leem o manifesto de *uma só* — e o UnooTube, que não declara `viewport`, aparece **dentro da casca de celular da Instanooria**.

B.3 Experiências — variáveis de *build* do Vite

Variável	Papel
`VITE_VIDEOS_BASE_URL`	exp-02: base dos 21 arquivos MP4 (52 MB), que ficam fora do *git* e fora do *bundle*. Vazio = `./videos` (`public/videos/`, *gitignorado*, em *dev*). Em produção: `https://<cdn-assets>/exp-02-unootube/videos`.

NOTA

A exp-70 não tem nenhuma variável de ambiente. Ela não usa IA, não usa *bucket* e não tem prazo de retenção declarado — o que é registrado explicitamente em `.env.example`, para que a ausência seja lida como decisão e não como esquecimento.

B.4 `vars` do repositório (CI/CD)

O *workflow* de *deploy* não conhece o nome de nenhum recurso: tudo entra por `vars` do repositório, cujos valores saem de `terraform output`. Há um *script* que os configura (`scripts/configurar-github-`

	Conteúdo
`GCP_PROJECT`	Projeto do contrato.
`REGION`	Região do Cloud Run e do Artifact Registry.
`AR_REPO`	Repositório do Artifact Registry.
`API_SERVICE`, `HUB_SERVICE`	Nomes dos serviços do Cloud Run.
`MIGRATE_JOB`	Nome do Cloud Run Job de migração.
`EXP_BUCKET`	*Bucket* dos *bundles*.
`ASSETS_BASE_URL`	Base pública de *assets* (entra no *build* da exp-02).
`CLARITY_PROJECT_ID`	Id do projeto Microsoft Clarity, por *environment* (`dev`, `hml`, `prd`). O `deploy.yml` o passa como `--build-arg` ao construir a imagem do hub. **Não** sai de `terraform output` — é criada à mão nas *Variables* do *environment*, e ausente significa hub com o Clarity desligado.
`API_URL`, `HUB_ORIGIN`	Endereços públicos, usados pelo *script* de manifesto.
`WIF_PROVIDER`, `DEPLOY_SA`	Identidade federada do CI.

Secret do CI	Conteúdo
`TOKEN_SIGNING_KEY_PEM`	A mesma chave RS256 do hub (`{contrato}-token-signing-key` no Secret Manager). O CI a usa para assinar o *token* de cinco minutos do *upsert* de manifesto.

B.5 Configuração mínima para subir sem GCP

Listing B.1 – A combinação que faz **RNF-14** ser verdade

```
# API
DATABASE_URL=postgresql://postgres:pass@localhost:5432/unooria
OIDC_ISSUER=http://localhost:3000
OIDC_JWKS_URI=http://localhost:3000/.well-known/jwks.json
AI_PROVIDER=mock          # sem GCP nenhum; devolve texto de exemplo
ASSETS_DRIVER=local       # a API serve /assets
PLANOS_DRIVER=local       # grava em storage-privado/
TELEMETRIA_TOPIC=         # vazio: eventos so no console
# TELEMETRIA_PUSH_* vazias  => /telemetry/ingest responde 401 (correto em dev)

# HUB
NEXT_PUBLIC_API_URL=http://localhost:3001
```

```
API_INTERNAL_URL=http://localhost:3001
NEXT_PUBLIC_CDN_EXP_URL=http://localhost:5173
# NEXT_PUBLIC_CLARITY_PROJECT_ID=  # ausente: Clarity desligado (o normal em dev)
HUB_ORIGIN=http://localhost:3000
AUTH_SECRET=troque-me
TOKEN_SIGNING_KEY_PEM=    # node apps/hub/scripts/gerar-chave-token.mjs --env
```

Apêndice C

Catálogo de *endpoints*

Todas as rotas da API, com o papel autorizado. **Este apêndice é gerado** a partir do `@Contrato` de cada rota (**ADR-36**): a coluna de autorização vem do `@Roles` real, e a semântica, do resumo e das notas declarados no *controller*. Convenções transversais e justificativas estão no Capítulo 13.

Marca	Significado
`papel`	`@Roles(...)` restringe aos papéis listados.
(qualquer)	`OidcGuard` apenas: qualquer papel autenticado.
público	Sem *guard*, deliberadamente.
serviço	Chave em cabeçalho; **503** se a variável estiver ausente.
push	`PubSubPushGuard`; **401** se as variáveis estiverem ausentes.
token no corpo	O *token* vai no corpo, não no cabeçalho — `sendBeacon` não manda cabeçalho.

C.1 ai

Rota	Autorização	Semântica
`POST /ai/gerar`	*(qualquer)*	Conversa com o tutor de IA da experiência. Única porta de IA para o cliente. O prompt de sistema é escolhido no servidor pelo `expId` e **nunca sai daqui** — segredo em bundle de experiência é regra 2. **Restrições:** • Quota diária POR PERFIL (`AI_DAILY_TOKEN_QUOTA_PER_ESTUDANTE` / `_PROFESSOR`), persistida em `ia_uso_diario`: `temSaldo` ANTES, `consumir` depois. Sem saldo, **429**. • A série do estudante vai ao prompt a partir do TOKEN, não do corpo.

C.2 analise

Rota	Autorização	Semântica
`GET /analise/atividades/{id}`	`admin-contrato`	O detalhe de uma atividade, com a eficácia da escrita por perfil. `eficaciaDaEscrita` é **a única métrica do sistema capaz de refutar a premissa do produto**: o dwell dos posts de um perfil entre estudantes daquele perfil, menos o dwell dos MESMOS posts entre estudantes de outro — sempre com a exposição no denominador. Sem o denominador, mediria o próprio viés do feed (que serve mais posts do perfil do estudante) e "confirmaria" a premissa por construção. Se o delta for zero, as fichas de linguagem não funcionam — e a gente descobre com número em vez de achismo.
`GET /analise/csv`	`admin-contrato`	Export do nível de produto — por run ou por evento. Dois níveis porque as perguntas são diferentes: **por run** responde "como as turmas se comparam"; **por evento** é a matéria-prima para quem quer recalcular tudo fora daqui, inclusive contestando as definições. Um sistema de análise que não deixa auditar o próprio cálculo não deveria ser levado a sério. **Restrições:** • Teto de 200.000 eventos no nível `evento`.
`GET /analise/resumo`	`admin-contrato`	Uma linha por atividade — o nível de produto. Guarda de papel própria: `admin-contrato` não é professor nem estudante, e as guardas existentes recusariam os dois. **O professor NÃO alcança esta rota.**

C.3 atividades

Rota	Autorização	Semântica
`GET /atividades`	`professor`	As atividades DESTE professor. Posse por `professor_id` + `escola_id`: atividade é autoral.
`POST /atividades`	`professor`	Cria a atividade em rascunho, com os defaults do motor. **Restrições:** • `qtdPostsTese` precisa ser múltiplo de 8 (2 lados × 4 perfis) para a matriz fechar sem célula quebrada — **recusar é melhor que arredondar em silêncio**. É um `.refine()`, invisível no esquema.

Rota	Autorização	Semântica
`GET /atividades/{id}`	`professor`	O detalhe de uma atividade. `_count` espelha o `select` do Prisma da página de origem — o porte dos painéis consome exatamente esse shape —, e os agregados nomeados (`estudantes`, `postsAprovados`) ficam junto.
`DELETE /atividades/{id}`	`professor`	Exclui a atividade. Devolve o título e quantas sessões foram junto — "pronto" sem número deixa o professor sem saber se apagou a atividade certa.
`GET /atividades/{id}/analise`	`professor`	A análise pedagógica da turma — identificada. É a turma DELE, e o professor precisa saber com quem falar. Só o professor DONO da atividade lê. A peça central é `conclusaoPorLadoSemeado`: se o lado em que o feed nasceu prevê a conclusão que o estudante escreveu, o resultado experimental da turma está nessa grade — e é ela que o professor projeta no momento-espelho. Tudo o mais na tela existe para explicar essa grade. **Restrições:** • `imersao.total` nunca aparece sem os componentes — regra do doc, e vale para o relatório, o monitor e a análise. • `ociosoHaMs` é `null` quando a run acabou: ninguém precisa ser cutucado por estar "ocioso" num relatório. • `colecao` conta o ÚLTIMO evento por post (salvar e dessalvar se anulam); `interacoes.salvos` conta eventos.
`GET /atividades/{id}/analise/csv`	`professor`	A mesma análise da turma, em planilha. O nome do arquivo sai no `content-disposition`. O caminho **não** termina em `.csv` porque no app de origem segmento de rota com ponto quebrava a validação de tipos do App Router — mantido igual, porque é o path que as telas portadas chamam. **Restrições:** • BOM UTF-8 **e** `charset=utf-8`: um sem o outro ainda quebra acento em alguma planilha.
`PATCH /atividades/{id}/antagonismo`	`professor`	O professor aprova (ou reescreve) os dois lados e os neutros. Nada é gerado antes desta aprovação: os dois lados e as justificativas de eixo são o que decide de que lado cada estudante começa, e isso não sai da IA direto para a turma. **Restrições:** • `forcar: true` passa por cima do medidor de equilíbrio, de propósito e explicitamente.

Rota	Autorização	Semântica
`POST /atividades/{id}/ativar`	`professor`	Ativa a atividade para os estudantes entrarem. Devolve a MATRIZ 2 lados × 4 perfis: é ela que diz se a geração fechou. Célula vazia significa que um perfil de um lado não tem post — e o feed daquele estudante nasceria torto. **Restrições:** • `forcar: true` ativa com a matriz incompleta, de propósito — e a resposta devolve `forcado: true` para a tela dizer isso.
`POST /atividades/{id}/destilar`	`professor`	Destila a pergunta-chave e os dois lados a partir do recorte. O professor pode mandar a `perguntaChave` pronta — e aí `perguntaOrigem` sai como `professor`. Sem ela, a IA destila e a origem é `ia`.
`GET /atividades/{id}/distribuicao`	`professor`	Onde cada estudante vai cair — antes de começar. Existe para o professor **ver antes**. A distribuição de perfis varia por sala, e aí os lados nascem desequilibrados: não é defeito do sistema, é a turma — mas o professor precisa saber antes, não descobrir no meio da aula. **Restrições:** • `semLado` são os que caem no SORTEIO (perfil ausente ou `misto`): não entram na projeção porque o lado só é decidido quando a run nasce. • `precisaRedistribuir` é `desequilibrio > 0.15`.
`GET /atividades/{id}/estudantes/{runId}`	`professor`	Tudo o que UM estudante fez, durante a atividade. As interações vêm com os **salvos primeiro**: é a coleção que o estudante vai defender no debate. **Restrições:** • `doLadoSemeado` é a definição de COERÊNCIA: o post é do lado em que o feed dele nasceu? • `nomeNoApp` é apelido dentro da ficção do Instanooria — nunca identidade em tela de professor ou relatório.
`GET /atividades/{id}/etapa`	`professor`	Etapa atual e quantos estudantes em cada tela. É com isto que o painel decide se dá para avançar a turma.
`POST /atividades/{id}/etapa`	`professor`	O RELÓGIO DA AULA: move a turma inteira de momento. Escreve a etapa na atividade e leva as runs junto. `voltou: true` quando a turma RETROCEDE de etapa. **Restrições:** • `foraDoAparelho` conta quem ficou no onboarding: não entrou e não pode ser movido.

Rota	Autorização	Semântica
`GET /atividades/{id}/geracao/progresso`	`professor`	O progresso da geração — o alvo do polling da tela. As imagens são contadas em três estados porque `fallback` **não é erro**: a arte procedural entra no lugar e o feed nunca fica com buraco.
`POST /atividades/{id}/gerar`	`professor`	Enfileira a geração dos posts. **Duas respostas, e a diferença importa**: `retomado: true` quando havia uma geração parada no meio (nada novo foi enfileirado), e `lotes: N` quando ela começou agora.
`PATCH /atividades/{id}/ia`	`professor`	Liga ou desliga a IA ao vivo durante a aula.
`POST /atividades/{id}/limpar`	`professor`	Apaga as sessões dos estudantes, mantendo a atividade e os posts. É o botão de rodar a mesma atividade com outra turma.
`GET /atividades/{id}/monitor`	`professor`	O painel da turma ao vivo. **'ociosoHaMs' é o único campo que pede AÇÃO** — é o número que faz o professor levantar da cadeira. Tudo o mais é retrato. **Restrições:** • `ladoDominante: "indefinido"` sai como `neutro`: o painel não tem coluna para "ainda não dá para dizer". • A imersão nunca vem só com o total — os componentes vão junto, e essa é a regra em toda tela.
`GET /atividades/{id}/perfis`	`professor`	O estado do perfil de cada estudante, para conferência. **Restrições:** • `origem: "ausente"` = a criança ainda não tem perfil; ela cai no sorteio.
`POST /atividades/{id}/perfis`	`professor`	Importa perfis à mão — o fallback do professor. Aceita **duas formas**: escores completos (colados de planilha) ou só o `dominante` (seleção manual na lista). A segunda existe porque, na prática, o professor às vezes tem só o resultado final do jogo impresso. **Restrições:** • `username` que não existe nesta escola volta em `desconhecidos` — dito, nunca engolido.

Rota	Autorização	Semântica
`POST /atividades/{id}/perfis/da-vila`	`professor`	Traz os perfis da Vila do Lume para a largada do Instanooria. Colapsa o perfil Schwartz da exp-70 nos quatro quadrantes e grava com `origem: "jogo"` — é o que decide de que lado o feed nasce. O disparo é do PROFESSOR: importar sozinho desfaria em silêncio uma redistribuição feita antes da aula. **Restrições:** • O colapso tem duas travas que CUSTAM estudantes, de propósito: valor não observado (`indicative`/`none`) fica **fora da média**, nunca entra como zero; e quadrante sem nenhum membro observado **invalida o perfil inteiro**. • Sem perfil, o Instanooria cai no sorteio balanceado — que é honesto sobre não saber. • Exige `amostraSuficiente`: o `dominante` vira coluna no CSV do `/analise` ao lado da identidade da criança. • `ladoForcado` é PRESERVADO no upsert.
`GET /atividades/{id}/posts`	`professor`	A grade de revisão do professor, paginada e filtrável. A ordenação é `(created_at, id)`, e o `id` **não é enfeite**: os posts entram por `createMany` numa transação, então o lote inteiro compartilha o mesmo `created_at`. Com a chave empatada, `LIMIT/OFFSET` não tem ordem definida — a mesma linha vem em duas páginas e outra nunca vem (medido no app de origem: 207 de 208 visíveis, de forma reprodutível). **Restrições:** • A `matriz` 2×4 existe para o professor ver de relance se alguma linguagem ficou fraca: os lados podem estar 50/50 e um perfil inteiro estar vazio. • 24 posts por página.
`POST /atividades/{id}/redistribuir`	`professor`	Rebalanceia a turma quando ela cai quase toda num lado. Duas regras definem o comportamento: 1. **Move os de afinidade mais fraca primeiro** — `misto` antes de todos, depois por margem crescente. Preserva a pedagogia onde o sinal do perfil é forte. 2. **Só age em quem ainda não começou.** Trocar o lado de quem já está rolando o feed invalidaria a sessão: os scores acumulados passariam a contar para o lado errado. **Restrições:** • `bloqueados` conta quem já começou e por isso ficou — o rebalanceamento pode não ter chegado ao alvo, e o professor precisa saber. • Pergunta e lados não aprovados ⇒ **409**.

C.4 auth

Rota	Autorização	Semântica
`POST /auth/login`	**público**	Autentica uma credencial da rede. Consumida pelo `authorize()` do NextAuth no hub — **nunca pelo navegador**. A api é a única dona do banco; o hub não conhece hash nenhum. Devolve as escolas em que a pessoa tem lotação ativa: professor de rede estadual tem mais de uma, e o token carrega UMA. **Restrições:** • Conta com `ativo = false` é recusada com a MESMA mensagem genérica de senha errada — dizer "conta desativada" confirmaria que o login existe. • `username` é único por ESCOLA, não por contrato: a senha desambigua, e empate entre escolas é recusado. • 8 tentativas por minuto por usuário (429). Não há limite por IP: a api só enxerga o IP do hub.
`POST /auth/trocar-senha`	**público**	Primeiro acesso: troca a senha provisória. A senha do lote (ou a temporária de um reset) vale uma vez. Exige a senha ATUAL — é o mesmo nível de prova do login, e por isso a rota não pede token. Limpa `senha_provisoria` **ADR-29**. O hub refaz a sessão com a senha nova depois disto: o cookie antigo ainda diz "precisa trocar". **Restrições:** • A senha nova precisa ser diferente da atual (400). • 8 tentativas por minuto por usuário (429).

C.5 bemestar

Rota	Autorização	Semântica
`POST /bemestar/checkins/{id}/respostas`	`professor`	Responde as três frases do dia. No **7º** check-in a api soma por subescala (×2) e classifica pelas faixas publicadas — correção server-side (regra 3) —, e o ciclo fecha com `resultado` preenchido. **Restrições:** • **Sinal de apoio**: item 17 ou 21 ≥ 2, ou termo de risco no texto ⇒ mensagem FIXA (CVV 188, SAMU 192, CAPS/UBS) e **a IA não é chamada**. • Responder duas vezes no mesmo dia é **409** com `codigo: JA_RESPONDIDO`. • O texto livre nunca vai a log nem à telemetria — só contagens.
`DELETE /bemestar/dados`	`professor`	Apaga TUDO do bem-estar do professor logado. Direito de eliminação (LGPD). Apaga de verdade — ciclos e check-ins —, e a resposta diz quanto sumiu.
`GET /bemestar/historico`	`professor`	Os ciclos concluídos e as anotações — só do próprio professor. Posse por `professor_id` em toda query, com RLS por cima. **Não existe rota de gestor nem agregado** em lugar nenhum: DASS-21 é dado sensível de saúde (LGPD art. 11), e só o professor lê o dele.
`GET /bemestar/hoje`	`professor`	O check-in de hoje — o já respondido, o pendente, ou um novo. **3 frases por dia; 7 check-ins fecham os 21 itens** do DASS-21. Cria o pendente se ainda não houver, e nesse caminho chama a IA para o entorno. Os 21 itens são FIXOS e *verbatim*: a IA nunca reescreve um item — reformular quebra a validação psicométrica e as faixas de corte. O que ela gera é a abertura do dia, a pergunta aberta e o acolhimento. **Restrições:** • Todo texto de IA passa por `validarTextoDaIa` (sem vocabulário clínico, ordens, promessas, gestão, links). Barrado, sem saldo ou sem provider ⇒ texto fixo do catálogo: **o check-in nunca deixa de abrir**. • A IA nunca recebe nome, escola, pontuação ou faixa — só dia da semana, mês, nº do check-in e o texto livre, delimitado como conteúdo. • 1 por dia, em `America/Sao_Paulo`. O ciclo expira em 21 dias sem fechar.

Rota	Autorização	Semântica
`GET /bemestar/resumo`	`professor`	Estado leve do check-in, para o card do hub. **Não abre ciclo e não chama IA** — é o que o card do hub lê a cada carregamento, e um card não pode custar uma geração.

C.6 escolas

Rota	Autorização	Semântica
`GET /escolas`	`admin-contrato`	Todas as escolas do contrato, com contagem. Só `admin-contrato` chega aqui. O filtro `paiId` é o insumo da lotação em lote: ele resolve "as unidades desta diretoria" em LISTA, que é o que a tela mostra para o admin confirmar.
`POST /escolas`	`admin-contrato`	Cria uma escola. **'admin-contrato' e ninguém mais**, inclusive no `orgs.csv`: escola é *tenant*, e abrir um inquilino com RLS e dados de menores por trás não é operação de quem administra uma unidade. **Restrições:** • Id já existente é **400**. • `pai_id` liga unidade a regional, e um trigger no banco recusa pai que seja `escola` ou a própria linha.
`PATCH /escolas/{id}`	`admin-contrato`	Edita os dados da escola. `id` e `tipo` não se editam: os dois circulam em token e em RLS.
`POST /escolas/{id}/lotacoes`	`gestor`, `admin-contrato`	Lota alguém nesta escola. Para professor e gestor é **ACRÉSCIMO**: a lotação que a pessoa tem em outra escola da rede continua valendo — é o caso que a rede estadual tem aos milhares, e a razão de `lotacoes` existir. **Restrições:** • `admin-contrato` não recebe lotação: ele é a exceção cross-escola.
`GET /escolas/{id}/pessoas`	`gestor`, `admin-contrato`	Quem está lotado nesta escola. Gestor vê a própria; `admin-contrato`, qualquer uma. `papel` diferente de `estudante`/`gestor` cai em `professor`. **Restrições:** • O CPF vem FORMATADO e visível: o gestor confere identidade contra o sistema da rede e atende a família no balcão **ADR-33**.

Rota	Autorização	Semântica
`POST /escolas/lotacoes-em-lote`	`admin-contrato`	Lota uma pessoa em MUITAS escolas de uma vez. É assim que o supervisor de uma Diretoria de Ensino nasce: um `gestor` lotado nas unidades dela, **sem papel novo e sem tocar na RLS** — o token continua carregando UMA escola. Idempotente: reaplicar reabre o que estava encerrado e não duplica. **Restrições:** • Recebe a LISTA de escolas, nunca o critério ("todas da regional X"): autorizar por hierarquia seria uma segunda fonte de verdade sobre acesso, divergente da lotação no dia em que alguém corrigisse um pai. • `regional` e `rede` entram em `ignoradas`: não têm turma nem dado, e virariam escola fantasma no seletor do cabeçalho. • Uma transação POR escola — cinquenta numa só seguraria conexão por segundos, e falhar na décima não é motivo para desfazer as nove.
`GET /escolas/minhas`	`estudante`, `professor`, `gestor`, `admin-contrato`	As escolas de QUEM ESTÁ PEDINDO. Declarada antes de `:id` de propósito: no Nest a primeira rota que casa vence. É com ela que o hub monta o seletor de escola do professor de duas unidades — e **valida a troca** antes de assinar um token com a escola nova. A validação é sempre contra esta rota, nunca contra a lista que o cookie carrega: senão bastaria pedir uma escola qualquer. **Restrições:** • Barata de propósito: está no caminho do LOGIN. Contagens ficam em `/escolas/minhas/resumo`. • Professor e gestor ACUMULAM lotação; estudante tem UMA ativa, garantida por índice único parcial no banco.
`GET /escolas/minhas/resumo`	`professor`, `gestor`, `admin-contrato`	O panorama de quem tem muitas escolas. A tela do supervisor de diretoria. **LISTA, não soma**: uma linha por unidade, com turmas, estudantes e quantos ainda não fizeram o primeiro acesso. Somar escolas numa métrica só seria autorizar por hierarquia — exatamente o que a **ADR-34** recusa. O supervisor não é papel novo: é um `gestor` lotado nas unidades da diretoria.

C.7 estudantes

Rota	Autorização	Semântica
`GET /estudantes/{id}/historico`	`professor`, `gestor`, `admin-contrato`	O histórico do estudante ATRAVESSANDO escolas. A única leitura fora de `admin-contrato` que atravessa escolas — e por isso é uma porta só. Alcança apenas quem tem lotação na escola **onde o estudante está HOJE**: é a unidade responsável por ele. **Restrições:** • Devolve PARTICIPAÇÕES, **nunca o conteúdo dos instrumentos**: o retrato de valores da Vila e a transcrição do unooGPT continuam atrás das telas de quem tem a posse pedagógica na escola onde aquilo aconteceu. • A escola de HOJE é a PRIMEIRA da lista `escolas` — a tela imprime nessa ordem, e a primeira linha é a que o adulto lê como "a escola dela".
`GET /estudantes/{id}/perfil-portatil`	`gestor`, `admin-contrato`	Monta o documento SEM assinatura, para o hub assinar. A api monta; **o hub assina**, porque é ele que tem a chave e publica o JWKS. Os campos `emissor` e `emitidoEm` são preenchidos na assinatura, e por isso não vêm daqui. Transferir é dentro do contrato; o perfil portátil é para FORA dele — a criança que vai da rede estadual para a municipal sai da instalação, e não há garantia de que o destino comprou o produto **ADR-35**. **Restrições:** • Mesma tranca de `GET /estudantes/:id/historico`: quem pede precisa de lotação na escola onde a criança está HOJE. • **Sem CPF não há perfil portátil** (400): numa rede de destino nenhum id daqui existe, e o CPF é o que reconhece a criança. • Bem-estar (exp-91) **nunca** entra: documento que o gestor emite não carrega o que nem o gestor lê. • Sem `CONTRATO_ID` no hub, a emissão é **503**.
`GET /estudantes/{id}/perfis-recebidos`	`gestor`, `admin-contrato`	Os documentos que esta escola já recebeu sobre esta criança. Contagens, não conteúdo: a tela diz o que veio sem despejar o documento inteiro. **Restrições:** • O que vem de fora vive SÓ em `perfis_portateis`: não vira matrícula, progresso nem perfil da Vila, e não entra em média. É a afirmação de outra instalação.

Rota	Autorização	Semântica
`POST /estudantes/{id}/transferir`	`gestor`, `admin-contrato`	Transfere o estudante entre escolas da rede. A escola de DESTINO é a de quem pede: a unidade que recebe a criança que a família matriculou nela. `admin-contrato` pode mover para qualquer uma. **O dado que o estudante produziu NÃO se move** — continua com o `escola_id` de onde aconteceu, que é verdade histórica. O que segue o estudante é a identidade **ADR-32**. **Restrições:** • É a única operação que troca o `app.escola_id` no MEIO de uma transação: sem isso o update na origem não veria linha nenhuma (a RLS o esconderia, sem erro) e a criança ficaria nas duas. • `jaEstava: true` quando o destino já era a escola atual — nada foi movido.

C.8 experiencias

Rota	Autorização	Semântica
`GET /experiencias`	*(qualquer)*	O catálogo de experiências do contrato. Tabela GLOBAL, sem RLS: manifesto de versões não é dado de estudante. O hub monta a URL do iframe com `versaoAtiva`, e filtra por série na interface. **Restrições:** • `series: []` quer dizer TODAS as séries, não nenhuma — é como uma experiência transversal se declara (exp-70).
`GET /experiencias/{expId}`	*(qualquer)*	Uma linha do manifesto. Devolve `experiencia: null` quando não há linha — e isso não é erro: o hub cai no `experiencia.json` servido ao lado do bundle. É o caso de dev e o de uma stack nova, antes do primeiro publish do CI.

Rota	Autorização	Semântica
`PUT /experiencias/{expId}`	`admin-contrato`	Publica ou faz rollback de uma versão. Chamada pelo CI ao publicar `gs://{contrato}-exp/{expId}/{versao}/`, com o conteúdo do `public/experiencia.json` da experiência. **Rollback é trocar 'versaoAtiva' para a anterior** — não há deploy a desfazer, porque o bundle antigo continua no bucket. **Restrições:** • `permissoes` é POR EXPERIÊNCIA: prender o ponteiro do estudante é capacidade de quem declarou, nunca default do frame. • `viewport` ausente ⇒ o hub NÃO embeda dentro da casca de aparelho (a exp-02 foi desenhada para desktop).

C.9 healthz

Rota	Autorização	Semântica
`GET /healthz`	**público**	O processo está vivo. Não toca no banco de propósito: é a sonda que decide reiniciar o container, e um Postgres lento derrubaria um processo saudável.

C.10 lotacoes

Rota	Autorização	Semântica
`DELETE /lotacoes/{id}`	`gestor`, `admin-contrato`	Encerra o vínculo da pessoa com a escola. **Não apaga**: grava `fim`. É a lotação que diz que a pessoa esteve ali. Fecha junto as matrículas de turma daquela escola (sem escola, não há turma), e se era a principal, a próxima lotação ativa assume — senão `users.escola_id` apontaria para onde a pessoa não está mais. **Restrições:** • A checagem de escola vem DEPOIS de achar a lotação, porque é ela que diz de qual escola é — e um gestor não encerra vínculo de outra unidade (403).

C.11 matriculas

Rota	Autorização	Semântica
`PATCH /matriculas/{id}`	`gestor`, `admin-contrato`	Move de turma, ou torna esta a turma de referência. Mover é UM passo de propósito: à mão, o gestor tira da antiga e esquece de pôr na nova, e o estudante fica sem turma — invisível para o professor, sem nada na tela dizendo o que faltou. Nenhuma das duas operações apaga linha. **Restrições:** • Informe `turmaDestinoId` OU `principal`, nunca os dois nem nenhum — é um `.refine()`, que o JSON Schema não sabe expressar.
`DELETE /matriculas/{id}`	`gestor`, `admin-contrato`	Encerra a matrícula. **Não apaga linha**: grava `fim`. O dado da Vila, do UnooTube e do unooGPT continua pendurado nela, e apagar deixaria esse dado órfão de turma. Sair da REDE é outra coisa — `ativo = false` na pessoa.

C.12 perfis

Rota	Autorização	Semântica
`POST /perfis/importar`	**serviço**	Ponte externa do Ato 1 — compatibilidade. Chamada pelo JOGO, que roda fora do navegador e não tem sessão: autenticação por chave de serviço no cabeçalho, não por cookie. Continua viva como compatibilidade; hoje o caminho normal é `POST /atividades/:id/perfis/da-vila`, disparado pelo professor. **Restrições:** • `escolaId` vem no CORPO porque esta rota não tem token: sem ele não há escola no contexto, e `comTenancy` falha alto. O `activityId` é conferido DENTRO dessa escola, então um `escolaId` errado não alcança atividade de outra rede. • Sem `PERFIL_IMPORT_TOKEN` a rota responde **503** — desligada, nunca aberta.

C.13 perfis-recebidos

Rota	Autorização	Semântica
`POST /perfis-recebidos`	`gestor`, `admin-contrato`	Recebe um documento de outra rede. Receber é **ato da escola de destino** — ninguém empurra perfil para dentro de uma rede; alguém de lá importa o arquivo, olhando para ele. A api verifica porque verificar é o passo que **não pode confiar em quem chama**. **Restrições:** • **Assinatura inválida é RECUSA (400)** — isso é adulteração. Emissor inalcançável **não** é: entra marcado `nao-verificada`, e a tela sai em âmbar em vez de fingir que conferiu. • A classificação é pelo `code` da jose, NUNCA pelo texto: "signature verification failed" contém "failed", e uma versão anterior classificava documento FORJADO como "não deu para conferir". • `PORTABILIDADE_EMISSORES` é a decisão de confiança: o `emissor.hub` vem de dentro do arquivo, então o documento SELECIONA entre origens que o operador escreveu e nunca introduz uma (sem isso, SSRF). Vazia = tudo entra `nao-verificada`. • **Importar não cria estudante**: liga pelo CPF a quem já está aqui. Matricular é ato da escola.
`GET /perfis-recebidos/{id}`	`gestor`, `admin-contrato`	O documento recebido inteiro, como veio. Guardado *verbatim*: alterar qualquer byte destruiria a assinatura. É a **única exceção** à contenção do CPF em `users.cpf` — o número é o conteúdo do documento **ADR-33**.
`DELETE /perfis-recebidos/{id}`	`gestor`, `admin-contrato`	Apaga um perfil recebido. Existe porque o documento é de FORA: importado na criança errada, ou emitido por engano, ele tem de sair — e apagá-lo não apaga nada que esta rede tenha medido.

C.14 periodos-letivos

Rota	Autorização	Semântica
`GET /periodos-letivos`	`professor`, `gestor`, `admin-contrato`	O calendário do contrato, com quantas turmas cada período tem. Catálogo do CONTRATO: sem `escola_id` e sem RLS, por isso controller separado. A contagem de turmas é o que impede apagar um período por engano. **Restrições:** • `vigente` é DERIVADO das datas, nunca coluna — estado que ninguém atualiza mente mais do que ajuda.
`POST /periodos-letivos`	`gestor`, `admin-contrato`	Cria um período letivo — a primeira etapa da virada do ano. A partir da data de início ele passa a ser o vigente, e toda turma nova nasce nele. **A segunda etapa é importar o enturmamento**, e nenhuma das duas é automática: as turmas do ano anterior ficam onde estão, e é isso que preserva o histórico. **Restrições:** • `fim` antes de `inicio` é **400**.

C.15 planos

Rota	Autorização	Semântica
`GET /planos`	`professor`	Os planos DESTE professor. Posse por `professor_id` em toda query — plano de aula é autoral.
`POST /planos`	`professor`	Pede a geração de um plano de aula. **202**: a geração corre em background (claim atômico na linha), e o cliente acompanha por `GET /planos/:id` até o status virar `pronto` ou `erro`. **Restrições:** • 3 gerações por hora por professor: cada uma custa ~220k tokens só de prompt (currículo da rede + as 4 unidades do ano). Acima disso, **429** com `codigo: RATE_LIMIT`. • O teto de saída é 64k tokens, não 32k: com o material real o gemini-2.5-flash era cortado e o professor recebia “o plano ficou grande demais” sem plano nenhum. • Passa pelo `QuotaService` por PERFIL (`AI_DAILY_TOKEN_QUOTA_PER_PROFESSOR`).

Rota	Autorização	Semântica
`GET /planos/{id}`	`professor`	Um plano, com o conteúdo quando estiver pronto. É o alvo do polling que segue o 202 de `POST /planos`. **Restrições:** • `html: null` enquanto o status não for `pronto` — é assim que o cliente sabe que ainda deve continuar o polling.
`PATCH /planos/{id}`	`professor`	Renomeia o plano. **Restrições:** • Título entre 3 e 160 caracteres; fora disso, **400**.
`DELETE /planos/{id}`	`professor`	Exclui o plano.
`PUT /planos/{id}/conteudo`	`professor`	Salva o conteúdo editado pelo professor. Sobrescreve o `atual.html` no object storage PRIVADO. O `gerado.md` é imutável e continua lá — é o que `POST /planos/:id/restaurar` devolve. **Restrições:** • O HTML é sanitizado com allowlist antes de gravar. • Teto de 1 MB: um plano anual tem 30–150 KB. Acima disso, **400**.
`POST /planos/{id}/restaurar`	`professor`	Descarta a edição e volta ao plano gerado. O `gerado.md` é IMUTÁVEL de propósito: é o que torna esta rota possível. Sem ele, salvar por cima seria irreversível.
`GET /planos/referencias`	**serviço**	O inventário das referências curriculares do contrato. **Sem o texto**: existe para o script de importação ser idempotente e para auditoria — o texto de um currículo inteiro não cabe numa listagem. **Restrições:** • Catálogo GLOBAL do contrato: sem `escola_id`, fora da RLS.
`POST /planos/referencias`	**serviço**	Importa (ou sobrescreve) uma referência curricular. Chamada pelo `scripts/importar-referencias-plano.mjs`, que extrai o texto dos PDF/DOCX **localmente** e manda só texto. Upsert por `(disciplina, serie, nome)`. **Restrições:** • `serie: null` = vale para todas as séries daquela disciplina. • O upsert é find + update/create, e não `upsert`: `serie` nulo não entra em `where` de unique composto no Prisma, e o Postgres trata NULLs como distintos no índice. • Sem `PLANOS_IMPORT_TOKEN` a rota responde **503** — desligada, nunca aberta.

Rota	Autorização	Semântica
`DELETE /planos/referencias/{id}`	**serviço**	Remove uma referência do catálogo.
`GET /planos/referencias/disponiveis`	`professor`	Disciplinas e séries que têm material curricular importado. Alimenta os seletores da exp-90. Vem com os rótulos prontos: a tela imprime o que o servidor mandou, em vez de manter um segundo dicionário que divergiria na primeira disciplina nova.

C.16 posts

Rota	Autorização	Semântica
`PATCH /posts/{id}`	`professor`	Edita o post: legenda, hashtags, comentários e aprovação.
`DELETE /posts/{id}`	`professor`	Exclui um post da grade.
`GET /posts/{id}/comentarios`	*(qualquer)*	Os comentários de um post do feed. Na ordem de uma CONVERSA: o comentário do estudante no topo (como o Instagram faz com o seu), as reações da IA logo depois, e os do roteiro por último. **Restrições:** • `doEstudante` distingue o que a criança escreveu do que veio do roteiro — só o estudante da própria run vê os dele.
`POST /posts/{id}/regenerar`	`professor`	Regenera o texto ou a imagem de um post.

C.17 progresso

Rota	Autorização	Semântica
`GET /progresso/{expId}`	*(qualquer)*	O progresso do estudante autenticado nesta experiência. Escopado por `escola_id` + `estudante_id` do token — nunca por parâmetro. O estudante da requisição é o único progresso que esta rota alcança. **Restrições:** • `dados: null` = a criança ainda não começou; não é erro.

Rota	Autorização	Semântica
`POST /progresso/{expId}`	*(qualquer)*	Salva o progresso da experiência. Substitui `dados` inteiro. É a experiência que decide o que cabe ali.
`POST /progresso/{expId}/concluir`	*(qualquer)*	Registra a conclusão da experiência. O `exp:concluir` do protocolo da bridge, gravado pelo hub. **Merge** no jsonb: nunca sobrescreve o que a experiência salvou. **Restrições:** • Isto é progresso (“terminou”), NÃO nota. Pontuação que vale nota é recalculada server-side pelo módulo de avaliação (regra 3).

C.18 readyz

Rota	Autorização	Semântica
`GET /readyz`	**público**	As dependências estão prontas (banco). Banco fora do ar responde **503**, não `ok: false` — o balanceador tira a instância do rodízio pelo código, não pelo corpo.

C.19 rostering

Rota	Autorização	Semântica
`GET /rostering/contas-de-assento`	`gestor`, `admin-contrato`	As contas sem matrícula da rede — as que podem ser de assento. As experiências vieram de projetos onde a conta era do APARELHO: outra turma reusava o mesmo login. Esta lista mostra quanto cada conta acumulou de dado, para o gestor decidir o que desativar. **Restrições:** • **Não existe fundir contas**: juntar duas move o que uma criança fez para outra pessoa, e depois ninguém audita de quem era. • Teto de 200 linhas, dito na resposta em `truncado` em vez de cortado em silêncio.

Rota	Autorização	Semântica
`POST /rostering/importar`	`gestor`, `admin-contrato`	Importa a planilha de pessoas. Corpo é o **CSV cru** (`Content-Type: text/csv`), até 30MB. Uma linha por pessoa, cabeçalho lido por NOME (ordem livre): `matricula,nome,usuario,email,papel,turma,serie,turno`. Só `nome` e `papel` são obrigatórios — estudante também exige `turma` e `serie`. Idempotente: a chave é a MATRÍCULA da pessoa na rede e, na falta dela, o login. **Restrições:** • **O CPF é a primeira chave**: achado em outra escola da rede, a linha vira transferência (estudante) ou lotação a mais (professor). • Reimportar NÃO reseta a senha de quem já entrou — seria trancar a escola inteira para fora. • Coluna desconhecida é ERRO, não coluna ignorada. • 5 importações por minuto por usuário (429); acima de 30MB ou 50k linhas, 400 — o teto cabe debaixo dos 32 MiB do Cloud Run, para o 413 do ambiente nunca ser o que o gestor vê.
`GET /rostering/modelos/exemplo.csv`	`gestor`, `admin-contrato`	O mesmo modelo, preenchido com um exemplo.
`GET /rostering/modelos/oneroster/{arquivo}`	`gestor`, `admin-contrato`	O modelo em branco de um arquivo do OneRoster 1.2. Só o cabeçalho, na grafia do padrão. É um SUBCONJUNTO das colunas que a api sabe ler — as que ela ignora ficam de fora, porque modelo com 24 colunas não é modelo, é a especificação. **Restrições:** • Modelo e exemplo são arquivos SEPARADOS: linha de exemplo dentro do modelo vira pessoa fantasma importada no primeiro uso.
`GET /rostering/modelos/oneroster/{arquivo}/exemplo`	`gestor`, `admin-contrato`	O mesmo arquivo do OneRoster, preenchido com um exemplo. Gerado com a escola de VERDADE no `sourcedId` do `orgs.csv` e no `schoolSourcedId` das turmas: um id fictício faria o lote voltar como “escola não é a desta importação” para o gestor e, para o `admin-contrato` (que PODE criar escola por CSV), abriria um tenant de mentira.

Rota	Autorização	Semântica
`GET /rostering/modelos/usuarios.csv`	`gestor`, `admin-contrato`	O modelo em branco da planilha de pessoas. Gerado da MESMA constante que o leitor consome — um modelo escrito à mão divergiria do parser no primeiro campo novo, e o sintoma seria "importei o modelo de vocês e deu erro de cabeçalho". **Restrições:** • Sai com BOM e `charset=utf-8`: um sem o outro ainda quebra acento em alguma planilha.
`POST /rostering/oneroster/{arquivo}`	`gestor`, `admin-contrato`	Binding CSV do OneRoster 1.2 — UM arquivo por chamada. Ordem: `manifest`, `orgs`, `academicSessions`, `classes`, `users`, `enrollments`, `roles`. **Fora de ordem não corrompe nada**: a linha que aponta para um `sourcedId` inexistente volta no relatório dizendo QUAL arquivo falta. O padrão vive só nesta fronteira: nada do domínio conhece `sourcedId`. **Restrições:** • `orgs.csv` só APRENDE o código INEP desta escola: escola é *tenant*, não nasce de upload. Criar escola por CSV é só `admin-contrato`. • Sem a coluna `role` (que saiu de `users.csv` na 1.2) a pessoa entra como **estudante**, o menor privilégio; o `roles.csv` corrige. • A coluna `password` do padrão é IGNORADA, e a contagem vai ao relatório. • Em `bulk`, ausência de linha **não desativa ninguém** — é contada em `candidatosAInativacao`: lote que chegou pela metade não pode apagar uma rede por omissão. • `status = tobedeleted` desativa (pessoa) ou encerra (matrícula), nunca apaga.

C.20 runs

Rota	Autorização	Semântica
`POST /runs`	`estudante`	Cria a run do estudante na atividade ativa. A LARGADA: é aqui que o lado do feed é decidido — pelo perfil do Ato 1, por redistribuição do professor, ou por sorteio balanceado quando não há perfil.

Rota	Autorização	Semântica
`GET /runs/{id}/busca`	`estudante`	Busca no acervo da atividade. Usar a busca é o sinal de INVESTIGAÇÃO que o relatório conta — por isso ela é uma rota, e não um filtro no cliente.
`POST /runs/{id}/conclusao`	`estudante`	A escolha de um lado no fim da missão. Enviar duas vezes é **409**, com linguagem de app real: "isso já foi enviado", nada de "fase" ou "atividade".
`GET /runs/{id}/dossie`	`estudante`, `professor`	O dossiê da sessão — o relatório da revelação. Aqui o disfarce JÁ CAIU, e por isso o `lado` de cada post pode aparecer. A seção mais importante é `explicacaoDoLado`: **por que o feed começou daquele lado**. **Restrições:** • `conclusao` é anexada na LEITURA, não no congelamento: o dossiê é computado no `finish`, antes de o estudante escrever. `undefined` quando a sessão foi interrompida — e o relatório precisa funcionar assim. • Os rótulos de perfil vêm prontos no payload em vez de o cliente importar as FICHAS: elas carregam o vocabulário dos prompts, que não tem por que viajar até o navegador. • `imersao.total` nunca vem sem os componentes.
`POST /runs/{id}/entrar`	`estudante`	O estudante entra no aparelho e vai ao feed. `nome` é APELIDO dentro da ficção do Instanooria — nunca identidade em tela de professor ou relatório.
`POST /runs/{id}/eventos`	`estudante`	Lote de eventos do cliente — o canal OPERACIONAL. Síncrono e transacional, lido em tempo quase real para curar o próximo bloco. **'dwell' nunca passa por Pub/Sub**: latência e falta de ordenação quebrariam a adaptação e o lock por run. **Restrições:** • Idempotência por `(client_ts, tipo, postId)`: `descartados` conta o que já existia — reenvio é rotina no cliente, não erro. • Cota por SESSÃO, não por IP: a turma toda compartilha a rede da escola.
`GET /runs/{id}/fase`	`estudante`	A tela que o estudante deve estar vendo agora. Já reconciliada com a etapa da TURMA — o cliente faz polling porque os momentos coletivos são do professor, não do estudante.

Rota	Autorização	Semântica
`GET /runs/{id}/feed`	`estudante`	O próximo bloco do feed. O post sai **sem 'lado', sem 'perfil' e sem 'intensidade'** — o disfarce depende disso: um bundle que soubesse o lado de cada post entregaria o experimento antes da revelação. O `diagnostico` do motor (domínio, lado dominante, bloco) fica no SERVIDOR. **Restrições:** • Lock consultivo por run: duas abas pedindo bloco ao mesmo tempo se sobrescreviam, e um bloco se perdia.
`POST /runs/{id}/finish`	`estudante`	Congela a run e computa o dossiê. Encerrar antes da hora é **409** — e a mensagem é de app real ("sua sessão ainda está rolando"), não de sistema.
`GET /runs/{id}/perdidos`	`estudante`	O que o estudante PERDEU — posts do lado oposto que nunca foram servidos. **Só das etapas 'perdidos' em diante**: antes disso derrubaria o disfarce, que é o que a atividade inteira depende de manter até a revelação.
`POST /runs/{id}/reacoes`	`estudante`	Reações das personas ao que o estudante escreveu. Nunca falha para o estudante: sem IA, sem cota ou com erro, devolve `reacoes: []` e diz por quê. **Restrições:** • `desligada: true` = o professor desligou a IA ao vivo. • `limite: true` = teto de 12 reações por sessão — estudante comentando sem parar não queima a cota da turma.
`GET /runs/{id}/reflexao`	`estudante`	O texto da reflexão que o estudante já escreveu. Lê a reflexão desta run — a dele, e só a dele: o `where` casa `estudanteId` do token junto com o id. **Restrições:** • `texto: ""` quando ainda não escreveu — o campo abre vazio, não com erro.
`POST /runs/{id}/reflexao`	`estudante`	O fechamento escrito da atividade. É sobre a ATIVIDADE (letramento digital), não sobre a pergunta-chave — a conclusão já foi a escolha de um lado. **Restrições:** • 60 caracteres é o PISO: passa "o algoritmo me mostrou mais de um lado" e barra "gostei". Não é medida de qualidade, é filtro de vazio.

Rota	Autorização	Semântica
`POST /runs/{id}/reset`	`professor`	Reabre a run de um estudante, sorteando a largada de novo. Rota de PROFESSOR: é o socorro para a sessão que travou no meio da aula.
`GET /runs/{id}/salvos`	`estudante`	A coleção do estudante — o que ele salvou. O ÚLTIMO evento por post decide: salvar e dessalvar se anulam. É a coleção que ele vai defender no debate.
`POST /runs/{id}/student-post`	`estudante`	O estudante publica no app, e as personas reagem. **Falha fechada**: IA desligada, erro de rede ou reação recusada ⇒ lista vazia, e o post é publicado do mesmo jeito — como num app real quando ninguém responde. A chamada de IA fica FORA da transação, de propósito.
`GET /runs/boot`	`estudante`	O bootstrap do aparelho — o que a tela de entrada precisa saber. Só os RÓTULOS dos lados — os únicos campos do antagonismo que o cliente pode ver antes da revelação. Zero lado/intensidade dos posts.

C.21 telemetry

Rota	Autorização	Semântica
`POST /telemetry/batch`	*(qualquer)*	Recebe um lote do canal ANALÍTICO da SDK. Publica no Pub/Sub, que entrega em `POST /telemetry/ingest`. É o canal analítico — **não** o operacional: `dwell` e o feed adaptativo do motor passam por `/runs/:id/eventos`, que é síncrono e ordenado. **Restrições:** • `estudanteId` e `escolaId` NÃO vêm do corpo: são acrescentados a partir do token. Aceitá-los do cliente deixaria um bundle gravar evento no nome de outra criança. • `recebidos` conta o que foi PUBLICADO, não o que foi gravado — a gravação acontece do outro lado do Pub/Sub.
`POST /telemetry/beacon`	**token no corpo**	Último lote de uma sessão que está fechando. Chamada no `pagehide` com `navigator.sendBeacon`. **Responde 204 SEMPRE**: a página já foi descartada e ninguém lê o corpo — erro aqui só poluiria o log do cliente. Corpo inválido ou token que não confere viram descarte silencioso, com aviso no servidor.

Rota	Autorização	Semântica
`POST /telemetry/ingest`	**push**	Consome a push subscription do Pub/Sub e grava os eventos. Push, e não um worker próprio, é uma peça a menos para operar. Grava em `telemetria_eventos` (particionada por mês, schema `particoes`). **Restrições:** • `ON CONFLICT (event_id, ts) DO NOTHING`: o Pub/Sub entrega *at least once*, então reentrega é rotina, não exceção. • UM INSERT multi-linha por escola, não um por evento — no pico da manhã o laço custava ~800 INSERTs/s. • Sem `TELEMETRIA_PUSH_SA`/`TELEMETRIA_PUSH_AUDIENCE` a rota responde 401, nunca aberta: ela escreve dado de estudante.

C.22 turmas

Rota	Autorização	Semântica
`GET /turmas`	`gestor`, `admin-contrato`	As turmas da escola. Filtra por período letivo quando `periodoLetivoId` vem. Sem ele, TODAS as turmas de todos os anos — é isso que preserva o histórico depois da virada.
`POST /turmas`	`gestor`, `admin-contrato`	Cria uma turma. Sem `periodoLetivoId`, nasce no período VIGENTE — derivado das datas. Sem nenhum período cadastrado, **400**: turma sem período é a coluna `users.turma_id` de novo, com outro nome. **Restrições:** • `UNIQUE (escola_id, periodo_letivo_id, nome)`: o “8A de 2026” e o “8A de 2027” são turmas DIFERENTES.
`GET /turmas/{id}`	`professor`, `gestor`, `admin-contrato`	Os matriculados da turma — a lista nominal. Para o PROFESSOR a checagem é de MATRÍCULA, não de escola: o professor do 8A pedindo o 9B leva **403**. Antes disto, qualquer professor da escola lia qualquer turma dela — a única prova exigida era “mesma escola”, que é a regra de tenancy e não a de quem dá aula ali. **Restrições:** • `matriculaId` é da MATRÍCULA e `id` é da PESSOA — mover e encerrar operam sobre o primeiro. • `senhaProvisoria: true` = ainda não fez o primeiro acesso.

Rota	Autorização	Semântica
`PATCH /turmas/{id}`	`gestor`, `admin-contrato`	Edita nome, série ou turno da turma. Não move de período: isso seria reescrever o histórico do ano.
`POST /turmas/{id}/matriculas`	`gestor`, `admin-contrato`	Matricula uma pessoa na turma. `papel` só admite `estudante` e `professor`. Gestor é papel da ESCOLA, não matrícula de turma — e é por isso que ele continua vendo a escola inteira.
`GET /turmas/minhas`	`professor`, `gestor`, `admin-contrato`	As turmas DO PROFESSOR — as que ele tem matrícula, e só elas. Declarada antes de `:id` de propósito: no Nest a primeira rota que casa vence, e `/turmas/minhas` bateria em `/turmas/:id` com o id "minhas". Gestor e admin-contrato caem nas turmas da escola inteira — é a diferença entre administrar a escola e dar aula numa turma.

C.23 unoogpt

Rota	Autorização	Semântica
`GET /unoogpt/agregado/escola`	`gestor`, `admin-contrato`	O agregado da escola — uma linha por turma. Controller separado do professor de propósito. A alternativa — um `@Roles` misto com exceção por método — é como uma rota individual acaba aberta para gestor sem ninguém notar. **Nada aqui identifica uma criança.** **Restrições:** • `admin-contrato` precisa dizer `?escolaId=`; gestor só vê a própria escola (403).
`POST /unoogpt/alocacao`	`estudante`	Manda uma equipe para um bairro. `bairro: null` tira a equipe do destino. A justificativa é **portão de servidor**: só vale fonte que o estudante carregou, ou "foi intuição" — que está sempre disponível, porque sem essa saída a pergunta vira pegadinha e o estudante aprende a mentir para o formulário.
`PUT /unoogpt/conjuntos`	`estudante`	Carrega fontes no modelo pelo painel. `PUT` porque é atribuição de conjunto. As camadas ACUMULAM.

Rota	Autorização	Semântica
`POST /unoogpt/contrafactual`	`estudante`	"E se eu tivesse mandado para…" — calculado e descartado. **Não grava e não debita**: é a pergunta que o estudante faz ao VER o resultado, e transformá-la em tentativa nova apagaria a diferença entre a decisão que ele tomou e a que ele imaginou depois.
`POST /unoogpt/passo`	`estudante`	Move a sessão para outro passo da missão.
`POST /unoogpt/pergunta`	`estudante`	O turno do Cientista — a rota que fala com o modelo. Rate limit apertado (**12/min**) porque cada chamada custa tokens de verdade: o teto por sessão é o controle de custo, e este é o freio contra o laço. **Restrições:** • Falha do PROVEDOR **não** chega ao estudante como 500: vira `cientista-indisponivel` (**503**), que a tela traduz em "tente falar de novo". E **não debita** — o débito acontece só depois de resposta válida. • O log fica com o motivo técnico, **sem o texto do estudante**: é caixa livre escrita por menor de idade. • O número que um botão de opção mostra sai do MOTOR, nunca da LLM.
`GET /unoogpt/professor/estudantes/{estudanteId}`	`professor`, `admin-contrato`	A linha de UMA criança: leitura em graus e a Transcrição. **Professor only.** O relatório abre pelo que a missão MEDIU, em graus. **Restrições:** • Número só com `grau: "scored"`. E `zero` ("teve a chance e não foi") nunca se confunde com `none` ("a oportunidade não chegou") — é a distinção mais importante do modelo, e ela está no TIPO. • `resumo` nunca afirma o que o estudante É, e partida curta some com o resto: dois minutos de jogo não sustentam frase.
`GET /unoogpt/professor/turmas`	`professor`, `admin-contrato`	Turmas para o seletor de resultados do unooGPT. Filtradas por MATRÍCULA para o professor. **Gestor não entra aqui** — ele tem `/unoogpt/agregado/escola`.
`GET /unoogpt/professor/turmas/{turmaId}`	`professor`, `admin-contrato`	A folha da turma: relógio, retrato de conduta e uma linha por estudante. O retrato fala de **conduta, não de nota** — conta "decidiram no escuro" e "justificaram por intuição" em vez de somar acertos, porque a Missão não tem resposta certa.

Rota	Autorização	Semântica
`DELETE /unoogpt/professor/turmas/{turmaId}`	`professor`, `admin-contrato`	Zera a turma: apaga as partidas E o relógio. É o caminho de exclusão que substitui o `expires_at` do app de origem — prazo real, apertado por quem tem a posse pedagógica, em vez de coluna que nenhuma rotina cumpre. O relógio vai junto porque rodar de novo com a aula "já começada" faria a segunda rodada nascer no meio da tempestade.
`POST /unoogpt/professor/turmas/{turmaId}/relogio`	`professor`, `admin-contrato`	Dá a largada, soma tempo, encerra, troca o modo ou redefine. É o único lugar onde alguém que não é o estudante mexe no que ele vê — e mesmo aqui **não toca o Passo dele**: o que o professor controla é QUANDO dá para falar com o Cientista, não o que a criança decide.
`POST /unoogpt/reiniciar`	`estudante`	Recomeça — abre uma tentativa NOVA. **Não apaga a conversa anterior.** Cada tentativa fica no resultado com o desfecho e a faixa DELA: é essa comparação que ensina que a mesma cidade dá números diferentes.
`GET /unoogpt/relogio`	`estudante`	O relógio da turma — polling de ~5s. Leve de propósito: sem IA e sem mapa. `agora` é o instante do SERVIDOR, senão o relógio da máquina da escola (que atrasa) decide quanto tempo a turma tem. **Restrições:** • 240 chamadas por minuto por estudante.
`GET /unoogpt/resultado`	`estudante`	O resultado da tentativa, com a faixa e o que ele nunca consultou. A **faixa é a manchete**: corridas da MESMA decisão, para "não repete igual" sair do texto e virar gráfico. Uma rodada é anedota — o estudante lê 1.345 e conclui que tirou uma nota. **Restrições:** • `sobrou` é um FATO com nome de bairro e número de pessoas, **não um veredito**: quem transforma fato em lição é o professor, no debate. • `naoConsultados` traz só `comoFalar`/`rotuloCurto`/`serve` — o nome técnico, o viés e a cobertura são o instrumento, e ficam com o professor.

Rota	Autorização	Semântica
`GET /unoogpt/sessao`	`estudante`	O bootstrap da Missão Tempestade. Cria a sessão na primeira visita. Repare no que NÃO existe nestas rotas: **id de sessão**. No app de origem toda rota era `/api/missao/:id/*` com uma guarda `donoDaSessao` para provar a posse; aqui a identidade é `(escola, estudante)` vinda do TOKEN, e não há como pedir a partida de outra pessoa. A guarda desapareceu junto com a classe de bug que ela protegia. **Restrições:** • A `projecao` é SERVIDA, nunca calculada no navegador: um bundle capaz de computá-la deixaria o estudante prever o desfecho antes de enviar. • O mapa começa vazio: só o que uma Pergunta revelou chega ao navegador — `estimativa: null` é o cartão em branco, e **branco é informação**. • A `ficha` sai por allowlist, sem os números por bairro: eles são a verdade do mundo, e mandá-los daria de graça a resposta que a missão existe para o estudante não ter.
`POST /unoogpt/simular`	`estudante`	Enfrenta a tempestade e fecha a tentativa. Exige justificativa de **cada equipe com destino** — sem isso, `falta-justificar`. Devolve o mesmo que `GET /unoogpt/resultado`.

C.24 unootube

Rota	Autorização	Semântica
`POST /unootube/credibilidade`	`estudante`	Fecha a missão trocando inscritos por credibilidade. É a única rota que acrescenta `score`, e ele é `derivados.fontesConfiaveis` — a correção server-side. É o valor que a experiência devolve ao hub no `exp:concluir`.
`POST /unootube/curtidos/{videoId}`	`estudante`	Curte (ou descurte) um vídeo. **Restrições:** • Curtir não conta para nada na missão: o que vale é o que a criança SALVA.

Rota	Autorização	Semântica
`GET /unootube/ranking`	`estudante`	O ranking — a virada da missão. Os colegas são **fictícios**: ranquear colegas reais exporia pontuação de menores (regra 6). `voce` marca a linha do estudante. É aqui que a popularidade que o feed exibia se desmancha: o vídeo de números grandes era o falso.
`POST /unootube/reiniciar`	`estudante`	Começa a missão do zero.
`POST /unootube/salvos`	`estudante`	Salva um vídeo na playlist. A api corrige contra o `factual` e devolve inscritos e fontes recalculados no servidor (regra 3) — o cliente manda só qual vídeo salvou. **Restrições:** • A playlist tem 3 lugares. Cheia, a api recusa — é o diálogo de limite da experiência. • Vídeo fora do catálogo é **400** com `codigo: VIDEO_DESCONHECIDO`.
`POST /unootube/salvos/{videoId}/remover`	`estudante`	Tira um vídeo da playlist. `POST` em vez de `DELETE` porque a SDK só expõe get/post — a semântica idempotente é a mesma.
`GET /unootube/sessao`	`estudante`	O estado da missão e o catálogo de vídeos. O catálogo sai **sem 'factual'** — o tipo do servidor é `Omit<Video, "factual">`, então o gabarito é impossível de vazar aqui, não só ausente por descuido. **Restrições:** • As métricas de popularidade são COSMÉTICAS: nenhuma entra em conta nenhuma. O vídeo fake é o que tem os números grandes, e é esse contraste que o ranking desmonta. • `ranking: null` até ele ser desbloqueado — é a virada da missão.
`GET /unootube/turmas`	`professor`, `gestor`, `admin-contrato`	Turmas para o seletor de resultados. Filtradas por MATRÍCULA para o professor; o gestor vê a escola. `admin-contrato` precisa dizer `?escolaId=` — o token dele tem escola vazia e o TenancyInterceptor não abre contexto.
`GET /unootube/turmas/{turmaId}/resultados`	`professor`, `gestor`, `admin-contrato`	O que a turma fez na missão do UnooTube. Uma linha por estudante mais o resumo. `distribuicaoFontes` conta só entre os que CONCLUÍRAM — misturar quem parou no meio faria a média dizer outra coisa.

C.25 usuarios

Rota	Autorização	Semântica
`GET /usuarios`	`gestor`, `admin-contrato`	As pessoas da escola, paginadas. A tela `/admin/usuarios`. Busca por nome, login, matrícula **e CPF — com ou sem pontuação**. **Restrições:** • `situacao: sem-primeiro-acesso` lista quem ainda não trocou a senha da importação. • A consulta roda dentro de `comTenancy` porque inclui MATRÍCULAS, que têm RLS: fora do contexto a lista voltaria com todo mundo "sem turma", em silêncio.
`POST /usuarios`	`gestor`, `admin-contrato`	Cadastra UMA pessoa da escola (estudante, professor ou gestor). O formulário de `/admin/usuarios`. Monta a mesma linha da planilha e delega ao importador **ADR-59** — por isso as regras de CPF, série, turno e login são idênticas às da importação. **Restrições:** • **Criar é criar**: CPF, matrícula ou login já existentes devolvem 409 e nada é escrito — o caminho de quem já existe é `PATCH /usuarios/:id`. • CPF achado em OUTRA escola da rede devolve 409 sem id, sem nome e sem dizer qual escola: transferir é pela importação, que move a matrícula junto. • A senha sai UMA vez no corpo da resposta e é sempre provisória. Em branco, é sorteada. • `admin-contrato` NÃO entra aqui: ele não tem escola nem lotação, e tem rota própria. • Limite de 30 cadastros por minuto por usuário — é balcão, não carga.
`PATCH /usuarios/{id}`	`gestor`, `admin-contrato`	Edita nome, CPF, e-mail, papel ou situação da pessoa. Caminho de balcão: a planilha veio com o dígito trocado e a família traz o documento. **Restrições:** • `cpf: ""` **APAGA** o campo — é o que se faz quando o número era de outra pessoa. • Sair da REDE é `ativo: false`, não DELETE: há dado de estudante pendurado em oito tabelas.

Rota	Autorização	Semântica
`POST /usuarios/{id}/reset-senha`	`gestor`, `admin-contrato`	Gera uma senha temporária e a devolve UMA vez. Marca a conta como provisória: quem entrar com ela cai em `/trocar-senha` sem saída — trava tripla, no middleware, no BFF e no `/api/exp-token`. **Restrições:** • A senha **não fica em coluna nem em log**: se a tela perder, gera-se outra.
`POST /usuarios/admin-contrato`	`admin-contrato`	Cadastra outro admin do contrato (só admin do contrato pode). O único papel que NÃO nasce por importação: não pertence a escola nenhuma e enxerga a rede inteira. Conceder esse alcance é decisão do contrato, nunca de uma unidade — por isso gestor recebe 403 aqui. **Restrições:** • Nasce com `escola_id` nulo e nenhuma lotação — é o mesmo formato que o seed cria. • O login é único na REDE inteira, não na escola: o índice `(escola_id, username)` não cobre linha com `escola_id` nulo, e a autenticação procura o username na rede. • A senha sai UMA vez e é provisória: o primeiro acesso cai em `/trocar-senha`.
`POST /usuarios/importar-csv`	`gestor`, `admin-contrato`	Compatibilidade: delega para `POST /rostering/importar`. Continua respondendo porque scripts e documentação a citam. **Delega** — dois importadores lado a lado divergiriam na primeira regra nova.

C.26 vila

Rota	Autorização	Semântica
`GET /vila/agregado/turmas`	`professor`, `gestor`	Turmas para o seletor da Vila. Sem filtro por série — a Vila é transversal —, mas **com** filtro por matrícula: o professor vê as turmas dele, o gestor vê a escola.

Rota	Autorização	Semântica
`GET /vila/agregado/turmas/{turmaId}`	`professor`, `gestor`	O agregado da turma — contagens, nunca por criança. Abre para GESTOR também, e nenhum campo aqui identifica uma criança: porta a regra do app de origem (“pesquisador nunca vê a linha de uma criança; só agregados”). A distribuição de bolhas é o número que o professor precisa **antes** do debate: sem as três bolhas na sala, o momento em que a turma descobre que recebeu mensagens diferentes não acontece.
`GET /vila/professor/cartoes`	`professor`	TODOS os cartões, das três bolhas, COM o gabarito. É a revelação final da missão: o professor projeta as três versões lado a lado, marcando quais eram verdadeiras, e é a primeira vez que a turma vê as outras. Vem da api, e não de uma cópia no hub, porque **os textos SÃO o instrumento**: uma vírgula divergente entre o que o jogo serviu e o que o professor projeta põe a sala discutindo uma mensagem que ninguém recebeu. **Restrições:** • **COM ‘turmaId‘, devolve só as bolhas que AQUELA sala recebeu.** Sem isso, o guia de uma turma de 3º ano projetava o golpe da bolha B — que a api nunca serviu ali — e mandava o professor esperar a reação de um grupo que não existe na sala. • `bolhasDaTurma: null` = nenhuma turma escolhida; a tela mostra as três e avisa.
`GET /vila/professor/estudantes/{estudanteId}`	`professor`	O relatório da última partida concluída de um estudante. **Professor only** — o gestor tem o agregado, nunca a linha de uma criança. São controllers separados de propósito: um `@Roles` misto com exceção por método é como uma rota individual acaba aberta para gestor sem ninguém notar. A seção das mensagens abre pelo **NÍVEL** que ela recebeu, antes do que ela fez: “caiu em 2 de 3” sem dizer que era o nível C faz o adulto concluir que a criança é ingênua — o rótulo que este projeto existe para não produzir. **Restrições:** • `relatorio: null` é resposta legítima: o estudante existe e não jogou. Um 404 faria a tela dizer “não encontrado” para uma criança da turma dele. • `grau` é invariante de API: só a variante `scored` tem número; as outras têm `null` POR TIPO, então o hub não consegue imprimir “Poder 100” a partir de um sinal só. • `mensagens: null` = nenhum cartão chegou a aparecer — **não** é “ela ignorou tudo”.

Rota	Autorização	Semântica
`GET /vila/professor/turmas/{turmaId}`	`professor`	Uma linha por estudante da turma. Quantas partidas, a bolha, e se a última sustenta leitura. **Sem valor de Schwartz nenhum**: listagem é ordenável, e uma coluna “Poder” viraria ranking de crianças. Os valores vivem na tela individual, uma criança por vez, com o grau ao lado.
`DELETE /vila/professor/turmas/{turmaId}/dados`	`professor`	Zera os dados da Vila desta turma. `DELETE`, e não um `POST /limpar`: é remoção, e o verbo diz isso a quem lê o log. Devolve as contagens do que sumiu — “pronto” sem número deixa o professor sem saber se apagou a turma certa. **Restrições:** • As **bolhas** somem junto: rodar de novo com a mesma bolha serviria à criança o mesmo golpe — ela reconhece, e a segunda rodada não mede nada. • `perfis_importados` **não** é tocado: aquilo pertence à atividade do Instanooria, e apagar de lá trocaria os lados de uma aula em andamento.
`POST /vila/sessoes`	`estudante`	Abre a partida e entrega os cartões DESTA criança. **A bolha é do SERVIDOR.** Atribuída A/B/C de forma balanceada dentro da TURMA e persistida em `vila_bolhas` (estável entre partidas: a folha do professor diz “esse estava na C”). Era `estudanteSlot % 3` no navegador, o que punha no bundle a condição do próprio instrumento e deixava as três versões legíveis antes da aula. **Restrições:** • Os cartões vêm **sem gabarito** e só os desta bolha: a explicação final da missão depende de ser a primeira vez que a turma vê as outras versões. • Nos anos iniciais (1EF–5EF) a bolha B **não é servida** — “suporte” exige entender o que é suporte, e só a api sabe a série. • Há lock consultivo por turma: a sala clica junto, e sem ele os 25 caíam todos na bolha A. • Identidade sempre do token: o payload não traz estudante nem turma.

Rota	Autorização	Semântica
`POST /vila/sessoes/{id}/concluir`	`estudante`	Fecha a partida e grava o payload bruto. **Idempotente**: a criança pode recarregar, e o `pagehide` pode disparar duas vezes — daí `status: "jaEstava"`. O servidor RECALCULA o retrato; nunca aceita o perfil do cliente. É isso que permite reprocessar o histórico quando `VERSAO_MAPEAMENTO` subir. **Restrições:** • `CHAVES_DE_PERFIL` recusa payload que traga valores/graus/perfil calculados: o jogo sabe computar o retrato e deliberadamente não o envia. • `eventos[].dados` só aceita número, booleano ou string de um **vocabulário fechado** — o `texto` livre era o caminho por onde o nome de uma criança chegaria a um JSONB. • O gabarito da missão fica no servidor: o cliente manda qual opção escolheu, a api corrige (regra 3).

Apêndice D

Payloads e esquemas de validação

Exemplos reais dos *payloads* que atravessam as fronteiras da plataforma, com os esquemas Zod que os validam. Todo *input* externo é validado; nenhum esquema aqui é decorativo.

D.1 Protocolo `postMessage` (v1)

Listing D.1 – `packages/sdk/src/types.ts` — os esquemas do protocolo

```ts
export const PROTOCOL_VERSION = 2;

export const EstudanteSchema = z.object({
  id: z.string(), nome: z.string(),
  serie: z.string(),          // "6EF", "2EM"
  turmaId: z.string(), escolaId: z.string(),
});

export const HubInitSchema = z.object({
  type: z.literal("hub:init"),
  protocolVersion: z.number(),
  token: z.string(),
  estudante: EstudanteSchema,
  apiUrl: z.string().url(),
  config: z.record(z.unknown()).default({}),
});

/** Mensagens POS-init, validadas com Zod nos DOIS lados. Unioes
 *  discriminadas — mensagem que nao casa e DESCARTADA com aviso, nunca
 *  despachada crua. Tudo aqui e ADITIVO ao v1: hub antigo ignora mensagem
 *  que nao conhece; experiencia antiga nunca recebe as novas sem pedir. */
export const HubParaExpSchema = z.discriminatedUnion("type", [
  z.object({ type: z.literal("hub:token"), token: z.string() }),
  z.object({ type: z.literal("hub:pause") }),
  z.object({ type: z.literal("hub:resume") }),
]);

export const ExpParaHubSchema = z.discriminatedUnion("type", [
  z.object({ type: z.literal("exp:ready"), protocolVersion: z.number() }),
  z.object({ type: z.literal("exp:concluir"),
             payload: ExpConcluirPayloadSchema.default({}) }),
  z.object({ type: z.literal("exp:token-expirando") }),
  z.object({ type: z.literal("exp:estado"), estado: z.record(z.unknown()) }),
]);
```

Listing D.2 – `hub:init` — o que a experiência recebe

```
{
  "type": "hub:init",
  "protocolVersion": 2,
  "token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "estudante": {
    "id": "8f2c1a44-...", "nome": "Ana",
    "serie": "8EF",
    "turmaId": "b17e...", "escolaId": "sp-001"
  },
  "apiUrl": "https://api-entreredes.unooria.com",
  "config": { "clarityProjectId": "ybo0odt6cn", "expId": "exp-01-instanooria" }
}
```

D.2 *Token* de experiência

Listing D.3 – *Claims* do JWT — contrato congelado

```
{
  "sub":       "8f2c1a44-...",       // usuarioId (nao "estudanteId": professor
                                     // passa pelo MESMO guard)
  "escola_id": "sp-001",             // vazio APENAS para admin-contrato
  "turma_id":  "b17e...",            // opcional
  "perfil":    "estudante",              // estudante | professor | gestor | admin-contrato
  "aud":       "plataforma-api",
  "iss":       "https://entreredes.unooria.com",
  "exp":       1756290000            // TTL de 15 minutos
}
```

NOTA

`sub` vira `usuarioId` nas *claims* tipadas da API (`src/auth/claims.ts`) — e não `estudanteId`, porque professor, gestor e admin passam pelo mesmo *guard*. O nome do campo é documentação: um `estudanteId` que às vezes contém um professor é uma armadilha para quem lê a *query* seis meses depois.

D.3 Telemetria — canal analítico

Listing D.4 – `src/telemetry/telemetry.controller.ts` — os esquemas

```
const EventoSchema = z.object({
  eventId:    z.string().uuid(),
  ts:         z.string(),
  expId:      z.string(),
  expVersion: z.string(),
  sessaoId:   z.string(),
```

```
  verbo:      z.string(),
  objeto:     z.string(),
  resultado:  z.record(z.unknown()).optional(),
  contexto:   z.record(z.unknown()).optional(),
});
// estudanteId e escolaId NAO estao no esquema de entrada: sao SOBRESCRITOS a
// partir do token. O cliente nao PODE colocar PII no lugar do id.

const BatchSchema  = z.object({ eventos: z.array(EventoSchema).max(50) });

/** O beacon do pagehide NAO envia headers, entao o token vem no CORPO. */
const BeaconSchema = BatchSchema.extend({
  beacon: z.literal(true).optional(),
  token:  z.string().min(1),
});

/** Envelope de push subscription do Pub/Sub. */
const PushSchema = z.object({
  message: z.object({ data: z.string().optional(), messageId: z.string().optional() }),
  subscription: z.string().optional(),
});
```

Listing D.5 – `POST /telemetry/batch` — corpo

```
{
  "eventos": [
    {
      "eventId":    "3f9a2c10-8b41-4d2e-9c77-1a5e6b0d4f22",
      "ts":         "2026-08-27T11:14:03.512Z",
      "expId":      "exp-02-unootube",
      "expVersion": "1.1.0",
      "sessaoId":   "c81b5e77-2d94-4a10-b3f6-77c2e9a4d001",
      "verbo":      "interacted",
      "objeto":     "video/v07",
      "resultado":  { "acao": "salvou", "salvos": 2 },
      "contexto":   { "tela": "feed", "ordenacao": "relevancia" }
    }
  ]
}
```

D.4 exp-70 — o *payload* de uma partida

Este é o *payload* mais elaborado da plataforma, e o que carrega mais decisões de projeto por campo.

Listing D.6 – `POST /vila/sessoes/:id/concluir` — corpo (abreviado)

```
{
  "v": 1,
  "bundleVersion": "1.0.0",
  "duracaoSeg": 1042,          // do "ENTRAR NA VILA" ao fim: tempo de JOGO
  "duracaoTotalSeg": 1387,     // desde a pagina abrir, incluindo a intro
  "status": "completa",

```

```
  "resumo": {
    "fogueirasAcesas": 8, "pedrasQuebradas": 3, "aldeoesAjudados": 2,
    "lampioesAcesos": 4, "tochasAcesas": 6,
    "dilemasVistos": 3, "dilemasResolvidos": 2, "distanciaM": 1840
  },

  "medidas": {
    "cobertura": { "celulas": 62, "total": 144, "pisoObrigatorio": 31 },
    "movimento": { "movendoSeg": 712, "escuroSeg": 128, "luzSeg": 584,
                   "atalhos": 2, "desvioM": 340, "raioMax": 58 },
    "lume":      { "conversas": 4, "perguntasCaminho": 1, "lore": 2,
                   "ajuda": 1, "sugestaoSeguida": null },
    "tocha":     { "aldeao": 2, "lampiao": 3, "pedra": 1,
                   "passarela": 0, "retomadas": 1 },
    "pedra":     { "borda": true, "destaque": false,
                   "nomesApagados": 0, "nomesLidos": 5 },
    "fumaca":    { "avistadas": 3, "seguidas": 2 },
    "brinquedos": 4,
    "resgates":  [ { "at": 512, "firesLit": 5 } ]
  },

  "oportunidades": { "lampiao": 6, "aldeao": 4, "pedra": 1 },

  "dilemas": [ { "pairId": 0, "escolha": "aldeao", "emMs": 214000 } ],

  "eventos": [
    { "tipo": "chama_acesa",  "emMs": 61000,  "dados": { "zona": "praca" } },
    { "tipo": "tocha_gasta",  "emMs": 188000, "dados": { "destino": "aldeao" } },
    { "tipo": "pedra_gravou", "emMs": 402000, "dados": { "acao": "borda" } }
  ],

  "seguranca": {
    "apareceram": 4, "respondidas": 3, "ignoradas": 1,
    "cartoes": [
      { "id": "c1", "emMs": 240000, "viuOpcoes": true,
        "escolha": 2, "decisaoMs": 9400 },
      { "id": "c2", "emMs": 431000, "viuOpcoes": true,
        "escolha": 1, "decisaoMs": 1800 },
      { "id": "c3", "emMs": 655000, "viuOpcoes": false,
        "escolha": null, "decisaoMs": null }
    ]
  }
}
```

D.4.1 As decisões que cada campo carrega

Campo	Decisão registrada
Ausência de `sessaoLocalId`, `destino`, `iniciadaEm`	Identidade e turma vêm da *sessão autenticada*; repeti-las no *payload* criaria uma segunda fonte de verdade para o servidor conferir contra a primeira. O *id* da partida é emitido pela API (é a chave de idempotência) e o instante de início é cravado por ela.

`seguranca.cartoes[]` sem classificação	**Regra 3.** O aplicativo de origem recebia `seguras`, `risco`, `golpeAceito`, `legitimaRecusada` e `bolha` por cartão — o *cliente* classificando as próprias respostas. Agora ele manda o que *fez*, e a API corrige contra `catalogo.ts`. Um *bundle* adulterado já não pode se reportar como tendo recusado tudo.
`viuOpcoes`	Separa “deixei passar” de “nem vi”. As duas caíam em `ignoradas`, e são crianças diferentes.
`decisaoMs`	O campo mais informativo do conjunto: cair em dois segundos e cair depois de dez são duas crianças diferentes — a primeira não avaliou, a segunda avaliou e foi convencida.
`oportunidades` — **a ausência de uma chave**	Significa “não observado”, que é *diferente* de zero. Zero é “teve a chance e não foi” e entra na média do polo; não observado **sai da conta**.
`pedra.borda` *versus* `pedra.destaque`	Gravou onde ainda cabia, sem apagar ninguém (Realização) *versus* apagou um nome tendo a borda livre (Poder). **A distinção é o ponto.**
`lume.sugestaoSeguida`: `null` = a sugestão não chegou a ser testada. **Não** é “não seguiu”. `null`	
`cobertura.pisoObriga`	Quantas células a rota obrigatória já toca. Sem isso, cobertura mediria *cumprimento do objetivo* em vez de iniciativa.
`eventos[].emMs`	Tempo **relativo** ao início. Nunca relógio de parede.
`eventos[].dados`	Número, booleano ou *string* de **vocabulário fechado** (ids das 8 zonas + Fogo Sagrado, destinos da Tocha, as duas ações da Pedra, ids dos cartões). Nome de criança não cabe num vocabulário fechado.
`bundleVersion`	Fica na linha para o *skew* de *publish* ser **visível**: quem estava com a aba aberta durante uma publicação termina a partida com o *bundle* anterior, e uma partida dura 15–25 minutos.

D.4.2 As duas travas estruturais

Listing D.7 – Por que `.strict()` e por que a inspeção acontece no BRUTO

```
/**
 * ATENCAO AO `.strict()` E A ORDEM DE `validarPayload`.
 *
 * A primeira versao disto no app de origem tinha as recusas mais importantes
 * escritas como `.refine()`, e elas NAO FUNCIONAVAM: o zod remove chaves
 * desconhecidas ANTES de rodar refine, entao `{ nome: "Lara" }` era descartado
 * em silencio e o refine recebia um objeto ja limpo. O payload passava e o nome
 * simplesmente sumia sem ninguem saber que tinha chegado. Por isso:
 *
 *   1. `.strict()` — chave desconhecida e ERRO, nunca descarte silencioso;
 *   2. `validarPayload()` inspeciona o dado BRUTO antes do parse, porque so o
 *      bruto ainda contem o que nao deveria ter vindo.
 */
```

Trava	Como funciona
`CHAVES_PROIBIDAS` (por *substring*)	`nome`, `name`, `apelido`, `nick`, `estudante`, `crianca`, `jogador`, `player`, `cpf`, `matricula`, `email`, `telefone`, `responsavel`, `professor`, `endereco`, `nascimento`. Por *substring*, porque `nomeCompleto`, `nome_do_estudante` e `playerName` são a mesma coisa para quem for reidentificar a criança.
`CHAVES_PROIBIDAS_EXATA`	`rg`, `ra`, `id_estudante`, `doc` — **só por igualdade**. `rg` casaria dentro de `perguntasCaminho`, e um falso positivo recusa a partida inteira de uma criança.
`CHAVES_DO_CONTRATO`	`nomesApagados` e `nomesLidos` são chaves legítimas que contêm um termo proibido como *substring* (contam nomes na Pedra — números, nunca texto). A lista é **fechada**: campo novo que colida tem de ser adicionado aqui *de forma consciente*.
`CHAVES_DE_PERFIL`	Recusa *payload* que traga o perfil calculado. O jogo sabe computá-lo e deliberadamente não o envia — é isso que permite reprocessar o histórico quando `VERSAO_MAPEAMENTO` subir.

> **NOTA**
>
> A varredura por nome de criança importa *mais* aqui do que no aplicativo de origem: lá o *bundle* não tinha nome nenhum em memória; aqui o *shim* entrega o primeiro nome da criança ao jogo, para o Lume chamá-la e para a Pedra dos Nomes. **Existe um nome a vazar agora** — e rejeitar é melhor que ignorar em silêncio, porque um campo de nome aparecendo é *bug do jogo*.

D.4.3 O cap de eventos, e por que ele caiu de 2000 para 200

```
.max(200)
/**
 * 200, e o teto REAL do jogo e ~22.
 *
 * MEDIDO no bundle: o jogo tem 7 pontos que gravam evento, e todos sao atos
 * discretos com guarda propria — 8 Chamas Sagradas, 4 aldeoes, 1 Borda + 3
 * Destaques, 4 cartoes da missao, 1 primeira conversa com o Lume. Nada por
 * quadro, nada por metro andado. `restart()` zera o log, entao replay nao
 * acumula.
 *
 * Herdamos 2000 do app de origem. Cabe baixar: o cap e a UNICA coisa que
 * limita o tamanho da linha se um cliente adulterado mandar lixo, e
 * 2000 x ~80 bytes = 160KB por partida (1,6GB num contrato de 10 mil estudantes)
 * contra ~2KB reais. Com 200 ha 9x de folga para eventos novos.
 */
```

D.5 Manifesto de experiência

Listing D.8 – `public/experiencia.json` — exp-01, com `viewport`

```json
{
  "id": "exp-01-instanooria",
  "nome": "Instanooria — O Conteúdo que não chegou",
  "versao": "1.0.0",
  "series": ["8EF"],
  "capabilities": ["telemetria", "progresso", "ia"],
  "viewport": { "largura": 375, "altura": 812 },
  "verbos": {
    "attempted":  "entrou no aparelho",
    "completed":  "concluiu a missão (dossiê visto)",
    "progressed": "mudou de fase"
  }
}
```

Listing D.9 – `public/experiencia.json` — exp-90, ferramenta de professor

```json
{
  "id": "exp-90-plano-de-aula",
  "nome": "Gerador de Plano de Aula",
  "versao": "1.0.0",
  "series": [],
  "capabilities": ["telemetria", "ia"],
  "publico": "professor",
  "verbos": {
    "attempted":  "pediu a geração de um plano anual (disciplina + série)",
    "completed":  "salvou o plano pela primeira vez",
    "interacted": "exportou .doc, restaurou o original ou excluiu um plano"
  }
}
```

> **NOTA**
>
> `series: []` na exp-90 significa "não se aplica" porque `publico: "professor"`; na exp-70 significa **todas as séries**, porque é experiência de estudante *transversal*. O mesmo valor, duas leituras — e o que as distingue é `publico`. Quando o filtro por série voltar a valer, vazio tem de continuar querendo dizer "todas" para o público `estudante`.

D.6 Validação de variáveis de ambiente

Listing D.10 – `src/config/env.ts` — o *refine* que evita um incidente

```ts
}).superRefine((env, ctx) => {
  if (env.AI_PROVIDER === "vertex" && !env.VERTEX_PROJECT_ID) {
    ctx.addIssue({
      code: z.ZodIssueCode.custom,
      path: ["VERTEX_PROJECT_ID"],
      message:
        "obrigatorio com AI_PROVIDER=vertex (em dev, use AI_PROVIDER=mock " +
        "para subir sem GCP)",
    });
  }
});
```

```
12
13 export function validarEnv(config: Record<string, unknown>): Env {
14   const parsed = EnvSchema.safeParse(config);
15   if (!parsed.success) {
16     const erros = parsed.error.issues
17       .map((i) => `  ${i.path.join(".")}: ${i.message}`)
18       .join("\n");
19     throw new Error(`Variaveis de ambiente invalidas:\n${erros}`);
20   }
21   return parsed.data;
22 }
```

NOTA

Sem esse `superRefine`, a URL do Vertex viraria `.../projects/undefined/...` e o erro só apareceria na *primeira geração real* — provavelmente de um professor, em aula. A mensagem diz o que fazer, não só o que está errado.

D.7 Importação de usuários por CSV

Listing D.11 – `POST /usuarios/importar-csv` — corpo `text/csv` cru

```
1 nome,usuario,senha,papel,turma,serie
2 Ana Souza,ana.souza,trocar123,estudante,8A,8EF
3 Bruno Lima,bruno.lima,trocar123,estudante,8A,8EF
4 Carla Dias,carla.dias,trocar123,professor,,
```

NOTA

O corpo é `text/csv` **cru**, com teto de 2 MB — por isso `src/main.ts` registra um *body parser* explícito de texto além do JSON. `username` é único **por escola**: uma rede tem muitos `joao.silva`.

Apêndice E

Dicionário de dados

Catálogo das 27 tabelas de `public`, mais o *schema* `particoes`. Os diagramas entidade-relacionamento estão no Capítulo 8.

Regime	Significado
Tenant	Tem `escola_id NOT NULL` + `ENABLE`/`FORCE ROW LEVEL SECURITY` + política por escola. Acesso só por `comTenancy`.
Tenant+posse	O acima, *mais* filtro por `professor_id` em toda *query*: dois professores da mesma escola não se leem.
Catálogo	Sem `escola_id`, **fora** da RLS. Pertence ao contrato, não à escola.

E.1 Índice de tabelas

Tabela	Regime	Conteúdo
`escolas`	Tenant raiz	Unidades escolares do contrato
`periodos_letivos`	**Catálogo**	Ano letivo e seus recortes
`turmas`	Tenant	Turmas por escola, período e série
`matriculas`	Tenant	**O enturmamento** (estudante e professor)
`users`	Tenant	Pessoas e papéis
`experiencias`	**Catálogo**	Manifesto de versões
`progresso`	Tenant	Progresso genérico da SDK
`ia_uso_diario`	Tenant	Contador de quota de IA
`telemetria_eventos`	Tenant, particionada	Canal analítico
`exp_01_activities`	Tenant	Atividade preparada pelo professor
`exp_01_authors`	Tenant	Personas fictícias
`exp_01_posts`	Tenant	Posts gerados

Tabela	Regime	Conteúdo
`exp_01_post_comments`	Tenant	Comentários dos posts
`exp_01_generation_jobs`	Tenant	Fila de geração
`exp_01_runs`	Tenant	Sessão de um estudante
`exp_01_events`	Tenant	Canal **operacional** de telemetria
`exp_01_student_posts`	Tenant	Posts publicados pelo estudante
`exp_01_conclusoes`	Tenant	Inclinação escolhida — a ÚLTIMA em `inclinacao` e a PRIMEIRA em `inclinacao_inicial`, com `trocas`. A tela convida a mudar de ideia (“vale a última”) e o `upsert` apagava a primeira; `atualizado_em` existe porque `created_at` congelava no primeiro envio e passava a mentir (ADR-43)
`exp_01_reflexoes`	Tenant	Texto de fechamento
`exp_01_dossiers`	Tenant	Dossiê calculado
`exp_01_comentarios_da_sessao`	Tenant	Comentários da sessão
`exp_01_exp_70_perfis_imp`	Tenant	**Contrato entre duas experiências**
`exp_02_sessoes`	Tenant	Estado da missão do UnooTube
`exp_03_sessoes`	Tenant	Estado da partida da Missão Tempestade
`exp_03_mensagens`	Tenant	A Transcrição (texto livre do estudante)
`exp_03_marcos`	Tenant	Trilha da decisão
`exp_03_simulacoes`	Tenant	Desfecho por tentativa
`exp_03_relogio_turma`	Tenant	O Relógio da Aula, por turma
`exp_70_sessoes`	Tenant	Uma partida da Vila do Lume
`exp_70_perfis`	Tenant	Retrato Schwartz recalculado
`exp_70_bolhas`	Tenant	Bolha por estudante
`exp_90_planos`	Tenant+posse	Metadados do plano de aula
`exp_90_referencias`	**Catálogo**	Referências curriculares
`exp_91_ciclos`	Tenant+posse	Ciclo do DASS-21 (**sensível**)
`exp_91_checkins`	Tenant+posse	*Check-in* diário (**sensível**)

E.2 Tabelas da plataforma

E.2.1 `escolas`

Coluna	Tipo	Nota

id	`TEXT` PK	**Texto, não UUID.** É o *mesmo* valor que circula no *claim* `escola_id` do JWT e no `set_config` da RLS — um id legível (`sp-001`) vale mais que um UUID quando o valor aparece em três lugares de um diagnóstico.
`nome`	`TEXT`	
`codigo_inep`	`TEXT?`	O `orgs.identifier` do OneRoster. É por onde a escola daqui se reconhece na base da rede.
`tipo`	`TEXT`	`escola` | `regional` | `rede` — *CHECK* na migração. Só a `escola` é *tenant*: regional e rede não têm turma, estudante nem lotação. **Agrupa, não autoriza** (ADR-34).
pai_id	`TEXT?` FK	Auto-relacionamento. Um *trigger* (`escolas_pai_agrupa`) recusa pai que seja `tipo = 'escola'` ou a própria linha — é o que impede uma unidade de virar pai de outra e a hierarquia de perder o sentido em silêncio.
`fonte_id`	`TEXT?`	`sourcedId` desta organização no sistema de origem. Chave de reconciliação da importação — nunca é o id daqui.
`created_at`	`TIMESTAMPTZ(6)`	
`atualizado_em`	`TIMESTAMPTZ(6)`	

E.2.2 `perfis_portateis`

O documento assinado que veio de OUTRA instalação da plataforma (ADR-35). Ele NÃO se mistura com o dado local: não vira matrícula, progresso nem perfil da Vila, e não entra em média nenhuma.

Coluna	Tipo	Nota
id	`UUID` PK	
escola_id	`TEXT` FK	RLS por escola. O documento pertence à unidade que o recebeu.
estudante_id	`UUID` FK	Ligado pelo CPF na importação — nunca cria a pessoa: matricular é ato da escola, e só depois o perfil se anexa.
`emissor_contrato`	`TEXT`	Quem emitiu. Pode ser uma rede que esta instalação nunca viu.
`emissor_hub`	`TEXT`	Origem `https` onde o JWKS é procurado.
`emissor_nome`	`TEXT?`	Nome legível da rede de origem.
`emitido_em`	`TIMESTAMPTZ(6)`	Do documento, não da importação.
`documento`	`JSONB`	O documento como veio — fonte para reprocessar. **Única exceção à contenção do CPF** (ADR-33): o número é o conteúdo, e alterá-lo destruiria a assinatura. A exceção está numa lista explícita dentro do teste estrutural.

assinatura	TEXT	JWS compacto; o *payload* é { hash } do documento canônico.
verificacao	TEXT	verificada | nao-verificada — *CHECK*. Assinatura INVÁLIDA nunca chega aqui: é recusa.
importado_por	UUID FK	Quem importou. ON DELETE RESTRICT.
importado_em	TIMESTAMPTZ(6)	

> **DECISÃO DE PROJETO**
>
> UNIQUE (escola_id, estudante_id, emissor_contrato, emitido_em): a escola recebe o arquivo por e-mail e alguém importa duas vezes. Duas cópias do mesmo fato pareceriam duas medidas. Reimportar reconfere a assinatura e atualiza só o veredito — a rede pode ter voltado, e "não consegui falar com o emissor" deixar de valer não é fato novo sobre a criança.

E.2.3 periodos_letivos (catálogo)

Coluna	Tipo	Nota
id	UUID PK	
tipo	TEXT	ano_letivo | semestre | bimestre | trimestre — *CHECK* na migração.
titulo	TEXT	UNIQUE (tipo, titulo).
ano	SMALLINT	
inicio, fim	DATE	*CHECK*: fim >= inicio. O período "de hoje" é derivado destas datas — sem coluna ativo, que é estado que ninguém atualiza.
pai_id	UUID? FK	Hierarquia do padrão: ano letivo → bimestre.
fonte_id	TEXT?	

Sem escola_id e sem RLS, de propósito: o calendário é do **contrato**, mesmo precedente de exp_90_referencias. Uma rede compartilha o ano letivo, e duplicá-lo por escola só criaria cem linhas divergentes.

E.2.4 turmas

Coluna	Tipo	Nota
id	UUID PK	
escola_id	TEXT FK	

periodo_letivo_id	UUID FK	**NOT NULL** de propósito: turma sem período é a coluna `users.turma_id` de novo, com outro nome — e é exatamente o que a virada do ano quebra.
nome	TEXT	`UNIQUE (escola_id, periodo_letivo_id, nome)`: duas escolas podem ter "8A", e o "8A" de 2026 e o de 2027 são turmas diferentes.
serie	TEXT	`1EF`–`9EF` | `1EM`–`3EM`, validado com Zod na aplicação.
tipo	TEXT	`regular` | `componente` — o `classType` do padrão (`homeroom`|`scheduled`). *CHECK* na migração.
turno	TEXT?	`manha` | `tarde` | `noite` | `integral`.
codigo, fonte_id	TEXT?	Código da turma na rede e o `sourcedId` dela no sistema de origem.

E.2.5 `matriculas`

Coluna	Tipo	Nota
id	UUID PK	
escola_id	TEXT	**RLS** por esta coluna. Consulta fora de `comTenancy` volta **vazia**, sem erro — o sintoma é lista de pessoas "sem turma".
turma_id	UUID FK	`ON DELETE CASCADE`.
usuario_id	UUID FK	`ON DELETE CASCADE`.
papel	TEXT	`estudante` | `professor` — *CHECK* na migração. **Gestor não entra**: o papel dele é da escola, não da turma.
principal	BOOLEAN	A turma de referência da pessoa. É ela que alimenta o cache `users.turma_id` e, por ele, o *claim* `turma_id` do *token*.
inicio	DATE	
fim	DATE?	Preenchido em vez de `DELETE` quando a pessoa sai. *CHECK*: `fim >= inicio`.
fonte_id	TEXT?	

`UNIQUE (escola_id, turma_id, usuario_id, papel)` mais índices por `(escola_id, usuario_id)` — "as turmas do professor" — e por `(escola_id, turma_id)` — "quem está nesta turma".

E.2.6 `users`

Coluna	Tipo	Nota
id	UUID PK	
role	TEXT	`estudante` | `professor` | `gestor` | `admin-contrato`.

escola_id	`TEXT?` FK	***CHECK* na migração:** `NOT NULL` *exceto* quando `role = 'admin-contrato'`. Nenhuma outra combinação é aceita pelo banco.
turma_id	`UUID?` FK	
`username`	`TEXT`	`UNIQUE (escola_id, username)` — único **por escola**, não globalmente: uma rede tem muitos `joao.silva`.
`display_name`	`TEXT`	
`password_hash`	`TEXT`	
`avatar`	`TEXT?`	
`matricula_externa`	`TEXT?`	O RA da pessoa na rede. É a chave de reconciliação da importação: com ela, corrigir nome ou login continua sendo a mesma pessoa; sem ela o *upsert* cai no `username` e um login corrigido vira um segundo estudante, com o histórico preso no primeiro. Índice único **parcial** por escola — vive na migração, porque o Prisma não modela índice parcial.
`cpf`	`TEXT?`	Os 11 dígitos, **sem pontuação** (*CHECK*) — a formatação é da tela, e guardar as duas formas faria "529.982.247-25" e "52998224725" coexistirem como duas pessoas. É a identidade IMUTÁVEL na rede: sobrevive a troca de nome, *login*, matrícula e ESCOLA, e é por ela que uma transferência é reconhecida como a mesma criança. Único por **contrato** (índice parcial), não por escola. **O gestor vê o número** — é como ele confere identidade contra o sistema da rede. Esta é a ÚNICA coluna do banco onde o CPF existe (**ADR-33**).
`fonte_id`	`TEXT?`	`sourcedId` da pessoa no sistema de origem. **Distinto** do `sub` do provedor de identidade: sistema de gestão e IdP são coisas diferentes, e as duas reconciliações não são a mesma.
`email`	`TEXT?`	
`ativo`	`BOOLEAN`	Saiu da rede → `false`. Nunca `DELETE`: há dado de estudante pendurado. `ativo = false` recusa *login* com a mesma mensagem genérica de credencial inválida.
`senha_provisoria`	`BOOLEAN`	A senha veio da importação ou de um reajuste e ainda não foi trocada. Enquanto verdadeira, o hub prende em `/trocar-senha` e não emite *token* de experiência (**ADR-29**).
`senha_definida_em`	`TIMESTAMPTZ?`	
`atualizado_em`	`TIMESTAMPTZ(6)`	

`turma_id` permanece, com novo significado: **cache** da matrícula principal. O *login* e o *claim* `turma_id` do *token* — contrato congelado — o leem antes de haver escola no contexto, onde `matriculas` não é alcançável.

E.2.7 `experiencias` (catálogo)

Coluna	Tipo	Nota
exp_id	`TEXT` PK	Slug da experiência.
`nome`	`TEXT`	
`versao_ativa`	`TEXT`	***Rollback* = `UPDATE` deste campo.** O hub monta a URL do `iframe` com ele.
`series`	`TEXT[]`	**Vazio = TODAS as séries**, não nenhuma. É assim que uma experiência transversal se declara.
`capabilities`	`TEXT[]`	`telemetria`, `progresso`, `ia`, `avaliacao`, `tempo-real`.
`viewport`	`JSONB?`	`{largura, altura}` em pixels lógicos. Presente ⇒ o hub embeda dentro do `DispositivoFrame`.
`publico`	`TEXT`	`estudante` (default) | `professor`, com *CHECK*. `professor` ⇒ `/exp/*` responde 404.
`permissoes`	`TEXT[]`	Permissões de `iframe`; hoje só `pointer-lock`, com *CHECK* do vocabulário. **`fullscreen` NÃO entra aqui**: o `allow` do *frame* já o delega a todas, e campo de manifesto que não muda nada é pior que campo nenhum.

E.2.8 `progresso`

PK composta `(escola_id, estudante_id, exp_id)`; `dados JSONB DEFAULT '{}'`; `atualizado_em`. É o progresso *genérico* da SDK — estado rico de experiência vai para a tabela da experiência.

E.2.9 `ia_uso_diario`

PK composta `(escola_id, usuario_id, dia)`, com `dia` no formato `YYYY-MM-DD`; `tokens_usados INT`. Incremento atômico por `ON CONFLICT DO UPDATE ... RETURNING`, para que requisições simultâneas não se sobrescrevam.

E.2.10 `telemetria_eventos` (particionada)

Coluna	Tipo	Nota
event_id	`UUID` PK_1	Chave de *dedupe*, gerada pela SDK.
ts	`TIMESTAMPTZ(6)` PK_2	**Chave de partição.** A PK é composta porque o PostgreSQL exige a chave de partição em todo índice único — e a *dedupe* segue correta, porque a reentrega da mesma mensagem traz o *mesmo* `ts`.
`escola_id`, `estudante_id`	`TEXT`	`estudante_id` é **opaco**: nunca nome nem *e-mail*. Derivado do *token* no servidor.

exp_id, exp_version	TEXT	
sessao_id	UUID	
verbo, objeto	TEXT	Vocabulário xAPI simplificado.
resultado, contexto	JSONB?	Livres, por experiência.

Objeto	Nota
idx_tel_escola_exp_ts	Índice (escola_id, exp_id, ts).
particoes.telemetria_eventos_YYYY_MM	Uma por mês, faixas declaradas até 2028-12. Vivem no *schema* particoes — em public cada uma viraria *drift* de *schema*.
particoes.…_default	**Rede de segurança.** Sem ela, evento com ts fora das faixas seria **recusado** — e telemetria perdida em silêncio é pior que telemetria mal guardada. Encher = faltou criar partição.

E.3 Tabelas da exp-01

E.3.1 exp_01_activities

Coluna	Tipo	Nota
id	UUID PK	
professor_id	UUID FK	
turma_id	UUID? FK	Para qual TURMA é esta atividade (**ADR-49**). NULL = atividade anterior à coluna, que vale para a escola inteira — o comportamento antigo, preservado sem *backfill*. Com turma, o painel de perfis, a importação da Vila e a entrada do estudante em POST /runs falam todos da MESMA sala. ON DELETE RESTRICT.
titulo, disciplina, tema_geral, recorte	TEXT	
contexto_aula	TEXT?	
pergunta_chave	TEXT	Gerada pela IA a partir do tema, validada pelo professor.
pergunta_origem	TEXT	ia | professor — **auditoria de autoria pedagógica**, e custa uma coluna.

`etapa`	`TEXT`	`investigacao` | `conclusao` | `debate` | `explicabilidade` | `perdidos` | `reflexao` | `fim`. **Relógio COLETIVO da turma:** o professor abre e fecha, e a tela do estudante trava conforme.
`ia_ao_vivo`	`BOOLEAN`	Personas reagem ao vivo. Desligável no meio da aula, **sem *deploy***.
`antagonismo`	`JSONB?`	Destilação **aprovada** pelo professor.
`categorias_neutras` `neutros_no_tema`	`TEXT[]`	Categorias dos neutros fora do tema; rótulos dos neutros dentro dele.
`estilo_visual`	`TEXT`	
`qtd_posts_tese`	`INT`	Posts que carregam o antagonismo; divisível por 8 (2 lados × 4 perfis).
`proporcoes`, `engine_config`	`JSONB`	Pesos e mixagem do motor.
`status`	`TEXT`	`rascunho` | `antagonismo` | `gerando` | `revisao` | `ativa` | `encerrada`.

E.3.2 `exp_01_posts`

Coluna	Tipo	Nota
`lado`	`TEXT`	`A` | `B` | `neutro`.
`perfil`	`TEXT`	`abertura` | `autotranscendencia` | `autopromocao` | `conservacao`.
`intensidade`	`SMALLINT`	1, 2 ou 3.

> **REGRA INVIOLÁVEL**
>
> `lado`, `perfil` e `intensidade` são **a variável independente do experimento**. Eles existem no banco e são **removidos pelo sanitizador** antes de o *feed* sair da API. Se chegassem ao cliente, a experiência poderia exibi-los — e o estudante saberia de que lado o algoritmo o colocou antes da revelação.

Coluna	Tipo	Nota
`formato`	`TEXT`	`feed` | `story` | `enquete`.
`caption`, `hashtags`, `emotional_hook`	`TEXT`/`TEXT[]`	Gerados pela IA.
`image_brief`	`TEXT`	*Prompt* da arte.
`image_url`	`TEXT?`	URL absoluta da CDN de *assets*.

`image_status`	`TEXT`	`pendente` | `ok` | `fallback`. `fallback` = arte procedural, porque a geração falhou — e a atividade não trava.
`likes_count`, `timestamp_label`	`INT`/`TEXT`	Ficção visual do clone.
`aprovado`	`BOOLEAN`	O professor revisa antes de ativar.

Índice de curadoria: `(activity_id, lado, perfil, intensidade, aprovado)` — é exatamente a consulta que o motor faz a cada bloco.

E.3.3 `exp_01_runs`

Coluna	Tipo	Nota
`fase`	`TEXT`	`onboarding` | `feed` | `encerramento` | `conclusao` | `revelacao` | `dossie` | `comparacao`.
`lado_inicial`	`TEXT`	`A` | `B`. Vem da afinidade do perfil; **nunca muda**.
`lado_origem`	`TEXT`	`perfil` | `sorteio` | `redistribuido` — **auditoria de por que** aquele estudante caiu naquele lado. Sem isso, uma reclamação de professor não tem resposta.
`perfil_jogo`	`JSONB?`	Perfil do Ato 1, **congelado** na criação da `run`.
`perfil_origem`	`TEXT`	`jogo` | `professor` | `ausente`.
`engine_state`	`JSONB`	Estado do motor. Lido e escrito *inteiro*, sempre dentro do mesmo *lock* — normalizá-lo daria *joins* sem ganho de consulta.
`nome_no_app`	`TEXT?`	O nome que a criança digitou. **Fica na `run`, não em `users`**, porque a conta é **do aparelho**: outra turma reusa o mesmo *login*.
`avatar_escolhido`	`TEXT?`	Mesma razão.

`UNIQUE (activity_id, estudante_id)`: uma sessão por estudante por atividade.

E.3.4 `exp_01_events` — o canal operacional

Coluna	Tipo	Nota
id	`BIGSERIAL` PK	Ordenação por inserção é significativa aqui — diferente do canal analítico.
`tipo`	`TEXT`	`visto` | `curtiu` | `descurtiu` | `salvou` | `abriu_comentarios` | `comentou` | `compartilhou` | `seguiu` | `deixou_seguir` | `nao_interessa` | `denunciou` | `enquete` | `rever` | `abriu_perfil`.
`payload`	`JSONB?`	`{dwellMs, dwellNorm}`, `{texto}`, `{opcao}`, `{authorId}`…

`client_ts`	`BIGINT`	Relógio do cliente, guardado como *dado*, não usado como verdade.

REGRA INVIOLÁVEL

***Dwell* nunca passa pelo Pub/Sub.** A latência e a falta de ordenação quebrariam a adaptação do *feed* e o *lock* por `run`. Esta tabela é o canal *operacional*; `telemetria_eventos` é o analítico (Capítulo 10).

E.3.5 `exp_01_exp_70_perfis_importados`

Coluna	Tipo	Nota
`escores`	`JSONB`	Os quatro quadrantes normalizados 0–1.
`dominante`	`TEXT`	Um dos quatro, ou `misto`.
`origem`	`TEXT`	`jogo` | `professor`.
`lado_forcado`	`TEXT?`	`A` | `B` quando o professor redistribuiu a turma; `NULL` = segue a afinidade. **PRESERVADO no *upsert*** — importar sozinho, ou levar o objeto inteiro no `update`, desfaria em silêncio uma redistribuição feita antes da aula.
`vila_sessao_id`	`UUID?`	De QUE partida da Vila esta medida saiu. **SEM chave estrangeira**, de propósito (**ADR-50**): zerar a turma na Vila apaga sessões, e uma cascata — ou um `RESTRICT` — levaria junto o perfil que está decidindo o lado de uma aula em andamento. É RASTRO da medida, não integridade. `NULL` = não veio da Vila desta instalação.
`versao_mapeament`	`SMALLINT?`	A `VERSAO_MAPEAMENTO` sob a qual o perfil foi medido. É o que permite o painel dizer que o mapeamento mudou desde a importação — sem ela, subir a constante deixava as cópias velhas decidindo o feed em silêncio.
`medido_em`	`TIMESTAMPTZ`	

E.4 Tabelas das demais experiências

E.4.1 `exp_02_sessoes`

Coluna	Tipo	Nota
(escola_id, estudante_id)	PK composta	Uma linha por estudante.

`salvos`	`TEXT[]`	Ids do catálogo. *CHECK* `cardinality(salvos) <= 3` na migração.
`curtidos`	`TEXT[]`	Segue livre mesmo após a conclusão.
`ranking_desbloqueado`	`BOOLEAN`	***Latch***: remover um salvo depois *não* tranca o ranking de novo. Estado que anda para trás confunde a criança e o professor.
`credibilidade_adicionada`	`BOOLEAN`	A conclusão pedagógica. **Congela os salvos**: o *score* já foi reportado, e nota e estado não podem divergir.
`concluido_em`	`TIMESTAMPTZ?`	*CHECK*: conclusão implica `concluido_em`.
`reinicios`	`SMALLINT`	Reiniciar descongela.

REGRA INVIOLÁVEL

O gabarito (`factual`) **não está aqui** e não é devolvido em nenhuma rota de estudante: vive em `src/unootube/catalogo.ts`. O que *sai* de lá é `VideoPublico`, isto é, tudo menos `factual` — e há teste que enumera os campos, para que um campo novo não escape por distração. Desde a 1.1.0 são também públicos `visualizacoesRotulo` e `curtidasRotulo`: rótulos **cosméticos** de popularidade, sem peso em conta nenhuma, mas escolhidos para que o vídeo *fake* seja o dos números grandes — o contraste que o *ranking* desmonta depois.

E.5 Tabelas da exp-03

E.5.1 `exp_03_sessoes`

Coluna	Tipo	Nota
(escola_id, estudante_id)	PK composta	Uma linha por estudante. **Não há id de sessão**: a identidade vem do *token*.
`turma_id`	`UUID?`	Denormalizado: o painel do professor consulta por turma o tempo todo, e resolver por *join* a cada leitura era o caminho para esquecer o filtro numa consulta nova.
`passo`	`TEXT`	*CHECK* nos quatro Passos. Derivado do que aconteceu na conversa, não de um clique de avançar.
`tentativa`	`SMALLINT`	Recomeçar abre a seguinte. *CHECK* ≥ 1.
`minutos_gastos`	`INT`	Minuto narrativo desde as 17h00. *Snapshot* do que o relógio da turma já produziu — a fonte da verdade é `exp_03_relogio_turma`. É também o minuto de **saída** das equipes quando a tempestade é enfrentada. *CHECK* 0–60.
`conjuntos`	`TEXT[]`	Fontes carregadas, em camadas. *CHECK* `cardinality <= 8`.

`perguntas_gastas`	`INT`	Conta o **gasto**, não o resto: mudar o teto passa a valer para partida em curso. *CHECK* 0–3.
`alocacao_alfa`, `alocacao_bravo`	`TEXT?`	Slug do bairro de destino. O agente **não escolhe** — registra a escolha do estudante.
`justificativa_alfa`, `justificativa_bravo`	`TEXT?`	Slug de um conjunto que o estudante carregou, ou `'intuicao'`. *CHECK*: justificativa sem destino é estado impossível; destino sem justificativa é legítimo (o portão exige antes de *enviar*).
`tokens_gastos`, `chamadas_ia`	`INT`	O teto por sessão (**ADR-24**). Somam voltas de ferramenta e tentativas rejeitadas.

E.5.2 `exp_03_mensagens`

Coluna	Tipo	Nota
id	`BIGSERIAL`	Paginação por id é estável; por *timestamp* não.
`papel`	`TEXT`	*CHECK* `IN ('estudante','agente')`.
`texto`	`TEXT`	**Caixa livre escrita por menor de idade**. Nunca vai a log nem à telemetria; nenhuma rota de gestor a alcança (**ADR-27**).
`anexo`	`JSONB?`	O cartão que acompanha a fala. É o que faz a conversa voltar inteira depois de um F5.
`debitou`	`BOOLEAN`	Se esta fala custou uma Pergunta — a trilha de auditoria do orçamento. *CHECK*: só fala do estudante pode debitar.

E.5.3 `exp_03_marcos`

Coluna	Tipo	Nota
id	`BIGSERIAL`	*UNIQUE* `(escola_id, estudante_id, tentativa, ordem)`.
`tipo`	`TEXT`	*CHECK* `IN ('fontes','leitura','alocacao','envio')`.
`minuto`	`INT`	Minuto do relógio em que aconteceu. *CHECK* 0–60.
`dados`	`JSONB`	*Payload* por tipo. **Só dado público**: este JSON chega ao navegador.

E.5.4 `exp_03_simulacoes`

Coluna	Tipo	Nota
id	`UUID`	*UNIQUE* `(escola_id, estudante_id, ordem)`.
`alocacao`, `desfecho`	`JSONB`	A decisão e o que ela rendeu.

`minuto_da_saida`	`INT`	O minuto em que as equipes saíram: é o que decidiu se cada uma chegou. Guardado junto do desfecho para a linha ser autoexplicativa.
`versao_prompt`	`INT`	Sem ela não se sabe qual *prompt* produziu o que está em produção.

E.5.5 `exp_03_relogio_turma`

Coluna	Tipo	Nota
(escola_id, turma_id)	PK composta	O relógio é da **turma** (**ADR-25**).
`modo`	`TEXT`	*CHECK* `IN ('cronometrado','livre')`.
`iniciado_em`	`TIMESTAMPTZ?`	**Nulo = a missão ainda não começou**: o estudante vê o cronômetro parado e não fala com o agente.
`duracao_seg`	`INT`	Segundos de parede. "+2 min" soma 120. *CHECK* > 0 — zero dividiria por zero no minuto narrativo.
`encerrado_em`	`TIMESTAMPTZ?`	*CHECK*: encerrar o que nunca começou é estado impossível.
`pivo_seg`, `pivo_min`	`INT`	O **pivô** da extensão: onde o relógio da cidade estava quando a duração mudou. É o que impede o minuto narrativo de rebobinar — aos 4:00 de 5:00 o estudante está no minuto 48, e recalcular com 7:00 daria 34.

REGRA INVIOLÁVEL

O gabarito da exp-03 não está em tabela nenhuma: os oito conjuntos, com o viés (`instrucao`) e a cobertura de cada um, vivem em `src/unoogpt/modelo/conjuntos.ts`, e os parâmetros ocultos do mundo em `modelo/parametros.ts`. Nada disso é persistido, nada disso é servido, e há conferência por `grep` no `dist/` da experiência.

E.5.6 `exp_70_sessoes`

Coluna	Tipo	Nota
id	`UUID` PK	**Emitido pela API.** No aplicativo de origem o cliente escolhia a própria chave de idempotência, e duas crianças na mesma máquina colidiam — a segunda partida voltava "duplicada" e desaparecia.
turma_id	`UUID?` FK	**Desnormalizado:** turma *no momento da partida*. O estudante troca de turma entre anos letivos, e o resultado precisa continuar na turma em que foi medido.
`exp_version`	`TEXT`	Versão do *bundle* que jogou — torna o *skew* de *publish* visível em vez de silencioso.

`schema_version`	`SMALLINT`	Versão do contrato de *payload*.
`bolha`	`TEXT`	`A` | `B` | `C`. Atribuída pela **API**, nunca pelo cliente, e copiada aqui porque **o relatório abre pelo NÍVEL**, antes do que ela fez.
`cartoes_servidos`	`SMALLINT`	Quantos a API *entregou* — ≠ **quantos apareceram**. Zero ⇒ a missão não aconteceu, e o relatório precisa dizer isso em vez de "ela ignorou tudo".
`aberta_em`, `concluida_em`	`TIMESTAMPTZ`	Cravados pelo **servidor**. O aplicativo de origem derivava do relógio do cliente, e ele mesmo documentava que a hora numa máquina de escola costuma estar errada.
`duracao_seg`	`INT?`	Tempo de **jogo** — denominador dos ritmos.
`duracao_total_seg`	`INT?`	Desde a página abrir, incluindo a intro. Uma criança de 6 anos leva vários minutos ali, e sem o total o professor cronometra a atividade inteira e vê um número bem menor no painel.
`status`	`TEXT`	`aberta` | `completa` | `abandonada`. `aberta` que nunca fecha é partida perdida — e saber quantas são é *um dado sobre o jogo*.
`fogueiras_acesas` … `distancia_m`	`SMALLINT`/`INT`	Resumo **tipado**: o que o painel consulta e ordena. O detalhe fica no *payload*.
`payload`	`JSONB?`	O corpo bruto, íntegro. **Única fonte para recomputar** o perfil quando `VERSAO_MAPEAMENTO` subir.
`leitura_mensagens`	`JSONB?`	A leitura da missão, **persistida**: o relatório sobrevive a qualquer expurgo futuro do *payload* e não precisa recomputar a cada visita.

> **DECISÃO DE PROJETO**
>
> **Não há `expires_at`** (ADR-13). Houve, de 90 dias, e nenhuma rotina que o cumprisse. Coluna de prazo que ninguém lê é pior que prazo nenhum.

E.5.7 `exp_70_perfis`

Coluna	Tipo	Nota
`versao_mapeamento`	`SMALLINT`	Hoje **3**, herdada do piloto para os dados serem comparáveis. Sobe a cada mudança de peso, *probe* ou fórmula; **nunca** reescreve uma versão publicada.
`valores`	`JSONB`	Os dez valores em 0–100, e `null` **NÃO É ZERO**: é valor que não recebeu número, porque tem uma fonte de medida só ou porque a oportunidade não chegou.
`graus`	`JSONB`	`scored` | `indicative` | `zero` | `none` por valor. **A distinção mais importante do modelo.**

`abertura_vs_conservac`	`FLOAT?`	Eixo do circumplexo. `NULL` **quando um dos polos não foi observado** — eixo com polo não observado não é zero, é indefinido.
`transcendencia_vs_autopromocao`	`FLOAT?`	Idem.
`amostra_suficiente`	`BOOLEAN`	Falso ⇒ o painel abre avisando que **não sabe**. Não é rigor estatístico: é impedir que uma partida sem escolha vire uma frase sobre quem a criança é.

`UNIQUE (sessao_id, versao_mapeamento)`: quando a hipótese mudar, nasce uma linha nova e as antigas ficam — porque comparar as duas é o único jeito de saber o que a mudança fez.

E.5.8 `exp_70_bolhas`

PK composta `(escola_id, estudante_id)`; `bolha`, `atribuida_em`.

> **DECISÃO DE PROJETO**
>
> **Por estudante, não por partida.** Repetir a partida não pode trocar o nível: a folha do professor diz "esse estava na C", e o relatório abre pelo nível. A atribuição é balanceada **dentro da turma** — o aplicativo de origem fazia `nº do estudante mod 3`, que dava terços de graça porque os números eram contíguos de 1 a 40; *hash* de UUID dá terços na média da rede e pode dar 14/6/5 numa sala de 25 — e aí o debate da missão não acontece, porque ele depende de as três bolhas estarem na mesma sala.

E.5.9 `exp_90_planos` e `exp_90_referencias`

Coluna	Tipo	Nota
`status`	`TEXT`	`pendente` | `gerando` | `pronto` | `erro`, com *CHECK*.
`tentativas`	`SMALLINT`	Retomadas de geração *interrompida* (*restart*, *deploy*), não de falha da IA. Máximo 2.
`iniciado_em`	`TIMESTAMPTZ?`	`gerando` há mais de 12 minutos sem *worker* nesta instância = órfã.
`tokens_usados`	`INT`	
`caminho_gerado_md`	`TEXT?`	`planos/{escolaId}/{id}/gerado.md` — Markdown da IA, **imutável**.
`caminho_html`	`TEXT?`	`atual.html` — o que o editor salvou, sanitizado por *allowlist*.

`exp_90_referencias` é **catálogo do contrato**: sem `escola_id`, fora da RLS, com `serie NULL` significando "vale para todas as séries da disciplina" (ex.: o currículo da rede). Escrita só pelo *script* de importação.

E.5.10 `exp_91_ciclos` e `exp_91_checkins`

Coluna	Tipo	Nota
`ordem_itens`	`INT[]`	Permutação dos itens 1..21 sorteada na abertura do ciclo. Cada *check-in* leva 3 itens, um de cada subescala.
`expira_em`	`TIMESTAMPTZ`	21 dias. Ciclo não fechado vira `expirado`, sem pontuação.
`depressao`, `ansiedade`, `estresse`	`INT?`	Já $\times 2$ (escala do DASS-42), preenchidas **ao concluir** o sétimo *check-in*, no servidor.
`numero`	`SMALLINT`	1..7 dentro do ciclo.
`dia`	`TEXT`	`YYYY-MM-DD` no fuso `America/Sao_Paulo`. `UNIQUE (escola_id, professor_id, dia)`: um por dia.
`itens`	`INT[]`	Os 3 itens deste *check-in*.
`respostas`	`JSONB?`	`[{ item, valor }]` $\times 3$, valor 0..3.
`texto_livre`	`TEXT?`	**Nunca vai a log nem à telemetria.** Só contagens.
`abertura`, `pergunta`, `acolhimento`	`TEXT`	Os textos de entorno que o professor *viu* — guardados, não regerados.
`origem_texto`	`TEXT`	`ia` | `fallback` (IA indisponível ou barrada pelo *guardrail*) | `seguranca` (sinal de apoio — texto fixo, a IA nem é chamada).
`sinal_de_apoio`	`BOOLEAN`	Item 17 ou 21 ≥ 2, ou termo de risco no texto.
`tokens_ia`	`INT`	

REGRA INVIOLÁVEL

Dado sensível de saúde (LGPD art. 11). RLS por `escola_id` **mais** posse por `professor_id` em toda *query*; **nenhuma** rota de gestor ou agregado; e o próprio titular apaga tudo com `DELETE /bemestar/dados`.

E.6 O que o Prisma não modela

ATENÇÃO

Ler apenas `schema.prisma` dá uma visão **incompleta** do banco. Vivem *só* na migração:

- políticas de **RLS** (`ENABLE`, `FORCE`, `CREATE POLICY`);
- ***CHECK* constraints** (papel *versus* `escola_id`, cardinalidade dos salvos, vocabulário de `permissoes`, coerência de conclusão, XOR de ancoragem de comentário);
- **partições** e o *schema* `particoes`;
- `GRANT` para a *role* de aplicação.

> Os campos correspondentes são `TEXT`/`SMALLINT` no *schema*, e a validação de aplicação é feita com Zod.

Apêndice F

Operação

F.1 Comandos de desenvolvimento

Listing F.1 – Subir a plataforma

```
# --- Tudo em container, sincronizado com o editor (RECOMENDADO)
pnpm docker:dev          # pg + api + hub + as 6 experiencias, cada um com watcher
pnpm docker:dev:logs     # acompanha a recompilacao
pnpm docker:dev:deps     # DEPOIS de mexer em dependencia; e reiniciar
pnpm docker:dev:down     # desce

# --- So o banco em container, o resto no host
docker compose up -d pg
cp .env.example apps/api/.env && cp .env.example apps/hub/.env.local   # e editar
node scripts/gerar-chave-token.mjs --env      # chave RS256 do hub
pnpm --filter @edu/api migrate:dev            # migrations (Prisma + RLS)
pnpm --filter @edu/api seed                   # escola-demo, prof/gestor/estudante01-40
pnpm --filter @edu/api seed:demo              # atividade DEMO com 92 posts, sem IA
pnpm dev                                      # hub :3000, api :3001, exp :5173

# --- Zerar o banco e recomecar do zero absoluto (DEV)
pnpm --filter @edu/api exec prisma migrate reset --force --skip-seed
SEED_MINIMO=1 pnpm --filter @edu/api seed      # escola + gestor.demo + admin, nada mais

# --- Imagens congeladas (mais proximo da producao)
docker compose up -d --build                  # migrations e seed no 1o up
```

> **ATENÇÃO**
>
> As portas são as mesmas nos três modos, então **não dá para rodar `pnpm docker:dev` e `pnpm dev` ao mesmo tempo**. Se salvar um arquivo e nada recompilar: `DEV_POLLING=1 pnpm docker:dev`.

Listing F.2 – Experiências, uma a uma

```
pnpm --filter @edu/exp-01-instanooria    dev   # :5173
pnpm --filter @edu/exp-02-unootube       dev   # :5174 (videos em public/videos)
pnpm --filter @edu/exp-90-plano-de-aula  dev   # :5175 -> /professor/plano-de-aula
pnpm --filter @edu/exp-91-bem-estar      dev   # :5176 -> /professor/bem-estar
pnpm --filter @edu/exp-70-vila-do-lume   dev   # :5177 -> /exp/exp-70-vila-do-lume

```

```
pnpm new:exp exp-07-fracoes                  # scaffolda do _template
```

Listing F.3 – O IdP sobe à parte

```
docker compose -f wso2/docker-compose.yml up -d    # IS :9443, pg dele :5433
docker compose -f wso2/docker-compose.yml watch    # olha conf/ e bootstrap/
docker compose -f wso2/docker-compose.yml logs bootstrap   # imprime as envs OIDC
```

Listing F.4 – Verificação e build

```
pnpm turbo typecheck lint test               # o portao da Definition of Done
pnpm turbo build --filter=@edu/exp-01        # 1 experiencia (+ deps)
pnpm turbo build --filter='...[HEAD^]'       # so o que mudou (o CI usa isso)
pnpm build:isolado                           # build de PRODUCAO do hub
                                             # (HUB_DIST_DIR=.next-prod)
pnpm --filter @edu/api test
```

ATENÇÃO

Validar *build* de produção **sempre** com `pnpm build:isolado`. Um `next build` comum apaga o `.next` do *dev*, e o sintoma é "tela branca" ou "CSS bugado" depois — que se confunde com bug de aplicação. E `apps/hub/next-env.d.ts` alterna entre `./.next` e `./.next-prod`: modificado depois de um `build:isolado` = **descartar, não *commitar***.

F.2 Testes de ponta a ponta

```
node scripts/e2e/rodar.mjs               # tudo, na ordem canonica
node scripts/e2e/rodar.mjs professor     # so um papel
node scripts/e2e/rodar.mjs estudante/02  # so uma suite
node scripts/e2e/rodar.mjs --deck        # tudo + deck (.pptx) + videos (.webm)
node scripts/probe-pointerlock.mjs
```

A suíte vive em `scripts/e2e/`, organizada por PAPEL — `publico`, `estudante`, `professor`, `gestor` e `admin-contrato` —, e as evidências saem em `testes/<papel>/<suíte>/`: os *prints*, um `passos.json` e um `relatorio.md`.

NOTA — a ordem dos papéis não é alfabética

O `publico` não depende de nada e falha rápido se o *hub* estiver fora — é o canário mais barato. O `estudante` PRODUZ o estado (*runs*, partidas, sessões) que as telas do `professor` leem: invertê-los deixaria as telas de resultado vazias e as asserções de conteúdo sem objeto. E `gestor`/`admin-contrato` vêm por último porque importam pessoas e trocam senhas — efeitos que atrapalhariam as contas usadas pelos roteiros de cima.
Cada suíte roda em processo PRÓPRIO. É mais lento que um processo só, e é deliberado: uma suíte que trava o navegador não leva as outras junto.

NOTA — o manifesto é a fonte do deck e dos vídeos

Cada `registrar()` escreve, junto com o *print*, o que a tela MOSTRA e a regra do projeto que ela PROVA. É desse `passos.json` que nascem os slides e as legendas dos vídeos. Antes existia um roteiro de 66 KB mantido à mão, separado dos scripts, e ele divergiu: descrevia telas que já não eram fotografadas e omitia passos novos. A descrição passou a morar onde o passo mora.

NOTA — os roteiros de DOIS contextos

As missões conduzidas pelo professor (`estudante/02-instanooria.mjs` e `estudante/04-unoogpt.mjs`) sobem DOIS navegadores: um com o docente no relógio da turma, outro com o estudante no aparelho. Nada avança sem o botão do professor — um contexto só não chega ao fim, e passaria com o relógio quebrado.
O da exp-01 mede a `boundingBox` do *iframe* antes e depois do "Ver meu relatório" ($375 \rightarrow 1280 \rightarrow 349$px), escreve uma marca dentro do *frame* para provar que ele NÃO foi remontado, e lê o `backgroundColor` COMPUTADO — `toBeVisible()` não pega nenhuma das três.
Duas armadilhas medidas ao escrevê-lo. O *frame* precisa vir do próprio elemento (`elementHandle().contentFrame()`): procurar por `"exp-01"` na lista de *frames* devolve o frame DO HUB, cuja rota é `/exp/exp-01-instanooria` — e aí a verificação de "não recarregou" fica verde por construção, medindo a página errada. E a medição precisa esperar um elemento do FIM do relatório: o conteúdo vem de `React.lazy`, e medir logo após o título pega a tela montando, o que sai como fundo transparente e zero gráfico.

NOTA — erro provocado não é erro descoberto

Quando um roteiro exercita uma trava de propósito — ativar uma atividade com conteúdo de menos (422), subir um perfil portátil adulterado (400), abrir a análise de produto com o papel errado (403) — o erro que aparece é a PROVA de que a trava existe. Ele é declarado com `r.provocado(padrão, motivo)` e sai numa seção própria do relatório, em vez de pintar de vermelho a corrida que estava justamente demonstrando o comportamento.

Pré-requisito	Detalhe
Browsers do Playwright	`pnpm exec playwright install chromium`, uma vez por máquina — o pacote está nas `devDependencies` desde a fundação do monorepo, mas os navegadores ficam FORA dele, num cache do usuário. **Não** `npx playwright install`: o `npx` resolve o *playwright* mais NOVO do npm e baixa a *build* de navegador DELE, deixando `node_modules` e cache em versões diferentes.
Plataforma no ar	Contêineres **ou** `pnpm dev` — um *ou* outro, nunca os dois na mesma porta: o navegador e o servidor passam a ler *dev servers* diferentes.
Chrome do **sistema**	Obrigatório para a suíte da exp-70 e para o *probe* de *pointer lock*: o Chromium do Playwright não concede *pointer lock* nem no topo da página.

Duas escolas no *seed*	`escola-demo` (8A, 8B, 4A, 2A) e `escola-demo-2` (5A), com `prof.demo` lotado nas DUAS — é o que faz aparecer o seletor de escola no cabeçalho dele. `gestor.vilanova` administra a segunda.
Logins do *seed*	`gestor.demo` → `/admin/usuarios`; `prof.demo` → `/professor`; `estudante01`–`estudante40` → `/estudante`; `estudante4a01` para a exp-02. Senha de todos: `unooria123`. Eles nascem com `senha_provisoria = false` de propósito: prendê-los na troca do primeiro acesso pararia as suítes de estudante na mesma tela. `prof.demo` é matriculado nas quatro turmas — sem isso, as telas dele abrem vazias.
Bancada de *embed*	`/dev-embed?src=http://localhost:5174`. Sanidade do *design system*: `/ds-smoke` (só em *dev*).

F.3 *Design system*

```
npm view @unooria/design-system version         # qual e a ultima publicada
pnpm up -r @unooria/design-system --latest      # sobe em ui/hub/exps de uma vez
                                                # (^0.1.x NAO sobe minor)
cd ../storybook-unooria && npm run storybook     # :6007, o catalogo das 6 secoes
```

ATENÇÃO

Commitar o *lockfile* **e** os `package.json` juntos. E lembrar: **quem importa, declara** — todo aplicativo que escreve `@unooria/design-system` no código tem a dependência no próprio `package.json`, hub incluído.

F.4 Publicação e reversão

Listing F.5 – Reversão — as três formas, por velocidade

```
# 1. EXPERIENCIA (o mais rapido: sem build, sem imagem, sem deploy)
node apps/hub/scripts/publicar-manifesto.mjs exp-02-unootube --versao 0.9.0
#    Requer: TOKEN_SIGNING_KEY_PEM, API_URL, HUB_ORIGIN

# 2. HUB ou API (revisao do Cloud Run)
gcloud run services update-traffic "$HUB_SERVICE" \
  --region "$REGION" --project "$GCP_PROJECT" --to-revisions "<revisao>=100"

# 3. CONTEUDO DE PROFESSOR (versionamento de objetos do bucket de planos)
gcloud storage ls --all-versions \
  "gs://$PLANOS_BUCKET/planos/$ESCOLA/$PLANO/atual.html"
```

Mudou uma experiência? Suba a `versao` do `public/experiencia.json`. O caminho no bucket é `{expId}/{versao}/` e o `rsync` usa `--delete-unmatched-destination-objects`: publicar sem subir a versão sobrescreve um caminho que o próprio *pipeline* trata como imutável

(`Cache-Control` longo em tudo menos o `index.html`). Quem está no meio de uma sessão tem os *assets* daquela versão em cache e vai pedir arquivos que acabaram de ser apagados. Suba a versão e o problema não existe: caminho novo, nada a sobrescrever, e a versão anterior continua no bucket — que é o que faz a reversão nº 1 acima ser possível.

Listing F.6 – Publicar tudo (primeiro *deploy* de um contrato)

```
# GitHub Actions -> workflow "deploy" -> Run workflow -> forcar: true
# Ignora o diff e publica hub, api e TODAS as experiencias.
```

F.5 Provisionamento de um contrato novo

1. No repositório de Terraform: instanciar o *module* `contrato` com o novo prefixo. **Nada é criado por *script* deste repositório.**
2. `terraform output` → configurar as `vars` do repositório (`scripts/configurar-github-ci.sh`).
3. Gerar o par RS256 e gravar em `{contrato}-token-signing-key` no Secret Manager; espelhar no *secret* `TOKEN_SIGNING_KEY_PEM` do GitHub.
4. Conceder `roles/aiplatform.user` à conta de serviço `{contrato}-api`. **Sem esse papel, toda geração falha com 403 do Vertex** — e o 403 vem do Vertex, o que se confunde com quota esgotada.
5. Verificar que a *role* de banco **não** tem `BYPASSRLS` nem `SUPERUSER` (SQL de verificação no *runbook* do repositório de infra).
6. Executar o *deploy* com `forcar: true`.
7. Importar usuários por CSV (`POST /usuarios/importar-csv`).
8. Importar as referências curriculares: `node scripts/importar-referencias-plano.mjs` com `PLANOS_IMPORT_TOKEN`.
9. **Manual, e pendente de automação:** `gsutil rsync` da mídia pesada da exp-02 para o *bucket* de *assets* (`R-09`).
10. Rodar `node scripts/e2e/rodar.mjs` contra o ambiente novo.

F.6 Diagnóstico: sintoma → causa

Esta tabela existe porque, nesta plataforma, o sintoma quase nunca aponta para a causa.

Sintoma	Causa provável
Ícone virando texto (“addInvestigação”)	O Next removeu o `@import` externo do CSS do *design system*, e `Material Symbols Rounded` é fonte de ligadura. Verificar `FontesDaPlataforma` em `casca-plataforma.tsx`. O `getComputedStyle` *responde* “Open Sans”, então não confie nele.

Sintoma	Causa provável
O professor não vê a turma dele	Falta **matrícula**, não falta permissão. Desde o enturmamento (**ADR-28**) "as turmas do professor" são as turmas em que ele tem linha em `matriculas`; o caminho é `/admin/turmas/:id` → "Adicionar professor", ou a coluna `turma` na linha dele da planilha.
A lista de pessoas mostra todo mundo "sem turma"	Consulta que lê `matriculas` **fora** de `comTenancy`: a RLS devolve zero linhas, sem erro nenhum. É a armadilha registrada na migração do enturmamento.
A pessoa troca a senha e volta para a tela de trocar a senha	Sessão antiga: o *cookie* ainda diz `precisaTrocarSenha`. O fluxo **refaz** o *login* com a senha nova justamente por isso (**ADR-29**); se ele falhar, a saída é `/login` — a senha JÁ foi trocada.
Importei e ninguém consegue entrar	A senha do lote não foi anotada. Ela é devolvida **uma vez**, na folha da tela de importação. O caminho de volta é `POST /usuarios/:id/reset-senha`, uma pessoa por vez.
Reimportei a planilha e nasceu gente duplicada	A coluna `matricula` estava vazia. Sem o RA, a chave de reconciliação cai no `usuario`, e um *login* corrigido vira uma segunda pessoa — com o histórico preso na primeira.
"Sem credencial do Google Cloud" — e o token funciona quando testado no container	O `GoogleAuth` é resolvido no **import** do módulo (`ai/vertex.provider.ts`), uma vez por processo. Se o arquivo do ADC estava sendo reescrito naquele instante (o `gcloud` renova o *refresh token* sozinho), a api fica presa num cliente quebrado até o fim da vida do processo, e um teste manual no mesmo contêiner passa. Conserto: reiniciar a api.
"Unique constraint failed on `cpf`" na importação	Comportamento correto: o índice único de CPF é da REDE, não da escola. Duas pessoas com o mesmo CPF não podem existir no contrato — e se a intenção era transferência, o caminho é importar na escola de DESTINO, que o reconhece. Em paralelo (duas importações simultâneas) a linha volta como erro dela, sem derrubar o lote.
Toda criança transferida virou cadastro novo	A planilha veio **sem** a coluna `cpf`. Sem ela, a chave de reconciliação cai na matrícula da rede — que muda de escola para escola — e a mesma criança nasce duas vezes.
O professor não vê a segunda escola no cabeçalho	Falta **lotação** nela. O seletor só aparece com duas ou mais: com uma, seria um campo de uma opção. O caminho é `/admin/escolas` → a escola → lotar, ou a planilha com o CPF dele importada na segunda unidade.
Lotei a supervisora na diretoria e ela continua sem ver as escolas	Estar numa regional não dá acesso a nada — a hierarquia agrupa, quem autoriza é a lotação (**ADR-34**). O caminho é `/admin/escolas` → filtrar pela regional → marcar as unidades → "Lotar pessoa nestas escolas".
Lotei em lote e o relatório diz menos escolas do que marquei	Havia `regional` ou `rede` na seleção, e elas não recebem lotação: não têm turma, estudante nem dado, e virariam escola fantasma no seletor do cabeçalho. A barra de seleção da tela já avisa quantas serão ignoradas.

Sintoma	Causa provável
O `orgs.csv` entrou mas as escolas ficaram sem regional	`parentSourcedId` apontou para uma organização que não veio no arquivo, ou veio DEPOIS. Isso é *aviso*, não erro: a escola entra sem regional, porque perdê-la por causa do agrupamento seria trocar o essencial pelo acessório. Reimportar com o pai presente corrige.
"Sem CPF não há perfil portátil"	É a regra, não um erro: numa rede de destino nenhum id daqui existe, e o CPF é a única chave que reconhece a criança. Preencha o CPF em `/admin/usuarios` e emita de novo.
Todo perfil importado entra "não verificado"	`PORTABILIDADE_EMISSORES` vazia, ou a origem do documento não está nela. Sem emissores configurados a api não busca JWKS nenhum, e marcar visivelmente é melhor que recusar. **Em contêiner, a entrada precisa apontar o JWKS**: `http://localhost:3000=http://hub:3000/.well-known/jwks.json` — o `emissor.hub` do documento é o endereço do navegador, e de dentro da api `localhost:3000` é ela mesma. É o *default* do `docker-compose.dev.yml`.
"em produção o JWKS só é buscado por https"	A entrada da allowlist aponta para `http`. Fora de produção isso é aceito (com aviso no *log*); em produção, não — chave pública por canal reescrevível não prova nada.
A emissão responde 503	Falta `CONTRATO_ID` no hub. Documento cujo emissor não se identifica não vale nada no destino, que precisa dele para achar o JWKS e para dizer de quem veio.
"Não há estudante com este CPF nesta rede"	O documento não cria pessoa. Matricule a criança primeiro (planilha ou OneRoster) e importe o perfil depois — a matrícula é ato da escola.
O item "Minhas escolas" não aparece na navegação	A pessoa tem UMA lotação. O panorama é a tela de quem administra várias unidades; com uma escola ele seria uma linha só, e item de menu que não leva a nada é pior que item nenhum.
O admin do contrato abre uma tela de escola e ela vem vazia	Falta `?escolaId=` na URL. O *token* dele não tem escola (é a exceção cross-escola), então a tela precisa dizer qual — e o caminho que já leva o parâmetro é `/admin/escolas`, pelos botões "Turmas" e "Pessoas".
"Nenhum período letivo cadastrado" ao importar ou criar turma	Banco novo, ou virada de ano sem o período criado. Turma vive DENTRO de um período letivo: o caminho é `/admin/periodos-letivos`. O *seed* cria o do ano corrente; um ambiente novo sem *seed* não tem nenhum.
Turma nova nasceu no ano errado	`vigente` é derivado das DATAS. Se nenhum período contém hoje, a plataforma cai no mais recente para não travar — e a tela do ano letivo avisa isso em faixa. Corrija as datas em vez de criar outro período.
O OneRoster recusa a linha dizendo que a turma é desconhecida	Ordem dos arquivos. `enrollments.csv` referencia `classSourcedId`, que vem de `classes.csv`, que referencia `termSourcedIds`, que vêm de `academicSessions.csv`. O relatório nomeia o arquivo que falta.
Todo mundo importado pelo OneRoster virou estudante	Na 1.2 o papel não está em `users.csv` — foi para `roles.csv`. Sem a coluna `role`, a pessoa nasce com o menor privilégio de propósito; aplique `roles.csv` para corrigir.

Sintoma	Causa provável
Mandei `bulk` e ninguém foi desativado	É o comportamento. Ausência de linha num lote completo não é ordem de exclusão: o relatório conta e lista os que não vieram, e a desativação é ato do gestor.
Elemento branco no branco	Classe de cor que não existe em `plataforma.css`. Não é erro em lugar nenhum. Checar o `backgroundColor` **computado**.
Experiência esticada na janela em vez de dentro do aparelho	`viewport` não lido: tabela `experiencias` vazia *e* `EXP_INTERNAL_HOST` ausente ou errado.
O UnooTube aparece dentro da casca de celular da Instanooria	`EXP_INTERNAL_URL` fixo em *dev*: as experiências estão lendo o manifesto de *uma só*. Usar `EXP_INTERNAL_HOST`, que preserva a porta.
`iframe` vazio	*Handshake* expirado, *dev server* fora do ar, ou *origin* divergente. Ver o console: a SDK avisa o descarte por esquema.
Câmera do jogo não anda *desde o começo*	`allow-pointer-lock` ausente: `permissoes` não chegou (manifesto vazio *e* `experiencia.json` não lido). `requestPointerLock()` morre com `SecurityError`. Desde 2026-08-27 o hub **avisa** no log (`embed-experiencia.ts`) nomeando URL, causa e consequência — antes o `catch {}` engolia, e uma configuração errada virava bug de jogo. O *sandbox* do `iframe` é a evidência: sem o *token*, o suíte da exp-70 falha no primeiro passo.
A câmera congela *depois* de já ter andado — ao apertar Esc ou trocar de aba — **e o aviso "clique pra voltar" não aparece**	O `pointerlockchange` não foi registrado. No jogo de origem os `<script>` ficavam no fim do `index.html` e o `load` disparava depois deles; no porte o `carregador.ts` injeta os *scripts* depois do *handshake* com o hub, quando o `load` já passou. Como o `pausedHint` é o **único** `controls.lock()` fora dos diálogos, a partida ficava sem caminho de volta. Corrigido em `16-boot.js` (2026-08-27); coberto pelo passo *pausa-e-volta* da suíte da exp-70.
Dois *dev servers* da mesma experiência na mesma porta	Um no *host* e um no *compose*. O navegador e o servidor podem alcançar **servidores diferentes** (`localhost` resolve para um, `host.docker.internal` para o outro), e o mais antigo responde com o `vite.config.ts` que ele tinha ao subir — daí um `403 Blocked request` no `allowedHosts` que *está* no arquivo. `pnpm docker:dev` e `pnpm dev` são um **ou** outro.
Vídeo não toca (`play()` rejeitado em silêncio)	`autoplay` não delegado no `allow` do *frame*: a *Permissions-Policy* tem *default* `'self'`.
O vídeo para no primeiro toque *dentro* da experiência e não volta — e a suíte acusa "autoplay BLOQUEADO"	**Não é o `allow` do *frame***, e é aí que o sintoma engana: o `play()` manual funciona e o `readyState` é 4. Clicar dentro do `iframe` tira o foco da janela do hub, o `ExpFrame.tsx` traduzia esse `blur` em `hub:pause` e o `focus` correspondente nunca voltava enquanto o estudante continuasse ali. A evidência é `visivel: false` em `lib/visibilidade.ts`. Corrigido em 2026-08-27: `blur` com `document.activeElement` igual ao próprio `iframe` não é perda de visibilidade nenhuma.

Sintoma	Causa provável
`confirm()` da experiência não aparece	`allow-modals` ausente no *sandbox*: o navegador **ignora** e devolve `false`.
Toda geração de imagem sai como *fallback* procedural	`VERTEX_IMAGE_MODEL` aponta para um modelo que saiu do catálogo. Verificar no Vertex.
Arte cortada pela metade	Proporção não pedida no `imageConfig`: o modelo devolve 1408×768 e o `cover` de 1080×1350 descarta a largura. Pedir `4:5`.
Plano de aula volta vazio com `finishReason: MAX_TOKENS`	`VERTEX_THINKING_BUDGET` diferente de `0`: os *tokens* de pensamento saem do `maxOutputTokens`.
Toda chamada de IA falha com 403	A conta de serviço não tem `roles/aiplatform.user`. O 403 vem do *Vertex*, não da API — e se confunde com quota esgotada.
IA não funciona no *host* nem no contêiner	ADC ausente. `gcloud auth application-default login`; os *compose* montam o arquivo em `/adc`.
API não sobe, reclamando de `VERTEX_PROJECT_ID`	Correto: com `AI_PROVIDER=vertex` a variável é obrigatória. Em *dev*, use `AI_PROVIDER=mock`.
`/telemetry/ingest` responde 401	Em *dev*, correto (`TELEMETRIA_PUSH_*` vazias = rota fechada). Em produção, conferir SA e *audience* contra a *subscription*.
Validação de *token* falha com "*issuer* inválido"	`iss` montado por concatenação. O do WSO2 é *tenant-qualified* e deve ser copiado **literalmente** do *discovery*.
Um bloco de *feed* desapareceu; posts repetidos	*Lock* consultivo não tomado. Verificar `pg_advisory_xact_lock(runId)` e o `timeout` de 20 s.
`comTenancy sem escola`	Rota sem `OidcGuard`/`TenancyInterceptor`, ou chamada fora de requisição sem `escolaId` explícito. **É o comportamento correto** — a alternativa era consultar sem filtro.
A sala inteira caiu na bolha A	*Lock* consultivo por turma não tomado em `POST /vila/sessoes`. A sala clica junto.
Partida da exp-70 recusada	`.strict()` pegou chave desconhecida, ou a varredura achou chave de nome, ou o *payload* trouxe perfil. **Recusa é o comportamento correto**; chave nova legítima precisa entrar no contrato *de forma consciente*.
Dependência nova não aparece no contêiner	`node_modules` do contêiner é volume próprio. `pnpm docker:dev:deps` + reiniciar.
Vite recusa a conexão do hub em contêiner	`allowedHosts` sem `host.docker.internal`. O Vite 5.4+ recusa `Host` que não conheça.
Vite subiu numa porta diferente	Segunda instância da mesma experiência. O hub passa a ler o manifesto da experiência errada.
***Link* sublinhado, marcador de lista de volta**	*Preflight* do Tailwind desligado. O pacote estiliza *componentes*, não elementos.

Sintoma	Causa provável
Sobrescrita de classe do *design system* não pega	Peso igual: o CSS do pacote é injetado *depois*. Usar **dois** seletores (`.app-shell > .x`).
`exp_01_exp_01_..._pkey` depois de uma migração de renomeação	`FOR ... IN SELECT` sobre o catálogo releu o que o laço alterou, ou `RENAME CONSTRAINT` renomeou também o índice. Ver a migração de referência.
Na exp-03, o estudante digita a pergunta, aperta Enter e NADA acontece — a conversa parece viva, porque a fala de abertura do agente está na tela	Falta `allow-forms` no *sandbox* do `iframe`: o `Composer` do *design system* é um `<form>`. O manifesto tem de declarar `"permissoes": ["forms"]`. O único rastro é um aviso de console sobre "*form's frame is sandboxed*" — e *typecheck* e *lint* passam.
Tela BRANCA dentro do `iframe`, sem erro de console	Uma rota de mutação devolveu a sessão *crua* em vez do envelope `{ sessao }`; o cliente leu `undefined` e derrubou a árvore do React. Toda rota de mutação da exp-03 devolve o envelope, e há teste fixando isso.
Na exp-03, "o Cientista não conseguiu responder" em toda tentativa	Credencial do Vertex ausente ou `AI_PROVIDER=vertex` sem ADC montado. A API traduz falha do provedor em `cientista-indisponivel` (503) de propósito — em desenvolvimento, use `AI_PROVIDER=mock`. Confira também `tokens_gastos` da sessão: teto estourado dá a mesma cara para o estudante, com outra frase.
Na exp-03, a turma inteira vê o cronômetro parado e ninguém consegue falar	**A largada não foi dada.** O relógio é da turma (**ADR-25**), e sai de `/professor/entre-redes/2-serie/unidade-1/resultados`. É o motivo nº 1 de "a missão não abre", e por isso a ficha do professor abre com a faixa avisando.
"Sem credencial do Google Cloud para falar com o Vertex", com o ADC montado e legível	A credencial de usuário precisa de **reautenticação periódica** (política da organização). O erro real, visível só ao pedir o token à mão, é `invalid_grant` com `error_subtype: invalid_rapt` — *RAPT* é *Reauth Proof Token*. **A janela é de 1 hora** (MEDIDO: login 22h20, expirado 23h31), ou seja, ela vence NO MEIO de uma sessão de trabalho e a suíte de ponta a ponta perde dois roteiros sem que uma linha de código tenha mudado. Antes de investigar "regressão" nos caminhos de IA, peça o token. Resolve com `gcloud auth application-default login` no HOST. Não é permissão nem *mount*: confira nessa ordem (existe? legível? token?), porque as três falham com a mesma mensagem na API. O `gcloud` reescreve o arquivo *no lugar*, então o *bind mount* do container acompanha sem recriar — mas confirme comparando o `inode`.
Teste ou script ignora o valor que você passou ao `ConfigService`	O `ConfigService` do Nest lê `process.env` com **prioridade** sobre o objeto do construtor. Num container com a variável definida, o valor do código não vale. Foi o que fez uma medição de custo bater na quota de 20.000 com 5.000.000 declarados no script.

Sintoma	Causa provável
Dependência nova não chega no container; a API não sobe depois de `prisma migrate`	O `node_modules` **e o Prisma Client** dos containers são próprios. `pnpm docker:dev:deps`, ou `prisma generate` dentro do container, e **recriar** o serviço — `restart` não relê o `compose`, então variável de ambiente nova (como `CORS_ORIGINS`) não entra.
Snyk verde no PR, `SNYK-0005` `Authentication error` só no *push* para `main`	Workflow **chamado** não herda *secrets*: no PR o `ci.yml` roda direto e recebe `SNYK_TOKEN`; chamado pelo `deploy.yml`, precisa de `secrets: inherit` no *job* `verificar`. O passo falha alto de propósito — token ausente nunca vira portão aberto.
`vite build` morre com "Transforming destructuring … is not supported yet" em todas as experiências	`esbuild` fora da faixa que o Vite pina (um *override* de segurança, tipicamente): o 0.28 não faz mais o *lowering* para os *targets default* do Vite 6 (`chrome87`/`es2020`). Reverter o *override*; a vulnerabilidade fica em `.snyk` com prazo até o Vite pinar o esbuild corrigido.
Build da imagem morre em `corepack: not found` (exit 127) depois de subir a base `node:NN-slim`	A partir do Node 25 a imagem oficial **não traz mais o corepack**. Os três *Dockerfiles* instalam o pnpm pelo npm, pinado na versão do `packageManager`. O sintoma engana porque o `docker compose up` segue usando a imagem **antiga** do cache sem avisar — os containers continuam no Node velho e só o `node --version` lá dentro denuncia.
Dois números diferentes para os posts que o algoritmo escondeu — a criança lê "32 posts que nunca chegaram até você" e o relatório do professor, sobre ela, diz "escondeu 30"	Dossiê **congelado** numa versão anterior à **ADR-42**. As duas contas já vêm de `dossie/exposicao.ts`, mas o dossiê é gravado no `finish` e só é recomputado na leitura quando `versaoDoRelatorio` está atrás de `VERSAO_DO_RELATORIO`. Se os números seguirem divergindo com a versão em dia, o caso é outro: alguém desaprovou posts entre o encerramento da investigação e a tela dos perdidos — a matriz é viva.
O envolvimento da turma cai sozinho conforme a aula avança (52,7 → 42,8 → 38,7 → 35,1 no painel ao vivo)	**Não é a turma desengajando.** O *buffer* de eventos da experiência vive enquanto a *run* vive, e cada `hub:pause` durante o debate, a explicação ou a reflexão ainda registra `perdeu_foco`. Se a janela do tempo não terminar no último `visto` (`fimDoFeedDe`, **ADR-44**), `feedMs` cresce depois de o feed fechar e `ativoMs / feedMs` derrete. Conferir a versão do relatório antes de olhar para o estudante.
O aparelho do estudante recarrega e ele volta para a revelação	Alguém tirou o `children` de dentro do `DispositivoFrame` em vez de ligar a *prop* `expandido`. Trocar a árvore desmonta o `<iframe>`; o modo expandido é ESTILO, e só (**ADR-41**).

Sintoma	Causa provável
TODA chamada de IA morre com `fetch failed` — e o `getAccessToken()` à mão devolve token	**Não é credencial**: é o `Agent` do pacote `undici` passado ao `fetch` GLOBAL, que usa o undici EMBUTIDO na *runtime* — outro pacote, outra versão (Node 26.7 traz a 8.x; a api declarava `^6`). A v6 não satisfaz mais a interface de *handler* da v8. A razão real, `invalid onError method`, vive só em `error.cause`, e o `catch` logava só `message` — por isso passou dias parecendo ADC vencido, que era um problema DE VERDADE por cima. Conferir com `node -p "process.versions.undici"` contra `require("undici/package.json").version`. Resolvido pela **ADR-45**: `Agent` e `fetch` do MESMO pacote.
Reautenticou o ADC, o token novo funciona, e a api continua dizendo "Sem credencial"	O processo cacheia a credencial: `new GoogleAuth()` é criado UMA vez, no escopo do módulo (`vertex.provider.ts`), e segura o *refresh token* lido na subida. `gcloud auth application-default login` reescreve o arquivo, mas quem já subiu não relê. **Reautenticar não basta: reinicie a api.** Comparar `docker inspect -f '{{.State.StartedAt}}'` com o `mtime` do ADC responde na hora.
A suíte acusa "turma não está no seletor (rode o seed)" — e a turma ESTÁ no banco, com o professor matriculado nela	O roteiro pegava `locator("select").first()`, e quem tem mais de uma lotação (`prof.demo` tem duas) ganhou um seletor de ESCOLA no cabeçalho, que vem ANTES no DOM. Ele procurava "2A" entre nomes de escola. **A mensagem culpava o banco por uma mudança de *header*** — e o seed é o primeiro lugar onde ninguém acha nada. Usar `select#turma`, como os outros roteiros já faziam. Corrigido em 2026-08-29; era o único `select` cru em `scripts/`.
Toda suíte morre em `Executable doesn't exist` — com o cache de navegadores CHEIO	O executável que falta é o `chromium_headless_shell-1234`, e alguém rodou `npx playwright install` em vez de `pnpm exec playwright install`. O `npx` ignora o `node_modules` do repo, resolve o *playwright* mais novo do npm e baixa a *build* de navegador DELE (a 1237); o repo fixa a 1.62.1, que procura a 1234. **O sintoma mente duas vezes**: o cache tem navegador de sobra, e o banner do próprio Playwright manda rodar exatamente o `npx` que causou o descasamento. Conferir com `pnpm exec playwright --version` e reinstalar por ali.
O relatório do estudante abre em tela cheia no meio do FEED	Alguém inverteu a condição do `ExpHost` para expandir por *default*. O `IntersectionObserver` do *dwell* se refere ao *iframe*: numa área larga o sinal central do motor infla em silêncio, e nada na tela denuncia. A condição é positiva — `fase === "explicabilidade"` E `passo === "relatorio"` —, tudo o mais é aparelho.
O seed morre com "SEED_SENHA não definida"	Não é regressão: a senha das contas de demonstração saiu do fonte (**ADR-47**). Nos dois *compose* ela já vem; no *host*, ponha `SEED_SENHA` no `apps/api/.env` (o Prisma carrega esse arquivo) ou prefixe o comando.
A experiência lança "hubOrigin não é quem embedou"	O `?hubOrigin=` da URL não bate com `location.ancestorOrigins[0]` (**ADR-46**). Em desenvolvimento, quase sempre é `HUB_ORIGIN` do hub apontando para outro endereço — `host.docker.internal` onde o navegador vê `localhost`, por exemplo. **Antes desta trava o sintoma era outro**: *handshake* expirando em 8 s sem dizer por quê.

Sintoma	Causa provável
O CSV grande é recusado — e a mensagem diz QUAL dos três tetos foi	São três, em ordem (**ADR-48**). "CSV acima de 30MB — divida o arquivo": o nosso, e é o esperado. `PayloadTooLarge` do Express: o *bodyParser* de `main.ts` ficou menor que o `TETO_DO_ARQUIVO` da rota — os dois saem da mesma constante, e o `mb` do *parser* é MEBIbyte justamente para ficar acima. 413 SEM mensagem nossa e sem linha nenhuma no log: é o Cloud Run, que recusa acima de 32 MiB em HTTP/1 antes do contêiner — só acontece se alguém chamar a rota fora da tela. Caminho, nos três: dividir o `enrollments.csv` por escola ou por diretoria.
O *bootstrap* do WSO2 falha em "TLS com identity-server:9443"	Ele agora PINA o certificado do IS e falha se não conseguir lê-lo, onde antes desligava a verificação de TLS do processo e seguia (**ADR-47**). O IS ainda está subindo, ou a porta 9443 não responde — o *healthcheck* do *compose* deveria ter segurado; conferir `docker compose -f wso2/docker-compose.yml ps`.
O painel da atividade lista estudantes que não são meus	Se a atividade tem turma, é bug — abrir um chamado. Se ela NÃO tem (`exp_01_activities.turma_id IS NULL`), é o comportamento das atividades anteriores à **ADR-49**: elas valem para a escola inteira, e o painel diz isso numa faixa azul. Criar a atividade de novo, agora escolhendo a turma. **Antes da ADR-49 o sintoma era pior e mudo**: a consulta rodava sem `escola_id` numa tabela FORA da RLS, então apareciam crianças de OUTRAS escolas do contrato.
Os perfis da Vila não mudam de lado depois de subir a `VERSAO_MAPEAMENTO`	Esperado: o que a constante nova recalcula é `exp_70_perfis`, não a cópia em `exp_01_exp_70_perfis_importados`, que é a que decide o lado do feed. O painel do Ato 1 mostra quantos vieram de versão anterior; "Trazer perfis da Vila do Lume" atualiza, e NÃO desfaz quem o professor moveu de lado (**ADR-50**). Não há rotina automática, de propósito.
O estudante entra e vê "Nada rolando por aqui agora"	Com **ADR-49**, `POST /runs` só entrega atividade SEM turma ou da turma DELE — e a turma sai de `users.turma_id`, o cache da matrícula principal. Estudante sem `turma_id` (importado sem enturmar, ou movido pela metade) só alcança atividade sem turma. Conferir em `/admin/turmas`.
A planilha volta com "papel não reconhecido"	A coluna `papel` aceita `estudante`, `professor` e `gestor`, mais os sinônimos de `SINONIMOS_DE_PAPEL` (`discente`, `docente`, `coordenacao`, `diretor`… e o `student`/`teacher` do OneRoster). Fora disso a linha vira relatório, não erro mudo. O modelo para download (`GET /rostering/modelos/usuarios.csv`) sai da MESMA constante que o parser lê.
`Protocolo incompatível: hub vN, experiência vM` no console	O *bundle* servido tem `PROTOCOL_VERSION` de outro *major* que o hub. É a falha correta — alta e legível, em vez do *handshake* expirando em 8s — e o conserto é republicar a experiência (`deploy.yml`, trabalho `publicar-exps`). Apontar o hub para uma versão anterior no CDN NÃO resolve quando é justamente a versão anterior que está atrás do protocolo: *rollback* só funciona entre versões do mesmo *major*.

Sintoma	Causa provável
O *deploy* sobe os *bundles*, fica verde em `publicar-exps` e a experiência continua na versão antiga	Olhe o trabalho `ativar-exps`: é ele que escreve `versaoAtiva`, e o *bundle* no *bucket* é inerte até esse *PUT*. A falha que produz exatamente este quadro é `ERR_MODULE_NOT_FOUND: Cannot find package 'jose'` — o trabalho pulava o `pnpm install` sob a justificativa de que o `publicar-manifesto.mjs` era "Node puro", e ele assina o *token* de `admin-contrato` com `SignJWT` do `jose`. Apareceu no primeiro *deploy* completo de uma *stack* nova, porque antes o caminho nunca havia sido exercitado. O sintoma engana duas vezes: o trabalho que *parece* responsável (`publicar-exps`) está verde, e nada no ar explica a versão velha.
`P1013: invalid port number in database URL` no túnel, com a porta certa na string	A porta está certa; o que quebra o *parse* é a SENHA. O `db_password` do Terraform é aleatório e pode conter `@`, `:` ou `/` — e aí o `@` da senha vira o separador de *host*, e o resto da senha vira "porta". Não é bug do Prisma nem da instância: **percent-encode a senha** antes de montar a URL. Passa despercebido de ambiente para ambiente porque uma senha sorteada sem esses caracteres funciona — em `dev` funcionou, em `hml` não. O Cloud Run não sofre disso: lá a string é montada pelo Terraform, não à mão.
`ativar-exps` falha numa *stack* nova — de DUAS formas diferentes, e a segunda mente	Causa única nas duas: **certificado ainda `PROVISIONING`**. Este é o único trabalho que fala HTTPS por **hostname público** (o *PUT* do manifesto); os outros usam `gcloud`. Qual cara aparece depende de QUAL certificado falta: ■ Falta o da **api** → o TLS quebra no *runner*: `fetch failed` → "`socket disconnected before secure TLS`". Aponta para a causa. ■ Falta o do **hub** → `401 {"message":"Token inválido"}`. **Este engana**: parece chave errada ou *secret* trocado, e não é — a api valida o *token* contra o JWKS que o HUB serve, e é ELA que toma o `ECONNRESET` ao buscá-lo. A prova está no *log* da api, não no do CI: `[TokenService] Token rejeitado: ECONNRESET ...`. Confirmar com `gcloud certificate-manager certificates list`; se os CNAMEs `_acme-challenge` já batem com `dns-authorizations list`, **não há nada a consertar**: é tempo de emissão. Repetir o *dispatch* com `forcar` quando virarem `ACTIVE`. O estado intermediário é seguro por desenho — os *bundles* já estão no *bucket* e são inertes até a ativação.

Sintoma	Causa provável
Feed da exp-02 em "Carregando o vídeo…" para sempre, e Salvar/Curtir desabilitados junto	**O *bucket* de *assets* do contrato está sem os 21 *mp4*** — o *upload* da mídia pesada é deliberado e não acontece no CI (**ADR-63**). Confirmar com `gcloud storage ls gs://{contrato}-assets/exp-02-unootube/videos/` e no *log* do *load balancer*, onde os `GET` do `/exp-02-unootube/videos/vNN.mp4` aparecem em 404. **Não é o *bundle* nem a variável**: se a URL pedida traz o domínio de *assets* e o caminho certo, a configuração está correta e o que falta é o arquivo. O sintoma engana porque o `ShortsPlayer` não escuta `error` — o estado de carregamento nunca é desfeito, e a criança lê como internet lenta. Se a URL pedida NÃO traz o domínio de *assets*, aí sim é a var `ASSETS_BASE_URL` vazia no *environment*: interpolada vazia ela produz um caminho relativo válido, e o *fallback* de dev nunca dispara.
Controle dentro de um *modal* "sai fora" — o `<select>` fecha sozinho, o clique parece cancelado	**Handler instável passado ao `Modal` do *design system***, combinado com algo que renderiza o pai periodicamente. O `Modal` das seções `chatgpt` e `unootube` mantém `onClose` nas dependências do *mesmo* `useEffect` que faz o foco: identidade nova ⇒ *cleanup* devolve o foco para fora do *modal* e o efeito o joga no primeiro focável, que é o botão de fechar. Perder o foco fecha um `<select>` nativo aberto. Não há rastro no servidor — as requisições que saem voltam 201. Procurar arrow *inline* em `onClose`/`onFechar` e *polling* ou cronômetro que renderize mais de uma vez por segundo. O `Modal` da seção `unooria` já separa os dois efeitos e é o desenho a copiar.
Tela de resultados do professor vazia, sem erro em lugar nenhum	**`POST /telemetry/ingest` bloqueado pelo Cloud Armor** antes de chegar à api (**ADR-62**) — por isso não há *log* de aplicação. Confirmar no *log* do *load balancer*: `jsonPayload.statusDetails="body_denied_by_security_policy"`, com `enforcedSecurityPolicy.preconfiguredExprIds` nomeando a assinatura. O Pub/Sub reentrega para sempre (não há DLQ), então o volume de 403 cresce sozinho e não indica ataque nenhum: o agente é `APIs-Google`.
O professor relata um erro e manda um código	É o `requestId` do corpo do 500 (`{codigo: "erro-interno", requestId}`), igual ao *header* `x-request-id` de toda resposta e ao `traceId` do *load balancer*. No Logs Explorer: `jsonPayload.requestId="<código>"` traz a linha `ERROR` com `stack_trace`, escola e perfil; o *request log* do Cloud Run com o mesmo `trace` vem agrupado ao lado. Sem código na tela (rota antiga, 4xx), filtre por `jsonPayload.codigo` — todo `logger.*` da api carrega um.

F.7 *Checklist* de operação

Frequência	Item

Contínuo (fase 0 do Capítulo 17, **pendente**)	Os quatorze alertas por sintoma e os cinco *uptime checks* tornam as linhas "semanal" e "mensal" abaixo confirmação, não descoberta. Até lá, elas são o único mecanismo.
Após cada *deploy*	Conferir que a migração rodou (`--wait` no *job*); abrir `/readyz`; abrir uma experiência de estudante e uma ferramenta de professor. Com a fase 0: o *smoke* do `deploy.yml` faz os três e notifica se falhar.
Semanal, e após *deploy*	Error Reporting: erro **novo** na revisão que acabou de receber tráfego, e erro de cliente novo por `(expId, versao)` — reverter pelo manifesto antes de investigar.
Semanal	Taxa de 5xx do Cloud Run; conexões ativas do Cloud SQL; contagem de `status = 'aberta'` em `exp_70_sessoes` (partidas perdidas); planos em `status = 'erro'`.
Mensal	**Contagem da partição `DEFAULT` de telemetria** — encher significa que faltou criar partição (`R-08`); volume de `telemetria_eventos` por mês; consumo de quota de IA por professor.
Trimestral	`pnpm up -r @unooria/design-system --latest` e rodar os suítes de ponta a ponta; revisar as lacunas do *design system* ainda compensadas localmente; conferir se os modelos de IA em uso continuam no catálogo (`R-12`).
Anual, ou por *release*	Revisar as ADRs cujo gatilho de reavaliação pode ter sido acionado (Capítulo 18).

Listing F.7 – As consultas do *checklist*

```sql
-- A particao DEFAULT esta enchendo? (deveria ser sempre 0)
SELECT count(*) FROM particoes.telemetria_eventos_default;

-- Partidas perdidas (aba fechada no meio) por turma
SELECT turma_id, count(*) FROM exp_70_sessoes
 WHERE status = 'aberta' AND aberta_em < now() - interval '1 day'
 GROUP BY turma_id ORDER BY 2 DESC;

-- Planos que falharam
SELECT id, serie, disciplina, tentativas, erro FROM exp_90_planos
 WHERE status = 'erro' ORDER BY updated_at DESC;

-- Consumo de IA no dia
SELECT usuario_id, tokens_usados FROM ia_uso_diario
 WHERE dia = to_char(now(), 'YYYY-MM-DD') ORDER BY 2 DESC LIMIT 20;

-- Volume de telemetria por mes (nome da particao ja diz o mes)
SELECT relname, n_live_tup FROM pg_stat_user_tables
 WHERE schemaname = 'particoes' ORDER BY relname;
```

> **ATENÇÃO**
>
> Toda consulta manual acima roda **fora** de `comTenancy`, ou seja, com a *role* de aplicação a RLS a filtraria para nada. Diagnóstico em produção usa conexão administrativa — e é

por isso que a *role* de aplicação **não pode** ter `BYPASSRLS`: a separação entre "a aplicação" e "um humano diagnosticando" tem de ser real.

F.8 Compilar esta documentação

```
cd documentacao-latex
./compilar.sh              # build completo (Tectonic: baixa o que falta e roda o BibTeX)
./compilar.sh --rapido     # uma passagem, para iterar na escrita
./compilar.sh --limpar     # remove os auxiliares

# Com uma distribuicao de TeX completa instalada, alternativamente:
latexmk -pdf main.tex
```

Requisito	Detalhe
Motor	**Tectonic** (recomendado: baixa os pacotes sob demanda e chama o BibTeX sozinho) *ou* qualquer distribuição com `pdflatex` + `bibtex`.
Fontes	`roboto` e `roboto-mono`.
Diagramas	`tikz` e `pgfplots` — todos vetoriais, sem renderização externa.
Bibliografia	`abntex2cite` (estilo `alf`) processado por **BibTeX** — *não* biblatex/biber, para que o Tectonic feche o ciclo sem ferramenta externa.

REGRA INVIOLÁVEL

Atualizar esta documentação faz parte da *Definition of Done* (Capítulo 16). Implementação que mude arquitetura, contrato, modelo de dados, decisão ou risco atualiza o capítulo correspondente **e regera o `main.pdf` no mesmo *commit***. A regra está registrada em `AGENTS.md`, não apenas aqui.

Referências Bibliográficas

Advanced Distributed Learning Initiative. *Experience API (xAPI) Specification, version 2.0*. 2023. <https://github.com/adlnet/xAPI-Spec>. Acesso em: 27 ago. 2026.

Autoridade Nacional de Proteção de Dados. *Guia orientativo: tratamento de dados pessoais de crianças e adolescentes*. 2024. ANPD, Brasília. Acesso em: 27 ago. 2026.

BEYER, B. et al. *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O'Reilly Media, 2016.

BEYER, B. et al. *The Site Reliability Workbook: Practical Ways to Implement SRE*. Sebastopol: O'Reilly Media, 2018.

Brasil. *Lei n. 13.709, de 14 de agosto de 2018 — Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. Diário Oficial da União, Brasília.

BRIKMAN, Y. *Terraform: Up & Running — Writing Infrastructure as Code*. 3. ed. Sebastopol: O'Reilly Media, 2022.

BROWN, S. *Software Architecture for Developers: Volume 2 — Visualise, document and explore your software architecture*. Victoria: Leanpub, 2024.

BROWN, S. *The C4 model for visualising software architecture*. 2026. <https://c4model.com>. Acesso em: 27 ago. 2026.

DöRING, A. K. et al. Assessing values at an early age: The picture-based value survey for children (pbvs-c). *Journal of Personality Assessment*, v. 92, n. 5, p. 439–448, 2010.

Google Cloud. *Deploy the Google-Built OpenTelemetry Collector on Cloud Run*. 2026. <https://cloud.google.com/stackdriver/docs/instrumentation/opentelemetry-collector-cloud-run>. Acesso em: 1 set. 2026.

Google Cloud. *Google Cloud Observability Documentation: Cloud Logging structured logs, Cloud Monitoring alerting and SLOs, Cloud Trace, Error Reporting*. 2026. <https://cloud.google.com/stackdriver/docs>. Acesso em: 1 set. 2026.

Google Cloud. *Google Cloud Observability pricing*. 2026. <https://cloud.google.com/stackdriver/pricing>. Acesso em: 1 set. 2026. Valores de referência: Cloud Logging 50 GiB/projeto/mês gratuitos e US$ 0,50/GiB; métricas 150 MiB gratuitos e US$ 0,258/MiB; Cloud Trace 2,5 milhões de *spans*/mês gratuitos e US$ 0,20/milhão; *uptime checks* 1 milhão de execuções/projeto/mês gratuitas; políticas de alerta US$ 0,35 por referência de métrica por mês, com cobrança anunciada para não antes de setembro de 2026.

International Organization for Standardization. *ISO/IEC/IEEE 42010: Systems and software engineering — Architecture description*. Genebra, 2022.

International Organization for Standardization. *ISO/IEC 25010: Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. Genebra, 2023.

KRUCHTEN, P. The 4+1 view model of architecture. *IEEE Software*, v. 12, n. 6, p. 42–50, 1995.

LOVIBOND, P. F.; LOVIBOND, S. H. The structure of negative emotional states: Comparison of the depression anxiety stress scales (dass) with the beck depression and anxiety inventories. *Behaviour Research and Therapy*, v. 33, n. 3, p. 335–343, 1995.

MAJORS, C.; FONG-JONES, L.; MIRANDA, G. *Observability Engineering: Achieving Production Excellence*. Sebastopol: O'Reilly Media, 2022.

MCDONNELL, C. *Zod: TypeScript-first schema validation with static type inference*. 2026. <https://zod.dev>. Acesso em: 27 ago. 2026.

Mozilla Developer Network. *The iframe sandbox attribute — HTML*. 2026. <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe>. Acesso em: 27 ago. 2026.

Mozilla Developer Network. *Window.postMessage() — Web APIs*. 2026. <https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage>. Acesso em: 27 ago. 2026.

NestJS. *NestJS Documentation: Guards, Interceptors and Modules*. 2026. <https://docs.nestjs.com>. Acesso em: 27 ago. 2026.

NYGARD, M. *Documenting Architecture Decisions*. 2011. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 27 ago. 2026.

OpenTelemetry Authors. *OpenTelemetry Documentation: signals, SDKs, sampling and the Collector*. 2026. <https://opentelemetry.io/docs/>. Acesso em: 1 set. 2026.

OWASP Foundation. *OWASP Top 10: 2021 — The Ten Most Critical Web Application Security Risks*. 2021. <https://owasp.org/Top10/>. Acesso em: 27 ago. 2026.

SCHWARTZ, S. H. An overview of the schwartz theory of basic values. *Online Readings in Psychology and Culture*, v. 2, n. 1, 2012.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: Row Security Policies*. 2026. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>. Acesso em: 27 ago. 2026.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: Table Partitioning*. 2026. <https://www.postgresql.org/docs/current/ddl-partitioning.html>. Acesso em: 27 ago. 2026.

Vercel. *Next.js Documentation: App Router, Server Components and Middleware*. 2026. <https://nextjs.org/docs>. Acesso em: 27 ago. 2026.

Vercel. *Turborepo Documentation: task pipelines, filtering and remote caching*. 2026. <https://turborepo.com/docs>. Acesso em: 27 ago. 2026.

VIGNOLA, R. C. B.; TUCCI, A. M. Adaptation and validation of the depression, anxiety and stress scale (dass) to brazilian portuguese. *Journal of Affective Disorders*, v. 155, p. 104–109, 2014.

W3C. *Trace Context — W3C Recommendation*. 2021. <https://www.w3.org/TR/trace-context/>. Acesso em: 1 set. 2026.